

PDI & VC

Processamento Digital de Imagens e Visão Computacional

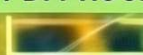
SUNFLOWER | B 86 CONF. →



SUNFLOWER | 0.98 CONF. (PDI->VC)



PDI PROCESS | EDGE DET.



GIRASSOL | 95% (VC USE)



Fundamentos, Algoritmos
e Aplicações Integradas

AUTOR: FRANCISCO DE ASSIS ZAMPIROLI



Processamento Digital de Imagens e Visão Computacional

Livro interativo com Python

Francisco de Assis Zampirolli

Sumário

PDI e VC — Python	14
Organização	14
Prefácio	15
Um livro vivo	15
Uso de ferramentas de Inteligência Artificial	15
Como este livro é produzido	15
A biblioteca <code>morph.py</code>	16
Exercícios de Programação (EPs) e Validação Automática	16
Estrutura e Validação Local	16
Integração com o Moodle (VPL)	16
Como Publicar no Moodle	17
Idiomas e Conversão de Código	17
Código aberto	18
Antes de começar: <i>Notebooks</i> em Python	18
I Parte I — Fundamentos de PDI	19
1 Fundamentos e Primeiros Passos	20
1.1 Objetivos	20
1.2 Antes de começar: <i>Notebooks</i> em Python	20
1.3 Fundamentos	20
1.3.1  Visão Computacional	21
1.3.2  Processamento Digital de Imagens (PDI)	21
1.4 Etapas do PDI	22
1.5 Formação da Imagem e o Espectro	22
1.6 O que é uma Imagem Digital?	25
Representação Matemática	25
1.7 Configuração do Ambiente	26
1.8 Fundamentos de Matrizes - atenção à cópia de referências	27
1.9 Lendo e Exibindo Imagens	28
Alternativa: <i>Download</i> da Imagem para Armazenamento Local	29
1.10 Conversão de Tipos e Limiarização	30
Conversão para Tons de Cinza	30
Limiarização (<i>Thresholding</i>)	30
Exemplo prático de conversão e limiarização	30
1.11 Limiarização pelo método de Otsu	31
1.12 Acesso a Pixels	32
1.13 Resumo	34
1.14  Uso do NotebookLM como Tutor Complementar	34
Funcionalidades Disponíveis na Plataforma	34
1.15 Lista de Exercícios	35
Referências do Capítulo	36
1.16  Parte Prática com Exercícios de Programação	36
 Objetivo deste Caderno	36

§ Instruções Passo a Passo	36
⚠ Importante: Regras e Boas Práticas	38
1.16.1 EP01_01 📏 Três métricas de distância em PDI	38
1.16.2 EP01_02 📊 Desempenho Preditivo — Métricas de ML em VC	45
1.16.3 EP01_03 ↗ <i>Mean Average Precision</i> (mAP) — Curva Precisão-Sensibilidade	47
1.16.4 EP01_04 🖼️ Leitura e Informações de uma Imagem em Matriz	50
1.16.5 EP01_05 🔄 Negativo de uma Imagem em Tons de Cinza	53
1.16.6 EP01_06 🎨 Conversão RGB → Tons de Cinza (ITU-R BT.601)	54
1.16.7 EP01_07 ⬛ Limiarização Manual: Imagem Binária	56
1.16.8 EP01_08 🎨 Remapeamento por Faixa de Intensidade	58
1.16.9 EP01_09 🏁 Padrão de Tabuleiro: Matriz de Xadrez	59
1.16.10 EP01_10 📄 Metadados: Leitura de Arquivo PGM	61
1.16.11 EP01_11 ↗ Análise de Vizinhaça: Filtro de Máximo 1D	63
2 Da Captura ao Pixel - Amostragem, Quantização e Conectividade	66
2.1 Objetivos	66
2.2 O Olho e a Câmera - Elementos da Percepção Visual	66
2.2.1 Visão humana	66
2.2.2 Sensores digitais	66
2.3 Ilusões de Ótica: Os Desafios da Percepção Visual	67
2.3.1 Ambiguidade e Contexto	67
2.3.2 Brilho e Contraste Local	68
2.3.3 Geometria e Perspectiva	68
2.4 O Modelo Matemático da Formação da Imagem	68
2.4.1 Representação Digital e Canais Espectrais	69
2.4.2 Preparando o Ambiente Prático	70
2.4.3 Implementação da Leitura de Imagem em <code>morph.py</code>	70
2.4.4 RGB e RGBA: Exemplo Prático com a Imagem Mandrill	71
2.5 Digitalização: Amostragem e Quantização	72
2.5.1 Amostragem - Discretização do espaço	72
2.5.2 Quantização - Discretização da intensidade	72
2.5.3 Efeitos da amostragem e quantização	73
2.5.4 Análise Técnica	75
2.6 Relações entre Pixels - Topologia da Imagem	76
2.6.1 Vizinhaça	76
2.6.2 Adjacência, conectividade e caminhos	76
2.6.3 Distâncias entre pixels	77
2.7 Armazenamento de Imagens	77
2.7.1 Exemplo: Extração de Metadados e Localização GPS	78
2.7.2 Por que esta separação é importante?	80
2.8 Transformações Geométricas Básicas	80
2.8.1 Translação	81
2.8.2 Rotação	82
2.8.3 Escala (Redimensionamento)	83
2.8.4 Cisalhamento (<i>Shear</i>)	84
2.9 Resumo	85
2.10 📖 Uso do NotebookLM como Tutor Complementar	85
2.11 Lista de Exercícios	86
Referências do Capítulo	86
2.12 🛠️ Parte Prática com Exercícios de Programação	86
🎯 Objetivo deste Caderno	86
2.12.1 EP02_01 ☉ Ajuste de Brilho e Contraste	87
2.12.2 EP02_02 📍 Subamostragem Espacial	89
2.12.3 EP02_03 🎨 Quantização de Níveis de Cinza	91

2.12.4	EP02_04	 Transformada de Distância em Imagem Binária	93
2.12.5	EP02_05	Translação de Imagem	95
2.12.6	EP02_06	 Rotação de Imagem	96
2.12.7	EP02_07	 Redimensionamento (Escala)	98
2.12.8	EP02_08	 Cisalhamento (Shear)	102
2.12.9	EP02_09	 Transformação Afim Genérica	103
2.12.10	EP02_10	 Correção de Perspectiva (Homografia)	107
2.12.11	EP02_11	 Correção de Perspectiva (Homografia) em Imagem Real	109
3	Operações Espaciais: Intensidade, Histograma e Filtragem		114
3.1	Objetivos		114
3.2	Operações em Nível de Intensidade		114
3.2.1	Preparando o Ambiente Prático		115
3.2.2	Operações Aritméticas		116
3.2.3	Mistura Ponderada (<i>Alpha Blending</i>)		117
3.2.4	Operações Lógicas e Máscaras Bit a Bit		119
3.3	Histograma de Imagens		120
3.3.1	Equalização de Histograma		121
3.3.2	Especificação de Histograma		125
3.4	Fundamentos Espaciais: Vizinhança, Convolução e <i>Kernels</i>		126
3.4.1	Vizinhança		126
3.4.2	Tratamento de Bordas		127
3.4.3	Correlação e Convolução		129
3.4.4	Correlação vs. Convolução no OpenCV		129
3.4.5	O Papel do <i>Kernel</i>		131
3.4.6	Exemplo Numérico: Correlação Passo a Passo		133
3.5	Filtragem Espacial de Suavização		134
3.5.1	Filtro de Média (<i>Box Filter</i>)		134
3.5.2	Filtro Gaussiano		135
3.6	Filtragem Espacial de Realce		137
3.6.1	Laplaciano		138
3.6.2	Operador de Sobel		141
3.6.3	Operador de Prewitt		143
3.6.4	<i>Unsharp Masking</i> (USM)		144
3.6.5	Detector de Canny		146
3.7	Filtros de Ordem: Filtro da Mediana		147
3.7.1	Ruído Impulsivo: Sal e Pimenta		147
3.7.2	Filtro da Mediana		148
3.8	Aplicação Prática: Pré-processamento para Segmentação		151
3.9	Resumo		152
3.10	 Uso do NotebookLM como Tutor Complementar		152
3.11	Lista de Exercícios		153
	Referências do Capítulo		154
3.12	 Parte Prática com Exercícios de Programação		154
	 Objetivo deste Caderno		154
3.12.1	EP03_01	 Adição Saturada de Constante	155
3.12.2	EP03_02	 <i>Alpha Blending</i> de Duas Imagens	157
3.12.3	EP03_03	Inversão de Imagem (Negativo Fotográfico)	158
3.12.4	EP03_04	 Equalização de Histograma (L bits)	160
3.12.5	EP03_05	Aplicação de Máscara AND Binária	161
3.12.6	EP03_06	Filtro de Média com <i>Kernel</i> $N \times N$	165
3.12.7	EP03_07	 Operador Laplaciano (w4) para Realce de Bordas	167
3.12.8	EP03_08	Gradiente de Sobel: G_x e G_y	168
3.12.9	EP03_09	 Filtro da Mediana 3×3	172

3.12.10	EP03_10	<i>Unsharp Masking (USM)</i>	174
4	Morfologia Matemática e Segmentação de Imagens		177
4.1	Objetivos		177
4.2	Limiarização		178
4.2.1	Imagem de Moedas		179
4.2.2	Escolha do Pré-processamento para o Otsu		180
4.2.3	Resultado: CLAHE como Melhor Pré-processamento		182
4.3	Morfologia Matemática		183
4.3.1	Erosão e Dilatação		183
4.3.2	Abertura e Fechamento		190
4.3.3	Operadores Geodésicos		193
4.3.4	Reconstrução Morfológica		197
4.3.5	Preenchimento de Buracos e Remoção de Bordas		199
4.3.6	<i>Pipeline</i> de Limpeza Binária com CLAHE		202
4.3.7	Morfologia em Tons de Cinza		203
4.4	Segmentação de Imagens: Fundamentação e Taxonomia		206
4.4.1	Rotulação		207
4.4.2	Transformada de Distância		210
4.4.3	Transformada de Distância Euclidiana em quatro passos		212
4.4.4	Segmentação por <i>Watershed</i>		214
4.5	Extração de Componentes e Descritores de Forma		219
4.5.1	Rotulação e Estatísticas de Componentes		220
4.5.2	Descritores de Forma		221
4.5.3	Conexão com Detecção de Objetos Moderna		223
4.6	Resumo		225
4.7	 Uso do NotebookLM como Tutor Complementar		226
4.8	Lista de Exercícios		226
	Referências do Capítulo		227
4.9	 Parte Prática com Exercícios de Programação		227
	 Objetivo deste Caderno		228
4.9.1	EP04_01	Limiarização Global por Limiar Fixo	228
4.9.2	EP04_02	 Limiarização Automática de Otsu	230
4.9.3	EP04_03	 Dilatação Binária Plana (mm.dil0)	233
4.9.4	EP04_04	Erosão Binária Plana (mm.ero0)	234
4.9.5	EP04_05	Abertura Morfológica (Remoção de Ruído)	238
4.9.6	EP04_06	 Fechamento Morfológico (Preenchimento de Falhas)	240
4.9.7	EP04_07	Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)	242
4.9.8	EP04_08	Gradiente Morfológico, Top-hat e Black-hat	243
4.9.9	EP04_09	Transformada de Distância e o “Miolo” do Objeto	246
4.9.10	EP04_10	Separação de <i>Blobs</i> , Rotulação e Descritores	248
5	Transformadas e Compressão		251
5.1	Objetivos		251
5.2	Configuração do Ambiente		251
5.3	Transformada de Fourier Discreta 2D		252
5.3.1	Fenômeno: o que uma imagem “parece” no domínio da frequência?		252
5.3.2	O Experimento da Grade: Construindo a Imagem do Zero		253
5.3.3	Definição Matemática		253
5.3.4	Implementação e Visualização		254
5.3.5	O que a Magnitude e a Fase carregam?		254
5.3.6	Simulador: Reconstruindo Sinais com Senoides		256
5.4	Teorema da Convolução e Estratégias de Filtragem		257
5.4.1	<i>Kernel</i> Separável vs. Não Separável		257

5.4.2	Análise de Eficiência Computacional em Alta Resolução	257
5.4.3	Discussão do Gráfico Experimental	257
5.5	Filtros no Domínio da Frequência	261
5.5.1	Filtros Passa-Baixa	262
5.5.2	Filtros Passa-Alta e Passa-Banda	262
5.5.3	Remoção de Ruído Periódico	265
	📌 Síntese — Filtros Espectrais	265
5.6	<i>Wavelets</i> e Multirresolução	265
5.6.1	O Limite de Fourier: Onde ocorreu o evento?	267
5.6.2	Transformada Wavelet Discreta 2D	267
5.6.3	Famílias de <i>Wavelets</i>	268
5.6.4	📌 Síntese - Fourier vs <i>Wavelets</i> : quando usar cada uma?	272
5.7	Compressão de Imagens	273
5.7.1	Taxonomia das Redundâncias	273
5.7.2	As Funções de Base da DCT	273
5.7.3	Transformada de Cossenos Discreta (DCT)	273
5.7.4	Quantização: a principal fonte de compressão	275
5.7.5	Pipeline JPEG	277
5.7.6	Simulador Interativo: Quantização DCT	279
5.8	Comparação de Formatos de Imagem	279
5.8.1	Características dos Formatos	279
5.8.2	Métricas de Qualidade	280
5.8.3	Inspeção Visual: Qualidade não é só PSNR	281
	📌 Síntese — Compressão JPEG	285
5.9	Aplicação Prática: Remoção de Ruído por Filtragem Espectral	285
5.10	Resumo do Capítulo	287
	🔑 Fundamentos Essenciais	287
5.11	📁 Uso do NotebookLM como Tutor Complementar	287
5.12	Lista de Exercícios	288
	Referências do Capítulo	288

II Parte II — Visão Computacional 289

6	Inspeção Industrial e Análise de Documentos 290
6.1	Objetivos do Capítulo 290
6.2	Configuração do Ambiente 291
6.3	Bases de Imagens para Experimentação 291
6.3.1	Imagens Públicas com <code>skimage.data</code> 292
6.3.2	Imagens Reais com OpenCV 292
6.4	Fundamentos de OMR e Inspeção Industrial 293
6.5	Projetos Práticos: Construção de um <i>Pipeline</i> de Análise Documental 294
6.5.1	Alinhamento Automático de Documentos (<i>OCR/OMR Pre-processing</i>) 294
6.5.2	Algoritmo de Retificação de Inclinação (<i>Deskew</i>) 295
6.5.3	Modelagem Matemática 295
6.5.4	Limitações Práticas 298
6.6	Normalização de Fundo e Equalização Local de Contraste 298
6.6.1	Deteção de Bordas e Contornos 300
6.6.2	Isolamento, Segmentação e Decodificação do <i>QRCode</i> 302
6.6.3	Decodificação de Código de Barras 306
6.6.4	O MCTest como estudo de caso: do protótipo ao sistema em produção 307
6.7	Inspeção Industrial Automatizada 314
6.7.1	Referências e <i>Datasets</i> Públicos 314
6.7.2	Deteção de Defeitos por Análise de Textura 315

6.8	Resumo	318
6.9	 Uso do NotebookLM como Tutor Complementar	318
6.10	Exercícios Práticos	318
6.10.1	Exercício 1: Adaptação do <i>Pipeline</i> para Código de Barras	319
6.10.2	Exercício 2: Validação Robusta da Leitura	319
6.10.3	Exercício 3: Detecção de Marcação Inválida	319
6.10.4	Exercício 4: <i>Pipeline</i> Completo	319
6.11	Próximos Passos	320
Referências		321

Lista de Figuras

1.1	Diagrama relacional detalhando as distinções fundamentais, sinergias e interconexões entre PDI e VC no contexto de sistemas baseados em imagens.	21
1.2	Representação do fluxo sequencial de processamento: a saída de cada nível torna-se a entrada do nível subsequente.	23
1.3	Fluxo detalhado do PDI: da aquisição sensorial à extração de atributos e reconhecimento automatizado, incluindo em processamento colorido e compressão.	23
1.4	Representação do espectro visível e sua posição em relação às demais radiações eletromagnéticas, destacando a variação dos comprimentos de onda de 380 nm a 750 nm.	24
1.5	(A) Espectro eletromagnético completo em escala logarítmica, com destaque para a faixa visível. (B) Detalhe da luz visível (380-750 nm) e suas cores. (C) Decomposição da luz branca pelo prisma: menor comprimento de onda sofre maior refração, separando UV, visível e infravermelho.	24
1.6	Exemplos de imagens sintéticas representadas matricialmente.	28
1.7	Mandrill (Mandrillus sphinx) em ambiente natural. Crédito: Julien Renoult (CC BY 4.0). . .	29
1.8	Processamento básico de imagens: (a) imagem original, (b) imagem em tons de cinza, (c) imagem binarizada por limiar ($T=128$).	31
1.9	Comparação entre limiarização manual ($T=128$) e automática (Otsu) sobre a imagem em tons de cinza.	32
1.10	Acesso a um pixel específico (r, c) e a representação de sua vizinhança na imagem.	33
1.11	Estúdio do NotebookLM.	35
1.12	Simulador: Distâncias Euclidiana, <i>City-block</i> e <i>Chessboard</i>	40
1.13	Simulador: Desempenho Preditivo — Métricas de ML	47
1.14	Simulador: Mean Average Precision (mAP) - Curva P.S	51
1.15	Simulador: Estatísticas de Pixels	52
1.16	Simulador: Transformação de Negativo	54
1.17	Simulador: Percepção de Cor e Pesos ITU-R	56
1.18	Simulador: Limiarização Interativa	57
1.19	Simulador: Ajuste de Brilho e Contraste	59
1.20	Simulador: Gerador de Xadrez	61
1.21	Simulador: Estrutura do Arquivo PGM	62
1.22	Simulador: Filtro de Máximo Local 1D	65
2.1	Comparativo didático entre o sistema visual biológico e o eletrônico: no topo, a anatomia do olho humano destacando a retina e os fotorreceptores (cones e bastonetes); abaixo, a estrutura de uma câmera digital detalhando o sensor CMOS, a matriz de filtros de Bayer (RGGB) e o processo de quantização digital realizado pelo ADC.	67
2.2	Coletânea de desafios perceptivos: (topo esquerdo) Vaso de Rubin — ambiguidade figura-fundo; (topo central) Elefante de Shepard — incongruência geométrica; (topo direita) Sombra de Adelson — constância de brilho baseada no contexto; (inferior esquerdo) Grade de pontos — inibição lateral; (inferior direito) Escada de Schröder.	68
2.3	Imagem RGB (3 canais, $M \times N \times 3$) e versão RGBA (4 canais, $M \times N \times 4$) com transparência alfa gradual da esquerda para a direita. Imagem Mandrill — USC SIPI Image Database (domínio público).	72
2.4	Efeito da subamostragem. Os títulos exibem as dimensões ($W \times H$) e o tamanho da matriz em memória (KB).	74

2.5	Efeito da redução da profundidade de bits. Os títulos exibem a quantidade de bits, níveis e o tamanho em memória (KB).	75
2.6	Ilustração de vizinhanças 4 em uma matriz 3x3. No centro (1,1), o pixel de interesse.	76
2.7	Area de Proteção Ambiental Quilombos do Médio Ribeira - Barbudo-rajado (<i>Malacoptila striata</i>). Crédito: Thomas Fuhrmann (CC BY-SA 4.0).	79
2.8	Exemplo de translação da imagem do pássaro com deslocamentos (50,50) e (100,50).	81
2.9	Exemplo de rotação da imagem do pássaro em 30° e 45° usando interpolação bilinear.	82
2.10	Comparação de interpolação com zoom no detalhe do olho (recorte 60x60, ampliado 4x). Note o efeito de blocos no vizinho mais próximo vs. a suavização na bilinear.	84
2.11	Exemplo de cisalhamento da imagem do pássaro: horizontal (shx=0.3), vertical (shy=0.3) e combinado (shx=0.2, shy=0.2).	85
2.12	Simulador: Ajuste de Brilho e Contraste	89
2.13	Simulador: Subamostragem	91
2.14	Simulador: Quantização	93
2.15	Simulador: Ajuste de Brilho e Contraste	95
2.16	Simulador: Translação (tx, ty)	97
2.17	Simulador: Rotação (ângulo)	99
2.18	Simulador: Redimensionamento (Nearest vs Bilinear)	101
2.19	Simulador: Cisalhamento (shx, shy)	104
2.20	Simulador: Transformação Afim (matriz 2x3)	106
2.21	Simulador: Correção de Perspectiva (Homografia)	108
2.22	Aquisição da imagem de um Sudoku à esquerda. À direita, conversão para tons de cinza e redimensionamento. Crédito: Héctor Rodríguez de Guardamar, Espanha (CC BY 2.0).	110
2.23	Correção de perspectiva: original e vista frontal retificada.	111
2.24	Simulador: correção de perspectiva do Sudoku.	112
3.1	<i>Mandrill</i> (<i>Mandrillus sphinx</i>) fotografado em ambiente natural na África do Sul. A imagem possui resolução de 500 px e contém metadados EXIF com coordenadas GPS aproximadas (-26.20473, 28.22662). Crédito: Carlos Guilherme Rodrigues (CC BY-SA 3.0).	116
3.2	Operações aritméticas saturadas: adição de constante (clareamento) e subtração de constante (escurecimento com saturação em 0).	117
3.3	Retrato de um leopardo (<i>Panthera pardus</i>) em ambiente natural. Crédito: C. Brück (CC BY-SA 4.0).	118
3.4	<i>Alpha blending</i> entre recortes alinhados de mandrill e do leopardo (Figure 3.3) para diferentes valores de α . Em $\alpha=1$ vê-se apenas mandrill; em $\alpha=0$, apenas o leopardo; valores intermediários fundem os olhares das duas imagens proporcionalmente.	119
3.5	Operações lógicas bit a bit com máscara circular: AND (isolamento da ROI), OR (iluminação da ROI) e NOT (negativo).	120
3.6	Histogramas da imagem original, de uma versão escurecida e de uma versão clareada. Os valores na segunda linha indicam a quantidade de pixels saturados em 0 (preto) e 255 (branco).	121
3.7	Equalização de histograma em imagem 5x5 com 3 bits (L=8): imagem original com tons concentrados em {2,3,4}, imagem equalizada com tons redistribuídos para {1,5,7}, e os respectivos histogramas evidenciando o espalhamento das frequências.	123
3.8	Equalização de histograma: mm.equalize (global via CDF) vs. CLAHE (adaptativa com limitação de contraste). Os histogramas revelam o espalhamento progressivo das intensidades.	125
3.9	Especificação de histograma: mandril mapeada para o perfil tonal do leopardo. A CDF da saída aproxima a CDF de referência.	126
3.10	Correlação com <i>kernel</i> de média 3x3: versão didática (mm.conv0) vs. vetorizada (mm.conv via cv2.filter2D). A diferença máxima de 39 ocorre nas bordas.	133
3.11	<i>Patch</i> 5x5 extraído da imagem do leopardo (posição [250:255, 250:255]). A janela amarela destaca a vizinhança 3x3 centrada no pixel [1,1] onde a correlação será calculada.	134
3.12	Filtro de média com <i>kernels</i> de tamanho crescente (3x3, 7x7, 15x15). O borramento das bordas aumenta com o tamanho do <i>kernel</i>	135

3.13	<i>Kernel</i> Gaussiano 5×5 ($=1$): pesos normalizados, maiores no centro e decrescentes radialmente. A grade facilita a leitura de cada coeficiente.	136
3.14	Comparação entre filtro de média e Gaussiano (<i>kernel</i> 9×9 , $=0$). O Gaussiano preserva melhor as bordas, visível no detalhe do rosto.	137
3.15	Simulador: Filtragem Espacial de Realce 1D.	138
3.16	<i>Kernel</i> Laplaciano w4 aplicado ao <i>patch</i> 5×5 : a janela amarela destaca a vizinhança 3×3 onde o operador de segunda derivada é calculado.	140
3.17	Laplaciano aplicado à imagem do leopardo: resposta bruta (bordas detectadas) com w4 e w8, e imagens realçadas pela subtração do Laplaciano. w8 é mais sensível às diagonais.	141
3.18	Operador de Sobel na imagem do leopardo: Gx detecta bordas verticais, Gy detecta bordas horizontais, e a magnitude $ f $ combina ambos, revelando todas as bordas independente de direção.	143
3.19	Operador de Prewitt: Gx, Gy e magnitude — comparável ao Sobel, mas sem ponderação Gaussiana perpendicular.	143
3.20	Realce de nitidez por <i>Unsharp Masking</i> na imagem do leopardo: a imagem suavizada é subtraída da original para gerar a máscara de alta frequência, que é então reintroduzida para realçar bordas e detalhes.	145
3.21	<i>Unsharp Masking</i> na imagem do leopardo com $=1$ e k variando de 0.5 a 8.0. Para $k > 2$ surgem artefatos nas bordas (halos) e o ruído de fundo começa a ser visível.	146
3.22	Detector de Canny com diferentes pares de limiar: limiares baixos capturam mais bordas (inclusive ruído); limiares altos retêm apenas as bordas mais fortes.	147
3.23	Ruído sal e pimenta com densidades crescentes (2%, 5%, 10%). O parâmetro prob indica a fração total de pixels corrompidos, metade sal (255) e metade pimenta (0).	148
3.24	Comparação de filtros para remoção de ruído sal e pimenta (10%): Gaussiano, Média, Mediana, Bilateral e Morfológico. Linha superior: imagens completas; linha inferior: crop da região de interesse.	151
3.25	<i>Pipeline</i> de pré-processamento: CLAHE $\rightarrow$ Gaussiano $\rightarrow$ Canny. Cada etapa prepara a imagem para a seguinte, resultando em bordas limpas e bem definidas.	152
3.26	Simulador: Adição Saturada de Constante	156
3.27	Simulador: Alpha <i>Blending</i> de Duas Imagens	158
3.28	Simulador: Inversão de Imagem (Negativo Fotográfico)	160
3.29	Simulador: Equalização de Histograma (L bits)	162
3.30	Simulador: Aplicação de Máscara AND Binária	164
3.31	Simulador: Filtro de Média com <i>Kernel</i> $N \times N$	166
3.32	Simulador: Operador Laplaciano (w4) para Realce de Bordas	169
3.33	Simulador: Gradiente de Sobel: Gx e Gy	171
3.34	Simulador: Filtro da Mediana 3×3	173
3.35	Simulador: <i>Unsharp Masking</i> (USM)	176
4.1	Imagem com moedas de vários tipos. Crédito: GAZI.MD.AHAD (CC BY-SA 4.0).	179
4.2	Comparação dos pré-processamentos: imagem histograma+T* $^2B(T)$ Otsu. A melhor separação bimodal indica o limiar mais confiável.	182
4.3	Elemento estruturante em formato de cruz (B_{cruz}) utilizado para conectividade-4.	184
4.4	Erosão e dilatação em imagem binária 10×10 com elemento estruturante 'L' assimétrico 3×3 . Validação da dualidade erosão-dilatação.	190
4.5	Simulador interativo de erosão morfológica: visualização do critério de inclusão do elemento estruturante assimétrico B_L sobre a imagem binária A.	191
4.6	Abertura, fechamento, composição e filtro sequencial alternado aplicados à binarização Otsu das moedas com CLAHE. Elemento estruturante: disco 13×13	193
4.7	Simulador interativo de dilatação geodésica: o marcador f (verde) propaga-se passo a passo pelos caminhos livres da máscara g , sem atravessar paredes — ilustrando como $\text{delta}_g^{(n)}(f)$ encontra a saída do labirinto.	194

4.8	Resolução de labirinto via Operações Complementares: Invertendo a máscara e o marcador no início do <i>pipeline</i> , o fluxo é resolvido diretamente no domínio complementar através de <i>mm.cero</i> e <i>mm.suprec</i> , eliminando re-inversões redundantes.	197
4.9	<i>Pipeline</i> de Reconstrução Morfológica por Dilatação Condicionada: a máscara contém dois objetos, o marcador isola apenas o núcleo do objeto principal, e as iterações subsequentes em laço reconstroem sua forma exata até a convergência.	199
4.10	<i>Pipeline</i> de preenchimento de buracos (<i>clohole</i>): o marcador é obtido a partir da borda da imagem e restringido ao complemento f^c . As iterações de dilatação geodésica reconstruem o fundo externo; após a complementação final, os buracos internos ficam preenchidos.	201
4.11	<i>Pipeline</i> de eliminação de estruturas de borda (<i>edgeoff</i>): o marcador captura as raízes conectadas às extremidades da matriz, a reconstrução delimita a extensão desses elementos e a subtração booleana preserva unicamente os objetos totalmente internos.	202
4.12	<i>Pipeline</i> completo de segmentação com CLAHE: Otsu → abertura ($r=9$) → <i>clohole</i> → abertura ($r=33$) → <i>edgeoff</i>	203
4.13	Simulador interativo avançado de morfologia com elemento estruturante editável e perfil 1D. .	204
4.14	Morfologia em tons de cinza e seus respectivos histogramas. A erosão reduz intensidades locais, a dilatação as amplia, o gradiente destaca bordas, e os operadores Top-hat e Black-hat evidenciam detalhes locais brilhantes e escuros. Elemento estruturante: disco de diâmetro 19.	206
4.15	Algoritmo de rotulação por <i>flood-fill</i> com pilha: passo a passo, cards, linha do tempo, código Python e fluxograma.	208
4.16	Simulador interativo de rotulação de componentes conexas (<i>connected component labeling</i>): visualização da expansão <i>flood-fill</i> , conectividade-4 e conectividade-8.	208
4.17	Efeito da conectividade na rotulação: a imagem binária 10×10 contém pixels diagonalmente adjacentes. Com conectividade-4, esses pixels formam componentes distintas; com conectividade-8, fundem-se em uma única componente.	209
4.18	Algoritmo da Transformada de Distância: passo a passo, cards, linha do tempo, código Python e fluxograma.	210
4.19	Simulador interativo da Transformada de Distância (TD) iterativa via erosão em tons de cinza. Pixels fora da imagem agora assumem o valor máximo (144), propagando os custos apenas a partir do fundo interno.	211
4.20	Transformada de Distância em imagem binária 10×10 . Esquerda: imagem original (<i>foreground</i> = 255). Centro: <i>mm.dist1</i> iterativa (erosões com elemento cruz). Direita: <i>mm.dist</i> (L2 Euclidiana).	212
4.21	Simulador interativo 2D (4×4) com sincronização estrita de filas de propagação horizontal (b) para obter a convergência exata descrita no artigo.	213
4.22	Menor caminho geodésico em um labirinto. As distâncias geodésicas são calculadas a partir da entrada e da saída usando <i>mm.gdist</i> . Os pixels cujo somatório das duas distâncias é igual à distância mínima entre os marcadores pertencem a um caminho ótimo.	214
4.23	Algoritmo Didático de <i>Watershed</i> por Crescimento de Regiões Limitado por Máscara: passo a passo, cards, linha do tempo, código Python e fluxograma.	215
4.24	Simulador interativo do Algoritmo <i>Watershed</i> por propagação morfológica. Desenhe a máscara, posicione os marcadores e ajuste o Elemento Estruturante para observar a inundação. Quando as bacias se encontram simultaneamente, o empate é resolvido assumindo uma das regiões de forma aleatória.	216
4.25	<i>Pipeline watershed</i> delimitado por máscara em imagem binária 20×20	217
4.26	<i>Pipeline Watershed</i> para separação de moedas sobrepostas: da máscara binária dilatada até os contornos finais anotados sobre a imagem original.	219
4.27	Componentes conexas extraídos após o <i>pipeline</i> CLAHE → Otsu → limpeza morfológica. Cada objeto é colorido com cor distinta e anotado com sua área em pixels.	221
4.28	Contornos extraídos com <i>findContours</i> . Cada moeda é anotada com sua circularidade — valores próximos de 1 confirmam forma circular.	223

4.29	<i>Bounding boxes</i> derivadas dos componentes conexos sobrepostas à imagem original. As anotações são exportadas no formato YOLO (<i>classe cx cy w h</i>), com coordenadas normalizadas para o intervalo $[0,1]$	224
4.30	Simulador: Limiarização Global por Limiar Fixo	230
4.31	Simulador: Limiarização Automática de Otsu	232
4.32	Simulador: Dilatação Binária Plana (mm.dil0)	235
4.33	Simulador: Erosão Binária Plana (mm.ero0)	237
4.34	Simulador: Abertura Morfológica (erosão + dilatação)	239
4.35	Simulador: Fechamento Morfológico (dilatação + erosão)	241
4.36	Simulador: Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)	244
4.37	Simulador: Gradiente Morfológico, Top-hat e Black-hat	246
4.38	Simulador: Transformada de Distância	248
4.39	Simulador: Separação de Blobs, Rotulação e Descritores	250
5.1	Decomposição de Fourier 1D: uma onda quadrada (linha tracejada) é aproximada pela soma das primeiras senoides (linhas coloridas). Quanto mais termos, melhor a aproximação.	252
5.2	Toda frequência no espectro (ponto isolado) corresponde a uma onda senoidal 2D rotacionada no domínio espacial.	253
5.3	Diagrama conceitual do espectro de Fourier 2D centrado.	254
5.4	Experimento de troca de fase: Imagem A (moedas) e Imagem B (padrão geométrico) reconstruídas com magnitudes e fases trocadas. O resultado confirma que a estrutura visual segue a fase — a imagem reconstruída com a fase de B se parece com B, independentemente de qual magnitude foi usada.	256
5.5	Simulador interativo da decomposição de Fourier 1D: visualização da soma de senoides com diferentes frequências, amplitudes e fases. Adicione termos e observe a convergência para formas de onda arbitrárias.	256
5.6	Comparação de eficiência: Convolução Não Separável (Espacial 2D), Separável (Espacial 1D) e via FFT.	258
5.7	Sem padding, um deslocamento severo faz a imagem vazar para o lado oposto (convolução circular).	259
5.8	Verificação do Teorema da Convolução: a diferença pixel a pixel entre a convolução espacial (cv2.filter2D) e a multiplicação em frequência (FFT) é numericamente nula — confirmando a equivalência teórica.	260
5.9	A Dualidade Perigosa: O corte abrupto na Frequência (Cilindro) transforma-se obrigatoriamente numa Sinc espacial. Suas ondulações causam o <i>ringing</i> fantasma nas bordas da imagem.	261
5.10	Filtros passa-baixa.	262
5.11	Simulador interativo de filtros no domínio da frequência.	263
5.12	Comparação entre filtros passa-baixa: Ideal ($D=30$), Gaussiano ($D=30$) e Butterworth ($D=30$, $n=2$). Perfis de $H(u,v)$ ao longo de uma linha central e imagens filtradas correspondentes.	264
5.13	Filtro passa-alta Gaussiano. (a) Original; (b) Filtro passa-alta ($D=30$) - as bordas das moedas e fundo texturizado são realçados.	265
5.14		266
5.15	Fourier global é cego para a posição. Os espectros não dizem onde as bordas estão.	267
5.16	Funções da Wavelet (). Note como elas rapidamente decaem para zero (suporte compacto), ao contrário das senoides infinitas de Fourier.	269
5.17	Diagrama da decomposição wavelet 2D em dois níveis.	269
5.18	Decomposição wavelet 2D de 2 níveis com wavelet Haar: subbandas LL, LH, HL, HH em cada nível. As subbandas de detalhe revelam estruturas orientadas em diferentes escalas.	270
5.19	Comparação entre famílias de wavelets: Haar, db4, sym4 e bior2.2. Subbanda LL (aproximação) e HH (diagonal) para cada escolha, ilustrando o compromisso entre compactação e suavidade.	271

5.20	Reconstrução wavelet com limiamento de coeficientes (hard thresholding): à medida que o limiar aumenta, mais detalhes são zerificados, produzindo imagens progressivamente mais suaves. Métrica PSNR quantifica a perda de qualidade.	272
5.21	O Alfabeto Visual do JPEG: As 64 funções de base da DCT-II. O coeficiente DC fica no topo esquerdo (suave). Ao descer e avançar à direita, a oscilação espacial aumenta drasticamente. .	275
5.22	DCT 2D em bloco 8×8: coeficientes e reconstrução progressiva.	277
5.23	Pipeline JPEG simplificado: DCT em blocos 8×8, quantização com diferentes fatores de qualidade e reconstrução via IDCT. O artefato de blocos torna-se visível para qualidades baixas (Q=10–20).	279
5.24	Simulador interativo de compressão DCT-JPEG: ajuste o fator de qualidade e visualize em tempo real os coeficientes zerados, o bloco reconstruído e o erro de quantização.	280
5.25	Forte compressão (Q=10). À esquerda o artefato de bloco clássico da DCT. À direita, a suavização de bordas característica do WebP.	281
5.26	Curva taxa-distorção: PSNR vs tamanho de arquivo para JPEG, WebP e PNG.	283
5.27	Mapas de erro JPEG em diferentes qualidades: artefatos de bloco nas bordas.	284
5.28	Pipeline completo de remoção de ruído misto: (1) ruído gaussiano + periódico adicionado; (2) picos periódicos identificados no espectro; (3) filtro notch de supressão suave (Gaussiana) aplicado; (4) filtro bilateral remove o gaussiano residual.	286
5.29	Mapa conceitual do domínio da frequência.	287
6.1	Algumas imagens públicas disponíveis em <i>skimage.data</i>	293
6.2	<i>Pipeline</i> de ingestão de documentos: rasterização adaptativa de páginas PDF para matrizes discretas em formato PNG, exibindo a página dois.	295
6.3	Simulador interativo de correção de inclinação (<i>deskew</i>): mova o slider para inclinar o documento e observe as três etapas do <i>pipeline</i> — imagem inclinada, bordas Canny e resultado corrigido.	296
6.4	<i>Pipeline</i> de retificação axial: exibição comparativa entre a entrada rotacionada original, o mapa de gradientes estruturais de Canny e o resultado final alinhado com fundo normalizado em branco.	297
6.5	Efeito da normalização de fundo e do CLAHE na limiarização por Otsu: da esquerda para a direita, qualidade crescente de segmentação.	299
6.6	Detecção dos discos marcadores, extração de contornos e retificação por transformação de perspectiva.	302
6.7	Simulador interativo de filtragem por circularidade: mova o slider para ajustar o limiar C e observe quais componentes são aceitos (verde) ou rejeitados (vermelho).	303
6.8	<i>Pipeline</i> de processamento do <i>QRCode</i> : limiarização, abertura morfológica, inversão e recorte final para decodificação.	305
6.9	Simulador interativo do <i>pipeline</i> de isolamento do <i>QRCode</i> : navegue pelas etapas de filtragem, ajuste o elemento estruturante do fechamento morfológica e veja a detecção do contorno quadrado sobre a imagem original do gabarito.	306
6.10	Decodificação de código de barras linear com <i>pyzbar</i> : imagem de entrada e dados extraídos. .	307
6.11	Imagem da folha de respostas em escala de cinza carregada a partir do PDF rasterizado. . .	309
6.12	Área de respostas extraída por <i>getAnswerArea</i> : região retificada contendo os quadros de marcação.	310
6.13	Região do <i>QRCode</i> isolada por <i>segmentQRcode</i> dentro da área de respostas retificada. . . .	311
6.14	Recorte inferior da área de respostas, concentrando os quadros de bolhas a serem segmentados.	311
6.15	Respostas lidas automaticamente pelo MCTest após segmentação e classificação de todas as bolhas.	313
6.16	Detecção de defeito por subtração de imagem: produto de referência, imagem com defeito simulado e máscara de anomalia detectada.	315
6.17	Simulador interativo de inspeção industrial por subtração de imagens: controle as distorções geométricas de desalinhamento (registro) e o limiar de detecção para observar o impacto nos falsos positivos.	316
6.18	Detecção de heterogeneidade de textura: mapa de variância local e máscara de anomalia. . .	317

Lista de Tabelas

1.1	Conexão entre PDI, VC e outras áreas da ciência.	22
1.2	Principais tipos de imagem digital e seus respectivos conjuntos de valores possíveis para cada pixel.	25
1.3	Principais bibliotecas utilizadas ao longo deste livro para manipulação matricial, PDI e visualização de resultados.	26
2.1	Convenções de escala e ordem de canais nas principais bibliotecas de PDI.	69
2.2	Comparativo entre estratégias de compactação e eficiência de processamento.	76
2.3	Comparativo de métricas de distância aplicadas à malha de pixels.	77
2.4	Principais formatos de armazenamento de imagens digitais e suas aplicações em PDI.	77
2.5	Métodos de interpolação disponíveis em <code>mm.resize</code> e seus respectivos números de vizinhos utilizados no cálculo.	83
3.1	Algoritmo de equalização de histograma.	121
3.2	Interpretação típica dos coeficientes do <i>kernel</i>	131
3.3	Etapas do <i>Unsharp Masking</i>	144
3.4	Média vs. mediana com um pixel corrompido. A mediana ignora o outlier; a média é deslocada ~40 níveis.	147
4.1	Técnicas de pré-processamento avaliadas para melhorar a separação entre moedas e fundo antes da aplicação do método de Otsu.	180
4.2	Propriedades de abertura e fechamento.	190
4.3	Sequências do filtro sequencial alternado <code>mm.asf</code>	192
4.4	Comparação entre os operadores <i>clohole</i> e <i>edgeoff</i>	200
4.5	Taxonomia simplificada das principais abordagens de segmentação e refinamento estudadas neste capítulo.	207
4.6	<i>Pipeline</i> completo do <i>watershed</i> baseado em marcadores para imagens reais.	215
4.7	<i>Pipeline</i> simplificado utilizado no exemplo didático da Figure 4.25.	217
4.8	Comparação entre as abordagens baseadas em componentes conexos e em contornos.	219
5.1	Comparação de complexidade entre convolução direta, separável e via FFT.	257
5.2	Tipos de redundância em imagens e como são exploradas.	273
5.3	Comparação entre os principais formatos de imagem rasterizados.	280
6.1	Principais imagens públicas disponíveis no módulo <i>skimage.data</i>	292

PDI e VC — Python

Bem-vindo ao livro de PDI e Visão Computacional — versão Python / Português.

Organização

- **Parte I — Fundamentos de PDI** — Representação, histogramas, filtragem, morfologia
 - **Parte II — Visão Computacional** — Segmentação, descritores, detecção, deep learning
-

Prefácio

Este livro é uma **obra em constante atualização** sobre **Processamento Digital de Imagens (PDI) e Visão Computacional (VC)**, concebido como material didático interativo para cursos de graduação e pós-graduação em Computação, Engenharia e áreas correlatas.

! Destaque

A obra segue a metodologia descrita em Zampirolli et al. (2025) — extensão de Zampirolli et al. (2024), trabalho premiado na trilha de “Recursos e Ambientes Educacionais” da **EduComp 2024**. O conteúdo integra a biblioteca `morph.py`, desenvolvida pelo autor.

Um livro vivo

O material não é estático: o conteúdo evolui continuamente à medida que exemplos são refinados, novas seções são incorporadas e as abordagens pedagógicas são atualizadas com base no retorno dos usuários. Por essa razão, o material é tratado como um *projeto em andamento* — uma base em permanente expansão.

Uso de ferramentas de Inteligência Artificial

A produção deste livro conta com o apoio de **ferramentas de Inteligência Artificial (IA)** para elevar a qualidade do texto e do código. A versão inicial dos capítulos foi submetida a processos de revisão por meio de assistentes em versões gratuitas, como **DeepSeek**, **Gemini** e **Claude**. Por meio de *prompts* iterativos, tais ferramentas auxiliaram nas seguintes tarefas:

- refinamento da clareza e da precisão das explicações;
- detecção e correção de erros de sintaxe, lógica e digitação nos exemplos de código;
- sugestão de melhorias na organização e na fluidez textual.

Adicionalmente, **ilustrações foram geradas com o Gemini** com o propósito de auxiliar a visualização de conceitos abstratos da área.

Como este livro é produzido

O conteúdo é integralmente redigido em **Quarto** (armazenado na pasta `all` do repositório github.com/fzampirolli/pdi-vc), um sistema de publicação científica que permite renderizar o código-fonte em múltiplos formatos:

Formato	Descrição
HTML	Versão web com navegação interativa: fzampirolli.github.io/pdi-vc
PDF	Versão para impressão ou leitura <i>offline</i> : livro.pt.py.pdf
Notebooks (.ipynb)	Versão processada por filtro personalizado que resolve citações e numeração de elementos, formatando referências no padrão ABNT. Compatível com Jupyter e Google Colab .

No início de cada capítulo, botões “*Open in Colab*” direcionam para dois ambientes:

1. **Parte Teórica:** localizada na abertura do capítulo.
2. **Parte Prática:** localizada na seção  **Parte Prática com Exercícios de Programação**, ao final do capítulo.

O *script* de pós-processamento (`gerar_notebooks_alunos.py`) prepara os arquivos para distribuição, garantindo seu funcionamento em ambientes interativos sem necessidade de instalação local do Quarto.

A biblioteca `morph.py`

A biblioteca `morph.py` (Zampirolli et al., 2025) é utilizada ao longo de toda a obra. Desenvolvida para fins pedagógicos, ela oferece uma interface simplificada sobre bibliotecas consolidadas — como **OpenCV**, **NumPy** e **Matplotlib** —, permitindo que o leitor concentre-se nos conceitos teóricos sem se preocupar com detalhes específicos de API.

Herança Técnica

A estrutura da `morph.py` tem como base ferramentas de morfologia matemática desenvolvidas no Brasil, como **MMachLib** (Lotufo et al., 1997) e **MMach** (Barrera et al., 1998), além da **mmorph**, empregada na obra de Dougherty; Lotufo (2003). Essas ferramentas serviram de referência para o ensino da área por décadas.

A biblioteca permite realizar leitura, exibição, conversões de cores, filtragem e morfologia matemática de forma concisa:

```
import morph as mm

img = mm.read('lena.jpg')      # leitura da imagem
mm.show(img)                   # exibição
neg = mm.neg(img)               # inversão (negativo)
mm.show(neg)
```

O código-fonte está disponível em: github.com/fzampirolli/pdi-vc/tree/master/morph.

Exercícios de Programação (EPs) e Validação Automática

Cada unidade do livro inclui **Exercícios de Programação (EPs)** práticos e de complexidade crescente, projetados para consolidar a teoria por meio da biblioteca `morph.py` ou do padrão da linguagem escolhida.

Estrutura e Validação Local

A validação das soluções é feita localmente pela classe `TestSuite` (`testsuite.py`), que compara a saída do programa com os arquivos de casos de teste (`.cases`):

```
TestSuite("EP01_01.py").run()
```

O sistema suporta múltiplas linguagens (Python, Java, C++, C, JavaScript e R) e pode ser integrado diretamente ao Moodle.

Integração com o Moodle (VPL)

Os EPs dos *notebooks* podem ser convertidos em atividades **VPL** (*Virtual Programming Lab*) no Moodle. O *script* `ep_tools.py` (executado via `make build`) automatiza a exportação dos EPs do final de cada capítulo em duas etapas:

1. **Extração** (`make eps`): gera um HTML interativo independente por EP, com enunciado, exemplos e simulador, em `gen/book/eps/<versao>/`. Veja um exemplo em: https://fzampirolli.github.io/pdi-vc/eps/py.pt/EP01_01.html
2. **Conversão** (`make moodle`): transforma o HTML em um fragmento autocontido, sem dependência de CSS externo, pronto para ser colado no editor do Moodle, em `gen/book/eps/<versao>_moodle/`. Veja um exemplo em: https://fzampirolli.github.io/pdi-vc/eps/py.pt_moodle/EP01_01.html

Ao publicar com `make publish`, essas **duas versões** de cada EP ficam disponíveis nos links indicados. O professor pode combinar as duas estratégias: colar a versão Moodle no editor VPL (com simulador funcional) e incluir no enunciado um link para a versão completa, onde as fórmulas são renderizadas corretamente pelo MathJax.

Revisão obrigatória antes de publicar no Moodle

Embora automatizada, a conversão exige revisão do professor em razão das limitações do editor TinyMCE/HTML Purifier do Moodle:

- **Fórmulas matemáticas** — O TinyMCE descarta barras invertidas (`\`), corrompendo equações LaTeX. Por isso, a versão Moodle converte fórmulas simples para HTML puro (entidades como `≥`, `×`). Fórmulas complexas (`\begin{cases}`, matrizes, integrais) podem sair incompletas — revise e, se necessário, reescreva-as em texto ou indique a versão completa via link.
- **Referências cruzadas** — Links como `@fig-...` ou `@tbl-...` são removidos, mantendo apenas o texto do rótulo.
- **Figuras externas** — Imagens complementares devem ser inseridas manualmente na atividade VPL.
- **Simuladores** — Funcionam perfeitamente desde que utilizem JavaScript padrão com estilos *inline* e manipulação direta do DOM (`getElementById`). **Atenção:** se incluir fórmulas em LaTeX que contenham o caractere `\` no Moodle, os simuladores podem parar de funcionar.

Como Publicar no Moodle

1. Acesse a versão Moodle do EP desejado:

```
https://fzampirolli.github.io/pdi-vc/eps/py.pt_moodle/EPXX_YY.html
```

2. Copie o código-fonte completo da página (`Ctrl+U` → `Ctrl+A` → `Ctrl+C`).
3. No Moodle, crie a atividade VPL, acesse o editor HTML, cole o conteúdo (`Ctrl+V`) e salve.
4. Opcionalmente, inclua no enunciado VPL um link para a versão completa:

```
https://fzampirolli.github.io/pdi-vc/eps/py.pt/EPXX_YY.html
```

5. Importe os arquivos `.cases` da pasta `all/capXX/casos/` nas configurações de testes do VPL.

Reaproveitamento dos Casos de Teste

Os arquivos `.cases` são 100% compatíveis com o VPL, permitindo usar o mesmo conjunto de testes tanto no desenvolvimento local quanto na correção automatizada no Moodle.

Idiomas e Conversão de Código

O **núcleo de referência** é escrito em **Português** com exemplos em **Python**. Versões em outros idiomas e linguagens de programação são geradas via **APIs de IA**, por meio de um fluxo automatizado que realiza a tradução textual e a conversão de sintaxe do código.

 Nota

As versões traduzidas via IA servem como **ponto de partida** e podem conter imprecisões; recomenda-se revisão antes da aplicação em sala de aula.

Código aberto

O projeto é regido por princípios de código aberto. O repositório público reúne o texto, os códigos, as imagens e os *scripts*: github.com/fzampirolli/pdi-vc

Cada versão do livro é arquivada permanentemente no [Zenodo](https://zenodo.org/record/2078460) com um DOI citável. Para citar este material em trabalhos acadêmicos:

ZAMPIROLI, Francisco de Assis. **PDI+VC — Processamento Digital de Imagens e Visão Computacional**. UFABC, 2026. DOI: [10.5281/zenodo.20784606](https://doi.org/10.5281/zenodo.20784606)

Antes de começar: *Notebooks* em Python

O conceito de *Literate Programming* (Programação Literária), proposto por Donald Knuth (Knuth, 1984), fundamenta a estrutura deste material. A lógica inverte o paradigma tradicional: o programa é escrito para a leitura humana, assemelhando-se a um ensaio, enquanto o código é extraído separadamente para execução computacional.

O conteúdo é estruturado em *notebooks* — documentos que intercalam **células de texto** (em [Markdown](#)) e **células de código** (em Python).

- **Execução:** células de código são identificadas por []. A execução pode ser realizada com **Shift + Enter** ou pelo botão ▶ da interface.
- **Ambientes:** os *notebooks* podem ser executados localmente, por meio do [Jupyter](#) ou do [Visual Studio Code \(VS Code\)](#), bem como em ambientes de nuvem, como o [Google Colab](#).

 Nota sobre o formato

Em ambientes interativos, o código pode ser modificado e executado. Nas versões estáticas (HTML ou PDF), os blocos de código têm finalidade de leitura e referência, sem prejuízo à integridade das explicações.

Parte I

Parte I — Fundamentos de PDI

Capítulo 1

Fundamentos e Primeiros Passos



Este capítulo inaugura a **Parte 1** do livro, dedicada aos fundamentos do Processamento Digital de Imagens (PDI). São apresentados a representação matemática de imagens digitais e os principais métodos para sua manipulação, utilizando a linguagem Python e a biblioteca `morph.py` (Zampirolli et al., 2025).

1.1 Objetivos

Ao final deste capítulo, você será capaz de:

- Compreender a natureza física e matemática da imagem digital $f(x, y)$.
- Identificar as faixas do espectro eletromagnético relevantes para PDI.
- Configurar o ambiente de desenvolvimento em Python.
- Realizar operações básicas: leitura, exibição e salvamento de imagens.
- Manipular estruturas de matrizes (NumPy) sem cair em armadilhas de memória.
- Acessar e modificar intensidades de pixels individualmente.
- Aplicar limiarização manual.

1.2 Antes de começar: *Notebooks* em Python

Este material foi construído sob o conceito de *Literate Programming* (Programação Literária), idealizado por Donald Knuth na década de 1980 (Knuth, 1984). Knuth — também criador do sistema **TeX** para tipografia digital — propôs que os programas fossem escritos como uma narrativa lógica para seres humanos, intercalando código e documentação.

Para executar uma célula, pressione Shift + Enter ou clique no botão ▷.

i Nota sobre o formato

Nas versões renderizadas (PDF ou HTML), o código é apresentado em blocos estáticos para fins de leitura e referência. A execução interativa requer o acesso via Google Colab (disponível no topo da página) ou em ambiente local via VSCode ou Jupyter Notebook.

1.3 Fundamentos

O estudo de sistemas baseados em imagens compreende um ecossistema de disciplinas integradas que transformam dados visuais brutos em conhecimento estruturado. Enquanto algumas áreas focam na geração de representações, outras dedicam-se ao tratamento e à análise desses dados para dar suporte a aplicações tecnológicas complexas.

O diagrama apresentado no Figure 1.1 estabelece a distinção e complementaridade entre o Processamento Digital de Imagens (PDI) e a Visão Computacional (VC). O PDI, destacado em **verde**, tem como foco

a transformação de imagem para imagem, visando melhoria de qualidade ou pré-processamento, como a remoção de ruídos e realce de contraste.

Em contrapartida, a VC, assinalada em **azul**, foca na interpretação do conteúdo visual para extrair modelos ou informações, como o reconhecimento de objetos e gestos. A região de intersecção ilustra a sinergia entre as áreas, onde o PDI prepara os dados visuais para a interpretação pela VC. O mapa também demonstra as interconexões de ambas as disciplinas com áreas como Robótica, Computação Gráfica, Inteligência Artificial (IA) e Neurociência.

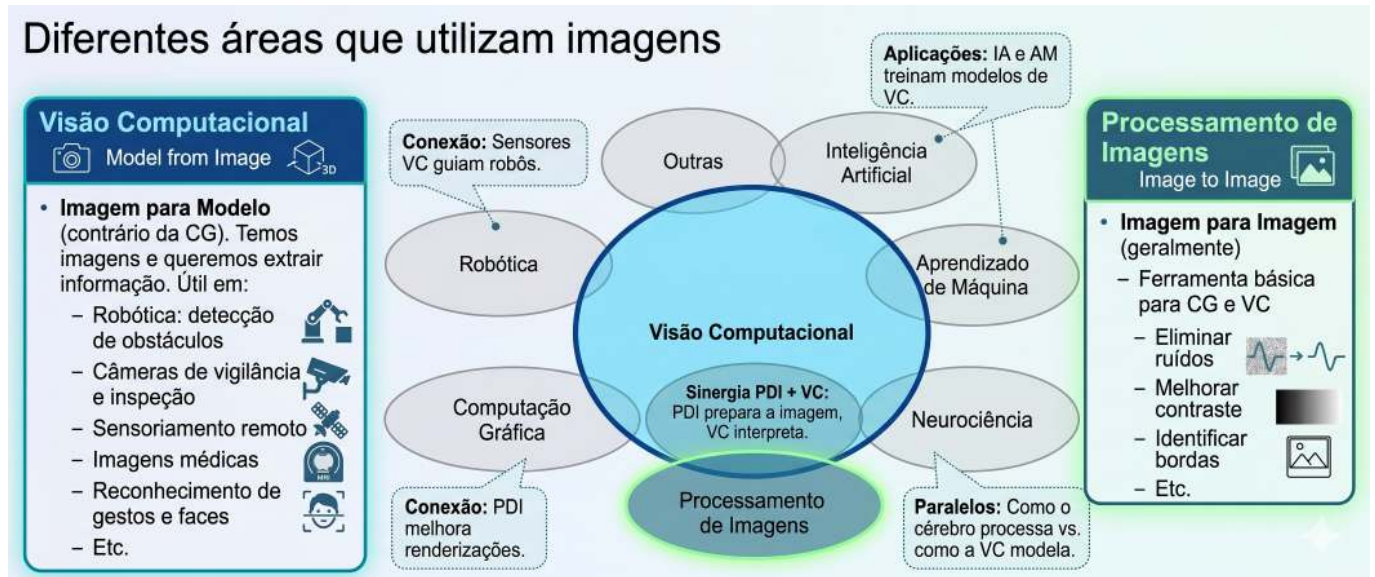


Figura 1.1: Diagrama relacional detalhando as distinções fundamentais, sinergias e interconexões entre PDI e VC no contexto de sistemas baseados em imagens.

1.3.1 Visão Computacional

- Foco:** *Imagem* → *Modelo* (caminho inverso da Computação Gráfica).
- Objetivo:** Extrair informação de alto nível a partir de imagens ou vídeos.
- Aplicações típicas:**
 - Robótica - detecção de obstáculos, localização e navegação autônoma.
 - Vigilância e inspeção - reconhecimento de eventos, leitura de placas, controle de qualidade.
 - Sensoriamento remoto - análise de imagens de satélite, mapeamento ambiental.
 - Imagens médicas - detecção de tumores, segmentação de órgãos, auxílio ao diagnóstico.
 - Interação humano-computador - reconhecimento de gestos, expressões faciais, rastreamento ocular.
- Relação com outras áreas:** utiliza técnicas de Aprendizado de Máquina e IA para classificar e interpretar cenas; serve como “olhos” da Robótica.

1.3.2 Processamento Digital de Imagens (PDI)

- Foco:** *Imagem* → *Imagem* (geralmente - transformação de uma imagem em outra).
- Objetivo:** Melhorar a qualidade visual ou extrair características de baixo nível.
- Aplicações comuns:**
 - Eliminação de ruídos (filtros de média, mediana, gaussiano).
 - Melhoria de contraste (equalização de histograma, ajuste gamma).
 - Detecção de bordas (Sobel, Canny, Laplaciano).
 - Segmentação (limiarização, crescimento de regiões, *watershed*).

- Transformações geométricas (redimensionamento, rotação, correção de perspectiva).
- **Relação com outras áreas (ver Table 1.1):**
 - É a **base** para a maioria dos sistemas de VC (pré-processamento).
 - A Computação Gráfica frequentemente aplica PDI para pós-processamento (ex.: suavização, realce).
 - Técnicas de IA podem otimizar parâmetros de processamento (ex.: aprendizado de filtros).

Tabela 1.1: Conexão entre PDI, VC e outras áreas da ciência.

Área	Relação com PDI e VC
Inteligência Artificial	Fornecer modelos (redes neurais, SVM) que interpretam saídas da VC.
Robótica	Consome dados de VC para tomar decisões (navegação, manipulação).
Aprendizado de Máquina	Usa descritores extraídos pelo PDI/VC para treinar classificadores.
Computação Gráfica	Caminho inverso: <i>modelo</i> $\rightarrow$ <i>imagem</i> ; muitas vezes aplica PDI para renderização realista.
Neurociência	Inspira modelos de PDI (ex.: filtros semelhantes a células ganglionares da retina).

1.4 Etapas do PDI

As etapas do PDI são apresentadas na Figure 1.2, que podem ser compreendidas como uma cadeia de transformações que reduz a redundância dos dados em busca de significado:

- **Baixo Nível:** Atua diretamente sobre os pixels da imagem ruidosa para realizar melhorias e filtragens, gerando como saída uma imagem limpa ou realçada.
- **Médio Nível:** Recebe a imagem tratada e realiza a segmentação e descrição, transformando a matriz de pixels em atributos estruturados (forma, tamanho e textura).
- **Alto Nível:** Utiliza a tabela de atributos para alimentar processos de lógica e inteligência artificial, resultando na decisão ou reconhecimento final (como o diagnóstico médico).

A Figure 1.3 detalha a sequência completa do PDI, desde a captura até a interpretação. O fluxo inicia-se na Aquisição da Imagem (1) e prossegue com o Melhoramento (2) e a Restauração (3). Em seguida, o conteúdo é isolado pela Segmentação (4) e refinado pela Morfologia (5). A transição crucial ocorre na Representação e Descrição (6), onde objetos visuais são convertidos em dados matemáticos (área, perímetro etc.), permitindo o Reconhecimento (7). Processos auxiliares incluem o Processamento de Imagem Colorida e a Compressão, que contribuem para a eficiência do armazenamento e da análise.

1.5 Formação da Imagem e o Espectro

O processo de formação de uma imagem é fundamentado na interação entre a matéria e a energia radiante. Essencialmente, uma imagem é concebida quando um **sensor registra a radiação** resultante da interação com um objeto físico. No contexto da visão humana e da fotografia convencional, este fenômeno depende de uma fonte de luz que ilumine a cena; as características dos objetos são então codificadas através das **variações de intensidade e cor** da luz que atinge o sensor, conforme ilustrado na Figure 1.4.

A **luz visível** ocupa apenas uma pequena faixa do espectro eletromagnético — entre **380 nm (violeta)** e **750 nm (vermelho)** — conforme ilustrado na Figure 1.5. Sensores digitais convencionais operam nessa mesma janela, mas equipamentos especializados podem captar radiações invisíveis ao olho humano, como o infravermelho e os raios X. Em PDI, a imagem formada depende diretamente da sensibilidade espectral do sensor utilizado.

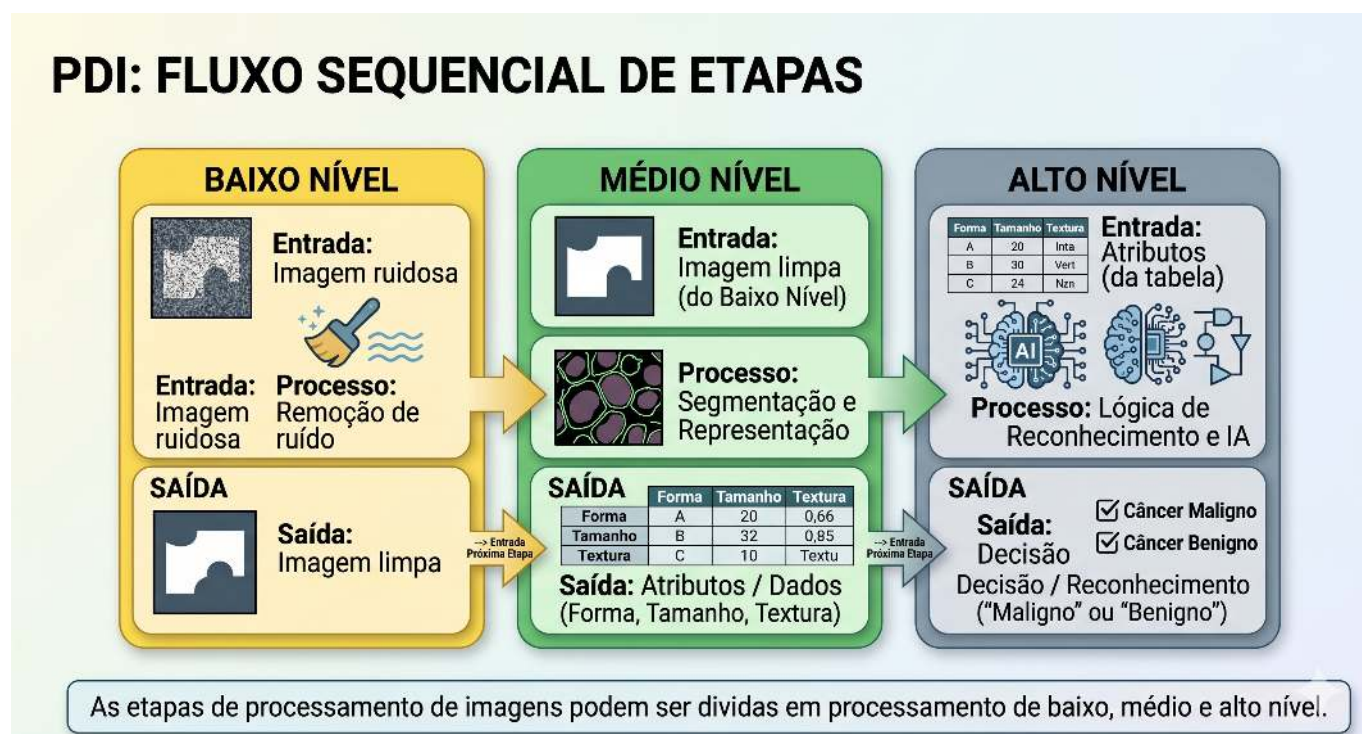


Figura 1.2: Representação do fluxo sequencial de processamento: a saída de cada nível torna-se a entrada do nível subsequente.

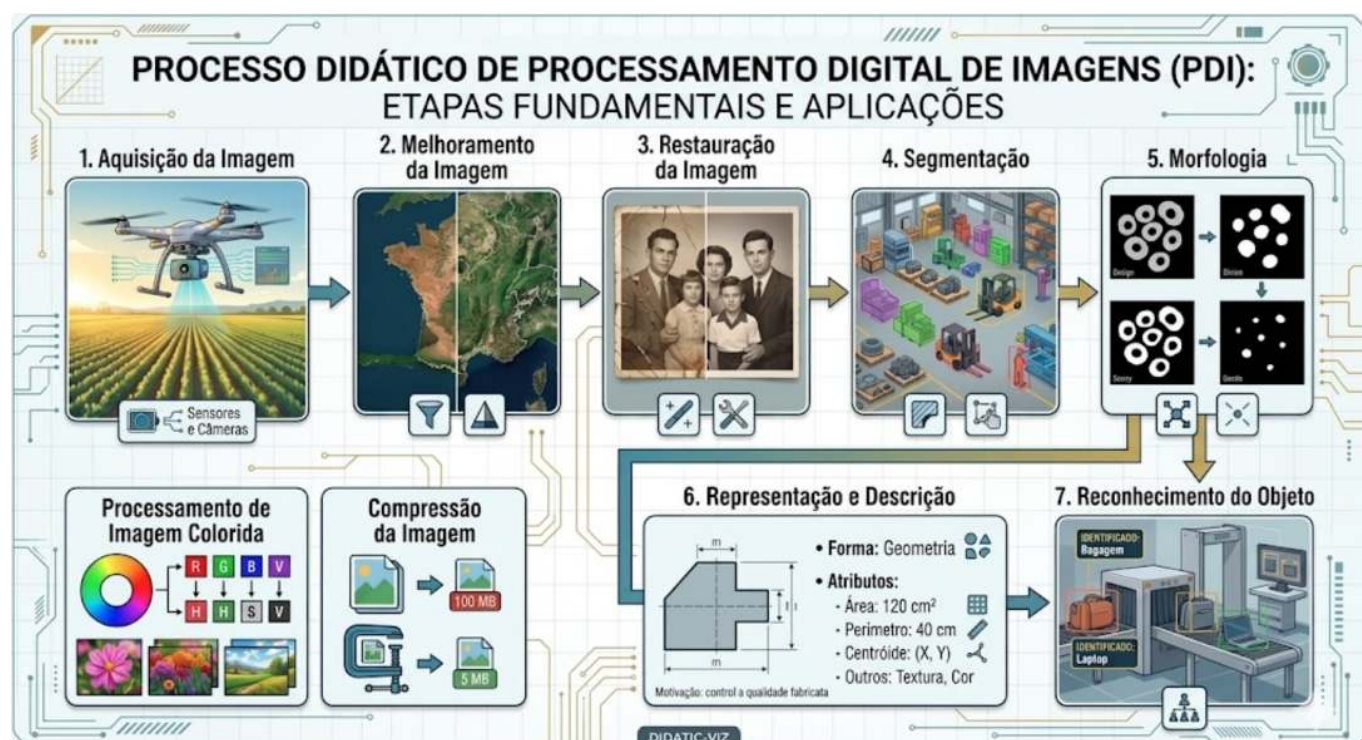


Figura 1.3: Fluxo detalhado do PDI: da aquisição sensorial à extração de atributos e reconhecimento automatizado, incluindo em processamento colorido e compressão.

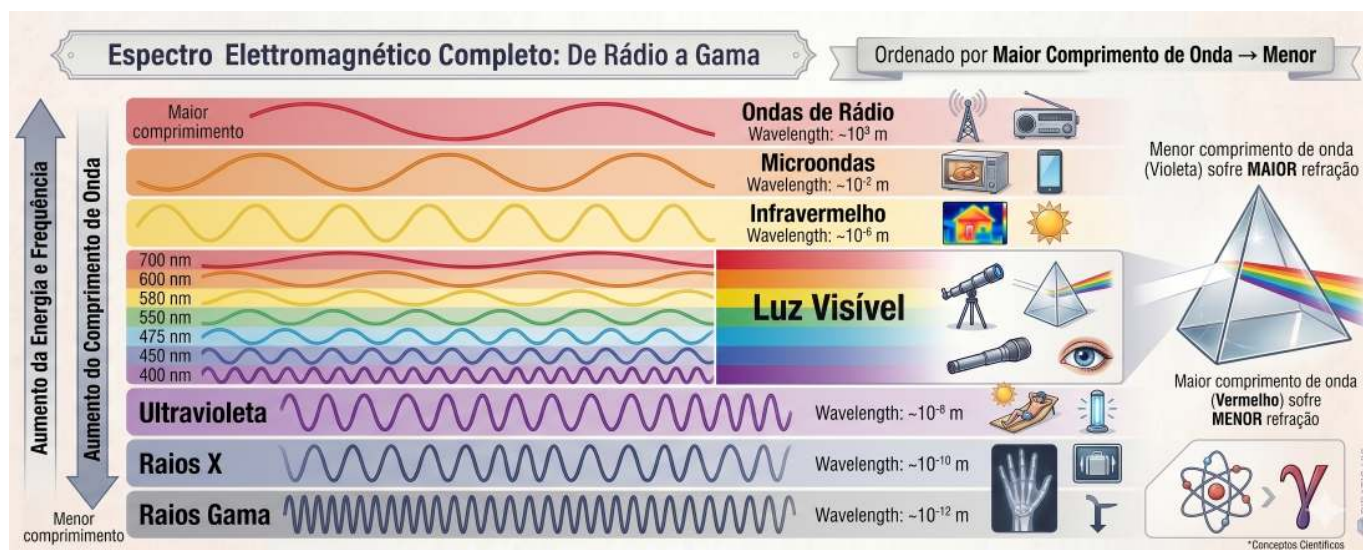


Figura 1.4: Representação do espectro visível e sua posição em relação às demais radiações eletromagnéticas, destacando a variação dos comprimentos de onda de 380 nm a 750 nm.

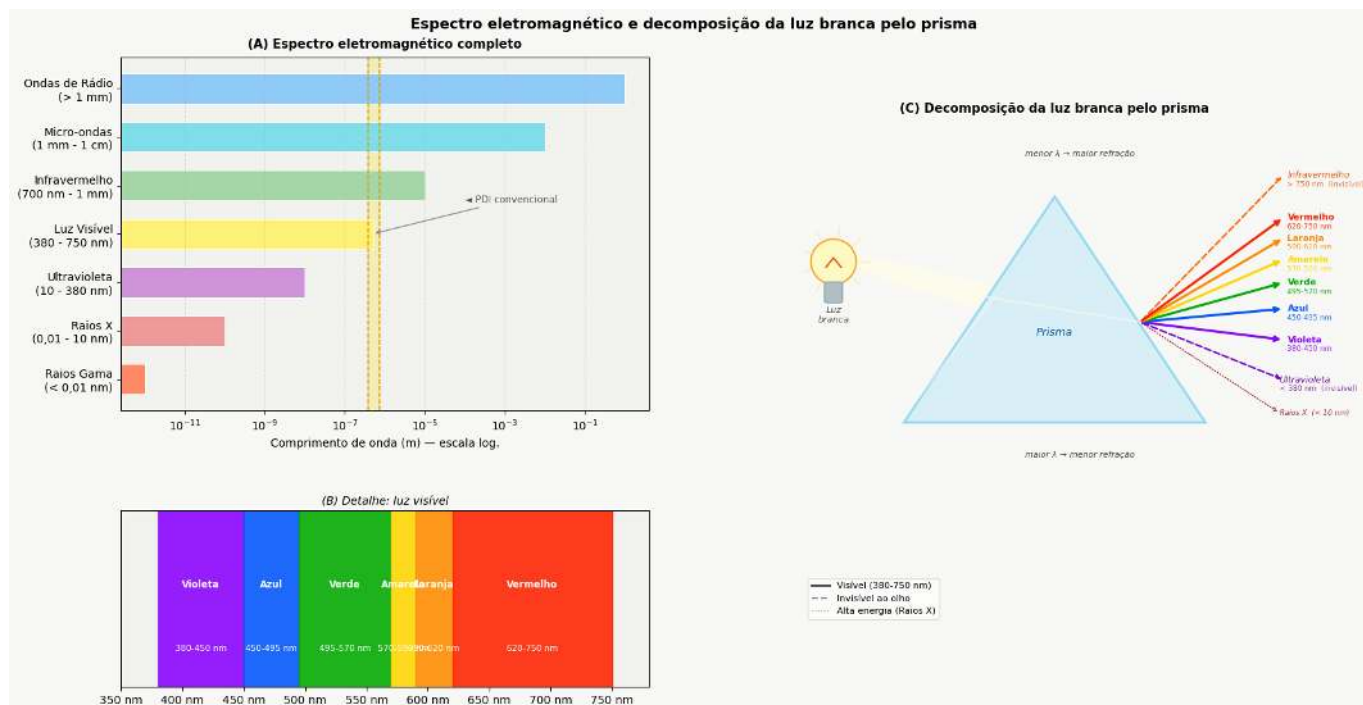


Figura 1.5: (A) Espectro eletromagnético completo em escala logarítmica, com destaque para a faixa visível. (B) Detalhe da luz visível (380-750 nm) e suas cores. (C) Decomposição da luz branca pelo prisma: menor comprimento de onda sofre maior refração, separando UV, visível e infravermelho.

A partir desse processo físico de aquisição, torna-se possível modelar matematicamente a imagem digital como uma função bidimensional discreta, na qual cada ponto da cena é representado por amostras numéricas de intensidade luminosa, formalizando assim os conceitos de pixel e de imagem digital apresentados na próxima seção.

1.6 O que é uma Imagem Digital?

Uma **imagem digital** é formada por uma grade de **pixels** (*Picture Elements*), onde cada pixel é a menor unidade elementar da imagem.

 O que é um Pixel?

Um **pixel** é a menor unidade endereçável que compõe uma imagem digital. Cada pixel ocupa uma posição única na grade e armazena um ou mais valores numéricos que representam sua intensidade ou cor.

Representação Matemática

Diferente de uma função contínua, o domínio de uma imagem digital é um plano retangular finito $\mathbb{E} \subset \mathbb{Z}^2$, que representa a grade de amostragem. Este domínio é indexado por coordenadas inteiras:

$$\mathbb{E} = \{(x, y) \in \mathbb{Z}^2 \mid 0 \leq x < L, 0 \leq y < H\} \tag{1.1}$$

Onde:

- L : representa a largura da imagem (número de colunas).
- H : representa a altura da imagem (número de linhas).

A imagem digital é uma função que associa cada par de coordenadas (x, y) a um ou mais valores que descrevem a aparência do pixel.

$$f : \mathbb{E} \rightarrow \mathcal{V} \tag{1.2}$$

O conjunto $\mathcal{V}$ define os valores possíveis para o pixel (codomínio), variando conforme o tipo de imagem, como demonstrado na Table 1.2.

Tabela 1.2: Principais tipos de imagem digital e seus respectivos conjuntos de valores possíveis para cada pixel.

Tipo de imagem	$\mathcal{V}$ (valores do pixel)	Representação
Binária	$\{0, 1\}$ ou $\{0, 255\}$	 
Tons de cinza	$\{0, 1, \dots, 255\}$	[] [=] [#] 
Colorida (RGB)	$\{0, \dots, 255\}^3$ (triplas ordenadas de valores)	  

Exemplo prático: Uma imagem colorida no modelo RGB pode ser representada matematicamente por uma função que associa três valores de intensidade a cada pixel. Computacionalmente, essa representação corresponde a três matrizes sobrepostas — os canais vermelho (*Red*), verde (*Green*) e azul (*Blue*) — nas quais cada elemento armazena a intensidade luminosa do respectivo canal em uma determinada posição da imagem.

Para viabilizar os experimentos práticos de PDI e VC, este livro utiliza o ecossistema científico do Python, integrando bibliotecas voltadas à manipulação matricial, ao processamento de imagens e à visualização de resultados, apresentadas a seguir.

1.7 Configuração do Ambiente

Este material fundamenta-se no ecossistema científico do Python, com destaque para o **NumPy** (matemática matricial), **OpenCV** (VC padrão de mercado) e a biblioteca **morph.py** (Zampirolli et al., 2025), desenvolvida especificamente para fins didáticos neste livro, ver Table 1.3.

Atualmente, o projeto disponibiliza a versão em **Python** e **Português**. A arquitetura do sistema, no entanto, foi projetada para expansão futura, permitindo a adaptação automatizada para outras linguagens de programação (como C++ e Java) e idiomas, por meio de APIs de tradução.

Os **Exercícios de Programação (EPs)** do final de cada capítulo integram o módulo `testsuite.py`, já disponível para práticas em Python, C, C++, Java, JavaScript e R, garantindo a validação imediata das soluções propostas nos *notebooks* de estudo.

Tabela 1.3: Principais bibliotecas utilizadas ao longo deste livro para manipulação matricial, PDI e visualização de resultados.

Biblioteca	Função principal
<code>numpy</code>	Representação matricial de imagens
<code>opencv-python</code>	Leitura, escrita e operações de VC
<code>matplotlib</code>	Visualização de imagens e gráficos
<code>morph.py</code>	Abstração didática das operações de PDI

i Versões do `morph.py`

A biblioteca `morph.py` possui duas versões públicas:

- **Versão 1.0** — versão original, publicada junto ao artigo apresentado no EduComp 2024: <https://github.com/fzampirolli/morph>
- **Versão 1.1** — versão compacta, utilizada nas atividades deste livro: <https://github.com/fzampirolli/pdi-vc/blob/master/morph/morph.py>

A versão 1.1 foi necessária para uso nas atividades do **Moodle/VPL** (Laboratório Virtual de Programação). Na versão 1.0, bibliotecas como `matplotlib`, `requests` e `skimage` eram carregadas no momento do `import morph`, causando erro de memória (Jail: out of memory, 128MiB) no ambiente restrito do VPL.

Na versão 1.1, esses *imports* foram convertidos para **carregamento lazy**: cada biblioteca é importada apenas dentro do método que a utiliza, e somente quando esse método é de fato chamado. Assim, um simples `import morph` carrega apenas `numpy` e `cv2`, que são suficientes para a maioria dos EPs.

```
import os, importlib, urllib.request, numpy as np, matplotlib.pyplot as plt

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

# Verifica se o atributo existe antes de imprimir
version = getattr(morph, "__version__", "local_file")

print(f" Ambiente pronto. Módulo 'morph' carregado com sucesso (Versão: {version}).")
print(f" Funções disponíveis no mm: \n{dir(mm)[-5:]}...")
```

Ambiente pronto. Módulo 'morph' carregado com sucesso (Versão: 1.1.0).
 Funções disponíveis no mm:
 ['verifyBoundingBox', 'watershed', 'watershed0', 'watershedB', 'write']...

1.8 Fundamentos de Matrizes - atenção à cópia de referências

Como uma imagem digital é uma matriz, precisamos saber criá-las corretamente. Em Python, existe uma armadilha comum ao usar o operador `*` em listas:

⚠ Atenção à cópia de referências

Ao fazer `m = [[0]*2]*3`, você não cria 3 linhas independentes, mas sim 3 referências para a **mesma** linha. Alterar um valor em uma linha alterará todas as outras!

Para visualizar esse comportamento, você pode testar o código no [Python Tutor](#) e comparar com a forma correta de criar matriz com listas: `m = [[0]*2 for _ in range(3)]`.

A forma recomendada e padrão em PDI é usar **NumPy**, que cria matrizes com dados independentes e oferece eficiência computacional. O código a seguir mostra diferentes formas de criação de imagens sintéticas — o resultado é exibido na Figure 1.6.

```
# Criando uma imagem preta (zeros) de 4, 6 pixels
img_preta = np.zeros((4, 6), dtype='uint8')
img_preta[0,0] = 255 # pixel branco no canto superior esquerdo

# Criando uma imagem branca (255) de 4, 6 pixels
img_branca = np.ones((4, 6), dtype='uint8') * 255
img_branca[3,5] = 0 # pixel preto no canto inferior direito

# Criando uma imagem aleatória para testes (ruído)
img_random = mm.randomImage(4, 6, maxValue=255)

print("Matriz aleatória gerada:")
print(mm.drawImg(img_random))

mm.show(
    [img_preta, img_branca, img_random],
    titles=[
        "Predominantemente preta\n(com 1 pixel branco em (0,0))",
        "Predominantemente branca\n(com 1 pixel preto em (3,5))",
        "Imagem aleatória\n(simulação de ruído)"
    ],
    cols=3,
    figsize=(9, 3),
    axis=True
)
```

Matriz aleatória gerada:

77	111	74	119	46	136
219	115	171	222	22	29
76	66	110	242	146	133
133	42	182	15	37	103

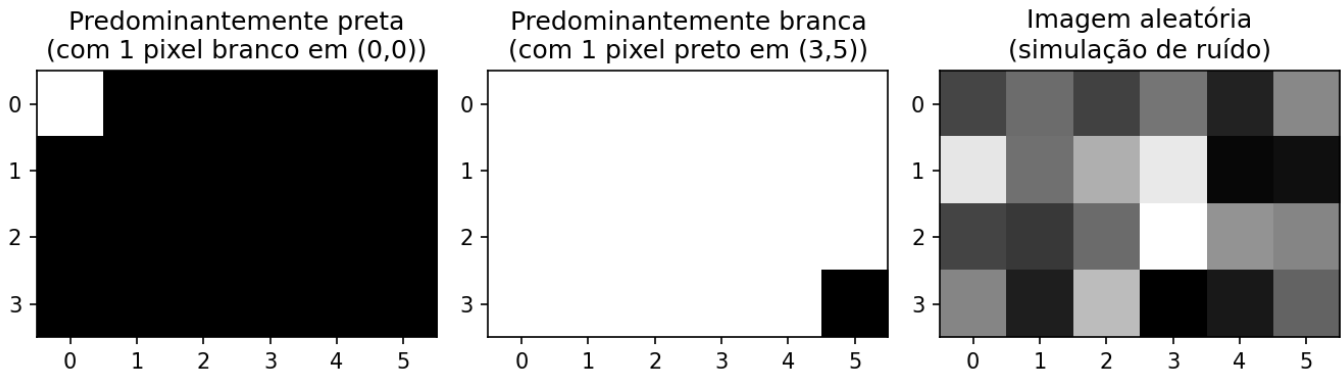


Figura 1.6: Exemplos de imagens sintéticas representadas matricialmente.

1.9 Lendo e Exibindo Imagens

Em bibliotecas como o [NumPy](#), [OpenCV](#) e [scikit-image](#), imagens digitais são frequentemente representadas computacionalmente como *arrays* do NumPy. Dessa forma, operações matriciais e conceitos de álgebra linear podem ser aplicados diretamente ao PDI.

Uma das operações fundamentais em PDI é a leitura de imagens. Na biblioteca `morph.py`, a função `mm.read()` permite carregar imagens tanto de arquivos locais quanto de URLs, como ilustrado na Figure 2.7.

```
import os
import numpy as np

url = "https://upload.wikimedia.org/wikipedia/commons/8/87/Mandrillus_sphinx_339428057.jpg"
caminho = "imagens/mandrill.png"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img = np.array(img_obj)

print(f"Dimensões (H, W, Canais): {img.shape}")
print(f"Tipo de dado: {img.dtype}")

mm.show(img, title="Exemplo: Captura RGB")
```

```
Dimensões (H, W, Canais): (1365, 2048, 3)
Tipo de dado: uint8
```


Exemplo: Captura RGB



Figura 1.7: Mandrill (*Mandrillus sphinx*) em ambiente natural. Crédito: Julien Renoult (CC BY 4.0).

Alternativa: *Download* da Imagem para Armazenamento Local

Em ambientes nos quais a leitura direta de URLs esteja indisponível — devido a restrições de rede, políticas de *firewall* ou ausência de conectividade — uma alternativa consiste em realizar previamente o *download* da imagem para o sistema de arquivos local. Após o armazenamento, a imagem pode ser carregada normalmente pela função `mm.read()`. No exemplo a seguir, o arquivo é salvo localmente com o nome `mandrill.png`.

```
!wget -O mandrill.png \
    https://upload.wikimedia.org/wikipedia/commons/8/87/Mandrillus_sphinx_339428057.jpg
```

```
img = mm.read('mandrill.png')
mm.show(img, title="Exemplo: Captura RGB (arquivo local)")
```

Explicação

- `wget -O mandrill.png <URL>` realiza o *download* da imagem e a armazena localmente com o nome especificado;
- `mm.read('mandrill.png')` efetua a leitura do arquivo diretamente do sistema de arquivos, sem necessidade de requisições HTTP adicionais;
- essa estratégia reduz a dependência de conectividade durante a execução dos experimentos e evita *downloads* repetidos da mesma imagem.

Nota

O prefixo `!` é utilizado em ambientes baseados em *notebooks*, como [Jupyter Notebook](#), [JupyterLab](#) e [Google Colab](#), para executar comandos do sistema operacional diretamente em células de código. Em terminais convencionais, o comando deve ser utilizado sem o prefixo `!`.

Caso o utilitário `wget` não esteja instalado, pode-se utilizar alternativamente:

```
!curl -o mandrill.png \
    https://upload.wikimedia.org/wikipedia/commons/8/87/Mandrillus_sphinx_339428057.jpg
```

1.10 Conversão de Tipos e Limiarização

Conforme vimos, uma imagem colorida no espaço RGB é representada pela função:

$$f : \mathbb{E} \rightarrow \{0, 1, \dots, 255\}^3$$

Ou seja, para cada pixel (x, y) , temos três valores (R, G, B) que definem sua cor.

Conversão para Tons de Cinza

Para converter uma imagem RGB em tons de cinza (*grayscale*), é necessário combinar os três canais em um único valor de intensidade g , que representa o brilho percebido. Como o olho humano não é igualmente sensível ao vermelho, verde e azul, utiliza-se uma **média ponderada**. O padrão [ITU-R BT.601](#) (ITU-R, 2011) define os seguintes pesos:

$$g = 0.299 R + 0.587 G + 0.114 B \quad (1.3)$$

Após o cálculo, o valor g é arredondado para o inteiro mais próximo e ajustado ao intervalo $[0, 255]$. O resultado é uma nova imagem, agora em tons de cinza, representada por:

$$f_{\text{cinza}} : \mathbb{E} \rightarrow \{0, 1, \dots, 255\}$$

Limiarização (*Thresholding*)

A partir da imagem em tons de cinza $f_{\text{cinza}}(x, y)$, uma operação fundamental é a **limiarização**, que produz uma imagem **binária** (apenas preto e branco). Para isso, escolhe-se um valor de corte T (geralmente no intervalo $[0, 255]$) e define-se:

$$f_{\text{bin}}(x, y) = \begin{cases} 255 & \text{se } f_{\text{cinza}}(x, y) > T \\ 0 & \text{caso contrário} \end{cases} \quad (1.4)$$

Exemplo: Com $T = 128$, pixels com intensidade acima de 128 tornam-se brancos (255); os demais tornam-se pretos (0).

A limiarização é amplamente usada para **segmentar objetos do fundo**, extrair bordas ou criar máscaras binárias para processamento posterior.

Nota: O valor 255 representa o branco máximo em imagens de 8 bits, enquanto 0 representa o preto absoluto.

Exemplo prático de conversão e limiarização

A Figure 1.8 ilustra os principais passos para transformar uma imagem colorida em tons de cinza e, em seguida, convertê-la em uma imagem binária por limiarização. O código a seguir implementa essas etapas:

```
# 1. Converter para Tons de Cinza
img_gray = mm.gray(img)

# 2. Aplicar limiar (Pixels > 128 tornam-se 255, outros 0)
limiar = 128
img_binaria = mm.threshold(img_gray, limiar)

# Uso da nova função
mm.show(
    [img, img_gray, img_binaria],
```

```
titles=["Original", "Tons de Cinza", f"Binária (T={limiar})"],
cols=3
)
```



Figura 1.8: Processamento básico de imagens: (a) imagem original, (b) imagem em tons de cinza, (c) imagem binarizada por limiar ($T=128$).

1.11 Limiarização pelo método de Otsu

Conforme apresentado na Equation 1.4, a limiarização converte uma imagem em tons de cinza para binária usando um valor de corte T . Até agora fixamos $T = 128$ manualmente.

```
img_bin_fixo = mm.threshold(img_gray, T=128)
```

Entretanto, a escolha manual de T nem sempre é trivial. A biblioteca `mm` oferece uma alternativa automática: quando o parâmetro **limiar não é fornecido**, a função `mm.threshold(img_gray)` calcula o valor de T pelo **método de Otsu** (Otsu, 1979). Esse método, que será detalhado em capítulos futuros, maximiza a variância entre classes do histograma (frequência de cada tom de cinza), separando automaticamente os pixels de objeto e fundo.

O código abaixo compara a limiarização manual ($T = 128$) com a automática (Otsu), mostrando também o valor de T calculado:

```
import cv2
# Limiarização com T fixo (manual)
T_fixo = 128
img_bin_fixo = mm.threshold(img_gray, T_fixo)

# Limiarização pelo método de Otsu (T automático)
T_otso, img_bin_otso = cv2.threshold(img_gray, 0, 255,
                                     cv2.THRESH_BINARY + cv2.THRESH_OTSU)

print(f'Limiar calculado por Otsu: T = {T_otso}')
# ou simplesmente:
# img_bin_otso = mm.threshold(img_gray)

# Exibição lado a lado
mm.show(
    [img_gray, img_bin_fixo, img_bin_otso],
    titles=["Tons de Cinza", f"Binária (T={T_fixo})", f"Binária (Otsu, T={T_otso})"],
    cols=3
)
```

```
Limiar calculado por Otsu: T = 95.0
```



Figura 1.9: Comparação entre limiarização manual ($T=128$) e automática (Otsu) sobre a imagem em tons de cinza.

A Figure 1.9 mostra que o limiar obtido por Otsu se adapta automaticamente à imagem, resultando em uma binarização mais eficiente do que um valor fixo, especialmente quando as intensidades do objeto e do fundo são bem separadas no histograma. Essa técnica é amplamente utilizada em sistemas de VC para binarização de documentos, detecção de objetos e pré-processamento de imagens.

💡 Simplicidade da biblioteca `morph.py`

Enquanto o OpenCV exige a chamada completa:

```
T_otsu, img_bin = cv2.threshold(img_gray, 0, 255,
                                cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

a biblioteca `mm` abstrai toda essa complexidade: basta chamar `mm.threshold(img_gray)`. O limiar de Otsu é calculado automaticamente e a imagem binária é retornada diretamente. Essa abordagem permite concentrar-se no conceito, não nos detalhes de implementação.

1.12 Acesso a Pixels

Em Python com NumPy, uma imagem é estruturada como um **array multidimensional**. O acesso a um pixel específico utiliza a convenção matricial **linha** (eixo Y) e **coluna** (eixo X): `img[linha, coluna]`.

O código da Figure 1.10 demonstra como extrair esses valores em imagens coloridas (RGB) e em tons de cinza, além de isolar a vizinhança imediata do ponto de interesse.

```
import matplotlib.pyplot as plt

# 1. Definição das coordenadas do pixel alvo
r, c = 600, 800

# 2. Acesso direto aos valores dos pixels
pixel_cinza = img_gray[r, c]
pixel_rgb = img[r, c]

print(f" VALORES NO PIXEL ALVO ({r}, {c}):")
print(f"   • Tons de cinza (Escalar) : {pixel_cinza}")
print(f"   • Colorido (Vetor RGB)      : R={pixel_rgb[0]}, G={pixel_rgb[1]}, B={pixel_rgb[2]}")
print(f"   -" * 50)

# 3. Exibição didática da vizinhança 3x3 ao redor do pixel (r, c)
# Recorta da linha r-1 até r+1, e da coluna c-1 até c+1
vizinhanca_gray = img_gray[r-1:r+2, c-1:c+2]

print(f" MATRIZ DE VIZINHANÇA 3x3 EM TONS DE CINZA:")
```



```

print(f"    (0 pixel alvo central está destacado com entre colchetes)\n")

# Impressão formatada para destacar o pixel central
for i, linha in enumerate(vizinhanca_gray):
    linhas_str = []
    for j, valor in enumerate(linha):
        if i == 1 and j == 1:
            linhas_str.append(f"[{valor:3d}]") # Destaca o centro (100, 100)
        else:
            linhas_str.append(f" {valor:3d} ")
    print("    " + " ".join(linhas_str))

# 4. Demonstração visual usando o mm.show (opcional, mas excelente para o livro)
# Mostra a imagem inteira com um ponto demarcando a região
plt.figure(figsize=(5, 5))
plt.imshow(img)
plt.plot(c, r, 'ro', markersize=8, label=f'Pixel ({r},{c})') # Plota (x, y) -> (c, r)
plt.title(f"Localização do Pixel ({r}, {c}) na Imagem")
plt.legend()
plt.axis('on') # Mantém os eixos para o aluno ver as coordenadas 100, 100
plt.show()

```

VALORES NO PIXEL ALVO (600, 800):

- Tons de cinza (Escalar) : 81
- Colorido (Vetor RGB) : R=92, G=78, B=67

MATRIZ DE VIZINHANÇA 3x3 EM TONS DE CINZA:

(0 pixel alvo central está destacado com entre colchetes)

```

83   96  105
85  [ 81]  96
83   77   84

```

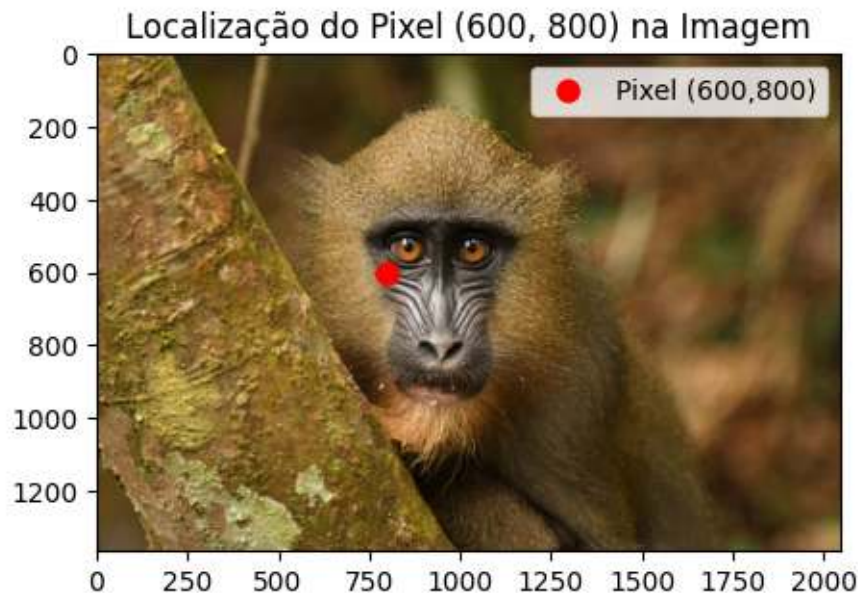


Figura 1.10: Acesso a um pixel específico (r, c) e a representação de sua vizinhança na imagem.

Explicação linha a linha:

1. `img_gray[r, c]` — Retorna um único valor inteiro (escalar) entre 0 e 255, que representa a intensidade de brilho do cinza.

2. `img[r, c]` — Retorna um vetor com três valores `[R, G, B]`, correspondentes às intensidades dos canais Vermelho, Verde e Azul.
3. `img_gray[r-1:r+2, c-1:c+2]` — Realiza um fatiamento (*slicing*) para extrair a submatriz 3×3 ao redor do pixel. Esta operação é a base para a implementação de filtros espaciais e convoluções.
4. O laço `for` subsequente apenas formata essa submatriz no console, exibindo o pixel central entre colchetes `[ ]` para fins didáticos.
5. A função `plt.plot(c, r, 'ro')` plota um ponto vermelho sobre a imagem para correlacionar a coordenada numérica com sua posição visual. Note que o Matplotlib utiliza a ordem padrão de tela `(X, Y)`, invertendo para `(c, r)`.

Como a indexação em Python começa em zero (*zero-based*), o canto superior esquerdo da imagem é a coordenada `(0,0)`. Guarde sempre esta regra: a primeira dimensão da matriz controla a **altura** (linhas/Y) e a segunda controla a **largura** (colunas/X).

1.13 Resumo

Neste capítulo, apresentaram-se os fundamentos de representação de imagens digitais: a definição de pixel, a estruturação de imagens em matrizes e o impacto da amostragem e da quantização na qualidade final:

- **Imagem digital** = função $f(x, y)$ que mapeia coordenadas para intensidades (escalares ou vetoriais).
- **Domínio**: conjunto finito $\mathbb{E} = \{(x, y) \in \mathbb{Z}^2 \mid 0 \leq x < L, 0 \leq y < H\}$.
- **Tipos principais**: binária ($\mathcal{V} = \{0, 255\}$), tons de cinza ($\mathcal{V} = [0, 255]$) e RGB ($\mathcal{V} = [0, 255]^3$).
- A biblioteca `morph.py` (ou `mm`) oferece funções didáticas para operações básicas de PDI, como `mm.gray()`, `mm.threshold()`, `mm.show_multiple()`.
- **Armadilha NumPy**: nunca use `[[0]*n]*m` para criar matrizes — use sempre `np.zeros()` ou `np.ones()`.
- **Limiarização** converte tons de cinza em binária; o **método de Otsu** determina o valor de corte automaticamente maximizando a variância entre classes.
- Acesso a pixels via `img[linha, coluna]`, com indexação *zero-based*.

O Capítulo 2 abordará **histogramas e equalização de contraste**.

1.14 Uso do NotebookLM como Tutor Complementar

Nesta edição, além dos *notebooks* interativos no Google Colab, o **NotebookLM** está disponível como ferramenta complementar de estudo. A plataforma utiliza exclusivamente os documentos fornecidos pelo autor como base de conhecimento, garantindo respostas coerentes com o conteúdo do livro.

Estude com o Tutor Inteligente

Acesse o ambiente do capítulo pelo *link* abaixo e explore especialmente as opções **Guia de Estudo** e **Conversa** para aprofundar sua compreensão.

 [ACESSAR NOTEBOOKLM: CAPÍTULO 01](#)

Aviso sobre Conteúdo Gerado por IA

A IA é uma poderosa aliada nos estudos, mas o conteúdo gerado pode conter **erros ou imprecisões**. Consulte sempre **livros, artigos científicos e outras fontes acadêmicas confiáveis** para validar as informações. Sempre que possível, execute os exemplos práticos fornecidos neste capítulo para verificar os resultados.

Funcionalidades Disponíveis na Plataforma

O **NotebookLM** oferece uma suíte avançada de ferramentas baseadas em IA para transformar o conteúdo estático do livro em uma experiência de aprendizado dinâmica e multimídia. Conforme ilustrado na

Figure 1.11, a plataforma utiliza técnicas de **RAG** (*Retrieval-Augmented Generation*), fundamentadas no trabalho de Lewis et al. (2020), para basear as respostas estritamente nos documentos fornecidos e minimizar a ocorrência de alucinações.

As principais funcionalidades incluem:

- **Resumos Multimodais (Áudio e Vídeo):** Geração de conversas naturais entre especialistas no formato de **Resumo em Áudio** (estilo *podcast*) e **Resumo em Vídeo**, discutindo os temas centrais do capítulo, como as diferenças entre PDI e VC, ou a interpretação de transformações como a limiarização e o método de Otsu.
- **Visualização de Estruturas (Mapa Mental e Infográfico):** Criação automática de diagramas que conectam visualmente os conceitos, por exemplo, o fluxo de processamento desde a captura da imagem digital, passando pela conversão para tons de cinza, limiarização e segmentação binária.
- **Ferramentas de Avaliação (Teste e Cartões Didáticos):** Geração de **Testes** de múltipla escolha e **Cartões Didáticos** (*flashcards*) para fixação de conhecimento, baseados no texto autoral (ex.: perguntas sobre a fórmula de conversão RGB→cinza ou sobre o funcionamento do limiar global e de Otsu).
- **Apoio à Apresentação (Slides e Relatórios):** Auxílio na estruturação de **Apresentações de Slides** e na redação de **Relatórios** técnicos, facilitando a comunicação de resultados de experimentos com imagens.
- **Análise de Dados (Tabela de Dados):** Organização de dados extraídos do texto em tabelas estruturadas, auxiliando na compreensão de exemplos práticos, como a comparação entre diferentes valores de limiar.
- **Chat Contextualizado:** Permite o questionamento direto sobre o código e a teoria, como: “*Como implementar a conversão de RGB para tons de cinza usando os pesos do padrão ITU-R BT.601?*” ou “*O que acontece com a imagem binária se eu escolher um limiar $T=200$ em vez de $T=128$?*”.



Figura 1.11: Estúdio do NotebookLM.

1.15 Lista de Exercícios

1. (15%) Com suas próprias palavras, defina **imagem digital** e **pixel**. Dê um exemplo concreto de como uma imagem colorida (RGB) é representada matricialmente no computador.
2. (15%) Explique as diferenças entre **imagem binária**, **tons de cinza (8 bits)** e **colorida RGB**, indicando a faixa de valores possíveis para cada pixel em cada tipo.
3. (20%) Considerando a fórmula de conversão RGB → tons de cinza do padrão ITU-R BT.601:

$$g = 0.299 R + 0.587 G + 0.114 B$$

Calcule o valor do pixel em tons de cinza para $(R, G, B) = (80, 180, 30)$. Arredonde para o inteiro mais próximo.

4. (20%) O que é **limiarização** (*thresholding*)? Explique a diferença entre escolher um limiar T fixo (ex.: $T = 128$) e utilizar o **método de Otsu** para determinação automática do limiar. Em poucas palavras, como o método de Otsu escolhe o limiar?
5. (15%) No contexto da biblioteca didática `mm` discutida no capítulo, responda:
 - a) (7,5%) Como se acessa o valor do pixel na posição (linha=50, coluna=60) de uma imagem em tons de cinza `img_gray`?
 - b) (7,5%) Qual é a vantagem de usar `mm.threshold(img_gray)` sem passar o limiar? Compare com a chamada equivalente no OpenCV.
6. (15%) O que a propriedade `img.shape` retorna para uma imagem NumPy no formato RGB? Dê um exemplo concreto com uma imagem de 640×480 pixels.

Referências do Capítulo

A fundamentação teórica deste capítulo compreende as seguintes obras de PDI e VC:

- Gonzalez; Woods (2018) para os fundamentos de **Processamento Digital de Imagens** (PDI).
- Singh (2019) para a implementação prática de métodos de **processamento e análise de imagens**.
- Szeliski (2022) para o estudo de VC e algoritmos fundamentais.
- Bradski; Kaehler (2008) para a aplicação da biblioteca **OpenCV** em ambiente Python.
- Lewis et al. (2020) para o conceito de **geração aumentada por recuperação (RAG)**, utilizado no suporte ao processamento de informações deste material.

 Executar  Colab  Abrir si-md2  GitHub

1.16 Parte Prática com Exercícios de Programação

Objetivo deste Caderno

Os **Exercícios de Programação (EPs)** apresentados a seguir podem ser submetidos também em atividades do Moodle (atividades VPL) que fornecem *feedback* automático.

Este caderno foi desenvolvido para superar limitações de uso do Moodle. Com ele, deve-se:

1. **Desenvolver:** Escrever e editar sua solução diretamente no ambiente Colab.
2. **Validar:** Testar seu código localmente utilizando os **mesmos casos de teste** do Moodle.
3. **Organizar:** Salvar seus códigos das atividades VPL de forma segura.
4. **Avaliar:** Quando estiver conectado ao Moodle, basta copiar sua solução e clicar em **Avaliar** no Moodle (se estiver na rede da UFABC) para registrar sua nota oficial.

§ Instruções Passo a Passo

Em um ambiente de execução (como VSCode, Jupyter ou Colab), siga a ordem abaixo para configurar o ambiente e validar seus exercícios:

Preparação do Ambiente

Execute a célula de código abaixo para baixar `morph.py` e `testsuite.py` do repositório da disciplina — apenas se ainda não existirem no diretório local. Com ambos em `./`, o *notebook* e os subprocessos do `TestSuite` encontram o módulo sem configurações extras de caminho.

Nota: O script `testsuite.py` buscará automaticamente os casos de teste em `all/{cap}/cases` no [GitHub](#).

Escrevendo o Código

Salve sua solução em uma célula de código usando o comando mágico `%%writefile`. O nome do arquivo deve seguir o padrão `EPX_Y.*`, onde `X` é o capítulo, `Y` é o exercício e `*` é a extensão da linguagem.

Exemplo: `%%writefile EP01_01.py`

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.1 | TestSuite: 1.1.0

Executando os Testes

Após salvar o arquivo com sua solução, execute o comando abaixo (em uma nova célula) para avaliar os testes automáticos:

```
TestSuite("EP01_01.extensão").run()
```

Substitua a extensão conforme a linguagem usada:

Linguagem	Extensão
Python	.py
Java	.java
C	.c
C++	.cpp
JavaScript	.js
R	.r

Como funciona: O `TestSuite` baixa os casos de teste do GitHub, executa seu programa com cada entrada e compara a saída com a esperada – calculando automaticamente sua nota.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:

```
codigo = """
from morph import mm
# ... seu código aqui ...
"""

TestSuite("EP04_01").run_code(codigo)
```

⚠ Importante: Regras e Boas Práticas

♦ Sobre a Entrada de Dados

O seu programa deve ler a entrada padrão (teclado).

- **Python:** Utilize `input()`.
- **Outras linguagens:** Utilize o comando de leitura padrão equivalente (`cin`, `Scanner`, etc.).

♦ Configuração de IA no Colab

Para um melhor aprendizado, recomenda-se desativar o preenchimento automático de código por IA, pois não estará disponível durante as avaliações. Por exemplo no navegador Chrome:

- Vá em: **Ferramentas > Configurações > IA generativa**
- Desmarque: **Habilitar geração de código**

♦ Integridade Acadêmica (Plágio)

Este recurso de testes locais aplica-se a EPs **sem variações**. No entanto:

- **Individualidade:** Cada aluno deve desenvolver sua própria solução.
- **Deteção de Similaridade:** O professor utiliza ferramentas que detectam cópias, **mesmo com alteração de nomes de variáveis ou espaços em branco**.

1.16.1 EP01_01 📏 Três métricas de distância em PDI

Nesta atividade, você deve escrever um programa que calcule as três distâncias clássicas em PDI: **Euclidiana (L2)**, **City-Block (L1)** e **Chessboard (L ∞)**.

- Leia **4 números reais** que representam as coordenadas: A_x, A_y, B_x, B_y .
- Calcule as três distâncias utilizando as fórmulas:

$$d_{\text{Euclidiana}} = \sqrt{(B_x - A_x)^2 + (B_y - A_y)^2}$$

$$d_{\text{City-block}} = |B_x - A_x| + |B_y - A_y|$$

$$d_{\text{Chessboard}} = \max(|B_x - A_x|, |B_y - A_y|)$$

- Imprima os três resultados, cada um em uma linha, formatados com **duas casas decimais**, na ordem: **Euclidiana**, **City-block**, **Chessboard**.

📌 Importante:

- Utilize as funções matemáticas padrão da sua linguagem: `math.sqrt`, `abs` (ou `fabs`) e `max`.
- A saída deve conter **apenas os números** (um por linha), sem textos adicionais.
- Ver um simulador interativo para esta questão na **Figure 1.12** (gráfico com arrasto dos pontos e visualização das três métricas).

1.16.1.1 🖼 Por que isso importa? – Custo computacional

Em uma imagem **1000×1000 pixels** (1 milhão de pixels), calcular a distância de cada pixel a um ponto de referência exige **1 milhão de operações**. A escolha da métrica afeta o desempenho:

Métrica	Operações por pixel	Custo relativo (1M pixels)	Quando usar
Euclidiana (L2)	2 subtrações, 2 multiplicações, 1 soma, 1 sqrt	 Mais custosa – sqrt é cara	Distância “real” no espaço contínuo
City-block (L1)	2 subtrações, 2 abs , 1 soma	 Moderada – sem raiz quadrada	Grids, robótica, imagens binárias
Chessboard (L∞)	2 subtrações, 2 abs , 1 max	 Mais eficiente	Movimentos de peças, morfologia

A função **sqrt** é computacionalmente mais cara que operações como adição, subtração, multiplicação e valor absoluto. Em CPUs modernas, a diferença pode ser pequena (cerca de $1,5\times$ a $3\times$), mas em sistemas embarcados ou em laços de milhões de iterações, qualquer ganho importa. Por isso, **quando o objetivo é apenas comparar distâncias** (ex.: encontrar o ponto mais próximo), use a **distância euclidiana ao quadrado**.

1.16.1.2 Tarefa (especificação para VPL)

Entrada:

Uma única linha com quatro números reais: A_x A_y B_x B_y

Saída:

Três linhas, cada uma com um número real de duas casas decimais (Euclidiana, City-block, Chessboard).

1.16.1.3 Exemplos

Entrada	Saída	Observação
0	5.00	Triângulo 3-4-5
0	7.00	
3	4.00	
4		
0	1.41	Diagonal unitária
0	2.00	
1	1.00	
1		

Exemplo de teste de **sqrt** em Python, com **timeit** isolando cada operação:

```
import math
import timeit

N = 50_000_000

def apenas_soma():
    a, b = 3.0, 4.0
    return a + b

def soma_e_sqrt():
    a, b = 3.0, 4.0
    return math.sqrt(a*a + b*b)
```

```

t_soma = timeit.timeit(apenas_soma, number=N)
t_sqrt = timeit.timeit(soma_e_sqrt, number=N)

print(f"Soma simples      : {t_soma:.3f} s")
print(f"Soma + sqrt       : {t_sqrt:.3f} s")
print(f"Razão (sqrt/soma)  : {t_sqrt/t_soma:.2f}x")

```

```

Soma simples      : 2.967 s
Soma + sqrt       : 5.276 s
Razão (sqrt/soma) : 1.78x

```

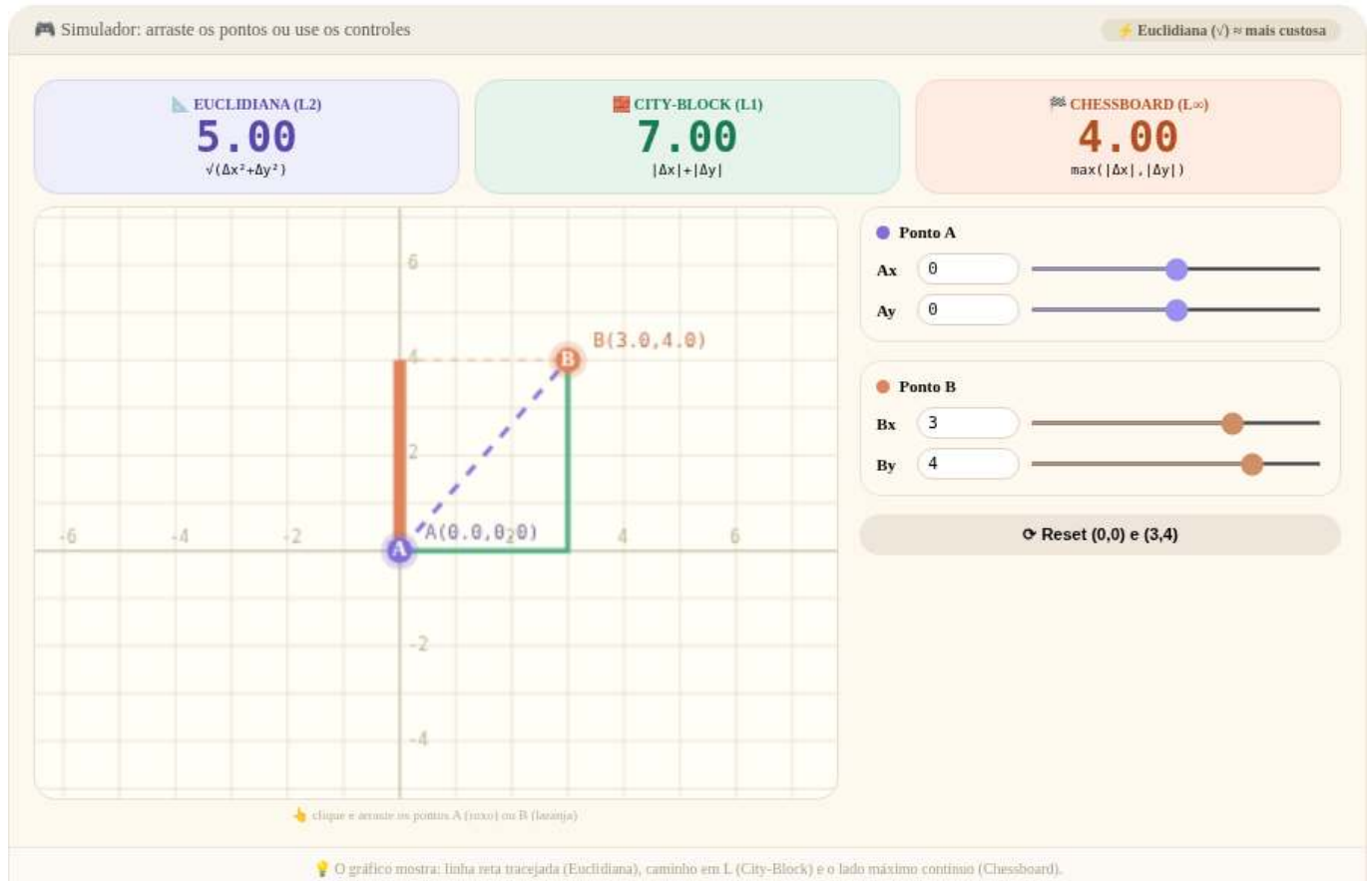


Figura 1.12: Simulador: Distâncias Euclidiana, *City-block* e *Chessboard*

1.16.1.4 🐍 Python

Basta criar uma célula de código normal e inserir o código Python. A entrada pode ser simulada usando `input()`, que funciona normalmente.

Exemplo de célula:

```

%%writefile EP01_01.py
# Código Python
x1,y1,x2,y2 = int(input()), int(input()), int(input()), int(input())
# Cálculo das diferenças
dx = abs(x2 - x1)
dy = abs(y2 - y1)

# 1. Distância Euclidiana (L2)
dist_euclidiana = (dx**2 + dy**2)**0.5

```

```
# 2. Distância City-block / Manhattan (L1)
dist_city_block = dx + dy

# 3. Distância Chessboard / Chebyshev (Linf)
dist_chessboard = max(dx, dy)

# Saída formatada conforme os casos de teste
print(f"{dist_euclidiana:.2f}")
print(f"{dist_city_block:.2f}")
print(f"{dist_chessboard:.2f}")
```

Overwriting EP01_01.py

```
# Espera que você digite 4 números inteiros ao executar esta célula.
# No Jupyter ou Google Colab, a mágica %run -i permite que o script leia do teclado.
# No terminal comum, você usaria: python3 EP01_01.py (sem o '!' e sem '%run').

# %run -i EP01_01.py
```

```
# Envia 4 inteiros como entrada padrão (stdin) para o script EP01_01.py usando um pipe
!echo -e "0\n0\n4\n4" | python3 EP01_01.py
```

```
5.66
8.00
4.00
```

```
TestSuite("EP01_01.py").run()
```

1.16.1.5 Java

Para executar Java no Colab, você precisa usar uma célula com o prefixo `%%writefile` para salvar o código em arquivo, compilar e executar.

```
%%writefile EP01_01.java
import java.util.Scanner;

class EP01_01 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        double x1 = s.nextDouble(), y1 = s.nextDouble();
        double x2 = s.nextDouble(), y2 = s.nextDouble();

        double dx = Math.abs(x2 - x1);
        double dy = Math.abs(y2 - y1);

        System.out.printf("%.2f\n", Math.sqrt(dx*dx + dy*dy));
        System.out.printf("%.2f\n", dx + dy);
        System.out.printf("%.2f\n", Math.max(dx, dy));
    }
}
```

Overwriting EP01_01.java

```
!javac EP01_01.java
!echo -e "0\n0\n4\n4" | java EP01_01
```

```
5,66
8,00
4,00
```

```
TestSuite("EP01_01.java").run()
```

1.16.1.6 C

De forma similar, use `%%writefile` para salvar o código, depois compile e execute. Para C, utilizamos o compilador **GCC**.

```
# Instalar o compilador GCC e ferramentas de build
# O build-essential inclui gcc, g++, make, etc.
import platform, shutil, subprocess

def instalar_gcc():
    if shutil.which("gcc"):
        print(" GCC já disponível."); return
    if platform.system() != "Linux":
        print(" Mac: xcode-select --install | Windows: WSL ou MinGW"); return
    try: import google.colab; cmd = ["apt-get", "install", "-y", "build-essential"]
    except ImportError: cmd = ["sudo", "apt-get", "install", "-y", "build-essential"]
    subprocess.run(cmd, check=True)
    print(" build-essential instalado!")

instalar_gcc()
```

GCC já disponível.

```
%%writefile EP01_01.c
#include <stdio.h>
#include <math.h>

int main() {
    double x1, y1, x2, y2;
    if (scanf("%lf %lf %lf %lf", &x1, &y1, &x2, &y2) != 4) return 0;

    double dx = fabs(x2 - x1);
    double dy = fabs(y2 - y1);

    // Euclidiana, City-block e Chessboard
    printf("%.2f\n", sqrt(dx * dx + dy * dy));
    printf("%.2f\n", dx + dy);
    printf("%.2f\n", fmax(dx, dy));

    return 0;
}
```

Overwriting EP01_01.c

```
# Compila o arquivo .c gerando o executável EP01_01
# -lm é usado para linkar a biblioteca matemática (math.h) se necessário

!gcc EP01_01.c -o EP01_01 -lm
!echo -e "0\n0\n4\n4" | ./EP01_01
```

```
5.66
8.00
4.00
```

```
TestSuite("EP01_01.c").run()
```

1.16.1.7 C++

De forma similar ao Java, use `%%writefile` para salvar o código, depois compile e execute. Lembre-se de que, no Colab, também é necessário instalar o seguinte:

```
# Instalar o compilador G++ para C++
# O build-essential inclui o g++, make, etc.
import platform, shutil, subprocess

def instalar_gpp():
    if shutil.which("g++"):
        print(" G++ já disponível."); return
    if platform.system() != "Linux":
        if platform.system() == "Darwin":
            print(" Mac: xcode-select --install")
        else:
            print(" Windows: use WSL ou MinGW (https://www.mingw-w64.org)")
        return
    try: import google.colab; cmd = ["apt-get", "install", "-y", "build-essential"]
    except ImportError: cmd = ["sudo", "apt-get", "install", "-y", "build-essential"]
    subprocess.run(cmd, check=True)
    print(" Compilador C++ pronto.")

instalar_gpp()
```

G++ já disponível.

```
%%writefile EP01_01.cpp
#include <iostream>
#include <iomanip>
#include <cmath>
#include <algorithm>

int main() {
    double x1, y1, x2, y2;
    if (!(std::cin >> x1 >> y1 >> x2 >> y2)) return 0;

    double dx = std::abs(x2 - x1);
    double dy = std::abs(y2 - y1);

    std::cout << std::fixed << std::setprecision(2);

    // Euclidiana, City-block e Chessboard
    std::cout << std::sqrt(dx*dx + dy*dy) << std::endl;
    std::cout << (dx + dy) << std::endl;
    std::cout << std::max(dx, dy) << std::endl;

    return 0;
}
```

Overwriting EP01_01.cpp


```
!g++ EP01_01.cpp -o EP01_01
!echo -e "0\n0\n4\n4" | ./EP01_01
```

```
5.66
8.00
4.00
```

```
TestSuite("EP01_01.cpp").run()
```

1.16.1.8 JavaScript (Node.js)

Para JavaScript, use `%%writefile` para criar o arquivo e execute com Node:

```
%%writefile EP01_01.js
function escreva(s) { try { document.write(s + "<br>"); } catch(e) { console.log(s); } }

process.stdin.once('data', data => {
  const valores = data.toString().trim().split(/\s+/);
  const x1 = parseFloat(valores[0]);
  const y1 = parseFloat(valores[1]);
  const x2 = parseFloat(valores[2]);
  const y2 = parseFloat(valores[3]);

  const dx = Math.abs(x2 - x1);
  const dy = Math.abs(y2 - y1);

  // Euclidiana, City-block e Chessboard
  escreva(Math.sqrt(dx * dx + dy * dy).toFixed(2));
  escreva((dx + dy).toFixed(2));
  escreva(Math.max(dx, dy).toFixed(2));
});
```

```
Overwriting EP01_01.js
```

```
TestSuite("EP01_01.js").run()
```

1.16.1.9 R

No Colab, pode-se executar código R diretamente utilizando a mágica `%%R`.

O programa deve ler **quatro números** (x1, y1, x2, y2) e exibir a distância euclidiana com **duas casas decimais**.

Exemplo de célula:

```
%%R
dados <- scan(file = "stdin", n = 4, quiet = TRUE)
x1 <- dados[1]; y1 <- dados[2]; x2 <- dados[3]; y2 <- dados[4]
dist <- sqrt((x2 - x1)^2 + (y2 - y1)^2)
cat(sprintf("%.2f", dist))
```

```
# Instalar o R e o Rscript
# O r-base instala o ambiente R completo, incluindo o Rscript
import platform, shutil, subprocess
```

```
def instalar_r():
    if shutil.which("R"):
        print(" R já disponível."); return
    if platform.system() != "Linux":
        if platform.system() == "Darwin":
            print(" Mac: https://cran.r-project.org/bin/macosx/")
        else:
            print(" Windows: https://cran.r-project.org/bin/windows/base/")
        return
    try: import google.colab; cmd = ["apt-get", "install", "-y", "r-base"]
    except ImportError: cmd = ["sudo", "apt-get", "install", "-y", "r-base"]
    subprocess.run(cmd, check=True)
    print(" Ambiente R pronto.")

instalar_r()
```

R já disponível.

Para testar da mesma forma que nos exemplos anteriores, deve-se usar a entrada padrão (stdin) no terminal ou adaptar o código conforme abaixo:

```
%%writefile EP01_01.r
dados <- scan(file = "stdin", n = 4, quiet = TRUE)
dx <- abs(dados[3] - dados[1])
dy <- abs(dados[4] - dados[2])

# Saídas: Euclidiana, City-block e Chessboard
cat(sprintf("%.2f\n%.2f\n%.2f\n",
    sqrt(dx^2 + dy^2),
    dx + dy,
    max(dx, dy)))
```

Overwriting EP01_01.r

```
!echo -e "0\n0\n4\n4" | Rscript EP01_01.r
```

5.66
8.00
4.00

```
TestSuite("EP01_01.r").run()
```

1.16.2 EP01_02 Desempenho Preditivo — Métricas de ML em VC

Nesta atividade, você entrará no mundo do **Aprendizado de Máquina** (*Machine Learning*). Seu objetivo é avaliar o desempenho de um classificador binário calculando métricas a partir de uma **Matriz de Confusão**.

1.16.2.1 Por que a métrica certa importa?

Imagine um detector de **moedas de R\$ 1,00**. O impacto do erro define a métrica prioritária:

Métrica	Exemplo Prático	Importância em PDI/VC
Acurácia	Contagem de Grãos	Útil quando as classes são equilibradas (ex: metade dos grãos com defeito, metade saudáveis).

Métrica	Exemplo Prático	Importância em PDI/VC
Precisão	Segurança/Biometria	Crucial para evitar Falsos Positivos (ex: não permitir que um impostor acesse um sistema por erro de reconhecimento).
Sensibilidade	Saúde (Tumores)	Crucial para evitar Falsos Negativos (ex: não deixar um tumor passar despercebido em um exame de Raio-X).
F1-score	Cédulas de Dinheiro	Ideal para um equilíbrio entre não rejeitar notas verdadeiras e não aceitar notas falsas.

1.16.2.2 A Matriz de Confusão

	Predita Positiva	Predita Negativa
Real Positiva	VP (Verdadeiro Positivo)	FN (Falso Negativo)
Real Negativa	FP (Falso Positivo)	VN (Verdadeiro Negativo)

Tarefa:

1. Leia **4 valores inteiros** na ordem: VP, FN, FP, VN.
2. Calcule as métricas utilizando as fórmulas:

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN}$$

$$\text{Precisão} = \frac{VP}{VP + FP}$$

$$\text{Sensibilidade (Recall)} = \frac{VP}{VP + FN}$$

$$\text{F1-score} = \frac{2 \times \text{Precisão} \times \text{Sensibilidade}}{\text{Precisão} + \text{Sensibilidade}}$$

3. Imprima os resultados formatados com **duas casas decimais**, um por linha.

Importante:

- Use divisão em ponto flutuante para evitar resultados truncados.
- A ordem de saída deve ser: **Acurácia**, **Precisão**, **Sensibilidade** e **F1-score**.
- Veja a simulação interativa deste EP na Figure 1.13.

1.16.2.3 Exemplo de Execução

Entrada	Saída	Observação
40	0.75	Acurácia
10	0.73	Precisão
15	0.80	Sensibilidade

Entrada	Saída	Observação
35	0.76	F1-score

(Nota: $VP=40$, $FN=10$, $FP=15$, $VN=35$. Total de casos = 100)

Simulador: desempenho de classificadores

Ajuste os valores da matriz de confusão e veja as métricas calculadas em tempo real.

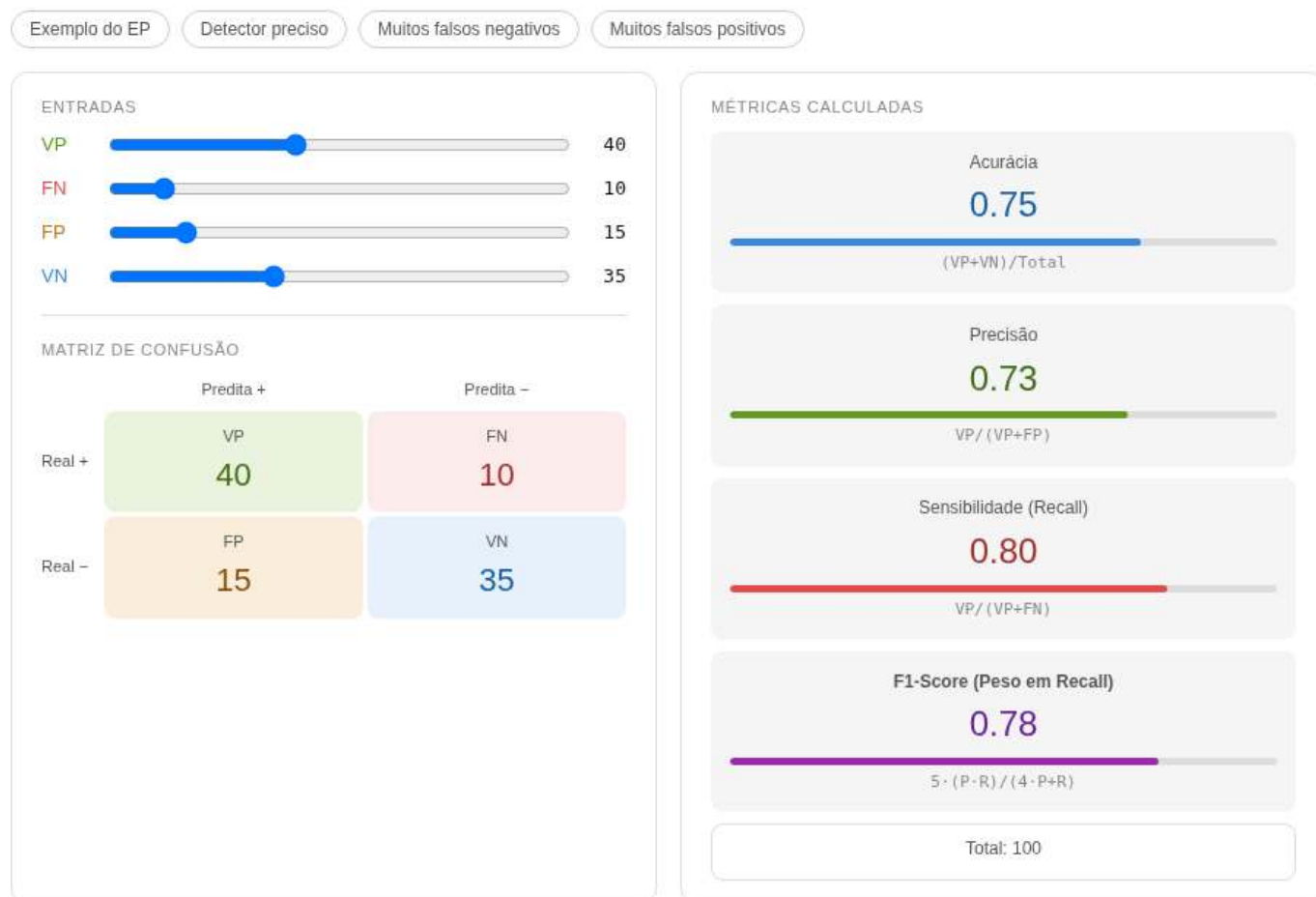


Figura 1.13: Simulador: Desempenho Preditivo — Métricas de ML

```
%%writefile EP01_02.py
# sua solução
```

Overwriting EP01_02.py

```
TestSuite("EP01_02.py").run()
```

1.16.3 EP01_03 ↗ *Mean Average Precision* (mAP) — Curva Precisão-Sensibilidade

Nesta atividade, você avaliará um classificador binário (ex.: detecção de desmatamento em imagens de satélite, ver dgi.inpe.br) através da **curva Precisão-Sensibilidade** e da métrica **mAP** (*Mean Average*

Precision). O mAP é padrão em competições como [COCO \(Common Objects in Context\)](#) e [PASCAL VOC \(Visual Object Classes\)](#) e dos modelos [YOLO \(You Only Look Once\)](#).

1.16.3.1 🧠 Por que o mAP é a métrica-padrão?

Na EP01_02 você viu que a escolha do limiar altera significativamente a Precisão e a Sensibilidade. O **mAP (Mean Average Precision)** resolve isso: avalia o modelo em **vários limiares** (cada limiar deve gerar uma matriz de confusão diferente) e resume o desempenho pela **área sob a curva Precisão-Sensibilidade (P-S)**.

Enquanto o *F1-Score* olha para um único ponto de equilíbrio, o mAP considera a curva inteira. Quanto mais próximo de **1,0**, melhor o detector em todos os limiares e classes (ex.: moedas de 25, 50 e 1 real).

Métrica	O que resume	Limitação
F1-Score	Equilíbrio $P \times S$ num único limiar	Depende do limiar escolhido
AP	Área sob a curva P-S de uma classe	Válida apenas para uma classe
mAP	Média das APs sobre todas as classes	Mais complexo de implementar

Referências: [Roboflow — mAP](#) · [Vídeo explicativo](#)

1.16.3.2 Como o mAP é calculado — passo a passo

1. **Limiares fixos** (use sempre esta lista):

```
limiares = [0.00, 0.09, 0.21, 0.31, 0.39, 0.52, 0.60, 0.71, 0.81, 0.89, 1.00]
```

2. Para cada limiar (t), classifique as amostras: **predito = 1 se confiança t , senão 0**. Calcule VP, FP, FN, VN e obtenha $\text{Precisão}((t))$ e $\text{Sensibilidade}((t))$.
3. Monte a curva P-S: pares ($\text{Sensibilidade}((t))$, $\text{Precisão}((t))$), ordenados por Sensibilidade crescente.
4. **Monotonize a Precisão**:

$$P_{\text{mono}}[i] = \max_{j \geq i} P[j]$$

5. Calcule a **AP** (área sob a curva monotônica) usando a **regra do trapézio** (aproximação mais precisa que a simples soma de Riemann):

$$AP = \sum_{i=1}^{m-1} \frac{P_{\text{mono}}[i-1] + P_{\text{mono}}[i]}{2} \cdot (S[i] - S[i-1])$$

6. **mAP** = média das APs de todas as classes. Neste EP há apenas **1 classe**, portanto $\text{mAP} = \text{AP}$.

Note

Diferença resumida:

A *soma de Riemann* aproxima a área por retângulos, podendo subestimar ou superestimar. A **regra do trapézio** usa trapézios, reduzindo o erro ao considerar a média entre os valores nos extremos do intervalo, sendo geralmente mais precisa para funções suaves por partes, como a curva Precisão-Sensibilidade.

1.16.3.3 📋 Tarefa

Leia um inteiro n (quantidade de amostras). Em seguida leia n linhas, cada uma com: **verdade** (0 ou 1) e **confiança** (float 0.0–1.0).

Calcule e imprima, para o **limiar 0.85** (índice 9 da lista):

- Matriz de Confusão (VP, FN, FP, VN)
- Acurácia, Precisão, Sensibilidade e F1-Score

Em seguida, para **todos os limiares**, imprima:

- Precisões cruas, Precisões monotônicas e Sensibilidades, separadas por ,
- mAP final

1.16.3.4 Importante

- Limiar fixo para as métricas individuais: **0.85**
- Divisão segura: se denominador for zero, use 0
- Formatação: duas casas decimais
- Monotonize de trás para frente
- A Figure 1.14 apresenta uma simulação desta questão

1.16.3.5 Exemplo de Execução

Entrada	Saída Esperada
7	# MÉTRICAS PARA O LIMAR 0.85 #
0 0.94	Matriz de Confusão:
1 0.80	VP = 0, FN = 5
1 0.69	FP = 1, VN = 1
0 0.67	
1 0.30	Métricas de Avaliação:
1 0.15	Acurácia: 0.14
1 0.15	Precisão: 0.00
	Sensibilidade: 0.00
	F1-Score: 0.00
	# MÉTRICAS PARA TODOS OS LIMIARES #
	Precisões: 0.00, 0.00, 0.00, 0.50, 0.50, 0.50, 0.50, 0.50, 0.60, 0.71, 0.71
	Precisões mon.: 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71
	Sensibilidades: 0.00, 0.00, 0.00, 0.20, 0.40, 0.40, 0.40, 0.40, 0.60, 1.00, 1.00
	mAP: 0.71

1.16.3.6 Dica para calcular o AP (com regra do trapézio)

```
def calcular_AP(verdades, confiancas, limiares):
    m = len(limiares)
    precisoes = [0.0] * m
    sensibilidades = [0.0] * m
    for i in range(m):
        p, s = calcular_metricas(verdades, confiancas, limiares[i])
        precisoes[m-1-i] = p
        sensibilidades[m-1-i] = s
    prec_mono = precisoes.copy()
    for i in range(m-2, -1, -1):
        if prec_mono[i] < prec_mono[i+1]:
            prec_mono[i] = prec_mono[i+1]
    AP = 0.0
    for i in range(1, m):
```

```
# Regra do trapézio: média das alturas vezes a base
area_trapezio = (prec_mono[i-1] + prec_mono[i]) / 2.0
AP += area_trapezio * (sensibilidades[i] - sensibilidades[i-1])
return precisoes, prec_mono, sensibilidades, AP
```

```
%%writefile EP01_03.py
# sua solução
```

Overwriting EP01_03.py

```
TestSuite("EP01_03.py").run()
```

1.16.4 EP01_04 Leitura e Informações de uma Imagem em Matriz

Nesta atividade, você deve escrever um programa que processe uma imagem digital representada como uma matriz de pixels em tons de cinza.

- Leia dois inteiros **L** e **C**, representando o número de linhas e colunas.
- Leia os **L * C** valores inteiros que compõem a matriz da imagem (cada valor entre 0 e 255).
- Calcule e imprima as seguintes informações:
 1. O número de linhas.
 2. O número de colunas.
 3. O valor do maior pixel (**Máximo**).
 4. O valor do menor pixel (**Mínimo**).
 5. A **Média** aritmética de todos os pixels.

Importante:

- A saída deve seguir exatamente o formato rotulado (ex: **Linhas: X**).
- O valor da média deve ser formatado com **duas casas decimais**.
- Ver um simulador interativo para esta questão na **Figure 1.15** (grade interativa para visualização de intensidades e cálculos em tempo real).

1.16.4.1 Por que isso importa? – A Imagem como Dados

Toda imagem digital é, no fundo, uma estrutura de dados. Em tons de cinza de 8 bits, cada pixel é um valor escalar. Extrair estatísticas básicas é o primeiro passo para:

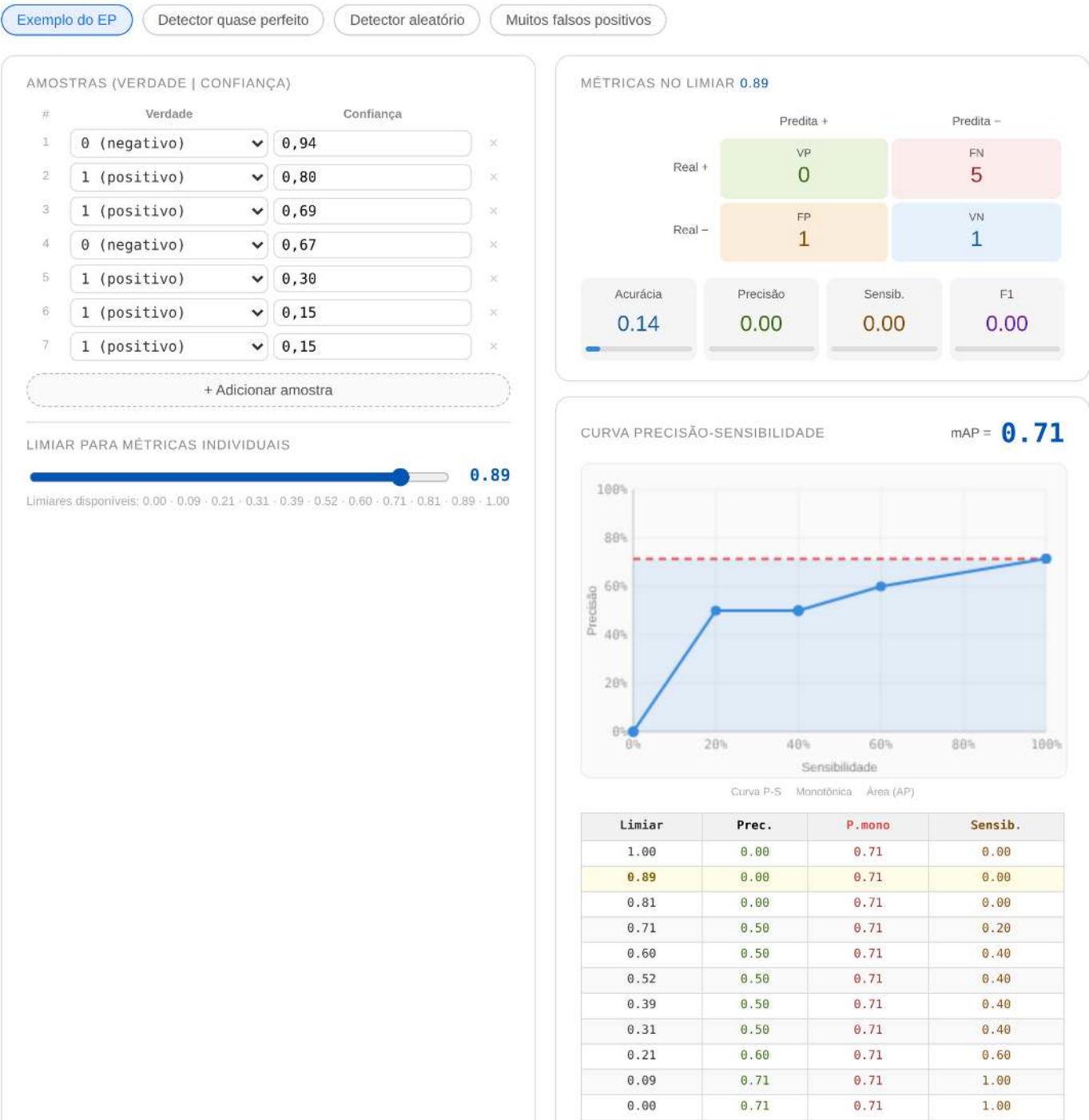
Operação	Utilidade Prática
Máximo/Mínimo	Identificar se a imagem está “lavada” (baixo contraste) ou saturada.
Média	Calcular o brilho global da cena para ajustes de exposição.
Normalização	Redimensionar os valores para intervalos como [0, 1] em redes neurais.

1.16.4.2 Tarefa (especificação para VPL)

Entrada:

Simulador: Curva P-S e mAP

Edite as amostras (verdade e confiança) e veja a curva Precisão-Sensibilidade e o mAP calculados em tempo real.



CURVA PRECISÃO-SENSIBILIDADE

mAP = 0.71

Limiar	Prec.	P.mono	Sensib.
1.00	0.00	0.71	0.00
0.89	0.00	0.71	0.00
0.81	0.00	0.71	0.00
0.71	0.50	0.71	0.20
0.60	0.50	0.71	0.40
0.52	0.50	0.71	0.40
0.39	0.50	0.71	0.40
0.31	0.50	0.71	0.40
0.21	0.60	0.71	0.60
0.09	0.71	0.71	1.00
0.00	0.71	0.71	1.00

Figura 1.14: Simulador: Mean Average Precision (mAP) - Curva P.S

A primeira linha contém o inteiro L (linhas).

A segunda linha contém o inteiro C (colunas).

As linhas seguintes contêm os elementos da matriz.

Saída:

Cinco linhas formatadas conforme o exemplo:

Linhas: L

Colunas: C

Max: V

Min: V

Media: V.VV

1.16.4.3 Exemplos

Entrada	Saída	Observação
2	Linhas: 2	Imagem pequena de alto contraste
3	Colunas: 3	
0 128 255	Max: 255	
50 100 200	Min: 0	
	Media: 122.17	



Figura 1.15: Simulador: Estatísticas de Pixels

```
%%writefile EP01_04.py
# sua solução
```

Overwriting EP01_04.py

```
TestSuite("EP01_04.py").run()
```

1.16.5 EP01_05 Negativo de uma Imagem em Tons de Cinza

Nesta atividade, deve-se escrever um programa que calcule o negativo de uma imagem digital.

- Leia dois inteiros **L** e **C**, representando o número de linhas e colunas.
- Leia os valores inteiros que compõem a matriz da imagem.
- Para cada pixel, aplique a transformação de inversão:

$$pixel_{negativo} = 255 - pixel_{original}$$

- Imprima a matriz resultante, mantendo o formato original (L linhas e C colunas).

Importante:

- Os valores de cada linha na saída devem ser separados por um **espaço em branco**.
- A saída deve conter **apenas os números** da matriz resultante.
- Ver um simulador interativo para esta questão na **Figure 1.16** (comparação em tempo real entre a matriz original e seu negativo).

1.16.5.1 Por que isso importa? – Inversão de Intensidade

O **negativo** é uma transformação linear básica que inverte a escala de brilho. É uma ferramenta essencial para o olho humano identificar detalhes claros que estão “escondidos” em fundos mais escuros, sendo amplamente utilizada em:

Aplicação	Utilidade
Imagens Médicas	Melhora a visualização de anomalias em tecidos densos (ex: Raios-X).
Astronomia	Destacar galáxias e nebulosas tênues contra o vazio do espaço.
Artes Digitais	Efeitos estéticos e preparação de máscaras de seleção.

1.16.5.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém o inteiro **L**.

A segunda linha contém o inteiro **C**.

As linhas seguintes contêm os elementos da matriz.

Saída:

A matriz invertida com **L** linhas e **C** colunas.

1.16.5.3 Exemplos

Entrada	Saída	Observação
2	255 127 0	Onde era 0 (preto) vira 255 (branco)
3	205 155 55	
0 128 255		
50 100 255		



Figura 1.16: Simulador: Transformação de Negativo

```
%%writefile EP01_05.py
# sua solução
```

Overwriting EP01_05.py

```
TestSuite("EP01_05.py").run()
```

1.16.6 EP01_06 🎨 Conversão RGB → Tons de Cinza (ITU-R BT.601)

Nesta atividade, você deve escrever um programa que converta pixels coloridos (RGB) para tons de cinza utilizando a ponderação fisiológica do padrão ITU-R BT.601.

- Leia dois inteiros **L** e **C**, representando o número de linhas e colunas.
- Leia **L** × **C** triplas de inteiros, onde cada tripla representa os canais **R** (Red), **G** (Green) e **B** (Blue) de um pixel.
- Para cada pixel, calcule o valor de cinza (*g*) usando a fórmula:

$$g = \text{round}(0.299 \times R + 0.587 \times G + 0.114 \times B)$$

- Imprima a matriz resultante (L linhas e C colunas) contendo os valores inteiros convertidos.

📌 **Importante:**

- Utilize a função `round()` da sua linguagem para garantir o arredondamento correto para o inteiro mais próximo.
- A saída deve conter apenas os valores de cinza, mantendo a estrutura de matriz (separados por espaço na linha).
- Ver um simulador interativo para esta questão na **Figure 1.17** (ajuste os sliders para ver como cada cor contribui para o brilho final).

1.16.6.1 🌸 Por que não usar apenas a média?

O olho humano não percebe todas as cores com a mesma intensidade. Somos muito mais sensíveis ao **Verde** do que ao **Azul** devido à nossa evolução biológica. O padrão ITU-R BT.601 utiliza pesos específicos para criar uma imagem em cinza que pareça naturalmente correta para nossa visão:

Canal	Peso	Percepção Humana
 Verde	58.7%	Máxima sensibilidade (distinção de folhagens).
 Vermelho	29.9%	Sensibilidade média.
 Azul	11.4%	Baixa sensibilidade (tons mais escuros).

1.16.6.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém o inteiro L.

A segunda linha contém o inteiro C.

As linhas seguintes contêm triplas de inteiros R G B para cada pixel.

Saída:

A matriz de cinzas com L linhas e C colunas.

1.16.6.3 📌 Exemplos

Entrada	Saída	Observação
1 3 255 0 0 0 255 0 0 0 255	76 150 29	Note como o Verde (150) é mais brilhante que o Azul (29)

```
%%writefile EP01_06.py
# sua solução
```

```
Overwriting EP01_06.py
```

```
TestSuite("EP01_06.py").run()
```

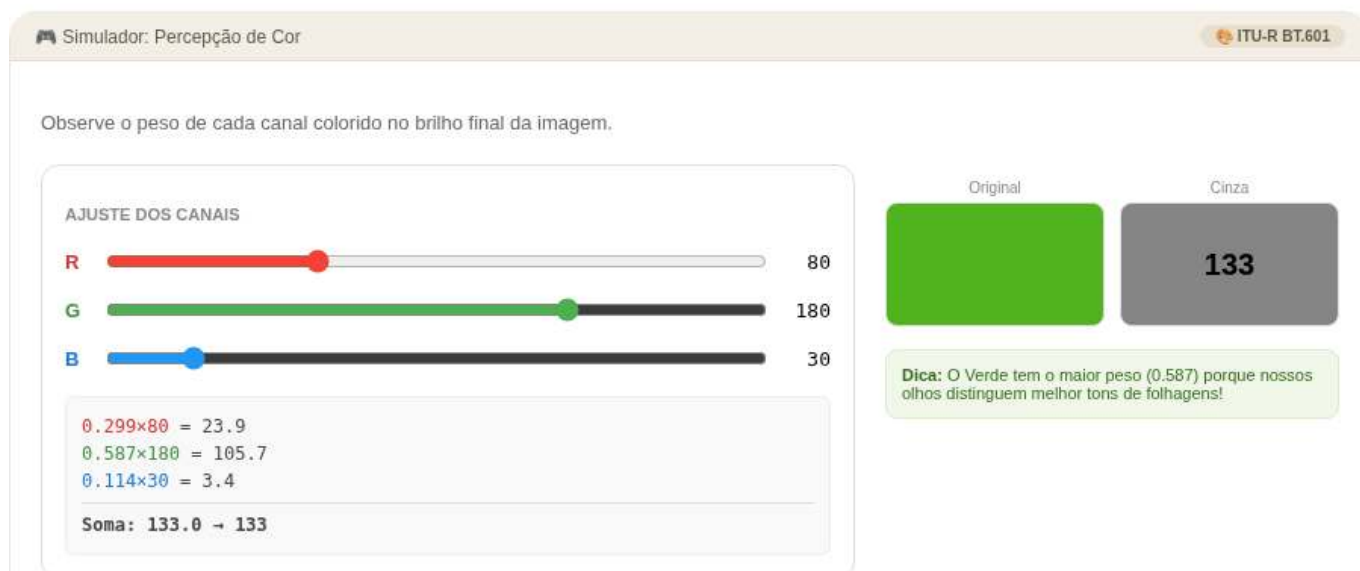


Figura 1.17: Simulador: Percepção de Cor e Pesos ITU-R

1.16.7 EP01_07 ● Limiarização Manual: Imagem Binária

Nesta atividade, você deve escrever um programa que realize a segmentação de uma imagem através da limiarização (*thresholding*).

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz.
- Leia um inteiro **T**, que será o valor do limiar (corte).
- Leia os valores inteiros da matriz.
- Para cada pixel p , aplique a seguinte regra de binarização:

$$\text{resultado} = \begin{cases} 255 & \text{se } p > T \\ 0 & \text{se } p \leq T \end{cases}$$

- Imprima a matriz resultante contendo apenas os valores 0 ou 255.

📌 Importante:

- Preste atenção ao operador: o pixel só vira branco (255) se for **estritamente maior** que T .
- A saída deve manter a estrutura de matriz (L linhas e C colunas).
- Ver um simulador interativo para esta questão na **Figure 1.18** (ajuste o slider de T para observar como os objetos são isolados do fundo).

1.16.7.1 🧠 O que é Segmentação?

A limiarização é o método mais simples de separar objetos de interesse do fundo da imagem. Ao transformar tons de cinza em preto e branco puro, criamos um mapa binário que facilita a contagem de objetos ou a identificação de formas:

Valor do Pixel (p)	Condição	Resultado Final
Escuro ($p \leq T$)	Fundo/Ruído	0 (Preto)
Claro ($p > T$)	Objeto/Destaque	255 (Branco)

1.16.7.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém o inteiro L .

A segunda linha contém o inteiro C .

A terceira linha contém o inteiro T (limiar).

As linhas seguintes contêm os elementos da matriz.

Saída:

A matriz binarizada (0 ou 255) com L linhas e C colunas.

1.16.7.3 📌 Exemplos

Entrada	Saída	Observação
2	0 0 0 255	Note que o valor 128 virou 0 (pois $128 \leq 128$)
4	0 255 255 0	
128		
0 100 128 200		
50 129 255 64		

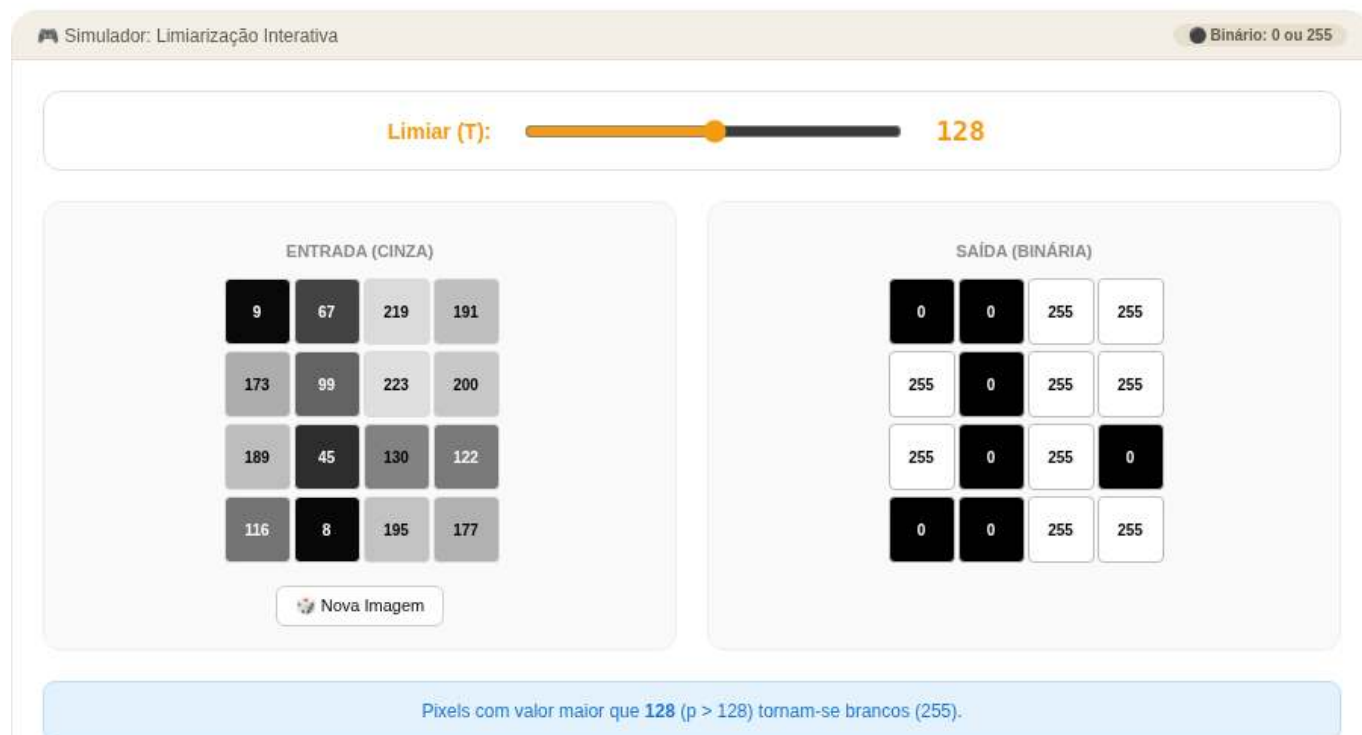


Figura 1.18: Simulador: Limiarização Interativa

```
%%writefile EP01_07.py
# sua solução
```

Overwriting EP01_07.py

```
TestSuite("EP01_07.py").run()
```

1.16.8 EP01_08 🧠 Remapeamento por Faixa de Intensidade

Nesta atividade, você deve escrever um programa que aplique transformações lineares distintas a diferentes regiões de intensidade da imagem.

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz.
- Leia os inteiros **T** (limiar), δ_1 (delta 1) e δ_2 (delta 2).
- Leia os valores inteiros da matriz.
- Para cada pixel p , aplique a regra de remapeamento condicional:

$$\text{resultado} = \begin{cases} p + \delta_1 & \text{se } p < T \\ p + \delta_2 & \text{se } p \geq T \end{cases}$$

- Imprima a matriz resultante com os novos valores de intensidade.

📌 Importante:

- δ_1 é o offset aplicado aos pixels escuros (abaixo do limiar).
- δ_2 é o offset aplicado aos pixels claros (maior ou igual ao limiar).
- Os casos de teste garantem que o resultado estará sempre no intervalo válido de **0 a 255**, portanto, não é necessário tratar saturação ou arredondamentos.
- A saída deve manter a estrutura de matriz (**L** linhas e **C** colunas).

1.16.8.1 🧠 Transformação Condicional de Pixels

Em Processamento Digital de Imagens (PDI), frequentemente precisamos tratar regiões de forma independente. Esta técnica permite, por exemplo, clarear apenas as sombras de uma fotografia (aumentando os pixels escuros) sem estourar o brilho das áreas já claras, ou vice-versa.

Faixa de Intensidade	Condição	Operação
Pixels Escuros	$p < T$	$p + \delta_1$
Pixels Claros	$p \geq T$	$p + \delta_2$

1.16.8.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém os inteiros **L** e **C**.

A segunda linha contém os inteiros **T**, δ_1 e δ_2 .

As linhas seguintes contêm os elementos da matriz.

Saída:

A matriz transformada com **L** linhas e **C** colunas, com valores separados por espaço.

1.16.8.3 📌 Exemplos

Entrada	Saída	Observação
2 4 128 60 -40 0 100 150 255 80 128 200 30	60 160 110 215 140 88 160 90	Pixels < 128 somam 60. Pixels 128 subtraem 40.

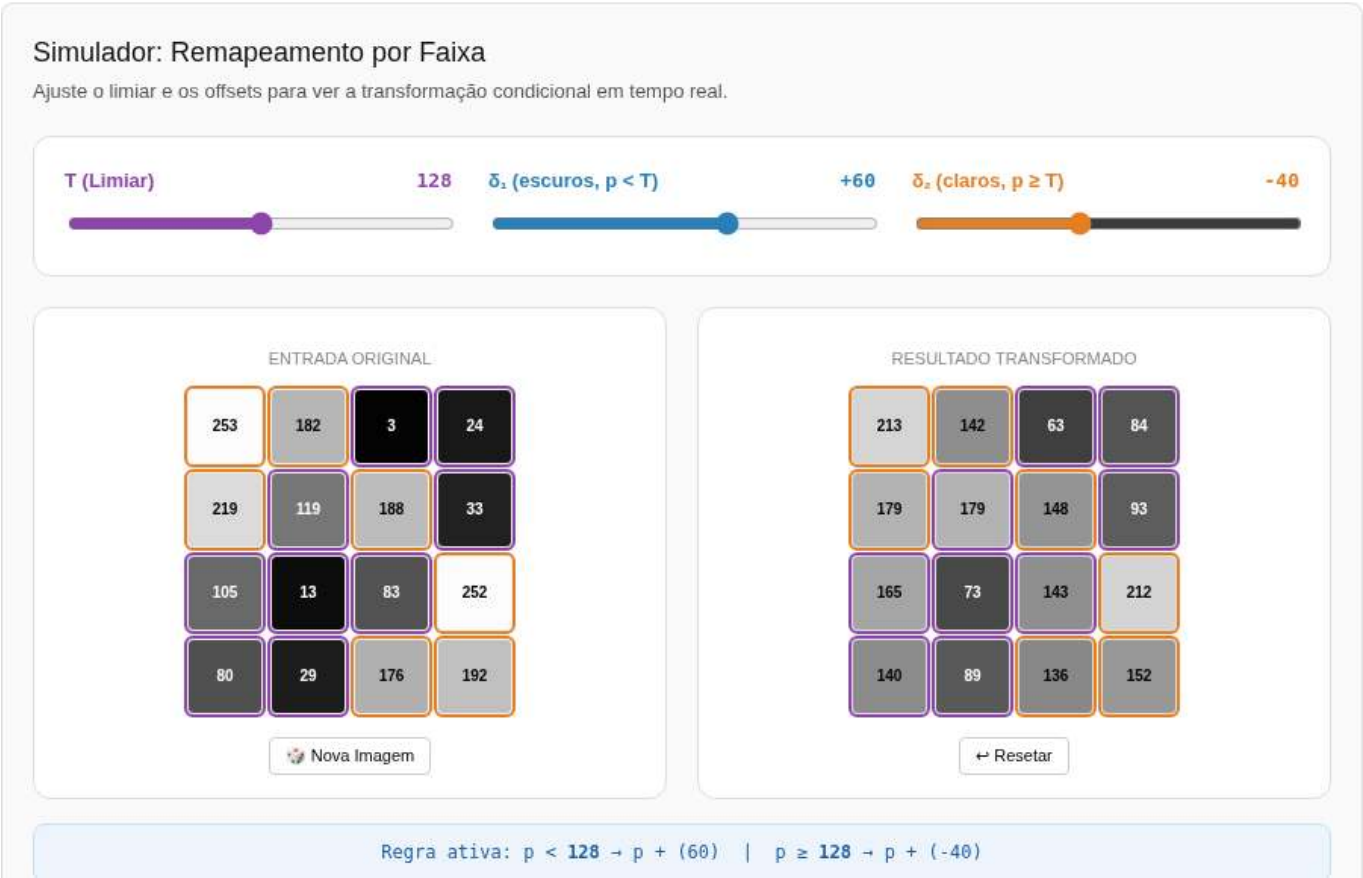


Figura 1.19: Simulador: Ajuste de Brilho e Contraste

```
%%writefile EP01_08.py
# sua solução

Overwriting EP01_08.py

TestSuite("EP01_08.py").run()
```

1.16.9 EP01_09 🏁 Padrão de Tabuleiro: Matriz de Xadrez

Nesta atividade, você deve escrever um programa que gere uma imagem sintética no padrão de um tabuleiro de xadrez.

- Leia dois inteiros **L** (linhas) e **C** (colunas).
- Gere uma matriz onde os valores alternam entre **0** (preto) e **1** (branco).

- A lógica de preenchimento deve seguir a regra da paridade:
- O elemento na posição $(0, 0)$ é sempre **0**.
- Um pixel na posição (i, j) será **1** se a soma dos índices $(i + j)$ for **ímpar**.
- Um pixel na posição (i, j) será **0** se a soma dos índices $(i + j)$ for **par**.

📌 Importante:

- As cores devem alternar corretamente tanto horizontalmente quanto verticalmente.
- A saída deve ser a matriz impressa linha por linha, com os elementos separados por um espaço.
- Ver um simulador interativo para esta questão na **Figure 1.20** (ajuste as dimensões para visualizar a construção da malha e a saída textual correspondente).

1.16.9.1 🧠 Padrões Sintéticos

Criar padrões geométricos é um exercício fundamental para dominar a **lógica de índices** em matrizes. Em Processamento Digital de Imagens, o padrão de xadrez não é apenas estético; ele é amplamente utilizado para:

Aplicação	Utilidade
Calibração de Câmera	Estimar parâmetros intrínsecos e extrínsecos da lente.
Correção de Distorção	Identificar e corrigir o efeito de “barril” ou “almofada” em lentes grande-angulares.
Mapeamento 3D	Projetar padrões conhecidos para reconstruir superfícies em sistemas de luz estruturada.

1.16.9.2 📋 Tarefa (especificação para VPL)

Entrada:

Uma linha contendo o inteiro L (linhas).

Uma linha contendo o inteiro C (colunas).

Saída:

A matriz de xadrez com L linhas e C colunas, impressa com espaços entre os elementos.

1.16.9.3 📌 Exemplos

Entrada	Saída	Observação
3	0 1 0 1	Note que cada linha começa com o inverso da anterior
4	1 0 1 0	
	0 1 0 1	

```
%%writefile EP01_09.py
# sua solução
```

```
Overwriting EP01_09.py
```

```
TestSuite("EP01_09.py").run()
```

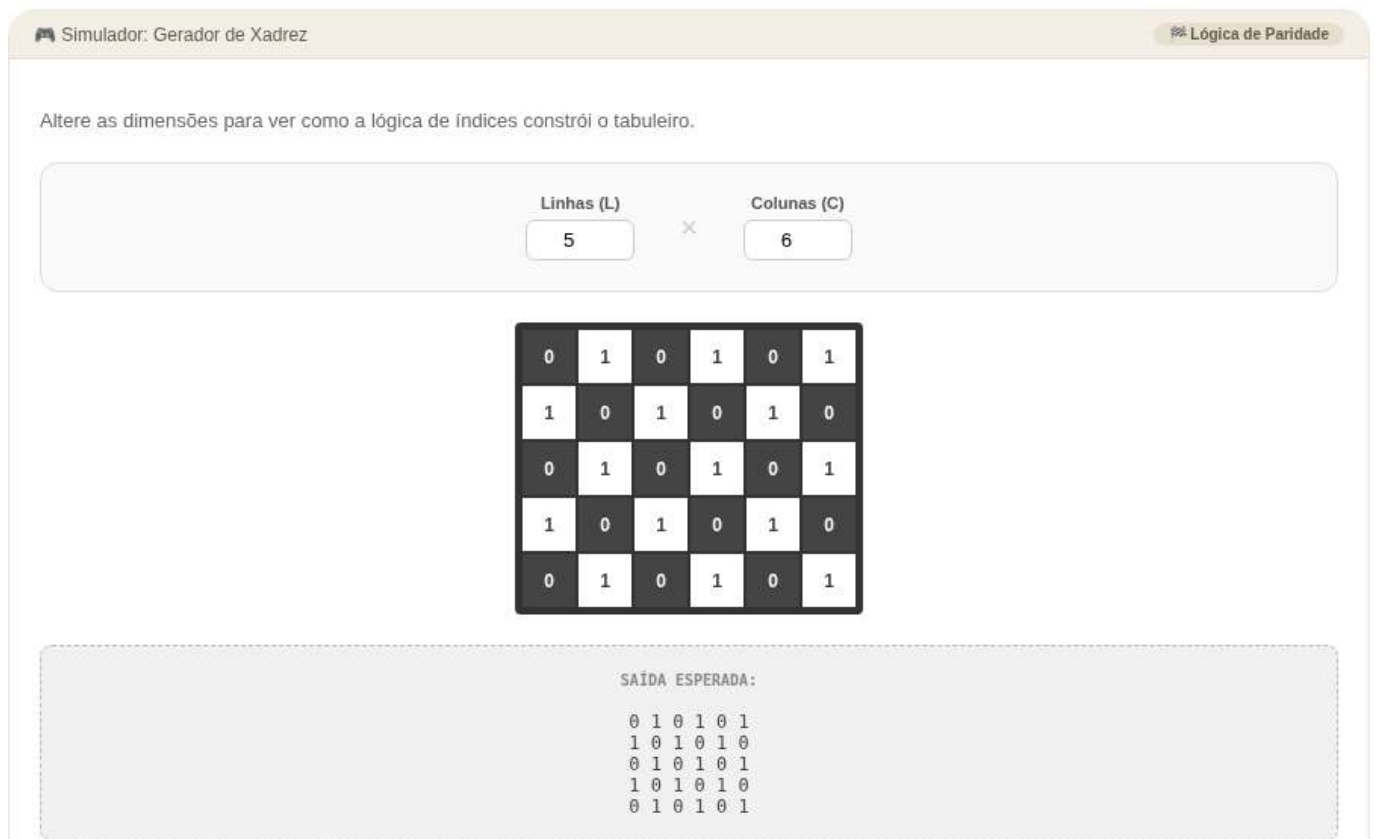



Figura 1.20: Simulador: Gerador de Xadrez

1.16.10 EP01_10 Metadados: Leitura de Arquivo PGM

Nesta atividade, você deve ler um arquivo de imagem no formato **PGM (Portable Gray Map)** e extrair suas dimensões a partir do cabeçalho.

- O formato **PGM (P2)** é um arquivo de texto simples (ASCII) que armazena imagens em tons de cinza.
- O arquivo possui um **cabeçalho** estruturado da seguinte forma:
 1. **Versão:** O identificador P2.
 2. **Comentários:** Linhas opcionais que começam com # (devem ser ignoradas).
 3. **Dimensões:** Dois inteiros representando **Largura** e **Altura**.
 4. **Máximo:** Um inteiro representando a intensidade máxima (geralmente 255).
- Após o cabeçalho, seguem-se os dados dos pixels.

📌 Importante:

- **Leitura de Arquivo:** Você deve abrir o arquivo indicado no exemplo utilizando a função `open()` do Python.
- **Ordem de Saída:** Ao contrário da ordem presente no arquivo, a saída esperada deve estar no formato de tupla: (**Altura**, **Largura**, **Canais**).
- Como arquivos PGM são em tons de cinza, o número de **Canais** é sempre **1**.
- Ver um simulador interativo para esta questão na **Figure 1.21** (ajuste as dimensões para ver como o cabeçalho ASCII é gerado).

1.16.10.1 🧠 Entendendo o formato PGM

O formato PGM é um dos mais simples para processamento de imagens. Por ser em texto puro, ele permite visualizar os metadados e até os valores dos pixels abrindo o arquivo em um bloco de notas:

Componente	Exemplo	Significado
Magic Number	P2	Identifica que é um PGM em formato de texto (ASCII).
Comentário	# CREATOR...	Linha informativa ignorada pelo processador.
Dimensões	397 343	397 colunas (Largura) e 343 linhas (Altura).
Intensidade	255	Define o valor do branco puro (escala de 0 a 255).

1.16.10.2 📋 Tarefa (especificação para VPL)

Entrada:

Nenhuma entrada via teclado. O programa deve ler o arquivo "aula01fig03b.pgm" presente no diretório de execução.

Saída:

Uma tupla contendo (Altura, Largura, 1).

1.16.10.3 📌 Exemplos

Nome do Arquivo	Saída Esperada	Observação
"aula01fig03b.pgm"	(343, 397, 1)	Note a inversão da ordem: Altura primeiro



Figura 1.21: Simulador: Estrutura do Arquivo PGM

i Nota

O arquivo necessário para este EP será baixado automaticamente do repositório por meio do seguinte código:

```
import os, urllib.request

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/all/cap01/imagens"
file = "aula01fig03b.pgm"

opener = urllib.request.build_opener()
opener.addheaders = [('User-agent', 'Mozilla/5.0')]
urllib.request.install_opener(opener)

for f in [file]:
    if not os.path.exists(f):
        url = f"{BASE_URL}/{f}"
        try:
            urllib.request.urlretrieve(url, f)
            print(f" Arquivo baixado: {f}")
        except urllib.error.HTTPError as e:
            print(f" Erro {e.code}: Não encontrado em {url}")
```

```
%%writefile EP01_10.py
# sua solução
```

```
Overwriting EP01_10.py
```

```
TestSuite("EP01_10.py").run()
```

1.16.11 EP01_11 ↗ Análise de Vizinhaça: Filtro de Máximo 1D

Nesta atividade, você deve implementar um filtro morfológico simples de máximo operando em um sinal unidimensional (vetor).

- Leia um inteiro **n**, representando o tamanho do vetor.
- Leia os **n** elementos inteiros que compõem o vetor original **v1**.
- Crie um novo vetor **v2**, onde cada posição *i* é o resultado da comparação entre o elemento atual e seus vizinhos imediatos:

$$v2[i] = \max(v1[i-1], v1[i], v1[i+1])$$

 Importante:

- **Bordas:** Nas extremidades do vetor (índices 0 e $n-1$), a vizinhaça possui apenas dois elementos (o próprio e o único vizinho disponível). No índice 0, compare apenas $v1[0]$ e $v1[1]$. No último índice, compare apenas $v1[n-2]$ e $v1[n-1]$.
- **Saída:** Imprima o cabeçalho “v2:” seguido pelos valores do vetor resultante, um por linha.
- Ver um simulador interativo para esta questão na **Figure 1.22** (passe o mouse sobre os resultados para visualizar a janela de vizinhaça utilizada no cálculo).

1.16.11.1 🧠 Por que analisar vizinhos?

Em processamento de imagens, o valor de um pixel raramente é isolado; ele depende do contexto ao seu redor. O **Filtro de Máximo** é a base da operação de **Dilatação** em morfologia matemática, servindo para:

Função	Efeito Visual
Realce	Expande estruturas brilhantes e “engorda” objetos claros.
Remoção de Ruído	Elimina pequenos pontos pretos (ruído de “sal e pimenta” escuro).
Preenchimento	Fecha pequenos buracos ou lacunas em formas binárias.

1.16.11.2 📋 Tarefa (especificação para VPL)

Entrada:

Um inteiro n .

Nas linhas seguintes, os n elementos inteiros do vetor.

Saída:

A string $v2$: na primeira linha.

Nas linhas seguintes, cada elemento de $v2$ (um por linha).

1.16.11.3 📌 Exemplos

Entrada	Saída	Observação
5	$v2$:	No índice 1: $\max(10, 20, 5) = 20$
10	20	
20	20	
5	30	
30	30	
15	30	

```
%%writefile EP01_11.py
# sua solução
```

Overwriting EP01_11.py

```
TestSuite("EP01_11.py").run()
```



Figura 1.22: Simulador: Filtro de Máximo Local 1D

Capítulo 2

Da Captura ao Pixel - Amostragem, Quantização e Conectividade

[Executar Colab](#) [Abrir si-md2](#) [GitHub](#)

Este capítulo aprofunda a compreensão da imagem digital, transitando da natureza física da captura para a sua representação matemática discreta. Investigamos como a luz se torna dado e como a organização espacial dos pixels define as relações de vizinhança e conectividade essenciais para algoritmos avançados de Visão Computacional (VC).

2.1 Objetivos

Ao final deste capítulo, você será capaz de:

- Explicar o modelo físico de formação da imagem baseado em iluminação e refletância.
- Diferenciar os mecanismos da visão humana e dos sensores digitais.
- Compreender os processos de **amostragem** (discretização do espaço) e **quantização** (discretização da intensidade).
- Descrever as relações topológicas entre pixels: vizinhança, adjacência, conectividade e distâncias.
- Realizar transformações geométricas básicas (translação, rotação, escala) preservando a qualidade.
- Visualizar na prática os efeitos da variação da resolução espacial e da profundidade de bits.

2.2 O Olho e a Câmera - Elementos da Percepção Visual

A formação de uma imagem digital começa com a captura da luz refletida pelos objetos. Como ilustrado na Figure 2.1, tanto o olho humano quanto as câmeras digitais seguem princípios ópticos semelhantes para focar a luz em uma superfície sensível, embora utilizem mecanismos biológicos e eletrônicos distintos para a transdução do sinal.

2.2.1 Visão humana

O olho funciona como um sistema óptico complexo: a luz atravessa a córnea, o humor aquoso, a pupila (controlada pela íris) e o cristalino — que ajusta o foco dinamicamente — até atingir a retina. Na retina, encontram-se os fotorreceptores: os **cones** (6 milhões), concentrados na fóvea, são responsáveis pela visão de cores e detalhes, enquanto os **bastonetes** (120 milhões) garantem a visão em baixa luminosidade (visão escotópica), detectando apenas intensidades de cinza. O ponto cego é a região de onde parte o nervo óptico, carecendo de receptores.

2.2.2 Sensores digitais

Nas câmeras, os sensores de imagem desempenham o papel da retina. Os dois tipos mais comuns são o **CCD** (*Charge-Coupled Device* - Dispositivo de Carga Acoplada) e o **CMOS** (*Complementary Metal-Oxide-Semiconductor* - Semicondutor de Óxido Metálico Complementar). O sensor é composto por uma matriz de fotossítios (pixels) que acumulam carga elétrica proporcional à luz incidente.

Para a reconstrução de cores, utiliza-se o **Filtro de Bayer**, uma matriz de filtros coloridos que permite que cada pixel capture apenas uma componente de cor: vermelho, verde ou azul (RGGB - *Red, Green, Green, Blue*). Posteriormente, um **ADC** (*Analog-to-Digital Converter* - Conversor Analógico-Digital) quantiza essa carga em valores numéricos, definidos por uma profundidade de bits (ex.: 8 bits, resultando em 256 níveis de intensidade).

i Curiosidade

Embora o olho humano tenha milhões de receptores, a resolução de alta definição é restrita à fóvea (visão central), equivalente a aproximadamente 120×120 pixels. A percepção de uma cena completa em alta resolução é resultado de intenso pós-processamento realizado pelo cérebro.

O Olho e a Câmera - Elementos da Percepção Visual

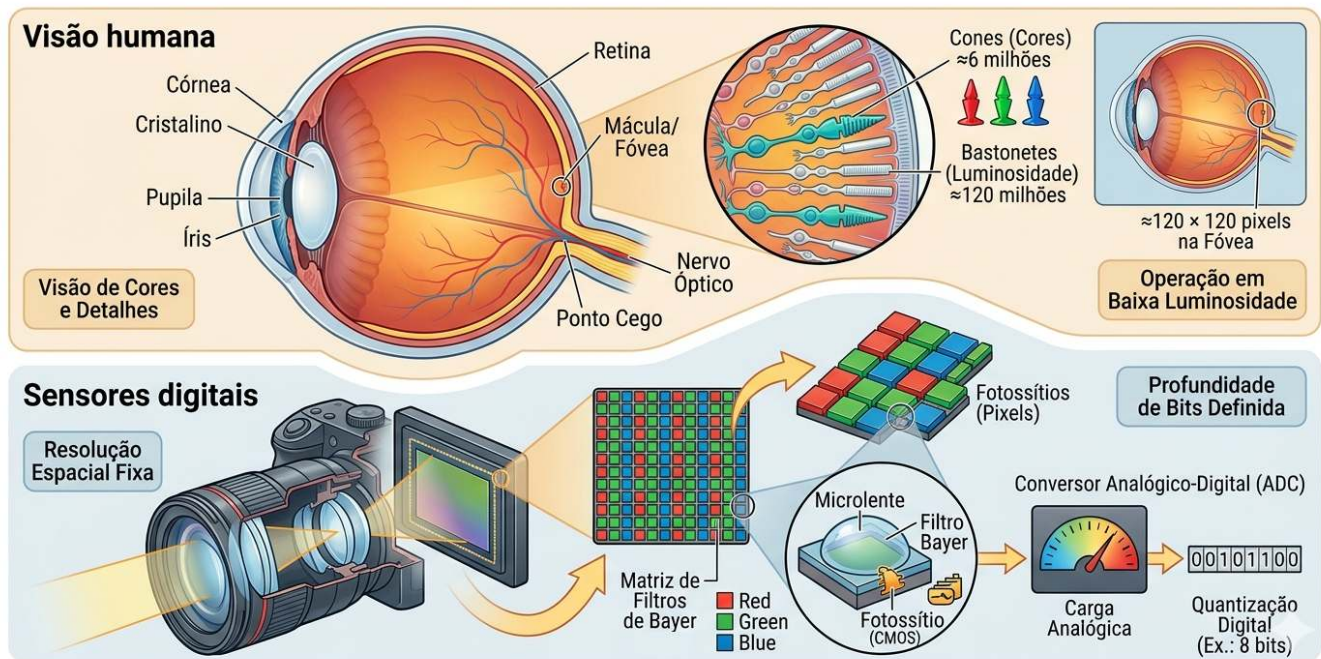


Figura 2.1: Comparativo didático entre o sistema visual biológico e o eletrônico: no topo, a anatomia do olho humano destacando a retina e os fotorreceptores (cones e bastonetes); abaixo, a estrutura de uma câmera digital detalhando o sensor CMOS, a matriz de filtros de Bayer (RGGB) e o processo de quantização digital realizado pelo ADC.

2.3 Ilusões de Ótica: Os Desafios da Percepção Visual

Enquanto os sensores digitais capturam a intensidade da luz de forma linear e objetiva, o sistema visual humano interpreta a cena com base em contexto, experiências prévias e mecanismos biológicos de sobrevivência. As ilusões de ótica não são “erros” do olho, mas evidências do intenso **pós-processamento cerebral** realizado no córtex visual.

2.3.1 Ambiguidade e Contexto

O cérebro busca constantemente dar sentido a padrões ambíguos. No exemplo do **Vaso de Rubin** (veja a Figure 2.2), a percepção alterna entre a figura (vaso) e o fundo (dois rostos), demonstrando que não conseguimos processar ambas as interpretações simultaneamente. Já a ilusão do **Elefante de Shepard** brinca com a nossa incapacidade de reconciliar linhas de contorno que sugerem volume em posições logicamente impossíveis.

2.3.2 Brilho e Contraste Local

Muitas ilusões decorrem da **inibição lateral**, mecanismo pelo qual neurônios vizinhos na retina competem entre si para realçar bordas. Na **Grade de Scintillating**, pontos escuros “fantasmas” parecem surgir nas interseções brancas devido a esse processamento local de contraste.

A ilusão da **Sombra no Tabuleiro de Adelson** é talvez a mais impactante para a VC: o quadrado “A” e o quadrado “B” possuem exatamente o mesmo valor de cinza no sensor (ou arquivo digital), mas o cérebro “corrige” o brilho de “B” por entender que ele está sob uma sombra projetada, percebendo-o como mais claro.

2.3.3 Geometria e Perspectiva

A percepção de profundidade pode ser enganada por construções geométricas que desafiam a lógica tridimensional a partir de um ângulo de visão específico. A **Escada de Schröder** utiliza a ambiguidade da perspectiva para criar um objeto que parece subir ou descer dependendo de como é observado, evidenciando como a nossa interpretação de “cima” e “baixo” depende do ponto de fuga.

Ilusões de Ótica: Os Desafios da Percepção Visual

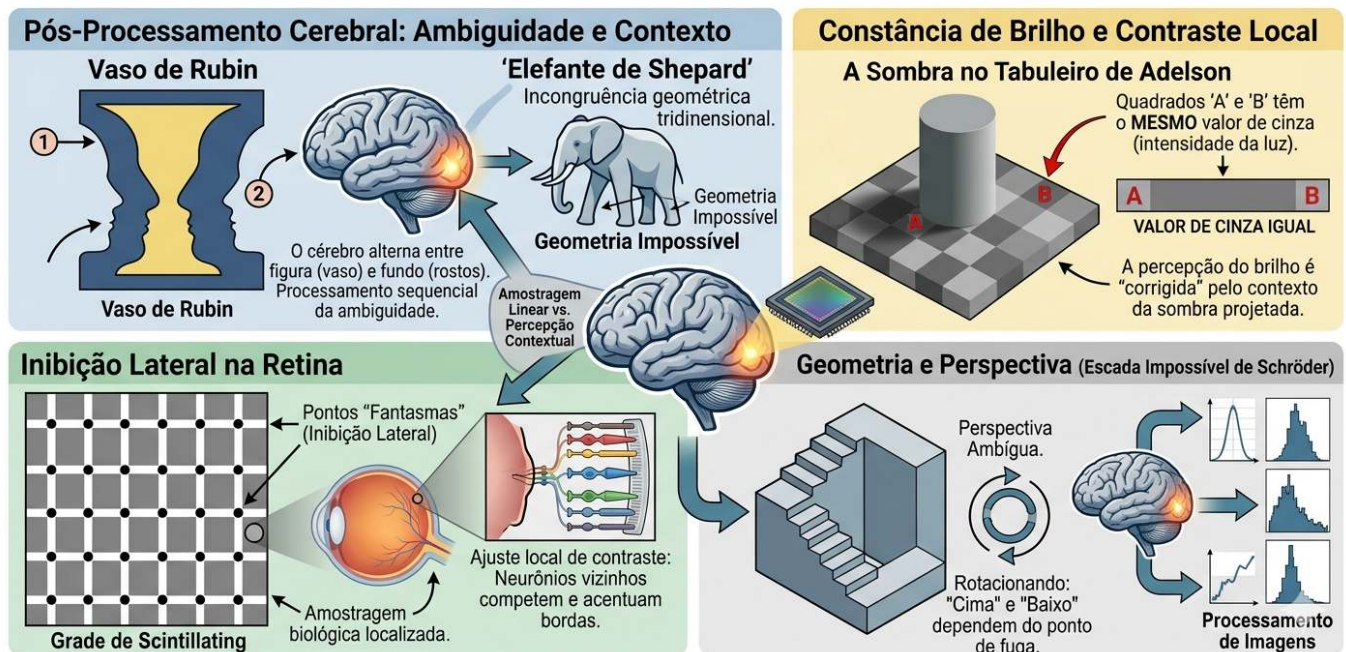


Figura 2.2: Coletânea de desafios perceptivos: (topo esquerdo) Vaso de Rubin — ambiguidade figura-fundo; (topo central) Elefante de Shepard — incongruência geométrica; (topo direita) Sombra de Adelson — constância de brilho baseada no contexto; (inferior esquerdo) Grade de pontos — inibição lateral; (inferior direito) Escada de Schröder.

2.4 O Modelo Matemático da Formação da Imagem

Uma imagem pode ser modelada como o produto de duas funções:

$$f(x, y) = i(x, y) \cdot r(x, y) \quad (2.1)$$

onde:

- $i(x, y)$ é a **iluminação** incidente sobre a cena (energia luminosa por unidade de área), determinada pela fonte de luz;

- $r(x, y)$ é a **refletância** do objeto (fração da luz refletida), determinada pelas propriedades ópticas da superfície, com $0 < r(x, y) < 1$.

Na prática, as duas componentes variam em escalas espaciais distintas: $i(x, y)$ tende a variar lentamente no espaço, enquanto $r(x, y)$ pode apresentar variações abruptas associadas a texturas, bordas e detalhes finos. Técnicas de PDI frequentemente procuram separar ou compensar essas componentes, como em métodos de correção de iluminação não uniforme.

A Figure 2.3 ilustra como esse modelo se manifesta em imagens digitais coloridas, representadas por múltiplos canais espectrais (RGB) e, opcionalmente, por um canal adicional de transparência (RGBA).

2.4.1 Representação Digital e Canais Espectrais

Em imagens digitais coloridas, a função $f(x, y)$ da Equation 2.1 é representada por múltiplos canais espectrais. No padrão **RGB**, cada pixel armazena três amostras independentes:

$$\mathbf{f}(x, y) = [R(x, y), G(x, y), B(x, y)]$$

correspondentes às intensidades das componentes vermelha (*Red*), verde (*Green*) e azul (*Blue*). Em imagens do tipo **RGBA**, adiciona-se um quarto canal:

$$\mathbf{f}(x, y) = [R(x, y), G(x, y), B(x, y), A(x, y)]$$

onde $A(x, y)$ representa o canal **alfa** (*Alpha*), responsável por codificar a opacidade do pixel. Por convenção, o valor $A = 0$ indica um pixel totalmente transparente, enquanto $A = 255$ (ou 1) representa um pixel totalmente opaco.

Assim, uma imagem digital colorida é estruturada como uma matriz multidimensional de dimensões:

- **RGB:** $M \times N \times 3$
- **RGBA:** $M \times N \times 4$

em que cada posição (x, y) armazena as amostras associadas às propriedades ópticas daquela coordenada espacial.

Convenção de escala entre bibliotecas: Diferentes ecossistemas de programação adotam faixas de valores e ordenações distintas para representar os canais digitais, conforme sintetizado na Table 2.1.

Tabela 2.1: Convenções de escala e ordem de canais nas principais bibliotecas de PDI.

Biblioteca	Tipo padrão	Faixa de valores	Ordem das bandas	Forma do array/tensor
OpenCV	uint8	[0, 255]	BGR	$H \times W \times C$
Pillow	uint8	[0, 255]	RGB	$H \times W \times C$
NumPy	uint8/float32	[0, 255] ou [0, 1]	—	$H \times W \times C$
scikit-image	float64	[0, 1]	RGB	$H \times W \times C$
PyTorch	float32	[0, 1]	RGB	$C \times H \times W$
Tensor-Flow/Keras	float32	[0, 1]	RGB	$H \times W \times C$

A conversão entre essas faixas de intensidade é direta e dada por:

$$f_{\text{norm}}(x, y) = \frac{f(x, y)}{255}$$

onde $f(x, y) \in [0, 255]$ e $f_{\text{norm}}(x, y) \in [0, 1]$.

Negligenciar essas discrepâncias é uma fonte frequente de erros em fluxos de trabalho de VC — como, por exemplo, injetar uma imagem lida via OpenCV (padrão BGR, tipo `uint8`) diretamente em um modelo profundo do PyTorch que pressupõe o formato RGB normalizado.

2.4.2 Preparando o Ambiente Prático

O bloco a seguir carrega o módulo `morph.py` do repositório e demonstra, para um pixel sintético, as diferentes convenções de escala e ordem de canais adotadas pelas principais bibliotecas de PDI.

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

version = getattr(morph, "__version__", "local_file")
print(f" Ambiente pronto. Módulo 'morph' carregado (versão: {version}).")

# Demonstração das convenções de escala
img_exemplo = np.array([[200, 100, 50]], dtype=np.uint8) # 1 pixel RGB

print("\n Convenções de escala por biblioteca ")
print(f" NumPy / Pillow / OpenCV (uint8): {img_exemplo[0,0]} → faixa [0, 255]")
print(f" scikit-image / PyTorch (float): {img_exemplo[0,0] / 255.0} → faixa [0.0, 1.0]")
print(f" OpenCV: atenção - lê em BGR: {img_exemplo[0,0,:-1]} (canais invertidos)")
```

Ambiente pronto. Módulo 'morph' carregado (versão: 1.1.0).

```
Convenções de escala por biblioteca
NumPy / Pillow / OpenCV (uint8): [200 100 50] → faixa [0, 255]
scikit-image / PyTorch (float): [0.78431373 0.39215686 0.19607843] → faixa [0.0, 1.0]
OpenCV: atenção - lê em BGR: [ 50 100 200] (canais invertidos)
```

2.4.3 Implementação da Leitura de Imagem em `morph.py`

O trecho a seguir exibe o código-fonte da função `mm.read`, permitindo verificar diretamente como a biblioteca `morph.py` implementa a leitura de imagens.

```
import inspect
print(inspect.getsource(mm.read))
```

```
@staticmethod
def read(file, pil=False, grayscale=False):
    """Lê imagem (local, URL ou Google Drive) → PIL.Image, ndarray 2D ou RGB 3D."""
    import re, requests, numpy as np
    from PIL import Image
    from io import BytesIO
    from urllib.request import urlopen, Request

    # - fonte: URL / Google Drive -
    if isinstance(file, str) and file.startswith(("http://", "https://", "id=")):
```



```

m = re.search(r"id=(\[w-]+\)", file) or re.search(r"/d/(\[w-]+\)", file)
url = f"https://drive.google.com/uc?export=view&id={m.group(1)}" \
    if m and ("id=" in file or "drive.google.com" in file) else file
hdr = {"User-Agent": "Mozilla/5.0 AppleWebKit/537.36 Chrome/124 Safari/537.36"}
try:
    r = requests.get(url, headers=hdr, timeout=20)
    if r.status_code == 429: raise requests.exceptions.HTTPError()
    r.raise_for_status()
    file = BytesIO(r.content)
except:
    file = BytesIO(urlopen(Request(url, headers=hdr), timeout=20).read())

img = Image.open(file)
img.load()
if pil:
    return img
if grayscale:
    return np.array(img.convert("L")) # (H,W)
if img.mode == "L":
    return np.array(img) # (H,W)
if img.mode == "RGBA":
    # fundo branco
    bg = Image.new("RGB", img.size, (255, 255, 255))
    bg.paste(img, mask=img.split()[3])
    return np.array(bg)
return np.array(img.convert("RGB")) # (H,W,3)

```

2.4.4 RGB e RGBA: Exemplo Prático com a Imagem Mandrill

O exemplo a seguir carrega a imagem Mandrill em formato RGB, constrói artificialmente um canal alfa com gradiente horizontal e exibe ambas as representações, ilustrando concretamente as estruturas matriciais $M \times N \times 3$ e $M \times N \times 4$.

```

import os
import numpy as np

# https://commons.wikimedia.org/wiki/File:Mandrill-k-means.png
url = "https://upload.wikimedia.org/wikipedia/commons/a/ab/Mandrill-k-means.png"
caminho = "imagens/mandrill.png"

# Leitura RGB
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_pil = mm.read(url, pil=True)
    mm.write(img_pil, caminho)
else:
    img_pil = mm.read(caminho, pil=True)

img_rgb = np.array(img_pil.convert("RGB")) # shape: (M, N, 3), uint8

# Criação do canal alfa (gradiente horizontal)
h, w, _ = img_rgb.shape
alpha = np.tile(np.linspace(0, 255, w, dtype=np.uint8), (h, 1))
img_rgba = np.dstack([img_rgb, alpha]) # shape: (M, N, 4), uint8

# Diagnóstico
print(f"RGB → shape={img_rgb.shape}, dtype={img_rgb.dtype}, "
      f"faixa=[{img_rgb.min()}, {img_rgb.max()}]")
print(f"RGBA → shape={img_rgba.shape}, dtype={img_rgba.dtype}, "
      f"faixa=[{img_rgba.min()}, {img_rgba.max()}]")
print(f"Pixel (0,0): RGB={img_rgb[0,0]} | RGBA={img_rgba[0,0]}")

# Normalização float32 (padrão scikit-image / PyTorch)
img_rgb_norm = img_rgb.astype(np.float32) / 255.0

```

```
print(f"\nNormalizado (float32): faixa=[{img_rgb_norm.min():.2f}, {img_rgb_norm.max():.2f}]")
print(f"Pixel (0,0) normalizado: {img_rgb_norm[0,0]}")

# Visualização
mm.show(
    [img_rgb, img_rgba],
    title=["RGB - $M \\times N \\times 3$", "RGBA - $M \\times N \\times 4$"],
    cols=2, axis=True
)
```

```
RGB → shape=(512, 512, 3), dtype=uint8, faixa=[14, 248]
RGBA → shape=(512, 512, 4), dtype=uint8, faixa=[0, 255]
Pixel (0,0): RGB=[125 110 58] | RGBA=[125 110 58 0]
```

```
Normalizado (float32): faixa=[0.05, 0.97]
Pixel (0,0) normalizado: [0.49019608 0.43137255 0.22745098]
```

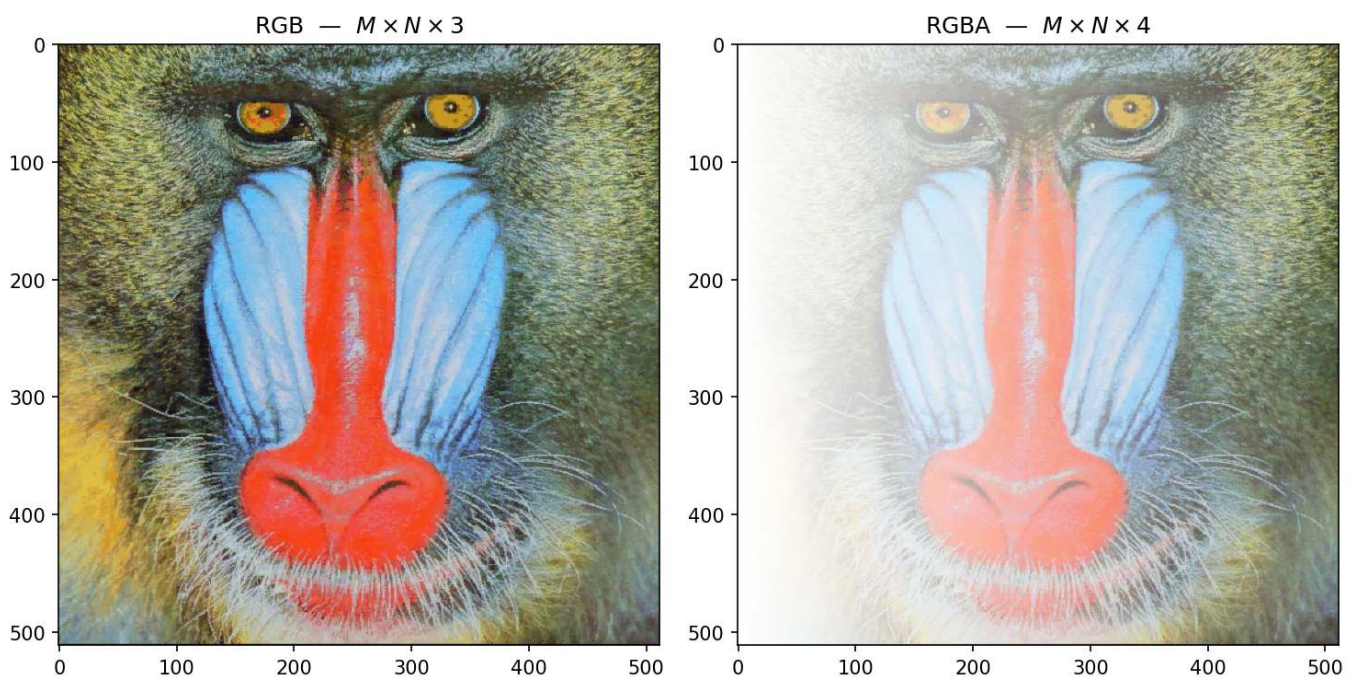


Figura 2.3: Imagem RGB (3 canais, $M \times N \times 3$) e versão RGBA (4 canais, $M \times N \times 4$) com transparência alfa gradual da esquerda para a direita. Imagem Mandrill — [USC SIPI Image Database](#) (domínio público).

2.5 Digitalização: Amostragem e Quantização

Para transformar uma cena contínua em uma imagem digital, dois processos são necessários: amostragem e quantização.

2.5.1 Amostragem - Discretização do espaço

A amostragem consiste em medir o valor da função $f(x, y)$ em pontos igualmente espaçados, formando uma matriz de M linhas (altura) e N colunas (largura). Cada elemento dessa matriz é um **pixel**. A **resolução espacial** é dada por $M \times N$. Quanto maior a resolução, mais detalhes espaciais são preservados, mas maior também o custo computacional e de armazenamento.

2.5.2 Quantização - Discretização da intensidade

A quantização associa a cada pixel um valor numérico discreto, geralmente representado por um inteiro de b bits. A **profundidade de bits** define o número de níveis de intensidade: 2^b . Imagens em tons de cinza

costumam usar 8 bits (256 níveis). Imagens coloridas usam três canais de 8 bits (24 bits no total).

Ilustração: Se usarmos apenas 1 bit por pixel (preto e branco), perdemos todos os tons intermediários. Com 2 bits (4 níveis) já se percebe degradês grosseiros. Com 8 bits, o olho humano dificilmente percebe a discretização (visão contínua).

⚠ Erro de quantização

Erro de quantização é a diferença entre o valor analógico real e o valor discreto atribuído. Ele se manifesta como ruído de quantização, visível em regiões com gradiente suave quando se usa poucos bits.

2.5.3 Efeitos da amostragem e quantização

Os experimentos a seguir mostram como a redução da resolução espacial (subamostragem) e da profundidade de bits degradam a qualidade visual. Use o código para explorar diferentes fatores e níveis de cinza.

```
# Carregar imagem de exemplo
base = "https://upload.wikimedia.org/wikipedia/commons"

arquivo = (
    "Area_de_Prote%C3%A7%C3%A3o_Ambiental_"
    "Quilombos_do_M%C3%A9dio_Ribeira_-_Thomas-"
    "Fuhrmann_%282023-02%29_Malacoptila_striata.jpg"
)

url = f"{base}/c/c5/{arquivo}"
caminho = "imagens/barbudo-rajado.jpg"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_pil = mm.read(url, pil=True) # retorna PIL Image com EXIF
    mm.write(img_pil, caminho)       # salva preservando EXIF
else:
    img_pil = mm.read(caminho, pil=True)

img_numpy = np.array(img_pil)

exif = img_pil._getexif() # agora funciona - img_pil é PIL Image

img_color = mm.read(caminho)
img_gray0 = mm.gray(img_color)
print(f"Imagem original: {img_color.shape}")
print(f"Tipo da imagem: {type(img_color)}")
```

```
Imagem original: (3067, 2047, 3)
Tipo da imagem: <class 'numpy.ndarray'>
```

O experimento apresentado na Figure 2.4 ilustra o compromisso entre a **resolução espacial** e o **custo de armazenamento** em memória. O código utiliza a técnica de subamostragem por fatiamento (*slicing*) para reduzir a matriz original de pixels conforme um fator f , resultando em uma economia de memória — por exemplo, um fator $f = 8$ reduz o tamanho do dado em 64 vezes (8^2). Para fins de comparação visual, as imagens reduzidas são restauradas às dimensões originais (512×512) através da interpolação por **vizinho mais próximo** (*nearest*). Este processo não recupera a informação perdida, mas torna evidente o efeito de **aliasing** e a estrutura de blocos (pixelização) gerada pela baixa densidade de dados da matriz amostrada.

```
def subsample_simple(image, f):
    # Subamostragem via fatiamento (slicing)
    reduced = image[::f, ::f]
```

```
# Cálculo de memória em KB
mem_kb = reduced.nbytes / 1024
label = f"{reduced.shape[1]}x{reduced.shape[0]}, {int(mem_kb)} KB\n(Fator {f})"

# Restaura o tamanho para visualização (H, W originais)
res = mm.resize(reduced, (image.shape[1], image.shape[0]), method='nearest')
return res, label

factors = [1, 4, 8, 12]
img_gray = img_gray0[820:1850, 890:1550] # Recorte para melhor visualização dos detalhes

# Gera os resultados e separa em listas para o mm.show
results = [subsample_simple(img_gray, f) for f in factors]
imgs_list = [r[0] for r in results]
titles_list = [r[1] for r in results]

mm.show(imgs_list, titles=titles_list, cols=4, figsize=(16, 12))
```

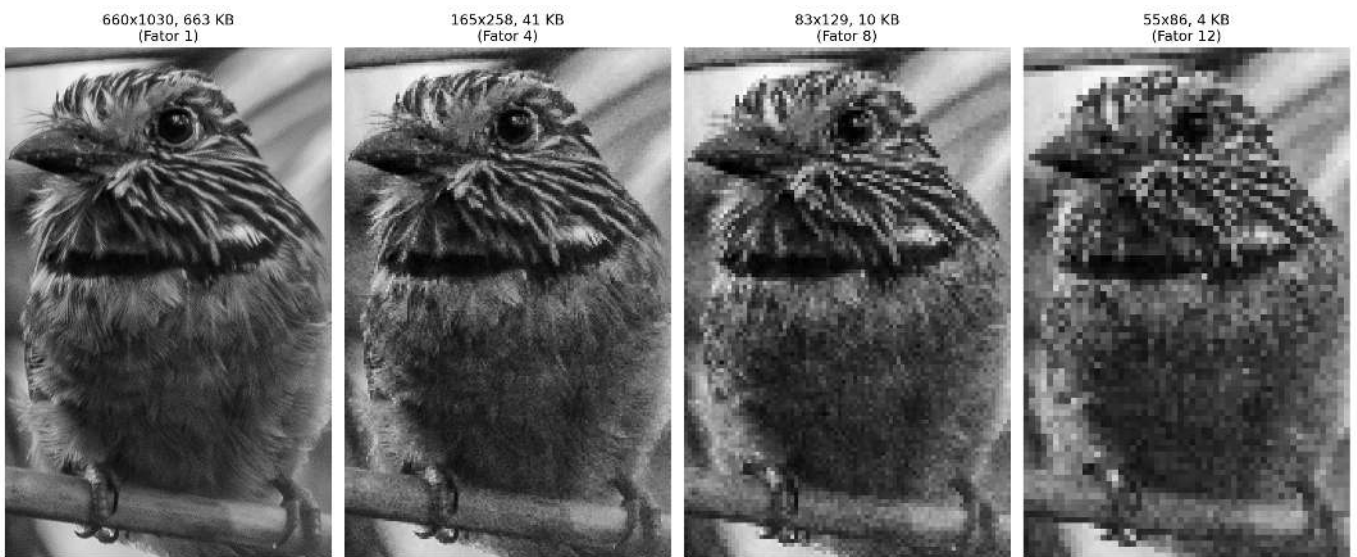


Figura 2.4: Efeito da subamostragem. Os títulos exibem as dimensões (W x H) e o tamanho da matriz em memória (KB).

O experimento na Figure 2.5 foca na **quantização de intensidade**, o processo de discretização da amplitude da função $f(x, y)$. Enquanto a subamostragem afeta a grade espacial, a redução da profundidade de bits limita a quantidade de tons de cinza disponíveis para representar o brilho.

Ao reduzir a profundidade de 8 bits (256 níveis) para valores menores, surge o efeito de **posterização**, onde os gradientes suaves de uma cena são substituídos por transições abruptas. No limite de 1 bit, a imagem torna-se estritamente binária, preservando apenas a silhueta e perdendo detalhes de textura e volume.

```
def quantize_simple(image, bits):
    """Reduz a profundidade de bits e calcula metadados de memória."""
    levels = 2 ** bits
    # Normalização e quantização uniforme
    quantized = (np.floor(image / 256 * levels) / levels * 255).astype(np.uint8)

    # Cálculo de memória em KB
    mem_kb = quantized.nbytes / 1024
    label = f"{bits} bits ({levels} níveis)\n{int(mem_kb)} KB"

    return quantized, label
```

```
# Lista de bits para teste (8 é o padrão, 1 é o binário)
bits_test = [8, 4, 2, 1]

# Gera os resultados e separa em listas para o mm.show
results_q = [quantize_simple(img_gray, b) for b in bits_test]
imgs_q = [r[0] for r in results_q]
titles_q = [r[1] for r in results_q]

mm.show(imgs_q, titles=titles_q, cols=4, figsize=(16, 12))
```

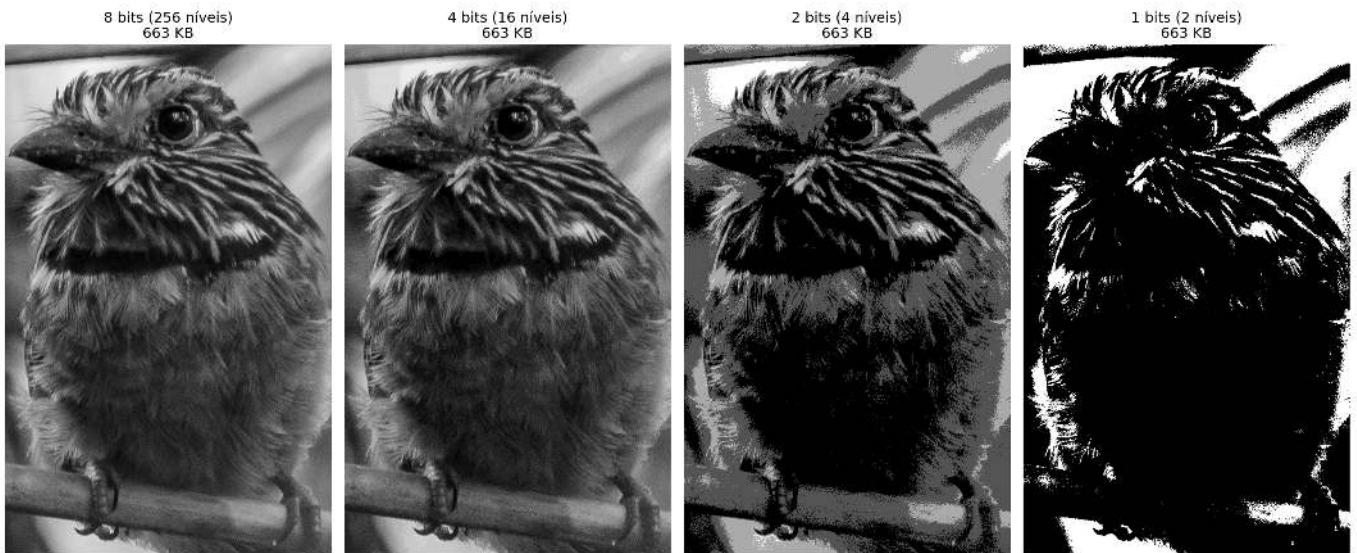


Figura 2.5: Efeito da redução da profundidade de bits. Os títulos exibem a quantidade de bits, níveis e o tamanho em memória (KB).

2.5.4 Análise Técnica

- **Domínio vs. Codomínio:** Note que a resolução espacial (dimensões da matriz) permanece constante em 512x512; o que se altera é apenas o codomínio da função da imagem.
- **Constância de Memória:** Observe nos títulos que o tamanho em KB não diminui. Isso ocorre porque o NumPy armazena cada pixel quantizado em um contêiner de 8 bits (`uint8`), independentemente de o valor real ser apenas 0 ou 1.
- **Percepção:** A degradação visual torna-se crítica abaixo de 4 bits, onde o olho humano começa a perceber as “fronteiras” artificiais criadas pela falta de tons intermediários.

A limitação de tipos de dados menores que um byte no ecossistema Python/NumPy deve-se à arquitetura de hardware, que endereça a memória em blocos de 8 bits (Bytes). Para que se mantenha a compatibilidade com o OpenCV e se garanta a eficiência, mapeiam-se até mesmo elementos binários para contêineres de 1 byte (`uint8` ou `bool8`).

Embora linguagens como ANSI C permitam a compactação de 8 pixels por byte (*bit-packing*), tal abordagem exige a descompactação constante para cálculos e impõe alta complexidade na manipulação de ponteiros. Conforme a Table 2.2, privilegia-se o uso de `uint8` pela facilidade no acesso a vizinhos e pela versatilidade em transformações geométricas. Além disso, métodos nativos do NumPy e OpenCV executam o processamento internamente em baixo nível (C/C++), o que torna operações vetorizadas mais rápidas do que implementações manuais com laços aninhados em Python.

Tabela 2.2: Comparativo entre estratégias de compactação e eficiência de processamento.

Característica	Python (NumPy/OpenCV)	ANSI C (Bit-packing)
Menor Unidade	1 Byte (8 bits)	1 Bit
Memória (Binária)	256 KB (para 512x512)	32 KB (para 512x512)
Velocidade	Alta (Vetorização em C)	Variável (Lenta se houver bit-shift)
Complexidade	Baixa: Métodos prontos	Alta: Ponteiros e Máscaras

2.6 Relações entre Pixels - Topologia da Imagem

Os pixels não são elementos isolados; suas posições relativas definem importantes conceitos para processamento.

2.6.1 Vizinhança

Dado um pixel de coordenadas (x, y) , definem-se dois tipos principais de vizinhança (para imagens em *grid* retangular):

- **Vizinhança-4** (von Neumann): inclui os pixels nas posições $(x-1, y)$, $(x+1, y)$, $(x, y-1)$, $(x, y+1)$.
- **Vizinhança-8** (Moore): inclui todos os oito pixels adjacentes (acrescenta os quatro diagonais).

A escolha da vizinhança influencia operações como detecção de bordas, cálculo de gradientes e conectividade.

```
# # Criação de uma matriz 3x3 para exemplo topológico
# viz = np.zeros((3, 3), dtype='uint8')

# # Definindo Vizinhança-4 (N4) com valor diferente para destaque
# viz[0, 1] = viz[2, 1] = viz[1, 0] = viz[1, 2] = viz[1, 1] = 255

# ou simplesmente (teste com números como argumentos):
viz = mm.secross()

# Exibição da matriz para análise de coordenadas
mm.drawImgPlt(viz, scale=40)
```

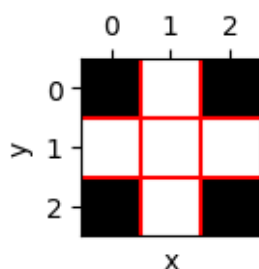


Figura 2.6: Ilustração de vizinhanças 4 em uma matriz 3x3. No centro (1,1), o pixel de interesse.

```
import morph
importlib.reload(morph)
from morph import mm
```

2.6.2 Adjacência, conectividade e caminhos

Dois pixels são **adjacentes** se estão em contato segundo uma vizinhança definida e satisfazem um critério de valor (ex.: mesmo nível de intensidade). Uma **conectividade** define uma relação de equivalência entre pixels que formam uma região conexa. Um **caminho** é uma sequência de pixels adjacentes.

A **conectividade-4** (N4) e **conectividade-8** (N8) podem produzir resultados diferentes na segmentação e no cálculo de componentes conexas (etiquetagem). Por exemplo, um padrão de tabuleiro de xadrez pode ser completamente desconexo em N4, mas totalmente conexo em N8.

2.6.3 Distâncias entre pixels

As métricas de distância são fundamentais para quantificar a proximidade física e a conectividade entre os elementos que compõem a grade digital. Conforme demonstrado na Table 2.3, a escolha da métrica define o custo de deslocamento entre pixels e altera o comportamento de algoritmos de segmentação e análise morfológica.

Para medir a distância entre dois pixels $p(x_1, y_1)$ e $q(x_2, y_2)$, utilizam-se diferentes funções métricas que impõem restrições de movimento distintas sobre o *grid*:

Tabela 2.3: Comparativo de métricas de distância aplicadas à malha de pixels.

Métrica	Definição	Interpretação
Euclidiana	$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$	Distância exata em linha reta (contínua)
Manhattan (<i>City block</i>)	$ x_1 - x_2 + y_1 - y_2 $	Movimentos horizontais + verticais
Chebyshev (<i>Tabuleiro</i>)	$\max(x_1 - x_2 , y_1 - y_2)$	Maior deslocamento entre os eixos

Essas distâncias são aplicadas em diversos contextos de PDI, incluindo algoritmos de interpolação geométrica, transformadas de distância, crescimento de regiões e análise de formas.

i Exemplo prático

Considerando dois pixels com deslocamentos relativos $\Delta x = 3$ e $\Delta y = 4$:

- **Euclidiana**: $\sqrt{3^2 + 4^2} = 5$ (hipotenusa do triângulo retângulo).
- **Manhattan**: $3 + 4 = 7$ (soma dos catetos).
- **Chebyshev**: $\max(3, 4) = 4$ (predomínio do maior deslocamento).

2.7 Armazenamento de Imagens

A escolha do formato de arquivo é um passo decisivo no fluxo de processamento, pois determina como os dados de amostragem e quantização serão preservados ou descartados. Conforme se apresenta na Table 2.4, cada extensão equilibra de forma distinta a fidelidade dos dados e a eficiência de armazenamento.

Tabela 2.4: Principais formatos de armazenamento de imagens digitais e suas aplicações em PDI.

Formato	Características	Uso típico
PGM	Formato simples de mapa de cinzas (texto ou binário).	Pesquisa acadêmica e ferramentas Unix.
BMP	Não comprimido (ou compressão simples).	Windows, aplicações legado.
PNG	Compressão sem perdas (<i>lossless</i>).	Web, imagens com transparência.
JPEG	Compressão com perdas (<i>lossy</i>), ideal para fotografias.	Fotos, câmeras digitais.
TIFF	Suporta múltiplas camadas e compressão variada.	Editoração, arquivamento.
RAW	Dados brutos do sensor, sem processamento.	Fotografia profissional.

Formato	Características	Uso típico
DICOM	Padrão médico com metadados clínicos embutidos (paciente, equipamento, protocolo).	Radiologia, tomografia, ressonância magnética.

Os metadados de uma imagem incluem parâmetros como largura, altura, profundidade de bits e codificação de cor. Em formatos científicos, preservam-se também informações de calibração e detalhes da captura. Ao utilizar-se a função `mm.read()`, a biblioteca `morph` preserva automaticamente esses dados para que se respeitem as propriedades originais da imagem.

Em contextos científicos e hospitalares, privilegia-se o padrão **DICOM** (*Digital Imaging and Communications in Medicine*) para garantir que não haja perda de precisão diagnóstica. Repositórios públicos como o [The Cancer Imaging Archive \(TCIA\)](#), o [Alzheimer’s Disease Neuroimaging Initiative \(ADNI\)](#) e o [PhysioNet](#) disponibilizam vastos conjuntos de dados neste formato, incluindo metadados clínicos anonimizados que são essenciais para a investigação científica.

2.7.1 Exemplo: Extração de Metadados e Localização GPS

Diferente da matriz de pixels pura obtida pela leitura convencional — como na imagem do pássaro apresentada no início deste capítulo —, o uso do argumento `pil=True` no método `mm.read()` altera a natureza do objeto retornado (ver Figure 2.7). Enquanto o padrão (`pil=False`) retorna um `numpy.ndarray` RGB, a leitura com `pil=True` retorna um objeto especializado da biblioteca Pillow, capaz de interpretar o cabeçalho **EXIF**.

O cabeçalho **EXIF** (*Exchangeable Image File Format*) funciona como um repositório técnico da captura, permitindo que o objeto Pillow interprete uma vasta gama de informações que vão muito além das coordenadas de GPS. Ao utilizar `pil=True`, o sistema ganha acesso ao “DNA” da imagem, incluindo metadados de hardware (marca e modelo da câmera), configurações ópticas (abertura, distância focal e tempo de exposição) e parâmetros de iluminação (uso de flash e balanço de branco).

Essa distinção é importante no PDI, pois transforma a matriz de amostragem em um conjunto de dados contextualizado, onde as características físicas do sensor e da lente podem ser utilizadas para normalizar brilhos ou corrigir distorções geométricas.

```
from PIL.ExifTags import TAGS
import os, numpy as np

base = "https://upload.wikimedia.org/wikipedia/commons"
arquivo = (
    "Area_de_Prote%C3%A7%C3%A3o_Ambiental_"
    "Quilombos_do_M%C3%A9dio_Ribeira_-_Thomas-"
    "Fuhrmann_%282023-02%29_Malacoptila_striata.jpg"
)
url = f"{base}/c/c5/{arquivo}"
caminho = "imagens/barbudo-rajado.jpg"

# 1. Leitura - preserva objeto PIL com EXIF
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho) # salva preservando EXIF
else:
    img_obj = mm.read(caminho, pil=True)

img_numpy = np.array(img_obj) # conversão para NumPy

# 2. Extração e conversão de GPS (Tag 34853)
exif = img_obj._getexif()
```



```

if exif and (gps := exif.get(34853)):
    to_dec = lambda dms, ref: float(-(dms[0]+dms[1]/60+dms[2]/3600) if ref in 'SW'
                                   else (dms[0]+dms[1]/60+dms[2]/3600))
    lat, lon = to_dec(gps[2], gps[1]), to_dec(gps[4], gps[3])
    print(f"GPS Decimal: {lat:.6f}, {lon:.6f}")
    print(f"Maps: https://www.google.com/maps/search/?api=1&query={lat},{lon}")

# 3. Diagnóstico de tipos, dimensões e acesso a pixels
print(f"\nTipo PIL : {type(img_obj)} | Dimensões (x,y): {img_obj.size}")
print(f"Pillow (0,0): {img_obj.getpixel((0, 0))}")
print(f"Tipo NumPy: {type(img_numpy)} | Dimensões [y,x,c]: {img_numpy.shape}")
print(f"NumPy [0,0]: {img_numpy[0, 0]}")

# 4. Exibição
mm.show(img_numpy, scale=30)

```

```

GPS Decimal: -24.587955, -48.629758
Maps: https://www.google.com/maps/search/?api=1&query=-24.587955,-48.629758333333335

Tipo PIL : <class 'PIL.JpegImagePlugin.JpegImageFile'> | Dimensões (x,y): (2047, 3067)
Pillow (0,0): (58, 96, 0)
Tipo NumPy: <class 'numpy.ndarray'> | Dimensões [y,x,c]: (3067, 2047, 3)
NumPy [0,0]: [58 96  0]

```

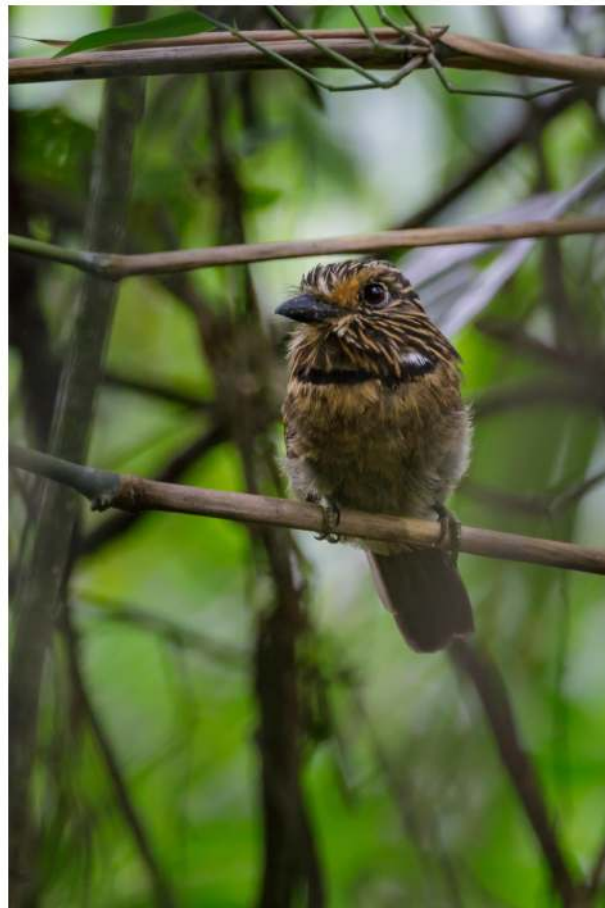


Figura 2.7: Área de Proteção Ambiental Quilombos do Médio Ribeira - Barbudo-rajado (*Malacoptila striata*). Crédito: Thomas Fuhrmann (CC BY-SA 4.0).

i Nota Pedagógica: A sutil diferença das dimensões

Repare que a representação das dimensões muda conforme a estrutura de dados utilizada:

- **No Pillow (.size):** Retorna (Largura, Altura) — no exemplo: (2047, 3067). É uma visão orientada ao arquivo de imagem.
- **No NumPy (.shape):** Segue a convenção matemática de matrizes: (Linhas/Altura, Colunas/Largura, Canais) — no exemplo: (3067, 2047, 3).

Esta distinção é fundamental para evitar erros de indexação ao implementar filtros manuais. Enquanto o objeto Pillow carrega o “**onde**” e o “**quando**” (contexto), o array NumPy carrega o “**quanto**” de luz (intensidade) existe em cada ponto da imagem.

2.7.2 Por que esta separação é importante?

Ao carregar uma imagem via `cv2.imread()`, o resultado é um `<class 'numpy.ndarray'>`, que contém estritamente os valores numéricos resultantes da **quantização** e **amostragem**. No entanto, ao utilizar `pil=True`, o `mm.read()` retorna um objeto da classe `PIL.JpegImagePlugin.JpegImageFile`.

Esta classe mantém o arquivo “aberto” para permitir o acesso ao contexto da captura antes que os dados sejam convertidos em uma matriz bruta. Note que o pixel (0, 0) é idêntico nas duas representações — (58, 96, 0) no Pillow e [58, 96, 0] no NumPy —, confirmando que ambos descrevem os mesmos dados, apenas com interfaces distintas. Essa separação é vital: pixels servem para algoritmos; metadados servem para georreferenciamento, catalogação científica e correções baseadas no hardware de aquisição.

Para inspecionar todos os metadados EXIF de um objeto Pillow:

```
from PIL import ExifTags

exif_raw = img_obj._getexif()
if exif_raw:
    for tag_id, valor in sorted(exif_raw.items()):
        tag_nome = ExifTags.TAGS.get(tag_id, f"TAG_{tag_id}")
        print(f" {tag_nome:40s} : {valor}")
```

2.8 Transformações Geométricas Básicas

As transformações geométricas alteram a posição dos pixels, mantendo os valores de intensidade. São fundamentais para alinhamento, correção de distorções e aumento de dados (*data augmentation*) em aprendizado de máquina.

Uma **transformação afim** é qualquer mapeamento que preserve colinearidade (pontos sobre uma reta permanecem sobre uma reta) e razões de distâncias entre pontos colineares. Em 2D, toda transformação afim pode ser expressa em **coordenadas homogêneas** por uma matriz 3×3 :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{T}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (2.2)$$

A sub-matriz 2×2 superior esquerda $\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$ controla rotação, escala e cisalhamento; o vetor $(t_x, t_y)^\top$ controla a translação. As transformações geométricas mais usadas em PDI — translação, rotação e escala — são casos particulares de $\mathbf{T}$, e podem ser **compostas** por multiplicação de matrizes, na ordem $\mathbf{T} = \mathbf{T}_n \cdots \mathbf{T}_2 \mathbf{T}_1$.

i Transformação inversa e interpolação

Na implementação prática (`cv2.warpAffine`), aplica-se a **transformação inversa**: para cada pixel (x', y') da imagem destino, calcula-se a posição de origem $(x, y) = \mathbf{T}^{-1}(x', y')$ e interpola-se o valor. Isso evita buracos na imagem resultante causados pelo mapeamento direto de pixels inteiros para posições não inteiras.

2.8.1 Translação

A translação é a transformação afim mais simples: desloca todos os pixels por um vetor (t_x, t_y) . Em coordenadas homogêneas, é expressa pela matriz:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow \begin{cases} x' = x + t_x \\ y' = y + t_y \end{cases} \quad (2.3)$$

A terceira linha da matriz garante que a operação permaneça no espaço afim, permitindo que translação, rotação e escala sejam combinadas por simples multiplicação de matrizes. Na prática, `cv2.warpAffine` usa apenas as duas primeiras linhas (matriz 2×3), pois a terceira é sempre $[0, 0, 1]$.

Pixels deslocados para além da área original são descartados; áreas descobertas são preenchidas com 0 (preto). Ver um exemplo na Figure 2.8.

```
img_tx1 = mm.translate(img_gray, 50, 50)
img_tx2 = mm.translate(img_gray, 100, 50)
mm.show([img_gray, img_tx1, img_tx2],
        titles=["Original", "Translação (50,50)", "Translação (100,50)"],
        cols=3, figsize=(16, 12))
```



Figura 2.8: Exemplo de translação da imagem do pássaro com deslocamentos (50,50) e (100,50).

2.8.2 Rotação

A rotação por ângulo θ em torno de um ponto central (c_x, c_y) é composta por três transformações afins: translação para a origem, rotação pura e translação de volta. A matriz resultante é:

$$\mathbf{T}_{\text{rot}} = \begin{bmatrix} \cos \theta & -\sin \theta & c_x(1 - \cos \theta) + c_y \sin \theta \\ \sin \theta & \cos \theta & c_y(1 - \cos \theta) - c_x \sin \theta \\ 0 & 0 & 1 \end{bmatrix} \quad (2.4)$$

Na implementação, `cv2.getRotationMatrix2D` gera diretamente as duas primeiras linhas de $\mathbf{T}_{\text{rot}}$ (matriz 2×3 para `warpAffine`), aceitando também um fator de escala s que multiplica $\cos \theta$ e $\sin \theta$. Ver exemplo na Figure 2.9.

Como a rotação desloca pixels para novas posições não-inteiras, `cv2.warpAffine` precisa estimar a cor de cada pixel de destino a partir dos vizinhos — processo chamado **interpolação**. O parâmetro `interp` controla esse comportamento:

- **nearest** (`INTER_NEAREST`): atribui o valor do pixel mais próximo. Rápido, mas produz serrilhado (*aliasing*) em bordas diagonais.
- **bilinear** (`INTER_LINEAR`, padrão): média ponderada dos 4 vizinhos mais próximos. Equilibra qualidade e desempenho — adequado para a maioria dos casos.
- **bicubic** (`INTER_CUBIC`): considera os 16 vizinhos em uma superfície cúbica. Produz bordas mais suaves, ao custo de maior processamento.

```
img_rot30 = mm.rotate(img_gray, 30, interp='bilinear')
img_rot45 = mm.rotate(img_gray, 45, interp='bilinear')

mm.show([img_gray, img_rot30, img_rot45],
        titles=["Original", "Rotação 30°", "Rotação 45°"],
        cols=3, figsize=(16, 12))
```



Figura 2.9: Exemplo de rotação da imagem do pássaro em 30° e 45° usando interpolação bilinear.

2.8.3 Escala (Redimensionamento)

O redimensionamento por fatores (s_x, s_y) é uma transformação afim com matriz:

$$\mathbf{T}_{\text{escala}} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.5)$$

Quando $s > 1$ (ampliação), pixels da imagem destino mapeiam para posições não inteiras na origem — exigindo interpolação para estimar o valor. Quando $s < 1$ (redução), múltiplos pixels de origem contribuem para um único pixel destino — exigindo **decimação**. Os mesmos três métodos descritos na rotação estão disponíveis em `mm.resize`, com a diferença de que aqui o impacto visual é mais perceptível: na ampliação, **nearest** produz efeito de blocos (*pixelation*), enquanto **bicubic** preserva melhor a nitidez das bordas, conforme a Table 2.5:

Tabela 2.5: Métodos de interpolação disponíveis em `mm.resize` e seus respectivos números de vizinhos utilizados no cálculo.

Método	Vizinhos usados	Característica
'nearest'	1	Rápido; produz efeito de blocos (<i>pixelation</i>)
'bilinear'	4	Bom compromisso qualidade/custo; bordas suaves
'bicubic'	16	Maior nitidez; preferido em softwares profissionais

O parâmetro `size_or_factor` aceita tanto um escalar (fator uniforme, e.g. 0.5 para reduzir à metade) quanto uma tupla (largura, altura) para dimensões absolutas.

```
# 1. Recorte da região do bico
y, x, offset = 210, 40, 40
crop = img_gray[y-offset:y+offset, x-offset:x+offset]
print(f"Imagem: {img_gray.shape} | Crop: {crop.shape}")

# 2. Ampliar 4× com mm.resize
crop_nearest = mm.resize(crop, size_or_factor=4, method='nearest')
crop_bilinear = mm.resize(crop, size_or_factor=4, method='bilinear')

# 3. Exibição comparativa
mm.show(
    [crop, crop_nearest, crop_bilinear],
    titles=["Original (recorte)", "Vizinho mais próximo (4×)", "Bilinear (4×)"],
    cols=3, figsize=(16, 12), dpi=200
)
```

Imagem: (1030, 660) | Crop: (80, 80)



Figura 2.10: Comparação de interpolação com zoom no detalhe do olho (recorte 60×60, ampliado 4×). Note o efeito de blocos no vizinho mais próximo vs. a suavização na bilinear.

2.8.4 Cisalhamento (*Shear*)

O cisalhamento é uma transformação afim que distorce a imagem ao deslocar cada pixel proporcionalmente à sua posição em um eixo. A matriz geral combina cisalhamento horizontal (sh_x) e vertical (sh_y):

$$\mathbf{T}_{\text{cisalh}} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.6)$$

Para $sh_x \neq 0$ e $sh_y = 0$, cada linha é deslocada horizontalmente proporcionalmente à sua posição vertical — produzindo o efeito de “inclinação” característico. Ver exemplo na Figure 2.11.

```
img_shx = mm.shear(img_gray, shx=0.3)
img_shy = mm.shear(img_gray, shy=0.3)
img_shc = mm.shear(img_gray, shx=0.2, shy=0.2)

mm.show(
    [img_gray, img_shx, img_shy, img_shc],
    titles=["Original", "Horiz. (shx=0.3)", "Vert. (shy=0.3)", "Combinado (0.2, 0.2)"],
    cols=4, figsize=(16, 12)
)
```

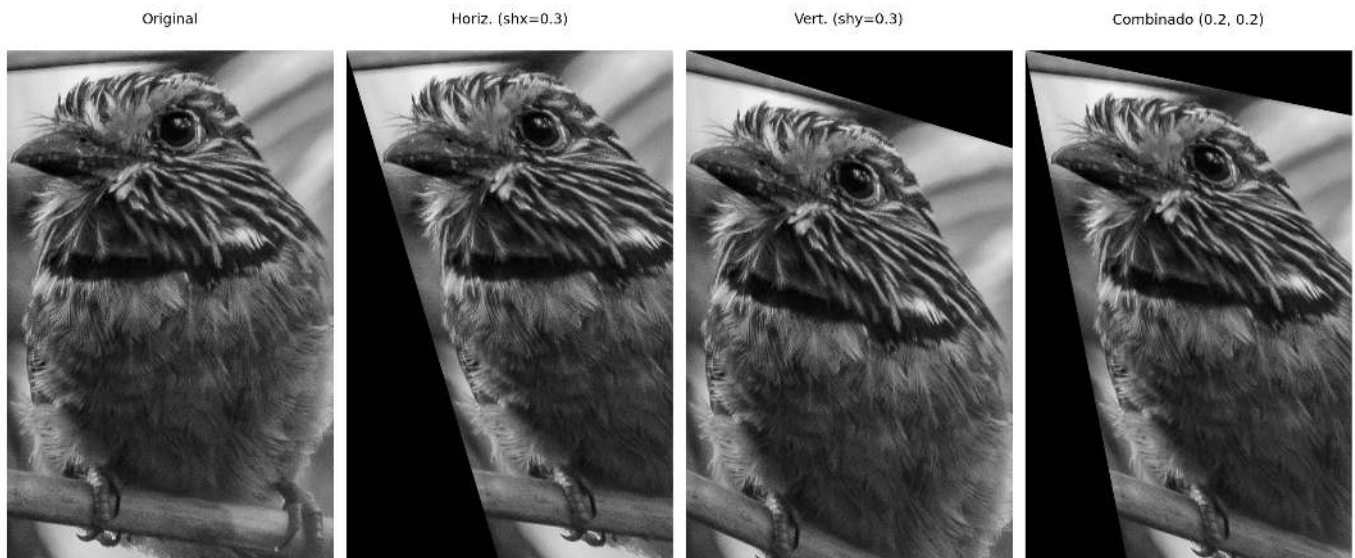



Figura 2.11: Exemplo de cisalhamento da imagem do pássaro: horizontal ($shx=0.3$), vertical ($shy=0.3$) e combinado ($shx=0.2$, $shy=0.2$).

2.9 Resumo

Neste capítulo foram apresentados os fundamentos de digitalização e topologia de imagens:

- **Formação da imagem:** $f(x, y) = i(x, y) \cdot r(x, y)$.
- **Amostragem:** discretização do espaço $\rightarrow$ resolução espacial $M \times N$.
- **Quantização:** discretização da intensidade $\rightarrow$ profundidade de bits b .
- **Relações topológicas:** vizinhança-4, vizinhança-8, conectividade, distâncias (Euclidiana, Manhattan, Chebyshev).
- **Transformações geométricas:** translação, rotação, escala (com interpolação bilinear ou vizinho mais próximo).
- **Formatos de arquivo:** BMP, PNG, JPEG, TIFF, RAW; cada um com diferentes compromissos entre qualidade e tamanho.

O Capítulo 3 abordará **operações espaciais** como convolução, filtragem e morfologia matemática (erosão, dilatação).

2.10 Uso do NotebookLM como Tutor Complementar

Nesta edição, incentivamos o uso do **NotebookLM** como ferramenta complementar de aprendizagem. Essa ferramenta de IA utiliza exclusivamente os documentos fornecidos pelo autor como base de conhecimento, garantindo respostas coerentes com o conteúdo do livro.

Para cada capítulo, preparamos um projeto específico na plataforma. Para uma experiência de estudo ampliada, utilize o acesso abaixo:

Estude com o Tutor Inteligente

Para interagir com o conteúdo deste capítulo, acesse o link a seguir. O ambiente contém materiais didáticos em diferentes formatos, gerados a partir do **PDF** do capítulo. Na plataforma, explore especialmente as opções **Guia de Estudo** e **Conversa** para aprofundar sua compreensão.

 [ACESSAR NOTEBOOKLM: CAPÍTULO 02](#)

Aviso sobre Conteúdo Gerado por IA

A IA é uma poderosa aliada nos estudos, mas o conteúdo gerado pode conter **erros ou imprecisões**. Consulte sempre **livros, artigos científicos e outras fontes acadêmicas confiáveis** para validar as informações. Sempre que possível, execute os exemplos práticos fornecidos neste capítulo para verificar os resultados.

2.11 Lista de Exercícios

1. (15%) Explique, com suas próprias palavras, a diferença entre **amostragem** e **quantização**. Dê um exemplo concreto de cada uma no contexto de uma imagem digital.
2. (15%) Considere uma imagem com resolução espacial de 1024×768 pixels e profundidade de 24 bits (8 bits por canal RGB). Calcule o tamanho total não comprimido da imagem em bytes e em megabytes.
3. (20%) Utilizando o código do laboratório, modifique o fator de subamostragem para 3 e para 6. Descreva visualmente o que ocorre com as bordas dos objetos. O que é o efeito de *aliasing*?
4. (20%) Para a imagem em tons de cinza, aplique quantização com 3 bits (8 níveis) e 5 bits (32 níveis). Compare os resultados e explique por que 5 bits já pode ser considerado suficiente para muitas aplicações.
5. (15%) Dados dois pixels $A = (10, 20)$ e $B = (15, 25)$, calcule as distâncias Euclidiana, Manhattan e Chebyshev entre eles.
6. (15%) Usando a função `mm.rotate`, gire a imagem do pássaro em ângulos de 90° , 180° e 270° com interpolação bilinear. Compare com a rotação usando `method='nearest'`. Em quais situações a interpolação vizinho mais próximo ainda é útil?

Referências do Capítulo

A fundamentação teórica deste capítulo baseia-se nas seguintes obras:

- Gonzalez; Woods (2018) para os conceitos de amostragem, quantização e relações entre pixels.
- Szeliski (2022) para transformações geométricas e conectividade.
- Bradski; Kaehler (2008) para a implementação prática com OpenCV e `morph.py`.

 Executar Colab  Abrir si-md2  GitHub

2.12 Parte Prática com Exercícios de Programação

Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.1 | TestSuite: 1.1.0

Executando os Testes

Para rodar os testes, execute `TestSuite("EP04_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (.py, .java, .c, .cpp, .js ou .r). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:

```
codigo = """
from morph import mm
# ... seu código aqui ...
"""
TestSuite("EP04_01").run_code(codigo)
```

2.12.1 EP02_01 ☉ Ajuste de Brilho e Contraste

Nesta atividade, o objetivo é implementar um operador de ponto para **transformação linear de intensidade**, aplicando o ajuste dinâmico de brilho e contraste em uma imagem digital.

2.12.1.1 📋 Diretrizes de Implementação

O algoritmo deve seguir o fluxo de execução abaixo:

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas) da matriz.
2. **Parâmetros:** Ler o valor real α (fator de contraste) e o inteiro β (fator de brilho).
3. **Dados:** Ler os valores inteiros da matriz original.
4. **Maapeamento:** Para cada pixel p , calcular o novo valor p' pela equação:

$$p' = \text{clip}(\text{round}(\alpha \cdot p + \beta))$$

5. **Saída:** Exibir a matriz resultante com as dimensões $L \times C$.

2.12.1.2 📌 Restrições Computacionais

- **Arredondamento (*Round*):** Aplica-se o arredondamento matemático para o inteiro mais próximo antes da conversão de tipo.
- **Saturação (*Clipping*):** Os valores devem ser confinados ao intervalo $[0, 255]$ para preservar o padrão de 8 bits:

$$\text{clip}(x) = \max(0, \min(255, x))$$

- **Simulação:** O efeito dos parâmetros α e β na correção histogramática pode ser observado na Figure 2.12.

2.12.1.3 Fundamentação Teórica

As alterações modificam o histograma da imagem para ajustar o perfil de iluminação e a distinção tonal.

Parâmetro	Função	Impacto Visual
α (Alpha)	Escalar	Modula o Contraste . Se $\alpha > 1$, expande o histograma; se $0 \leq \alpha < 1$, comprime.
β (Beta)	Aditiva	Modula o Brilho . Se positivo, translada o histograma para a direita; se negativo, para a esquerda.
clip	Limitador	Restringe a faixa dinâmica, impedindo erros de <i>underflow</i> e <i>overflow</i> .

2.12.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Valores de **alpha** (α) e **beta** (β).
- Linhas seguintes: Elementos numéricos da matriz original.

Saída:

- Matriz transformada estruturada em L linhas e C colunas.

2.12.1.5 Exemplos

A tabela a seguir apresenta um exemplo prático do comportamento esperado do algoritmo, destacando a atuação dos operadores de arredondamento e saturação.

Entrada	Saída	Observação
1 4 1.5 -30 0 100 180 255	0 120 240 255	Note o efeito de saturação no último pixel

Executando os Testes

Para rodar os testes, execute `TestSuite("EP02_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.



Figura 2.12: Simulador: Ajuste de Brilho e Contraste

```
%%writefile EP02_01.py
# sua solução
```

```
Overwriting EP02_01.py
```

```
TestSuite("EP02_01.py").run()
```

2.12.2 EP02_02 Subamostragem Espacial

Nesta atividade, você deve implementar a redução da resolução espacial de uma imagem através do processo de subamostragem.

- Leia dois inteiros L e C , representando as dimensões da matriz original.
- Leia um valor inteiro f ($f \geq 1$), que representa o fator de amostragem.
- Leia os valores inteiros da matriz original.
- A nova imagem deve ser construída selecionando o pixel da posição $(f \cdot i, f \cdot j)$ da imagem original.
- Imprima a matriz resultante com as novas dimensões.
- Ver na Figure 2.13 uma simulação deste EP.

Importante:

- **Dimensões Finais:** A imagem amostrada terá dimensões $\lceil L/f \rceil \times \lceil C/f \rceil$. No contexto de programação, isso equivale ao tamanho resultante de um fatiamento (slicing) com passo f .
- **Implementação:** Não utilize funções prontas de bibliotecas de processamento de imagens (como OpenCV ou PIL) para o redimensionamento. Implemente a lógica de seleção de pixels manualmente ou via fatiamento de matrizes.
- **Aliasing:** Note que este processo pode causar o efeito de *aliasing* (serrilhamento), onde detalhes finos são perdidos ou padrões indesejados aparecem.

2.12.2.1 🧠 Discretização do Espaço

A subamostragem reduz a resolução espacial de uma imagem, selecionando apenas um pixel a cada f pixels em cada direção. É o processo inverso da interpolação:

Parâmetro	Função	Efeito
Fator f	Salto de amostragem	Define o intervalo de seleção. Um fator 2 reduz a largura e altura pela metade.
Resolução	Densidade de pixels	Diminui a quantidade total de informação espacial da imagem.
Aliasing	Efeito colateral	Surgimento de padrões em escada ou blocos devido à perda de detalhes finos.

2.12.2.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém o fator f .

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz reduzida com as dimensões correspondentes ao fatiamento por f .

2.12.2.3 📌 Exemplos

Entrada	Saída	Observação
2 4 2 10 20 30 40 50 60 70 80	10 30	O fator 2 seleciona os pixels (0,0) e (0,2) da primeira linha. A segunda linha é ignorada.

```
%%writefile EP02_02.py
# Código Python
```

```
Overwriting EP02_02.py
```

```
TestSuite("EP02_02.py").run()
```

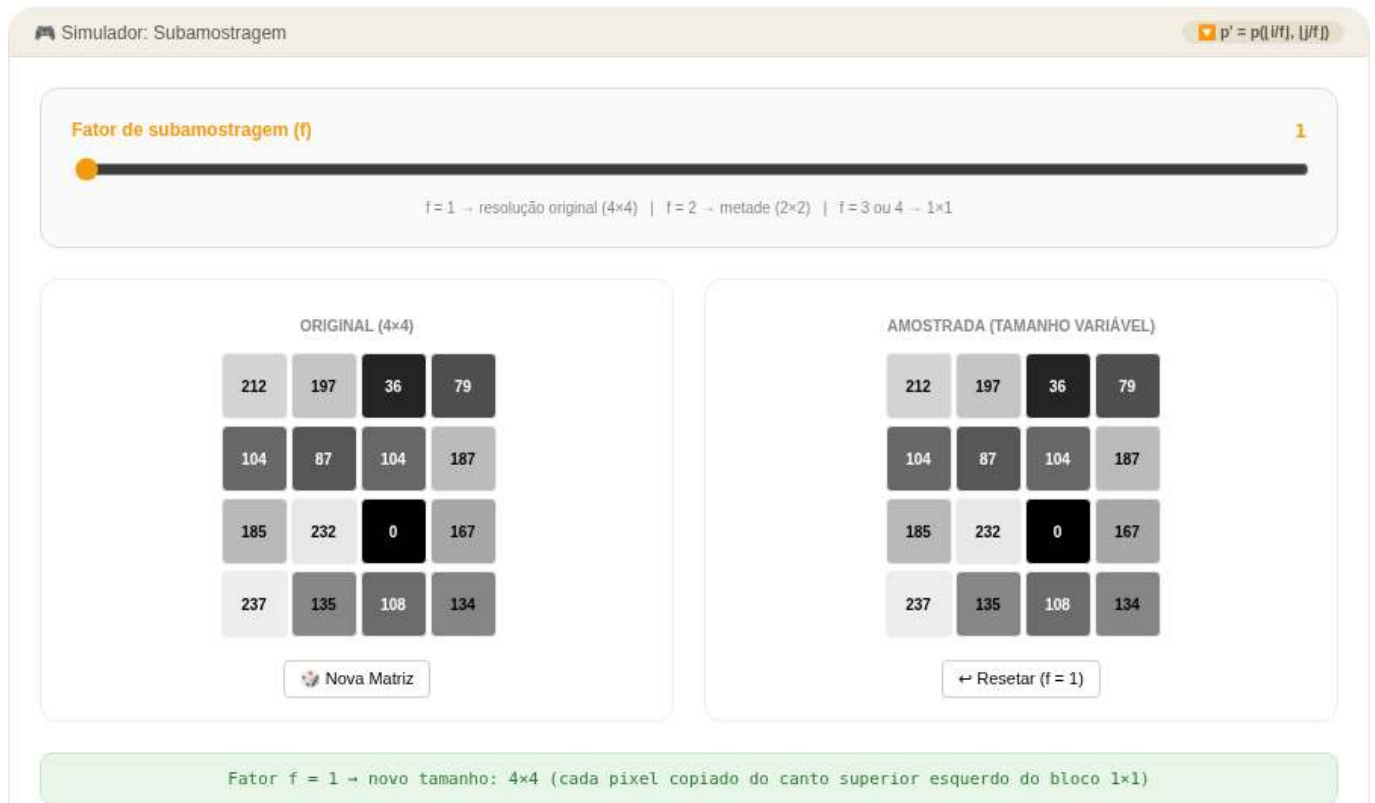



Figura 2.13: Simulador: Subamostragem

2.12.3 EP02_03 🎮 Quantização de Níveis de Cinza

Nesta atividade, você deve implementar a quantização uniforme de uma imagem, reduzindo a quantidade de níveis de intensidade de cinza originais para uma nova escala baseada em um número menor de bits.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia um inteiro k ($1 \leq k \leq 8$), representando o novo número de bits da imagem.
- Calcule o número de níveis ($N = 2^k$) e o tamanho do intervalo (passo).
- Para cada pixel p , calcule o novo valor p' mapeando-o para o índice do nível discretizado correspondente (variando de 0 a $2^k - 1$).
- Imprima a matriz resultante com os mesmos valores de dimensões originais.
- Ver na Figure 2.14 uma simulação deste EP.

📌 Importante:

- **Posterização:** Ao reduzir drasticamente os níveis (ex: $k = 2$), você notará que degradês suaves se transformam em faixas abruptas de cor devido à perda de resolução de amplitude.
- **Cálculo do Passo:** O intervalo entre cada nível é definido por $passo = 256/2^k$.
- **Mapeamento:** O método de quantização uniforme por truncamento que mapeia o pixel para o índice do seu respectivo nível discretizado é dado por:

$$p' = \left\lfloor \frac{p}{passo} \right\rfloor$$

Em termos de implementação (como em Python), isso equivale à divisão inteira: $p' = p // passo$.

2.12.3.1 🧠 Discretização da Amplitude

Enquanto a subamostragem lida com a resolução espacial, a quantização foca na precisão da cor (amplitude). Reduzir os bits significa simplificar a informação cromática:

Parâmetro	Função	Efeito
Bits (k)	Profundidade de cor	Define quantos tons diferentes a imagem pode ter (2^k).
Passo	Intervalo de tom	Espaçamento entre os níveis de cinza permitidos.
Posterização	Fenômeno visual	Transformação de variações contínuas em blocos de cor sólida.

2.12.3.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém o número de bits k .

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz transformada com os índices dos níveis quantizados, mantendo o tamanho original $L \times C$.

2.12.3.3 📌 Exemplos

Entrada	Saída	Observação
1 4 2 0 80 170 255	0 1 2 3	Com $k = 2$, temos $2^2 = 4$ níveis discretos disponíveis (0, 1, 2, 3). O passo é $256/4 = 64$. Aplicando a divisão inteira por elemento: $0//64 = 0$, $80//64 = 1$, $170//64 = 2$, $255//64 = 3$.
1 5 1 10 50 120 200 250	0 0 0 1 1	Com $k = 1$, temos $2^1 = 2$ níveis (0 e 1). Passo = $256/2 = 128$. Pixels menores que 128 resultam em 0, e pixels maiores ou iguais a 128 resultam em 1.

```
%%writefile EP02_03.py
# Código Python
```

```
Overwriting EP02_03.py
```

```
TestSuite("EP02_03.py").run()
```

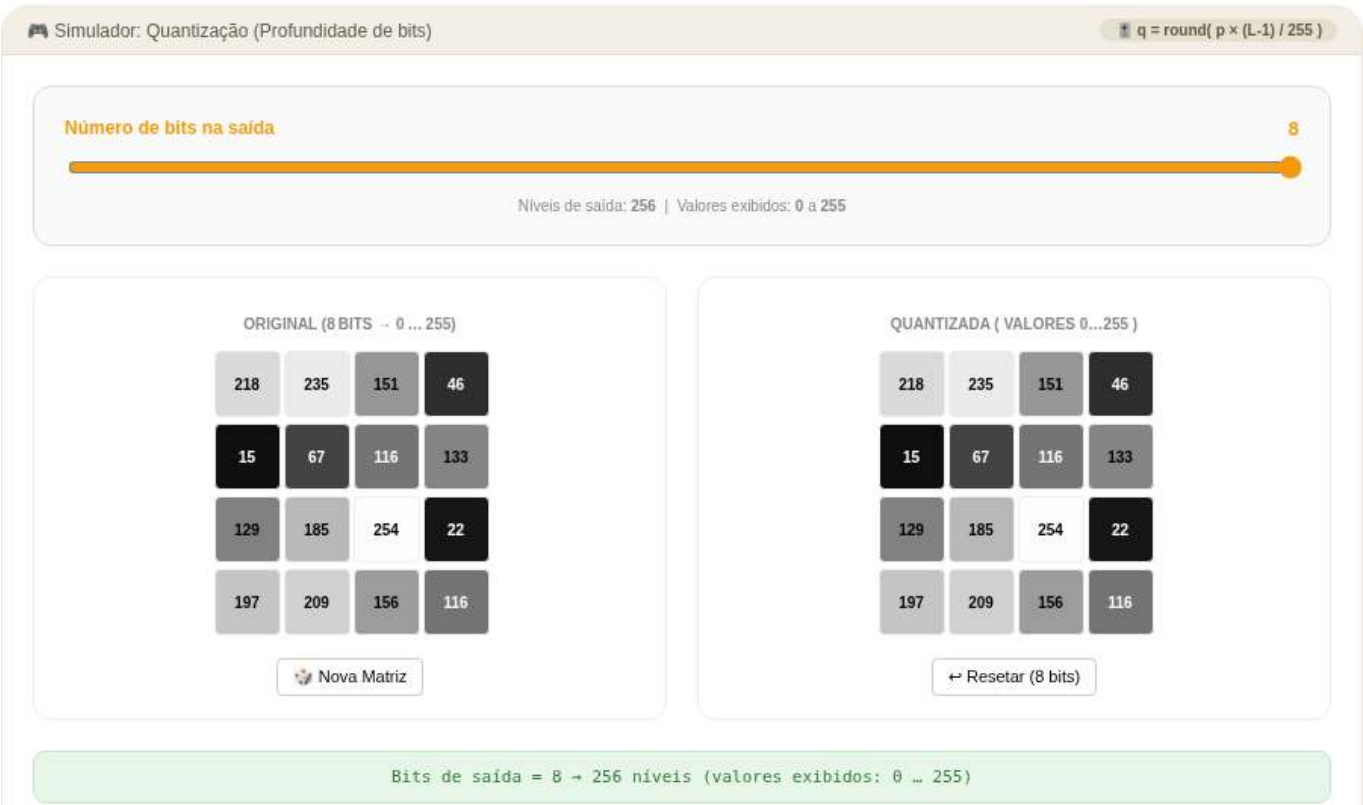


Figura 2.14: Simulador: Quantização

2.12.4 EP02_04 Transformada de Distância em Imagem Binária

Dada uma imagem binária onde pixels de valor **1** representam o *objeto* e pixels **0** representam o *fundo*, a distância de um pixel de fundo recebe a **menor distância ao pixel de objeto mais próximo**. Pixels de objeto recebem distância **0**. Para simplificar, considere que a imagem tem **apenas um único objeto com um pixel de valor 1**.

Problema: Leia uma imagem binária $L \times C$ e uma métrica, e calcule essa distância simplificada aplicando uma das três fórmulas:

$$d_{\text{Euclidiana}} = \sqrt{(\Delta r)^2 + (\Delta c)^2}$$

$$d_{\text{City-block}} = |\Delta r| + |\Delta c|$$

$$d_{\text{Chessboard}} = \max(|\Delta r|, |\Delta c|)$$

onde Δr é a diferença de linhas e Δc a diferença de colunas entre dois pixels.

2.12.4.1 Por que isso importa? - Aplicações da DT

A Transformada de Distância (DT) aparece em dezenas de pipelines de visão computacional:

Métrica	Complexidade	Aplicação típica
Euclidiana	$O(n^2)$ ingênuo	Esqueletização, matching de formas

Métrica	Complexidade	Aplicação típica
City-block	● $O(n)$ com 2 passes	Morfologia, dilatação/erosão
Chessboard	● $O(n)$ com 2 passes	Morfologia, dilatação/erosão

2.12.4.2 📌 Requisitos Técnicos

- **Entrada:** * Primeira linha: L e C (inteiros).
 - Segunda linha: nome da métrica (`euclidean`, `cityblock` ou `chessboard`).
 - Em seguida, a matriz binária $L \times C$ (valores 0 ou 1).
- **Pixels de objeto (1):** distância = 0 (ou 0.00 para euclidiana).
- **Pixels de fundo (0):** distância ao único pixel de objeto na imagem.
- **Arredondamento (euclidiana):** imprimir com **2 casas decimais** (formato `:.2f`). City-block e Chessboard produzem inteiros — imprimir sem decimais.
- **Saída:** valores separados por espaço, uma linha por linha da matriz.
- Ver na Figure 2.15 uma simulação deste EP.

2.12.4.3 📌 Exemplos

Entrada	Saída	Observação
4	2 2 2 2	A distância Chessboard é $\max(\ dx\ , \ dy\)$. O único pixel objeto é $(2, 2) = 0$; os demais armazenam sua distância mínima até ele.
4	2 1 1 1	
chessboard	2 1 0 1	
0 0 0 0	2 1 1 1	
0 0 0 0		
0 0 1 0		
0 0 0 0		

2.12.4.4 📌 Observações finais

- Como a imagem tem **apenas um objeto de um pixel**, a distância de cada pixel de fundo é simplesmente a distância desse pixel ao único ponto objeto.
- A implementação pode usar força bruta (percorrer todos os pixels da imagem e calcular a distância diretamente), pois L e C são pequenos nos casos de teste.
- Este problema é um aquecimento para a Transformada de Distância geral, que será trabalhada em capítulos posteriores com múltiplos objetos e algoritmos otimizados.

```
%%writefile EP02_04.py
# Código Python
```

```
Overwriting EP02_04.py
```

```
TestSuite("EP02_04.py").run()
```



Figura 2.15: Simulador: Ajuste de Brilho e Contraste

2.12.5 EP02_05 Translação de Imagem

Nesta atividade, você deve implementar o deslocamento espacial de uma imagem. A translação move cada pixel da imagem original para uma nova posição com base em um vetor de deslocamento.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia dois inteiros t_x (deslocamento horizontal) e t_y (deslocamento vertical).
- Leia os valores inteiros da matriz original.
- Calcule a nova posição (x', y') para cada pixel (x, y) original.
- Imprima a matriz resultante com as mesmas dimensões da original.
- Ver na Figure 2.16 uma simulação deste EP.

📌 Importante:

- **Preenchimento:** Pixels que “entram” na imagem devido ao deslocamento e não possuem correspondente na original devem ser preenchidos com **0** (preto).
- **Descarte:** Pixels que, após a translação, ficarem fora dos limites da matriz ($0 \dots L - 1$ ou $0 \dots C - 1$) devem ser ignorados.
- **Coordenadas:** Considere x como o índice da linha e y como o índice da coluna.

2.12.5.1 🧠 Deslocamento Espacial

Transladar uma imagem significa mover todos os seus pontos por uma distância fixa em direções especificadas. Matematicamente, usando coordenadas homogêneas, a operação é descrita como:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Que resulta nas equações simples:

- $x' = x + t_x$
- $y' = y + t_y$

2.12.5.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém os inteiros t_x e t_y .

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz resultante com as mesmas dimensões $L \times C$ após o deslocamento.

2.12.5.3 Exemplos

Entrada	Saída	Observação
2 2 1 1 10 20 30 40	0 0 0 10	Deslocamento ($t_x = 1, t_y = 1$): Cada pixel move uma posição para a direita (horizontal) e uma para baixo (vertical). O pixel $(0, 0) = 10$ vai para o destino $(1, 1)$ (canto inferior direito). As posições vazias são preenchidas com 0.
3 3 -1 0 1 2 3 4 5 6 7 8 9	2 3 0 5 6 0 8 9 0	
		Deslocamento ($t_x = -1, t_y = 0$): Cada pixel move uma posição para a esquerda (horizontal). A primeira coluna original (1, 4, 7) é descartada, as demais colunas movem-se para a esquerda, e a última coluna resultante é preenchida com zeros (0).

```
%%writefile EP02_05.py
# Código Python
```

```
Overwriting EP02_05.py
```

```
TestSuite("EP02_05.py").run()
```

2.12.6 EP02_06 Rotação de Imagem

Nesta atividade, você deve implementar a rotação de uma imagem em torno do seu centro geométrico. Esta operação requer o mapeamento de coordenadas e o uso de técnicas de interpolação para determinar os novos valores dos pixels.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia um valor real θ (ângulo em graus) e uma string representando o **método** de interpolação (**nearest** ou **bilinear**).

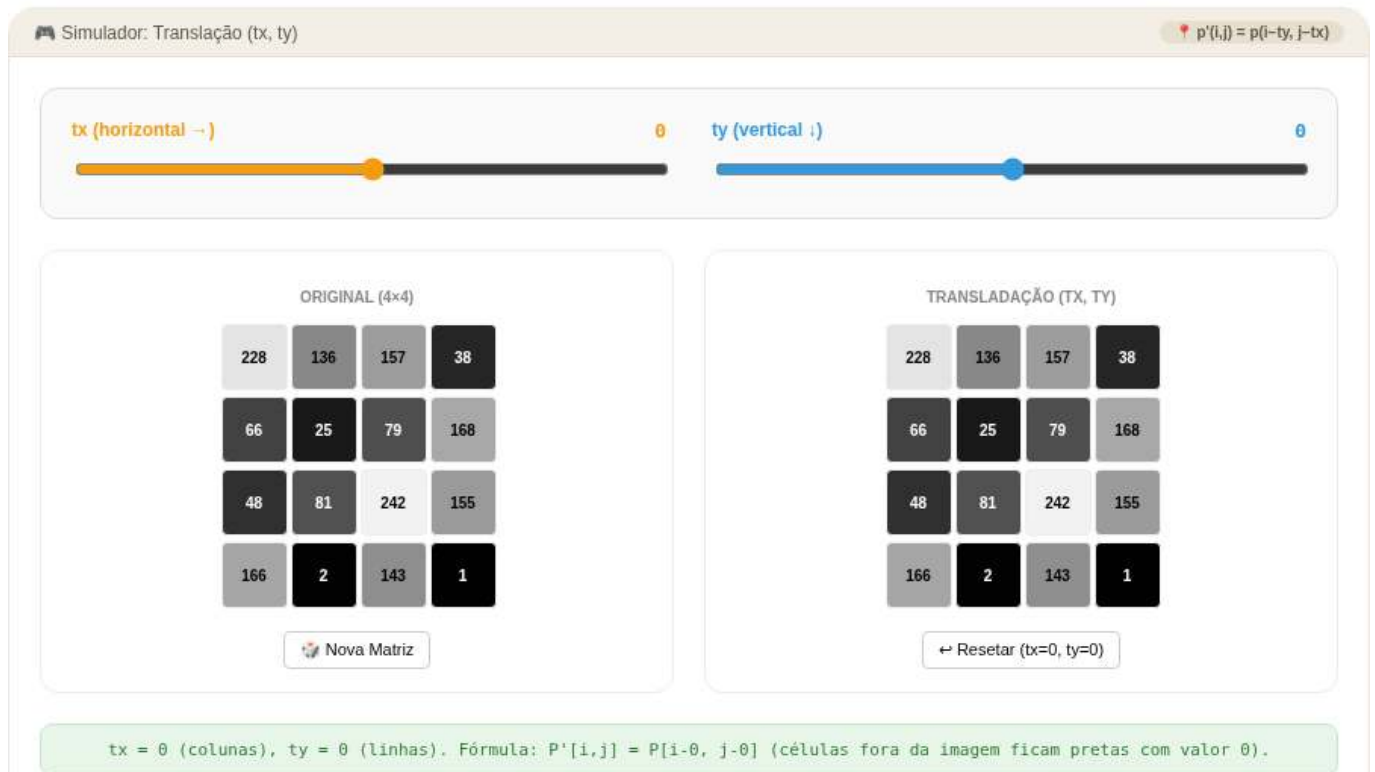


Figura 2.16: Simulador: Translação (tx, ty)

- Leia os valores inteiros da matriz original.
- Realize a rotação em torno do centro da imagem $(L/2, C/2)$.
- Imprima a matriz resultante com as mesmas dimensões da original.
- Ver na Figure 2.17 uma simulação deste EP.

📌 Importante:

- **Mapeamento Inverso:** Para evitar “buracos” na imagem final, percorra cada pixel (x', y') da imagem de destino e calcule sua posição correspondente (x, y) na imagem original usando a matriz de rotação inversa.
- **Interpolação:**
 - **nearest:** Atribui o valor do pixel mais próximo da coordenada calculada.
 - **bilinear:** Calcula uma média ponderada baseada nos 4 vizinhos mais próximos.
- **Bordas:** Pixels cuja origem (x, y) caia fora dos limites da imagem original devem ser preenchidos com 0.

2.12.6.1 🌸 Transformação por Ângulo

A rotação de um ponto (x, y) em relação à origem por um ângulo θ é dada pela matriz de transformação. Para rotacionar em torno de um centro (x_c, y_c) , primeiro transladamos o centro para a origem, rotacionamos e transladamos de volta:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & x_c \\ \sin \theta & \cos \theta & y_c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x - x_c \\ y - y_c \\ 1 \end{bmatrix}$$

Dica: Use o mapeamento inverso para garantir que todos os pixels da imagem de saída sejam preenchidos corretamente.

2.12.6.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém o ângulo **theta** (em graus) e o método **interp** (**nearest** ou **bilinear**).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz rotacionada com L linhas e C colunas.

2.12.6.3 Exemplos

Entrada	Saída	Observação
2	3 1	Rotação de 90° horário: a coluna 0 vira a linha 0 (de baixo para cima). $(0, 0) = 1 \rightarrow (1, 0)$, $(1, 0) = 3 \rightarrow (0, 0)$, $(0, 1) = 2 \rightarrow (1, 1)$, $(1, 1) = 4 \rightarrow (0, 1)$.
2	4 2	
90 nearest		
1 2		
3 4		
3	0 180 0	Rotação de 45°: o pixel central permanece 255; os vizinhos diretos recebem valor interpolado ≈ 180 por bilinear; os cantos permanecem 0.
3	180 255 180	
45 bilinear	0 180 0	
0 0 0		
0 255 0		
0 0 0		

```
%%writefile EP02_06.py
# Código Python
```

```
Overwriting EP02_06.py
```

```
TestSuite("EP02_06.py").run()
```

2.12.7 EP02_07 Redimensionamento (Escala)

Nesta atividade, você deve implementar o redimensionamento de uma imagem utilizando fatores de escala. Diferente da subamostragem simples, aqui utilizaremos técnicas de interpolação para permitir tanto a ampliação quanto a redução da imagem.

- Leia dois inteiros L e C , representando as dimensões da matriz original.
- Leia dois valores reais s_x (escala nas linhas) e s_y (escala nas colunas).
- Leia uma string representando o **método** de interpolação (**nearest** ou **bilinear**).
- Leia os valores inteiros da matriz original.

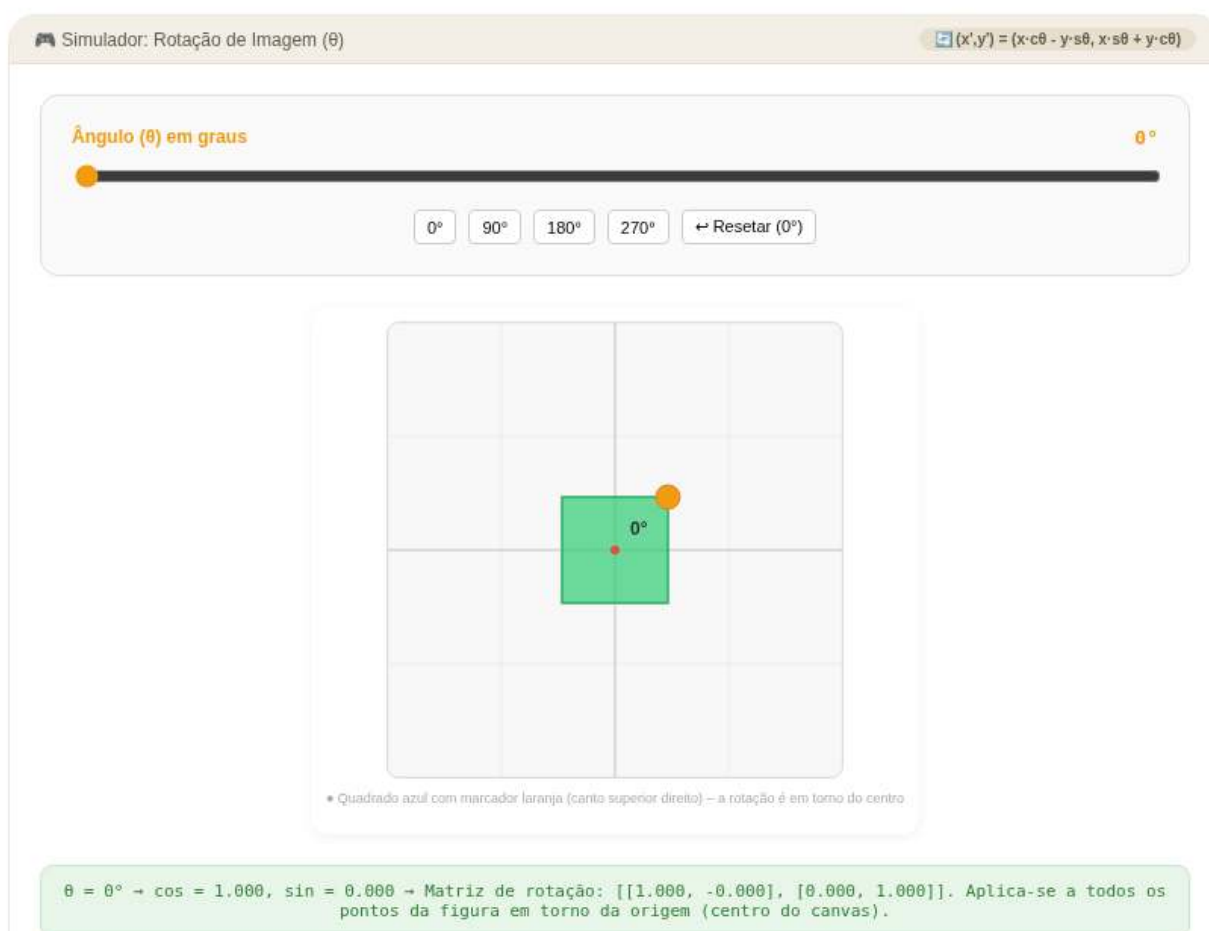


Figura 2.17: Simulador: Rotação (ângulo)

- Calcule as novas dimensões: $L' = \text{round}(L \times s_x)$ e $C' = \text{round}(C \times s_y)$.
- Imprima a matriz resultante com as novas dimensões.
- Ver na Figure 2.18 uma simulação deste EP.

Importante:

- **Mapeamento Inverso:** Para cada pixel (x', y') da imagem de destino, encontre a posição correspondente na origem usando $(x, y) = (x'/s_x, y'/s_y)$.
- **Interpolação:**
 - **nearest:** Seleciona o valor do pixel mais próximo (arredondamento das coordenadas).
 - **bilinear:** Realiza uma interpolação linear dupla entre os quatro pixels vizinhos mais próximos na imagem original.
 - **Bordas:** Certifique-se de que o mapeamento não tente acessar índices fora do intervalo $[0, L - 1]$ e $[0, C - 1]$.

2.12.7.1 Interpolação para Ampliação/Redução

Redimensionar uma imagem por fatores (s_x, s_y) exige o preenchimento de vazios (na ampliação) ou a fusão de informações (na redução). O método de interpolação define a qualidade visual do resultado:

Método	Funcionamento	Efeito Visual
Nearest	Pega o valor do vizinho mais próximo.	Rápido, mas gera efeito “pixelado” ou blocos.
Bilinear	Média ponderada dos 4 vizinhos (2×2) .	Suaviza a imagem, reduzindo o serrilhamento.

2.12.7.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém os fatores s_x e s_y .

A quarta linha contém o método `interp` (`nearest` ou `bilinear`).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz redimensionada com dimensões $L' \times C'$.

2.12.7.3 Exemplos

Entrada	Saída	Observação
2	1 1 2 2	Ampliação 2×: cada pixel original é replicado em um bloco 2×2. A imagem 2 × 2 vira 4 × 4.
2	1 1 2 2	
2.0 2.0	3 3 4 4	
nearest	3 3 4 4	
1 2		
3 4		Redução 0.5×: a imagem 2 × 2 vira 1 × 1. Com nearest, o único pixel de saída amostra a posição (0,0) = 10.
2	10	
2		
0.5 0.5		
nearest		
10 20		
30 40		

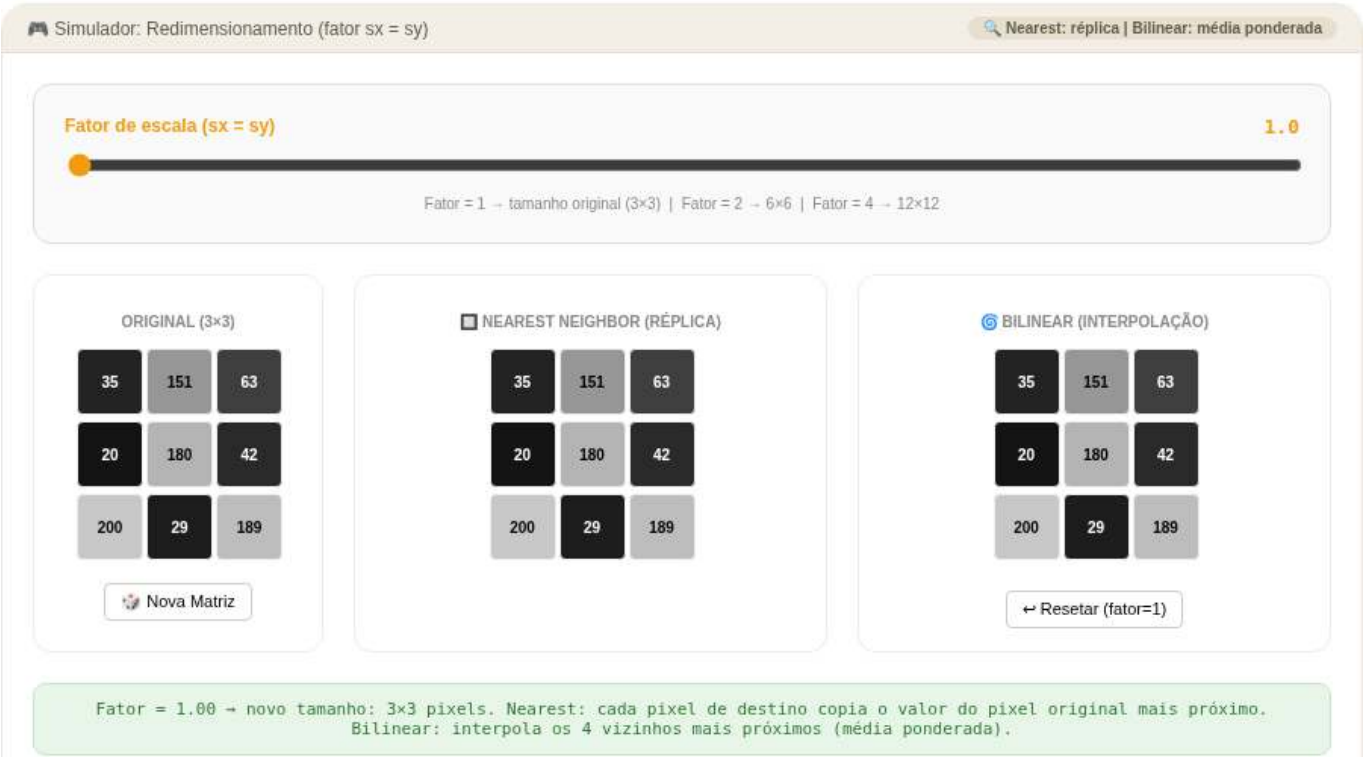


Figura 2.18: Simulador: Redimensionamento (Nearest vs Bilinear)

```
%%writefile EP02_07.py
# Código Python
import numpy as np
from morph import mm

# 1. Leitura das dimensões, fatores e método
l = int(input())
c = int(input())
sx, sy = map(float, input().split())
interp = input().strip()

# 2. Leitura da imagem original
img = mm.readImg(l, c)

# 3. Novas dimensões
```

```

l_new = round(l * sx)
c_new = round(c * sy)

# 4. Redimensionamento usando mm.resize
# cv2.resize usa (largura, altura) = (colunas, linhas)
resultado = mm.resize(img, (c_new, l_new), method=interp)

# 5. Exibição
print(mm.drawImg(resultado))

```

Writing EP02_07.py

TestSuite("EP02_07.py").run()

2.12.8 EP02_08 ✂ Cisalhamento (Shear)

Nesta atividade, você deve implementar a transformação de cisalhamento em uma imagem. O cisalhamento é uma transformação afim que desloca cada ponto em uma direção fixada, por um valor proporcional à sua distância de uma reta paralela a essa direção, resultando em um efeito de inclinação.

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz.
- Leia dois valores reais sh_x (cisalhamento horizontal) e sh_y (cisalhamento vertical).
- Leia uma string representando o **método** de interpolação (**nearest** ou **bilinear**).
- Leia os valores inteiros da matriz original.
- Aplique a transformação mantendo o tamanho original da imagem (cortando o que ultrapassar os limites).
- Imprima a matriz resultante com as dimensões $L \times C$.
- Ver na Figure 2.19 uma simulação deste EP.

📌 Importante:

- **Mapeamento Inverso:** Para cada pixel (x', y') da imagem de destino, calcule a posição correspondente na origem (x, y) utilizando a matriz de cisalhamento inversa.
- **Preenchimento:** Coordenadas que resultarem em posições fora da matriz original devem ser preenchidas com 0.
- **Coordenadas:** Para fins desta implementação, considere x como o índice da linha e y como o índice da coluna.

2.12.8.1 🧠 Distorção Afim

O cisalhamento altera a geometria da imagem inclinando seus eixos. A relação entre as coordenadas originais (x, y) e as transformadas (x', y') é dada por:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Isso resulta nas equações:

- $x' = x + sh_x \cdot y$
- $y' = y + sh_y \cdot x$

2.12.8.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém os fatores `shx` e `shy`.

A quarta linha contém o método `interp` (`nearest` ou `bilinear`).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz transformada com as mesmas dimensões $L \times C$.

2.12.8.3 Exemplos

Entrada	Saída	Observação
3	10 20 30	Cisalhamento horizontal: linha i desloca $\lfloor i \cdot 0.5 \rfloor$ pixels. Linha $0 \rightarrow 0\text{px}$, linha $1 \rightarrow 0\text{px}$, linha $2 \rightarrow 1\text{px}$. Os pixels deslocados para fora são descartados e as posições vazias preenchidas com 0.
3	0 40 50	
0.5 0.0	0 0 70	
nearest		
10 20 30		
40 50 60		Cisalhamento vertical: coluna j desloca $\lfloor j \cdot 1.0 \rfloor$ pixels para baixo. Coluna $0 \rightarrow 0\text{px}$ (inalterada), coluna $1 \rightarrow 1\text{px}$: 20 desce para $(1, 1)$ e $(0, 1)$ fica 0.
70 80 90		
2	10 0	
2	30 20	
0.0 1.0		
nearest		
10 20		
30 40		

```
%%writefile EP02_08.py
# Código Python
```

```
Overwriting EP02_08.py
```

```
TestSuite("EP02_08.py").run()
```

2.12.9 EP02_09 Transformação Afim Genérica

Nesta atividade, você deve implementar uma transformação afim arbitrária em uma imagem. Esta operação é a generalização de todas as transformações lineares (escala, rotação, cisalhamento) combinadas com a translação, permitindo manipulações geométricas complexas através de uma única matriz.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia **seis valores reais** (a, b, t_x, c, d, t_y) que compõem a matriz de transformação afim 2×3 .
- Leia uma string representando o **método** de interpolação (`nearest` ou `bilinear`).
- Leia os valores inteiros da matriz original.
- Aplique a transformação mantendo o tamanho original $L \times C$.

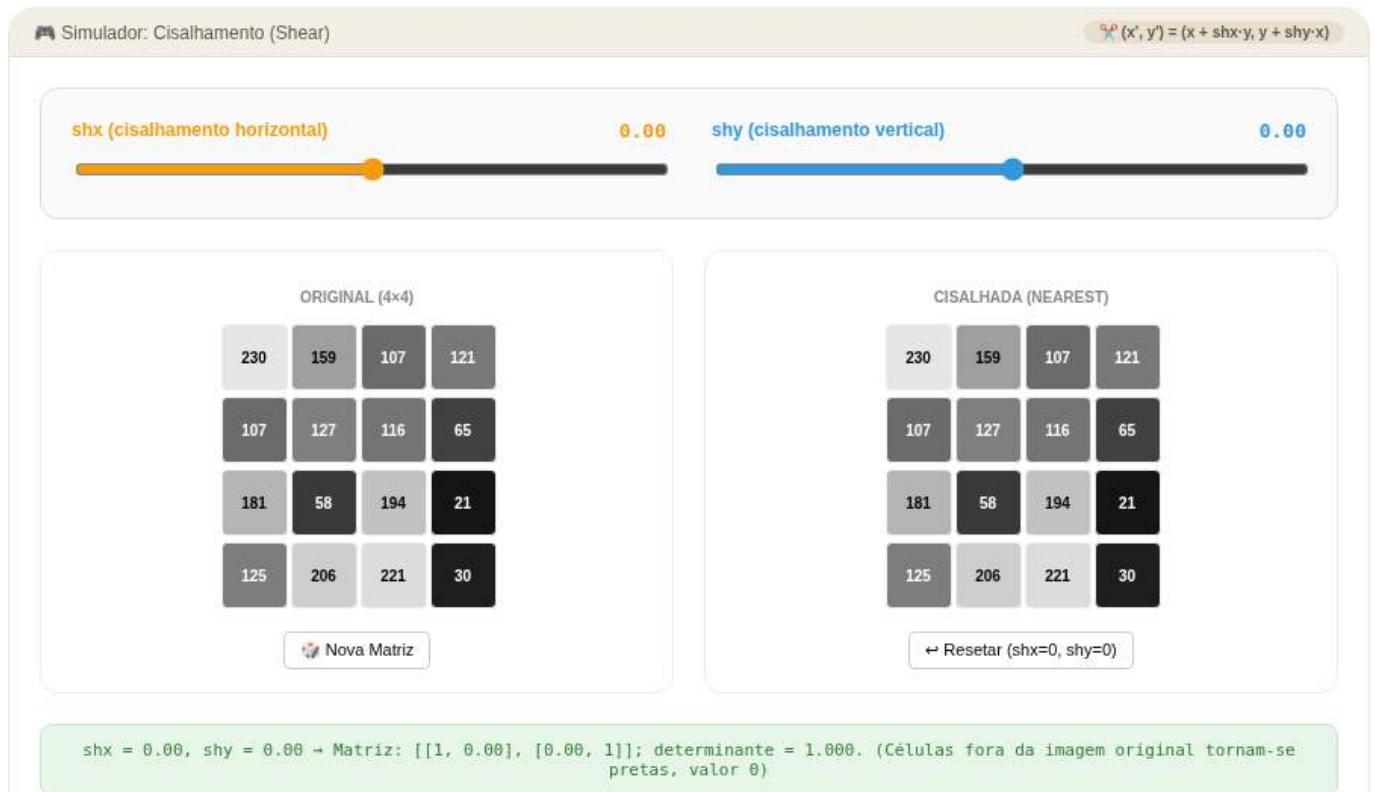


Figura 2.19: Simulador: Cisalhamento (shx, shy)

- Imprima a matriz resultante.
- Ver na Figure 2.20 uma simulação deste EP.

📌 Importante:

- **Mapeamento Inverso:** Para calcular o valor de cada pixel na imagem de destino, você deve utilizar a **inversa** da matriz de transformação afim fornecida para encontrar a coordenada correspondente na imagem original.
- **Preenchimento:** Coordenadas calculadas que caíam fora dos limites $[0, L - 1]$ e $[0, C - 1]$ da imagem original devem resultar em um pixel de valor **0**.
- **Flexibilidade:** Esta implementação deve ser capaz de realizar qualquer uma das tarefas anteriores (translação, rotação, etc.) bastando alterar os parâmetros da matriz.

Dica:

```
flags = cv2.INTER_NEAREST if interp == 'nearest' else \
        cv2.INTER_CUBIC   if interp == 'bicubic'   else \
        cv2.INTER_LANCZOS4 if interp == 'lanczos'  else \
        cv2.INTER_LINEAR

r = cv2.warpAffine(img, M, (C, L), flags=flags)
```

2.12.9.1 🧠 Combinação de Operações

A transformação afim preserva pontos, retas e planos. No processamento de imagens, ela mapeia a posição (x, y) para (x', y') seguindo o sistema:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Ou, de forma compacta em coordenadas homogêneas:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2.12.9.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém seis floats: a b t_x c d t_y .

A quarta linha contém o método `interp` (`nearest` ou `bilinear`).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz transformada com as dimensões originais $L \times C$.

2.12.9.3 Exemplos

Entrada	Saída	Observação
2	15 20	Translação fracionária ($t_x = 0.5, t_y = 0.5$): cada pixel de saída (i, j) amostra a posição $(i + 0.5, j + 0.5)$ da entrada via bilinear. Ex: $(0, 0)$ interpola os quatro vizinhos $\rightarrow 15$.
2	25 30	
1.0 0.0 0.5 0.0 1.0 0.5		
bilinear		
10 20		
30 40		Escala $2\times$ via matriz afim ($a = 2, d = 2$): cada pixel de saída (i, j) amostra a posição $(2i, 2j)$ da entrada com nearest. Ex: $(0, 2) \rightarrow (0, 4)$ fora da imagem $\rightarrow$ nearest clipa para $(0, 2) = 3$... aguarda confirmação da lógica de borda.
3	1 1 2	
3	1 1 2	
2.0 0.0 0.0 0.0 2.0 0.0	4 4 5	
nearest		
1 2 3		
4 5 6		
7 8 9		

```
%%writefile EP02_09.py
# Código Python
```

```
Overwriting EP02_09.py
```

```
TestSuite("EP02_09.py").run()
```

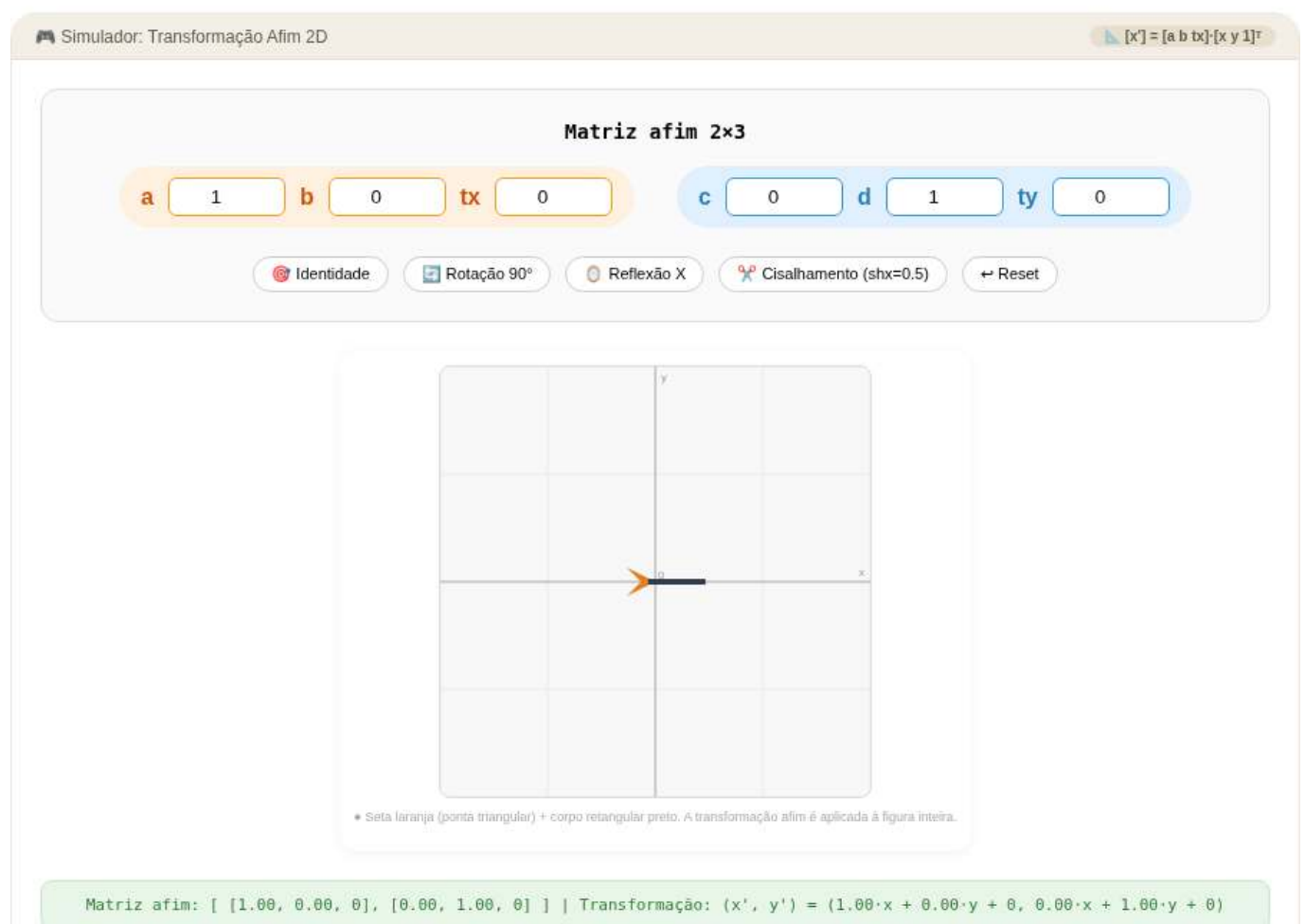


Figura 2.20: Simulador: Transformação Afim (matriz 2x3)

2.12.10 EP02_10 🎯 Correção de Perspectiva (Homografia)

Nesta atividade, você deve implementar a transformação de perspectiva, também conhecida como homografia. Diferente das transformações afins, a perspectiva não preserva o paralelismo, permitindo “retificar” objetos inclinados, como documentos ou placas capturados em ângulos oblíquos.

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz original.
- Leia **quatro pares de coordenadas** (x, y) representando os cantos do quadrilátero de origem (objeto distorcido).
- Leia **quatro pares de coordenadas** (x, y) representando os cantos do quadrilátero de destino (onde o objeto deve ser mapeado).
- Leia os valores da matriz original.
- Calcule a matriz de homografia 3×3 e aplique a transformação.
- Imprima a matriz resultante com as dimensões de saída especificadas.
- Ver na Figure 2.21 uma simulação deste EP.

📌 Importante:

- **Graus de Liberdade:** A homografia possui 8 graus de liberdade (o nono elemento da matriz 3×3 é uma constante de normalização, geralmente 1), exigindo no mínimo 4 pontos correspondentes para ser calculada.
- **Projeção:** Após multiplicar as coordenadas pela matriz, é necessário dividir os resultados x' e y' pela componente homogênea w para retornar ao plano 2D.
- **Uso de Bibliotecas:** Para esta tarefa, você pode utilizar as funções `cv2.getPerspectiveTransform` para obter a matriz e `cv2.warpPerspective` para aplicar a transformação, ou implementar o sistema linear e o mapeamento inverso manualmente para um desafio extra.

```
# Dimensões de saída: bounding box dos pontos destino + 1
w = int(max(pts2[:, 0])) + 1; h = int(max(pts2[:, 1])) + 1
# M = cv2.getPerspectiveTransform(pts1, pts2)
# dst = cv2.warpPerspective(img, M, (w, h))
# ou
dst = mm.perspective_transform(img, pts1, pts2, size=(w, h))
```

2.12.10.1 🧠 Deformação não-afim

Enquanto transformações afins mapeiam paralelogramos em paralelogramos, a homografia mapeia qualquer quadrilátero em outro quadrilátero. Isso é essencial para visão computacional:

Operação	Característica	Aplicação Típica
Homografia	Projeção em plano	Correção de documentos, escaneamento de placas.
Ponto de Fuga	Convergência de linhas	Reconstrução 3D a partir de imagens 2D.
Warping	Deformação de malha	Estabilização de vídeo e panoramas (stitching).

2.12.10.2 📌 Exemplos

Entrada	Saída	Observação
4 4	10 20 30 40	As 4 primeiras linhas após as dimensões são os pontos de origem; as 4 seguintes são os destinos. Com pontos idênticos, a transformação de perspectiva é a identidade e a imagem é preservada.
0 0	50 60 70 80	
3 0	90 100 110 120	
0 3	130 140 150 160	
3 3		
0 0		
3 0		
0 3		
3 3		
10 20 30 40		
50 60 70 80		
90 100 110 120		
130 140 150 160		

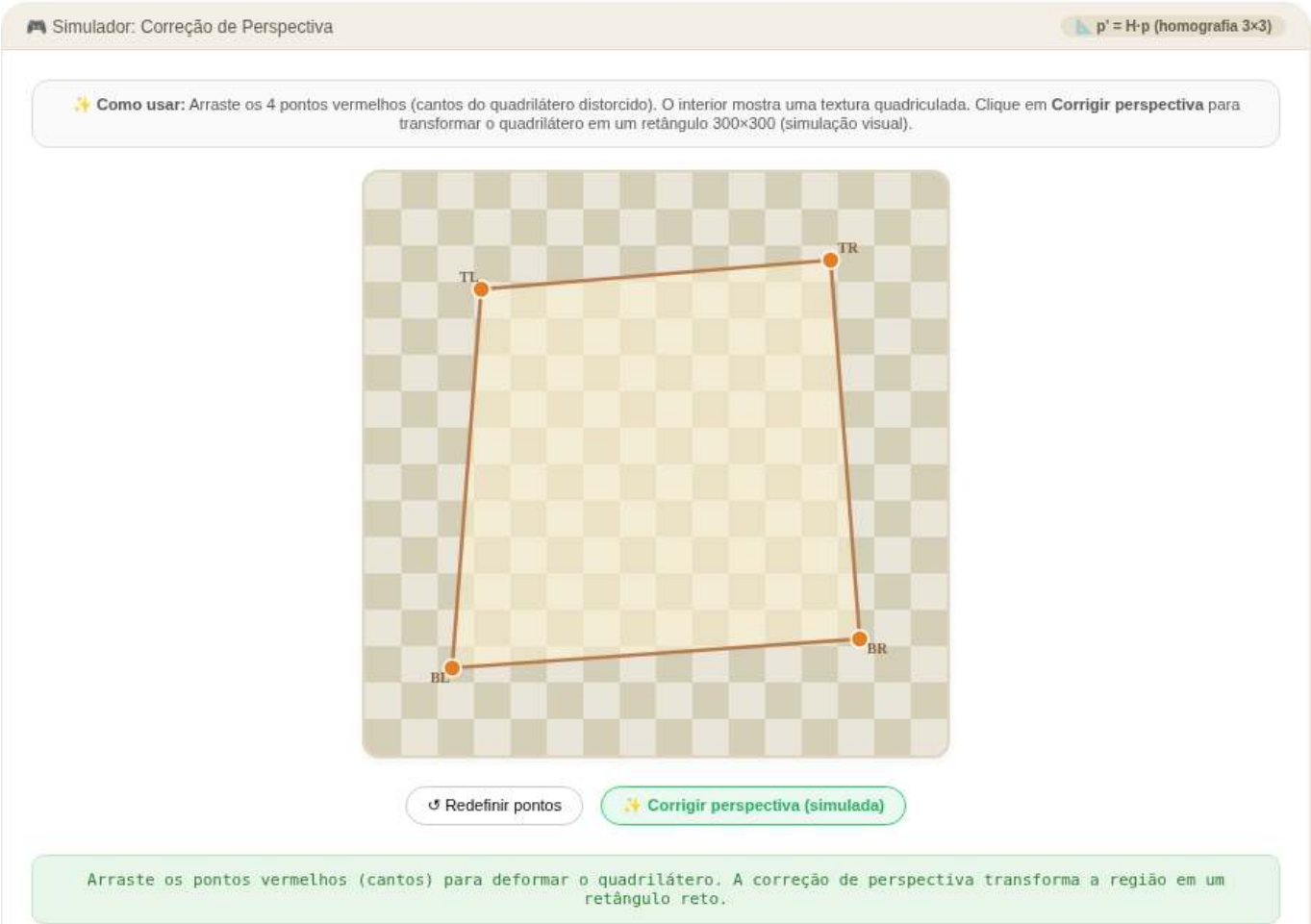


Figura 2.21: Simulador: Correção de Perspectiva (Homografia)

```
%%writefile EP02_10.py
# Código Python

Overwriting EP02_10.py

TestSuite("EP02_10.py").run()
```


2.12.11 EP02_11 🎯 Correção de Perspectiva (Homografia) em Imagem Real

Nesta atividade, o objetivo é aplicar a transformação de perspectiva (homografia) para “retificar” um objeto inclinado em uma fotografia real. Você lidará com a imagem de um jornal, onde a grade de um jogo de Sudoku está distorcida devido ao ângulo em que a foto foi capturada.

Seu programa deve ler os parâmetros de entrada do terminal, carregar a imagem, calcular a matriz de homografia 3×3 , aplicar a transformação geométrica e exibir um indicador global de validação.

- Leia dois inteiros **L** e **C**, representando as dimensões de linhas e colunas (altura e largura) que a imagem retificada de saída deve ter.
- Leia **quatro pares de coordenadas** (x, y) via terminal, representando os quatro cantos do quadrilátero de origem (o Sudoku distorcido na imagem original).
- Calcule automaticamente os **quatro pares de coordenadas de destino** utilizando as dimensões L e C fornecidas, mapeando os cantos para as extremidades da nova imagem: $(0, 0)$, $(C - 1, 0)$, $(0, L - 1)$ e $(C - 1, L - 1)$.
- Carregue a imagem local `sudoku.png` e converta-a para tons de cinza (grayscale).
- Calcule a matriz de homografia e aplique a transformação espacial na imagem.
- **Saída:** Calcule e imprima a **soma de todos os pixels** da imagem resultante.

📌 Importante:

- **Arquivo de entrada:** A imagem `sudoku.png` deve estar na mesma pasta do script. O programa deve lê-la diretamente do disco (ex: usando `mm.read("sudoku.png")` ou `cv2.imread`).
- **Ordem dos Pontos:** Garanta que a leitura dos 4 pontos de origem e a geração dos 4 pontos de destino sigam rigorosamente a mesma ordem dos cantos: Superior-Esquerdo (TL), Superior-Direito (TR), Inferior-Esquerdo (BL) e Inferior-Direito (BR).
- **Dimensões no OpenCV:** Lembre-se que funções como `cv2.warpPerspective` esperam o tamanho da imagem de saída no formato `(largura, altura)`, o que equivale a (C, L) .
- **Interpolação:** Para garantir a consistência matemática da soma dos pixels com o corretor automático, utilize a interpolação bilinear padrão (`flags=cv2.INTER_LINEAR`).
- **Créditos:** A imagem utilizada é “Sudoku en periódico” de Héctor Rodríguez, sob licença **CC BY 2.0**.

2.12.11.1 🧠 Contexto do Problema

A homografia possui 8 graus de liberdade, exigindo no mínimo 4 correspondências de pontos para ser calculada. Ao contrário de transformações afins, ela mapeia qualquer quadrilátero em outro quadrilátero, permitindo que linhas que convergem para pontos de fuga voltem a ser paralelas:

Operação	Característica	Aplicação Típica
Homografia	Projeção entre planos	Retificação de documentos, escaneamento de placas e QR Codes.
Mapeamento Inverso	Varredura do destino para a origem	Evita “buracos” ou pixels vazios na imagem final retificada.
Warping	Reamostragem espacial	Correção de distorção de lentes e montagem de panoramas (<i>stitching</i>).

2.12.11.2 📌 Exemplos

Entrada	Saída	Observação
500 500 100 120 420 95 80 440 450 460	32982820	As duas primeiras entradas são as dimensões de saída (L e C). As 4 linhas seguintes são as coordenadas (x, y) dos cantos do Sudoku na imagem original + PAD. A saída é a soma total dos pixels da imagem retificada.
200 200 100 120 420 95 80 440 450 460	5277150	

2.12.11.3 Aquisição da imagem do sudoku e conversão para níveis de cinza

A Figure 2.22 mostra a leitura da imagem original seguida da conversão para tons de cinza e redimensionamento para uma matriz de 500×500 pixels, preparando os dados para a etapa seguinte. A correção de perspectiva, aplicada na Figure 2.23 via matriz de homografia, elimina as deformações causadas pelo ângulo da câmera e produz uma visão frontal e regular da grade do Sudoku.



Figura 2.22: Aquisição da imagem de um Sudoku à esquerda. À direita, conversão para tons de cinza e redimensionamento. Crédito: Héctor Rodríguez de Guardamar, Espanha (CC BY 2.0).

```
import cv2
import numpy as np

# --- 1. Carrega a imagem salva (sudoku.png) ---
img = mm.read("sudoku.png")          # BGR, 500x500

# --- 2. Padding para não cortar vértices ---
```

```

PAD = 60
img_pad = cv2.copyMakeBorder(
    img, PAD, PAD, PAD, PAD,
    cv2.BORDER_CONSTANT, value=[255, 255, 255]
)

# --- 3. Pontos de origem (cantos da grade na imagem expandida) ---
pts1 = np.float32([
    [100, 160], # TL
    [390, 45],  # TR
    [200, 580], # BL
    [570, 420], # BR
])
#      W      H

# --- 4. Pontos de destino (vista frontal 500x500) ---
SIZE = 500
pts2 = np.float32([
    [0, 0],
    [SIZE, 0],
    [0, SIZE],
    [SIZE, SIZE],
])

# --- 5. Homografia e retificação ---
img_rect = mm.perspective_transform(img_pad, pts1, pts2, size=(SIZE, SIZE))

# --- 6. Exibição ---
mm.show(
    [img_pad, img_rect],
    titles=["Original (com padding)", "Vista frontal retificada"],
    cols=2, figsize=(10, 6), axis=True
)

```

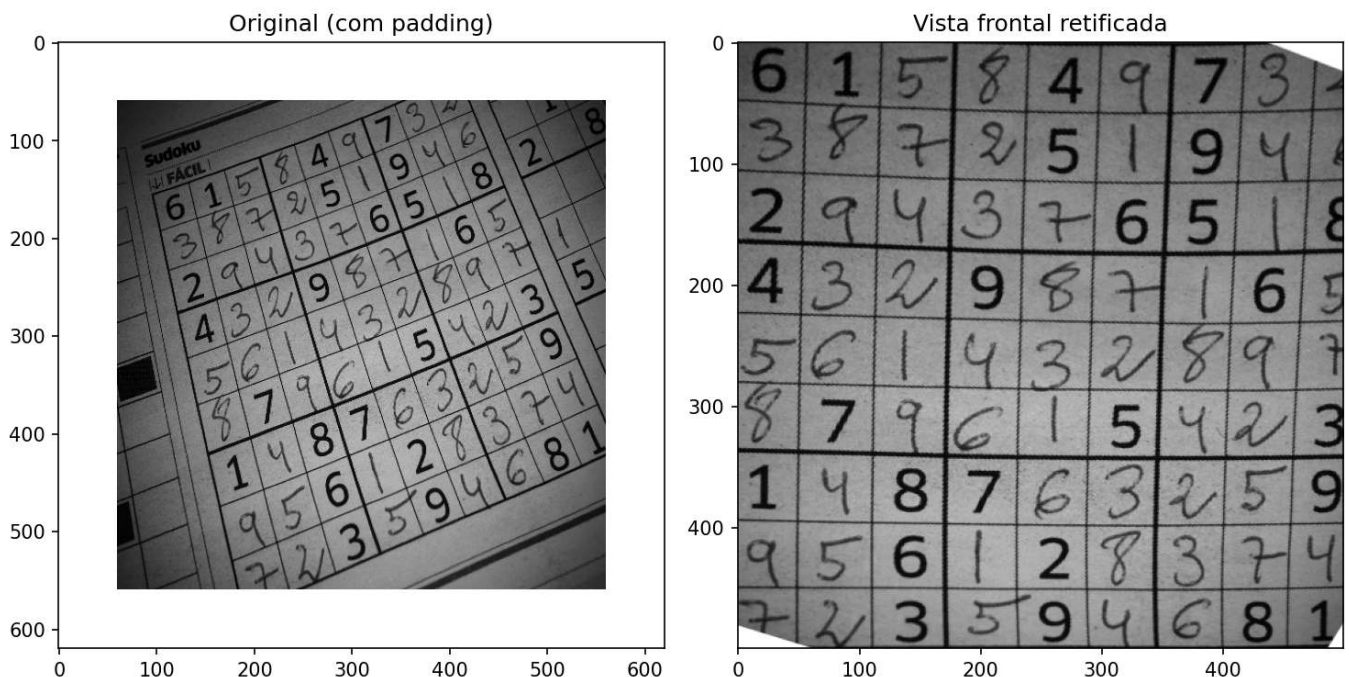


Figura 2.23: Correção de perspectiva: original e vista frontal retificada.

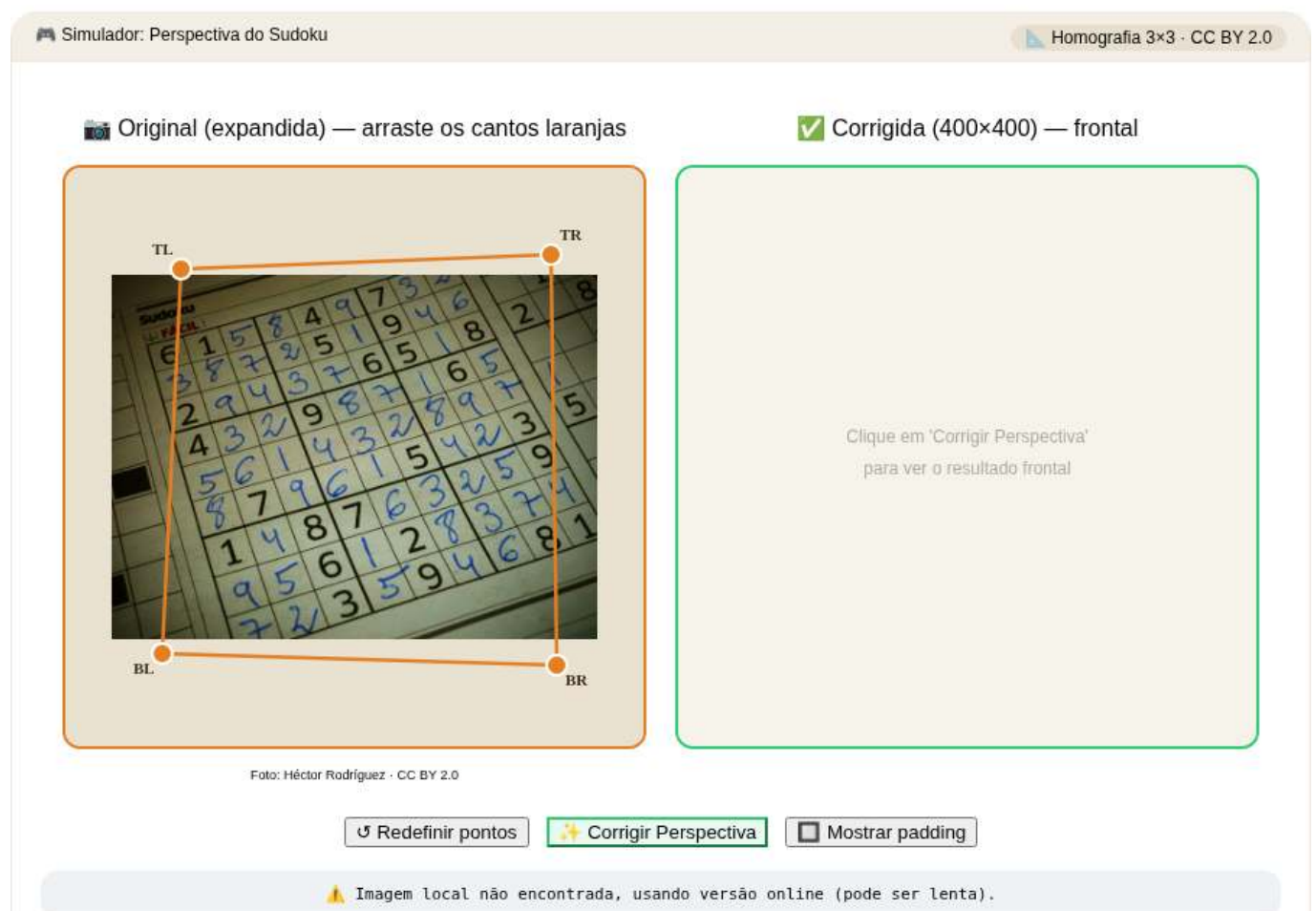


Figura 2.24: Simulador: correção de perspectiva do Sudoku.

```
%%writefile EP02_11.py  
# Código Python
```

```
Overwriting EP02_11.py
```

```
TestSuite("EP02_11.py").run()
```


Capítulo 3

Operações Espaciais: Intensidade, Histograma e Filtragem

 Executar  Colab  Abrir si-md2  GitHub

Este capítulo aprofunda o processamento de imagens no domínio espacial, partindo da manipulação direta de pixels e histogramas para o realce de contraste, até a aplicação de filtros locais por convolução para suavização, redução de ruído e detecção de bordas. O objetivo é desenvolver a intuição matemática e computacional que sustenta grande parte dos algoritmos modernos de Visão Computacional.

3.1 Objetivos

Ao final deste capítulo, você será capaz de:

- **Manipular intensidade e pixels:** Executar operações aritméticas saturadas (`mm.addm`, `mm.subm`) e lógicas bit a bit (`mm.band`, `mm.bor`, `mm.bnot`) para combinação e seleção de regiões de interesse (ROI), e aplicar *alpha blending* (`mm.blend`) para fusão ponderada de imagens;
- **Processar histogramas:** Interpretar o histograma como diagnóstico tonal, aplicar equalização global (`mm.equalize`) e adaptativa (CLAHE), e realizar especificação de histograma para transferência de perfil tonal entre imagens;
- **Compreender fundamentos espaciais:** Entender vizinhança, *padding* de borda, e a diferença entre correlação cruzada (`mm.conv`) e convolução — incluindo por que kernels assimétricos como Sobel produzem resultados distintos nas duas operações;
- **Aplicar filtragem de suavização:** Utilizar o filtro de média (`mm.conv` com kernel uniforme) e o filtro Gaussiano (`cv2.GaussianBlur`) para redução de ruído, compreendendo a vantagem da ponderação radial e da separabilidade Gaussiana;
- **Aplicar filtragem de realce:** Usar o Laplaciano (w_4 e w_8) para realce isotrópico de bordas, o operador de Sobel para gradiente direcional (G_x , G_y , magnitude e direção), e o *Unsharp Masking* para amplificação controlada de alta frequência pelo parâmetro k ;
- **Utilizar filtros de ordem:** Aplicar o filtro da mediana (`cv2.medianBlur`) para remoção eficaz de ruído sal e pimenta, compreendendo por que sua natureza não linear e robustez a outliers o tornam superior aos filtros lineares nesse cenário;
- **Resolver problemas práticos:** Encadear técnicas em pipelines de pré-processamento (ex.: CLAHE → Gaussiano → Canny) e utilizar as funções da biblioteca `morph.py` (`mm.conv`, `mm.histImg`, `mm.equalize`, `mm.drawImageKernel`) para análise e visualização didática de cada etapa.

3.2 Operações em Nível de Intensidade

O nível mais elementar de processamento de imagens atua diretamente sobre os valores dos pixels, sem considerar vizinhança. Essas operações — chamadas de **transformações de ponto** (*point operations*) — são as mais rápidas computacionalmente e formam a base para técnicas mais complexas.

Formalmente, uma transformação de ponto pode ser descrita como:

$$g(x, y) = T[f(x, y)] \quad (3.1)$$

onde $f(x, y)$ é a imagem de entrada, $g(x, y)$ é a saída e T é uma função aplicada a cada pixel individualmente.

3.2.1 Preparando o Ambiente Prático

O bloco a seguir carrega o módulo `morph.py` do repositório.

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

version = getattr(morph, "__version__", "local_file")
print(f" Ambiente pronto. Módulo 'morph' carregado (versão:{version}).")
```

Ambiente pronto. Módulo 'morph' carregado (versão: 1.1.1).

Como objeto de estudo ao longo deste capítulo, utilizaremos as imagens de vida selvagem apresentadas nas Figure 3.1 e Figure 3.3. A partir delas, exploraremos operações espaciais sobre intensidade, histogramas e filtragem, analisando seus efeitos no realce, na suavização, na redução de ruído e na detecção de bordas, de modo a compreender os fundamentos matemáticos e computacionais do PDI.

```
import os

# Fonte : https://commons.wikimedia.org/wiki/File:Carlos_Guilherme_Rodrigues_(76515283).jpeg
# Mapa : https://www.google.com/maps?q=-26.204734,28.226624

base = "https://upload.wikimedia.org/wikipedia/commons"
arquivo = "Carlos_Guilherme_Rodrigues_%2876515283%29.jpeg"
url = f"{base}/9/9b/{arquivo}"
caminho = "imagens/mandrill-exif.jpg"

# 1. Leitura com cache local
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_color = np.array(img_obj)
img_gray = mm.gray(img_color)

# 2. Diagnóstico de tipos, dimensões e acesso a pixels
print(f"\nTipo PIL : {type(img_obj)} | Dimensões (x,y): {img_obj.size}")
print(f"Tipo NumPy: {type(img_color)} | Dimensões [y,x,c]: {img_color.shape}")

# 3. Exibição
mm.show(img_color, scale=90)
```

Tipo PIL : <class 'PIL.JpegImagePlugin.JpegImageFile'> | Dimensões (x,y): (960, 540)
 Tipo NumPy: <class 'numpy.ndarray'> | Dimensões [y,x,c]: (540, 960, 3)



Figura 3.1: *Mandrill* (*Mandrillus sphinx*) fotografado em ambiente natural na África do Sul. A imagem possui resolução de 500 px e contém metadados EXIF com coordenadas GPS aproximadas (-26.20473, 28.22662). Crédito: Carlos Guilherme Rodrigues (CC BY-SA 3.0).

3.2.2 Operações Aritméticas

Operações aritméticas entre imagens são amplamente usadas em PDI para combinar, comparar ou realçar informações. A **subtração de imagens** é especialmente poderosa para detectar diferenças entre dois quadros — por exemplo, na remoção de fundo estático em câmeras de vigilância:

$$g(x, y) = f_1(x, y) - f_2(x, y) \quad (3.2)$$

A **adição saturada** limita o resultado ao intervalo $[0, 255]$: valores acima de 255 são fixados em 255, evitando o *overflow* silencioso do tipo `uint8` (ex.: $200 + 100 = 44$ em vez de 300). A **subtração saturada** aplica o mesmo princípio pelo lado inferior: valores negativos são fixados em 0.

⚠ Saturação e *overflow*

Operações aritméticas em `uint8` sofrem *overflow* silencioso: $200 + 100 = 44$ (não 300). As funções `mm.addm` e `mm.subm` usam `cv2.add` e `cv2.subtract`, que realizam saturação automática. Para o *blending*, converta para `float32` antes de operar e aplique `np.clip(..., 0, 255).astype(np.uint8)` ao final, pois a operação envolve pesos fracionários.

A Figure 3.2 demonstra adição de uma constante (clareamento) e subtração de uma constante (escurecimento com saturação em 0).

```
fundo = 60

img_add = mm.addm(img_gray, fundo)
img_sub = mm.subm(img_gray, fundo)

mm.show(
    [img_gray, img_add, img_sub],
    titles=["Original", "addm (+60)", "subm (-60)"],
    cols=3
)
```

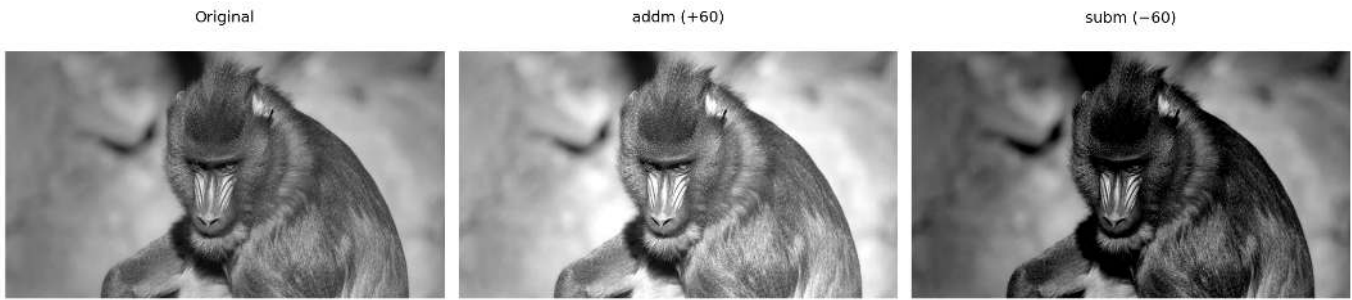


Figura 3.2: Operações aritméticas saturadas: adição de constante (clareamento) e subtração de constante (escurecimento com saturação em 0).

3.2.3 Mistura Ponderada (*Alpha Blending*)

A **mistura ponderada** (*alpha blending*) combina duas imagens utilizando pesos complementares α e $(1 - \alpha)$:

$$g(x, y) = \alpha f_1(x, y) + (1 - \alpha) f_2(x, y), \quad \alpha \in [0, 1] \quad (3.3)$$

Quando $\alpha = 1$, obtém-se apenas a imagem f_1 ; quando $\alpha = 0$, apenas f_2 . Valores intermediários produzem uma transição suave entre ambas, sendo amplamente utilizados em composição de imagens, sobreposição de camadas, marcas d'água e efeitos de fusão visual.

Para que a combinação produza um resultado coerente, é necessário alinhar previamente as regiões de interesse das imagens. Na Figure 3.4, utiliza-se um recorte do rosto do leopardo:

```
leo = img_gray_leo[250:-300, 100:-200]
```

e um recorte central da região facial do mandril:

```
mandrill = img_gray[100:400, 380:530]
```

As duas regiões são escolhidas de forma que olhos e estrutura facial permaneçam aproximadamente alinhados. Em seguida, o recorte do leopardo é redimensionado para coincidir com as dimensões do mandril antes da aplicação da mistura ponderada.

A operação é realizada em `float32` para evitar problemas de *overflow* durante as operações aritméticas, seguida de *clipping* e conversão final para `uint8`.

```
import os
from PIL.ExifTags import TAGS

base = "https://upload.wikimedia.org/wikipedia/commons"
arquivo = "Leopard_%28Panthera_pardus%29_portrait.jpg"
url = f"{base}/9/92/{arquivo}"
caminho = "imagens/leopardo.jpg"

# 1. Leitura com cache local
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_numpy = np.array(img_obj)
img_gray_leo = mm.gray(img_numpy)

# 2. Extração e conversão de GPS (Tag 34853)
```

```

exif = img_obj._getexif()
if exif and (gps := exif.get(34853)):
    to_dec = lambda dms, ref: float(
        -(dms[0]+dms[1]/60+dms[2]/3600) if ref in 'SW'
        else (dms[0]+dms[1]/60+dms[2]/3600)
    )
    lat, lon = to_dec(gps[2], gps[1]), to_dec(gps[4], gps[3])
    print(f"GPS Decimal: {lat:.6f}, {lon:.6f}")
    print(f"Maps: https://www.google.com/maps/search/?api=1&query={lat},{lon}")

# 3. Diagnóstico de tipos, dimensões e acesso a pixels
print(f"\nTipo PIL : {type(img_obj)} | Dimensões (x,y): {img_obj.size}")
print(f"Pillow (0,0): {img_obj.getpixel((0, 0))}")
print(f"Tipo NumPy: {type(img_numpy)} | Dimensões [y,x,c]: {img_numpy.shape}")
print(f"NumPy [0,0]: {img_numpy[0, 0]}")

# 4. Exibição
mm.show(img_numpy)

```

GPS Decimal: -19.140200, 23.814872

Maps: <https://www.google.com/maps/search/?api=1&query=-19.1402,23.814872222222224>

Tipo PIL : <class 'PIL.JpegImagePlugin.JpegImageFile'> | Dimensões (x,y): (1242, 1324)

Pillow (0,0): (192, 134, 86)

Tipo NumPy: <class 'numpy.ndarray'> | Dimensões [y,x,c]: (1324, 1242, 3)

NumPy [0,0]: [192 134 86]



Figura 3.3: Retrato de um leopardo (*Panthera pardus*) em ambiente natural. Crédito: C. Brück (CC BY-SA 4.0).

```

leo = img_gray_leo[250:-300, 100:-200]
mandrill = img_gray[100:400, 380:530]
img_leopardo = mm.resize(leo, (mandrill.shape[1], mandrill.shape[0]), method='bilinear')

alphas = [1.0, 0.8, 0.6, 0.4, 0.2, 0.0]
imgs_blend = []

```

```
titles_blend = []

for a in alphas:
    imgs_blend.append(mm.blend(mandrill, img_leopardo, alpha=a))
    titles_blend.append(f"={a:.1f}")

mm.show(imgs_blend, titles=titles_blend, cols=6, figsize=(18, 16))
```

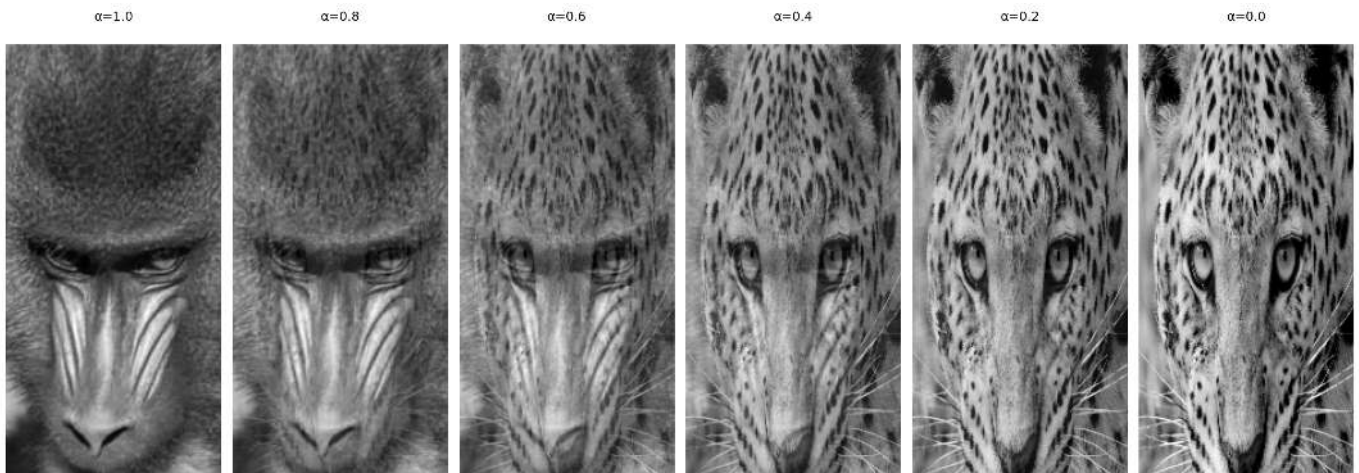


Figura 3.4: *Alpha blending* entre recortes alinhados de mandrill e do leopardo (Figure 3.3) para diferentes valores de α . Em $\alpha=1$ vê-se apenas mandrill; em $\alpha=0$, apenas o leopardo; valores intermediários fundem os olhares das duas imagens proporcionalmente.

3.2.4 Operações Lógicas e Máscaras Bit a Bit

As operações lógicas bit a bit (AND, OR e NOT) atuam diretamente sobre os bits de cada pixel e são a base para criação e aplicação de **máscaras** (*masks*) — imagens binárias com apenas 0 (preto) e 255 (branco) usadas para isolar **Regiões de Interesse (ROI)**.

O comportamento de cada operação decorre da representação binária do 255 (11111111) e do 0 (00000000):

- **AND** com a máscara: onde $m = 255$, os bits originais são preservados; onde $m = 0$, o pixel é zerado. Resultado: recorte da ROI.
- **OR** com a máscara: onde $m = 255$, o pixel é forçado a branco; onde $m = 0$, o valor original é mantido. Resultado: iluminação da ROI.
- **NOT** (sem máscara): inverte todos os bits ($g = 255 - f$), produzindo o negativo fotográfico da imagem.

A Figure 3.5 ilustra as três operações aplicadas à imagem do mandrill com uma máscara circular.

```
import cv2
h, w = img_gray.shape

# Máscara circular centrada na imagem
mask_circ = np.zeros((h, w), dtype=np.uint8)
cv2.circle(mask_circ, (w//2, h//2), min(h, w)//3 - 10, 255, -1)

# Operações via morph
img_not = mm.bnot(img_gray) # NOT: negativo fotográfico
img_and = mm.band(img_gray, mask_circ) # img_gray & mask_circ: preserva apenas a ROI circular
img_or = mm.bor(img_gray, mask_circ) # img_gray | mask_circ: ilumina a região da máscara

mm.show(
    [img_gray, img_and, img_or, img_not],
```



```
titles=["Original", "AND (ROI circular)", "OR (ilumina ROI)", "NOT (negativo)"],
cols=4
)
```



Figura 3.5: Operações lógicas bit a bit com máscara circular: AND (isolamento da ROI), OR (iluminação da ROI) e NOT (negativo).

3.3 Histograma de Imagens

O **histograma** de uma imagem em tons de cinza é uma função discreta que descreve a distribuição de frequências das intensidades:

$$h(r_k) = n_k, \quad k = 0, 1, \dots, L - 1 \quad (3.5)$$

onde r_k é o k -ésimo nível de intensidade, n_k é o número de pixels com essa intensidade e L é o total de níveis (tipicamente 256 para 8 bits). O histograma normalizado estima a probabilidade de cada nível:

$$p(r_k) = \frac{n_k}{MN} \quad (3.6)$$

onde MN é o total de pixels. Por ser uma **estatística global**, o histograma não carrega informação posicional, mas revela características essenciais como brilho médio, contraste e distribuição tonal. Na prática, `mm.hist(img)` retorna o vetor de contagens $h(r_k)$, que serve tanto para visualização (via `mm.histImg`) quanto para cálculos como **função de distribuição acumulada (CDF)** e equalização.

i Interpretação do Histograma

- **Estreito à esquerda:** imagem subexposta (escura).
- **Estreito à direita:** imagem superexposta (clara).
- **Concentrado no centro:** baixo contraste.
- **Distribuído por toda a faixa:** alto contraste, boa utilização dos tons disponíveis.

A Figure 3.6 apresenta o histograma da imagem do mandrill, bem como versões escurecida (`mm.subm`) e clareada (`mm.addm`). Observa-se o deslocamento da distribuição de intensidades para a esquerda e para a direita, respectivamente. Note que o intervalo representado no eixo x não corresponde necessariamente a toda a faixa de 0 a 255.

```
img_dark = mm.subm(img_gray, 80)
img_high = mm.addm(img_gray, 80)

images = [img_gray, img_dark, img_high]
titles = ["Original", "Escurecida (-80)", "Clareada (+80)"]

def hist_title(img, t):
    H = mm.hist(img)
    n0 = int(H[0])
    n255 = int(H[255]) if len(H) > 255 else 0
    return f"Histograma - {t}\n0:{n0:},px 255:{n255:},px"
```



```
mm.show(
    images + [mm.histImg(img) for img in images],
    titles=titles + [hist_title(img, t) for img, t in zip(images, titles)],
    rows=2, cols=3,
    figsize=(14, 8)
)
```

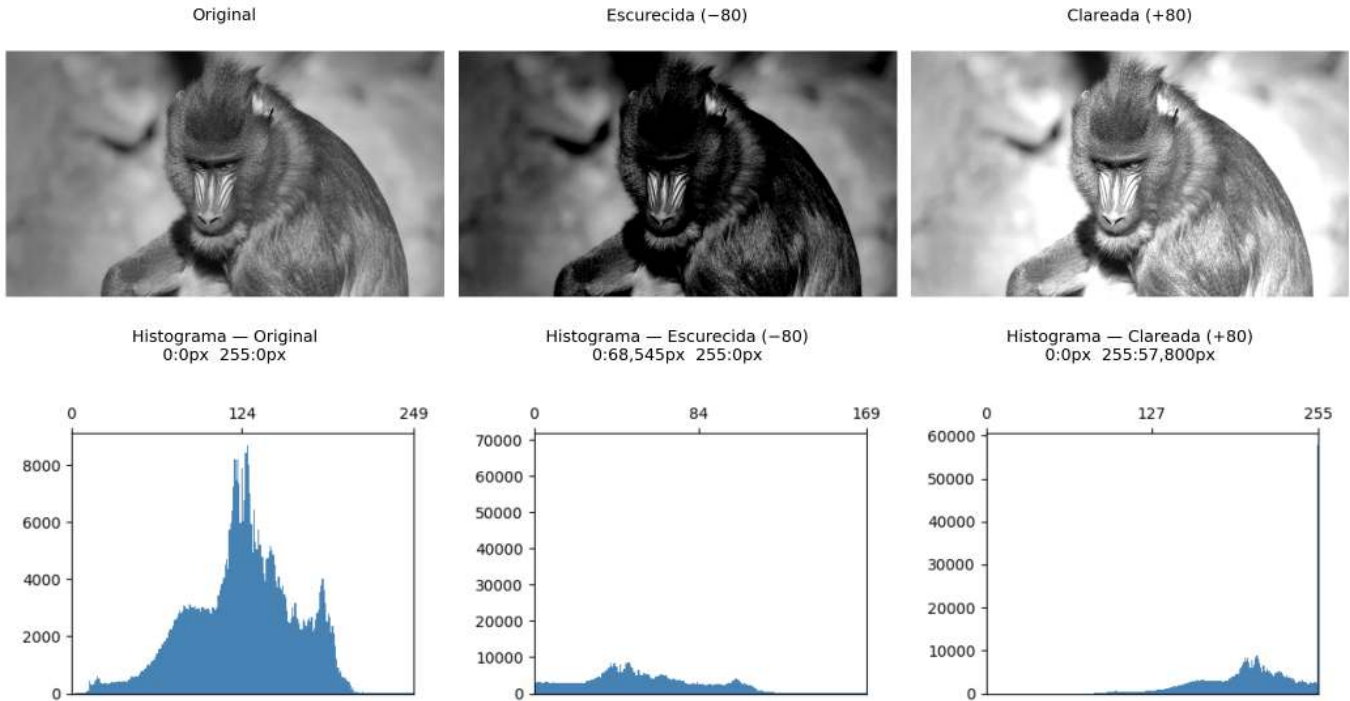


Figura 3.6: Histogramas da imagem original, de uma versão escurecida e de uma versão clareada. Os valores na segunda linha indicam a quantidade de pixels saturados em 0 (preto) e 255 (branco).

3.3.1 Equalização de Histograma

A **equalização de histograma** redistribui as intensidades para que o histograma resultante seja o mais uniforme possível. O mapeamento é dado pela **função de distribuição acumulada (CDF)**:

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p(r_j) = \frac{L - 1}{MN} \sum_{j=0}^k n_j \quad (3.7)$$

A transformação é monotônica: níveis frequentes recebem intervalos maiores no domínio de saída (maior separação → mais contraste), enquanto níveis raros são comprimidos.

O algoritmo completo, em cinco etapas, é apresentado na Table 3.1.

Tabela 3.1: Algoritmo de equalização de histograma.

Etapa	Operação	Fórmula
1	Histograma	$h[k] \leftarrow$ número de pixels com intensidade k , $k = 0 \dots L - 1$
2	Probabilidade	$p[k] \leftarrow h[k]/MN$
3	CDF	$\text{cdf}[k] \leftarrow \sum_{j=0}^k p[j]$ (soma acumulada)
4	Look-Up Table (mapeamento)	$\text{lut}[k] \leftarrow \text{round}(\text{cdf}[k] \times (L - 1))$

Etapa	Operação	Fórmula
5	Aplicação	$g[i, j] \leftarrow \text{lut}[f[i, j]]$ (para todo pixel)

Note na Figure 3.7 que a equalização **redistribui** os tons existentes para posições mais espaçadas na faixa $[0, L - 1]$, mas não cria novos tons — a imagem equalizada continua com exatamente 3 tons distintos, agora em $\{1, 5, 7\}$ em vez de $\{2, 3, 4\}$.

```
img5 = np.array([[3, 4, 2, 3, 4],
                 [4, 3, 3, 4, 3],
                 [2, 3, 4, 3, 2],
                 [3, 4, 3, 2, 3],
                 [4, 3, 2, 3, 4]], dtype=np.uint8)
L = 8 # 3 bits: níveis 0..7

# 1. Histograma com tamanho garantido até L
h = mm.hist(img5, 3) # B=3 → 2³=8 níveis, tamanho garantido

p = h / h.sum() # 2. Probabilidade de cada nível p[k] = h[k] / MN

cdf = np.cumsum(p) # 3. CDF normalizada ou
# cdf = np.zeros(len(p))
# cdf[0] = p[0]
# for k in range(1, len(p)):
#     cdf[k] = cdf[k-1] + p[k] # cdf[k] = Σ p[j], j=0..k

lut = np.round(cdf * (L - 1)).astype(np.uint8) # 4. LUT: mapeamento para [0, L-1]

img5_eq = lut[img5] # 5. Aplica LUT pixel a pixel ou, equivalente a:
# l, c = img5.shape
# img5_eq = np.zeros((l, c), dtype=np.uint8)
# for i in range(l):
#     for j in range(c):
#         img5_eq[i, j] = lut[img5[i, j]] # g[i,j] = lut[f[i,j]]

print("Imagem original (5×5, 3 bits):")
print(img5)
print()
header = f"{'k':>3} {'h[k]':>6} {'p[k]':>7} {'CDF[k]':>8} {'lut[k]':>7}"
print(header)
print("-" * len(header))
for k in range(L):
    print(f"{k:>3} {h[k]:>6} {p[k]:>7.4f} {cdf[k]:>8.4f} {lut[k]:>7}")
print()
print("Imagem equalizada (5×5):")
print(img5_eq)

# Conta tons distintos
tons_orig = len(np.unique(img5))
tons_eq = len(np.unique(img5_eq))

mm.show(
    [img5, img5_eq, mm.histImg(img5, L-1), mm.histImg(img5_eq, L-1)],
    titles=[f"Original ({tons_orig} tons: {sorted(np.unique(img5).tolist())})",
           f"Equalizada ({tons_eq} tons: {sorted(np.unique(img5_eq).tolist())})",
           "Histograma - Original",
           "Histograma - Equalizada"],
    rows=2, cols=2, figsize=(5, 4), dpi=100
```

Imagem original (5×5, 3 bits):

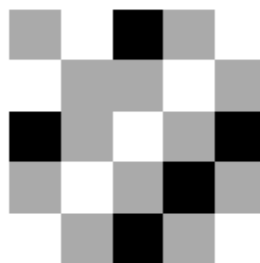
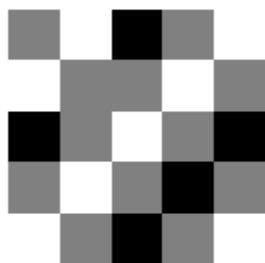
```
[[3 4 2 3 4]
 [4 3 3 4 3]
 [2 3 4 3 2]
 [3 4 3 2 3]
 [4 3 2 3 4]]
```

k	h[k]	p[k]	CDF[k]	lut[k]
0	0	0.0000	0.0000	0
1	0	0.0000	0.0000	0
2	5	0.2000	0.2000	1
3	12	0.4800	0.6800	5
4	8	0.3200	1.0000	7
5	0	0.0000	1.0000	7
6	0	0.0000	1.0000	7
7	0	0.0000	1.0000	7

Imagem equalizada (5×5):

```
[[5 7 1 5 7]
 [7 5 5 7 5]
 [1 5 7 5 1]
 [5 7 5 1 5]
 [7 5 1 5 7]]
```

Original (3 tons: [2, 3, 4]) Equalizada (3 tons: [1, 5, 7])



Histograma — Original

Histograma — Equalizada

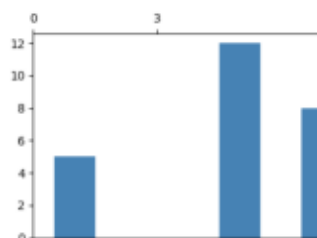
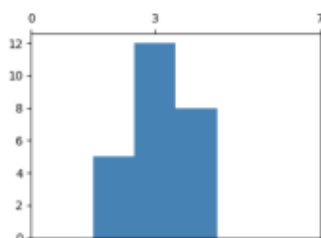


Figura 3.7: Equalização de histograma em imagem 5×5 com 3 bits ($L=8$): imagem original com tons concentrados em $\{2,3,4\}$, imagem equalizada com tons redistribuídos para $\{1,5,7\}$, e os respectivos histogramas evidenciando o espalhamento das frequências.

Limitação: a equalização global pode super-realçar ruídos e produzir contraste excessivo em regiões homogêneas. O **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*) reduz esse problema ao aplicar a equalização em blocos locais (*tiles*) e limitar a altura dos picos do histograma antes da equalização.

A Figure 3.8 compara a imagem original, a equalização global via `mm.equalize` e o CLAHE do OpenCV, exibindo também os histogramas resultantes. Diferentemente da equalização global, que utiliza uma única transformação baseada na CDF de toda a imagem, o CLAHE adapta o contraste a cada região, sendo particularmente útil em imagens com iluminação não uniforme.

No exemplo, foi utilizado `clipLimit=2.0` e `tileGridSize=(32,32)`. O parâmetro `clipLimit` define o quanto os picos do histograma local podem crescer antes de serem cortados (*clipped*). No OpenCV, esse valor é um fator relativo: o limite real é aproximadamente calculado como `clipLimit × (número de pixels do bloco / número de níveis de cinza)`. Por exemplo, em um bloco com 4096 pixels e uma imagem de 8 bits (256 níveis de cinza), a frequência média por nível é $4096/256 = 16$. Assim, `clipLimit=2.0` permite picos de aproximadamente $2 \times 16 = 32$ ocorrências antes do corte. As ocorrências excedentes não são descartadas: elas são redistribuídas entre os demais níveis de cinza do histograma, reduzindo a concentração excessiva em poucos níveis e evitando uma amplificação exagerada do contraste local. Valores menores limitam mais o contraste e reduzem a amplificação de ruído, enquanto valores maiores permitem um realce mais intenso, mas podem introduzir artefatos.

- `clipLimit=1.0`: realce suave e conservador;
- `clipLimit=2.0`: bom equilíbrio entre contraste e naturalidade;
- `clipLimit=4.0`: maior destaque para detalhes locais;
- `clipLimit=8.0`: contraste agressivo, com possível amplificação de ruído.

Assim, o CLAHE costuma produzir resultados mais naturais do que a equalização global, especialmente em imagens com sombras, reflexos ou iluminação desigual.

```
img_eq    = mm.equalize(img_gray)
clahe     = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(32, 32))
img_clahe = clahe.apply(img_gray)

images = [img_gray, img_eq, img_clahe]
titles = ["Original", "mm.equalize (CDF)", "CLAHE"]

mm.show(
    images + [mm.histImg(img) for img in images],
    titles=titles + [f"Histograma - {t}" for t in titles],
    rows=2, cols=3,
    figsize=(14, 8)
)
```

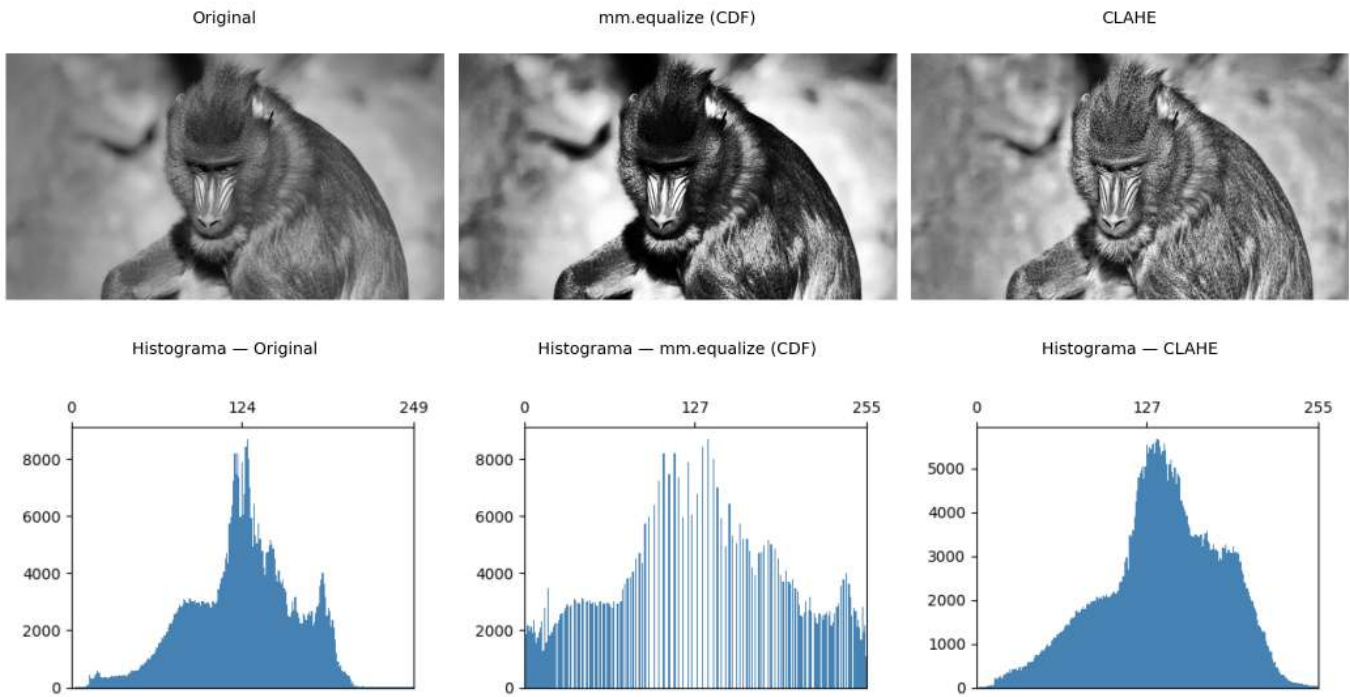


Figura 3.8: Equalização de histograma: mm.equalize (global via CDF) vs. CLAHE (adaptativa com limitação de contraste). Os histogramas revelam o espalhamento progressivo das intensidades.

3.3.2 Especificação de Histograma

Enquanto a equalização impõe uma distribuição uniforme, a **especificação de histograma** (*histogram matching*) permite que o histograma da imagem de saída siga uma distribuição **arbitrária** — por exemplo, o histograma de outra imagem de referência.

O procedimento envolve três etapas:

1. Calcular a CDF da imagem de entrada: $P_r(r_k)$.
2. Calcular a CDF da imagem de referência: $P_z(z_k)$.
3. Para cada nível r_k , encontrar o nível z que minimiza $|P_z(z) - P_r(r_k)|$.

$$T(r_k) = \arg \min_z |P_z(z) - P_r(r_k)| \quad (3.8)$$

Na Figure 3.9, transferimos o perfil tonal do leopardo (Figure 3.3) para a imagem do mandrill — uma aplicação direta do conceito visto no *blending*: em vez de fundir pixels, aqui fundimos distribuições tonais.

```
img_leop_gray = mm.gray(img_numpy)

def hist_specify(src, ref):
    """Mapeia src para o perfil tonal de ref via CDF."""
    cdf_src = np.cumsum(mm.hist(src) / src.size)
    cdf_ref = np.cumsum(mm.hist(ref) / ref.size)
    lut = np.array([np.argmin(np.abs(cdf_ref - v)) for v in cdf_src], dtype=np.uint8)
    return lut[src]

img_spec = hist_specify(img_gray, img_leop_gray)

images = [img_gray, img_leop_gray, img_spec]
titles = ["Mandrill (original)", "Leopardo (referência)", "Mandrill → perfil do leopardo"]

mm.show()
```

```

images + [mm.histImg(img) for img in images],
titles=titles + [f"Histograma - {t}" for t in titles],
rows=2, cols=3,
figsize=(12, 9)
)

```

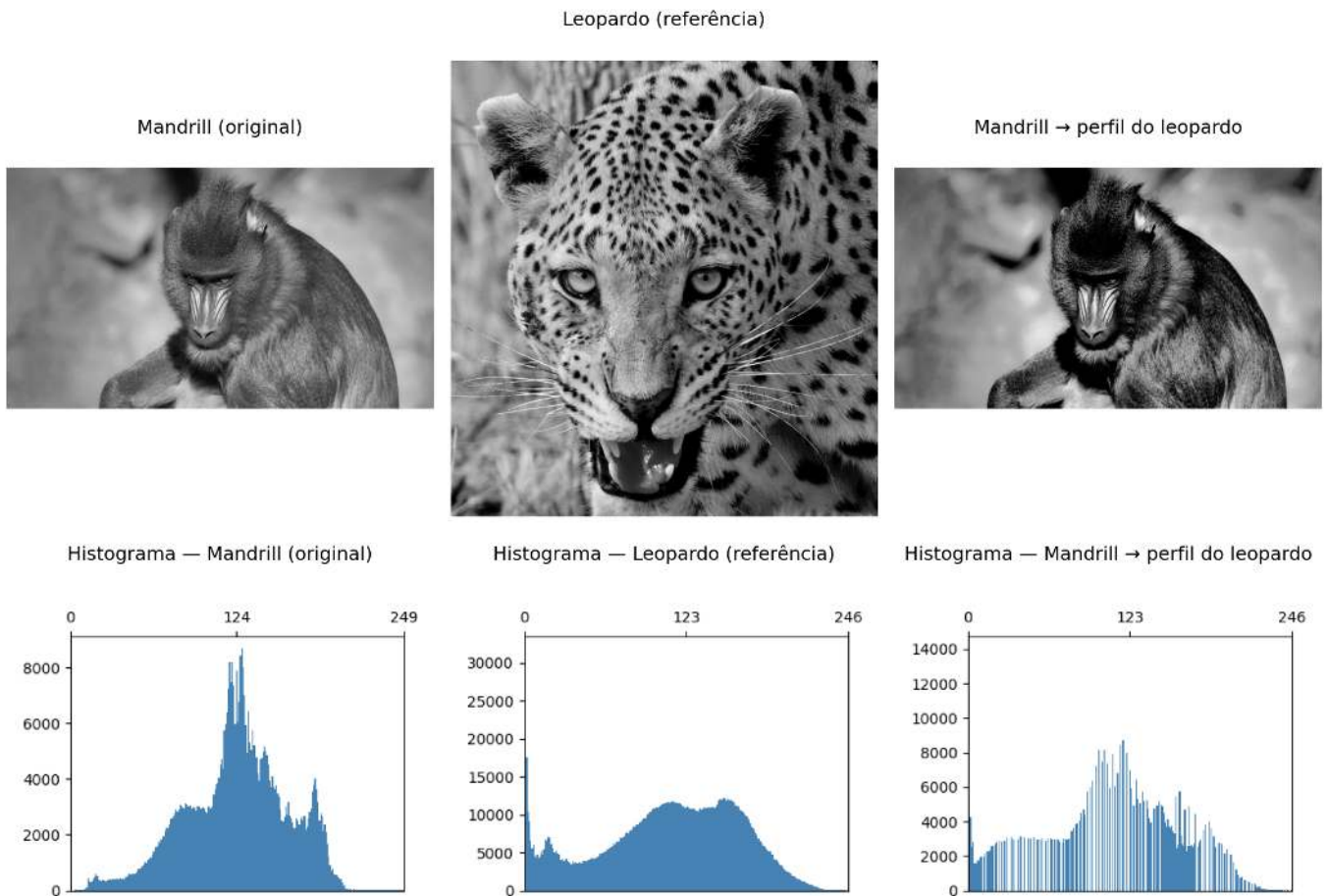


Figura 3.9: Especificação de histograma: mandril mapeada para o perfil tonal do leopardo. A CDF da saída aproxima a CDF de referência.

3.4 Fundamentos Espaciais: Vizinhança, Convolução e *Kernels*

As operações de filtragem espacial não atuam em um único pixel isolado, mas em uma **vizinhança** ao seu redor. Para isso, utiliza-se uma pequena matriz de coeficientes denominada **kernel** (ou máscara), que percorre toda a imagem por meio de uma **janela deslizante** (*sliding window*).

As janelas mais comuns são 3×3 , 5×5 e 7×7 . Em uma janela 3×3 , por exemplo, o pixel central é processado juntamente com seus oito vizinhos imediatos. Em cada posição da janela, os valores dos pixels são combinados com os coeficientes do *kernel*, produzindo um novo valor para o pixel central.

3.4.1 Vizinhança

Considere uma janela 3×3 centrada no pixel (x, y) :

$$\begin{bmatrix} (x-1, y-1) & (x, y-1) & (x+1, y-1) \\ (x-1, y) & (x, y) & (x+1, y) \\ (x-1, y+1) & (x, y+1) & (x+1, y+1) \end{bmatrix} \quad (3.9)$$

De forma geral, uma janela de tamanho $(2a + 1) \times (2b + 1)$ abrange todos os pixels situados até a posições na horizontal e até b posições na vertical em relação ao pixel central. Assim, uma janela 3×3 corresponde a $a = b = 1$, uma janela 5×5 a $a = b = 2$, e assim por diante.

Matematicamente, a vizinhança é definida por

$$\mathcal{V}(x, y) = \{(x + s, y + t) : -a \leq s \leq a, -b \leq t \leq b\} \quad (3.10)$$

3.4.2 Tratamento de Bordas

Pixels próximos às bordas possuem parte de sua vizinhança fora da imagem. Para aplicar filtros nessas regiões, é necessário definir como os valores externos serão obtidos. As estratégias mais comuns são:

- **Zero-padding** (BORDER_CONSTANT): completa a região externa com zeros.
- **Replicação** (BORDER_REPLICATE): repete os valores da borda.
- **Reflexão** (BORDER_REFLECT_101): espelha os pixels vizinhos à borda.

O OpenCV utiliza, por padrão, BORDER_REFLECT_101, pois essa estratégia preserva melhor a continuidade dos níveis de cinza e reduz artefatos introduzidos pelo tratamento das bordas.

O exemplo a seguir compara os três métodos usando uma matriz 3×3 . Observe como cada estratégia preenche os pixels externos necessários para que filtros 3×3 possam ser aplicados também nos cantos da imagem.

```
import cv2
import numpy as np

img = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
], dtype=np.uint8)

bordas = {
    "CONSTANT (zero-padding)": cv2.BORDER_CONSTANT,
    "REPLICATE": cv2.BORDER_REPLICATE,
    "REFLECT_101": cv2.BORDER_REFLECT_101
}

print("Imagem original:")
print(mm.drawImg(img))

for k in [3, 5]:
    b = k // 2

    print(f"=== Kernel {k}x{k} ===")

    for nome, tipo in bordas.items():
        pad = cv2.copyMakeBorder(
            img, b, b, b, b,
            tipo,
            value=0
        )

        print(f"{nome}")
        print(mm.drawImg(pad))
```

```
Imagem original:
1 2 3
4 5 6
7 8 9
```

```

=== Kernel 3x3 ===
CONSTANT (zero-padding)
0 0 0 0 0
0 1 2 3 0
0 4 5 6 0
0 7 8 9 0
0 0 0 0 0

REPLICATE
1 1 2 3 3
1 1 2 3 3
4 4 5 6 6
7 7 8 9 9
7 7 8 9 9

REFLECT_101
5 4 5 6 5
2 1 2 3 2
5 4 5 6 5
8 7 8 9 8
5 4 5 6 5

=== Kernel 5x5 ===
CONSTANT (zero-padding)
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 1 2 3 0 0
0 0 4 5 6 0 0
0 0 7 8 9 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

REPLICATE
1 1 1 2 3 3 3
1 1 1 2 3 3 3
1 1 1 2 3 3 3
4 4 4 5 6 6 6
7 7 7 8 9 9 9
7 7 7 8 9 9 9
7 7 7 8 9 9 9

REFLECT_101
9 8 7 8 9 8 7
6 5 4 5 6 5 4
3 2 1 2 3 2 1
6 5 4 5 6 5 4
9 8 7 8 9 8 7
6 5 4 5 6 5 4
3 2 1 2 3 2 1

```

Observe que o resultado de um filtro pode variar significativamente conforme o tratamento adotado para as bordas da imagem.

Na biblioteca didática `morph.py`, os algoritmos implementados diretamente em Python (sem utilizar OpenCV ou *scikit-image*) não utilizam *padding*. Para cada pixel, em geral, os elementos da vizinhança são examinados individualmente, e apenas os vizinhos cujas coordenadas pertencem ao domínio da imagem participam do cálculo. Assim, nas regiões próximas às bordas, a vizinhança efetiva pode conter menos pixels do que a janela especificada.

3.4.3 Correlação e Convolução

Existem dois mecanismos matematicamente relacionados:

Correlação cruzada (*cross-correlation*) — o *kernel* é aplicado diretamente:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t) \quad (3.11)$$

Convolução bidimensional — o *kernel* é rotacionado 180° antes da aplicação:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t) \quad (3.12)$$

Para *kernels* simétricos (Gaussiano, Laplaciano, média) as duas operações produzem resultados idênticos. Para *kernels* assimétricos (Sobel, Prewitt) a diferença é significativa.

3.4.4 Correlação vs. Convolução no OpenCV

`cv2.filter2D` implementa **correlação**. Para convolução verdadeira com *kernel* assimétrico, rotacione o *kernel* 180° (`np.rot90(w, 2)`) antes de passar para `filter2D`.

3.4.4.1 Correlação (*OpenCV*: `cv2.filter2D`)

```
import cv2
import numpy as np

# imagem 4x4
img = np.array([
    [ 1,  2,  3,  4],
    [ 5,  6,  7,  8],
    [ 9, 10, 11, 12],
    [13, 14, 15, 16]
], dtype=np.float32)

# kernel assimétrico
w = np.array([
    [0, 1, 2],
    [0, 0, 0],
    [0, 0, 0]
], dtype=np.float32)

corr = cv2.filter2D(
    img, -1, w, # -1 indica que o tipo da saída é o mesmo da entrada
    borderType=cv2.BORDER_CONSTANT
)

print("Imagem original:")
print(mm.drawImg(img))

print("Kernel:")
print(mm.drawImg(w))

print("Resultado da correlação:")
print(mm.drawImg(corr))
```

Imagem original:

1	2	3	4
---	---	---	---

```

5   6   7   8
9  10  11  12
13 14  15  16

```

Kernel:

```

0  1  2
0  0  0
0  0  0

```

Resultado da correlação:

```

0   0   0   0
5   8  11   4
17  20  23   8
29  32  35  12

```

O método `cv2.filter2D` realiza correlação, isto é, aplica o *kernel* exatamente na orientação fornecida.

3.4.4.2 Convolução

```

import cv2
import numpy as np

# mesmo kernel
w_conv = np.rot90(w, 2)

conv = cv2.filter2D(
    img, -1, w_conv,
    borderType=cv2.BORDER_CONSTANT
)

print("Imagem original:")
print(mm.drawImg(img))

print("Kernel original:")
print(mm.drawImg(w))

print("Kernel rotacionado 180°:")
print(mm.drawImg(w_conv))

print("Resultado da convolução:")
print(mm.drawImg(conv))

```

Imagem original:

```

1   2   3   4
5   6   7   8
9  10  11  12
13 14  15  16

```

Kernel original:

```

0  1  2
0  0  0
0  0  0

```

Kernel rotacionado 180°:

```

0  0  0
0  0  0
2  1  0

```

Resultado da convolução:

```

5  16  19  22

```

9	28	31	34
13	40	43	46
0	0	0	0

A convolução utiliza o *kernel* rotacionado em 180°. Por isso, para reproduzir a definição matemática de convolução usando `cv2.filter2D`, é necessário aplicar previamente `np.rot90(w, 2)`.

Quando o *kernel* é simétrico (por exemplo, filtros de média ou Gaussianos), correlação e convolução produzem o mesmo resultado. A diferença aparece apenas para *kernels* assimétricos, como o exemplo apresentado.

3.4.5 O Papel do *Kernel*

Os coeficientes do *kernel* determinam completamente o efeito produzido pelo filtro, conforme resumido na Table 3.2.

Tabela 3.2: Interpretação típica dos coeficientes do *kernel*.

Característica	Efeito típico
Coeficientes positivos com soma 1	Suavização (<i>passa-baixa</i>)
Soma igual a 0, com valores positivos e negativos	Detecção de bordas (<i>passa-alta</i>)
Coeficiente central positivo dominante e vizinhos negativos	Realce de nitidez
Coeficientes assimétricos	Gradiente direcional

Exemplos:

Suavização:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Detecção de bordas:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Realce de nitidez:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Gradiente direcional (Sobel):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

A Figure 3.10 demonstra o mecanismo passo a passo: para cada posição da janela, multiplica-se cada coeficiente do *kernel* pelo pixel correspondente da vizinhança e somam-se os produtos obtidos. O resultado é exatamente o valor definido pela Equation 3.11 para aquela posição da imagem. Embora as imagens produzidas por `mm.conv0` e `cv2.filter2D` (ou `mm.conv`) sejam visualmente muito semelhantes, a implementação baseada em OpenCV é milhares de vezes mais rápida, como mostrado a seguir.

⚠ Desempenho: laços Python vs. operações vetorizadas

A função `mm.conv0` implementa a correlação diretamente em Python por meio de laços aninhados. Embora essa abordagem seja adequada para fins didáticos, ela executa um grande número de operações e torna-se lenta para imagens maiores.

Já `mm.conv` utiliza `cv2.filter2D`, implementado em C++ e otimizado para operações matriciais. No exemplo apresentado, a versão vetorizada foi mais de **3000 vezes mais rápida** que a implementação didática, produzindo um resultado visualmente equivalente.

As diferenças numéricas observadas concentram-se principalmente nas bordas da imagem. Em `mm.conv0`, os pixels da borda permanecem inalterados, enquanto `mm.conv` utiliza uma estratégia de reflexão das bordas (`cv2.BORDER_REFLECT_101`, padrão do `cv2.filter2D`).

Por isso, `mm.conv0` deve ser utilizado para compreender o algoritmo, enquanto `mm.conv` é a opção recomendada para aplicações práticas.

```
import time

w_mean = np.ones((3,3), dtype=np.float32) / 9.0
img_gray = img_leop_gray
# Demonstração numérica em patch 5x5
patch = img_gray[250:255, 250:255].astype(np.float32)
roi = patch[0:3, 0:3]
print("Patch 5x5 (intensidades):")
print(patch.astype(np.int32))
print(f"\nKernel 3x3 de média:\n{w_mean}")
print(f"\nCorrelação no pixel central [1,1]: \
      {(w_mean * roi).sum():.1f} (original: {patch[1,1]:.0f})")

# Comparação de tempo na imagem completa
t0 = time.perf_counter()
img_conv0 = mm.conv0(img_gray, w_mean)
t_conv0 = time.perf_counter() - t0

t0 = time.perf_counter()
img_conv = mm.conv(img_gray, w_mean)
t_conv = time.perf_counter() - t0

diff = np.abs(img_conv0.astype(int) - img_conv.astype(int)).max()
print(f"\nTempos na imagem {img_gray.shape[0]}x{img_gray.shape[1]}:")
print(f"  mm.conv0 (laços Python): {t_conv0:.3f} s")
print(f"  mm.conv (cv2.filter2D): {t_conv:.5f} s")
print(f"  Aceleração: {t_conv0/t_conv:.0f}x mais rápido")
print(f"  Diferença máxima: {diff} (bordas com padding diferente)")

mm.show(
    [img_gray, img_conv0, img_conv],
    titles=["Original", f"conv0 ({t_conv0:.2f}s)", f"conv ({t_conv:.4f}s)"],
    cols=3
)
```

Patch 5x5 (intensidades):

```
[[ 91  95 108 116 114]
 [106 107 108 111 114]
 [102 103 107 112 116]
 [ 83  90 111 125 123]
 [ 79  88 110 126 124]]
```

Kernel 3x3 de média:

```
[[0.11111111 0.11111111 0.11111111]]
```



```
[0.11111111 0.11111111 0.11111111]
[0.11111111 0.11111111 0.11111111]]
```

Correlação no pixel central [1,1]: 103.0 (original: 107)

Tempos na imagem 1324x1242:

mm.conv0 (laços Python): 5.963 s

mm.conv (cv2.filter2D): 0.00213 s

Aceleração: 2796× mais rápido

Diferença máxima: 39 (bordas com padding diferente)

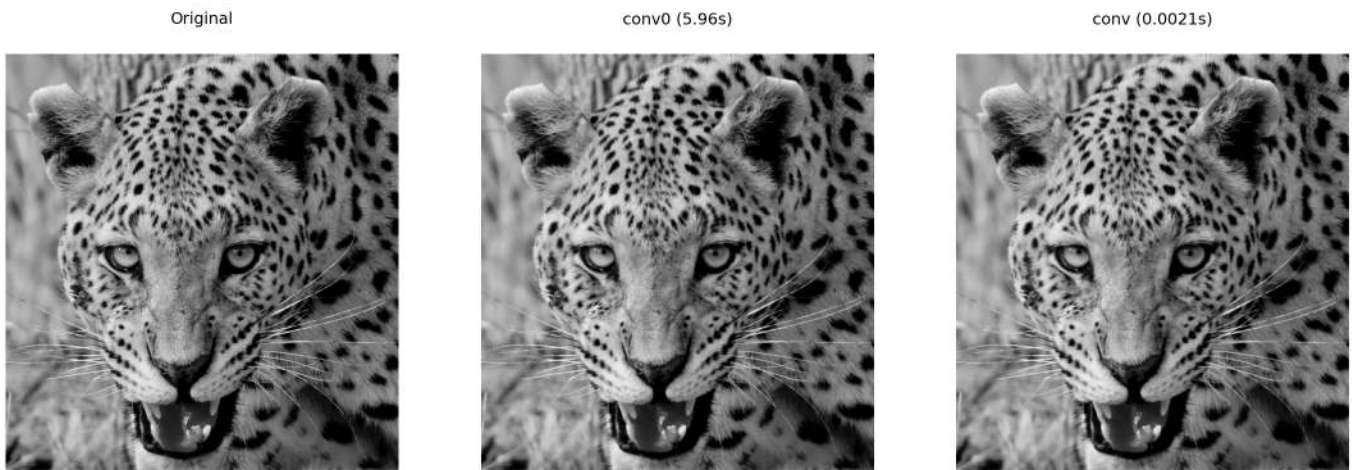


Figura 3.10: Correlação com *kernel* de média 3×3: versão didática (mm.conv0) vs. vetorizada (mm.conv via cv2.filter2D). A diferença máxima de 39 ocorre nas bordas.

3.4.6 Exemplo Numérico: Correlação Passo a Passo

Para tornar concreto o mecanismo da Equation 3.11, considere o *kernel* de média 3×3 ($a = b = 1$, todos os coeficientes = $1/9 \approx 0,111$) aplicado ao *patch* 5×5 extraído da imagem do leopardo. A Figure 3.11 exhibe o *patch* com grade e destaca em amarelo a janela 3×3 centrada no pixel [1,1]:

```
patch = img_gray[250:255, 250:255]
B      = np.ones((3,3), dtype=np.uint8)

print("Patch 5x5 (intensidades):")
print(mm.drawImg(patch))

mm.drawImgKernel(patch, B, x=1, y=1, scale=40.0)
```

```
Patch 5x5 (intensidades):
 91  95 108 116 114
106 107 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

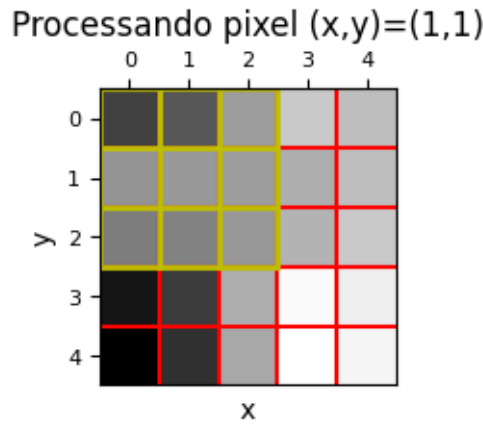


Figura 3.11: *Patch* 5×5 extraído da imagem do leopardo (posição [250:255, 250:255]). A janela amarela destaca a vizinhança 3×3 centrada no pixel [1,1] onde a correlação será calculada.

Para ilustrar o cálculo da correlação, considere o pixel na posição [1, 1] do *patch* 5×5 mostrado na Figure 3.11. Essa posição foi escolhida apenas por conveniência didática, pois possui uma vizinhança 3×3 completa ao seu redor.

Os valores dessa vizinhança correspondem à submatriz superior esquerda do *patch*:

$$\text{vizinhança} = \begin{bmatrix} 91 & 95 & 108 \\ 106 & 107 & 108 \\ 102 & 103 & 107 \end{bmatrix}$$

Aplicando a Equation 3.11 com o kernel de média:

$$g[1, 1] = \frac{91 + 95 + 108 + 106 + 107 + 108 + 102 + 103 + 107}{9} = \frac{927}{9} = 103$$

O resultado (103) é ligeiramente menor que o valor original do pixel central (107), pois a média incorpora vizinhos de menor intensidade, produzindo o efeito de suavização. Na prática, o algoritmo inicia o processamento em [0, 0] e repete esse mesmo cálculo para cada posição da imagem, deslocando a janela até cobrir todo o domínio.

3.5 Filtragem Espacial de Suavização

Os filtros de suavização (*smoothing filters*) atenuam variações bruscas de intensidade, reduzindo ruído e detalhes de alta frequência. São filtros **passa-baixa** — preservam as componentes de baixa frequência (estruturas grandes) e atenuam as de alta frequência (ruído, bordas).

3.5.1 Filtro de Média (*Box Filter*)

O filtro de média utiliza um *kernel* uniforme de tamanho $n \times n$, onde todos os coeficientes valem $1/n^2$:

$$w_{\text{média}} = \frac{1}{n^2} \begin{bmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{bmatrix}_{n \times n} \quad (3.13)$$

Cada pixel de saída é a média aritmética dos n^2 pixels de sua vizinhança. Note que a soma dos coeficientes é sempre 1 — o brilho médio da imagem é preservado. *Kernels* maiores produzem suavização mais agressiva, mas borram progressivamente as bordas.

A Figure 3.12 mostra o efeito do filtro de média com *kernels* 3×3 , 7×7 e 15×15 sobre um detalhe da imagem do leopardo. Os resultados foram obtidos com `mm.blur`, que implementa o filtro de média por meio da função `cv2.blur`, equivalente à convolução da imagem com um *kernel* uniforme cujos coeficientes são $h(x,y) = 1/N^2$; de forma equivalente, o mesmo resultado pode ser obtido com `mm.conv`, calculando ($g = f * h$). À medida que o *kernel* aumenta, mais pixels contribuem para cada valor de saída, intensificando a suavização, reduzindo o ruído e tornando detalhes finos e bordas progressivamente mais borrados.

```
# Detalhe da região do olho
y0, y1, x0, x1 = 580, 740, 680, 900
crop = lambda img: img[y0:y1, x0:x1]

img_gray_crop = crop(img_gray)

sizes = [3, 7, 15]
imgs = [img_gray_crop] + \
    [mm.blur(img_gray_crop, k) for k in sizes] # ou
    #[mm.conv(img_gray_crop, np.ones((k,k), dtype=np.float32)/(k*k)) for k in sizes]
titles = ["Original"] + [f"Média {k}x{k}" for k in sizes]

mm.show(imgs, titles=titles, cols=4)
```

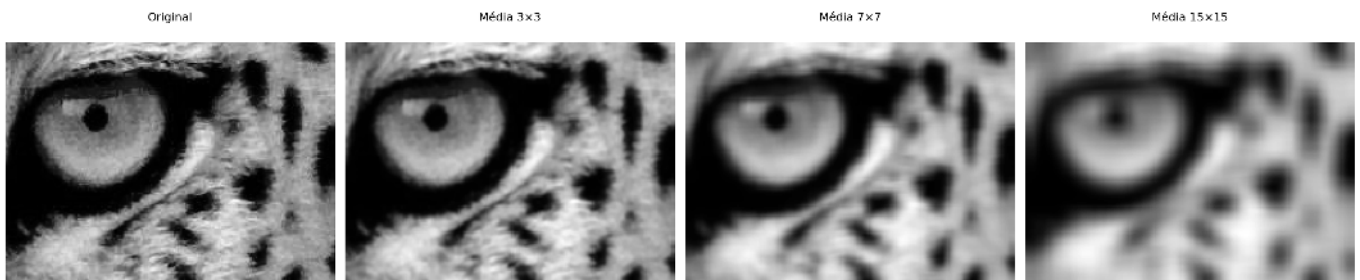


Figura 3.12: Filtro de média com *kernels* de tamanho crescente (3×3 , 7×7 , 15×15). O borramento das bordas aumenta com o tamanho do *kernel*.

3.5.2 Filtro Gaussiano

O filtro Gaussiano pesa os pixels da vizinhança de acordo com uma função Gaussiana bidimensional:

$$G(s, t) = \frac{1}{2\pi\sigma^2} e^{-\frac{s^2+t^2}{2\sigma^2}} \quad (3.14)$$

onde σ é o desvio padrão e controla o raio de influência. Pixels mais próximos do centro têm peso maior; pixels distantes são progressivamente ignorados.

A Figure 3.13 apresenta o *kernel* Gaussiano 5×5 gerado para $\sigma = 1$. O *kernel* foi construído a partir do produto externo de dois vetores Gaussianos unidimensionais e posteriormente normalizado para que a soma de seus coeficientes seja igual a 1. Observa-se que os maiores pesos concentram-se no centro da matriz, decrescendo radialmente em direção às bordas. Essa distribuição faz com que os pixels centrais tenham maior influência no resultado da filtragem, contribuindo para uma suavização mais natural e com melhor preservação de bordas do que o filtro de média.

```
# Gera kernel Gaussiano 5x5 via cv2
k_gauss = cv2.getGaussianKernel(5, 1)
w_gauss = k_gauss @ k_gauss.T          # @ => multiplicação matricial: G(s,t) = G(s)·G(t)
w_gauss = w_gauss / w_gauss.sum()      # garante soma = 1

print("Kernel Gaussiano 5x5 (=1), normalizado:")
for row in w_gauss:
```

```
print(" " + " ".join(f"{v:.4f}" for v in row))
print(f"\nSoma dos coeficientes: {w_gauss.sum():.6f}")
print(f"Peso central vs. canto: {w_gauss[2,2]:.4f} vs. {w_gauss[0,0]:.4f} "
      f"({w_gauss[2,2]/w_gauss[0,0]:.1f}× maior)")

mm.drawImgPlt((w_gauss * 1000).astype(np.uint8), scale=40)
```

Kernel Gaussiano 5×5 (=1), normalizado:

```
0.0030  0.0133  0.0219  0.0133  0.0030
0.0133  0.0596  0.0983  0.0596  0.0133
0.0219  0.0983  0.1621  0.0983  0.0219
0.0133  0.0596  0.0983  0.0596  0.0133
0.0030  0.0133  0.0219  0.0133  0.0030
```

Soma dos coeficientes: 1.000000

Peso central vs. canto: 0.1621 vs. 0.0030 (54.6× maior)

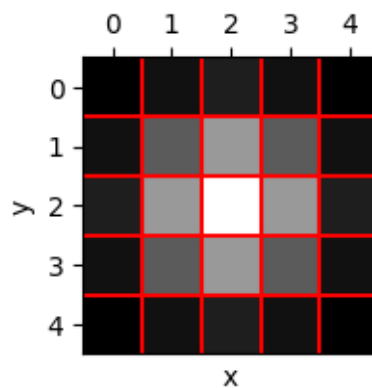


Figura 3.13: *Kernel* Gaussiano 5×5 (=1): pesos normalizados, maiores no centro e decrescentes radialmente. A grade facilita a leitura de cada coeficiente.

i Vantagem computacional da separabilidade

Considere um *kernel* quadrado de tamanho $n \times n$. Se esse filtro for separável (como o Gaussiano), a convolução 2D pode ser decomposta em duas convoluções 1D: uma horizontal e outra vertical. Nesse caso, o custo por pixel passa de aproximadamente $O(n^2)$ operações (convolução 2D direta) para $O(2n)$ operações (duas convoluções 1D). Assim, a complexidade é reduzida de forma significativa, tornando o processamento mais eficiente

Comparado ao filtro de média, o Gaussiano:

- **Preserva melhor as bordas** — a ponderação radial suaviza sem criar transições abruptas;
- **Não introduz anéis** (*ringing*) no domínio da frequência, pois a Gaussiana é sua própria transformada de Fourier (Capítulo 5);
- **É controlado por σ** — aumentar σ equivale a aumentar o raio de suavização de forma contínua e previsível.

A Figure 3.14 compara os filtros de média e Gaussiano aplicados à imagem do leopardo usando uma janela 9×9 . O filtro de média foi implementado por convolução com um *kernel* uniforme, em que todos os 81 pixels da vizinhança possuem o mesmo peso ($1/81$), enquanto o filtro Gaussiano foi obtido com `cv2.GaussianBlur`, utilizando pesos definidos por uma distribuição Gaussiana. Ambos reduzem ruído e suavizam a imagem, porém o filtro Gaussiano preserva melhor as bordas e os detalhes locais, como pode ser observado na região ampliada do olho.

A Figure 3.14 compara os filtros de média e Gaussiano aplicados a um detalhe da imagem do leopardo

com *kernels* 9×9 . O filtro de média foi obtido com `mm.blur`, equivalente à convolução com um *kernel* uniforme cujos coeficientes valem $1/81$, enquanto o filtro Gaussiano foi obtido com `mm.gaussian`, equivalente à convolução com um *kernel* gerado a partir de uma distribuição Gaussiana. Ambos promovem suavização e redução de ruído, porém o filtro Gaussiano atribui maior peso aos pixels centrais da vizinhança, preservando melhor as bordas e os detalhes locais, como pode ser observado na região ampliada do olho.

```
# w_media9 = np.ones((9,9), dtype=np.float32) / 81.0
# img_media9 = mm.conv(img_gray, w_media9) # ou
img_media9 = mm.blur(img_gray, 9)
# img_gauss9 = cv2.GaussianBlur(img_gray, (9,9), 0) # ou
img_gauss9 = mm.gaussian(img_gray, 9, 0)

# Detalhe da região do olho
y0, y1, x0, x1 = 580, 740, 680, 900
crop = lambda img: img[y0:y1, x0:x1]

img_gray_crop = crop(img_gray)

mm.show(
    [crop(img_gray), crop(img_media9), crop(img_gauss9)],
    titles=["Detalhe: Original", "Média 9x9", "Gaussiano 9x9"],
    cols=3, figsize=(12, 8)
)
```



Figura 3.14: Comparação entre filtro de média e Gaussiano (*kernel* 9×9 , $\sigma=0$). O Gaussiano preserva melhor as bordas, visível no detalhe do rosto.

3.6 Filtragem Espacial de Realce

Os filtros de realce (*sharpening filters*) enfatizam transições abruptas de intensidade, aumentando a nitidez e a visibilidade de bordas. São filtros **passa-alta** — amplificam as componentes de alta frequência (bordas, textura) e suprimem as de baixa frequência (regiões uniformes).

A intuição é simples: se subtrairmos de uma imagem sua versão suavizada (que contém apenas as baixas frequências), o que resta são as altas frequências — bordas e detalhes. Somando esse resíduo de volta à imagem original, o contraste local aumenta:

$$g = f + k(f - f_{\text{suave}}), \quad k > 0 \quad (3.15)$$

O Figure 3.15 ilustra esse processo em um sinal 1D sintético com três estruturas distintas: um degrau largo, um pico fino e uma rampa suave. No painel , o sinal original $f(x)$; no , a versão suavizada $f_{\text{suave}}(x)$ obtida por média móvel — note como o pico fino é atenuado. O painel exibe o resíduo $f - f_{\text{suave}}$, que retém apenas as transições abruptas. Por fim, o painel mostra $g(x)$: o pico, antes atenuado, é restaurado e amplificado

em relação ao original. Ajuste k e o tamanho da janela para observar o *trade-off* entre nitidez e amplificação de ruído.

Os filtros de realce formalizam essa ideia diretamente no *kernel*, sem precisar de duas etapas separadas.



Figura 3.15: Simulador: Filtragem Espacial de Realce 1D.

3.6.1 Laplaciano

O Laplaciano é um operador de **segunda derivada** isotrópico, ou seja, responde igualmente a variações em todas as direções, ao contrário de operadores de primeira derivada, como Sobel e Prewitt, que são direcionais:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \quad (3.16)$$

Uma propriedade importante da segunda derivada é que seu valor é **próximo de zero em regiões uniformes** e elevado nas transições de intensidade. Assim, ao subtrair o Laplaciano da imagem original, reforçam-se bordas e detalhes, aumentando o contraste local:

$$g(x, y) = f(x, y) - \nabla^2 f(x, y) \quad (3.17)$$

Na forma discreta, a segunda derivada em x é aproximada por $f(x+1, y) - 2f(x, y) + f(x-1, y)$, e analogamente em y . Somando as duas direções, obtém-se o *kernel* w_4 (4-vizinhos) ou w_8 (8-vizinhos, incluindo diagonais):

$$w_4 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad w_8 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (3.18)$$

i Soma zero e centro negativo

Ambos os *kernels* têm **soma dos coeficientes igual a zero**: em regiões uniformes, a saída é 0 — o Laplaciano não altera o brilho médio, apenas detecta variações. O **centro negativo** indica que o pixel é comparado com seus vizinhos: quanto mais ele se destacar (para cima ou para baixo), maior o valor absoluto do Laplaciano naquele ponto.

No exemplo a seguir, o pixel central $[1, 1] = 107$ possui vizinhos $\{95, 106, 108, 103\}$. Como esses valores são próximos entre si, a região é quase uniforme e o Laplaciano retorna um valor baixo, produzindo pouco realce. Em regiões de borda, onde há diferenças maiores entre o pixel central e seus vizinhos, o Laplaciano assume valores mais elevados (positivos ou negativos), e a operação de Equation 3.17 intensifica essas transições.

A Figure 3.16 ilustra o cálculo do Laplaciano com o *kernel* w_4 em uma vizinhança 3×3 destacada dentro de um *patch* 5×5 . O exemplo mostra o valor obtido pelo operador e o correspondente pixel realçado na imagem de saída, que passa de 107 para 123.

```
patch = img_gray[250:255, 250:255]
w4 = np.array([[0, 1, 0],
               [1,-4, 1],
               [0, 1, 0]], dtype=np.float32)
B = np.ones((3,3), dtype=np.uint8)

p = patch.astype(np.float32)
lap_11 = int(p[0,1] + p[1,0] - 4*p[1,1] + p[1,2] + p[2,1])
real_11 = int(np.clip(p[1,1] - lap_11, 0, 255))

print("Patch original:")
print(mm.drawImg(patch))
print(f"Laplac. w4 em [1,1]: ", end="")
print(f"{p[0,1]:.0f} + {p[1,0]:.0f} - 4*{p[1,1]:.0f} + {p[1,2]:.0f} + {p[2,1]:.0f} = {lap_11}")
print(f"Pixel realçado [1,1]: {p[1,1]:.0f} - ({lap_11}) = {real_11}")

mm.drawImgKernel(patch, B, x=1, y=1, scale=40.0)
```

```
Patch original:
 91  95 108 116 114
106 107 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

```
Laplac. w4 em [1,1]: 95 + 106 - 4*107 + 108 + 103 = -16
Pixel realçado [1,1]: 107 - (-16) = 123
```

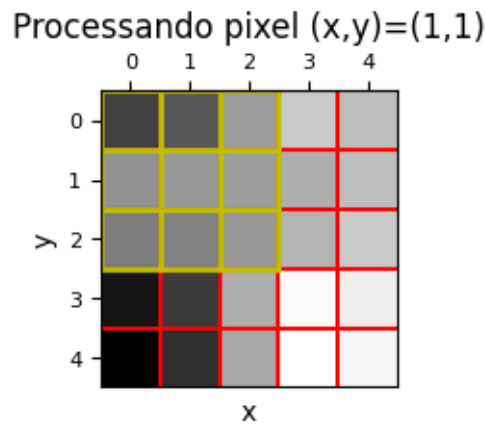


Figura 3.16: *Kernel* Laplaciano w_4 aplicado ao *patch* 5×5 : a janela amarela destaca a vizinhança 3×3 onde o operador de segunda derivada é calculado.

A Figure 3.17 compara a aplicação dos *kernels* Laplacianos w_4 e w_8 em um recorte maior da imagem do leopardo. Para cada caso, são mostradas a resposta bruta do operador, que evidencia as bordas e transições de intensidade, e a imagem obtida após o realce por subtração do Laplaciano. Observa-se que o *kernel* w_8 , por considerar também os vizinhos diagonais, produz uma resposta mais intensa e detecta variações em mais direções, resultando em um realce ligeiramente mais acentuado.

```
w4 = np.array([[0, 1, 0],
               [1,-4, 1],
               [0, 1, 0]], dtype=np.float32)
w8 = np.array([[1, 1, 1],
               [1,-8, 1],
               [1, 1, 1]], dtype=np.float32)

mm.show(
    [img_gray_crop, mm.laplacian_viz(img_gray_crop, w4), mm.laplacian(img_gray_crop, w4),
     img_gray_crop, mm.laplacian_viz(img_gray_crop, w8), mm.laplacian(img_gray_crop, w8)],
    titles=["Original", "Laplaciano w4", "Realce w4",
           "Original", "Laplaciano w8", "Realce w8"],
    rows=2, cols=3, figsize=(14, 8)
)
```



Figura 3.17: Laplaciano aplicado à imagem do leopardo: resposta bruta (bordas detectadas) com w4 e w8, e imagens realçadas pela subtração do Laplaciano. w8 é mais sensível às diagonais.

3.6.2 Operador de Sobel

O operador de Sobel estima as **derivadas parciais de primeira ordem** nas direções horizontal e vertical. Diferente do Laplaciano (segunda derivada), o Sobel é direcional e mais robusto ao ruído, pois cada *kernel* combina uma derivada com uma suavização Gaussiana perpendicular:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * f, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * f \quad (3.19)$$

G_x detecta bordas **verticais** (variação na direção x); G_y detecta bordas **horizontais** (variação na direção y). Os pesos $\{1, 2, 1\}$ na direção perpendicular correspondem à suavização Gaussiana 1D, que reduz a sensibilidade ao ruído.

i Sobel é correlação, não convolução

Os *kernels* de Sobel são **assimétricos** — a rotação de 180° altera o resultado. `cv2.Sobel` implementa correlação cruzada (como `cv2.filter2D`). Para obter a derivada direcional correta, os sinais já estão definidos para correlação: G_x retorna valores positivos onde a intensidade cresce da esquerda para a direita.

A magnitude do **gradiente** combina os dois componentes, representando a força da borda independente de direção:

$$|\nabla f| = \sqrt{G_x^2 + G_y^2} \quad (3.20)$$

E a **direção** do gradiente (perpendicular à borda) é:

$$\theta = \arctan \left(\frac{G_y}{G_x} \right) \quad (3.21)$$

Para ilustrar numericamente, calcula-se G_x e G_y manualmente no pixel central $[1, 1]$ do *patch* 5×5 :

```
patch = img_gray[250:255, 250:255]
p     = patch.astype(np.float32)

# Gx no pixel [1,1]: coluna direita - coluna esquerda (ponderada)
gx = (p[0,2] + 2*p[1,2] + p[2,2]) - (p[0,0] + 2*p[1,0] + p[2,0])

# Gy no pixel [1,1]: linha inferior - linha superior (ponderada)
gy = (p[2,0] + 2*p[2,1] + p[2,2]) - (p[0,0] + 2*p[0,1] + p[0,2])

mag  = np.sqrt(gx**2 + gy**2)
theta = np.degrees(np.arctan2(gy, gx))

print("Patch 5x5:")
print(mm.drawImg(patch))
print(f"Gx em [1,1]: ({p[0,2]:.0f} + 2*{p[1,2]:.0f} + {p[2,2]:.0f}) \
      - ({p[0,0]:.0f} + 2*{p[1,0]:.0f} + {p[2,0]:.0f}) = {gx:.1f}")
print(f"Gy em [1,1]: ({p[2,0]:.0f} + 2*{p[2,1]:.0f} + {p[2,2]:.0f}) \
      - ({p[0,0]:.0f} + 2*{p[0,1]:.0f} + {p[0,2]:.0f}) = {gy:.1f}")

# Substituído o | pelo prefixo mag para não quebrar o interpretador de tabelas do Quarto
print(f"mag(Grad f) = sqrt({gx:.1f}^2 + {gy:.1f}^2) = {mag:.1f}")
print(f"Theta      = arctan({gy:.1f}/{gx:.1f}) = {theta:.1f} graus")
```

Patch 5x5:

```
91  95 108 116 114
106 107 108 111 114
102 103 107 112 116
83   90 111 125 123
79   88 110 126 124
```

```
Gx em [1,1]: (108 + 2*108 + 107)      - (91 + 2*106 + 102) = 26.0
Gy em [1,1]: (102 + 2*103 + 107)      - (91 + 2*95 + 108) = 26.0
mag(Grad f) = sqrt(26.0^2 + 26.0^2) = 36.8
Theta      = arctan(26.0/26.0) = 45.0 graus
```

O valor reduzido de $|\nabla f|$ nesse *patch* confirma que a região é quase uniforme, pois o gradiente assume valores elevados apenas onde há mudanças significativas de intensidade. A Figure 3.18 aplica o operador de Sobel a um recorte maior da imagem do leopardo. São apresentadas as respostas horizontal (G_x) e vertical (G_y), obtidas por convolução com os respectivos *kernels* de Sobel, além da magnitude $|\nabla f|$, calculada a partir da combinação de ambas. Enquanto G_x destaca bordas verticais e G_y bordas horizontais, a magnitude evidencia bordas em qualquer direção.

```
def norm255(img):
    mn, mx = img.min(), img.max() #+1e-9 para evitar divisão por zero
    return ((img - mn) / (mx - mn + 1e-9) * 255).astype(np.uint8)

sobelx = cv2.Sobel(img_gray_crop, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img_gray_crop, cv2.CV_64F, 0, 1, ksize=3)
magnitudo = np.sqrt(sobelx**2 + sobely**2)
direcao = np.degrees(np.arctan2(sobely, sobelx))

# magnitudo = mm.sobel(img_gray_crop) # ou, mas com clip em vez de norm255

mm.show()
```

```
[img_gray_crop, norm255(np.abs(sobelx)), norm255(np.abs(sobely)),
 norm255(magnitude), norm255(direcao % 180)],
titles=["Original", "Gx (bordas vert.)", "Gy (bordas horiz.)",
        "Magnitude |f|", "Direção (mod 180°)",
cols=5
)
```



Figura 3.18: Operador de Sobel na imagem do leopardo: Gx detecta bordas verticais, Gy detecta bordas horizontais, e a magnitude $|f|$ combina ambos, revelando todas as bordas independente de direção.

3.6.3 Operador de Prewitt

O operador de Prewitt é estruturalmente idêntico ao Sobel, mas substitui a ponderação Gaussiana $\{1, 2, 1\}$ por pesos uniformes $\{1, 1, 1\}$:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} * f, \quad G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} * f \quad (3.22)$$

A magnitude e a direção do gradiente seguem as mesmas equações do Sobel (Equation 3.20 e Equation 3.21). A diferença prática é que o Prewitt é ligeiramente mais sensível ao ruído — a suavização perpendicular uniforme pondera menos o pixel central da linha — mas computacionalmente mais simples. Em imagens com baixo ruído os resultados são equivalentes.

```
Kx = np.array([[ -1, 0, 1], [ -1, 0, 1], [ -1, 0, 1]], dtype=np.float32)
Ky = np.array([[ -1, -1, -1], [ 0, 0, 0], [ 1, 1, 1]], dtype=np.float32)
f = img_gray_crop.astype(np.float32)

mm.show(
    [img_gray_crop,
     norm255(np.abs(cv2.filter2D(f, -1, Kx))),
     norm255(np.abs(cv2.filter2D(f, -1, Ky))),
     mm.prewitt(img_gray_crop)],
    titles=["Original", "Gx", "Gy", "Magnitude |f|"],
    cols=4
)
```

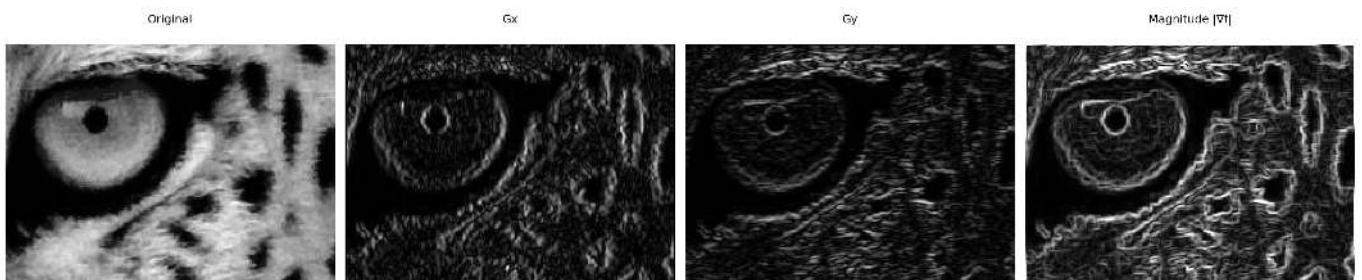


Figura 3.19: Operador de Prewitt: Gx, Gy e magnitude — comparável ao Sobel, mas sem ponderação Gaussiana perpendicular.

3.6.4 Unsharp Masking (USM)

O *Unsharp Masking* é uma técnica clássica de realce de nitidez originária da fotografia analógica, hoje amplamente usada em software de edição de imagens. A ideia central é extrair as **componentes de alta frequência** da imagem (bordas e detalhes) e somá-las de volta à original com um peso k :

Tabela 3.3: Etapas do *Unsharp Masking*.

Etapas	Operação	Descrição
1	$\bar{f} = f * G_{\sigma}$	Suaviza com Gaussiana — retém baixas frequências
2	$m = f - \bar{f}$	Máscara: diferença = altas frequências (bordas)
3	$g = f + k \cdot m$	Soma ponderada da máscara à original

Substituindo a etapa 2 na etapa 3, obtém-se a expressão compacta:

$$g = f + k(f - f * G_{\sigma}) = (1 + k)f - k(f * G_{\sigma}) \quad (3.23)$$

O parâmetro k controla a intensidade do realce:

- $k = 0$: sem realce ($g = f$);
- $k = 1$: USM clássico — duplica a contribuição das altas frequências;
- $k > 1$: *High Boost Filtering* — amplificação além do dobro, útil para imagens muito borradas.

⚠ Amplificação de ruído

O USM não distingue bordas de ruído — ambos são componentes de alta frequência. Para k elevado, o ruído presente na imagem é amplificado junto com as bordas. Por isso, é recomendável aplicar uma leve suavização antes do USM em imagens ruidosas, ou usar σ pequeno na Gaussiana.

Para ilustrar as etapas do USM, a Figure 3.20 aplica o método a um *patch* 30×30 da imagem do leopardo, utilizando $\sigma = 1$ e $k = 1$. Inicialmente, a imagem é suavizada por um filtro Gaussiano. Em seguida, a máscara de alta frequência é obtida pela diferença entre a imagem original e a suavizada. Por fim, essa máscara é somada à imagem original, reforçando bordas e detalhes. A figura apresenta as três etapas do processo e o resultado final do realce.

```
patch = img_gray_crop[35:65, 45:75]
k = 1.0

p = patch.astype(np.float32)
sigma = 1.0

# Etapa 1: suavização Gaussiana
ksize = int(6 * sigma + 1) | 1 # operador binário garante tamanho ímpar
p_suave = cv2.GaussianBlur(p, (ksize, ksize), sigma)

# Etapa 2: máscara de alta frequência
mascara = p - p_suave

# Etapa 3: realce
p_usm = np.clip(p + k * mascara, 0, 255).astype(np.uint8)

# p_usb = mm.usm(p, k) # ou

print(f"Pixel central [1,1]: original={int(p[1,1])} suave={p_suave[1,1]:.1f}")
```



```

f" mascara={mascara[1,1]:.1f} realcado={p_usm[1,1]}")

mm.show(
    [patch,
     p_suave.astype(np.uint8),
     np.clip(mascara + 128, 0, 255).astype(np.uint8), # tornar os valores negativos visíveis na figura
     p_usm],
    titles=["Patch original",
            f"Suavizado (={sigma})",
            "Máscara (alta freq., +128)",
            f"Realçado USM (k={k})"],
    cols=4, figsize=(12, 4)
)

```

Pixel central [1,1]: original=16 suave=20.2 mascara=-4.2 realcado=11

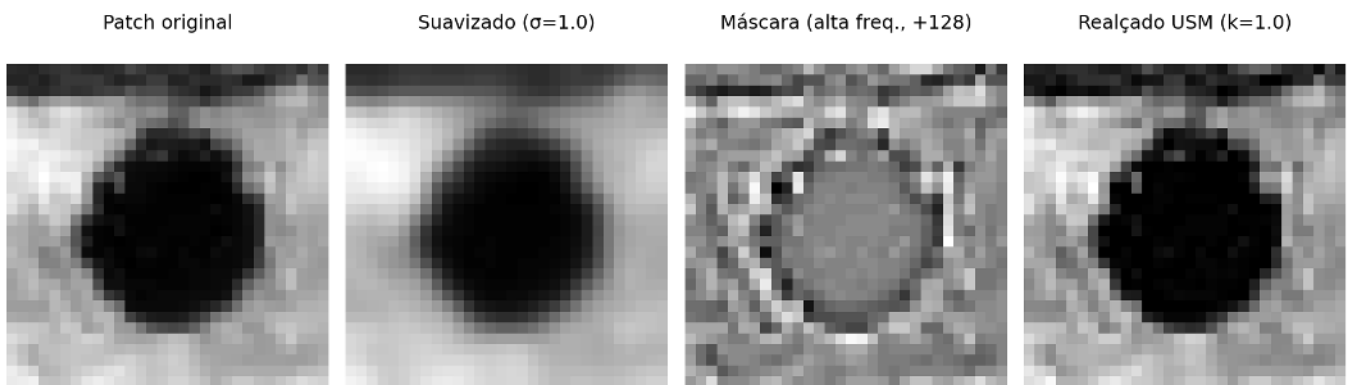


Figura 3.20: Realce de nitidez por *Unsharp Masking* na imagem do leopardo: a imagem suavizada é subtraída da original para gerar a máscara de alta frequência, que é então reintroduzida para realçar bordas e detalhes.

A Figure 3.21 aplica o método USM a um recorte maior da imagem do leopardo utilizando $\sigma = 1$ e diferentes valores do fator de ganho k . Em todos os casos, a máscara de alta frequência é obtida pela diferença entre a imagem original e sua versão suavizada por filtro Gaussiano. O parâmetro k controla a intensidade do realce: valores menores produzem um aumento sutil de nitidez, enquanto valores maiores reforçam progressivamente bordas e detalhes. Observa-se que, para valores elevados de k , surgem halos ao redor das bordas e o ruído presente na imagem passa a ser amplificado.

```

ks = [0.5, 1.0, 3.0, 5.0, 8.0]

imgs = [img_gray_crop] + [mm.usm(img_gray_crop, k) for k in ks]
titles = ["Original"] + [f"USM k={k}" for k in ks]

mm.show(imgs, titles=titles, cols=3, rows=2, figsize=(14, 8))

```

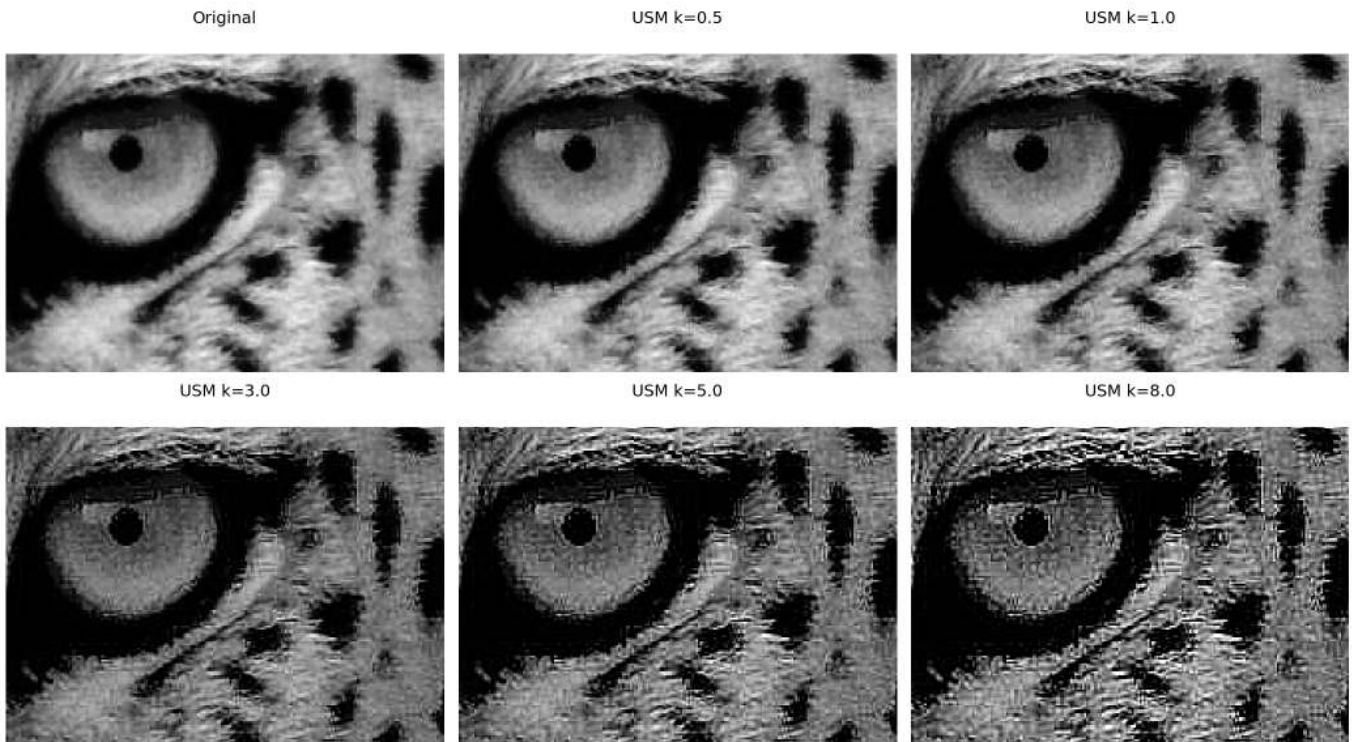


Figura 3.21: *Unsharp Masking* na imagem do leopardo com $\alpha=1$ e k variando de 0.5 a 8.0. Para $k>2$ surgem artefatos nas bordas (halos) e o ruído de fundo começa a ser visível.

3.6.5 Detector de Canny

O Canny combina quatro etapas em sequência — suavização Gaussiana, gradiente de Sobel, supressão de não-máximos e histerese por duplo limiar — para produzir bordas **finas, binárias e conectadas**. Diferente de Sobel e Prewitt, o resultado não é um mapa de gradiente contínuo, mas uma máscara onde cada pixel é borda ou não.

O parâmetro central é o par de limiares (T_{low}, T_{high}). Pixels com gradiente acima de T_{high} são bordas certas; abaixo de T_{low} , descartados. Os pixels ambíguos — entre os dois limiares — são decididos por **histerese**: tornam-se borda se estiverem conectados a uma borda certa, e descartados caso contrário. Isso evita tanto a perda de trechos fracos de bordas reais quanto a inclusão de ruído isolado. Uma heurística comum é $T_{high} = 3 \times T_{low}$.

```
mm.show(
    [img_gray_crop,
     mm.canny(img_gray_crop, 30, 90),
     mm.canny(img_gray_crop, 60, 180),
     mm.canny(img_gray_crop, 120, 240)],
    titles=["Original", "Canny (30/90)", "Canny (60/180)", "Canny (120/240)"],
    cols=4
)
```

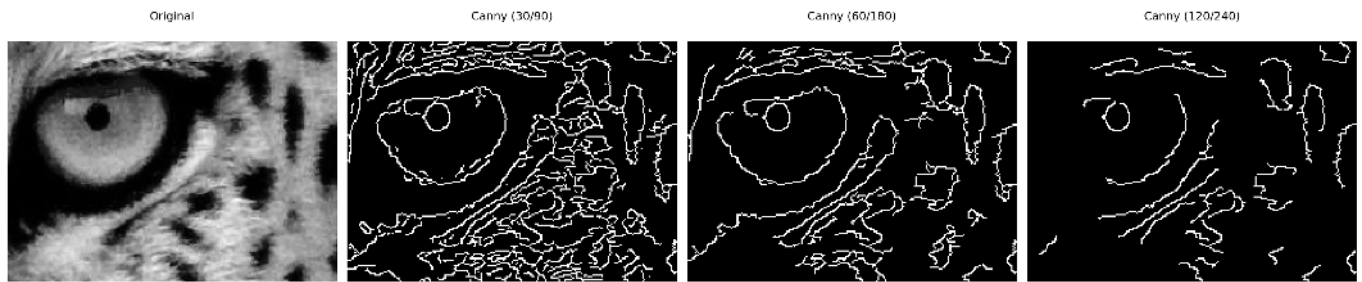


Figura 3.22: Detector de Canny com diferentes pares de limiar: limiares baixos capturam mais bordas (inclusive ruído); limiares altos retêm apenas as bordas mais fortes.

i Escolha dos limiares

Uma heurística comum é $T_{high} = 3 \times T_{low}$. Valores típicos dependem do intervalo de gradiente da imagem — `cv2.Canny` aceita valores absolutos em $[0, 255]$. Para imagens com contraste variável, calcular os limiares a partir de percentis da magnitude de Sobel é mais robusto do que valores fixos.

3.7 Filtros de Ordem: Filtro da Mediana

Os filtros de ordem (*order-statistic filters*) substituem o pixel central pelo valor de um **percentil** da distribuição de intensidades da vizinhança — ao contrário dos filtros lineares, que calculam combinações ponderadas. O mais importante é o **filtro da mediana**.

3.7.1 Ruído Impulsivo: Sal e Pimenta

O ruído **sal e pimenta** (*salt-and-pepper noise*) substitui pixels aleatórios por valores extremos: 0 (pimenta, preto) ou 255 (sal, branco). É comum em transmissão de imagens com erros de bit e em câmeras com sensores defeituosos.

Para entender por que os filtros lineares falham, considere uma vizinhança 3×3 onde um único pixel foi corrompido para 255:

$$\text{vizinhança} = \begin{bmatrix} 102 & 98 & 105 \\ 100 & \mathbf{255} & 97 \\ 103 & 99 & 101 \end{bmatrix}$$

Tabela 3.4: Média vs. mediana com um pixel corrompido. A mediana ignora o outlier; a média é deslocada ~40 níveis.

Método	Cálculo	Resultado
Média	$(102+98+\dots+255+\dots+101)/9$	140
Mediana	$\{97, 98, 99, 100, \mathbf{101}, 102, 103, 105, 255\}$	101

⚠ Por que filtros de média falham com ruído impulsivo?

A média é sensível a **outliers** — um único pixel com valor 255 em uma vizinhança de valor 100 eleva a saída para 140, espalhando o ruído pela imagem. A mediana, por ser um **estimador robusto**, seleciona o valor central da distribuição ordenada, descartando naturalmente os extremos sem nenhum ajuste especial.

O exemplo a seguir ilustra o comportamento da média e da mediana na presença de um pixel corrompido por ruído impulsivo. Observa-se que a média é fortemente influenciada pelo valor extremo (255), produzindo uma estimativa distante dos valores predominantes da vizinhança. Já a mediana permanece próxima do valor original da região, evidenciando sua maior robustez a *outliers* e justificando seu uso na remoção de ruído sal e pimenta.

```
# Exemplo numérico: média vs. mediana com pixel corrompido
vizinhanca = np.array([102, 98, 105, 100, 255, 97, 103, 99, 101])
print(f"Vizinhança: {sorted([int(x) for x in vizinhanca])}")
print(f"Média:      {vizinhanca.mean():.1f} (deslocada pelo outlier 255)")
print(f"Mediana:    {int(np.median(vizinhanca))} (ignora o outlier)")
print(f"Valor original: ~100")
```

```
Vizinhança: [97, 98, 99, 100, 101, 102, 103, 105, 255]
Média:      117.8 (deslocada pelo outlier 255)
Mediana:    101 (ignora o outlier)
Valor original: ~100
```

A Figure 3.23 apresenta o efeito do ruído sal e pimenta em diferentes densidades. O ruído foi gerado substituindo aleatoriamente uma fração dos pixels por valores mínimos (0, pimenta) e máximos (255, sal). À medida que a densidade aumenta de 2% para 10%, cresce a quantidade de pixels corrompidos, tornando a degradação visual mais evidente e dificultando a percepção de detalhes da imagem.

```
def add_salt_pepper(img, prob=0.05, seed=42):
    """Adiciona ruído sal e pimenta com probabilidade total prob."""
    noisy = img.copy()
    rnd = np.random.default_rng(seed).random(img.shape)
    noisy[rnd < prob / 2] = 0 # pimenta
    noisy[rnd > 1 - prob / 2] = 255 # sal
    n_corrompidos = int((rnd < prob/2).sum() + (rnd > 1-prob/2).sum())
    return noisy, n_corrompidos

probs = [0.02, 0.05, 0.10]
imgs_noise, titles_noise = [img_gray_crop], ["Original"]

for p in probs:
    noisy, n = add_salt_pepper(img_gray_crop, p)
    imgs_noise.append(noisy)
    titles_noise.append(f"Ruído {int(p*100)}%\n({n:,} pixels)")

mm.show(imgs_noise, titles=titles_noise, cols=4)
```

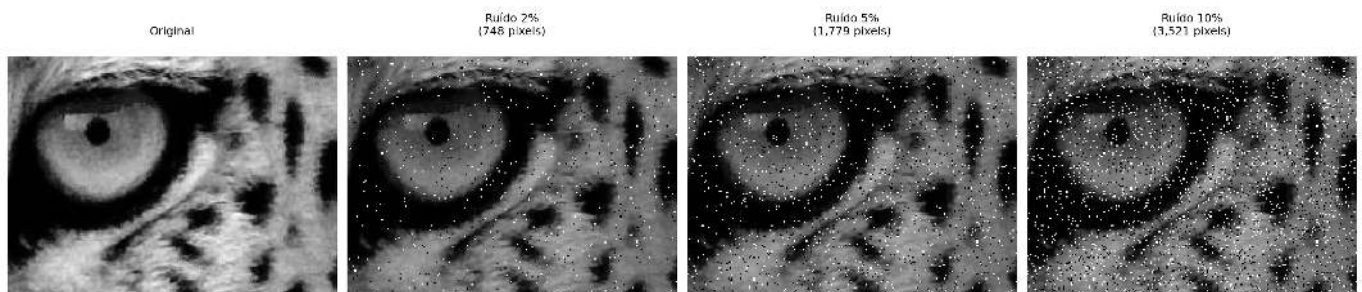


Figura 3.23: Ruído sal e pimenta com densidades crescentes (2%, 5%, 10%). O parâmetro *prob* indica a fração total de pixels corrompidos, metade sal (255) e metade pimenta (0).

3.7.2 Filtro da Mediana

O filtro da mediana substitui cada pixel pelo **valor mediano** dos pixels da sua vizinhança $n \times n$:

$$g(x, y) = \text{med}_{(s,t) \in \mathcal{V}_n} \{f(x + s, y + t)\} \quad (3.24)$$

O valor mediano é aquele que ocupa a posição central quando os n^2 valores da vizinhança são ordenados. Para uma janela 3×3 ($n^2 = 9$ pixels), a mediana é o 5º valor da sequência ordenada.

Para ilustrar, considere o mesmo *patch* 5×5 com um pixel corrompido artificialmente em $[1, 1]$:

```
patch = img_gray[250:255, 250:255].copy()
corrompido = patch.copy()
corrompido[1, 1] = 255 # injeta pixel sal no centro

# Vizinhança 3x3 centrada em [1,1]
viz_orig = sorted(patch[0:3, 0:3].ravel().tolist())
viz_corr = sorted(corrompido[0:3, 0:3].ravel().tolist())

print("Patch original:")
print(mm.drawImg(patch))
print("\nPatch com pixel corrompido [1,1]=255:")
print(mm.drawImg(corrompido))
print(f"\nVizinhança 3x3 original (ordenada): {viz_orig}")
print(f"Mediana original: {viz_orig[4]}")
print(f"\nVizinhança 3x3 corrompida (ordenada): {viz_corr}")
print(f"Mediana corrompida: {viz_corr[4]} ← ignora o 255")
print(f"Média corrompida: {np.mean(viz_corr):.1f} ← distorcida pelo 255")
```

```
Patch original:
 91  95 108 116 114
106 107 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

```
Patch com pixel corrompido [1,1]=255:
```

```
 91  95 108 116 114
106 255 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

```
Vizinhança 3x3 original (ordenada): [91, 95, 102, 103, 106, 107, 107, 108, 108]
Mediana original: 106
```

```
Vizinhança 3x3 corrompida (ordenada): [91, 95, 102, 103, 106, 107, 108, 108, 255]
Mediana corrompida: 106 ← ignora o 255
Média corrompida: 119.4 ← distorcida pelo 255
```

O exemplo confirma: mesmo com o pixel corrompido a 255, a mediana retorna o valor central correto — o *outlier* ocupa a última posição na ordenação e é descartado naturalmente.

Por ser baseada em ordenação e não em soma, a mediana possui três propriedades fundamentais que a diferenciam dos filtros lineares:

- **Robusta** ao ruído impulsivo — outliers vão para as extremidades da sequência ordenada e não afetam o valor central;
- **Preservadora de bordas** — transições abruptas de intensidade são mantidas, pois a mediana seleciona um valor que já existe na vizinhança, sem criar novos níveis intermediários;

- **Não linear** — não pode ser expressa como convolução, portanto `mm.conv` não se aplica; usa-se `cv2.medianBlur`.

A Figure 3.24 compara diferentes técnicas de remoção de ruído sal e pimenta aplicadas a uma imagem com 10% de pixels corrompidos. Foram avaliados os filtros Gaussiano, Média, Mediana, Bilateral e Morfológico (próximo capítulo), permitindo observar o compromisso entre remoção de ruído e preservação de detalhes. Em geral, os filtros de média e Gaussiano reduzem o ruído, mas tendem a borrar as bordas, enquanto a mediana apresenta melhor desempenho para ruído impulsivo. O filtro bilateral preserva melhor as bordas, e o filtro morfológico remove boa parte dos pixels corrompidos sem degradar excessivamente a estrutura da imagem.

```
# 10% de ruído para evidenciar diferenças entre filtros
noisy_5, _ = add_salt_pepper(img_gray_crop, prob=0.1)

# Filtros
f_gauss = cv2.GaussianBlur(noisy_5, (5, 5), 1.0)
f_media = mm.conv(noisy_5, np.ones((5, 5), dtype=np.float32) / 20.0)
f_median3 = cv2.medianBlur(noisy_5, 3)
f_median5 = cv2.medianBlur(noisy_5, 5)
f_bilat = cv2.bilateralFilter(noisy_5, d=9, sigmaColor=75, sigmaSpace=75)

# filtros morfológicos no próximo capítulo, mas já adiantados aqui para comparação
# B = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
# f_morf = cv2.morphologyEx( # apresentado no próximo capítulo
#     cv2.morphologyEx(noisy_5, cv2.MORPH_OPEN, B),
#     cv2.MORPH_CLOSE, B)
B = mm.seccross() # elemento estruturante de caixa 3x3
f_morf = mm.asf(noisy_5, 'OC', B) # equivalente via morph

# MSE
def mse(a, b):
    return float(np.mean((a.astype(np.float32) - b.astype(np.float32))**2))

print(f"{'Filtro':<22} {'MSE':>8}")
print("-" * 32)
pares = [("Gaussiano 5x5", f_gauss),
         ("Média 5x5", f_media),
         ("Mediana 3x3", f_median3),
         ("Mediana 5x5", f_median5),
         ("Bilateral", f_bilat),
         ("Morf. open+close", f_morf)]
for nome, img in pares:
    print(f"{nome:<22} {mse(img_gray_crop, img):>8.2f}")

imgs = [img_gray_crop, noisy_5, f_gauss, f_media,
        f_median3, f_median5, f_bilat, f_morf]
titles = ["Original", "Ruído 10%", "Gaussiano 5x5", "Média 5x5",
         "Mediana 3x3", "Mediana 5x5", "Bilateral", "Morf. open+close"]

mm.show(
    imgs,
    titles=[f"Crop: {t}" for t in titles],
    cols=4, rows=2, figsize=(14, 8)
)
```

Filtro	MSE
Gaussiano 5x5	260.84
Média 5x5	872.57
Mediana 3x3	31.26

Mediana 5×5	65.86
Bilateral	1001.22
Morf. open+close	65.16

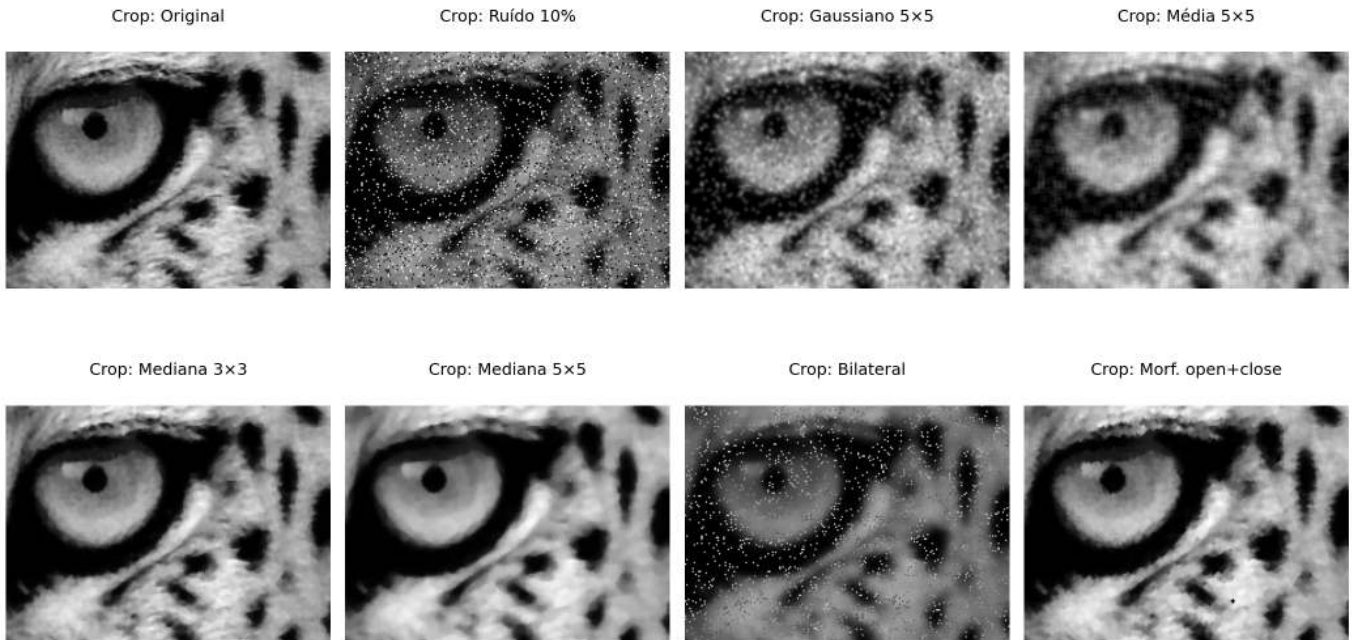


Figura 3.24: Comparação de filtros para remoção de ruído sal e pimenta (10%): Gaussiano, Média, Mediana, Bilateral e Morfológico. Linha superior: imagens completas; linha inferior: crop da região de interesse.

3.8 Aplicação Prática: Pré-processamento para Segmentação

Na prática, as técnicas deste capítulo raramente são usadas isoladamente. Um *pipeline de pré-processamento* típico combina várias etapas em sequência, adaptando-se ao tipo de imagem e à aplicação. A Figure 3.25 ilustra um *pipeline* completo:

1. **Equalização de histograma (CLAHE):** normaliza o contraste independente das condições de iluminação;
2. **Filtro Gaussiano:** suaviza ruído de aquisição sem destruir bordas;
3. **Deteção de bordas (Sobel/Canny):** extrai estruturas relevantes para segmentação.

i Ordem importa

A ordem das operações afeta o resultado final. Em geral: **(1) normalização de intensidade → (2) redução de ruído → (3) realce/segmentação**. Inverter a ordem pode amplificar ruído ou perder bordas antes de detectá-las.

```
# Etapa 1: CLAHE
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
img_clahe = clahe.apply(img_gray_crop)

# Etapa 2: Gaussiano
img_gauss = cv2.GaussianBlur(img_clahe, (5, 5), 0)

# Etapa 3: Canny
edges = cv2.Canny(img_gauss, 50, 150)

# Comparação: pipeline vs. Canny direto
edges_direct = cv2.Canny(img_gray_crop, 50, 150)
```

```
mm.show(
    [img_gray_crop, img_clahe, img_gauss, edges, edges_direct],
    titles=["Original", "1. CLAHE", "2. Gaussiano", "3. Canny (pipeline)", "Canny (direto)"],
    cols=5
)
```



Figura 3.25: *Pipeline* de pré-processamento: CLAHE → Gaussiano → Canny. Cada etapa prepara a imagem para a seguinte, resultando em bordas limpas e bem definidas.

3.9 Resumo

Neste capítulo foram apresentadas as principais técnicas de processamento no domínio espacial, da manipulação direta de pixels até a filtragem por vizinhança:

- **Operações de ponto:** aritméticas saturadas (`mm.addm`, `mm.subm`) e lógicas bit a bit (`mm.band`, `mm.bor`, `mm.bnot`) para recorte de ROI e combinação de imagens; *alpha blending* (`mm.blend`) para fusão ponderada com peso $\alpha \in [0, 1]$.
- **Histograma:** função discreta de distribuição de intensidades; visualizado com `mm.histImg` e calculado com `mm.hist`; base para diagnóstico tonal e para as técnicas de equalização e especificação.
- **Equalização:** redistribuição automática das intensidades pela CDF (`mm.equalize`), com variante adaptativa CLAHE para controle local do contraste.
- **Especificação de histograma:** transferência do perfil tonal de uma imagem de referência via mapeamento inverso da CDF — generalização da equalização para distribuições arbitrárias.
- **Correlação e convolução:** mecanismo de janela deslizante implementado em `mm.conv` (`cv2.filter2D`); diferenciados pela rotação de 180° do *kernel* — relevante apenas para kernels assimétricos.
- **Filtros de suavização:** média (*kernel* uniforme, borra bordas proporcionalmente ao tamanho) e Gaussiano (ponderação radial, separável, sem *ringing*, preserva melhor as bordas).
- **Filtros de realce:** Laplaciano (w_4/w_8 , segunda derivada isotrópica), Sobel (gradiente direcional de primeira ordem, com magnitude $|\nabla f|$ e direção θ) e *Unsharp Masking* (amplificação das altas frequências com parâmetro k).
- **Filtro da mediana:** não linear, robusto a outliers, preserva bordas — superior aos filtros lineares para ruído sal e pimenta.
- **Pipeline prático:** encadeamento CLAHE → Gaussiano → Canny como estratégia de pré-processamento; `mm.drawImgKernel` para visualização didática da janela deslizante.

O Capítulo 4 abordará a **morfologia matemática** (erosão, dilatação, abertura e fechamento), explorando em profundidade as funções `mm.ero` e `mm.dil` da biblioteca `morph.py`. Em seguida, o Capítulo 5 apresentará o **processamento no domínio da frequência**, com foco na Transformada de Fourier e em técnicas de filtragem espectral.

3.10 Uso do NotebookLM como Tutor Complementar

Nesta edição, incentivamos o uso do **NotebookLM** como ferramenta complementar de aprendizagem. Essa ferramenta de IA utiliza exclusivamente os documentos fornecidos pelo autor como base de conhecimento, garantindo respostas coerentes com o conteúdo do livro — incluindo as funções da biblioteca `morph.py` e os experimentos realizados neste capítulo.

Para cada capítulo, preparamos um projeto específico na plataforma com o PDF do capítulo, os *notebooks* e materiais auxiliares. Sugerimos explorar especialmente:

- **Guia de Estudo:** resumo estruturado dos conceitos, ideal para revisão antes de provas;
- **Conversa:** tire dúvidas sobre equalização, convolução, filtros e pipelines diretamente com o tutor;
- **Perguntas frequentes:** questões típicas sobre a diferença entre média e mediana, USM, Laplaciano vs. Sobel.

Estude com o Tutor Inteligente

Para interagir com o conteúdo deste capítulo, acesse o *link* a seguir. O ambiente contém materiais didáticos em diferentes formatos, gerados a partir do **PDF** do capítulo. Na plataforma, explore especialmente as opções **Guia de Estudo** e **Conversa** para aprofundar sua compreensão.

 [ACESSAR NOTEBOOKLM: CAPÍTULO 03](#)

Aviso sobre Conteúdo Gerado por IA

A IA é uma poderosa aliada nos estudos, mas o conteúdo gerado pode conter **erros ou imprecisões**. Consulte sempre **livros, artigos científicos e outras fontes acadêmicas confiáveis** para validar as informações. Sempre que possível, execute os exemplos práticos fornecidos neste capítulo para verificar os resultados.

3.11 Lista de Exercícios

1. (10%) Explique a diferença entre **convolução** e **correlação cruzada**. Para quais tipos de *kernel* os resultados são idênticos? Dê um exemplo de *kernel* assimétrico (como Sobel G_x) e mostre numericamente que os resultados diferem aplicando-o ao patch 5×5 do capítulo das duas formas.
2. (15%) Considere uma imagem 5×5 com intensidades concentradas entre os níveis 3 e 5 (baixo contraste, 3 bits). Aplique manualmente o algoritmo de equalização da Table 3.1, preenchendo todas as colunas da tabela (k , $h[k]$, $p[k]$, $\text{cdf}[k]$, $\text{lut}[k]$). Verifique o resultado com `mm.equalize`.
3. (15%) Usando `mm.conv`, aplique o filtro de média com kernels de tamanho 3×3 , 9×9 e 21×21 à imagem do mandrill. Para cada versão, calcule o **PSNR** (*Peak Signal-to-Noise Ratio*) em relação à original:

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right), \quad \text{MSE} = \frac{1}{MN} \sum_{i,j} (f - g)^2$$

Plote o PSNR em função do tamanho do kernel e explique o que a queda progressiva indica sobre a relação entre suavização e perda de informação.

4. (15%) Usando `add_salt_pepper` com densidade de 5%, aplique e compare: (a) `mm.conv` com média 3×3 , (b) `cv2.GaussianBlur` com $\sigma = 1$, (c) `cv2.medianBlur` com janela 3×3 e (d) `cv2.medianBlur` com janela 5×5 . Exiba as imagens com `mm.show` em grade 2×4 (linha 1: imagens, linha 2: histogramas via `mm.histImg`). Explique por que a mediana supera os filtros lineares usando o argumento da Table 3.4.
5. (15%) Implemente `mm.conv0` usando apenas operações NumPy vetorizadas — sem laços Python e sem `cv2.filter2D` — com o operador de *stride tricks* (`np.lib.stride_tricks.sliding_window_view`). Compare o resultado e o tempo de execução com `mm.conv0` (laços) e `mm.conv` (cv2) para kernels 3×3 e 15×15 na imagem do mandrill.
6. (15%) Aplique o *Unsharp Masking* com $\sigma = 1$ e $k \in \{0.5, 1.0, 2.0, 4.0\}$ usando a função `usm` do capítulo. Para cada valor de k : (a) calcule a diferença absoluta $|g - f|$, (b) exiba as imagens e as diferenças com `mm.show`, e (c) plote o histograma das diferenças com `mm.histImg`. Identifique a partir de qual k os artefatos (halos e amplificação de ruído) tornam-se visualmente inaceitáveis.

7. (15%) Escolha uma imagem de raio-X ou tomografia disponível publicamente (ex.: via `mm.read` de URL) e projete um pipeline de pré-processamento com pelo menos 4 etapas sequenciais, justificando cada escolha com base nos conceitos do capítulo. Exiba com `mm.show` em grade: imagem original, cada etapa intermediária e o resultado final com seus histogramas (`mm.histImg`).

Referências do Capítulo

A fundamentação teórica deste capítulo baseia-se nas seguintes obras:

- Gonzalez; Woods (2018) para os conceitos de operações de intensidade, histograma, convolução e filtragem espacial.
- Szeliski (2022) para a visão computacional e aplicações práticas de filtragem.
- Bradski; Kaehler (2008) para a implementação prática com OpenCV e `morph.py`.



3.12 Parte Prática com Exercícios de Programação

Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f"\u2705 Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.0 | TestSuite: 1.1.0

Executando os Testes

Para rodar os testes, execute `TestSuite("EP03_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:

```

codigo = """
from morph import mm
# ... seu código aqui ...
"""
TestSuite("EP03_01").run_code(codigo)

```

3.12.1 EP03_01 + Adição Saturada de Constante

Em sistemas de vigilância por vídeo, câmeras em ambientes com iluminação variável produzem imagens subexpostas. O ajuste de brilho por **adição saturada de uma constante** é a operação mais simples para correção imediata, sendo aplicada em tempo real nos *chips* de câmeras embarcadas e em *pipelines* de pré-processamento de robôs móveis.

Ver na ?@fig-03-sim-adicao uma simulação deste EP.

3.12.1.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Constante:** Ler o inteiro k (valor a ser somado).
3. **Dados:** Ler os valores inteiros da matriz original linha a linha.
4. **Mapeamento:** Para cada pixel p , calcular o novo valor pela equação:

$$p' = \text{clip}(p + k)$$

5. **Saída:** Exibir a matriz resultante com dimensões $L \times C$.

3.12.1.2 📌 Restrições Computacionais

- **Saturação (*Clipping*):** Os valores devem ser confinados ao intervalo $[0, 255]$:

$$\text{clip}(x) = \max(0, \min(255, x))$$

- **Tipo:** O resultado final deve ser inteiro (sem casas decimais).
- **k pode ser negativo:** valores negativos escurecem a imagem; positivos clareiam.

3.12.1.3 🧠 Fundamentação Teórica

Parâmetro	Tipo	Impacto Visual
$k > 0$	Inteiro	Clareia a imagem; pixels próximos de 255 saturam em branco
$k < 0$	Inteiro	Escurece a imagem; pixels próximos de 0 saturam em preto
$k = 0$	Inteiro	Imagem inalterada

3.12.1.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .

- Linha 2: Inteiro C .
- Linha 3: Inteiro k .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz transformada em L linhas e C colunas, valores inteiros separados por espaço.

3.12.1.5 Exemplos

Entrada	Saída	Observação
2	50 150 250	Saturação em 255 nos pixels altos
3	255 255 255	
50		
0 100 200		
210 240 255		
1	0 0 170 225	Saturação em 0 nos pixels baixos
4		
-30		
0 20 200 255		

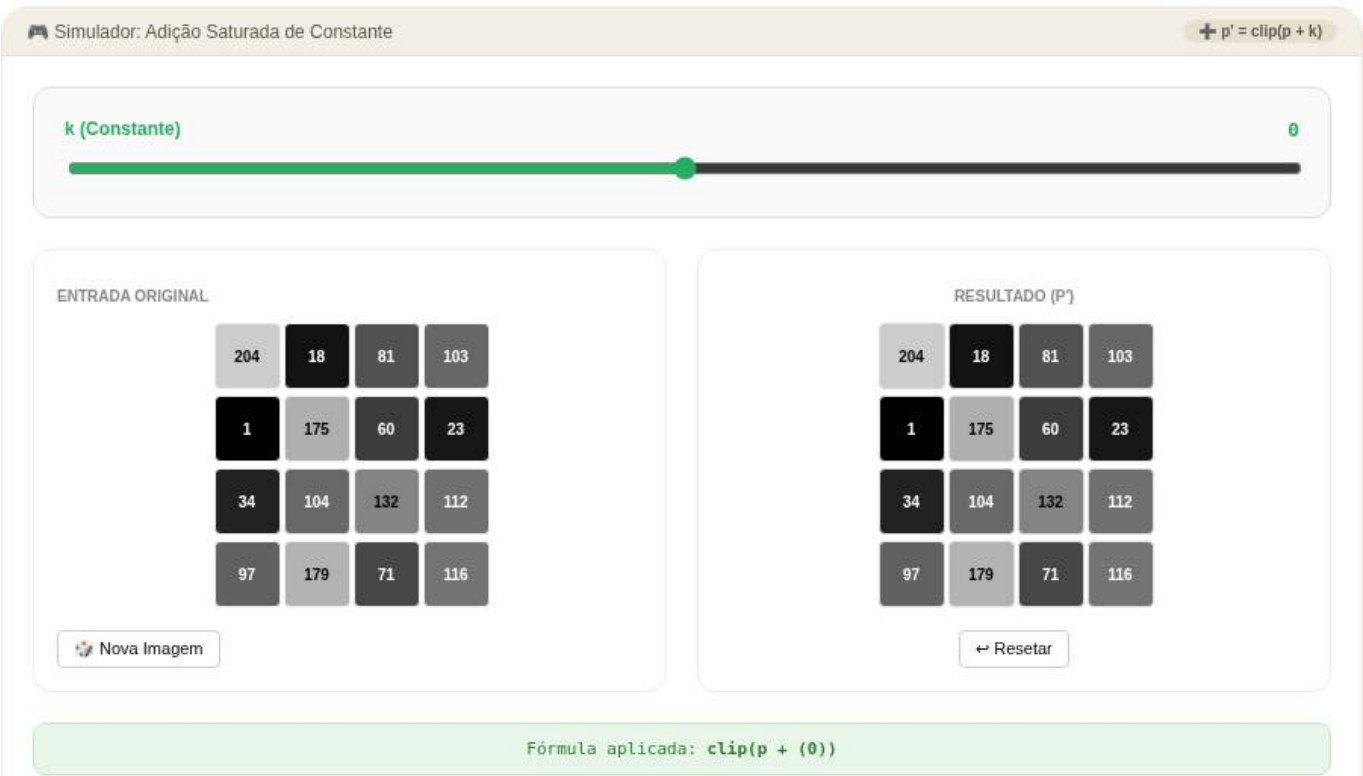


Figura 3.26: Simulador: Adição Saturada de Constante

```
%%writefile EP03_01.py
# Código Python

Writing EP03_01.py

TestSuite("EP03_01.py").run()
```


3.12.2 EP03_02 Alpha *Blending* de Duas Imagens

Em medicina nuclear, imagens de diferentes modalidades (tomografia computadorizada e ressonância magnética) são fundidas para auxiliar no diagnóstico. A **mistura ponderada** (*alpha blending*) é a operação fundamental desse processo, permitindo ao radiologista controlar interativamente o peso de cada modalidade na imagem exibida.

Ver na Figure 3.27 uma simulação deste EP.

3.12.2.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Parâmetro:** Ler o valor real $\alpha \in [0, 1]$.
3. **Dados:** Ler os valores inteiros da matriz f_1 (imagem 1) e em seguida da matriz f_2 (imagem 2).
4. **Mapeamento:** Para cada posição (i, j) , calcular:

$$g(i, j) = \text{clip}(\text{round}(\alpha \cdot f_1(i, j) + (1 - \alpha) \cdot f_2(i, j)))$$

5. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.2.2 Restrições Computacionais

- **Arredondamento:** Aplicar **round** antes da conversão para inteiro.
- **Saturação:** Confinar ao intervalo $[0, 255]$ com $\text{clip}(x) = \max(0, \min(255, x))$.
- **Operação em float:** Realize a operação em ponto flutuante antes de arredondar.

3.12.2.3 Fundamentação Teórica

Valor de α	Resultado
$\alpha = 1.0$	Apenas f_1
$\alpha = 0.5$	Média aritmética de f_1 e f_2
$\alpha = 0.0$	Apenas f_2

3.12.2.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Real α .
- Linhas seguintes: Elementos de f_1 (L linhas com C valores cada).
- Linhas seguintes: Elementos de f_2 (L linhas com C valores cada).

Saída:

- Matriz resultante $L \times C$.

3.12.2.5 📌 Exemplos

Entrada	Saída	Observação
1 3 0.5 0 100 200 100 200 50	50 150 125	Média entre as duas imagens
1 3 1.0 10 20 30 90 80 70	10 20 30	Apenas f_1 (alpha=1)



Figura 3.27: Simulador: Alpha *Blending* de Duas Imagens

```
%%writefile EP03_02.py
# Código Python

Writing EP03_02.py

TestSuite("EP03_02.py").run()
```

3.12.3 EP03_03 Inversão de Imagem (Negativo Fotográfico)

Em radiologia, as imagens de raio-X são tradicionalmente visualizadas em negativo: ossos aparecem em preto sobre fundo branco. A operação de **negativo fotográfico** é aplicada rotineiramente em PACS (*Picture Archiving and Communication Systems*) para facilitar a detecção de fraturas e densidades ósseas.

Ver na Figure 3.28 uma simulação deste EP.

3.12.3.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler os valores inteiros da matriz original.
3. **Mapeamento:** Para cada pixel p , calcular o negativo:

$$p' = 255 - p$$

4. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.3.2 Restrições Computacionais

- **Sem *clipping* necessário:** O resultado de $255 - p$ com $p \in [0, 255]$ é sempre $\in [0, 255]$.
- **Tipo inteiro:** Saída deve ser valores inteiros.
- **Equivalência lógica:** A operação é idêntica ao NOT bit a bit (`mm.bnot`) em imagens de 8 bits.

3.12.3.3 Fundamentação Teórica

Pixel Original p	Pixel Negativo p'	Observação
0 (preto)	255 (branco)	Inversão total
128 (cinza médio)	127 (cinza médio)	Valor central
255 (branco)	0 (preto)	Inversão total

3.12.3.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz negativa em L linhas e C colunas.

3.12.3.5 Exemplos

Entrada	Saída	Observação
140 128 200 255	255 127 55 0	Inversão de cada pixel
2 2 10 20 30 40	245 235 225 215	Matriz 2x2 invertida

```
%%writefile EP03_03.py
# Código Python
```

```
Writing EP03_03.py
```

```
TestSuite("EP03_03.py").run()
```



Figura 3.28: Simulador: Inversão de Imagem (Negativo Fotográfico)

3.12.4 EP03_04 Equalização de Histograma (L bits)

Em imagens de satélite de sensoriamento remoto, a variação de iluminação ao longo do dia produz imagens de baixo contraste. A **equalização de histograma** é aplicada automaticamente em satélites como o Landsat para redistribuir os tons, revelando detalhes de vegetação, relevo e zonas urbanas invisíveis na imagem original.

Ver na Figure 3.29 uma simulação deste EP.

3.12.4.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas), C (colunas) e B (número de bits, com $L_{\max} = 2^B$).
2. **Dados:** Ler a matriz de pixels f com valores em $[0, 2^B - 1]$.
3. **Histograma:** Calcular $h[k] = \text{número de pixels com intensidade } k$, para $k = 0 \dots 2^B - 1$.
4. **Probabilidade:** $p[k] = h[k]/(L \cdot C)$.
5. **CDF:** $\text{cdf}[k] = \sum_{j=0}^k p[j]$; função de distribuição acumulada.
6. **LUT:** $\text{lut}[k] = \text{round}(\text{cdf}[k] \cdot (2^B - 1))$; *Look-Up Table* (tabela de consulta).
7. **Aplicação:** $g[i, j] = \text{lut}[f[i, j]]$.
8. **Saída:** Exibir a matriz equalizada $L \times C$.

3.12.4.2 Restrições Computacionais

- **Arredondamento:** Usar arredondamento matemático (**round**) na LUT.
- **Bits:** O número de níveis é 2^B (ex.: $B = 3 \Rightarrow 8$ níveis, $B = 8 \Rightarrow 256$ níveis).
- **CDF acumulada:** $\text{cdf}[k] = \sum_{j=0}^k p[j]$, com $\text{cdf}[2^B - 1] = 1.0$.

3.12.4.3 Fundamentação Teórica

Etapa	Operação	Fórmula
1	Histograma	$h[k] \leftarrow \text{n}^\circ \text{ pixels com intensidade } k$

Etapa	Operação	Fórmula
2	Probabilidade	$p[k] = h[k]/(L \cdot C)$
3	CDF	$\text{cdf}[k] = \sum_{j=0}^k p[j]$
4	LUT	$\text{lut}[\text{Look} - \text{UpTable}(\text{tabeladeconsulta})k] = \text{round}(\text{cdf}[k] \cdot (2^B - 1))$
5	Aplicação	$g[i, j] = \text{lut}[f[i, j]]$

3.12.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro B (número de bits).
- Linhas seguintes: Elementos inteiros da matriz.

Saída:

- Matriz equalizada em L linhas e C colunas.

3.12.4.5 Exemplos

Entrada	Saída	Observação
5	5 7 1 5 7	Exemplo 3 bits do capítulo
5	7 5 5 7 5	
3	1 5 7 5 1	
3 4 2 3 4	5 7 5 1 5	
4 3 3 4 3	7 5 1 5 7	
2 3 4 3 2		
3 4 3 2 3		Histograma bimodal extremo
4 3 2 3 4		
1	0 0 7 7	
4		
3		
0 0 7 7		

```
%%writefile EP03_04.py
# Código Python
```

```
Writing EP03_04.py
```

```
TestSuite("EP03_04.py").run()
```

3.12.5 EP03_05 Aplicação de Máscara AND Binária

Em sistemas de inspeção industrial por visão computacional, é necessário isolar regiões de interesse (ROI) em imagens de peças para verificar defeitos de fabricação. A operação **AND bit a bit com uma máscara**

Simulador: Equalização de Histograma

Escolha o número de bits e gere uma imagem para ver a equalização em tempo real.

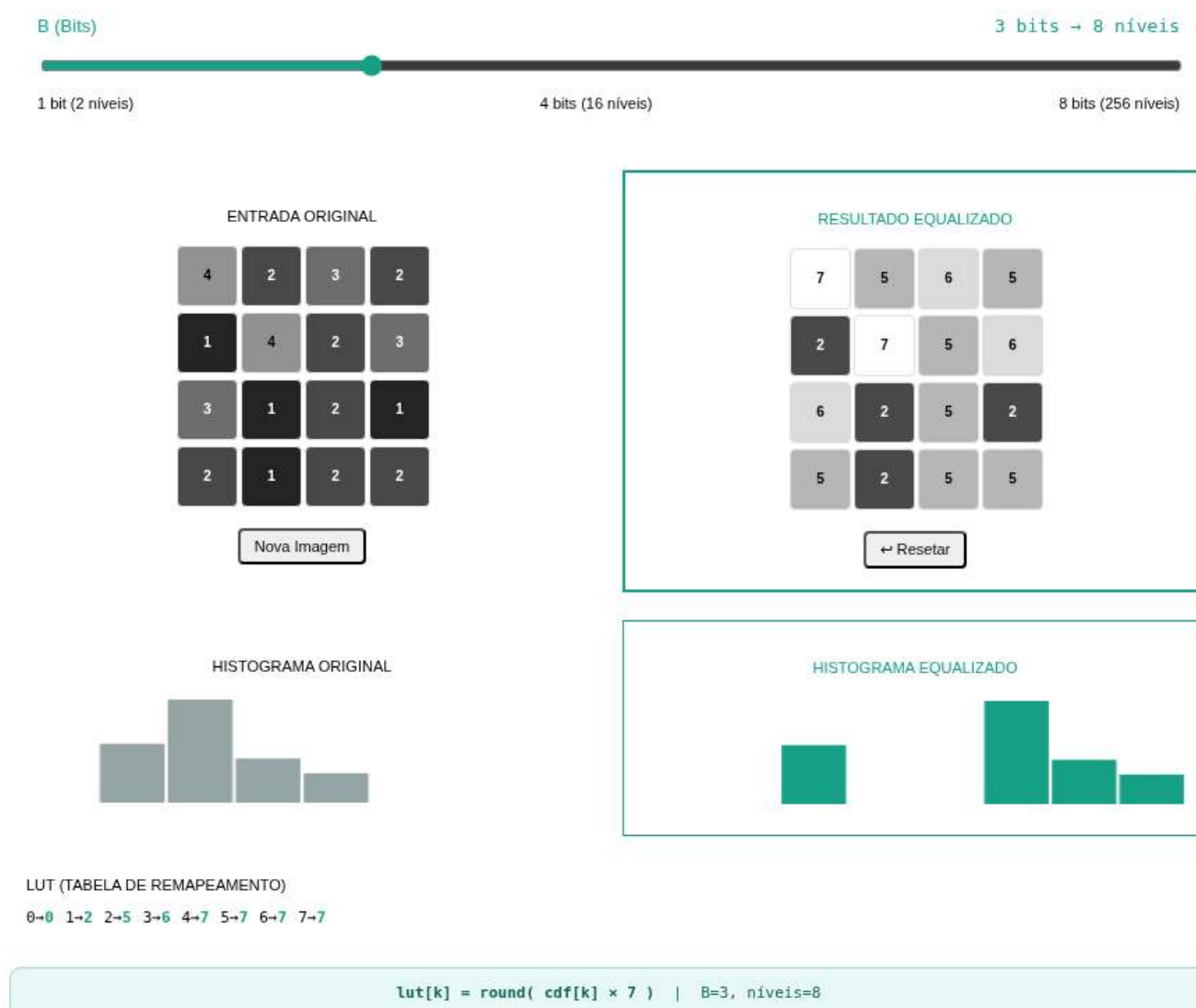


Figura 3.29: Simulador: Equalização de Histograma (L bits)

binária é o mecanismo fundamental para recortar exatamente a área de inspeção, zerando todos os pixels fora dela.

Ver na Figure 3.30 uma simulação deste EP.

3.12.5.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz de pixels f (valores $\in [0, 255]$).
3. **Máscara:** Ler a matriz binária m (valores: apenas 0 ou 255).
4. **Mapeamento:** Para cada pixel (i, j) , aplicar o AND bit a bit:

$$g(i, j) = f(i, j) \text{ AND } m(i, j)$$

onde $255 = 11111111$ e $0 = 00000000$ em binário.

5. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.5.2 Restrições Computacionais

- **AND com 255:** $p \text{ AND } 255 = p$ (todos os bits preservados).
- **AND com 0:** $p \text{ AND } 0 = 0$ (todos os bits zerados).
- **Máscara:** Os únicos valores possíveis na máscara são 0 e 255.
- **Implementação:** Em Python, o AND bit a bit entre inteiros usa o operador `&`.

3.12.5.3 Fundamentação Teórica

Pixel f	Máscara m	Resultado $f \text{ AND } m$
qualquer v	255 (11111111)	v (preservado)
qualquer v	0 (00000000)	0 (zerado)

3.12.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos de f (L linhas).
- Linhas seguintes: Elementos de m (L linhas com valores 0 ou 255).

Saída:

- Matriz resultante $L \times C$.

3.12.5.5 Exemplos

Entrada	Saída	Observação
2	100 150 0	Máscara seleciona região
3	0 80 120	
100 150 200		
50 80 120		
255 255 0		
0 255 255		Alternado preservado/zerado
1	10 0 30 0	
4		
10 20 30 40		
255 0 255 0		

Simulador de máscara AND binária: aplica operação AND pixel a pixel entre imagem e máscara interativa.

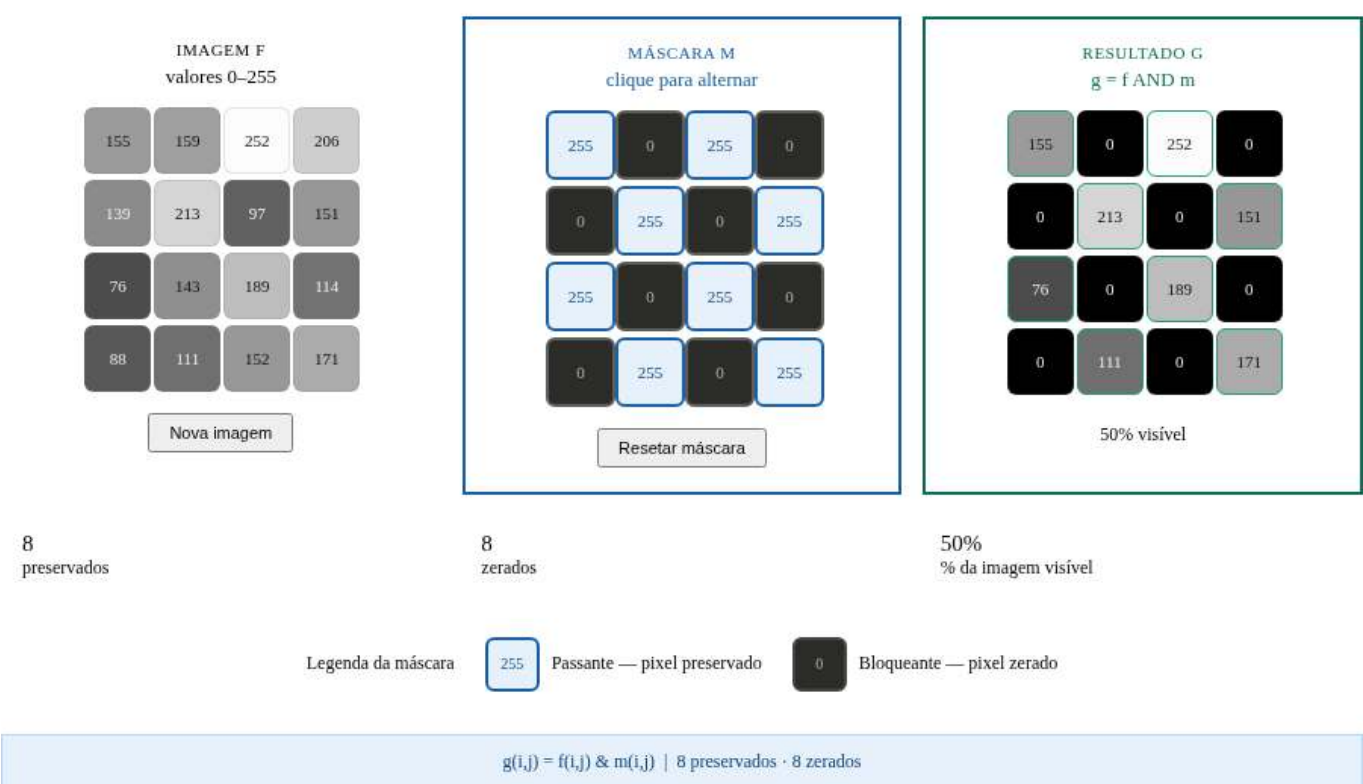


Figura 3.30: Simulador: Aplicação de Máscara AND Binária

```
%%writefile EP03_05.py
# Código Python

Writing EP03_05.py

TestSuite("EP03_05.py").run()
```

3.12.6 EP03_06 Filtro de Média com *Kernel* $N \times N$

Em câmeras de veículos autônomos, imagens capturadas sob chuva ou névoa apresentam ruído gaussiano. O **filtro de média** é amplamente utilizado para sua redução em tempo real, sendo implementado diretamente no **ISP** (*Image Signal Processor*) de sensores **CMOS** (*Complementary Metal-Oxide-Semiconductor*).

Os sensores CMOS são os sensores de imagem usados na maioria das câmeras modernas (*smartphones*, *webcams*, câmeras automotivas etc.). Eles convertem a luz em sinais elétricos, e o ISP processa esses sinais em tempo real — aplicando operações como redução de ruído, balanço de branco e outros ajustes de imagem.

Ver na Figure 3.31 uma simulação deste EP.

3.12.6.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas), C (colunas) e N (tamanho do *kernel*, sempre ímpar).
2. **Dados:** Ler a matriz de pixels f .
3. **Filtro de Média:** Para cada pixel (i, j) **interno** (sem bordas), calcular:

$$g(i, j) = \text{round} \left(\frac{1}{N^2} \sum_{s=-(r)}^r \sum_{t=-(r)}^r f(i+s, j+t) \right), \quad r = \lfloor N/2 \rfloor$$

4. **Tratamento de Borda:** Pixels na borda (onde a janela $N \times N$ ultrapassa os limites) devem ser **copiados diretamente** do original sem modificação.
5. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.6.2 Restrições Computacionais

- **Raio:** $r = \lfloor N/2 \rfloor$ (metade do *kernel*, inteiro).
- **Pixels internos:** (i, j) com $r \leq i < L - r$ e $r \leq j < C - r$.
- **Arredondamento:** Usar arredondamento matemático antes de converter para inteiro.
- **Sem *clipping*:** A média de valores $\in [0, 255]$ permanece em $[0, 255]$.

3.12.6.3 Fundamentação Teórica

Tamanho N	Coefficiente	Pixels na janela	Efeito
3	$1/9 \approx 0.111$	9	Suave
5	$1/25 = 0.04$	25	Médio
7	$1/49 \approx 0.020$	49	Forte

3.12.6.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro N (ímpar, $N \geq 3$).
- Linhas seguintes: Elementos da matriz original.

Saída:

- Matriz filtrada $L \times C$.

3.12.6.5 Exemplos

Entrada	Saída	Observação
3	10 20 30	Apenas borda ($3 \times 3 =$ borda total)
3	40 50 60	
3	70 80 90	
10 20 30		
40 50 60		
70 80 90		
5	0 0 0 0 0	Pixel isolado: todos os 9 pixels internos cuja janela 3×3 inclui o valor 100 recebem $\text{round}(100/9)=11$
5	0 11 11 11 0	
3	0 11 11 11 0	
0 0 0 0 0	0 11 11 11 0	
0 0 0 0 0	0 0 0 0 0	
0 0 100 0 0		
0 0 0 0 0		
0 0 0 0 0		

Simulador de filtro de média com kernel 3×3 ou 5×5 : passe o mouse nas células do resultado para visualizar a janela de vizinhança.

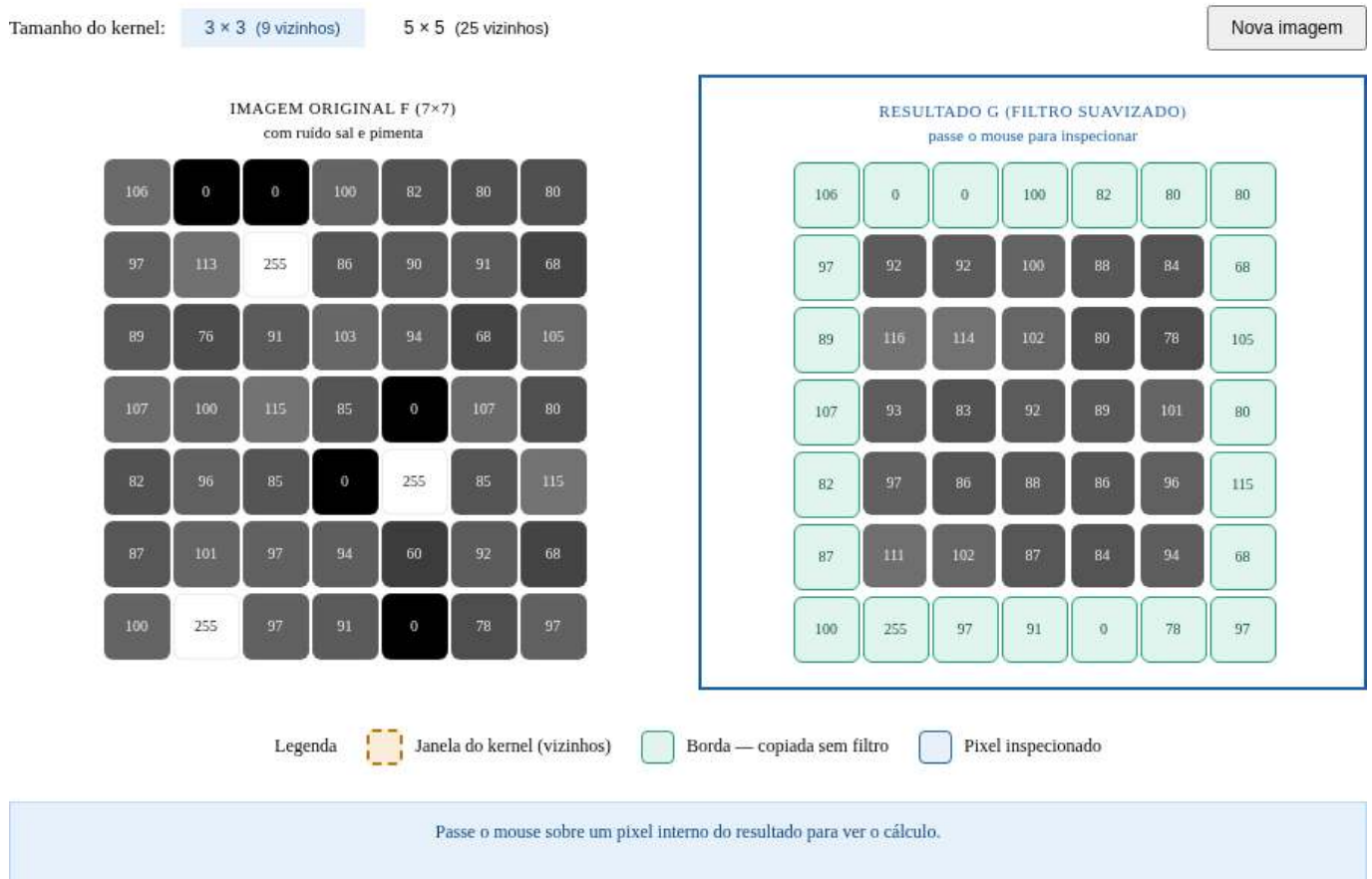


Figura 3.31: Simulador: Filtro de Média com *Kernel* $N \times N$

```
%%writefile EP03_06.py
# Código Python
```

```
Writing EP03_06.py
```

```
TestSuite("EP03_06.py").run()
```

3.12.7 EP03_07 Operador Laplaciano (w4) para Realce de Bordas

Em tomografias de alta resolução, a nitidez das bordas entre tecidos é crítica para diagnóstico. O **operador Laplaciano** é amplamente utilizado em *pipelines* de pré-processamento de imagens médicas para realçar automaticamente os contornos anatômicos antes da segmentação, evitando intervenção manual do radiologista.

Ver na Figure 3.32 uma simulação deste EP.

3.12.7.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz de pixels f .
3. **Laplaciano (w4):** Para cada pixel **interno** (i, j) com $1 \leq i < L - 1$, $1 \leq j < C - 1$, calcular:

$$\nabla^2 f(i, j) = f(i - 1, j) + f(i + 1, j) + f(i, j - 1) + f(i, j + 1) - 4 \cdot f(i, j)$$

4. **Realce:** Calcular a imagem realçada:

$$g(i, j) = \text{clip}(f(i, j) - \nabla^2 f(i, j))$$

5. **Borda:** Pixels na borda são copiados diretamente: $g(i, j) = f(i, j)$.
6. **Saída:** Exibir a matriz realçada $L \times C$.

3.12.7.2 Restrições Computacionais

- **Kernel w4:** $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ — apenas 4-vizinhos.
- **Saturação:** $\text{clip}(x) = \max(0, \min(255, x))$ aplicado ao resultado do realce.
- **Sem arredondamento:** O Laplaciano usa apenas somas/subtrações de inteiros.

3.12.7.3 Fundamentação Teórica

Região	$\nabla^2 f$	Efeito do Realce
Uniforme	≈ 0	Sem alteração
Borda crescente	< 0	Pixel clareado
Borda decrescente	> 0	Pixel escurecido

3.12.7.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .

- Linha 2: Inteiro C .
- Linhas seguintes: Elementos da matriz original.

Saída:

- Matriz realçada $L \times C$.

3.12.7.5 Exemplos

Entrada	Saída	Observação
3 3 0 0 0 0 100 0 0 0 0	0 0 0 0 255 0 0 0 0	Pico isolado: $\text{lap}=-400$, $g=100-(-400)=500 \rightarrow \text{clip}=255$
3 3 50 50 50 50 50 50 50 50 50	50 50 50 50 50 50 50 50 50	Região uniforme: Laplaciano=0, sem alteração

```
%%writefile EP03_07.py
# Código Python
```

```
Writing EP03_07.py
```

```
TestSuite("EP03_07.py").run()
```

3.12.8 EP03_08 Gradiente de Sobel: G_x e G_y

Em robôs exploradores de Marte (como o Perseverance), a detecção de obstáculos é realizada em tempo real por câmeras estereoscópicas. O **operador de Sobel** calcula o gradiente direcional da cena e é utilizado no algoritmo de detecção de bordas para identificar rochas, fissuras e desníveis do terreno que possam comprometer a navegação.

Ver na Figure 3.33 uma simulação deste EP.

3.12.8.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz f .
3. **G_x e G_y :** Para cada pixel **interno** (i, j) com $1 \leq i < L - 1$, $1 \leq j < C - 1$:

$$G_x(i, j) = [f(i - 1, j + 1) + 2f(i, j + 1) + f(i + 1, j + 1)] - [f(i - 1, j - 1) + 2f(i, j - 1) + f(i + 1, j - 1)]$$

$$G_y(i, j) = [f(i + 1, j - 1) + 2f(i + 1, j) + f(i + 1, j + 1)] - [f(i - 1, j - 1) + 2f(i - 1, j) + f(i - 1, j + 1)]$$

4. **Magnitude:** $|\nabla f(i, j)| = \text{clip}(\text{round}(\sqrt{G_x^2 + G_y^2}))$.

Simulador do operador Laplaciano para realce de bordas: imagem original, laplaciano e resultado.

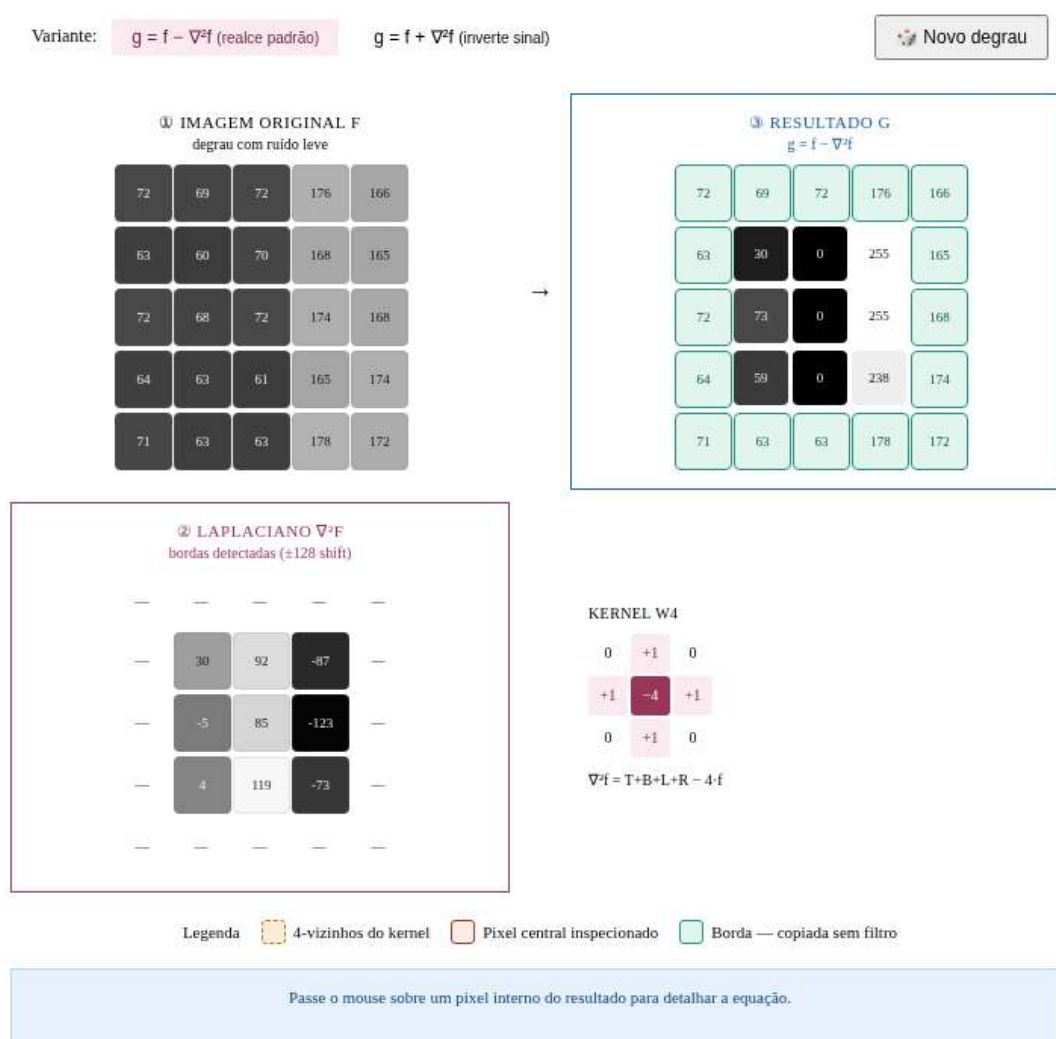


Figura 3.32: Simulador: Operador Laplaciano (w4) para Realce de Bordas

5. **Borda:** Pixels de borda recebem magnitude 0.
6. **Saída:** Exibir a magnitude $L \times C$.

3.12.8.2 Restrições Computacionais

- **Arredondamento:** Aplicar round antes de converter para inteiro.
- **Saturação:** $\text{clip}(x) = \max(0, \min(255, x))$.
- **Raiz quadrada:** Usar $\sqrt{G_x^2 + G_y^2}$ (não a aproximação $|G_x| + |G_y|$).

3.12.8.3 Fundamentação Teórica

Operador	Detecta	Coefficientes diagonais
G_x	Bordas verticais	± 1
G_y	Bordas horizontais	± 1
$ \nabla f $	Todas as bordas	Combinado

3.12.8.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos da matriz.

Saída:

- Magnitude do gradiente, matriz $L \times C$.

3.12.8.5 Exemplos

Entrada	Saída	Observação
3	0 0 0	Imagem nula: gradiente zero
3	0 0 0	
0 0 0	0 0 0	
0 0 0		
0 0 0		
3	0 0 0	Borda vertical central: Gx alto
3	0 255 0	
0 0 255	0 0 0	
0 0 255		
0 0 255		

```
%%writefile EP03_08.py
# Código Python
```

Writing EP03_08.py

```
TestSuite("EP03_08.py").run()
```

Simulador do gradiente de Sobel: imagem original, Gx, Gy e magnitude do gradiente.



Figura 3.33: Simulador: Gradiente de Sobel: Gx e Gy

3.12.9 EP03_09 Filtro da Mediana 3×3

Imagens de radar de abertura sintética (SAR) usadas em monitoramento ambiental e militar sofrem de um tipo específico de ruído chamado *speckle*, que possui características similares ao ruído sal e pimenta. O **filtro da mediana** é o método padrão de remoção desse ruído porque preserva as bordas das estruturas enquanto elimina os pontos espúrios.

Ver na Figure 3.34 uma simulação deste EP.

3.12.9.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz de pixels f .
3. **Filtro da Mediana 3×3:** Para cada pixel **interno** (i, j) com $1 \leq i < L - 1$, $1 \leq j < C - 1$:
 - Coletar os 9 pixels da vizinhança 3×3 : $\{f(i + s, j + t) : s, t \in \{-1, 0, 1\}\}$.
 - Ordenar os 9 valores em ordem crescente.
 - Atribuir $g(i, j)$ ao valor central (posição índice 4, considerando índice 0).

$$g(i, j) = \text{mediana}\{f(i + s, j + t) : s, t \in \{-1, 0, 1\}\}$$

4. **Borda:** Copiar diretamente: $g(i, j) = f(i, j)$.
5. **Saída:** Exibir a matriz filtrada $L \times C$.

3.12.9.2 Restrições Computacionais

- **Janela:** Sempre $3 \times 3 = 9$ elementos.
- **Mediana:** O elemento central da sequência ordenada (índice 4 de 0 a 8).
- **Sem clipping:** A mediana de valores em $[0, 255]$ permanece em $[0, 255]$.
- **Não linear:** O filtro mediana não pode ser expresso como convolução linear.

3.12.9.3 Fundamentação Teórica

Ruído	Filtro de Média	Filtro de Mediana
Sal e pimenta (0 ou 255)	Espalha o ruído	Remove sem distorcer bordas
Gaussiano	Reduz eficazmente	Reduz parcialmente

3.12.9.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos da matriz.

Saída:

- Matriz filtrada $L \times C$.

3.12.9.5 📌 Exemplos

Entrada	Saída	Observação
3	100 100 100	Ponto preto eliminado: mediana de $8 \times 100 + 1 \times 0 = 100$
3	100 100 100	
100 100 100	100 100 100	
100 0 100		
100 100 100		
3	50 50 50	Ponto branco (sal) eliminado
3	50 50 50	
50 50 50	50 50 50	
50 255 50		
50 50 50		

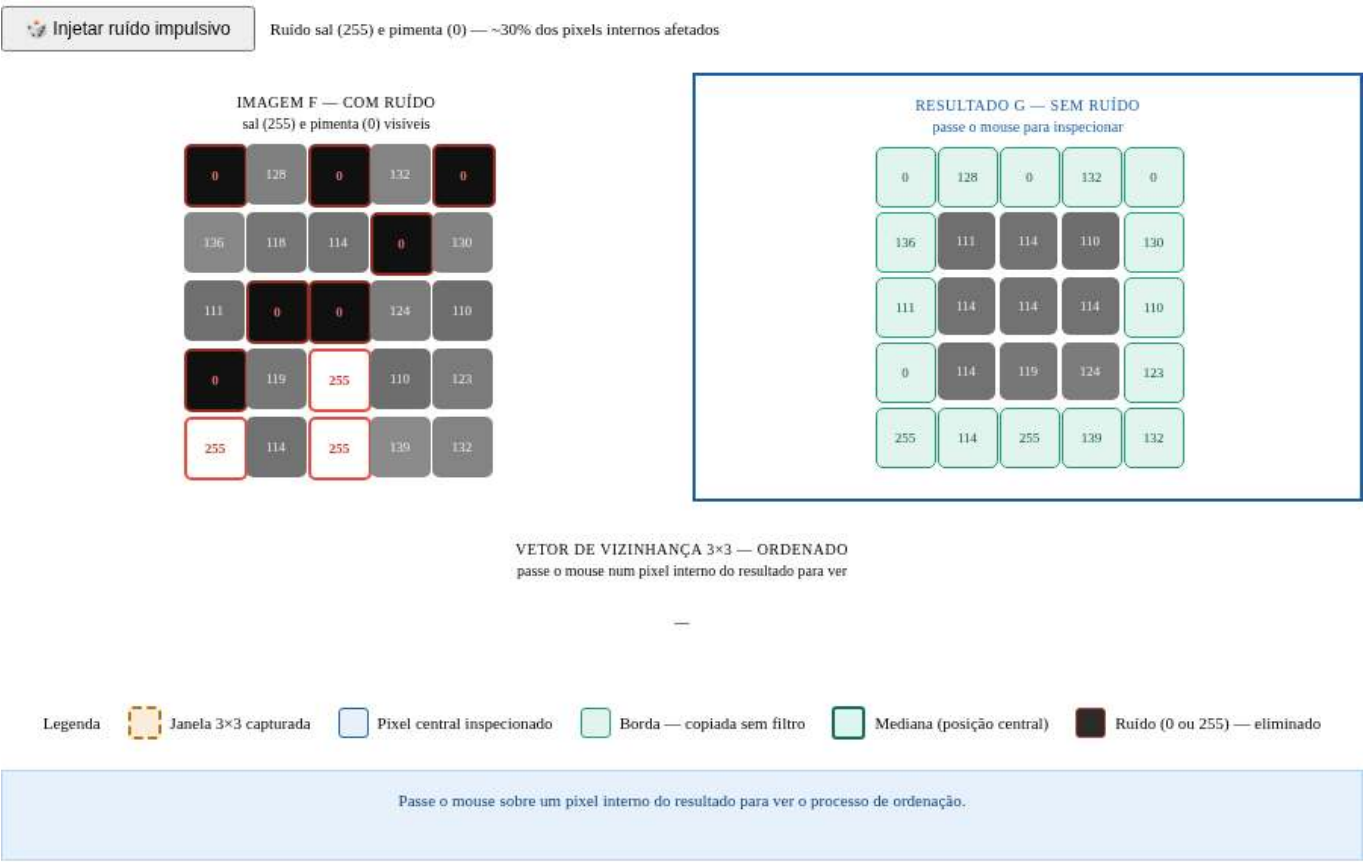


Figura 3.34: Simulador: Filtro da Mediana 3×3

```
%%writefile EP03_09.py
# Código Python

Writing EP03_09.py

TestSuite("EP03_09.py").run()
```

3.12.10 EP03_10 *Unsharp Masking* (USM)

Em sistemas de digitalização de documentos históricos e obras de arte, a nitidez das imagens é fundamental para leitura de textos manuscritos e detalhes ornamentais. O *Unsharp Masking* (USM) é o algoritmo de realce de nitidez padrão utilizado em *scanners* profissionais e softwares como Adobe Photoshop, controlado pelo parâmetro k que determina a intensidade do realce.

Ver na Figure 3.35 uma simulação deste EP.

3.12.10.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Parâmetro:** Ler o valor real k (intensidade do realce, $k \geq 0$).
3. **Dados:** Ler a matriz de pixels f .
4. **Suavização:** Calcular $\bar{f}$ com filtro de média 3×3 (apenas pixels internos; bordas mantidas):

$$\bar{f}(i, j) = \frac{1}{9} \sum_{s=-1}^1 \sum_{t=-1}^1 f(i+s, j+t)$$

5. **Máscara de alta frequência:** $m(i, j) = f(i, j) - \bar{f}(i, j)$.
6. **Realce USM:** Para cada pixel interno:

$$g(i, j) = \text{clip}(\text{round}(f(i, j) + k \cdot m(i, j)))$$

7. **Borda:** $g(i, j) = f(i, j)$ (cópia direta).
8. **Saída:** Exibir a matriz realçada $L \times C$.

3.12.10.2 Restrições Computacionais

- **Arredondamento:** Aplicar `round` antes do clipping.
- **Saturação:** $\text{clip}(x) = \max(0, \min(255, x))$.
- **Operações em float:** Calcular $\bar{f}$ e m em ponto flutuante antes de arredondar o resultado final.
- $k = 0$: Sem realce — a saída é idêntica à entrada (exceto pelas bordas).

3.12.10.3 Fundamentação Teórica

Etapas	Operação	Descrição
1	$\bar{f} = f * \frac{1}{9} \mathbf{1}_{3 \times 3}$	Suavização (baixas frequências)
2	$m = f - \bar{f}$	Máscara (altas frequências)
3	$g = \text{clip}(\text{round}(f + k \cdot m))$	Realce ponderado

3.12.10.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Real k .
- Linhas seguintes: Elementos da matriz original.

Saída:

- Matriz realçada $L \times C$.

3.12.10.5 Exemplos

Entrada	Saída	Observação
3	100 100 100	k=0: sem realce
3	100 100 100	
0.0	100 100 100	
100 100 100		
100 100 100		
100 100 100		
3	50 50 50	k=1: pixel central realçado e saturado
3	50 255 50	
1.0	50 50 50	
50 50 50		
50 200 50		
50 50 50		

```
%%writefile EP03_10.py
# Código Python
```

Writing EP03_10.py

```
TestSuite("EP03_10.py").run()
```

Simulador de Unsharp Masking (USM): visualização do pipeline com imagem original, versão desfocada, máscara de alta frequência e resultado realçado.

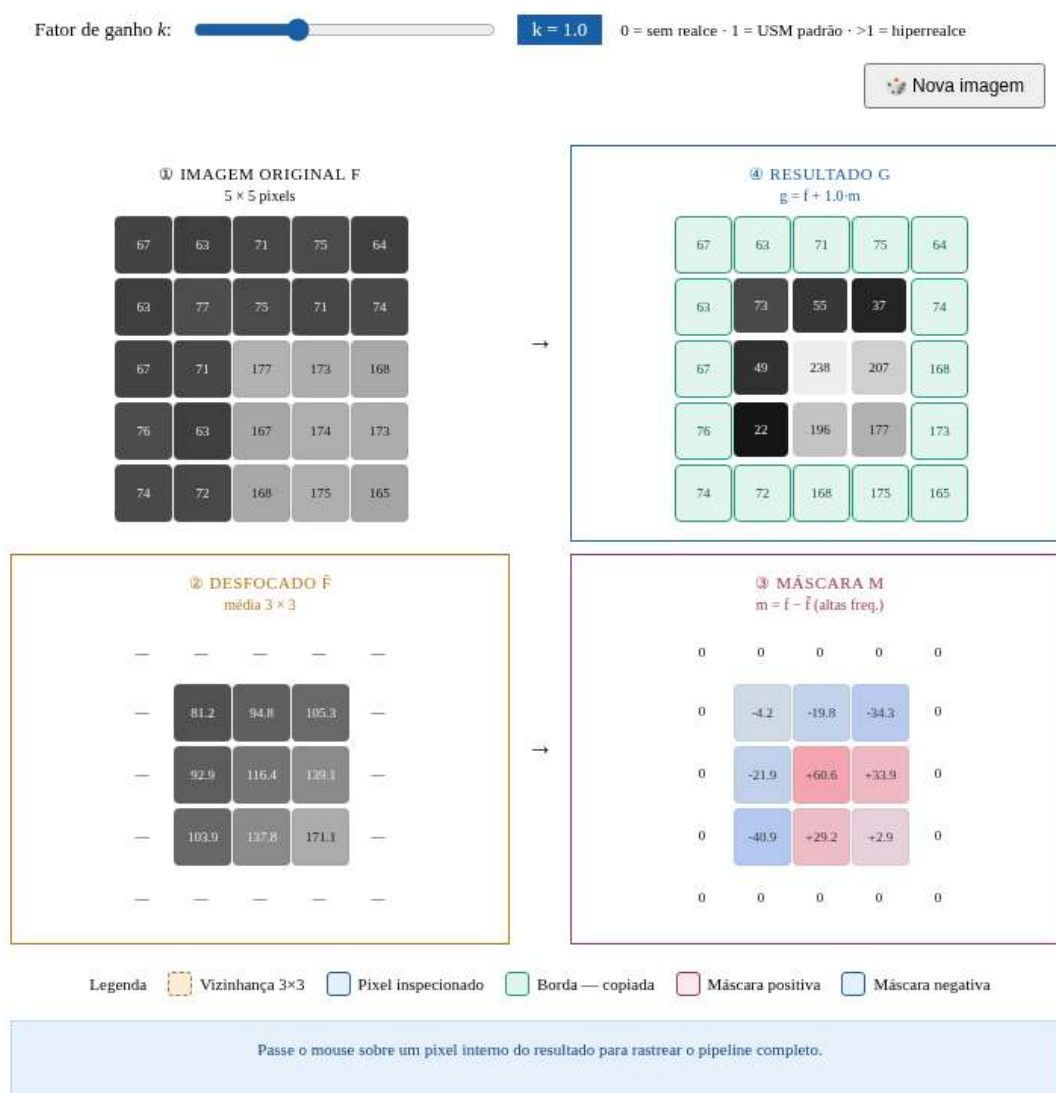


Figura 3.35: Simulador: *Unsharp Masking* (USM)

Capítulo 4

Morfologia Matemática e Segmentação de Imagens

 Executar  Colab  Abrir si-md2  GitHub

Este capítulo apresenta dois temas fundamentais do Processamento Digital de Imagens (PDI): a **morfologia matemática** e a **segmentação de imagens**. A morfologia matemática fornece um arcabouço teórico baseado na teoria dos conjuntos para analisar, refinar e quantificar a forma de objetos em imagens binárias e em tons de cinza, por meio de operadores fundamentais como **erosão** e **dilatação**. A segmentação, por sua vez, tem como objetivo particionar a imagem em regiões de interesse, separando objetos do fundo e produzindo representações adequadas para análise e interpretação.

O capítulo inicia com a **limiarização**, uma das técnicas mais importantes de segmentação, introduzindo o método automático de **Otsu** e revisitando a análise de histogramas por meio da variância interclasses, apresentada no Capítulo 1. Em seguida, são estudados os principais operadores da morfologia matemática, incluindo erosão, dilatação, abertura, fechamento e reconstrução morfológica, que permitem refinar máscaras binárias e preservar estruturas relevantes dos objetos. Por fim, são apresentadas técnicas de segmentação baseada em regiões, como a **rotulação de componentes conexos**, a **transformada de distância** e o algoritmo *watershed* baseado em marcadores, culminando na extração de descritores geométricos e na geração de *bounding boxes* compatíveis com sistemas modernos de detecção de objetos.

4.1 Objetivos

Ao final deste capítulo, você será capaz de:

- **Aplicar limiarização:** Compreender o critério automático de Otsu por maximização da variância interclasses (σ_B^2) e selecionar estratégias adequadas de pré-processamento para facilitar a segmentação;
- **Dominar morfologia binária:** Compreender e aplicar erosão ($A \ominus B$) e dilatação ($A \oplus B$) como operadores fundamentais, derivando abertura ($A \circ B$), fechamento ($A \bullet B$) e operações baseadas em reconstrução morfológica, como `mm.clohole` e `mm.edgeoff`;
- **Aplicar morfologia em tons de cinza:** Utilizar gradiente morfológico e filtros *top-hat* para realce e análise de estruturas locais;
- **Rotular componentes conexos:** Identificar e separar regiões conectadas em imagens binárias por meio de algoritmos de rotulação;
- **Aplicar transformada de distância:** Interpretar e calcular distâncias ao fundo utilizando abordagens morfológicas e métricas geométricas;
- **Segmentar por regiões:** Construir *pipelines* de segmentação baseados em marcadores utilizando Transformada de Distância e o algoritmo *watershed*;
- **Extrair descritores geométricos:** Calcular propriedades como área, perímetro, centróide, circularidade e *bounding boxes* por meio de `cv2.connectedComponentsWithStats` e `cv2.findContours`;
- **Relacionar PDI e visão computacional:** Compreender como descritores extraídos por segmentação podem ser convertidos para formatos utilizados por detectores modernos, como YOLO.

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

version = getattr(morph, "__version__", "local_file")
print(f" Ambiente pronto. Módulo 'morph' carregado (versão: {version}).")
```

Ambiente pronto. Módulo 'morph' carregado (versão: 1.1.2).

4.2 Limiarização

A **limiarização** (*thresholding*) é uma das formas mais simples e eficientes de segmentação de imagens. Seu objetivo é classificar cada pixel em duas classes de intensidade, normalmente associadas a *objeto* e *fundo*:

$$g(x, y) = \begin{cases} 255, & \text{se } f(x, y) > T \\ 0, & \text{caso contrário} \end{cases} \quad (4.1)$$

em que $f(x, y)$ representa a intensidade do pixel na imagem original e $g(x, y)$ a imagem binária resultante.

A escolha do limiar T é importante para a qualidade da segmentação. O método de **Otsu** determina automaticamente o limiar ótimo ao maximizar a **variância interclasses** σ_B^2 e definida por:

$$\sigma_B^2(T) = w_0(T) w_1(T) [\mu_0(T) - \mu_1(T)]^2 \quad (4.2)$$

em que:

- $w_0(T)$ e $w_1(T)$ são as probabilidades acumuladas das classes fundo e objeto;
- $\mu_0(T)$ e $\mu_1(T)$ são as médias de intensidade dessas classes;
- $\sigma_B^2(T)$ representa a variância interclasses para um dado limiar T .

O método funciona melhor quando o histograma apresenta dois grupos de intensidades relativamente separados. Para isso, o algoritmo avalia todos os limiares possíveis da imagem — tipicamente no intervalo $[0, 255]$ para imagens de 8 bits — e seleciona o valor que maximiza a variância entre classes, denotada por σ_B^2 :

$$T^* = \arg \max_{T \in [0, 255]} \sigma_B^2(T)$$

i Otsu assume histogramas bimodais

O método de Otsu produz melhores resultados quando o histograma apresenta dois picos bem definidos (*bimodalidade*), correspondentes ao fundo e ao objeto. Quanto maior a separação entre esses picos e mais pronunciado o máximo de σ_B^2 , mais confiável tende a ser o limiar obtido.

Em imagens com iluminação não uniforme ou múltiplas regiões de intensidade, técnicas de **limiarização adaptativa** — nas quais o limiar é calculado localmente — costumam produzir segmentações mais robustas.

O índice B em σ_B^2 significa *between classes* (*entre classes*). Assim, σ_B^2 representa a **variância entre as classes** (*between-class variance*).

4.2.1 Imagem de Moedas

A imagem utilizada para praticar a segmentação é uma fotografia de coleção de moedas de diferentes países e épocas (Figure 4.1). Crédito: GAZI.MD.AHAD (CC BY-SA 4.0). Ela apresenta objetos circulares com bordas bem definidas, sendo ideal para demonstrar limiarização, operadores morfológicos, transformada de distância, *watershed* e descritores de forma.

```
import os

url      = "https://upload.wikimedia.org/wikipedia/commons/2/25/GAZI.MD.AHAD_11.jpg"
caminho = "imagens/coins.jpg"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_coins_color = np.array(img_obj)
img_coins_gray  = mm.gray(img_coins_color)

print(f"Dimensões [y,x,c]: {img_coins_color.shape}")
mm.show(img_coins_color, scale=30)
```

Dimensões [y,x,c]: (2560, 1920, 3)



Figura 4.1: Imagem com moedas de vários tipos. Crédito: GAZI.MD.AHAD (CC BY-SA 4.0).

4.2.2 Escolha do Pré-processamento para o Otsu

A qualidade do método de Otsu depende diretamente de quão **bimodal** é o histograma da imagem de entrada. A imagem original das moedas apresenta iluminação não uniforme e moedas escuras (cobre oxidado) com intensidades próximas às do fundo escuro, tornando o histograma pouco bimodal.

Para minimizar essas limitações, serão avaliadas duas técnicas clássicas de pré-processamento, apresentadas no capítulo anterior, aplicadas antes da etapa de limiarização. A Table 4.1 resume as características de cada abordagem. Essas técnicas buscam aumentar a separação entre objeto e fundo, tornando o histograma mais próximo de uma distribuição bimodal.

Tabela 4.1: Técnicas de pré-processamento avaliadas para melhorar a separação entre moedas e fundo antes da aplicação do método de Otsu.

Técnica	O que faz	Quando usar
CLAHE	Equalização de histograma adaptativa local	Baixo contraste global ou regional
Gaussiano	Suavização por convolução com gaussiana	Ruído de alta frequência (textura do fundo)

A função `cv2.createCLAHE(clipLimit, tileGridSize)` divide a imagem em blocos e aplica equalização de histograma em cada um deles, limitando a amplificação de ruído por meio do parâmetro `clipLimit`. Já o filtro Gaussiano (`cv2.GaussianBlur`) suaviza texturas finas que poderiam criar falsos picos no histograma.

Para comparar objetivamente qual pré-processamento produz a melhor entrada para o método de Otsu, serão apresentados, para cada versão da imagem, a própria imagem, o histograma com o limiar ótimo T^* destacado, a curva $\sigma_B^2(T)$ e o resultado da binarização.

i Critério de comparação

A versão que apresentar o maior valor de σ_B^2 em seu pico fornece a melhor separação entre as classes de fundo e objeto e, conseqüentemente, a melhor entrada para o método de Otsu (Figure 4.2).

```
import cv2, io

def otsu_criterio(img):
    h=mm.hist(img); p=h/h.sum(); # histograma e probabilidades
    sigma2=np.zeros(len(p))
    for T in range(1,len(p)): # percorre limiares
        w0,w1=p[:T].sum(),p[T:].sum() # probabilidades das classes
        if w0*w1==0: continue # evita divisão por zero
        mu0=(np.arange(T)*p[:T]).sum()/w0 # média fundo
        mu1=(np.arange(T,len(p))*p[T:]).sum()/w1 # média objeto
        sigma2[T]=w0*w1*(mu0-mu1)**2 #  $\sigma_B^2(T)$ 
    return sigma2,np.argmax(sigma2) # curva e T ótimo

def fig2img(fig):
    b=io.BytesIO(); fig.savefig(b,format='png',dpi=100) # figura → buffer
    plt.close(fig); b.seek(0)
    return np.array(plt.imread(b)) # buffer → array

def plot_curve(y,T,title,ylabel,color):
    fig,ax=plt.subplots(figsize=(4,3))
    ax.plot(y,color=color) if ylabel==" $\sigma_B^2$ " else ax.bar(range(len(y)),y,color=color,width=1)
    ax.axvline(T,color='red',lw=2,label=f"T*={T}") # limiar ótimo
    ax.set(xlabel="T" if ylabel==" $\sigma_B^2$ " else "Intensidade",ylabel=ylabel)
    ax.legend(fontsize=8); plt.tight_layout()
```



```

return fig2img(fig)

# Pré-processamentos
clahe=cv2.createCLAHE(clipLimit=2.0,tileGridSize=(8,8))
img_clahe = clahe.apply(img_coins_gray)
img_gauss = cv2.GaussianBlur(img_clahe,(5,5),0)
imgs0=[("Original",img_coins_gray),
        ("CLAHE",img_clahe),
        ("CLAHE+Gauss",img_gauss)]

# Tabela comparativa
print(f"{'Versão':<18}{'T*':>6}{'2B pico':>14}")
print("-"*40)

imgs,titles=[],[]
for nome,img in imgs0:
    sigma2,T=otsu_criterio(img) # calcula 2B(T)
    print(f"{'nome':<18}{'T':>6}{'sigma2[T]:>14.4e}")
    imgs += [
        img, # imagem
        plot_curve(mm.hist(img),T,f"Hist T*={T}", "Freq.", "steelblue"),
        plot_curve(sigma2,T,"2B(T)", "2B", "darkorange"),
        mm.threshold(img) # Otsu final
    ]
    titles += [nome,f"Hist T*={T}", "2B(T)",f"Otsu T*={T}"]

# Exibição final
mm.show(imgs,titles=titles,cols=4,figsize=(12,12),dpi=200)

```

Versão	T*	² B pico
Original	105	1.9437e+03
CLAHE	122	2.4695e+03
CLAHE+Gauss	123	2.4283e+03

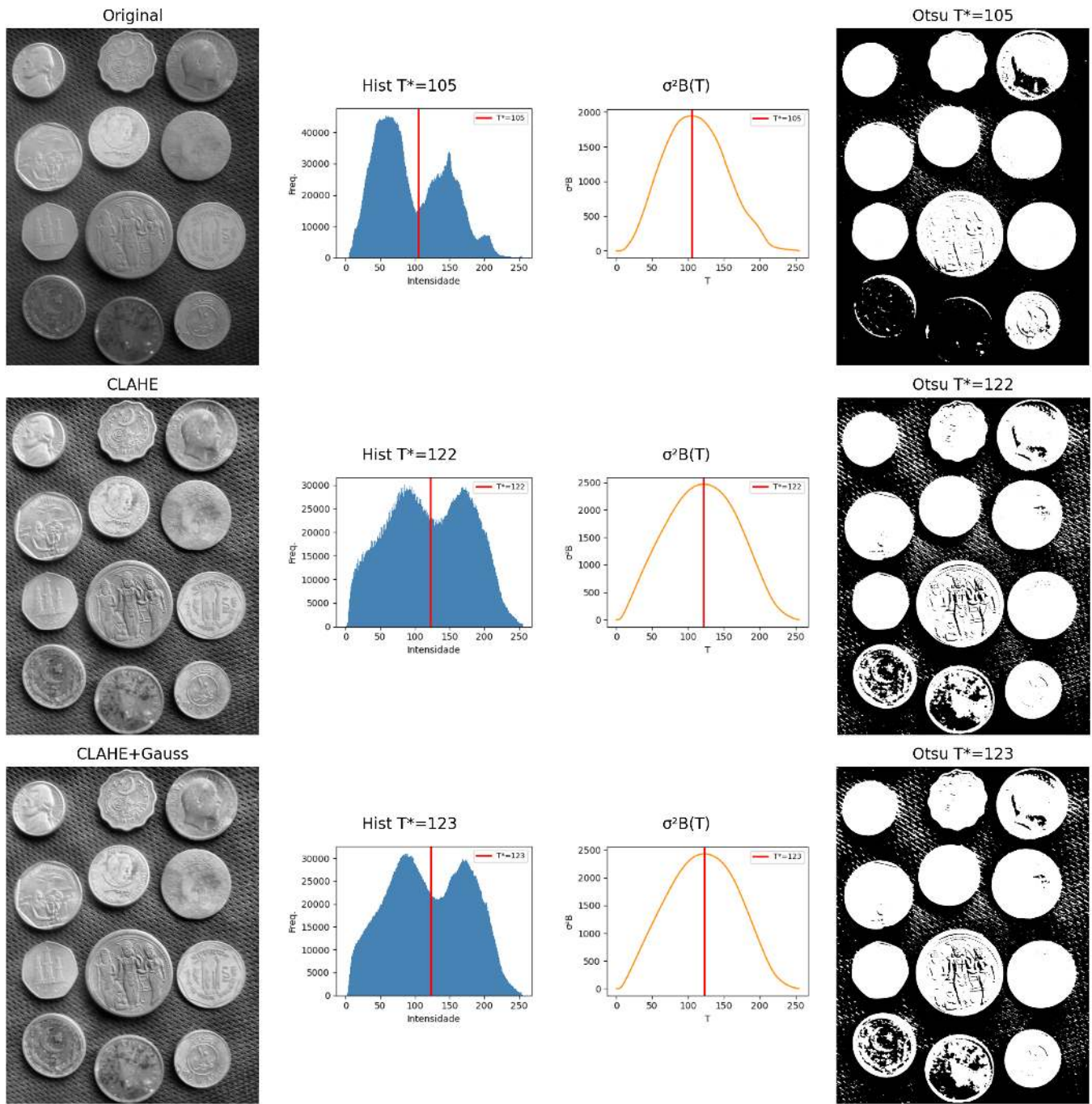


Figura 4.2: Comparação dos pré-processamentos: imagem | histograma+ T^* | $\sigma^2 B(T)$ | Otsu. A melhor separação bimodal indica o limiar mais confiável.

4.2.3 Resultado: CLAHE como Melhor Pré-processamento

A análise da Figure 4.2 indica que o **CLAHE** obteve o maior valor da variância inter-classes ($\sigma_B^2 \approx 2,47 \times 10^3$), com limiar ótimo $T^* = 122$. Embora a combinação **CLAHE+Gaussiano** tenha produzido resultado muito semelhante ($\sigma_B^2 \approx 2,43 \times 10^3$, $T^* = 123$), o critério quantitativo do método de Otsu favorece ligeiramente o uso do CLAHE isoladamente.

Em termos visuais, as imagens binarizadas obtidas com CLAHE e CLAHE+Gaussiano são praticamente equivalentes. A diferença entre as duas abordagens torna-se mais evidente na análise dos histogramas e dos valores de $\sigma_B^2(T)$ do que na inspeção direta das segmentações resultantes. Assim, a escolha do CLAHE baseia-se principalmente na maximização da separação estatística entre as classes de fundo e objeto.

💡 Interpretação dos resultados

Observe que os pré-processamentos com CLAHE e CLAHE+Gaussiano produzem histogramas e limiares ótimos muito próximos ($T^* = 122$ e $T^* = 123$). Consequentemente, as imagens binarizadas resultantes também são bastante semelhantes. Nesse caso, a decisão não é baseada em diferenças visuais marcantes, mas no critério objetivo do método de Otsu: o maior valor de σ_B^2 indica a melhor separação entre as classes.

4.3 Morfologia Matemática

A **morfologia matemática** é uma teoria baseada em conjuntos utilizada para analisar a forma e a estrutura de objetos em imagens. Diferentemente dos filtros lineares apresentados no Capítulo 3, os operadores morfológicos são **não lineares**, pois se baseiam em operações de mínimo, máximo e inclusão espacial, em vez de combinações lineares de intensidades. Esses operadores atuam sobre a vizinhança de cada pixel por meio de um **elemento estruturante** $\mathbb{B}$, responsável por definir a forma e o tamanho da região analisada.

Em imagens binárias e em tons de cinza com elementos planos, o elemento estruturante transladado para a posição x é definido espacialmente como:

$$\mathbb{B}_x = \{x + b \mid b \in \mathbb{B}\}$$

Nas regiões de borda da imagem, parte do conjunto $\mathbb{B}_x$ pode extrapolar o domínio físico da cena ($\mathbb{E}$). Para garantir a consistência matemática dos operadores primitivos nessas fronteiras, assume-se teoricamente que o espaço exterior ao domínio da imagem é preenchido com o **elemento neutro** da operação correspondente (infinito positivo para a erosão e infinito negativo para a dilatação), impedindo que o ambiente externo corrompa as estruturas internas do objeto.

Quando o elemento estruturante associa pesos a seus elementos — isto é, $b : \mathbb{B} \rightarrow \mathbb{Z}$ — ele é denominado **função estruturante** ou **elemento estruturante não plano**.

Desenvolvida por Matheron e Serra na década de 1960 para imagens binárias e posteriormente estendida a tons de cinza, a morfologia matemática fundamenta operadores como gradiente morfológico, *top-hat*, *watershed* e transformada de distância, todos derivados de dois primitivos: **erosão** e **dilatação** (Matheron, 1975; Serra, 1982).

4.3.1 Erosão e Dilatação

Os dois operadores primitivos são definidos de forma unificada para imagens em tons de cinza ($f : \mathbb{E} \rightarrow \mathbb{Z}$) e, por restrição ao domínio $\{0, 1\}$, também para imagens binárias.

4.3.1.1 Erosão

A **Erosão** de uma imagem f por uma função estruturante $b : \mathbb{B} \rightarrow \mathbb{Z}$ é definida formalmente por:

$$\varepsilon_b(f)(x) = (f \ominus b)(x) = \min_{z \in \mathbb{B}} \{ f(x + z) - b(z) \}, \quad \forall x \in \mathbb{E} \quad (4.3)$$

Na prática, a erosão substitui a intensidade do pixel x pelo mínimo valor resultante da diferença entre a imagem e o elemento estruturante na vizinhança definida pelo domínio $\mathbb{B}$. Valores positivos nos pesos de $b(z)$ forçam o resultado local para baixo, “cavando” o relevo da imagem mais profundamente e intensificando a erosão.

No caso **plano** (onde os pesos são nulos dentro do domínio, ou seja, $b \equiv 0$), a expressão simplifica-se para o mínimo local puro:

$$\varepsilon_B(f)(x) = \min\{f(y) : y \in \mathbb{B}_x\}$$

Em imagens binárias, essa operação equivale a exigir que o conjunto $\mathbb{B}$, transladado para a coordenada x , esteja **completamente contido** no objeto A :

$$A \ominus \mathbb{B} = \{z \in \mathbb{E} \mid \mathbb{B}_z \subseteq A\}$$

Efeito Visual: *Encolhe* objetos e estruturas claras, eliminando protuberâncias, picos brilhantes ou ruídos que sejam geometricamente menores que o domínio $\mathbb{B}$.

4.3.1.2 Implementação da erosão

A versão didática `mm.ero0` implementa o caso particular de erosão com **elemento estruturante plano**. Para cada pixel (y, x) , a função percorre os vizinhos espaciais permitidos por B e armazena o menor valor encontrado na imagem de entrada f , reproduzindo diretamente a operação de mínimo local descrita na Equation 4.3 para $b \equiv 0$.

Observe que os valores dos vizinhos são sempre lidos de forma estática da imagem original f ; a matriz de saída g é utilizada exclusivamente para registrar o mínimo acumulado da vizinhança corrente. Dessa forma, o resultado final é invariante em relação à ordem de varredura dos pixels (seja por linhas ou colunas).

A função auxiliar `_viz` calcula as coordenadas dos vizinhos válidos dentro dos limites físicos da imagem. Nas bordas, a inicialização do acumulador em 255 emula com exatidão o preenchimento por elemento neutro exigido pela teoria. Já a função de interface `mm.ero` recorre à implementação nativa e otimizada do OpenCV (`cv2.erode`) quando o elemento estruturante é plano, chaveando para a rotina geral `mm.ero1` caso o elemento possua pesos topográficos.

O exemplo computacional a seguir ilustra a aplicação de um elemento estruturante em cruz (`mm.secross()`) em destaque na Figure 4.3, comparando a execução da variante didática em laço (`mm.ero0`) com o motor computacional do OpenCV (`mm.ero`).

```
B_cruz = mm.secross()
mm.drawImgPlt(B_cruz, scale=20)
```

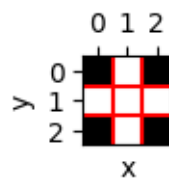


Figura 4.3: Elemento estruturante em formato de cruz (B_{cruz}) utilizado para conectividade-4.

```
def _viz(f, B, y, x):
    """Gera (vy, vx, b_val) para cada vizinho válido de (y,x)."""
    H, W = f.shape
    Bh, Bw = B.shape
    oh, ow = -Bh/2 + 0.5, -Bw/2 + 0.5 # offsets fixos
    for by, bx in np.ndindex(Bh, Bw):
        vy, vx = int(y + by + oh), int(x + bx + ow)
        if 0 <= vy < H and 0 <= vx < W:
            yield vy, vx, B[by, bx]

def ero(f, Bc=np.zeros((3,3),dtype='uint8')):
    """Erosão (OpenCV ou com pesos)."""
    try:
        return cv2.erode(f, Bc)
```

```

except: return mm.ero1(f, Bc)

def ero0(f, Bc=np.ones((3,3),dtype='uint8')):
    """Erosão clássica sem pesos."""
    g = np.empty_like(f)
    for y in range(f.shape[0]):
        for x in range(f.shape[1]):
            g[y,x] = 255
            for vy,vx,bv in mm._viz(f,Bc,y,x):
                if bv and g[y,x] > f[vy,vx]: g[y,x] = f[vy,vx]
    return g

# Script de Teste e Validação
B = mm.seccross()
print("Elemento estruturante B:")
print(mm.drawImage(B))

f = mm.randomImage(5,5)
print("Imagem original f:")
print(mm.drawImage(f))

print("Erosão Plana Didática (ero0):")
print(mm.drawImage(ero0(f, B)))

print("Erosão Otimizada OpenCV (ero):")
print(mm.drawImage(ero(f, B)))

```

```

Elemento estruturante B:
0 1 0
1 1 1
0 1 0

Imagem original f:
1 9 8 0 8
3 7 7 0 2
8 5 5 1 2
4 3 6 6 0
8 9 4 1 9

Erosão Plana Didática (ero0):
1 1 0 0 0
1 3 0 0 0
3 3 1 0 0
3 3 3 0 0
4 3 1 1 0

Erosão Otimizada OpenCV (ero):
1 1 0 0 0
1 3 0 0 0
3 3 1 0 0
3 3 3 0 0
4 3 1 1 0

```

A função `_viz` utiliza `yield` para gerar cada vizinho sob demanda, sem armazenar todos os resultados em memória em uma lista. No experimento abaixo, uma janela de 3000×3000 produz 9 milhões de vizinhos. A implementação baseada em lista consumiu mais de 1 GB de RAM e levou aproximadamente 46 s, enquanto a versão com `yield` consumiu memória desprezível e executou em 38 s. Em processamento de imagens, geradores são especialmente úteis para percorrer grandes vizinhanças de forma eficiente.

```
import tracemalloc

def lista(n): return [(i, i) for i in range(n)]
def gera(n):
    for i in range(n): yield i, i

for nome, f in [("LISTA", lista), ("YIELD", gera)]:
    tracemalloc.start()
    sum(x+y for x,y in f(9_000_000))
    _, pico = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    print(f"{nome}: {pico/1024/1024:.1f} MB")
```

```
LISTA: 830.8 MB
YIELD: 0.0 MB
```

4.3.1.3 Dilatação

A **Dilatação** de uma imagem f por uma função estruturante $b : \mathbb{B} \rightarrow \mathbb{Z}$ é definida formalmente por:

$$\delta_b(f)(x) = (f \oplus b)(x) = \max_{z \in \mathbb{B}} \{ f(x - z) + b(z) \}, \quad \forall x \in \mathbb{E} \quad (4.4)$$

Na prática, a dilatação substitui a intensidade do pixel x pelo maior valor resultante da soma entre a imagem e o elemento estruturante na vizinhança definida. O argumento de inversão espacial ($x - z$) indica que a dilatação avalia implicitamente o elemento transposto (refletido) $\hat{b}$, propriedade fundamental para assegurar a dualidade matemática em relação à erosão.

No caso **plano** (onde os pesos são nulos dentro do domínio, ou seja, $b \equiv 0$), a expressão reduz-se ao máximo local puro:

$$\delta_B(f)(x) = \max \{ f(y) : y \in \mathbb{B}_x \}$$

Em imagens binárias, essa operação equivale a exigir que o conjunto refletido $\hat{\mathbb{B}}$, transladado para a coordenada x , possua **interseção não vazia** com o objeto A :

$$A \oplus \mathbb{B} = \{ z \in \mathbb{E} \mid \hat{\mathbb{B}}_z \cap A \neq \emptyset \}$$

Efeito Visual: *Expand*e as estruturas claras da imagem, aumentando o preenchimento de objetos, conectando componentes próximos e eliminando canais, fossas escuras ou vales que sejam geometricamente menores que o domínio $\mathbb{B}$.

4.3.1.4 Implementação da dilatação

A versão didática `mm.dil0` implementa o caso particular de dilatação com **elemento estruturante plano**. Para cada pixel (y, x) , a função percorre os vizinhos espaciais permitidos por B e armazena o maior valor encontrado na imagem de entrada f , reproduzindo diretamente a operação de máximo local para $b \equiv 0$.

Antes de iniciar a varredura espacial, o elemento estruturante sofre uma reflexão geométrica por meio da instrução `np.flip(Bc)` para construir explicitamente a matriz transposta $\hat{B}$ exigida pela teoria. Em máscaras perfeitamente simétricas (como cruces, quadrados e discos centrados na origem), essa reflexão não altera o arranjo dos pixels; contudo, para elementos assimétricos, tal etapa é estritamente necessária para garantir a equivalência com as definições formais e salvaguardar as leis de dualidade.

Assim como verificado no operador de erosão, os valores dos vizinhos são sempre lidos de forma estática a partir da matriz original f , enquanto a matriz de saída g atua puramente como o registrador do máximo acumulado da vizinhança. Nas fronteiras da imagem, a inicialização do acumulador em 0 emula com exatidão

o preenchimento externo por elemento neutro ($-\infty$, ou zero em representações de 8 bits), garantindo que as bordas físicas da cena sejam dilatadas em perfeita conformidade com o padrão adotado pelo OpenCV.

O exemplo computacional abaixo ilustra a aplicação prática de um elemento em cruz (`mm.secross()`), validando a consistência entre a lógica em laços (`mm.dil0`) e o método nativo industrial (`mm.dil`).

```
def dil(f, Bc=np.zeros((3,3),dtype='uint8')):
    """Dilatação (OpenCV ou com pesos)."""
    try:    return cv2.dilate(f, Bc)
    except: return mm.dil1(f, Bc)

def dil0(f, Bc=np.zeros((3,3),dtype='uint8')):
    """Dilatação plana seguindo rigorosamente a teoria."""
    g = np.empty_like(f)
    Bc = np.flip(Bc)      # reflexão explícita: B
    for y in range(f.shape[0]):
        for x in range(f.shape[1]):
            g[y,x] = 0 # Inicializa com o valor mínimo para buscar o máximo
            for vy,vx,bv in mm._viz(f,Bc,y,x):
                if bv and g[y,x] < f[vy,vx]:
                    g[y,x] = f[vy,vx]

    return g

# Script de Teste e Validação
B = mm.secross()
print("Elemento estruturante B:")
print(mm.drawImage(B))

f = mm.randomImage(5,5)
print("Imagem original f:")
print(mm.drawImage(f))

print("Dilatação Plana Didática (dil0):")
print(mm.drawImage(dil0(f, B)))

print("Dilatação Otimizada OpenCV (dil):")
print(mm.drawImage(dil(f, B)))
```

```
Elemento estruturante B:
0 1 0
1 1 1
0 1 0

Imagem original f:
8 3 5 7 1
5 6 1 8 9
0 1 8 1 0
5 5 4 4 1
9 4 0 6 9

Dilatação Plana Didática (dil0):
8 8 7 8 9
8 6 8 9 9
5 8 8 8 9
9 5 8 6 9
9 9 6 9 9

Dilatação Otimizada OpenCV (dil):
8 8 7 8 9
8 6 8 9 9
```

```

5 8 8 8 9
9 5 8 6 9
9 9 6 9 9

```

i Nota: O Confronto dos Sinais ($f(x+z)$ vs $f(x-z)$)

Compare as definições formais da erosão (Equation 4.3) e da dilatação (Equation 4.4). Considere um elemento estruturante assimétrico à direita $\mathbb{B} = \{0, 1\}$ (origem e um pixel à direita) aplicado na posição $x = 10$.

1. Na Erosão (Equation 4.3):

$$\min\{f(x+z) - b(z)\}$$

- $z = 0 \Rightarrow f(10+0) = \mathbf{f(10)}$
- $z = 1 \Rightarrow f(10+1) = \mathbf{f(11)}$

O operador consulta o pixel atual (10) e o pixel à direita (11), preservando a orientação original de $\mathbb{B}$.

2. Na Dilatação (Equation 4.4):

$$\max\{f(x-z) + b(z)\}$$

- $z = 0 \Rightarrow f(10-0) = \mathbf{f(10)}$
- $z = 1 \Rightarrow f(10-1) = \mathbf{f(9)}$

Devido ao sinal negativo ($-z$), avançar no elemento estruturante corresponde a recuar na imagem, fazendo com que a dilatação consulte o pixel à esquerda (9).

A função `_viz`, utilizada em `morph.py`, gera vizinhos por deslocamentos aditivos da forma $x+z$. Por esse motivo, a implementação de `mm.dil0` reflete previamente o elemento estruturante por meio de `np.flip(B)`. Após a reflexão, a varredura baseada em $x+z$ passa a acessar exatamente os mesmos pontos definidos pela expressão teórica $f(x-z)$ da dilatação em Equation 4.4.

Para elementos estruturantes simétricos (como discos, quadrados e cruzes centradas), a reflexão não altera a máscara. Já para elementos assimétricos, essa etapa é indispensável para que a implementação reproduza corretamente a definição matemática da dilatação e preserve a dualidade erosão-dilatação.

i Dualidade erosão-dilatação

Erosão e dilatação são **duais pelo complemento**. Isso significa que um operador pode ser completamente obtido a partir do outro, desde que se atue sobre o complemento da imagem utilizando o elemento estruturante refletido $\hat{B}$:

$$(A \ominus B)^c = A^c \oplus \hat{B} \quad \Longleftrightarrow \quad A \ominus B = (A^c \oplus \hat{B})^c$$

De forma análoga, a **dilatação também pode ser obtida a partir da erosão**:

$$(A \oplus B)^c = A^c \ominus \hat{B} \quad \Longleftrightarrow \quad A \oplus B = (A^c \ominus \hat{B})^c$$

Em termos práticos, a erosão de um objeto pode ser obtida pela dilatação de seu complemento, seguida da complementação do resultado (e vice-versa). Na implementação do pacote `morph.py`, as versões didáticas `mm.ero0` e `mm.dil0` tornam explícita essa estrutura por meio de laços (*loops*), enquanto `mm.ero` e `mm.dil` delegam as operações ao OpenCV visando maior eficiência computacional.

i Condições de contorno e imagens finitas

Na morfologia matemática clássica, definida sobre um domínio infinito (tipicamente $\mathbb{Z}^2$), essa dualidade é exata. Em imagens digitais, entretanto, trabalha-se com matrizes finitas, e o resultado passa a depender da forma como são tratados os pixels localizados fora da imagem.

Para que as identidades de dualidade permaneçam válidas, o complemento deve ser definido em relação ao mesmo universo e as condições de contorno adotadas para a erosão e para a dilatação devem ser complementares entre si. Por exemplo, se a erosão assume que os pixels externos pertencem ao objeto (255), então a dilatação aplicada ao complemento deve assumir que esses mesmos pixels externos pertencem ao fundo (0).

Quando diferentes estratégias de preenchimento são utilizadas (replicação, reflexão, valor constante etc.), a dualidade teórica pode deixar de ser satisfeita exatamente nas regiões próximas às bordas da imagem.

Para ilustrar numericamente os operadores morfológicos e a dualidade erosão–dilatação, a Figure 4.4 apresenta uma imagem binária 10×10 processada com um elemento estruturante em formato de “L”. Na implementação de `morph.py`, a origem de B é fixada no centro geométrico da máscara — posição (1,1) para um *kernel* 3×3 — e deve corresponder a um elemento ativo para que a erosão se comporte corretamente (conforme discutido anteriormente). O elemento estruturante B_L definido a seguir satisfaz essa condição. A Figure 4.5 complementa a análise com um simulador interativo da erosão, permitindo visualizar o deslocamento do elemento estruturante sobre a imagem e identificar as posições em que ele permanece completamente contido no objeto.

```
A = np.array([
    [0,0,0,0,0,0,0,0,0,0],
    [0,0,0,1,1,1,0,0,0,0],
    [0,0,1,1,1,1,1,0,0,0],
    [0,1,1,1,1,1,1,1,0,0],
    [0,1,1,1,1,1,1,1,0,0],
    [0,1,1,1,1,1,1,1,0,0],
    [0,1,1,1,1,1,1,0,0,0],
    [0,0,1,1,1,1,1,1,0,0],
    [0,0,0,1,1,1,1,0,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0]], dtype=np.uint8) * 255

B_L = np.array([[1,0,0],
                [1,1,0],
                [1,1,0]], dtype=np.uint8) # origem no centro (1,1), elemento ativo

# Elemento estruturante refletido B
B_hat = np.flip(B_L)
print("Elemento estruturante B_L:")
print(mm.drawImg(B_L))
print("Elemento estruturante refletido B:")
print(mm.drawImg(B_hat))

# Erosão de A por B_L
img_ero0 = mm.ero0(A, B_L)

# Validação da dualidade: (A ∖ B)c = Ac ∖ B
A_c = 255 - A # complemento de A
ero_c = 255 - img_ero0 # complemento da erosão - lado esquerdo
dil_Ac = mm.dil0(A_c, B_hat) # lado direito

dualidade_ok = np.array_equal(ero_c, dil_Ac)
print(f"Dualidade (A ∖ B)c == Ac ∖ B : {dualidade_ok}")
# Transposto (reflexão em ambos os eixos): B usado na dilatação da dualidade
B_hat = np.flip(B_L)
mm.show(
    [A, A_c, img_ero0, ero_c, dil_Ac],
    titles=["A", "A ", "A ∖ B", "(A ∖ B) ", "A ∖ B"],
    cols=5,
```

```
figsize=(15, 3)
)
```

Elemento estruturante B_L:

```
1 0 0
1 1 0
1 1 0
```

Elemento estruturante refletido B:

```
0 1 1
0 1 1
0 0 1
```

Dualidade $(A \ominus B)^c == A^c \ominus B$: True

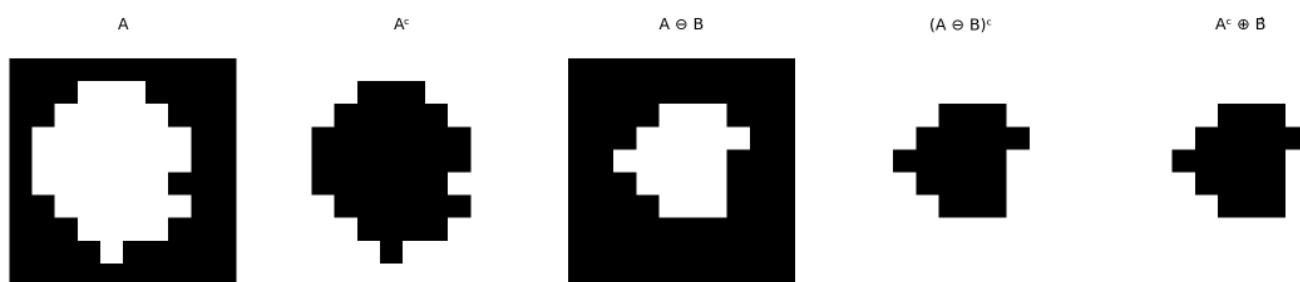


Figura 4.4: Erosão e dilatação em imagem binária 10×10 com elemento estruturante ‘L’ assimétrico 3×3. Validação da dualidade erosão–dilatação.

4.3.2 Abertura e Fechamento

Combinando erosão e dilatação obtêm-se dois operadores de grande utilidade prática: a **abertura** e o **fechamento**, definidos pelas Equações Equation 4.5 e Equation 4.6. Seus principais efeitos são resumidos na Tabela Table 4.2.

Abertura (*opening*) — erosão seguida de dilatação pelo mesmo B :

$$A \circ B = (A \ominus B) \oplus B \quad (4.5)$$

Fechamento (*closing*) — dilatação seguida de erosão pelo mesmo B :

$$A \bullet B = (A \oplus B) \ominus B \quad (4.6)$$

Na prática, o OpenCV aplica o mesmo elemento estruturante nas duas etapas. Para elementos estruturantes simétricos (os mais comuns), essa implementação coincide com a definição matemática apresentada acima.

Tabela 4.2: Propriedades de abertura e fechamento.

Operador	Sequência	Efeito principal
Abertura $A \circ B$	erosão $\rightarrow$ dilatação	Remove estruturas incapazes de conter o elemento estruturante; suaviza contornos externos
Fechamento $A \bullet B$	dilatação $\rightarrow$ erosão	Preenche buracos menores que B ; suaviza contornos internos

Propriedade importante: ambos são **idempotentes**. Por exemplo,

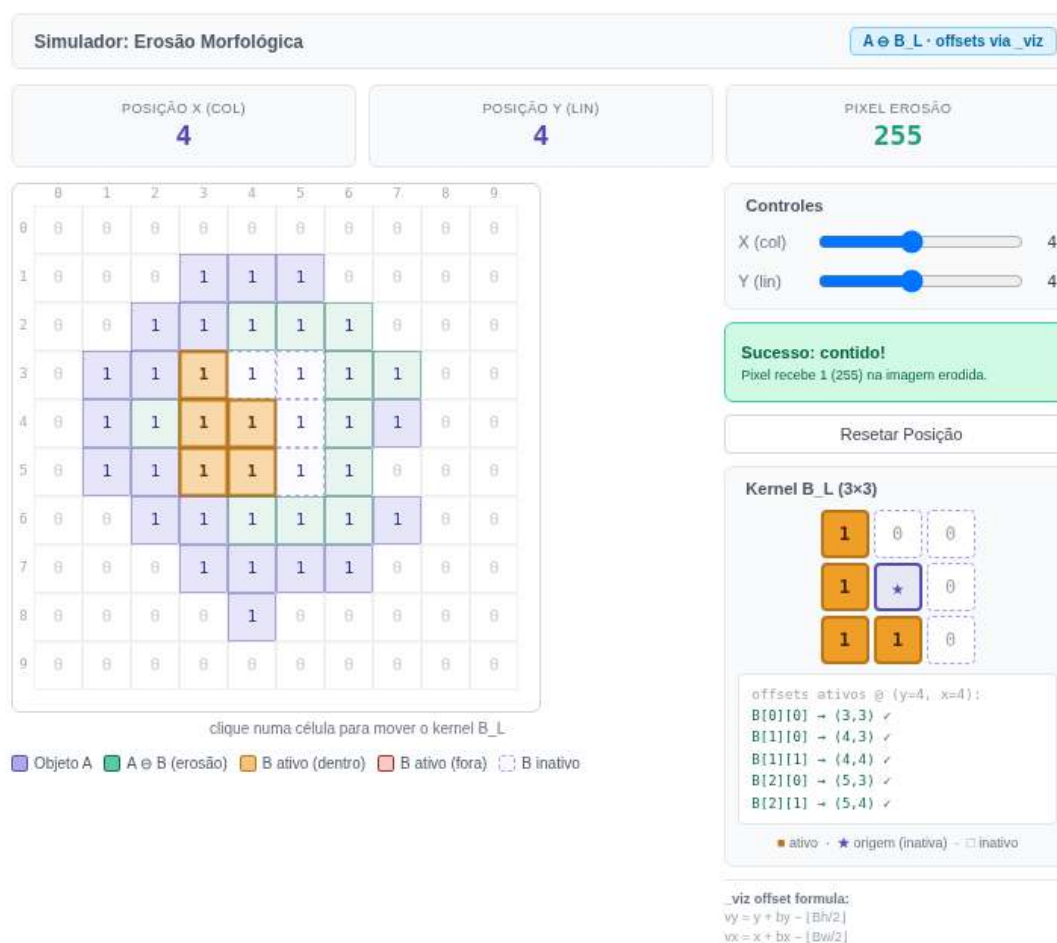


Figura 4.5: Simulador interativo de erosão morfológica: visualização do critério de inclusão do elemento estruturante assimétrico B_L sobre a imagem binária A .

$$(A \circ B) \circ B = A \circ B,$$

ou seja, após a primeira aplicação, novas aplicações do mesmo operador não alteram mais o resultado.

4.3.2.1 Implementação da abertura e do fechamento

Diferentemente da erosão e da dilatação, abertura e fechamento não introduzem novos mecanismos computacionais. Ambos são obtidos pela composição sequencial dos operadores primitivos já apresentados:

```
def open0(f, B):
    return mm.dil0(mm.ero0(f, B), B)

def close0(f, B):
    return mm.ero0(mm.dil0(f, B), B)
```

A função `mm.open` delega a operação para `cv2.morphologyEx(..., MORPH_OPEN, B)`, enquanto `mm.close` utiliza `cv2.morphologyEx(..., MORPH_CLOSE, B)`, produzindo o mesmo resultado de forma mais eficiente.

A abertura herda da erosão a capacidade de remover estruturas menores que o elemento estruturante e da dilatação a restauração parcial das regiões preservadas. O fechamento realiza o processo inverso: primeiro expande os objetos e depois restaura suas dimensões originais, preenchendo lacunas e buracos menores que o elemento estruturante.

4.3.2.2 Filtro Sequencial Alternado

Na prática, abertura e fechamento são frequentemente aplicados em sequência para remover simultaneamente ruído externo e preencher buracos internos. A função `mm.asf` (*Alternating Sequential Filter*) generaliza essa estratégia ao aplicar aberturas e fechamentos alternadamente com elementos estruturantes progressivamente maiores. As sequências disponíveis são apresentadas na Table 4.3.

Tabela 4.3: Sequências do filtro sequencial alternado `mm.asf`.

Sequência	Ordem	Uso típico
'OC'	abertura → fechamento	remove ruído externo antes de preencher pequenos buracos
'CO'	fechamento → abertura	preenche pequenos buracos antes de remover ruído externo
'OCO'	abertura → fechamento → abertura	ênfatisa a remoção de ruído externo
'COC'	fechamento → abertura → fechamento	ênfatisa o preenchimento de buracos e lacunas

O parâmetro `n` controla o número de escalas utilizadas pelo filtro. Em cada iteração i , o elemento estruturante é ampliado por soma de Minkowski (`mm.sesum(b, i)`), produzindo uma sequência de filtros morfológicos cada vez mais abrangentes. Diferentemente de uma única abertura ou fechamento com um elemento estruturante grande, o ASF realiza uma suavização progressiva em múltiplas escalas, preservando melhor a geometria dos objetos relevantes enquanto elimina estruturas menores. A Figure 4.6 apresenta um exemplo de aplicação da abertura, do fechamento e do ASF na imagem das moedas.

```
img_bin    = mm.threshold(img_clahe)
B_disk     = mm.sedisk(19)

img_open   = mm.open(img_bin, B_disk)           # erosão → dilatação
img_close  = mm.close(img_bin, B_disk)          # dilatação → erosão
img_oc     = mm.close(img_open, B_disk)         # abertura seguida de fechamento
img_asf    = mm.asf(img_bin, 'OC', mm.sedisk(3), n=7)
# ASF: disco base 3×3, cresce a cada iteração
```



```
mm.show(
    [img_bin, img_open, img_close, img_oc, img_asf],
    titles=["Binarização Otsu (CLAHE)", "Abertura (A∘B)", "Fechamento (A⋄B)",
           "Abertura→Fechamento", "ASF-OC (n=7)"],
    cols=5, figsize=(15, 12)
)
```

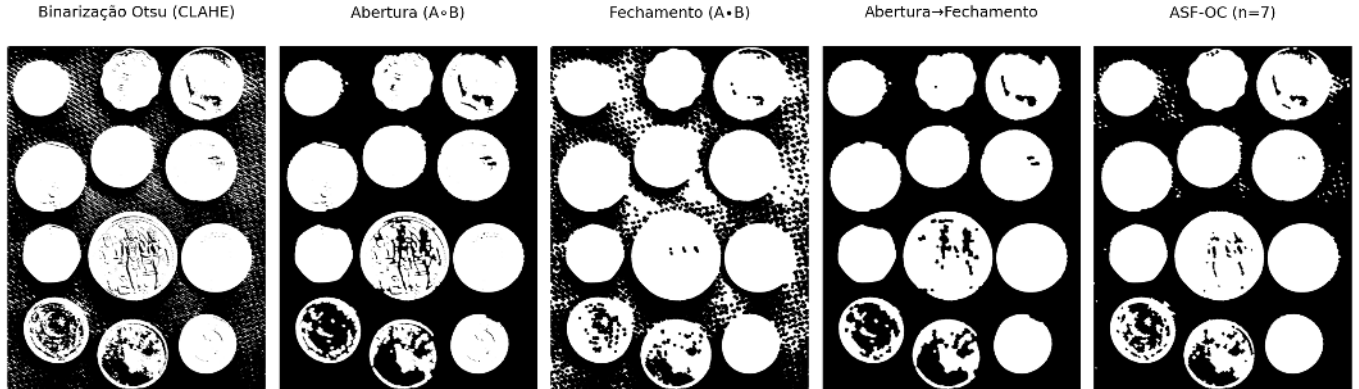


Figura 4.6: Abertura, fechamento, composição e filtro sequencial alternado aplicados à binarização Otsu das moedas com CLAHE. Elemento estruturante: disco 13×13 .

4.3.3 Operadores Geodésicos

Os operadores geodésicos introduzem uma restrição adicional aos operadores morfológicos clássicos por meio de uma imagem de controle denominada **máscara** g . Em vez de permitir que a erosão ou a dilatação se propaguem livremente pela imagem, o resultado de cada iteração é limitado ponto a ponto pelos valores da máscara, restringindo a evolução da operação às regiões permitidas.

4.3.3.1 Dilatação Geodésica

A **dilatação geodésica** de uma imagem marcador f sob uma imagem máscara g , utilizando um elemento estruturante plano b , é definida por:

$$f \oplus_g b = (f \oplus b) \wedge g, \quad (4.7)$$

onde $\wedge$ representa o mínimo ponto a ponto.

Em outras palavras, realiza-se inicialmente uma dilatação convencional sobre o marcador e, em seguida, o resultado é restringido pela máscara g . Dessa forma, a propagação nunca pode ultrapassar as regiões permitidas pela máscara.

A formulação clássica da dilatação geodésica pressupõe que o marcador esteja contido na máscara, isto é, $f \leq g$, garantindo que a evolução da operação permaneça sempre limitada pela máscara.

4.3.3.2 Implementação da dilatação geodésica

A função `mm.cdil` implementa diretamente esse operador e permite executar múltiplas iterações consecutivas:

```
def cdil(f, g, b=np.zeros((3,3),dtype='uint8'), n=1):
    """Dilatação geodésica do marcador f sob a máscara g."""
    y = f.copy()
    for _ in range(n):
        y = np.minimum(cv2.dilate(y, b), g)
    return y
```

A instrução `np.minimum(cv2.dilate(y, b), g)` implementa exatamente a definição matemática da dilatação geodésica, isto é, $(y \oplus b) \wedge g$.

Quando $n = 1$, a função executa uma única dilatação geodésica. Para $n > 1$, o resultado de cada etapa torna-se o marcador da etapa seguinte, produzindo uma propagação progressiva controlada pela máscara.

O simulador interativo Figure 4.7 permite acompanhar, passo a passo, a propagação do marcador f ao longo dos corredores do labirinto. A cada iteração de `mm.cdil`, a frente de dilatação avança para as células vizinhas livres — aquelas em que $g = 1$ —, enquanto as paredes ($g = 0$) permanecem intransponíveis. O número de passos necessários para que o marcador alcance a saída corresponde exatamente ao comprimento geodésico do caminho mais curto dentro da máscara, evidenciando a conexão direta entre a dilatação geodésica iterada e a noção de distância em grafos.

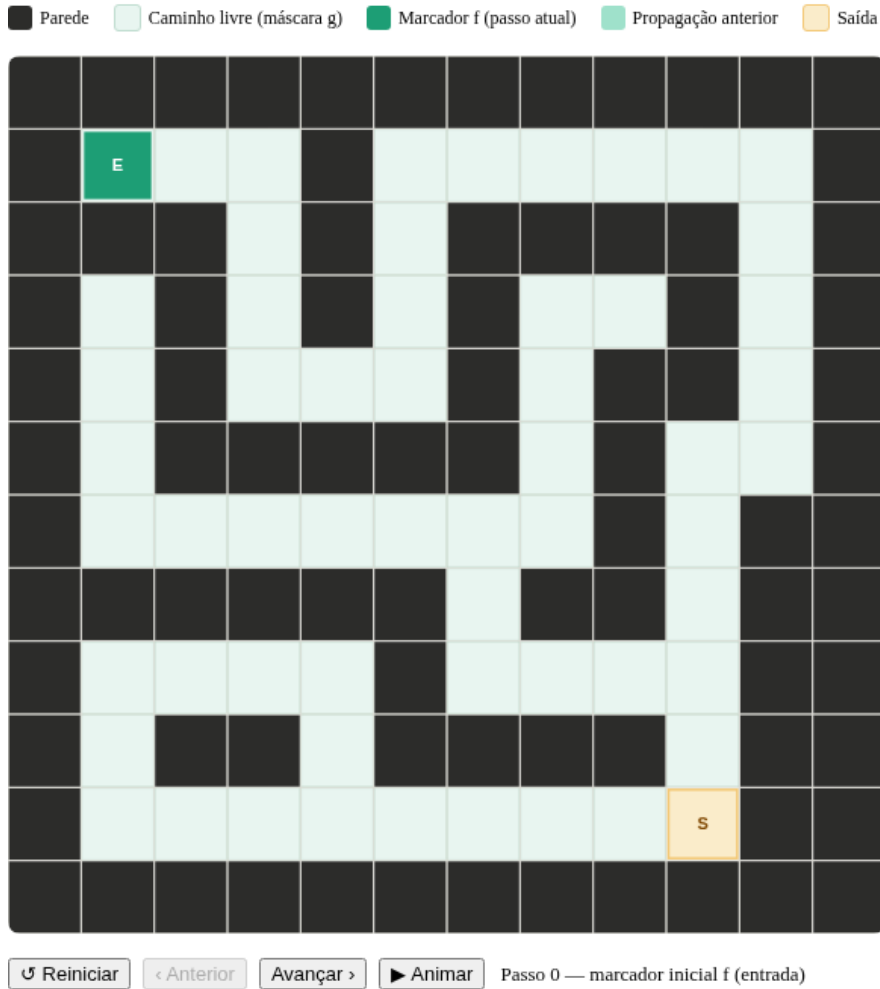


Figura 4.7: Simulador interativo de dilatação geodésica: o marcador f (verde) propaga-se passo a passo pelos caminhos livres da máscara g , sem atravessar paredes — ilustrando como $\delta_g^{(n)}(f)$ encontra a saída do labirinto.

4.3.3.3 Erosão Geodésica

De forma dual, a **erosão geodésica** de uma imagem marcador f sob uma imagem máscara g é definida por:

$$f \ominus_g b = (f \ominus b) \vee g, \quad (4.8)$$

onde $\vee$ representa o operador de máximo ponto a ponto.

Nesse caso, a erosão convencional do marcador é seguida por uma restrição inferior imposta pela máscara. Assim, nenhum pixel do resultado pode assumir valor inferior ao correspondente pixel da máscara.

A formulação clássica da erosão geodésica pressupõe a condição dual

$$f \geq g,$$

de modo que a máscara atue como limite inferior durante todo o processo.

4.3.3.4 Implementação da erosão geodésica

A função estática `mm.cero` materializa esse operador:

```
@staticmethod
def cero(f, g, b=np.zeros((3,3),dtype='uint8'), n=1):
    """Erosão geodésica do marcador f sob a máscara g."""
    y = f.copy()
    for _ in range(n):
        y = np.maximum(cv2.erode(y, b), g)
    return y
```

A instrução `np.maximum(cv2.erode(y, b), g)` implementa diretamente a expressão $(y \ominus b) \vee g$.

Assim como na dilatação geodésica, o parâmetro n define quantas erosões geodésicas sucessivas serão computadas antes de retornar a imagem final.

i Relação com a reconstrução morfológica

A reconstrução morfológica apresentada na próxima seção é obtida pela aplicação iterativa da dilatação geodésica (`mm.cdil`) até que um ponto fixo seja alcançado, isto é, até que nenhuma célula mude de valor entre duas iterações consecutivas. Em outras palavras, a reconstrução consiste em uma sequência de dilatações geodésicas sucessivas que se propagam dentro da máscara até não haver mais alterações. De forma dual, também é possível definir reconstruções baseadas em erosão geodésica por meio de aplicações sucessivas de `mm.cero`.

4.3.3.5 Exemplo: propagação em um labirinto via dualidade

A Figure 4.8 ilustra a resolução do problema de conectividade de um labirinto utilizando a dualidade morfológica por meio das funções `mm.cero` e `mm.suprec`.

Em vez de propagar um marcador pelos corredores livres através de dilatações geodésicas, o problema é formulado no domínio complementar. Inicialmente, a máscara original é invertida,

$$g = 1 - g_{orig},$$

fazendo com que as paredes passem a assumir valor 1 e os corredores valor 0. De forma análoga, o marcador é construído nesse mesmo domínio complementar, contendo um único valor 0 na posição de entrada do labirinto e valor 1 nos demais pixels.

Utilizando o elemento estruturante em cruz (`mm.secross()`), a erosão geodésica atua sobre o marcador complementado. A cada iteração de `mm.cero`, a região conectada ao marcador inicial sofre erosões sucessivas, enquanto a máscara impõe um limite inferior que impede a propagação através das paredes do labirinto.

As imagens intermediárias mostram estados da evolução após diferentes números de iterações ($n=5$, $n=12$ e $n=22$). O resultado final é obtido pela reconstrução geodésica por erosão (`mm.suprec`), que aplica erosões geodésicas sucessivas até atingir um ponto fixo, isto é, uma situação em que nenhuma alteração adicional ocorre entre duas iterações consecutivas.

No domínio complementar, a região reconstruída corresponde exatamente ao conjunto de corredores conectados à entrada do labirinto. Assim, a conectividade entre entrada e saída pode ser determinada diretamente a partir da imagem reconstruída.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

# 1. Máscara original g (0 = Parede, 1 = Corredor)
g_orig = np.array([
    [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 0, 1, 1, 1],
    [0, 0, 0, 0, 0, 1, 0, 1, 0, 1],
    [0, 1, 1, 1, 0, 1, 1, 1, 0, 1],
    [0, 1, 0, 1, 0, 0, 0, 0, 0, 1],
    [0, 1, 0, 1, 1, 1, 1, 1, 1, 1],
    [0, 1, 0, 0, 0, 0, 0, 0, 1, 0],
    [0, 1, 1, 1, 1, 1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0, 0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]], dtype=np.uint8)

# Inversão global no início (Dualidade Morfológica)
g = 1 - g_orig # Agora: 1 = Parede, 0 = Corredor
f = np.ones((10, 10), dtype=np.uint8)
f[0, 1] = 0 # Semente injetada como 0 no corredor

# Elemento estruturante em cruz
B_cruz = mm.secross()

# Processamento direto usando mm.cero e mm.suprec no domínio invertido
passo_5 = mm.cero(f, g, B_cruz, n=5)
passo_12 = mm.cero(f, g, B_cruz, n=12)
passo_22 = mm.cero(f, g, B_cruz, n=22)
ponto_fixo = mm.suprec(f, g, B_cruz)

# --- CONFIGURAÇÃO DA EXIBIÇÃO GRÁFICA ---

titles = [
    "Máscara (~g)", "Marcador (~f)",
    "Avanço (n=5)", "Avanço (n=12)",
    "Avanço (n=22)", "~mm.suprec"
]

images = [g, f, passo_5, passo_12, passo_22, ponto_fixo]

fig, axes = plt.subplots(1, 6, figsize=(16, 4), facecolor='#fcfcfc')

# Mapa de cores adaptado para o domínio complementar:
# No domínio invertido: 1 = Parede (Azul Escuro)
# Onde g == 0 e imagem == 1 = Corredor Livre (Cinza Claro)
# Onde g == 0 e imagem == 0 = Onda Geodésica Ativa (Ouro)
cmap_pipeline = mcolors.ListedColormap(['#ffc13b', '#e0e0e0', '#1e3d59'])

for i, ax in enumerate(axes):
    if i == 0 or i == 1:
        # Para as condições iniciais (~g e ~f)
        cmap_init = mcolors.ListedColormap(['#e0e0e0', '#1e3d59'])
        ax.imshow(images[i], cmap=cmap_init, vmin=0, vmax=1)
    else:
        # Renderização baseada nos estados complementares
        render_step = np.zeros_like(g, dtype=np.uint8)
```

```

render_step[g == 1] = 2      # Parede (1 original do mapa de cores)
render_step[g == 0] = 1      # Corredor padrão
render_step[images[i] == 0] = 0 # Onda ativa (0 original do mapa de cores)
ax.imshow(render_step, cmap=cmap_pipeline, vmin=0, vmax=2)

ax.set_title(titles[i], fontsize=10, fontweight='bold', color='#1e3d59', pad=12)
ax.set_xticks(np.arange(-0.5, 10, 1), minor=True)
ax.set_yticks(np.arange(-0.5, 10, 1), minor=True)
ax.grid(which='minor', color='ffffff', linestyle='--', linewidth=1)
ax.tick_params(which='both', bottom=False, left=False, labelbottom=False, labelleft=False)

# Indicadores gráficos de Entrada e Saída
ax.plot(1, 0, marker='v', color='#2ecc71', markersize=7, markeredgewidth=1.5)
ax.plot(8, 9, marker='o', color='#e74c3c', markersize=6, fillstyle='none', markeredgewidth=2)

plt.tight_layout()
plt.show()

```

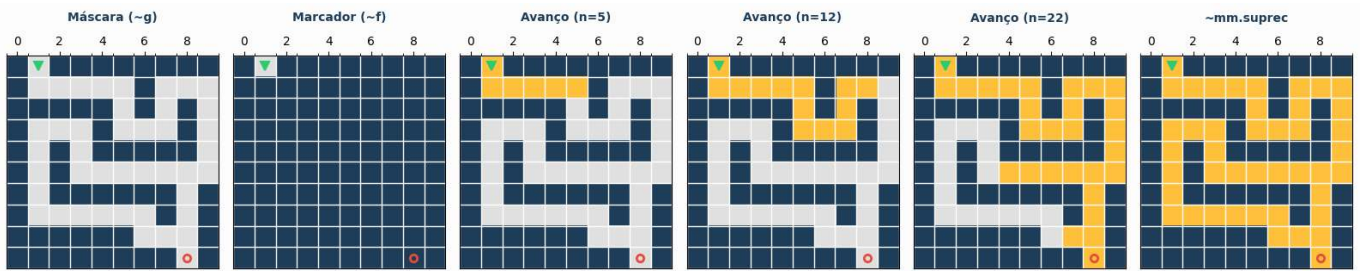


Figura 4.8: Resolução de labirinto via Operações Complementares: Invertendo a máscara e o marcador no início do *pipeline*, o fluxo é resolvido diretamente no domínio complementar através de *mm.cero* e *mm.suprec*, eliminando re-inversões redundantes.

4.3.4 Reconstrução Morfológica

A **reconstrução morfológica** propaga uma imagem **marcador** f dentro de uma imagem **máscara** g , garantindo que o resultado nunca ultrapasse os valores de intensidade impostos pela máscara. O operador fundamental que viabiliza essa propagação contida é a **dilatação geodésica**, definida pela Equation 4.7.

A reconstrução é obtida pela aplicação iterativa dessa dilatação condicionada. Inicialmente, o marcador é limitado pela máscara para estabelecer o estado inicial:

$$X^{(0)} = f \wedge g,$$

e as iterações subsequentes são definidas de forma recursiva por:

$$X^{(k)} = (X^{(k-1)} \oplus b) \wedge g.$$

A sequência cresce monotonicamente até atingir um ponto fixo, produzindo a **reconstrução morfológica por dilatação** (também conhecida na literatura como *inf-reconstrução*):

$$R_g^\delta(f) = \lim_{k \rightarrow \infty} X^{(k)} = X^{(k)} \quad \text{quando} \quad X^{(k)} = X^{(k-1)}. \quad (4.9)$$

A subida iterativa é interrompida assim que a estabilidade é alcançada, isto é, quando duas iterações consecutivas produzem matrizes com valores absolutamente idênticos.

4.3.4.1 Implementação da reconstrução morfológica

A rotina didática `mm.infrec` implementa diretamente o algoritmo iterativo de ponto fixo. Inicialmente, o marcador efetivo inicial $X^{(0)}$ é determinado pela operação `np.minimum(f, g)`. Para garantir que o laço de verificação execute a primeira passada sem disparar falsas convergências prematuras, a variável de controle da iteração anterior (`y1`) é inicializada preenchida com um valor sentinela fora do domínio de dados (ou simplesmente com uma matriz que force a primeira execução).

```
def infrec(f, g, b=np.zeros((3,3), dtype='uint8')):
    """Inf-reconstrução: dilata o marcador (f g) até convergir sob a máscara g."""
    y = np.minimum(f, g)
    # Inicializa y1 com valores impossíveis para forçar a entrada no laço
    y1 = np.full_like(f, 256, dtype=np.int16)
    while not np.array_equal(y, y1):
        y1 = y.copy()
        # Aplica a dilatação geodésica: (y b) g
        y = np.minimum(cv2.dilate(y, b), g)
    return y.astype('uint8')
```

No interior do laço `while`, a variável `y` armazena a estimativa corrente da reconstrução $X^{(k)}$, enquanto `y1` preserva a imagem do estágio imediatamente anterior $X^{(k-1)}$. A instrução de controle condicional `np.minimum(cv2.dilate(y, b), g)` traduz fielmente a dilatação geodésica teórica, onde a expansão morfológica convencional comandada pelo OpenCV é imediatamente “podada” e limitada pelas barreiras de intensidade da máscara `g`. O laço cessa quando nenhuma modificação de pixel é registrada entre os passos.

4.3.4.2 Vantagens da reconstrução morfológica

A reconstrução morfológica é significativamente mais robusta que a abertura convencional porque é capaz de remover estruturas indesejadas sem distorcer ou alterar a morfologia dos objetos que devem ser preservados.

Enquanto a abertura clássica suaviza cantos, elimina pontas e deforma contornos devido à imposição geométrica rígida do elemento estruturante, a reconstrução geodésica utiliza a máscara para recuperar com exatidão os limites e formatos originais dos objetos que possuem conectividade com o marcador original.

Em termos intuitivos, o marcador atua como uma semente de contágio que se expande progressivamente, mas apenas trafegando pelas regiões permitidas pela máscara. Componentes que não possuem nenhuma intersecção com o marcador jamais serão reconstruídas (sendo eliminadas), enquanto as componentes tocadas pela semente expandem-se até restaurar integralmente a sua geometria original.

Esse comportamento discriminatório e conservativo é ilustrado na Figure 4.9, utilizando o elemento estruturante em cruz apresentado na Figure 4.3.

```
# 1. Definição da Máscara (g): Imagem 10x10 com dois objetos isolados
g = np.array([
    [0,0,0,0,0,0,0,0,0,0],
    [0,1,1,1,0,0,0,1,1,0],
    [0,1,1,1,1,0,0,1,1,0],
    [0,1,1,1,1,0,0,0,0,0],
    [0,1,1,1,1,0,0,0,0,0],
    [0,1,1,1,0,0,0,0,0,0],
    [0,0,1,1,1,0,0,1,0,0],
    [0,0,1,1,1,0,0,1,1,0],
    [0,0,0,1,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0]], dtype=np.uint8) * 255

B_cruz = mm.secross()

# 2. Geração do Marcador (f)
f = mm.ero(g, mm.sebox())
```



```
# 3. Iterações da Dilatação Condicionada automatizadas em laço
iteracoes = []
titulos_iteracoes = []
img_atual = f.copy()

for i in range(1, 6):
    img_add = img_atual*80
    img_atual = mm.cdil(img_atual, g, B_cruz)
    iteracoes.append(img_add+img_atual)
    titulos_iteracoes.append(f"Dilatação Cond. (n={i})")

# Reconstrução Geodésica Final via biblioteca (ponto de controle)
img_reconstruida = mm.infrec(f, g, B_cruz)

# Verificação de convergência (o passo 5 deve ser idêntico à reconstrução final)
convergencia_ok = np.array_equal(img_reconstruida, iteracoes[-1])
print(f" Reconstrução alcançou estabilidade na iteração 5: {convergencia_ok}")

# 4. Exibição do pipeline evolutivo (Montagem dinâmica das listas)
mm.show(
    [g, f] + iteracoes + [img_reconstruida],
    titles=["Máscara (g)", "Marcador (f)"] + titulos_iteracoes + ["Reconstrução Final R_g(f)"],
    cols=8,
    figsize=(18, 3)
)
```

Reconstrução alcançou estabilidade na iteração 5: False

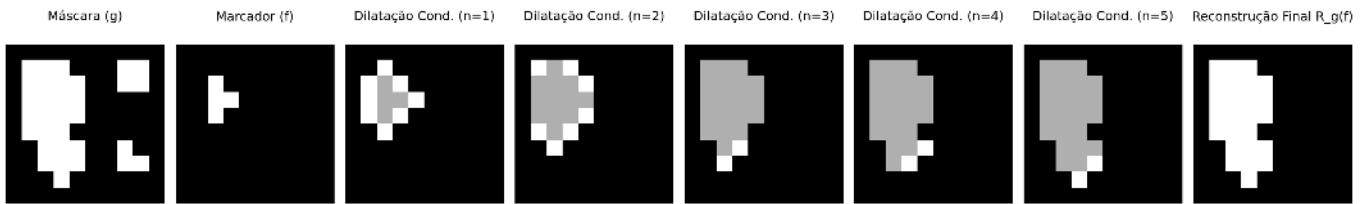


Figura 4.9: *Pipeline* de Reconstrução Morfológica por Dilatação Condicionada: a máscara contém dois objetos, o marcador isola apenas o núcleo do objeto principal, e as iterações subsequentes em laço reconstroem sua forma exata até a convergência.

4.3.5 Preenchimento de Buracos e Remoção de Bordas

Dois operadores baseados em reconstrução morfológica completam o *pipeline* de limpeza binária. Suas características principais são resumidas na Table 4.4.

Preenchimento de buracos (`mm.clohole`) remove cavidades completamente cercadas pelo objeto, independentemente do tamanho, sem alterar os contornos externos. O procedimento atua no complemento da imagem, utilizando como marcador uma restrição da moldura (*frame*) ao fundo:

$$\text{clohole}(f) = (R_{f^c}^{\delta}(\text{frame}(f) \wedge f^c))^c \quad (4.10)$$

Em termos operacionais, primeiro reconstrói-se o fundo externo e, em seguida, aplica-se a complementação para recuperar os objetos com buracos preenchidos.

Remoção de objetos de borda (`mm.edgeoff`) elimina todos os objetos que tocam a borda da imagem, preservando apenas os componentes totalmente internos. O marcador é obtido pela interseção entre a moldura (*frame*) e os objetos da imagem:

$$\text{edgeoff}(f) = f \setminus R_f^\delta(\text{frame}(f) \wedge f) \quad (4.11)$$

Tabela 4.4: Comparação entre os operadores *clohole* e *edgeoff*.

Operador	Marcador	Máscara	Efeito
<code>mm.clohole</code>	<i>frame</i> restrito ao fundo (f^c)	f^c	Preenche buracos internos
<code>mm.edgeoff</code>	<i>frame</i> restrito ao objeto (f)	f	Remove objetos conectados à borda

A evolução passo a passo dessas transformações geodésicas pode ser acompanhada nas figuras a seguir. A Figure 4.10 ilustra o mecanismo de inundação controlada do operador `mm.clohole`, no qual a reconstrução ocorre a partir do fundo externo e impede a propagação para regiões internas não conectadas ao exterior, resultando no preenchimento consistente das cavidades internas. Em contrapartida, a Figure 4.11 detalha a dinâmica do operador `mm.edgeoff`, em que apenas os componentes conectados à borda são reconstruídos e posteriormente removidos, preservando exclusivamente os objetos totalmente contidos no interior da imagem.

A conectividade da propagação geodésica é controlada pelo elemento estruturante: `mm.sebox()` (vizinhança de 8) inclui conexões diagonais, enquanto `mm.secross()` (vizinhança de 4) as exclui. Consequentemente, a escolha do elemento estruturante afeta quais componentes são alcançados pela reconstrução e, portanto, quais serão preservados ou removidos.

4.3.5.1 Conformidade com a implementação

As definições acima estão diretamente alinhadas com a implementação em `morph.py`, reproduzida a seguir:

```
@staticmethod
def clohole(f, b=np.ones((3,3),dtype='uint8')):
    # marcador restrito ao fundo da imagem
    marcador = mm.frame(f, border=1) & mm.neg(f)
    return mm.neg(mm.infrec(marcador, mm.neg(f), b))

@staticmethod
def edgeoff(f, b=np.ones((3,3),dtype='uint8')):
    # marcador restrito aos objetos da imagem
    marcador = mm.frame(f, border=1) & f
    return mm.subm(f, mm.infrec(marcador, f, b))
```

Essas implementações deixam explícito que ambos os operadores são instâncias diretas de reconstrução morfológica por dilatação geodésica com `mm.infrec`, diferindo apenas na escolha do marcador e da máscara: `clohole` atua no complemento da imagem, enquanto `edgeoff` atua diretamente no domínio dos objetos.

```
# Função auxiliar unificada para gerar o pipeline iterativo com realce visual
def gerar_pipeline_reconstrucao(marcador, mascara, kernel, passos=4, fator=80):
    iteracoes, titulos = [], []
    img_atual = marcador.copy()
    for i in range(1, passos + 1):
        img_visual = img_atual * fator
        img_atual = mm.cdil(img_atual, mascara, kernel)
        iteracoes.append(img_visual + img_atual)
        titulos.append(f"Iter. (n={i})")
    return iteracoes, titulos

# Imagem binária 10x10 com 3 cenários: objeto com buraco, objeto isolado e objeto na borda
f = np.array([
    [0,0,0,0,0,0,0,0,0,0],
```

```

[0,1,1,1,1,0,0,0,1],
[0,1,0,0,1,0,0,1,0,1],
[0,1,0,0,1,0,0,1,0,1],
[0,1,1,1,1,0,0,1,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,1,1,1,0,1,1,1,0],
[0,0,1,1,1,0,1,0,1,0],
[0,0,1,1,1,0,1,1,1,0],
[0,0,0,0,0,0,0,0,0,1]], dtype=np.uint8) * 255

B_cruz = mm.secross()
f_c = mm.neg(f)          # Utiliza o operador de negação nativo da mm

# PIPELINE CLOHOLE
# 0 marcador do clohole teórico é a borda externa
marcador_ch = mm.frame(f, border=1)
iters_ch, tits_ch = gerar_pipeline_reconstrucao(marcador_ch, f_c, B_cruz, passos=5)

# Resultado final calculado por extenso e validado com a função nativa mm.clohole
img_clohole = mm.neg(mm.infrec(marcador_ch, f_c, B_cruz))
print(f" Validação clohole: {np.array_equal(img_clohole, mm.clohole(f))}")

mm.show(
    [f, marcador_ch] + iters_ch + [img_clohole],
    titles=["f original", "Marcador (Borda)"] + tits_ch + ["clohole(f)"],
    cols=8, figsize=(18, 3), axis=True
)

```

Validação clohole: True

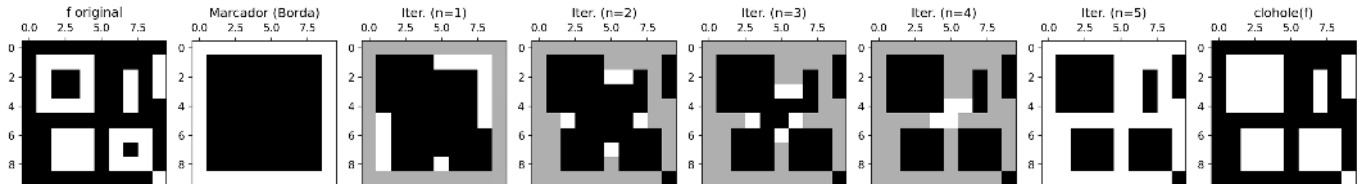


Figura 4.10: *Pipeline* de preenchimento de buracos (*clohole*): o marcador é obtido a partir da borda da imagem e restringido ao complemento f^c . As iterações de dilatação geodésica reconstruem o fundo externo; após a complementação final, os buracos internos ficam preenchidos.

```

# PIPELINE EDGEOFF
B_box = mm.sebox()
marcador_eo = marcador_ch & f
iters_eo, tits_eo = gerar_pipeline_reconstrucao(marcador_eo, f, B_box, passos=5)

# Resultado final por definição e validação nativa
img_edgoff = mm.edgoff(f, B_box)
print(f" Validação edgoff: {np.array_equal(img_edgoff, mm.edgoff(f))}")

mm.show(
    [f, marcador_eo] + iters_eo + [img_edgoff],
    titles=["f original", "Marcador (Borda)"] + tits_eo + ["edgoff(f)"],
    cols=8, figsize=(18, 3)
)

```

Validação edgoff: True

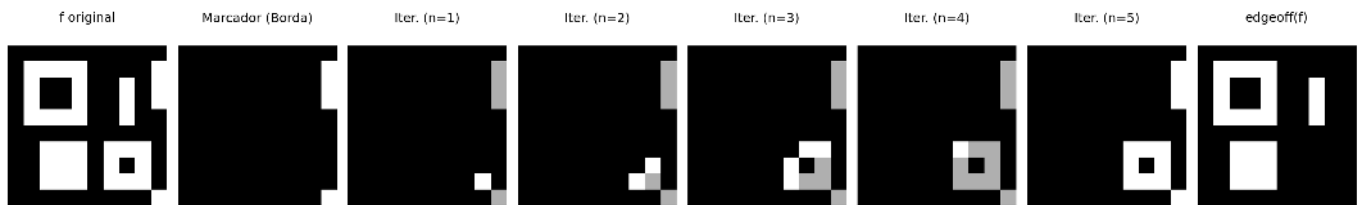
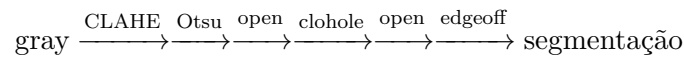


Figura 4.11: *Pipeline* de eliminação de estruturas de borda (*edgeoff*): o marcador captura as raízes conectadas às extremidades da matriz, a reconstrução delimita a extensão desses elementos e a subtração booleana preserva unicamente os objetos totalmente internos.

4.3.6 Pipeline de Limpeza Binária com CLAHE

Com base na análise anterior — na qual o CLAHE produziu o maior valor da variância interclasses ($\sigma_B^2 \approx 2,47 \times 10^3$), ver Figure 4.2, e o operador `mm.clohole` mostrou-se eficaz no preenchimento das cavidades internas —, o *pipeline* final de segmentação, ilustrado na Figure 4.12, é estruturado pelo seguinte fluxo computacional:



Após a etapa de `mm.clohole`, aplica-se uma segunda abertura morfológica com um elemento estruturante maior (`mm.sedisk(33)`, disco de diâmetro 33). Essa operação remove pequenas regiões residuais e artefatos que possam ter permanecido após a segmentação. Em particular, o preenchimento geodésico pode transformar pequenas cavidades isoladas em componentes conectados ao objeto, tornando conveniente uma etapa adicional de filtragem baseada em tamanho. O diâmetro foi escolhido de modo que as moedas permaneçam capazes de conter o elemento estruturante, enquanto componentes significativamente menores sejam eliminados.

Nesta imagem, nenhuma moeda está conectada à borda da matriz. Consequentemente, a aplicação de `mm.edgeoff` não altera o resultado obtido após a segunda abertura. Ainda assim, essa etapa é mantida no *pipeline* por robustez, pois em outras imagens podem existir objetos parcialmente visíveis ou conectados às bordas, que devem ser removidos antes da etapa de análise.

💡 Por que a abertura após o clohole?

O operador `mm.clohole` preenche todas as cavidades fechadas presentes nos objetos segmentados. Em algumas situações, pequenas regiões indesejadas podem permanecer após essa etapa ou tornar-se conectadas aos objetos principais. A abertura morfológica subsequente remove componentes menores que o elemento estruturante, preservando as moedas devido ao seu tamanho significativamente maior.

```
# Pré-processamento: apenas CLAHE
img_clahe0 = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(img_coins_gray)

# Etapa 1: Binarização Otsu
img_bin = mm.threshold(img_clahe0)

# Etapa 2: Abertura - remove ruídos brancos de fundo
img_open = mm.open(img_bin, mm.sedisk(9))

# Etapa 3: clohole - fecha todos os buracos internos
img_hole = mm.clohole(img_open)

# Etapa 4: Abertura (kernel grande) - remove artefatos do clohole
img_limpo = mm.open(img_hole, mm.sedisk(33))

# Etapa 5: edgeoff - remove objetos que tocam a borda
img_final = mm.edgeoff(img_limpo, border=1)
```

```
mm.show(
    [img_clahe0, img_bin,          img_open,
     img_hole,  img_limpo,        img_final],
    titles=["CLAHE",              "Otsu",          "Abertura (r=9)",
           "clohole",            "Abertura (r=33)", "Final (edgeoff)",
           cols=6, figsize=(18, 6)
)
```

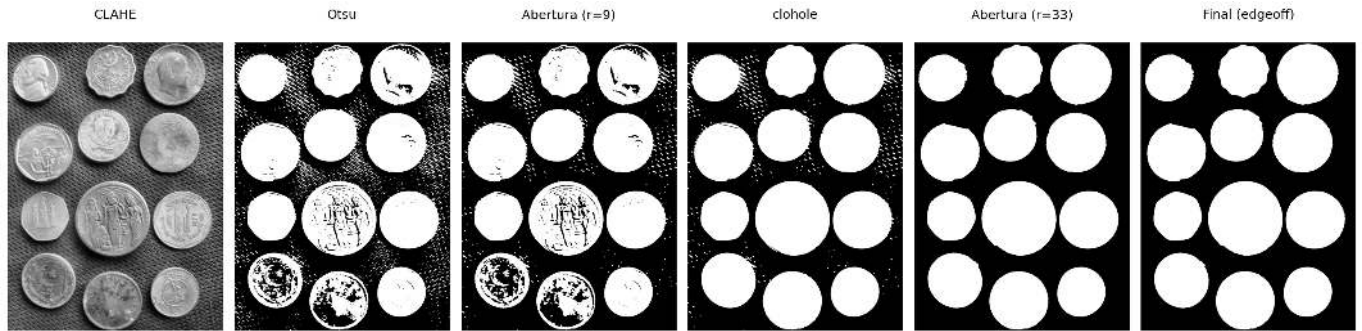


Figura 4.12: *Pipeline* completo de segmentação com CLAHE: Otsu $\rightarrow$ abertura ($r=9$) $\rightarrow$ clohole $\rightarrow$ abertura ($r=33$) $\rightarrow$ edgeoff.

4.3.7 Morfologia em Tons de Cinza

Os operadores morfológicos estendem-se naturalmente para imagens em tons de cinza. Nessa formulação, a erosão e a dilatação passam a atuar diretamente sobre os níveis de intensidade da imagem. Para elementos estruturantes planos ($b \equiv 0$), a erosão corresponde ao **mínimo local** e a dilatação ao **máximo local** dentro da vizinhança definida pelo elemento estruturante.

A interpretação intuitiva é simples: a erosão **escorece** regiões ao substituir cada pixel pelo menor valor presente em sua vizinhança, enquanto a dilatação **clareia** regiões ao utilizar o maior valor disponível. A combinação desses operadores permite construir transformações capazes de realçar bordas, remover tendências de iluminação e destacar estruturas locais.

Três operadores derivados são especialmente úteis:

Gradiente morfológico — destaca bordas como a diferença entre dilatação e erosão:

$$\text{grad}_B(f) = (f \oplus B) - (f \ominus B) \quad (4.12)$$

Top-hat — destaca estruturas brilhantes menores que o elemento estruturante (diferença entre a imagem original e sua abertura):

$$\text{top-hat}_B(f) = f - (f \circ B) \quad (4.13)$$

Black-hat — destaca estruturas escuras menores que o elemento estruturante (diferença entre o fechamento e a imagem original):

$$\text{black-hat}_B(f) = (f \bullet B) - f \quad (4.14)$$

O *Top-hat* extrai detalhes brilhantes que não sobrevivem à abertura, enquanto o *Black-hat* evidencia detalhes escuros removidos pelo fechamento. Já o gradiente morfológico realça transições abruptas de intensidade, produzindo uma representação semelhante à de um detector de bordas.

Para compreender o mecanismo desses operadores em nível local, a Figure 4.13 apresenta um simulador interativo de morfologia em tons de cinza. O simulador permite editar livremente o elemento estruturante,

visualizar seu deslocamento sobre a imagem e acompanhar simultaneamente o perfil unidimensional das intensidades. Dessa forma, torna-se possível observar diretamente como a erosão seleciona mínimos locais, como a dilatação seleciona máximos locais e como o gradiente morfológico emerge da diferença entre esses dois operadores.

O botão localizado no canto superior direito permite alternar entre a visualização original em tons de cinza e uma representação pseudocolorida (*colormap*) apenas nos três tipos gradientes. A versão colorida facilita a percepção visual das variações de intensidade, tornando mais evidente a ação dos operadores morfológicos sobre máximos, mínimos e transições locais da imagem.

Os operadores apresentados estão disponíveis em `morph.py` por meio das funções `mm.gradm`, `mm.tophat` e `mm.blackhat`.

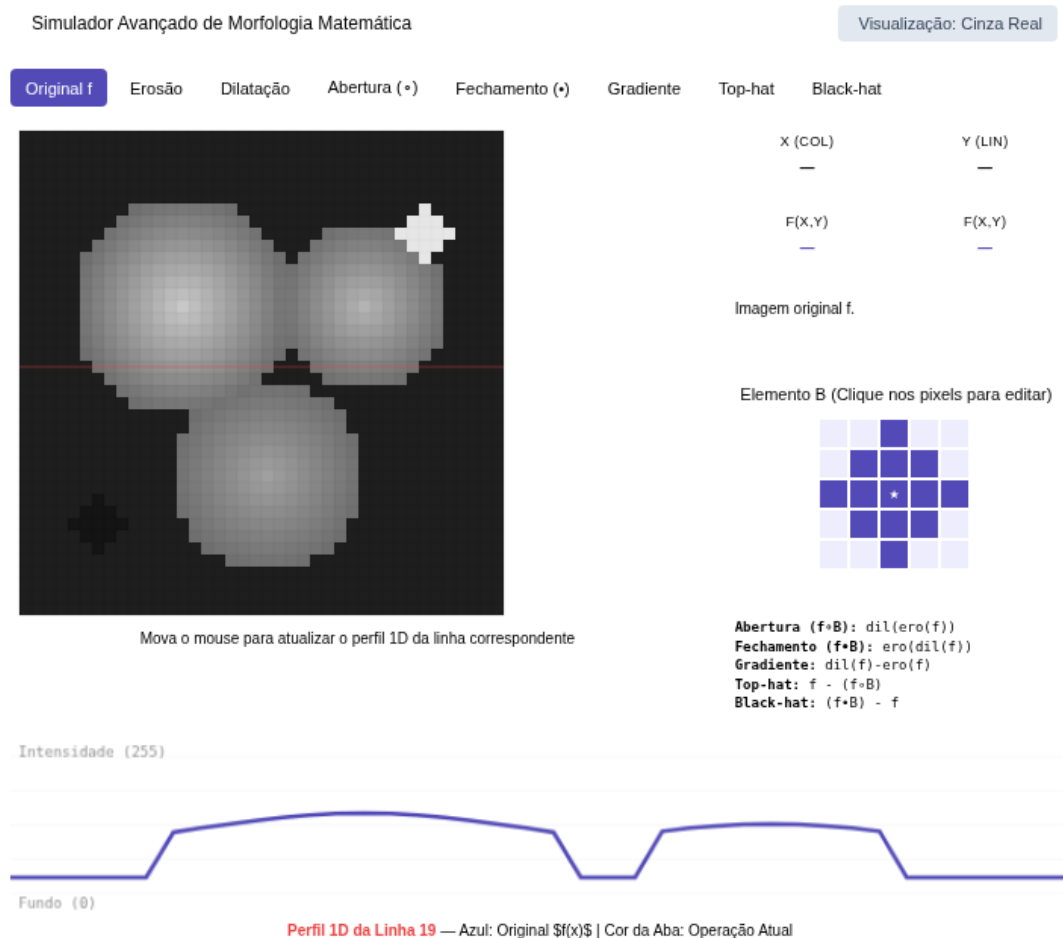


Figura 4.13: Simulador interativo avançado de morfologia com elemento estruturante editável e perfil 1D.

A Figure 4.14 ilustra os efeitos desses operadores sobre a imagem das moedas e seus respectivos histogramas. Observe que a erosão desloca a distribuição para intensidades mais baixas, enquanto a dilatação a desloca para intensidades mais altas. O gradiente concentra valores nas regiões de contorno, e os operadores *Top-hat* e *Black-hat* produzem histogramas fortemente concentrados em baixos níveis de intensidade, pois apenas pequenas estruturas locais são realçadas.

```
import io
import matplotlib.pyplot as plt
import numpy as np

def fig2img(fig):
    b = io.BytesIO(); fig.savefig(b, format='png', dpi=100); plt.close(fig); b.seek(0)
    return (plt.imread(b)[: , : , :3] * 255).astype(np.uint8)
```



```
def plot_hist(img, title):
    fig, ax = plt.subplots(figsize=(4, 3))
    h = mm.hist(img)
    # CORREÇÃO: range usa len(h) dinamicamente para casar com o retorno da biblioteca
    ax.bar(range(len(h)), h, color='steelblue', width=1, edgecolor='steelblue')
    ax.set(xlim=(0, 255)); plt.tight_layout()
    return fig2img(fig)

# 1. Processamento morfológico base
B = mm.sedisk(19)
operadores = [
    ("Original", img_coins_gray),
    ("Erosão", mm.ero(img_coins_gray, B)),
    ("Dilatação", mm.dil(img_coins_gray, B)),
    ("Gradiente Morf.", mm.gradm(img_coins_gray, B)),
    ("Top-hat", mm.tophat(img_coins_gray, B)),
    ("Black-hat", mm.blackhat(img_coins_gray, B))
]

# 2. Montagem dinâmica do par: [Imagem, Histograma]
imgs, titles = [], []
for nome, img in operadores:
    imgs += [img, plot_hist(img, f"Hist. {nome}")]
    titles += [nome, f"Hist. {nome}"]

# 3. Exibição final em grade de duas colunas (Imagem | Histograma)
mm.show(imgs, titles=titles, cols=4, figsize=(10, 8))
```

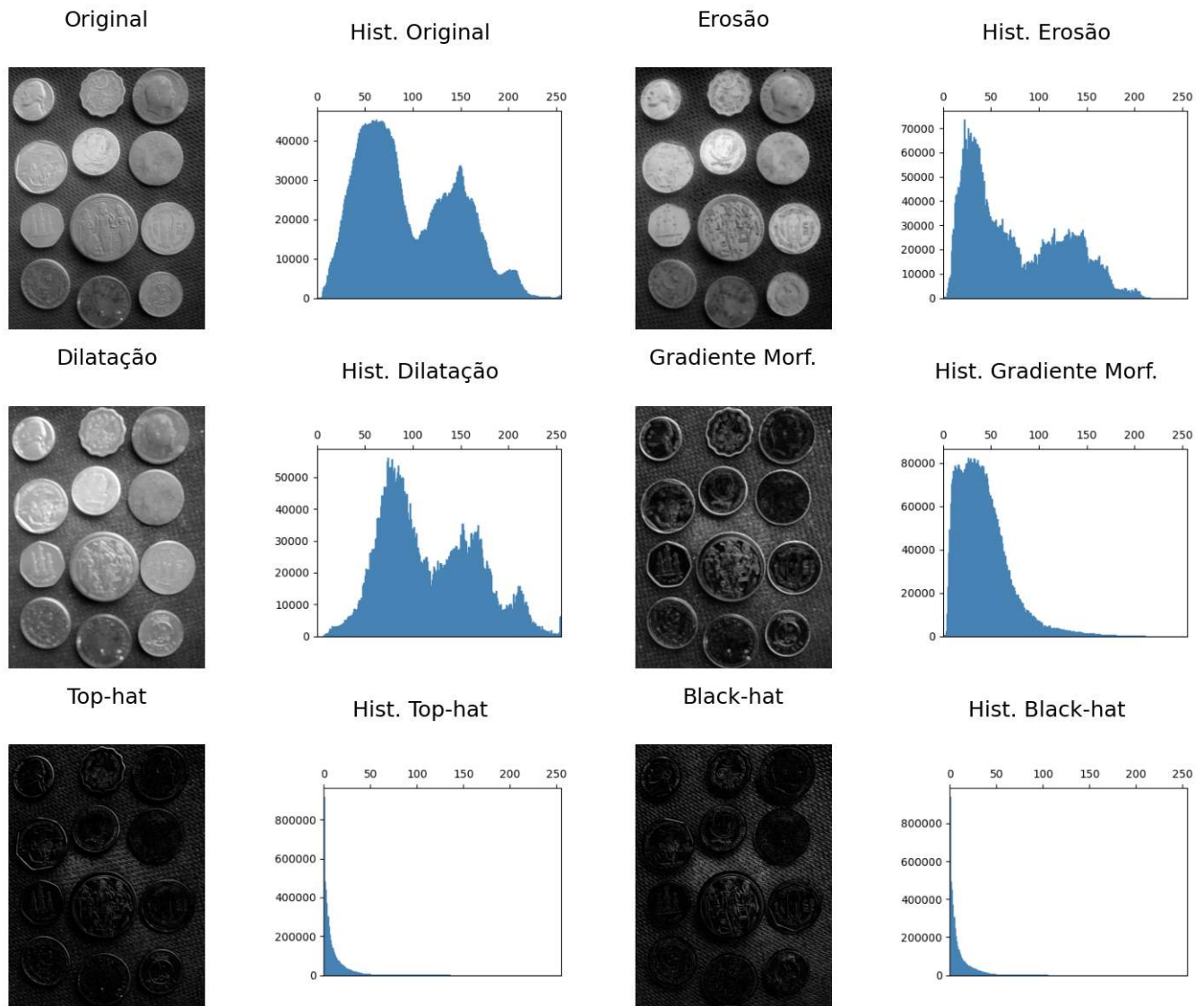


Figura 4.14: Morfologia em tons de cinza e seus respectivos histogramas. A erosão reduz intensidades locais, a dilatação as amplia, o gradiente destaca bordas, e os operadores Top-hat e Black-hat evidenciam detalhes locais brilhantes e escuros. Elemento estruturante: disco de diâmetro 19.

Os operadores morfológicos apresentados anteriormente serão agora utilizados como ferramentas de refinamento e geração de marcadores para métodos de segmentação mais avançados, apresentados a seguir.

4.4 Segmentação de Imagens: Fundamentação e Taxonomia

A **segmentação de imagens** consiste em dividir a imagem em regiões associadas a objetos ou estruturas de interesse. Em PDI, ela representa a transição entre o processamento de baixo nível — como filtragem e realce — e etapas de análise mais avançadas, como extração de características, reconhecimento e interpretação da cena.

Formalmente, o objetivo da segmentação consiste em decompor o domínio espacial completo de uma imagem, denotado por Ω , em uma partição de subconjuntos $\{R_1, R_2, \dots, R_n\}$ que satisfaça simultaneamente os critérios de **completeza** e **disjunção**:

$$\bigcup_{i=1}^n R_i = \Omega, \quad R_i \cap R_j = \emptyset \quad \forall i \neq j \quad (4.15)$$

Além das propriedades de completeza e disjunção expressas na Equation 4.15, cada sub-região R_i deve constituir um domínio **homogêneo** segundo um predicado de similaridade definido sobre propriedades locais — intensidade, cor ou textura — e, simultaneamente, ser **distinta** das regiões adjacentes.

As técnicas de segmentação podem ser organizadas em diferentes famílias. Neste capítulo serão enfatizadas as abordagens resumidas na Table 4.5, fundamentadas principalmente em critérios de intensidade, conectividade e proximidade espacial.

Tabela 4.5: Taxonomia simplificada das principais abordagens de segmentação e refinamento estudadas neste capítulo.

Abordagem	Critério de Segmentação	Operadores de Referência
Limiarização	Particionamento do espaço de intensidades	Critério de Otsu, limiarização global e local
Morfologia Matemática	Relações espaciais definidas por elementos/funções estruturantes	Erosão, dilatação, abertura, fechamento e reconstrução
Baseada em Regiões	Homogeneidade local e conectividade espacial	Rotulação de componentes conexos, Transformada de Distância e <i>Watershed</i>

Até este ponto, o desenvolvimento prático concentrou-se na **limiarização**, por meio da combinação entre equalização adaptativa CLAHE e o método global de Otsu. Essa etapa foi complementada por operadores de **reconstrução morfológica** baseados em dilatações geodésicas, implementados pelas funções `mm.infrec`, `mm.clohole` e `mm.edgeoff`, produzindo uma máscara binária limpa e adequada para análise.

Contudo, em cenários onde objetos distintos aparecem conectados na máscara binária — seja por contato físico, sobreposição parcial ou por pontes estreitas de pixels produzidas pela segmentação —, a limiarização deixa de ser suficiente para individualizar cada objeto. Nesses casos, múltiplos objetos passam a compor um único componente conexo, dificultando etapas posteriores de medição e interpretação.

Para superar essa limitação, as próximas seções introduzem três ferramentas complementares: a **Rotulação de Componentes Conexos**, a **Transformada de Distância** e o algoritmo de segmentação por **Watershed** baseado em marcadores. Em conjunto, essas técnicas permitem separar objetos adjacentes, identificar regiões individualmente e extrair descritores geométricos consistentes para análise quantitativa.

4.4.1 Rotulação

A **rotulação de componentes conexas** (*connected component labeling*) é o operador que atribui um identificador inteiro único a cada conjunto de pixels pertencentes à mesma componente conexa em uma imagem binária.

i Definição formal

Dada uma imagem binária f e uma relação de conectividade definida por um elemento estruturante B (tipicamente conectividade-4 ou conectividade-8), o algoritmo de rotulação da Figure 4.15 produz uma imagem g na qual todos os pixels pertencentes à mesma componente conexa recebem o mesmo rótulo inteiro positivo, enquanto pixels pertencentes a componentes distintas recebem rótulos diferentes.

A conectividade define quais pixels são considerados vizinhos diretos de um pixel (x, y) . As definições mais utilizadas são:

- **Conectividade-4:** considera apenas os quatro vizinhos ortogonais (norte, sul, leste e oeste).
- **Conectividade-8:** considera os quatro vizinhos ortogonais e os quatro diagonais, totalizando oito vizinhos.

A escolha da conectividade influencia diretamente a formação das componentes conexas e, consequentemente, o resultado da rotulação, como ilustrado na Figure 4.17. Um exemplo adicional pode ser explorado interativamente no simulador apresentado na Figure 4.16.

A implementação em `morph.py` disponibiliza duas versões desse operador. A função `mm.label0` reproduz explicitamente o algoritmo de *flood-fill* utilizando uma pilha e permite controlar a conectividade por meio do elemento estruturante adotado. Já `mm.label` delega a operação à implementação otimizada do OpenCV (`cv2.connectedComponents`). Em ambos os casos, o resultado é uma imagem rotulada na qual cada componente conexa recebe um identificador inteiro distinto.

Figura 4.15: Algoritmo de rotulação por *flood-fill* com pilha: passo a passo, cards, linha do tempo, código Python e fluxograma.

O auxiliar `_viz` itera sobre a janela estruturante `b` centrada em (i, j) , gerando somente os vizinhos válidos dentro dos limites da imagem — a conectividade desejada é inteiramente determinada pela forma de `b` passada ao algoritmo.

Exemplo didático — efeito da conectividade:



Figura 4.16: Simulador interativo de rotulação de componentes conexas (*connected component labeling*): visualização da expansão flood-fill, conectividade-4 e conectividade-8.

```
# Imagem binária 10x10 com componentes diagonalmente adjacentes
f = np.array([
    [0,0,0,0,0,0,0,0,0,0],
    [0,1,0,0,0,0,0,0,0,0],
    [0,0,1,0,0,0,0,0,0,0], # diagonal com linha anterior
    [0,0,0,1,0,0,1,1,0,0],
    [0,0,0,0,0,0,1,1,0,0],
    [0,0,0,0,0,0,0,0,0,0],
    [0,1,1,1,0,0,0,0,0,0],
    [0,1,0,1,0,0,0,1,0,0],
    [0,1,1,1,0,0,0,0,1,0], # diagonal com linha anterior
    [0,0,0,0,0,0,0,0,0,0]], dtype=np.uint8) * 255
```

```

# Elemento estruturante cruz (conectividade-4) e quadrado (conectividade-8)
B4 = mm.secross() # conectividade-4
B8 = mm.sebox()   # conectividade-8

# Rotulação com label0 (didático) e label (cv2)
lbl4_didatico = mm.label0(f, B4)
lbl8_didatico = mm.label0(f, B8)

_, lbl4_cv2 = cv2.connectedComponents(f, connectivity=4)
_, lbl8_cv2 = cv2.connectedComponents(f, connectivity=8)

# Validação cruzada
print(f" label0 (C4) == cv2 (C4): {np.array_equal(lbl4_didatico, lbl4_cv2)}")
print(f" label0 (C8) == cv2 (C8): {np.array_equal(lbl8_didatico, lbl8_cv2)}")
print(f" Componentes C4: {lbl4_cv2.max()} | Componentes C8: {lbl8_cv2.max()}")

# Normalização para visualização
def norm_label(lbl):
    out = np.zeros_like(lbl, dtype=np.uint8)
    for i, v in enumerate(np.unique(lbl)[1:], 1):
        out[lbl == v] = int(i * 255 / lbl.max())
    return out

mm.show(
    [f, norm_label(lbl4_cv2), norm_label(lbl8_cv2)],
    titles=[
        "f original",
        f"Rotulação C4\n({lbl4_cv2.max()} componentes)",
        f"Rotulação C8\n({lbl8_cv2.max()} componentes)"
    ],
    cols=3, figsize=(12, 4), axis=True
)

```

```

label0 (C4) == cv2 (C4): True
label0 (C8) == cv2 (C8): True
Componentes C4: 7 | Componentes C8: 4

```

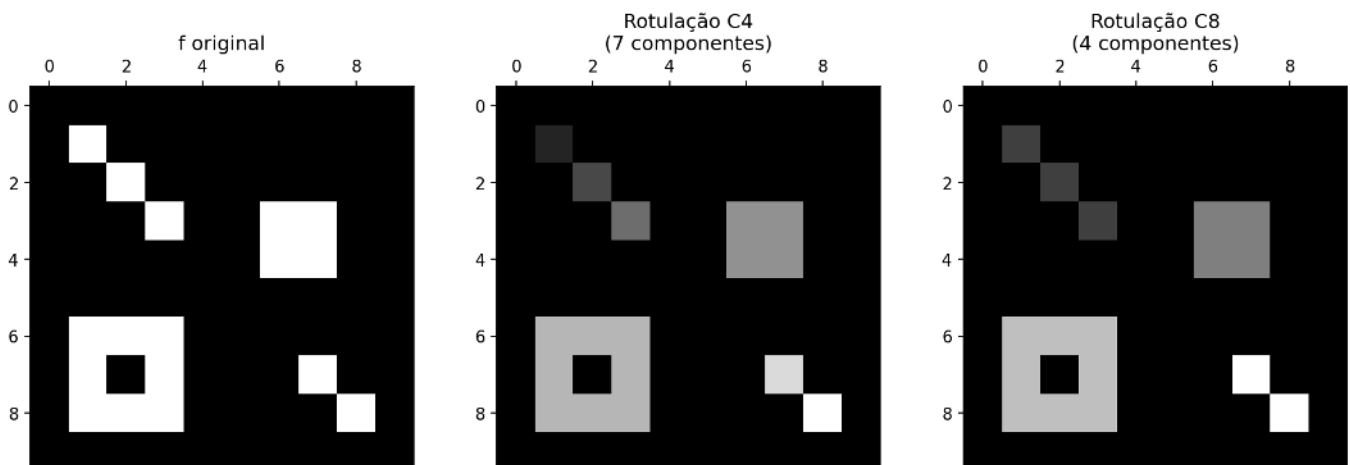


Figura 4.17: Efeito da conectividade na rotulação: a imagem binária 10×10 contém pixels diagonalmente adjacentes. Com conectividade-4, esses pixels formam componentes distintas; com conectividade-8, fundem-se em uma única componente.

4.4.2 Transformada de Distância

A **Transformada de Distância** (TD) é um operador que, aplicado a uma imagem binária f , produz uma imagem em níveis de cinza D na qual cada pixel pertencente ao objeto ($f(x, y) \neq 0$) recebe como valor a distância geométrica até o pixel de fundo ($f(x', y') = 0$) mais próximo:

$$D(x, y) = \min_{(x', y') : f(x', y') = 0} d((x, y), (x', y'))$$

onde $d(\cdot, \cdot)$ é uma métrica de distância — tipicamente a distância Euclidiana (L_2). O resultado é uma representação topográfica dos objetos: pixels localizados no interior assumem valores elevados, enquanto pixels próximos às bordas apresentam baixos valores de distância. Os **máximos locais** de D correspondem aos pontos mais afastados da borda do objeto, frequentemente próximos aos seus centros geométricos ou centros de máxima inscrição — propriedade particularmente útil para a geração automática de marcadores no algoritmo *watershed*.

i Definição formal usando erosões

A TD admite ainda uma interpretação morfológica iterativa, conforme algoritmo da Figure 4.18. Considere uma função estruturante b cujo valor central é nulo e cujos vizinhos possuem custos negativos associados ao deslocamento. Ao aplicar erosões sucessivas com essa função estruturante particular, os valores dos pixels dos objetos (que devem assumir a distância máxima possível da imagem) são progressivamente reduzidos segundo os custos definidos por b . O valor acumulado dessa propagação passa então a representar a distância ao fundo segundo a métrica induzida pela função estruturante.

Essa interpretação é implementada em `mm.dist1()`, que acumula erosões sucessivas utilizando a operação `mm.ero1()`. Já `mm.dist()` delega o cálculo da distância Euclidiana ao operador otimizado do OpenCV `cv2.distanceTransform(f, cv2.DIST_L2, 5)`, onde f é a imagem binária de entrada, `cv2.DIST_L2` especifica a métrica Euclidiana (L_2) e 5 indica o uso de uma máscara 5×5 para aproximar a distância com elevada precisão.

A função `dist1` produz uma transformada de distância discreta cuja métrica é determinada pela geometria e pelos pesos da função estruturante utilizada. Por exemplo, utilizando uma função estruturante em cruz com custo unitário para os quatro vizinhos ortogonais, obtém-se a distância de **Manhattan** (L_1). Outras escolhas de vizinhança e pesos induzem métricas diferentes. Já `mm.dist()` calcula uma aproximação eficiente da distância Euclidiana (L_2).

Por exigir sucessivas erosões sobre toda a imagem, a abordagem `dist1` possui custo computacional significativamente maior que `mm.dist()`, sendo empregada neste livro principalmente para fins didáticos e para evidenciar a relação entre morfologia matemática e transformadas de distância.

Figura 4.18: Algoritmo da Transformada de Distância: passo a passo, cards, linha do tempo, código Python e fluxograma.

A Figure 4.19 apresenta um simulador interativo da TD: é possível posicionar o cursor sobre diferentes pixels do objeto e observar, em tempo real, o valor da distância associado àquela posição, isto é, a distância até o pixel de fundo mais próximo. A Figure 4.20 apresenta um exemplo prático dessa execução em ambiente Python.

```
import numpy as np

# Imagem binária 10x10 com um único pixel de fundo (0,0) e o resto como objeto (255)
f = np.ones((10,10), dtype=np.uint8) * 255
f[0,0] = 0
```

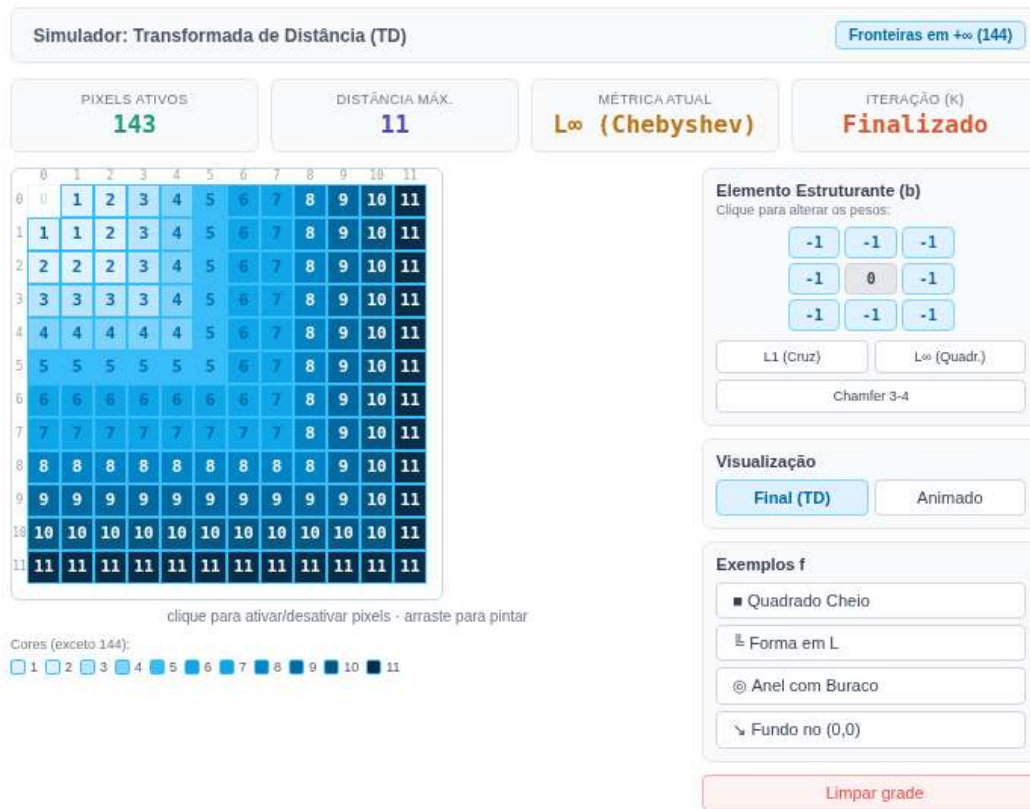



Figura 4.19: Simulador interativo da Transformada de Distância (TD) iterativa via erosão em tons de cinza. Pixels fora da imagem agora assumem o valor máximo (144), propagando os custos apenas a partir do fundo interno.

```

B_cruz = np.array([
    [-np.inf, -1, -np.inf],
    [-1,      0, -1],
    [-np.inf, -1, -np.inf]
], dtype=float)

d_iter = mm.dist1(f, B_cruz)
d_l2 = mm.dist(f)

print(f"Máx. dist1 (erosões) : {d_iter.max()} px")
print(f"Máx. dist (L2)       : {d_l2.max():.2f} px")
print(f"Posição do máximo dist1 : {np.where(d_iter == d_iter.max())}")
print(f"Posição do máximo dist  : {np.where(d_l2 == d_l2.max())}")

mm.show(
    [f, d_iter, d_l2],
    titles=["f original", "mm.dist1\n(erosões com cruz)", "mm.dist\n(L2 Euclidiana)"],
    cols=3, figsize=(12, 4), axis=True
)

```

```

Máx. dist1 (erosões) : 18 px
Máx. dist (L2)       : 12.00 px
Posição do máximo dist1 : (array([9]), array([9]))
Posição do máximo dist  : (array([9]), array([9]))

```

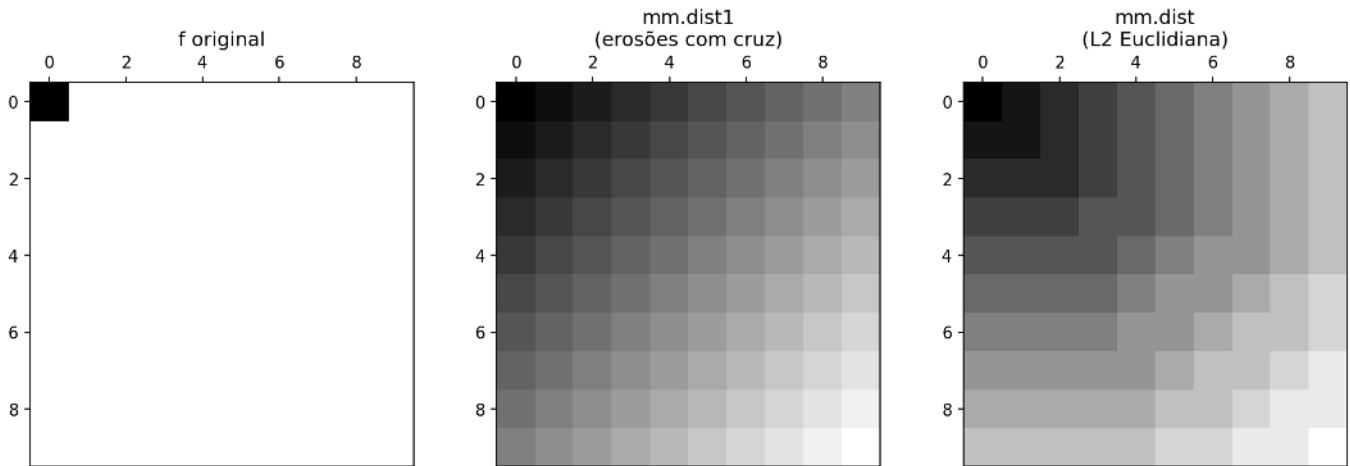


Figura 4.20: Transformada de Distância em imagem binária 10×10. Esquerda: imagem original (*foreground* = 255). Centro: `mm.dist1` iterativa (erosões com elemento cruz). Direita: `mm.dist` (L2 Euclidiana).

```
np.inf
```

```
inf
```

```
np.where(d_iter == d_iter.max())
```

```
(array([9]), array([9]))
```

A anotação dos valores numéricos diretamente sobre os pixels permite verificar como `dist1` propaga as distâncias segundo a métrica induzida pela função estruturante utilizada. No caso do elemento cruz com custo unitário, os valores obtidos correspondem à distância de *Manhattan* (L_1). Embora `dist1` e `mm.dist` produzam valores numéricos distintos por adotarem métricas diferentes, ambas as transformadas preservam a estrutura topográfica dos objetos, fazendo com que seus máximos ocorram em regiões centrais semelhantes. Essa propriedade justifica o uso de `mm.dist` em aplicações práticas, devido à sua elevada eficiência computacional.

4.4.3 Transformada de Distância Euclidiana em quatro passos

O simulador da Figure 4.21 implementa o algoritmo da TDE de Lotufo; Zampirolli (2001) em duas etapas. Na primeira etapa, a função `edt1` realiza uma transformação unidimensional vertical de forma sequencial (*in-place*), percorrendo cada coluna em *raster* ($\downarrow$) e *anti-raster* ($\uparrow$) para calcular as distâncias na direção vertical (dois passos: Sul e Norte). Na segunda etapa, a função `edt2` utiliza esse resultado como entrada e realiza uma propagação horizontal por filas: para cada linha da matriz, duas filas de prioridade, `Eq` e `Wq`, são inicializadas percorrendo os índices de coluna em sentidos opostos (`Eq` de `W-1` até 1, `Wq` de 2 até `W`), de modo que cada pixel atualizado enfileira imediatamente seus vizinhos para reprocessamento dentro da mesma rodada (mais dois passos: Leste e Oeste). Esse mecanismo de fila permite aplicar erosões sucessivas com pesos ímpares crescentes (`b = 1, 3, 5, ...`, incrementado a cada iteração do laço externo) sem que a propagação fique presa em valores desatualizados, já que cada rodada resolve a cadeia de dependências horizontal por completo antes do próximo incremento de `b`. A convergência dessa propagação produz a Transformada de Distância Euclidiana em toda a matriz, combinando a informação vertical obtida em `edt1` com a propagação horizontal em fila realizada em `edt2`.

4.4.3.1 Transformada de Distância Geodésica

A transformada de distância geodésica associa a cada pixel a menor distância até um marcador, sob a restrição imposta por uma máscara. Dessa forma, a propagação ocorre exclusivamente pelos pixels permitidos, preservando a conectividade do domínio.

A Figura Figure 4.22 ilustra esse processo em um labirinto: (a) a máscara `g`; (b) a distância geodésica `D1`



Figura 4.21: Simulador interativo 2D (4x4) com sincronização estrita de filas de propagação horizontal (b) para obter a convergência exata descrita no artigo.

calculada a partir da entrada; (c) a distância D2 calculada a partir da saída; e (d) o caminho mínimo obtido a partir dessas duas transformadas.

O caminho ótimo é determinado pela soma das distâncias ($D1 + D2$). Os pixels pertencentes à trajetória mínima são aqueles para os quais essa soma assume seu menor valor, definindo uma conexão entre entrada e saída com comprimento geodésico mínimo.

Esse princípio permite resolver labirintos sem a necessidade de explorar explicitamente todas as possibilidades de percurso. A solução emerge diretamente da propagação de distâncias em um domínio restrito. Essa abordagem é particularmente relevante em labirintos de elevada complexidade, como os construídos a partir de estruturas quasicristalinas e ciclos hamiltonianos descritos por Singh; Lloyd; Flicker (2024). Em Zampirolli et al. (2025), esse mesmo formalismo é empregado para resolver um labirinto complexo; a seguir, o método é ilustrado em uma versão simplificada do problema.

```
import numpy as np

# 1 = corredor, 0 = parede
g = np.array([
  [0,1,0,0,0,0,0,0,0,0],
  [0,1,1,1,1,1,0,1,1,1],
  [0,0,0,0,0,1,0,1,0,1],
  [0,1,1,1,0,1,1,1,0,1],
  [0,1,0,1,0,0,0,0,0,1],
  [0,1,0,1,1,1,1,1,1,1],
  [0,1,0,0,0,0,0,0,1,0],
  [0,1,1,1,1,1,1,0,1,0],
  [0,0,0,0,0,0,1,1,1,0],
  [0,0,0,0,0,0,0,0,1,0]
], dtype=np.uint8)

# marcador da entrada
entrada = np.zeros_like(g, dtype=np.uint8)
entrada[0,1] = 1

# marcador da saída
saida = np.zeros_like(g, dtype=np.uint8)
```

```

saida[9,8] = 1

# distâncias geodésicas
D1 = mm.gdist(g, entrada)
D2 = mm.gdist(g, saida)

# soma das distâncias
S = D1 + D2

# menor valor válido da soma
dmin = np.min(S[S > 0])

# pixels pertencentes a um caminho ótimo
caminho = (S == dmin)

print("Distância geodésica mínima:", dmin)

mm.show(
    [g, D1, D2, caminho],
    titles=[
        "Labirinto",
        "Distância da Entrada",
        "Distância da Saída",
        f"Menor Caminho\n(d={dmin})"
    ],
    cols=4,
    figsize=(14,4),
    axis=True
)

```

Distância geodésica mínima: 15

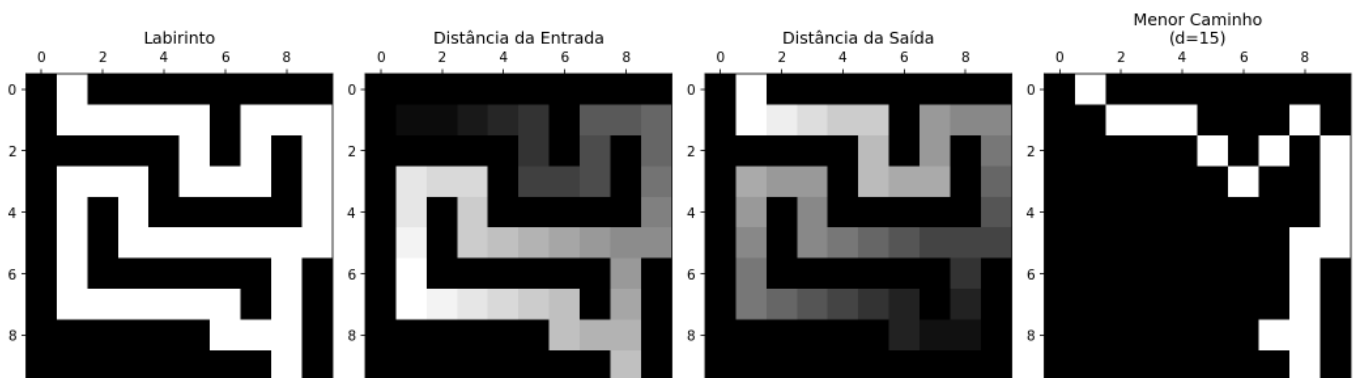


Figura 4.22: Menor caminho geodésico em um labirinto. As distâncias geodésicas são calculadas a partir da entrada e da saída usando `mm.gdist`. Os pixels cujo somatório das duas distâncias é igual à distância mínima entre os marcadores pertencem a um caminho ótimo.

4.4.4 Segmentação por *Watershed*

O algoritmo ***Watershed*** interpreta uma imagem em níveis de cinza como uma superfície topográfica, na qual valores elevados correspondem a montanhas e valores baixos correspondem a vales ou bacias de drenagem (*catchment basins*). No contexto da segmentação baseada em marcadores, os máximos da Transformada de Distância são frequentemente utilizados para identificar regiões internas aos objetos, fornecendo sementes confiáveis para o processo de inundação.

A segmentação é então realizada por uma simulação conceitual de inundação progressiva a partir desses marcadores. À medida que as bacias associadas a diferentes sementes se expandem, regiões vizinhas

eventualmente entram em contato. Nesse instante são construídas barreiras virtuais, denominadas *watershed lines*, que passam a delimitar os objetos da cena. Esse mecanismo permite separar objetos adjacentes ou parcialmente sobrepostos, mesmo quando eles formam uma única componente conexa após a limiarização. A implementação didática apresentada neste capítulo explora inicialmente o conceito de crescimento de regiões (*region growing*) confinado por uma máscara binária, conforme detalhado no algoritmo iterativo da Figure 4.23.

i Versão didática versus implementação clássica

A função `mm.watershed0` não implementa o algoritmo *watershed* clássico. Seu objetivo é ilustrar, de forma simplificada, a propagação de marcadores por crescimento de regiões (*region growing*), permitindo visualizar como diferentes sementes competem pela ocupação do espaço disponível. O crescimento é delimitado por uma máscara binária de suporte e monitorado por um controle de estagnação, gerando um resultado semelhante a uma partição de Voronoi restrita à geometria dos objetos de entrada. Já a função `mm.watershed` utiliza a implementação otimizada do OpenCV (`cv2.watershed`), que realiza a inundação sobre uma superfície topográfica definida pela imagem de entrada. Nesse caso, a propagação dos marcadores é influenciada pelos valores dos pixels, fazendo com que as linhas de separação se formem naturalmente sobre as cristas do relevo.

Figura 4.23: Algoritmo Didático de *Watershed* por Crescimento de Regiões Limitado por Máscara: passo a passo, cards, linha do tempo, código Python e fluxograma.

A Figure 4.24 apresenta um simulador iterativo que ilustra a propagação dos marcadores pela região de interesse. Cada marcador atua como uma fonte de inundação que expande sua área de influência até encontrar regiões provenientes de outras sementes. No algoritmo do OpenCV, os pixels pertencentes às linhas divisoras são identificados pelo valor `-1`, representando as fronteiras entre bacias adjacentes.

Pipeline Morfológico do *Watershed*

O *watershed* baseado em marcadores normalmente integra um fluxo mais amplo de segmentação. Em imagens reais, etapas de pré-processamento são frequentemente necessárias para melhorar o contraste, reduzir ruídos e gerar marcadores confiáveis. Esse fluxo completo está resumido na Table 4.6.

Tabela 4.6: *Pipeline* completo do *watershed* baseado em marcadores para imagens reais.

Etapas	Operação	Finalidade
1	CLAHE + Suavização	Realce de contraste e redução de ruído
2	Limiarização	Separação inicial entre objeto e fundo
3	Abertura/Fechamento	Remoção de ruídos e pequenas imperfeições
4	Dilatação da Máscara	Identificação do Fundo Certo (<i>Sure Background</i>)
5	Transformada de Distância + Limiar	Identificação do Objeto Certo (<i>Sure Foreground</i>)
6	Região Incerta	Diferença entre Fundo Certo e Objeto Certo
7	<code>cv2.watershed</code>	Propagação dos marcadores pela região incerta

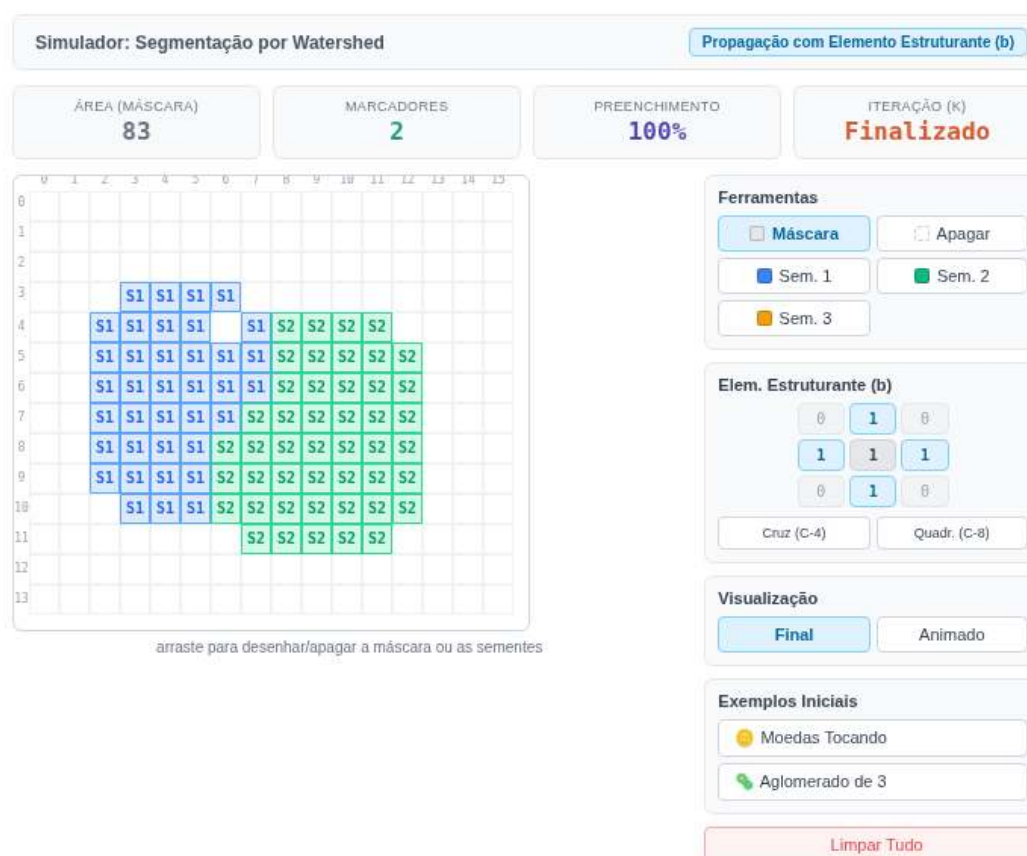


Figura 4.24: Simulador interativo do Algoritmo *Watershed* por propagação morfológica. Desenhe a máscara, posicione os marcadores e ajuste o Elemento Estruturante para observar a inundação. Quando as bacias se encontram simultaneamente, o empate é resolvido assumindo uma das regiões de forma aleatória.

Para enfatizar exclusivamente os conceitos de Transformada de Distância, marcadores e inundação topográfica, o exemplo da Figure 4.25 utiliza uma imagem binária sintética e adota um fluxo simplificado, resumido na Table 4.7.

Tabela 4.7: *Pipeline* simplificado utilizado no exemplo didático da Figure 4.25.

Etapas	Operação	Finalidade
1	Transformada de Distância	Construção da superfície topográfica
2	Limiar da TD	Extração dos marcadores (<i>Sure Foreground</i>)
3	Dilatação	Determinação do Fundo Certo (<i>Sure Background</i>)
4	Região Incerta	Diferença entre fundo e marcadores
5	<code>cv2.watershed</code>	Propagação dos marcadores e geração das fronteiras

```
import cv2

# 1. Imagem sintética e Transformada de Distância
f_sint = np.zeros((20, 20), dtype=np.uint8)
cv2.circle(f_sint, (6, 10), 5, 255, -1)
cv2.circle(f_sint, (14, 10), 5, 255, -1)
dist = mm.dist(f_sint)

# 2. Marcadores (picos da distância)
m = (dist > 0.8 * dist.max()).astype(np.uint8) * 255

# 3. Execução do Watershed0 condicional (m=marcadores primeiro, mask=f_sint)
w_reg = mm.watershed0(m, mask=f_sint, op='region')
w_line = mm.watershed0(m, mask=f_sint, op='line')

#w_reg = mm.watershedB(m, mask=f_sint, op='region')
#w_line = mm.watershedB(m, mask=f_sint, op='line')

w_reg = mm.watershed(m, mask=f_sint, op='region')
w_line = mm.watershed(m, mask=f_sint, op='line')

# 4. Exibição dos resultados
mm.show([f_sint, dist, m, w_reg, w_line], cols=5, figsize=(16, 4),
        titles=["Original", "Distância", "Marcadores", "Regiões", "Linhas"])
```

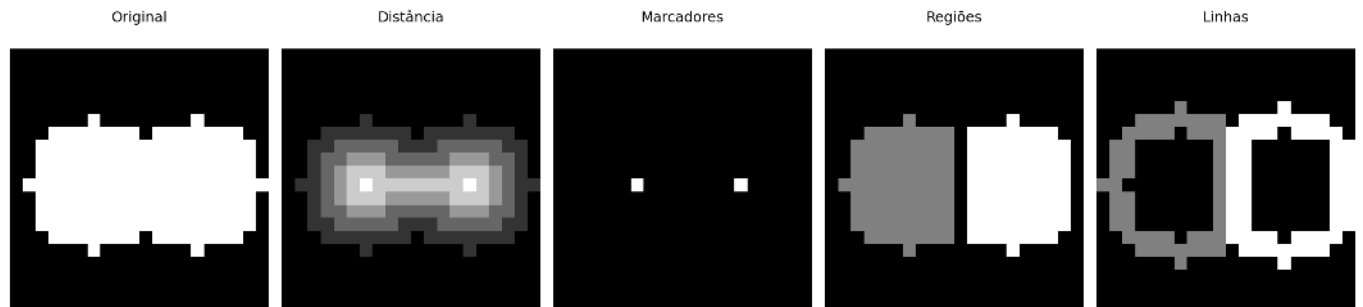


Figura 4.25: *Pipeline watershed* delimitado por máscara em imagem binária 20×20.

Aplicação em moedas sobrepostas:

```

img_base = img_coins_gray

# 1. Simular moedas sobrepostas/conectadas (usando a máscara binária base)
img_sobrepostas = mm.dil(img_final, mm.sebox(40))

# 2. Abertura morfológica para limpar ruídos
opening = mm.open(img_sobrepostas, mm.sebox(2))

# 3. Transformada de Distância
dist = mm.dist(opening)
dist_vis = (255 * (dist / dist.max())).astype(np.uint8) if dist.max() > 0 else dist.astype(np.uint8)

# 4. Picos seguros (Marcadores das moedas)
picos = (dist > 0.5 * dist.max()).astype(np.uint8) * 255

# 5. Execução do Watershed (Ajustado para usar a nova assinatura)
# Passamos 'opening' direto em mask, pois ela delimita o escopo de expansão das moedas
ws_region = mm.watershed(picos, mask=opening, op='region')
ws_line = mm.watershed(picos, mask=opening, op='line')

# 6. Contagem de objetos (Ignora fundo 0)
labels = np.unique(ws_region)
labels = labels[labels > 0]
print(f"Objetos detectados: {len(labels)}")

# 7. Anotação Final
img_annotated = cv2.cvtColor(img_base, cv2.COLOR_GRAY2BGR)

for idx, label_id in enumerate(labels):
    mask_reg = (ws_region == label_id).astype(np.uint8)
    if mask_reg.sum() < 500: continue

    cy, cx = np.mean(np.where(mask_reg), axis=1).astype(int)
    cv2.putText(img_annotated, str(idx + 1), (cx - 25, cy + 20),
                cv2.FONT_HERSHEY_SIMPLEX, 3.2, (0, 255, 0), 8, cv2.LINE_AA)

# Desenha as linhas de separação em vermelho
mask_ann = mm.dil(ws_line, np.ones((11, 11), np.uint8))
img_annotated[mask_ann > 0] = [255, 0, 0]

# Exibição
mm.show(
    [img_base, img_sobrepostas, dist_vis, picos, ws_region, img_annotated],
    titles=["Original", "Sobrepostas", "Distância", "Marcadores", "Watershed", "Anotado"],
    cols=3, rows=2, figsize=(12, 8)
)

```

Objetos detectados: 12

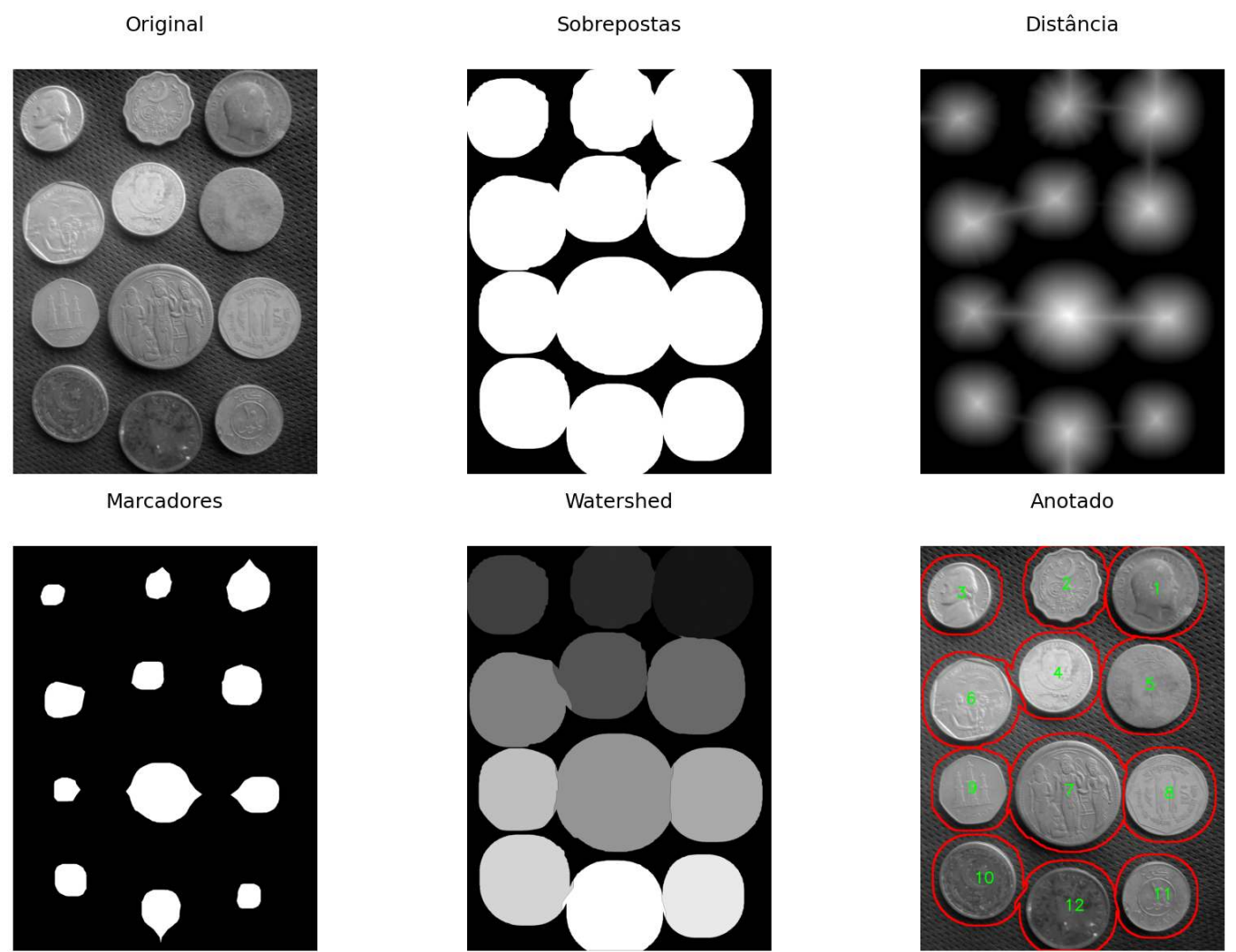


Figura 4.26: *Pipeline Watershed* para separação de moedas sobrepostas: da máscara binária dilatada até os contornos finais anotados sobre a imagem original.

4.5 Extração de Componentes e Descritores de Forma

Após a segmentação e o refinamento morfológico, o próximo passo consiste em identificar individualmente cada objeto presente na imagem e extrair suas propriedades geométricas. Essa etapa é fundamental para tarefas de medição, classificação e reconhecimento de padrões.

Um **componente conexo** é um conjunto maximal de pixels pertencentes ao objeto que permanecem mutuamente conectados segundo uma relação de conectividade previamente definida (4- ou 8-conectividade). Após a rotulação, cada componente recebe um identificador único, permitindo que suas características sejam analisadas individualmente.

O OpenCV oferece duas abordagens complementares para essa análise, resumidas na Table 4.8.

Tabela 4.8: Comparação entre as abordagens baseadas em componentes conexos e em contornos.

	<code>connectedComponentsWithStats</code>	<code>findContours</code>
Retorna	rótulo por pixel e estatísticas por componente	sequência de pontos que descreve a borda
Descritores diretos	área, <i>bounding box</i> e centróide	perímetro, forma e hierarquia
Objetos em contato	tende a fundir regiões conectadas	tende a produzir um único contorno externo

	<code>connectedComponentsWithStats</code>	<code>findContours</code>
Uso típico	contagem, filtragem e rotulação	análise geométrica e descritores de forma

4.5.1 Rotulação e Estatísticas de Componentes

A função `cv2.connectedComponentsWithStats` realiza simultaneamente a rotulação dos componentes conexos e a extração de descritores básicos de cada região. O operador retorna:

- uma imagem de rótulos (`labels`);
- estatísticas geométricas (`stats`);
- coordenadas dos centróides (`centroids`).

As estatísticas incluem área, largura, altura e posição da *bounding box* mínima alinhada aos eixos da imagem.

A Figure 4.27 ilustra a aplicação desse operador após o *pipeline* de segmentação das moedas. Cada componente conexo recebe uma cor distinta e sua área é anotada diretamente sobre a imagem.

```
# 1. Rotulagem e estatísticas
n, labels, stats, centroids = cv2.connectedComponentsWithStats(
    img_final, connectivity=8
)

# 2. Coloração dos componentes
np.random.seed(4)
colors = np.random.randint(50, 255, (n, 3), dtype=np.uint8)
colors[0] = [0, 0, 0] # fundo preto
img_colored = colors[labels]
img_annotated = img_colored.copy()

# 3. Tabela de descritores
print(f"Componentes detectados (excluindo fundo): {n - 1}")
print(f"\n{'ID':>4} {'Área':>8} {'cx':>6} {'cy':>6} {'w':>6} {'h':>6}")
print("-" * 42)
for i in range(1, n):
    area = stats[i, cv2.CC_STAT_AREA]
    cx, cy = int(centroids[i, 0]), int(centroids[i, 1])
    w, h = stats[i, cv2.CC_STAT_WIDTH], stats[i, cv2.CC_STAT_HEIGHT]
    print(f"{i:>4} {area:>8} {cx:>6} {cy:>6} {w:>6} {h:>6}")
    for cor, esp in [((0,0,0), 10), ((255,0,0), 5)]:
        cv2.putText(img_annotated, f"{i}: {area}",
                    (cx - 150, cy + 18),
                    cv2.FONT_HERSHEY_SIMPLEX, 2.0, cor, esp, cv2.LINE_AA)

mm.show(
    [img_coins_gray, img_final, img_colored, img_annotated],
    titles=["Original", "Segmentação Final", "Componentes Conexos", "Áreas Anotadas"],
    cols=4, figsize=(18, 6)
)
```

Componentes detectados (excluindo fundo): 12

ID	Área	cx	cy	w	h

1	231969	1484	280	553	542
2	158967	917	250	453	468
3	139229	250	312	433	419
4	175550	857	821	474	465
5	222882	1442	889	539	531

6	210043	316	977	527	516
7	343376	934	1559	667	665
8	213641	1555	1574	530	515
9	147880	328	1543	432	438
10	187280	363	2111	487	492
11	150110	1487	2215	433	444
12	215387	932	2292	528	522

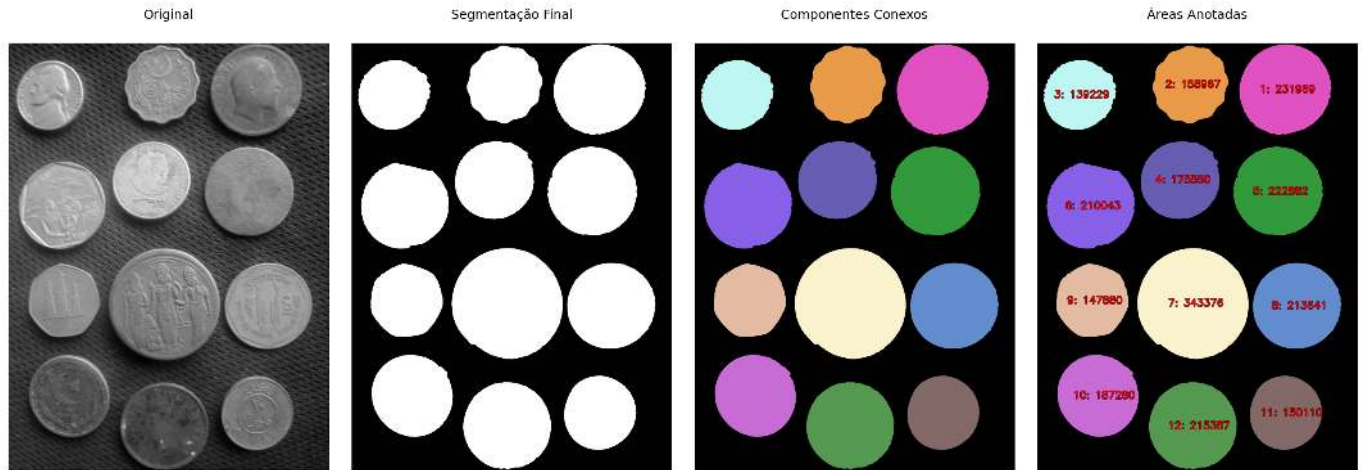


Figura 4.27: Componentes conexos extraídos após o *pipeline* CLAHE → Otsu → limpeza morfológica. Cada objeto é colorido com cor distinta e anotado com sua área em pixels.

4.5.2 Descritores de Forma

Enquanto `connectedComponentsWithStats` opera sobre regiões, `cv2.findContours` atua diretamente sobre suas fronteiras. O operador retorna, para cada objeto, uma sequência ordenada de pontos que descreve seu contorno.

A partir dessa representação é possível calcular descritores geométricos que não são fornecidos diretamente pela rotulação de componentes:

- **Área** (`cv2.contourArea`);
- **Perímetro** (`cv2.arcLength`);
- **Circularidade** ($C = \frac{4\pi A}{P^2}$, onde A é a área e P o perímetro);
- **Aproximação poligonal** (`cv2.approxPolyDP`);
- **Fecho convexo** (*convex hull*);
- **Hierarquia de contornos**, permitindo representar relações pai-filho entre regiões e seus buracos internos.

A circularidade assume valor máximo igual a 1 para um círculo perfeito e diminui à medida que o objeto se torna mais alongado ou apresenta irregularidades em sua borda.

A Figure 4.28 apresenta os contornos extraídos das moedas segmentadas, juntamente com os valores de circularidade calculados para cada objeto.

```
contornos, hierarquia = cv2.findContours(
    img_final, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

img_contornos = cv2.cvtColor(img_coins_gray, cv2.COLOR_GRAY2BGR)
img_circulares = img_contornos.copy()
```

```

print(f"Contornos detectados: {len(contornos)}")
print(f"\n{'ID':>4} {'Área':>8} {'Perímetro':>10} {'Circularidade':>14}")
print("-" * 42)

for i, cnt in enumerate(contornos, start=1):
    area = cv2.contourArea(cnt)
    perim = cv2.arcLength(cnt, closed=True)
    circ = (4 * np.pi * area / perim**2) if perim > 0 else 0
    M = cv2.moments(cnt)
    cx = int(M["m10"] / M["m00"]) if M["m00"] > 0 else 0
    cy = int(M["m01"] / M["m00"]) if M["m00"] > 0 else 0

    print(f"{'i':>4} {'area':>8.0f} {'perim':>10.1f} {'circ':>14.3f}")

    cv2.drawContours(img_contornos, [cnt], -1, (0, 255, 0), 3)
    cv2.drawContours(img_circulares, [cnt], -1, (0, 255, 0), 3)
    for cor, esp in [((0,0,0), 8), ((255,0,0), 3)]:
        cv2.putText(img_circulares, f"{circ:.2f}",
                    (cx - 80, cy + 15),
                    cv2.FONT_HERSHEY_SIMPLEX, 3.6, cor, esp, cv2.LINE_AA)

mm.show(
    [img_final, img_contornos, img_circulares],
    titles=["Segmentação Final", "Contornos", "Circularidade"],
    cols=3, figsize=(18, 6)
)

```

Contornos detectados: 12

ID	Área	Perímetro	Circularidade

1	214626	1769.6	0.861
2	149480	1465.1	0.875
3	186570	1654.9	0.856
4	147252	1458.6	0.870
5	212886	1756.3	0.867
6	342424	2232.5	0.863
7	209282	1767.7	0.842
8	222108	1809.7	0.852
9	174854	1616.4	0.841
10	138602	1471.5	0.804
11	158306	1538.7	0.840
12	231181	1847.4	0.851



Figura 4.28: Contornos extraídos com *findContours*. Cada moeda é anotada com sua circularidade — valores próximos de 1 confirmam forma circular.

4.5.3 Conexão com Detecção de Objetos Moderna

Os descritores extraídos nas seções anteriores — especialmente *bounding boxes*, centróides, áreas e medidas de forma — estabelecem uma ponte natural entre a segmentação morfológica clássica e os sistemas modernos de detecção de objetos. Embora as técnicas estudadas neste capítulo utilizem operações sobre pixels e regiões segmentadas, muitas das representações produzidas são diretamente compatíveis com os formatos empregados em modelos contemporâneos de visão computacional.

Detectores baseados em aprendizado profundo, como a família YOLO (*You Only Look Once*) (Redmon et al., 2016), operam diretamente sobre imagens coloridas e produzem, para cada objeto detectado, uma *bounding box* descrita pelo centro (cx, cy) e pelas dimensões (w, h), além de uma classe e uma pontuação de confiança. Essa representação compartilha a mesma estrutura geométrica básica obtida por `connectedComponentsWithStats`, embora seja produzida por um modelo aprendido e não por segmentação explícita.

A Figure 4.29 ilustra como as *bounding boxes* obtidas por morfologia podem ser exportadas no formato YOLO para compor conjuntos de dados utilizados no treinamento ou na avaliação de detectores.

```
H_img, W_img = img_coins_gray.shape
img_bbox = cv2.cvtColor(img_coins_gray, cv2.COLOR_GRAY2BGR)

CLASSE = 0 # 0 = moeda (única categoria neste exemplo)

print(f"{'cls':>4} {'cx_n':>8} {'cy_n':>8} {'w_n':>8} {'h_n':>8} ← formato YOLO")
print("-" * 54)

yolo_linhas = []
for i in range(1, n):
    x0 = stats[i, cv2.CC_STAT_LEFT]
    y0 = stats[i, cv2.CC_STAT_TOP]
    w = stats[i, cv2.CC_STAT_WIDTH]
    h = stats[i, cv2.CC_STAT_HEIGHT]

    cx_n = (x0 + w / 2) / W_img
    cy_n = (y0 + h / 2) / H_img
    w_n = w / W_img
    h_n = h / H_img

    yolo_linhas.append(f"{CLASSE} {cx_n:.4f} {cy_n:.4f} {w_n:.4f} {h_n:.4f}")
print(f"{CLASSE:>4} {cx_n:>8.4f} {cy_n:>8.4f} {w_n:>8.4f} {h_n:>8.4f}")
```

```

cv2.rectangle(img_bbox, (x0, y0), (x0 + w, y0 + h), (0, 255, 0), 4)
cv2.putText(img_bbox, f"moeda", (x0 + 8, y0 + 60),
            cv2.FONT_HERSHEY_SIMPLEX, 3.8, (255, 0, 0), 5, cv2.LINE_AA)

# Exportar arquivo de anotação no formato YOLO
with open("moedas.txt", "w") as f:
    f.write("\n".join(yolo_linhas))
print("\nAnotação salva em moedas.txt")

mm.show(
    [img_coins_gray, img_final, img_bbox],
    titles=["Original", "Segmentação Final", "Bounding Boxes (formato YOLO)"],
    cols=3, figsize=(18, 6)
)

```

cls	cx_n	cy_n	w_n	h_n	← formato YOLO
0	0.7753	0.1102	0.2880	0.2117	
0	0.4784	0.0984	0.2359	0.1828	
0	0.1331	0.1225	0.2255	0.1637	
0	0.4464	0.3217	0.2469	0.1816	
0	0.7523	0.3471	0.2807	0.2074	
0	0.1664	0.3812	0.2745	0.2016	
0	0.4862	0.6104	0.3474	0.2598	
0	0.8115	0.6154	0.2760	0.2012	
0	0.1724	0.6023	0.2250	0.1711	
0	0.1893	0.8258	0.2536	0.1922	
0	0.7763	0.8652	0.2255	0.1734	
0	0.4854	0.8953	0.2750	0.2039	

Anotação salva em moedas.txt

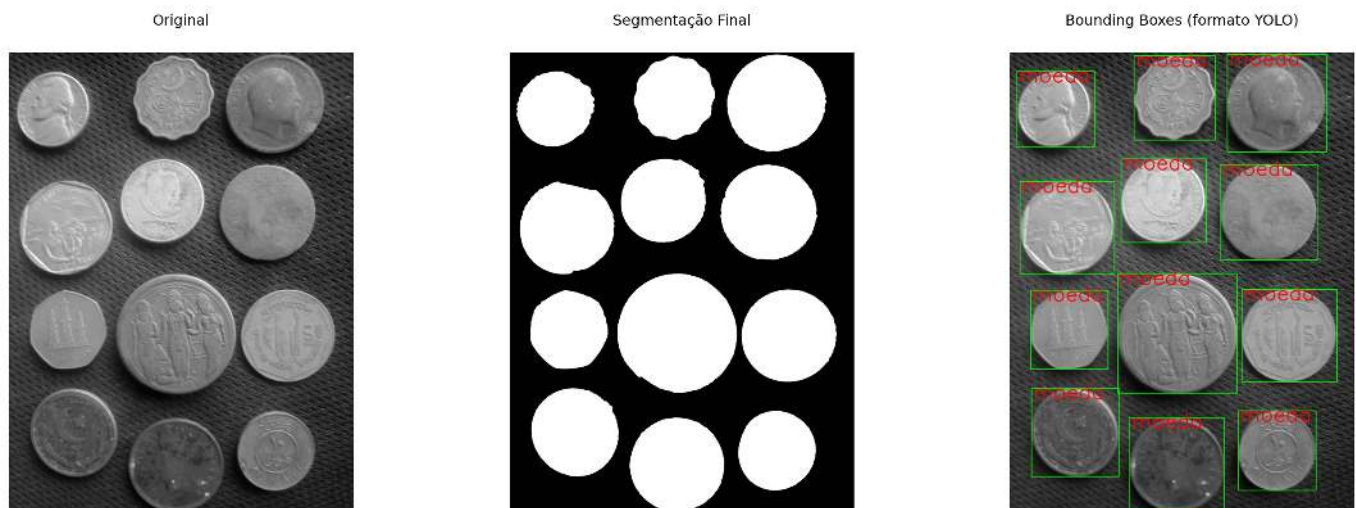


Figura 4.29: *Bounding boxes* derivadas dos componentes conexos sobrepostas à imagem original. As anotações são exportadas no formato YOLO (*classe cx cy w h*), com coordenadas normalizadas para o intervalo $[0,1]$.

O formato YOLO armazena cada objeto em uma linha contendo cinco campos:

classe cx cy w h

onde (cx, cy) representa o centro da *bounding box* e (w, h) suas dimensões. Todos os valores geométricos são normalizados para o intervalo $[0, 1]$ em relação à largura e à altura da imagem. A **classe** é um identificador

inteiro associado a uma categoria definida pelo conjunto de dados (por exemplo, $0 \rightarrow \text{moeda}$). Quando há múltiplas categorias — como moeda de ouro (0), moeda de prata (1) e disco plástico (2) — basta atribuir o identificador correspondente a cada objeto antes da exportação, mantendo exatamente o mesmo formato de anotação. No exemplo anterior, essas informações foram armazenadas no arquivo `moedas.txt`.

O fluxo apresentado neste capítulo — segmentação $\rightarrow$ rotulação $\rightarrow$ extração de *bounding boxes* — corresponde conceitualmente à etapa de **anotação** (*labeling*) empregada na construção de conjuntos de treinamento para detectores modernos. Ferramentas especializadas, como Label Studio e Roboflow, automatizam esse processo em imagens complexas, mas a lógica fundamental permanece a mesma: associar a cada objeto uma região de interesse e uma classe. Em cenários controlados, com fundo uniforme e objetos bem separados, técnicas morfológicas podem inclusive gerar anotações automaticamente ou servir como ponto de partida para a rotulação manual, reduzindo significativamente o esforço de construção do conjunto de dados. Em aplicações reais mais complexas, entretanto, a validação humana continua sendo necessária para garantir a qualidade das anotações.

Avaliação: IoU (*Intersection over Union*)

Uma forma simples de avaliar a qualidade de uma segmentação consiste em compará-la com uma máscara de referência (*ground truth*). A métrica mais utilizada para essa finalidade é a **IoU** (*Intersection over Union*):

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} \quad (4.16)$$

onde A representa a segmentação produzida pelo algoritmo e B a segmentação de referência.

O valor da IoU varia entre 0 e 1. Quanto maior o valor, maior a sobreposição entre as máscaras. Uma IoU igual a 1 indica correspondência perfeita entre a segmentação obtida e a referência.

A mesma métrica também é amplamente utilizada em detecção de objetos, sendo aplicada às *bounding boxes* previstas e anotadas. Nessa área, valores de IoU superiores a 0,5 são frequentemente adotados como critério mínimo para considerar uma detecção correta.

Além da Morfologia

As técnicas estudadas neste capítulo segmentam objetos explorando conectividade espacial, operações morfológicas e relevo topográfico. Existem, entretanto, abordagens alternativas baseadas em agrupamento de características, como o algoritmo *k-means*, modelos de mistura Gaussiana (GMM) e métodos mais recentes baseados em aprendizado profundo. Essas técnicas serão retomadas na Parte II do livro, dedicada à Visão Computacional.

4.6 Resumo

Neste capítulo foram apresentadas as principais técnicas de segmentação e morfologia matemática, concluindo o estudo do PDI no domínio espacial.

- **Pré-processamento e limiarização:** A combinação entre equalização adaptativa CLAHE e o método de Otsu mostrou-se eficaz para induzir separação bimodal no histograma e simplificar a binarização de imagens com iluminação não uniforme.
- **Erosão e dilatação:** Operadores morfológicos fundamentais baseados na busca por mínimos e máximos locais em uma vizinhança definida pelo elemento estruturante B . São operadores duais pelo complemento e foram implementados por meio das funções `mm.ero` e `mm.dil`.
- **Abertura e fechamento:** Composições de erosão e dilatação que permitem remover ruídos, suavizar contornos e preencher pequenas lacunas, preservando a estrutura global dos objetos.
- **Reconstrução morfológica:** Processo geodésico iterativo que propaga um marcador dentro dos limites impostos por uma máscara, constituindo a base de operadores como `mm.clohole` e `mm.edgeoff`.

- **Pipeline de limpeza binária:** Fluxo consolidado composto por CLAHE → Otsu → abertura → `mm.clohole` → abertura restrita → `mm.edgeoff`, produzindo máscaras adequadas para análise quantitativa.
- **Morfologia em tons de cinza:** Extensão algébrica baseada em mínimos e máximos ponderados, viabilizando operadores como gradiente morfológico e filtros *top-hat* para realce de estruturas locais.
- **Transformada de Distância e Watershed:** A Transformada de Distância permitiu gerar marcadores automáticos para o algoritmo *watershed*, possibilitando a separação de objetos adjacentes ou parcialmente sobrepostos.
- **Componentes conexos e descritores:** A rotulação de regiões (`cv2.connectedComponentsWithStats`) e a extração de contornos (`cv2.findContours`) permitiram calcular descritores geométricos como área, centróide, perímetro, circularidade e *bounding boxes*.
- **Conexão com Visão Computacional Moderna:** As *bounding boxes* extraídas por morfologia foram exportadas no formato YOLO, evidenciando a ligação entre técnicas clássicas de segmentação e sistemas modernos de detecção de objetos.

O Capítulo 5 introduzirá técnicas de processamento no **domínio da frequência**, abordando a **Transformada de Fourier**, filtragem espectral e os fundamentos da **compressão de imagens**, incluindo DCT, JPEG e *wavelets*.

4.7 Uso do NotebookLM como Tutor Complementar

Nesta edição, o uso do **NotebookLM** é incentivado como ferramenta complementar de aprendizagem. Baseado em inteligência artificial, o sistema utiliza exclusivamente os documentos fornecidos pelo autor como fonte de conhecimento, produzindo respostas alinhadas ao conteúdo e à abordagem adotada ao longo deste capítulo.

 Estude com o Tutor Inteligente

 [ACESSAR NOTEBOOKLM: CAPÍTULO 04](#)

 **Aviso sobre Conteúdo Gerado por IA**

Embora seja uma ferramenta valiosa de apoio aos estudos, o NotebookLM pode eventualmente produzir respostas incompletas, imprecisas ou incorretas. Recomenda-se validar as informações consultando o material do capítulo, livros, artigos científicos e outras fontes acadêmicas confiáveis. Sempre que possível, execute e experimente os exemplos práticos apresentados ao longo do texto para consolidar a compreensão dos conceitos.

4.8 Lista de Exercícios

1. (10%) Implemente manualmente o critério de Otsu sem utilizar `cv2.threshold`. Calcule a variância interclasses $\sigma_B^2(T)$ para todos os limiares $T \in [0, 255]$ usando `mm.hist`, identifique o limiar ótimo T^* e compare o resultado com o valor obtido pelo OpenCV. Plote σ_B^2 em função de T e destaque o ponto de máximo.
2. (15%) Aplique limiarização adaptativa com blocos de tamanho 11, 31 e 51 a uma imagem contendo iluminação não uniforme. Compare os resultados com a limiarização global de Otsu e discuta as vantagens e limitações de cada abordagem.
3. (15%) Execute o *pipeline watershed* da imagem de moedas variando o limiar aplicado à Transformada de Distância (0.3, 0.5 e 0.7 vezes o valor máximo). Explique como esse parâmetro influencia a geração dos marcadores, a separação de objetos adjacentes e a ocorrência de sobre-segmentação.
4. (15%) Utilizando `mm.drawImg`, construa uma demonstração visual passo a passo da erosão de uma imagem binária 7×7 com elemento estruturante quadrado 3×3 . Para cada posição analisada, indique

se o elemento estruturante está completamente contido no objeto e justifique o valor atribuído ao pixel de saída.

5. (15%) Demonstre experimentalmente a dualidade entre erosão e dilatação verificando a identidade $(A \ominus B)^c = A^c \oplus \hat{B}$ utilizando `mm.ero`, `mm.dil` e `mm.bnot`. Calcule a diferença pixel a pixel entre os dois lados da equação e apresente o resultado utilizando `mm.histImg` ou visualização equivalente.
6. (15%) Implemente manualmente o gradiente morfológico utilizando apenas `mm.ero` e `mm.dil`, comparando o resultado com `cv2.morphologyEx(img, cv2.MORPH_GRADIENT, B)`. Avalie o efeito de diferentes elementos estruturantes (quadrado 3×3 , disco 5×5 e linha 1×9) sobre a detecção de bordas.
7. (15%) Construa um *pipeline* completo para contagem e classificação de moedas por tamanho (pequena, média e grande) utilizando área e circularidade como descritores. Gere uma máscara de referência (*ground truth*) manualmente e calcule a métrica IoU (*Intersection over Union*) para avaliar a qualidade da segmentação. Apresente os resultados em tabela e por meio de visualizações produzidas com `mm.show`.

Referências do Capítulo

A fundamentação teórica deste capítulo baseia-se nas seguintes obras:

- Gonzalez; Woods (2018) para os conceitos de segmentação, limiarização de Otsu, Transformada de Distância, *watershed*, morfologia matemática e descritores de forma.
- Matheron (1975) e Serra (1982) para a fundamentação teórica original, formulação algébrica e desenvolvimento da Morfologia Matemática.
- Szeliski (2022) para segmentação baseada em regiões, rotulação de componentes conexos, *watershed* baseado em marcadores e avaliação de segmentação por meio da métrica IoU.
- Bradski; Kaehler (2008) para a utilização prática da biblioteca OpenCV, incluindo funções como `cv2.distanceTransform`, `cv2.watershed`, `cv2.connectedComponentsWithStats` e `cv2.findContours`.
- Redmon et al. (2016) para a introdução aos detectores modernos da família YOLO e sua relação com descritores geométricos como *bounding boxes* extraídas por segmentação.
- Singh; Lloyd; Flicker (2024) para a construção de labirintos complexos baseados em ciclos hamiltonianos sobre mosaicos quasicristalinos, utilizados como exemplo de aplicação da Transformada de Distância Geodésica e de algoritmos de busca de caminhos.
- Zampirolli et al. (2025) para a implementação dos operadores morfológicos, transformadas geodésicas e resolução de labirintos por propagação de distâncias em domínios restritos.

 Executar  Colab  Abrir si-md2  GitHub

4.9 Parte Prática com Exercícios de Programação

Em construção!

Esta seção está sendo preparada e será disponibilizada em breve com exercícios práticos, desafios de programação e exemplos.

Esta lista transforma os conceitos do Capítulo 4 em uma trilha prática de segmentação e morfologia matemática. Os EPs começam com limiarização e avançam até rotulação e descritores de componentes, sempre com matrizes pequenas para que cada pixel possa ser conferido à mão.

Regra comum dos EPs morfológicos

Nas operações com vizinhança, **não faça padding**. Para cada pixel, avalie apenas as posições do elemento estruturante que caem dentro do domínio da imagem. Essa é a mesma ideia das implementações

didáticas em `morph.py`, como `mm.dil0`, `mm.ero0`, `mm.dil1` e `mm.label0`: a vizinhança é recortada pelo domínio válido da imagem.

🎯 Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.0

Executando os Testes

Para rodar os testes, execute `TestSuite("EP04_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:

```
codigo = """
from morph import mm
# ... seu código aqui ...
"""
TestSuite("EP04_01").run_code(codigo)
```

4.9.1 EP04_01 Limiarização Global por Limiar Fixo

Em **scanners de documentos** e em **sistemas de leitura de código de barras**, a primeira etapa do processamento é sempre separar o que é “objeto” (tinta, texto, barras) do que é “fundo” (papel, embalagem). A **limiarização global** faz exatamente isso: compara cada pixel a um único limiar T e decide, em tempo real, se ele pertence à classe clara ou à classe escura. É o operador de segmentação mais simples — e mesmo assim, está por trás de boa parte dos *pipelines* industriais de inspeção visual. Ver na Figure 4.30 uma simulação deste EP.

4.9.1.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).

2. **Limiar:** Ler o inteiro T (limiar de decisão).
3. **Dados:** Ler os valores inteiros da matriz original linha a linha.
4. **Mapeamento:** Para cada pixel p , calcular o novo valor pela equação:

$$p' = \begin{cases} 255, & \text{se } p > T \\ 0, & \text{se } p \leq T \end{cases}$$

5. **Saída:** Exibir a matriz binarizada com dimensões $L \times C$.

4.9.1.2 Restrições Computacionais

- **Binarização:** A saída contém **apenas** os valores 0 ou 255.
- **Comparação estrita:** O critério usa $> T$ (pixels iguais a T tornam-se fundo).
- **Tipo:** O resultado final deve ser inteiro.
- **Observação:** Este EP segue a convenção da OpenCV (`cv2.THRESH_BINARY`): apenas pixels com valor **maior que** T tornam-se brancos (255); pixels com valor **igual a** T permanecem pretos (0).

4.9.1.3 Fundamentação Teórica

Parâmetro	Tipo	Impacto Visual
T pequeno	Inteiro	A maioria dos pixels torna-se branca
T grande	Inteiro	A maioria dos pixels torna-se preta
T bem escolhido	Inteiro	Separa nitidamente objeto e fundo

4.9.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro T .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz binarizada em L linhas e C colunas, valores 0 ou 255 separados por espaço.

4.9.1.5 Exemplos

Entrada	Saída	Observação
2 4 100 0 99 100 180 255 30 120 80	0 0 0 255 255 0 255 0	$T = 100$: apenas pixels com valor maior que 100 tornam-se brancos; por isso, 99 e 100 tornam-se pretos.
1 3 0 0 50 255	0 255 255	
		$T = 0$: apenas pixels com valor estritamente maior que 0 tornam-se brancos.

```
%%writefile EP04_01.py
# Código Python
```

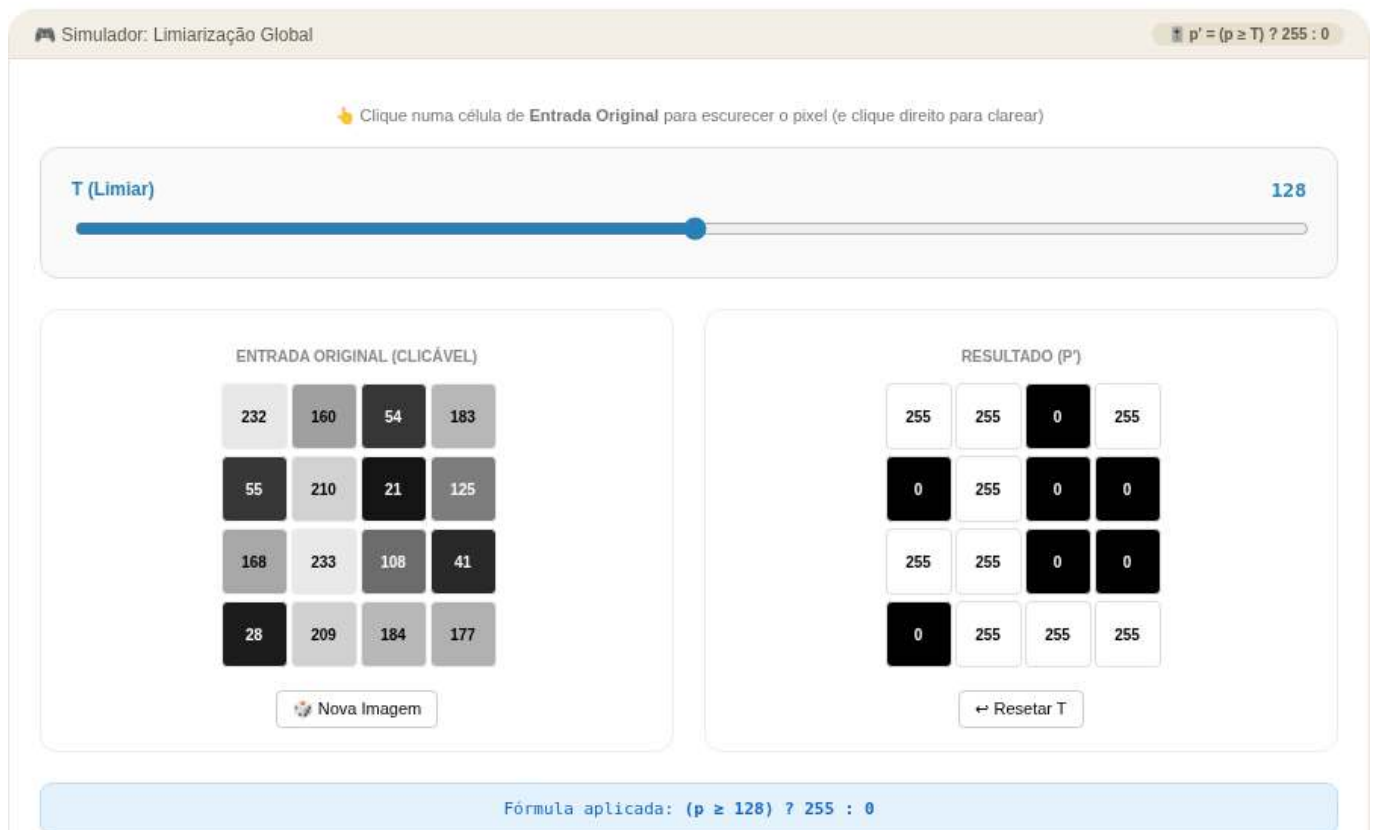


Figura 4.30: Simulador: Limiarização Global por Limiar Fixo

Writing EP04_01.py

TestSuite("EP04_01.py").run()

4.9.2 EP04_02 Limiarização Automática de Otsu

Escolher manualmente o limiar T funciona quando a iluminação é estável, mas em **microscopia digital** e em **inspeção de lâminas de sangue**, cada amostra tem um contraste diferente — um limiar fixo falharia de imagem para imagem. O **método de Otsu** resolve isso encontrando, sozinho, o limiar que **maximiza a separação estatística** entre as duas classes de pixels, tornando a segmentação automática e adaptativa. Ver na Figure 4.31 uma simulação deste EP.

4.9.2.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler os valores inteiros da matriz original linha a linha.
3. **Histograma:** Construir o histograma $h[i]$, $i = 0, \dots, 255$, contando quantos pixels têm valor i .
4. **Busca do limiar:** Para cada candidato T de 1 a 255, calcular a **variância entre classes**:

$$\sigma_B^2(T) = \frac{n_0 \cdot n_1}{N^2} (m_0 - m_1)^2$$

onde n_0, n_1 são as quantidades de pixels com valor $< T$ e $\geq T$, m_0, m_1 são suas médias, e $N = L \times C$.

5. **Escolha:** O limiar ótimo T^* é o que maximiza $\sigma_B^2(T)$ (em caso de empate, manter o **primeiro** encontrado).

6. Aplicação: Binarizar a imagem usando T^* , aplicando:

$$p' = \begin{cases} 255, & \text{se } p > T^* \\ 0, & \text{se } p \leq T^* \end{cases}$$

4.9.2.2 📌 Restrições Computacionais

- **Candidatos válidos:** Ignorar T que deixe $n_0 = 0$ ou $n_1 = 0$ (classe vazia).
- **Empate:** Sempre manter o **primeiro** T que atingiu o valor máximo de σ_B^2 .
- **Tipo:** T^* e a matriz de saída devem ser inteiros.
- **Convenção OpenCV:** A binarização segue `cv2.THRESH_BINARY`; pixels com valor exatamente igual a T^* tornam-se pretos.

4.9.2.3 🧠 Fundamentação Teórica

Conceito	Significado	Impacto
$\sigma_B^2(T)$ alta	Classes bem separadas em T	T é um bom candidato a limiar
Histograma bimodal	Dois “morros” distintos	Otsu encontra o vale entre eles
Histograma unimodal	Um único “morro”	Otsu ainda escolhe <i>algum</i> T , mas a segmentação é pouco confiável

4.9.2.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz binarizada em L linhas e C colunas, valores 0 ou 255.

4.9.2.5 📌 Exemplos

Entrada	Saída	Observação
4	0 0 0 255	Histograma bimodal claro: 12 e 200
4	0 0 255 255	
12 12 12 200	0 255 255 255	
12 12 200 200	255 255 255 255	
12 200 200 200		Apenas dois valores: T^* fica no maior
200 200 200 200		
1	0 250	
2		
10 250		

```
%%writefile EP04_02.py
# Código Python
```

Writing EP04_02.py

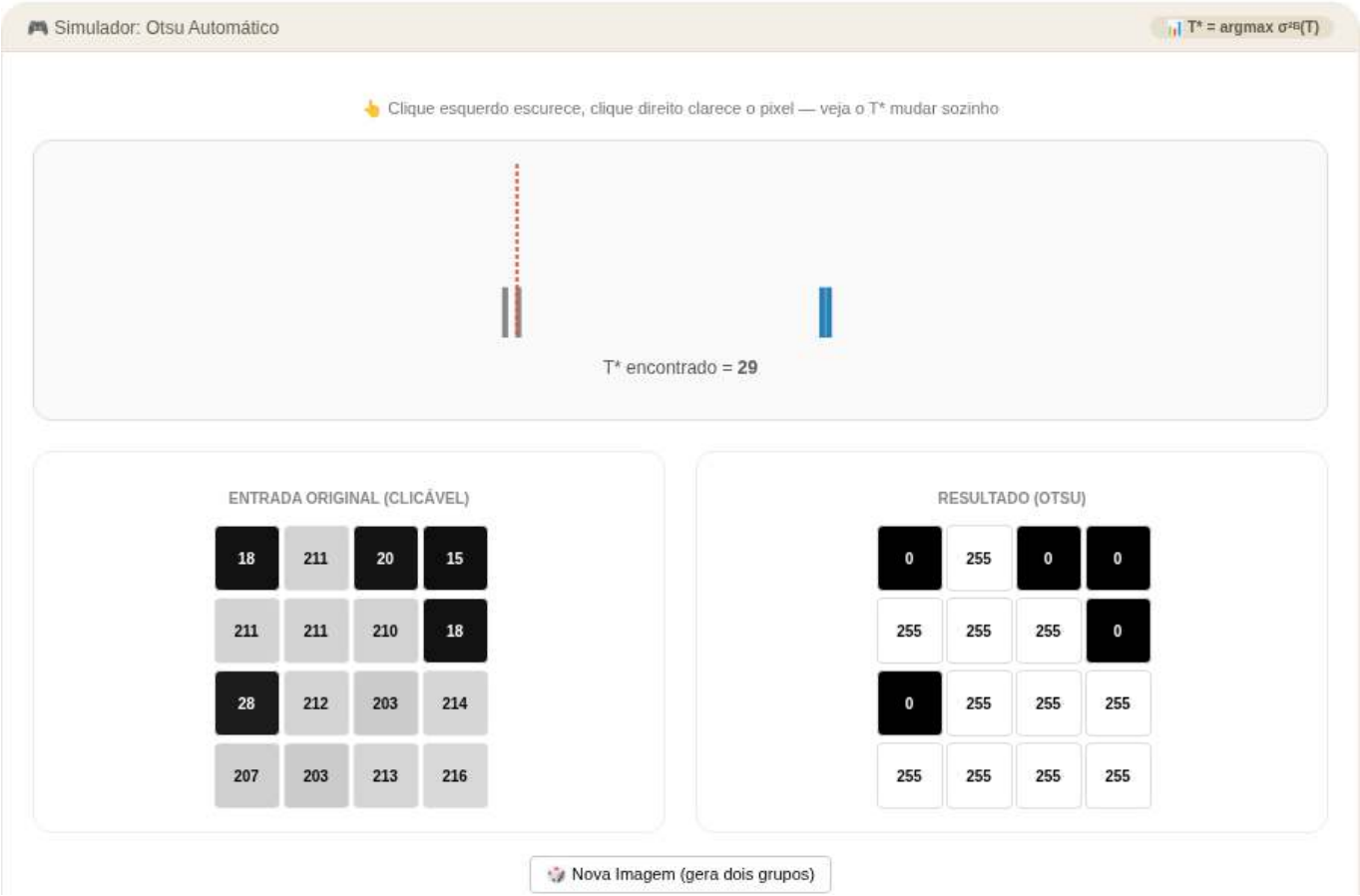


Figura 4.31: Simulador: Limiarização Automática de Otsu

```
TestSuite("EP04_02.py").run()
```

4.9.3 EP04_03 🌱 Dilatação Binária Plana (mm.dil0)

Em **microscopia de partículas** e em **OCR de placas de carro desgastadas**, traços finos ou descontínuos precisam ser “engrossados” para que o reconhecimento funcione. A **dilatação morfológica** faz exatamente isso: expande regiões claras usando um elemento estruturante B — a mesma operação implementada em `morph.py` como `mm.dil0(f, B)`, usada quando B é **plano** (sem pesos, só 0/1). Ver na Figure 4.32 uma simulação deste EP.

4.9.3.1 📋 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz f (a imagem original), linha a linha.
5. **Reflexão:** Construir B_{ref} , a versão de B refletida em 180° (linhas e colunas invertidas) — exatamente como faz `mm.dil0` internamente.
6. **Vizinhança sem padding:** Para cada pixel (y, x) , percorrer as posições (by, bx) de B_{ref} centradas em (y, x) , usando o deslocamento

$$v_y = y + by + o_y, \quad v_x = x + bx + o_x, \quad o_y = -\frac{L_B}{2} + 0,5, \quad o_x = -\frac{C_B}{2} + 0,5$$

Descartar todo (v_y, v_x) fora de $[0, L) \times [0, C)$ — **não preencher com zeros**.

7. **Mapeamento:** Calcular cada pixel de saída como o **máximo** entre $f(y, x)$ e todos os $f(v_y, v_x)$ válidos cuja posição correspondente em B_{ref} vale 1:

$$g(y, x) = \max \left(f(y, x), \max_{\substack{(v_y, v_x) \text{ válido} \\ B_{ref}(by, bx)=1}} f(v_y, v_x) \right)$$

8. **Saída:** Exibir a matriz g com dimensões $L \times C$.

4.9.3.2 🚫 Restrições Computacionais

- **Sem padding:** Jamais inventar vizinhos fora da imagem; usar apenas os que existem de fato.
- **Reflexão obrigatória:** B deve ser refletido antes de aplicado (é o que diferencia `mm.dil0` de uma simples busca por máximo).
- **Robustez de borda:** Se nenhuma posição válida de $B_{ref} = 1$ cair dentro do domínio para um dado pixel, ele **mantém seu valor original**.

4.9.3.3 🧠 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Dilatação	$g \geq f$ sempre (extensiva)	Regiões claras crescem, buracos escuros encolhem
B maior	Vizinhança mais ampla	Crescimento mais agressivo
Reflexão de B	$B_{ref}(y, x) = B(-y, -x)$	Garante a definição formal de Minkowski da dilatação

4.9.3.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Matriz g em L linhas e C colunas, valores inteiros separados por espaço.

4.9.3.5 Exemplos

Entrada	Saída	Observação
3	0 9 0	B em cruz simétrico: ponto isolado se expande em cruz
3	9 9 9	
3	0 9 0	
3		
0 1 0		
1 1 1		
0 1 0		
0 0 0		
0 9 0		
0 0 0		
1	200 200 200 80	B horizontal: cada pixel “puxa” o máximo dos vizinhos da linha
4		
1		
3		
1 1 1		
10 200 5 80		

```
%%writefile EP04_03.py
# Código Python
```

```
Writing EP04_03.py
```

```
TestSuite("EP04_03.py").run()
```

4.9.4 EP04_04 Erosão Binária Plana (mm.ero0)

Se a dilatação engrossa, a **erosão** afina. Em **sistemas de contagem de células**, ela é usada para **separar células que se tocam**: ao “comer” as bordas de cada região, conexões finas entre objetos desaparecem antes mesmo de qualquer contagem ser feita. Em **morph.py**, essa é a operação **mm.ero0(f, B)** — a **dual** exata da dilatação, e a única das duas que **não** reflete o elemento estruturante. Ver na Figure 4.33 uma simulação deste EP.

4.9.4.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz f (a imagem original), linha a linha.

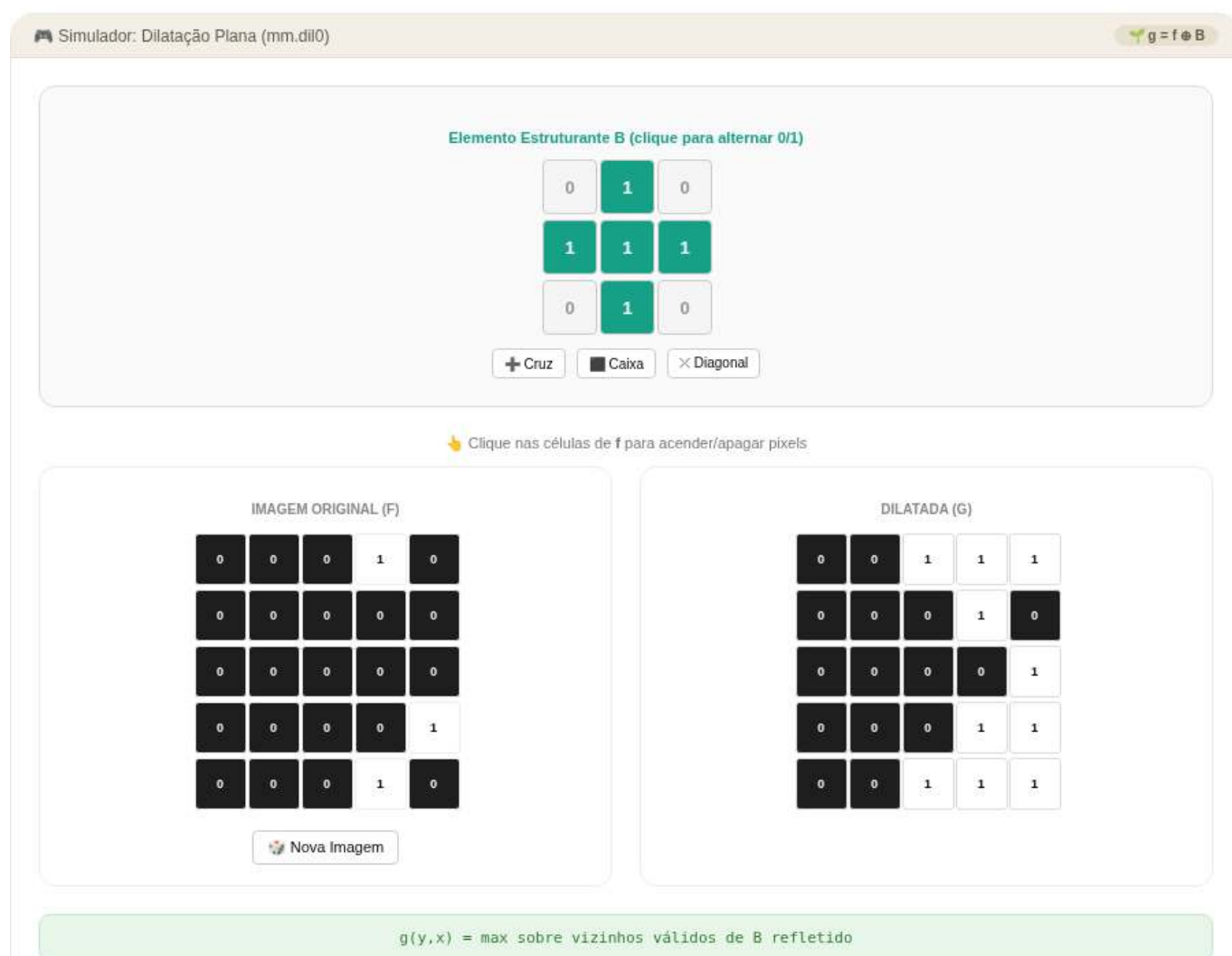


Figura 4.32: Simulador: Dilatação Binária Plana (mm.dil0)

5. **Vizinhança sem padding (sem reflexão!):** Para cada pixel (y, x) , percorrer as posições (by, bx) de B na ordem original (sem refletir), usando o mesmo deslocamento do EP04_03:

$$v_y = y + by + o_y, \quad v_x = x + bx + o_x, \quad o_y = -\frac{L_B}{2} + 0,5, \quad o_x = -\frac{C_B}{2} + 0,5$$

Descartar todo (v_y, v_x) fora de $[0, L) \times [0, C)$. 6. **Mapeamento:** Calcular cada pixel de saída como o **mínimo** entre $f(y, x)$ e todos os $f(v_y, v_x)$ válidos cuja posição correspondente em B vale 1:

$$g(y, x) = \min \left(f(y, x), \min_{\substack{(v_y, v_x) \text{ válido} \\ B(by, bx)=1}} f(v_y, v_x) \right)$$

7. **Saída:** Exibir a matriz g com dimensões $L \times C$.

4.9.4.2 📌 Restrições Computacionais

- **Sem reflexão:** Diferente da dilatação, B é usado **exatamente como lido** — refletir aqui seria um erro conceitual grave.
- **Sem padding:** Vizinhos fora da imagem são simplesmente ignorados, nunca tratados como 0.
- **Robustez de borda:** Se nenhuma posição válida de $B = 1$ cair dentro do domínio, o pixel mantém seu valor original.

4.9.4.3 🧠 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Erosão	$g \leq f$ sempre (anti-extensiva)	Regiões claras encolhem, ruído pontual desaparece
Dualidade	$\text{ero}(f, B) = -\text{dil}(-f, B_{ref})$	Erosão e dilatação são “espelhos” matemáticos
B maior	Erosão mais agressiva	Objetos finos somem completamente

4.9.4.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Matriz g em L linhas e C colunas, valores inteiros separados por espaço.

4.9.4.5 📌 Exemplos

Entrada	Saída	Observação
3	9 0 9	O “buraco” central (0) se propaga em cruz
3	0 0 0	
3	9 0 9	
3		
0 1 0		
1 1 1		
0 1 0		
9 9 9		
9 0 9		
9 9 9		
1	10 5 5 80	B horizontal: cada pixel “puxa” o mínimo dos vizinhos da linha
4		
1		
3		
1 1 1		
10 200 5 80		

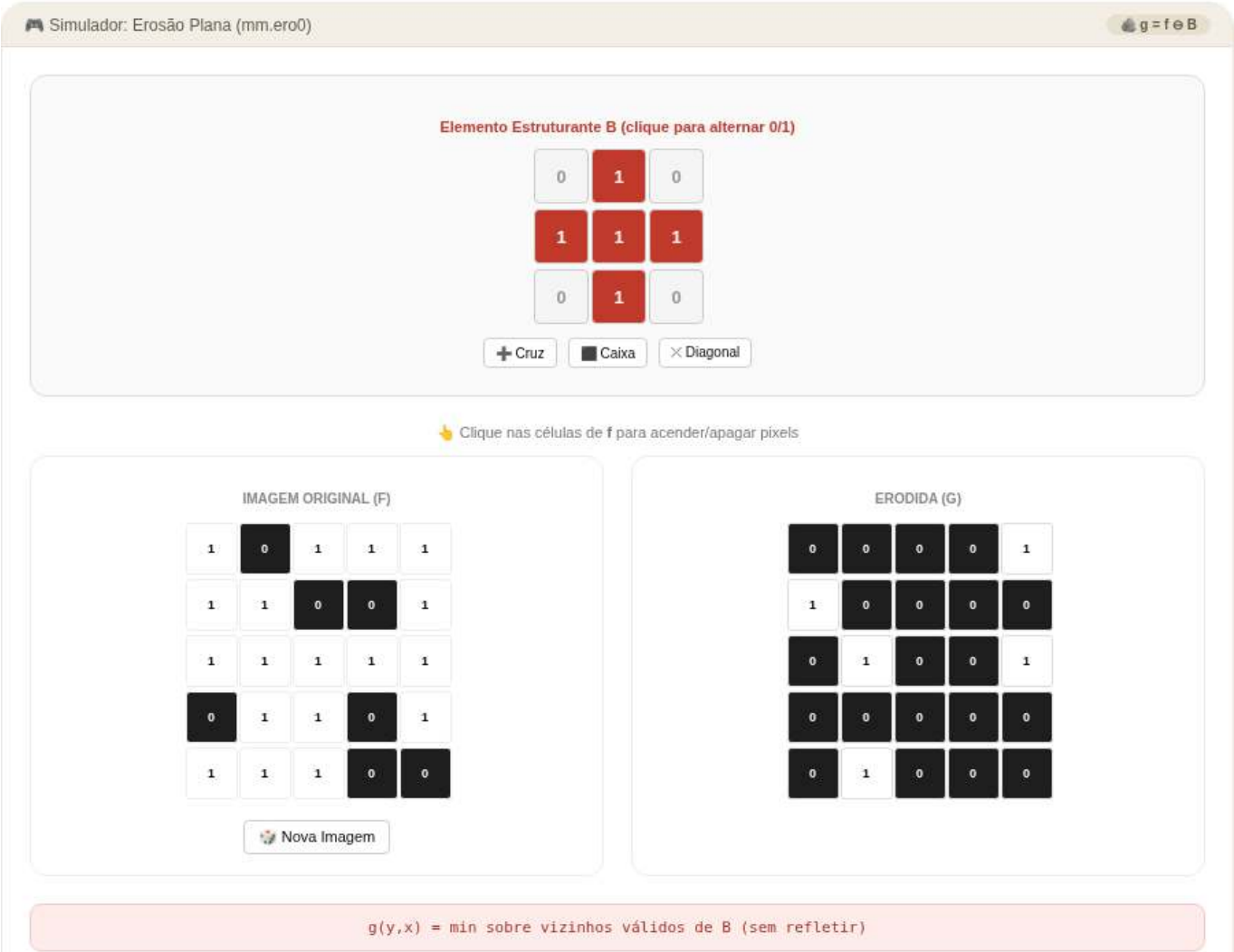


Figura 4.33: Simulador: Erosão Binária Plana (mm.ero0)

```
%%writefile EP04_04.py
# Código Python
```

```
Writing EP04_04.py
```

```
TestSuite("EP04_04.py").run()
```

4.9.5 EP04_05 Abertura Morfológica (Remoção de Ruído)

Imagens capturadas por **sensores de baixo custo**, como os de drones agrícolas, costumam vir salpicadas de pequenos pontos de ruído — pixels isolados que não representam nada real. Aplicar erosão seguida de dilatação com o **mesmo** elemento estruturante produz a **abertura**: ela “limpa” pontos e protuberâncias finas, mas devolve ao objeto principal praticamente seu tamanho original. É a combinação clássica usada em **pré-processamento de imagens de satélite** antes de qualquer contagem de área plantada. Ver na Figure 4.34 uma simulação deste EP.

4.9.5.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Erosão:** Calcular $e = f \ominus B$, usando exatamente o algoritmo do EP04_04 (sem refletir B , sem padding).
6. **Dilatação:** Calcular $g = e \oplus B$, usando exatamente o algoritmo do EP04_03 (refletindo B , sem padding) — mas agora aplicado sobre e , não sobre f .
7. **Saída:** Exibir a matriz resultante g (a **abertura** de f por B) com dimensões $L \times C$.

4.9.5.2 Restrições Computacionais

- **Ordem fixa:** É **sempre** erosão primeiro, depois dilatação — a ordem inversa define outro operador (fechamento, do próximo EP).
- **Mesmo B :** O elemento estruturante usado na erosão e na dilatação deve ser idêntico.
- **Sem padding em nenhuma das duas etapas.**

4.9.5.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Anti-extensividade	$g \subseteq f$ sempre	A abertura nunca cria pixel novo, só remove
Idempotência	$\text{abertura}(\text{abertura}(f)) = \text{abertura}(f)$	Aplicar de novo não muda mais nada
Pontos isolados	Menores que B	São completamente eliminados
Núcleo do objeto	Maior que B	É recuperado quase intacto pela dilatação final

4.9.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .

- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros (0 ou 1) da matriz f .

Saída:

- Matriz resultante em L linhas e C colunas, valores 0 ou 1.

4.9.5.5 Exemplos

Entrada	Saída	Observação
7	0 0 0 0 0 0	Pontos isolados e a protuberância fina desaparecem; o quadrado central sobrevive
7	0 0 0 0 0 0	
3	0 0 1 1 1 0 0	
3	0 0 1 1 1 0 0	
1 1 1	0 0 1 1 1 0 0	
1 1 1	0 0 0 0 0 0 0	
1 1 1	0 0 0 0 0 0 0	
0 0 0 0 0 0 0		
0 1 0 0 0 1 0		
0 0 1 1 1 0 0		
0 0 1 1 1 0 0		
0 0 1 1 1 1 0		
0 0 0 0 0 0 0		
0 1 0 0 0 0 1		

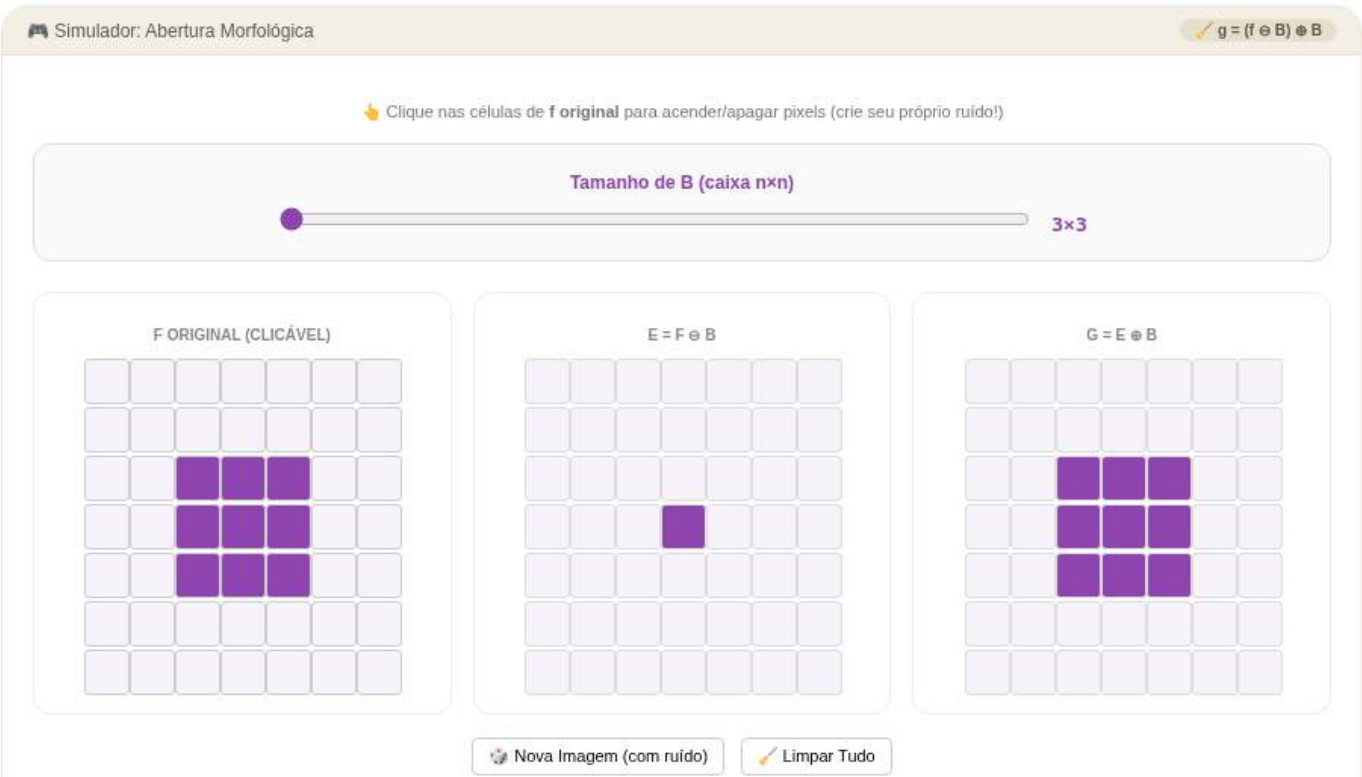


Figura 4.34: Simulador: Abertura Morfológica (erosão + dilatação)

```
%%writefile EP04_05.py
# Código Python
```

```
Writing EP04_05.py
```

```
TestSuite("EP04_05.py").run()
```

4.9.6 EP04_06 ✨ Fechamento Morfológico (Preenchimento de Falhas)

Em **digitalização de impressões digitais**, sulcos da pele às vezes ficam interrompidos por sujeira ou ressecamento, criando pequenas falhas na curva contínua que deveria existir. O **fechamento** — dilatação seguida de erosão com o mesmo elemento estruturante — é o operador dual da abertura: ele **preenche buracos pequenos e reentrâncias estreitas**, sem alterar significativamente o contorno externo do objeto. É a etapa padrão antes de extrair o esqueleto de uma impressão digital. Ver na Figure 4.35 uma simulação deste EP.

4.9.6.1 📋 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Dilatação:** Calcular $d = f \oplus B$, usando exatamente o algoritmo do EP04_03 (refletindo B , sem padding).
6. **Erosão:** Calcular $g = d \ominus B$, usando exatamente o algoritmo do EP04_04 (sem refletir B , sem padding) — agora aplicado sobre d , não sobre f .
7. **Saída:** Exibir a matriz resultante g (o **fechamento** de f por B) com dimensões $L \times C$.

4.9.6.2 📌 Restrições Computacionais

- **Ordem fixa:** É sempre dilatação primeiro, depois erosão — a ordem inversa é a abertura do EP04_05.
- **Mesmo B :** O elemento estruturante usado na dilatação e na erosão deve ser idêntico.
- **Sem padding em nenhuma das duas etapas.**

4.9.6.3 🧠 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Extensividade	$g \supseteq f$ sempre	O fechamento nunca remove pixel, só adiciona
Idempotência	$\text{fecha}(\text{fecha}(f)) = \text{fecha}(f)$	Aplicar de novo não muda mais nada
Buracos pequenos	Menores que B	São completamente preenchidos
Dualidade	$\text{fecha}(f) = \overline{\text{abre}(\overline{f})}$	É a abertura aplicada ao “negativo” da imagem

4.9.6.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros (0 ou 1) da matriz f .

Saída:

- Matriz resultante em L linhas e C colunas, valores 0 ou 1.

4.9.6.5 Exemplos

Entrada	Saída	Observação
8	0 0 1 1 1 1 0 0	Os dois buracos internos não-adjacentes são totalmente preenchidos
8	0 0 1 1 1 1 0 0	
3	0 0 1 1 1 1 0 0	
3	0 0 1 1 1 1 0 0	
1 1 1	0 0 1 1 1 1 0 0	
1 1 1	0 0 1 1 1 1 0 0	
1 1 1	0 0 1 1 1 1 0 0	
0 0 0 0 0 0 0 0	0 0 1 1 1 1 0 0	
0 0 1 1 1 1 0 0		
0 0 1 1 1 1 0 0		
0 0 1 0 1 1 0 0		
0 0 1 1 0 1 0 0		
0 0 1 1 1 1 0 0		
0 0 1 1 1 1 0 0		
0 0 0 0 0 0 0 0		

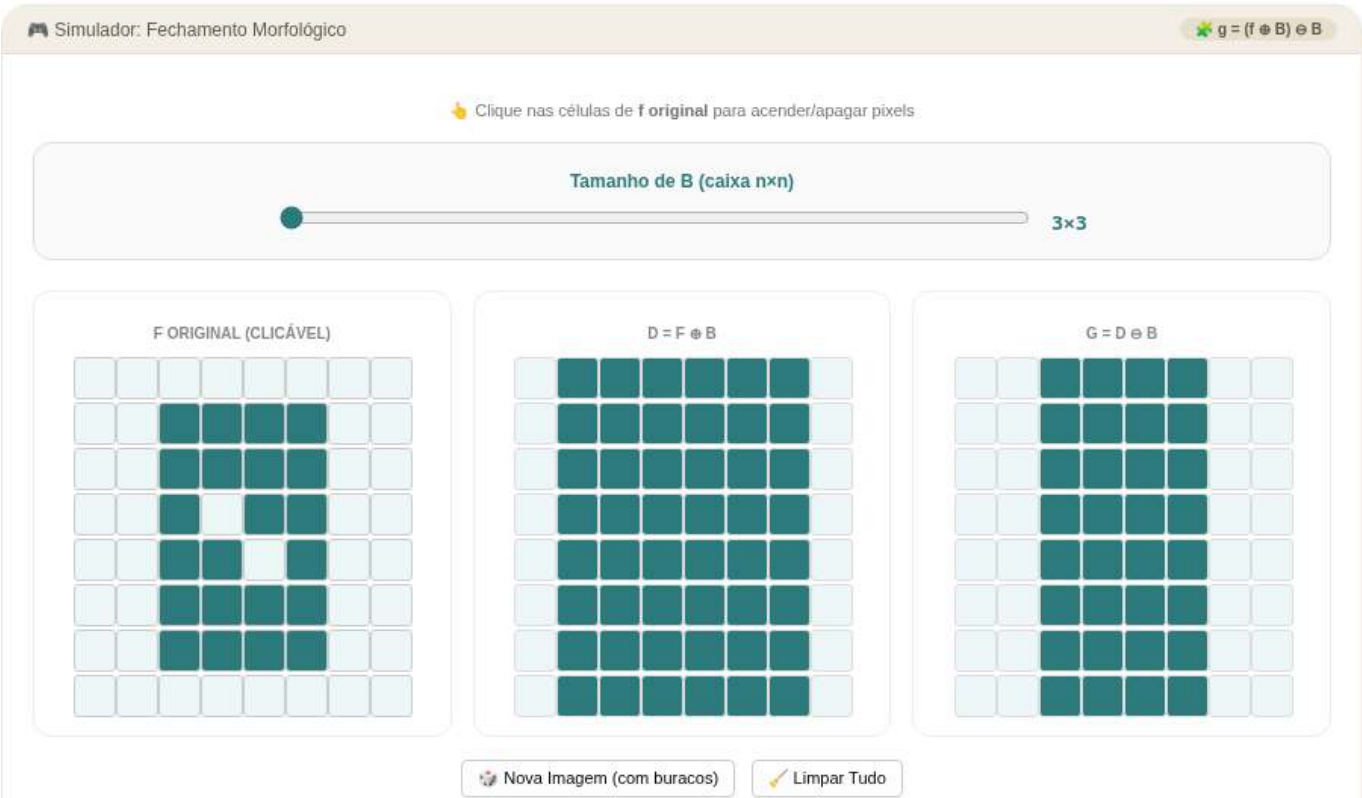


Figura 4.35: Simulador: Fechamento Morfológico (dilatação + erosão)

```
%%writefile EP04_06.py
# Código Python

Writing EP04_06.py
```

```
TestSuite("EP04_06.py").run()
```

4.9.7 EP04_07 Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)

Até agora, o elemento estruturante só dizia “este vizinho conta” ou “não conta” — mas em **modelos digitais de elevação** (usados em SIG e em planejamento de drenagem urbana), cada vizinho deveria ter um **peso diferente** dependendo da distância ou da direção do relevo. As versões **ponderadas** da dilatação e da erosão, implementadas em `morph.py` como `mm.dil1(f, b)` e `mm.ero1(f, b)`, somam (ou subtraem) o peso de cada vizinho antes de tomar o máximo (ou mínimo) — generalizando tudo o que foi feito nos EPs anteriores. Ver na Figure 4.36 uma simulação deste EP.

4.9.7.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de b :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante ponderado.
3. **Pesos:** Ler a matriz b de pesos **inteiros** (podem ser negativos, zero ou positivos), linha a linha.
4. **Dados:** Ler a matriz f (a imagem original), linha a linha.
5. **Vizinhança sem padding:** Para cada pixel (y, x) , percorrer **todas** as posições (by, bx) de b (não apenas onde valeria 1 — aqui **todo** peso participa), usando o mesmo deslocamento dos EPs anteriores:

$$v_y = y + by + o_y, \quad v_x = x + bx + o_x, \quad o_y = -\frac{L_B}{2} + 0,5, \quad o_x = -\frac{C_B}{2} + 0,5$$

Descartar todo (v_y, v_x) fora de $[0, L) \times [0, C)$.

6. **Dilatação ponderada:** Calcular

$$g_{dil}(y, x) = \max \left(f(y, x), \max_{(v_y, v_x) \text{ válido}} (f(v_y, v_x) + b(by, bx)) \right)$$

7. **Erosão ponderada:** Calcular, **usando o mesmo b e sem refletir**:

$$g_{ero}(y, x) = \min \left(f(y, x), \min_{(v_y, v_x) \text{ válido}} (f(v_y, v_x) - b(by, bx)) \right)$$

8. **Saída:** Exibir **primeiro** a matriz g_{dil} completa, e **depois** a matriz g_{ero} completa.

4.9.7.2 Restrições Computacionais

- **Nenhuma das duas reflete b** — a versão ponderada não usa reflexão, mesmo na dilatação (diferente de `mm.dil0`).
- **Todos os pesos participam:** Não existe aqui o filtro “ $B = 1$ ”; mesmo peso 0 entra na conta.
- **Sem padding:** vizinhos fora da imagem são ignorados, nunca virtualmente preenchidos.
- **Tipo:** A saída pode conter valores negativos ou maiores que 255 — **não** há *clipping* neste EP.
- **Dica:** Para remover mensagens de overflow ao ultrapassar limites do tipo `uint8`, incluir no início do código:

```
import warnings
warnings.filterwarnings("ignore")
```

4.9.7.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Peso positivo	“Puxa” o valor do vizinho para cima na dilatação	Simula relevo que sobe naquela direção
Peso negativo	Reduz a contribuição do vizinho	Simula distância ou atenuação direcional

Conceito	Significado	Impacto Visual
Dualidade ponderada	$\text{ero1}(f, b) = -\text{dil1}(-f, b)$	A simetria entre as duas operações se mantém mesmo com pesos

4.9.7.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (podem ser negativos) da matriz b .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Primeiro a matriz g_{dil} em L linhas e C colunas.
- Em seguida a matriz g_{ero} em L linhas e C colunas.

4.9.7.5 Exemplos

Entrada	Saída	Observação
3	50 60 61	Peso central 2 acelera o crescimento na dilatação e o encolhimento na erosão
3	80 90 91	
3	81 91 92	
3	8 9 19	
0 1 0	9 10 20	
1 2 1	39 40 50	
0 1 0		
10 20 30		
40 50 60		
70 80 90		

```
import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite
```

```
%%writefile EP04_07.py
# Código Python
```

Writing EP04_07.py

```
TestSuite("EP04_07.py").run()
```

4.9.8 EP04_08 Gradiente Morfológico, Top-hat e Black-hat

Em **inspeção automática de placas de circuito**, três perguntas aparecem o tempo todo: onde estão as **bordas** dos componentes? Quais **detalhes claros e pequenos** (como pontos de solda) se destacam do fundo? Quais **reentrâncias escuras** (como fissuras) o fundo esconde? Um único par erosão/dilatação responde às três: o **gradiente morfológico** evidencia contornos, o **top-hat** revela picos estreitos, e o

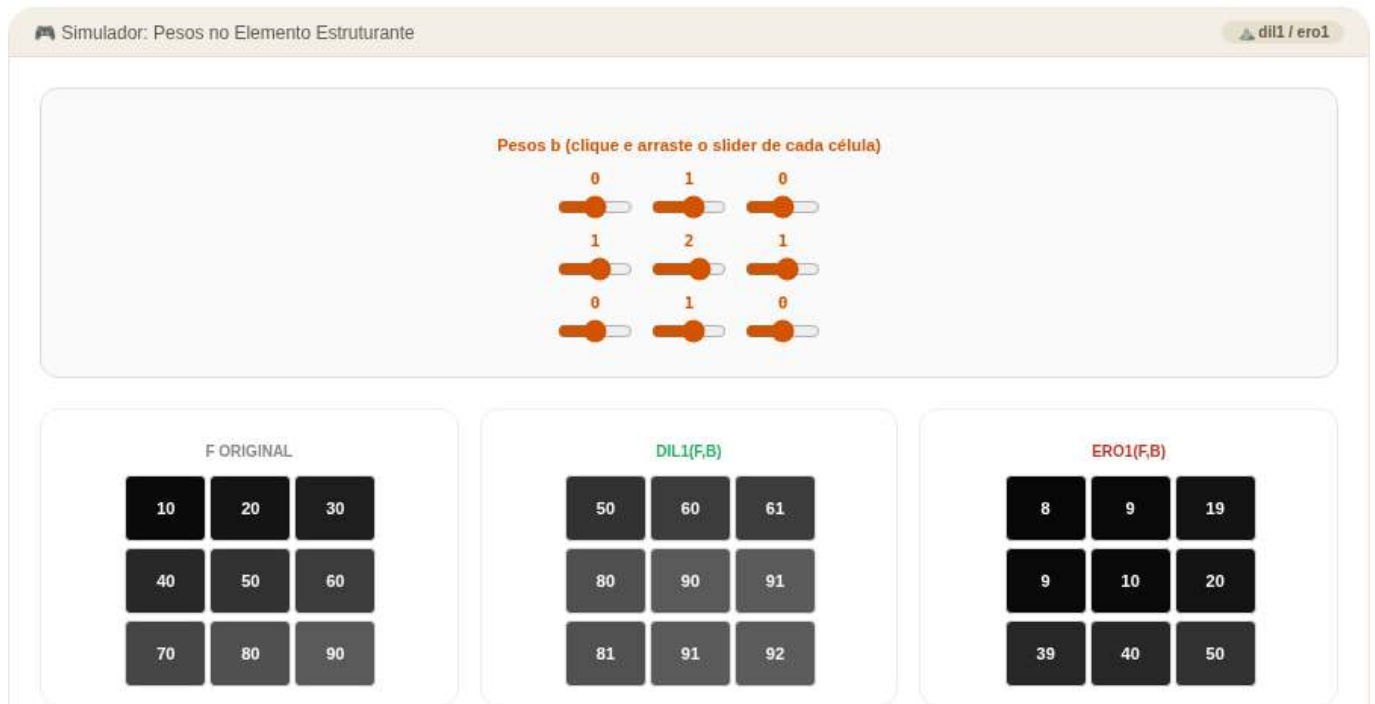


Figura 4.36: Simulador: Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)

black-hat revela vales estreitos — três ferramentas, uma só vizinhança. Ver na Figure 4.37 uma simulação deste EP.

4.9.8.1 📋 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz f (a imagem original, em tons de cinza), linha a linha.
5. **Operadores de base:** Calcular, exatamente como nos EPs 04_03 a 04_06:
 - $d = f \oplus B$ (dilatação),
 - $e = f \ominus B$ (erosão),
 - abertura = $e \oplus B$,
 - fechamento = $d \ominus B$.
6. **Gradiente morfológico:** $\text{grad}(y, x) = d(y, x) - e(y, x)$.
7. **Top-hat:** $\text{tophat}(y, x) = f(y, x) - \text{abertura}(y, x)$.
8. **Black-hat:** $\text{blackhat}(y, x) = \text{fechamento}(y, x) - f(y, x)$.
9. **Saída:** Exibir, **nesta ordem**, as três matrizes completas: gradiente, top-hat, black-hat.

4.9.8.2 📌 Restrições Computacionais

- **Sem padding em nenhuma etapa intermediária** — dilatação, erosão, abertura e fechamento seguem as mesmas regras de vizinhança dos EPs anteriores.
- **Não há clipping:** as três saídas podem conter qualquer valor inteiro (o gradiente é sempre ≥ 0 , mas top-hat e black-hat também).
- **Reaproveitamento:** d e e devem ser calculados **uma única vez** e reaproveitados para montar abertura, fechamento e gradiente.

4.9.8.3 🧠 Fundamentação Teórica

Operador	Fórmula	O que revela
Gradiente	$d - e$	Bordas: zero em regiões planas, alto nas transições
Top-hat	$f - \text{abertura}(f)$	Elementos claros e finos , menores que B
Black-hat	$\text{fechamento}(f) - f$	Elementos escuros e finos , menores que B

4.9.8.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Matriz gradiente em L linhas e C colunas.
- Matriz top-hat em L linhas e C colunas.
- Matriz black-hat em L linhas e C colunas.

4.9.8.5 Exemplos

Entrada	Saída	Observação
9	(gradiente: halo	Pico isolado vira top-hat; vale isolado vira
9	$3 \times 3 = 70$ em torno de	black-hat; ambos aparecem no gradiente
3	(2, 2) e halo $3 \times 3 = 8$ em	
3	torno de (6, 6), resto 0)	
1 1 1	(top-hat: único 70 em	
1 1 1	(2, 2), resto 0)	
1 1 1	(black-hat: único 8 em	
10 10 10 10 10 10 10 10 10	(6, 6), resto 0)	
10 10 10 10 10 10 10 10 10		
10 10 80 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 2 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		

```
%%writefile EP04_08.py
# Código Python
```

```
Writing EP04_08.py
```

```
TestSuite("EP04_08.py").run()
```

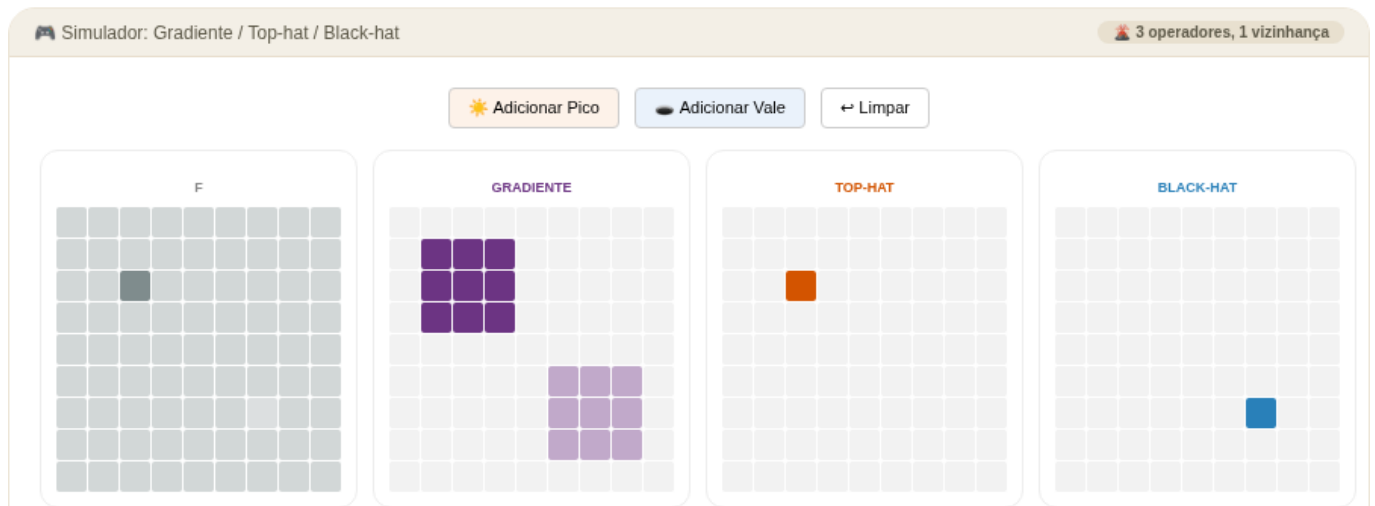


Figura 4.37: Simulador: Gradiente Morfológico, Top-hat e Black-hat

4.9.9 EP04_09 Transformada de Distância e o “Miolo” do Objeto

Em **robótica móvel**, ao planejar uma rota dentro de um corredor, o robô quer saber não apenas *onde* existe espaço livre, mas também **quão longe** cada ponto livre está da parede mais próxima. Os caminhos mais seguros tendem a passar pelo “miolo” do corredor, longe dos obstáculos.

A **transformada de distância morfológica** atribui a cada pixel um valor que representa sua distância até a borda mais próxima, segundo a métrica definida pelo elemento estruturante. Pixels próximos à borda recebem valores baixos, enquanto pixels mais internos recebem valores maiores. O pixel de valor máximo corresponde à região mais protegida do objeto, frequentemente associada ao seu centro morfológico.

Ver na Figure 4.38 uma simulação deste EP.

4.9.9.1 📋 Diretrizes de Implementação

1. **Dimensões da imagem:** ler os inteiros L (linhas) e C (colunas) da imagem f .
2. **Dimensões de B :** ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** ler a matriz b , contendo valor 0 no centro e valores negativos nas demais posições.
4. **Imagem:** ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Preparação:** multiplicar a imagem por $L \times C$, garantindo que os pixels internos tenham valor inicial suficientemente alto para a propagação das distâncias.
6. **Transformada de distância:** calcular a matriz de distâncias utilizando o método `mm.dist1(f,b)`.
7. **Saída:** exibir a matriz resultante da transformada de distância.

4.9.9.2 📌 Restrições Computacionais

- Utilizar a implementação de erosão ponderada fornecida pela biblioteca.
- O elemento estruturante pode conter valores negativos arbitrários.
- A transformada deve ser obtida pela aplicação iterativa de erosões ponderadas até atingir um ponto fixo.

⚠ **Nota Crucial sobre Leitura de Matrizes:** Como o elemento estruturante pode conter valores inteiros negativos (por exemplo, -1 e -99), **não utilize a função `mm.readImg` para ler a matriz b** . Essa função converte os dados para o tipo `uint8`, provocando *underflow* e corrompendo os valores negativos. Leia as L_B linhas de b manualmente utilizando o tipo padrão `int`. A imagem f pode continuar sendo lida normalmente por `mm.readImg`.

4.9.9.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
$\text{dist}(y, x)$	Distância morfológica até a borda mais próxima segundo a métrica definida por b	Pixels mais internos recebem valores maiores
Valor máximo	Pixel mais distante da borda	Aproxima o centro morfológico do objeto
Elemento estruturante ponderado	Define os custos de deslocamento entre pixels vizinhos	Determina a métrica de distância utilizada
Objetos finos	Regiões estreitas do objeto	Produzem valores baixos de distância

4.9.9.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro L .
- Linha 2: inteiro C .
- Linha 3: inteiro L_B .
- Linha 4: inteiro C_B .
- Próximas L_B linhas: elementos inteiros da matriz b .
- Próximas L linhas: elementos binários (0 ou 1) da matriz f .

 **Nota de implementação:** Os elementos da matriz f (0 ou 1) devem ser multiplicados por **255** para gerar uma imagem binária adequada (0 e 255) antes de aplicar a Transformada de Distância (TD).

Saída:

- Matriz da transformada de distância em L linhas e C colunas.

4.9.9.5 Exemplo

Entrada	Saída	Observação
5	0 0 0 0 0 0 0 0	Resultado da transformada de distância.
9	0 1 1 1 1 1 1 0	
3	0 1 2 2 2 2 2 0	
3	0 1 1 1 1 1 1 0	
-99 -1 -99	0 0 0 0 0 0 0 0	
-1 0 -1		
-99 -1 -99		
0 0 0 0 0 0 0 0		
0 1 1 1 1 1 1 0		
0 1 1 1 1 1 1 0		
0 1 1 1 1 1 1 0		
0 0 0 0 0 0 0 0		

Nota: o valor -99 atua como uma aproximação prática de $-\infty$, impedindo a propagação pelas diagonais. Dessa forma, apenas os vizinhos horizontal e vertical contribuem para a distância, produzindo a distância de Manhattan.

```
%%writefile EP04_09.py
# Código Python
```

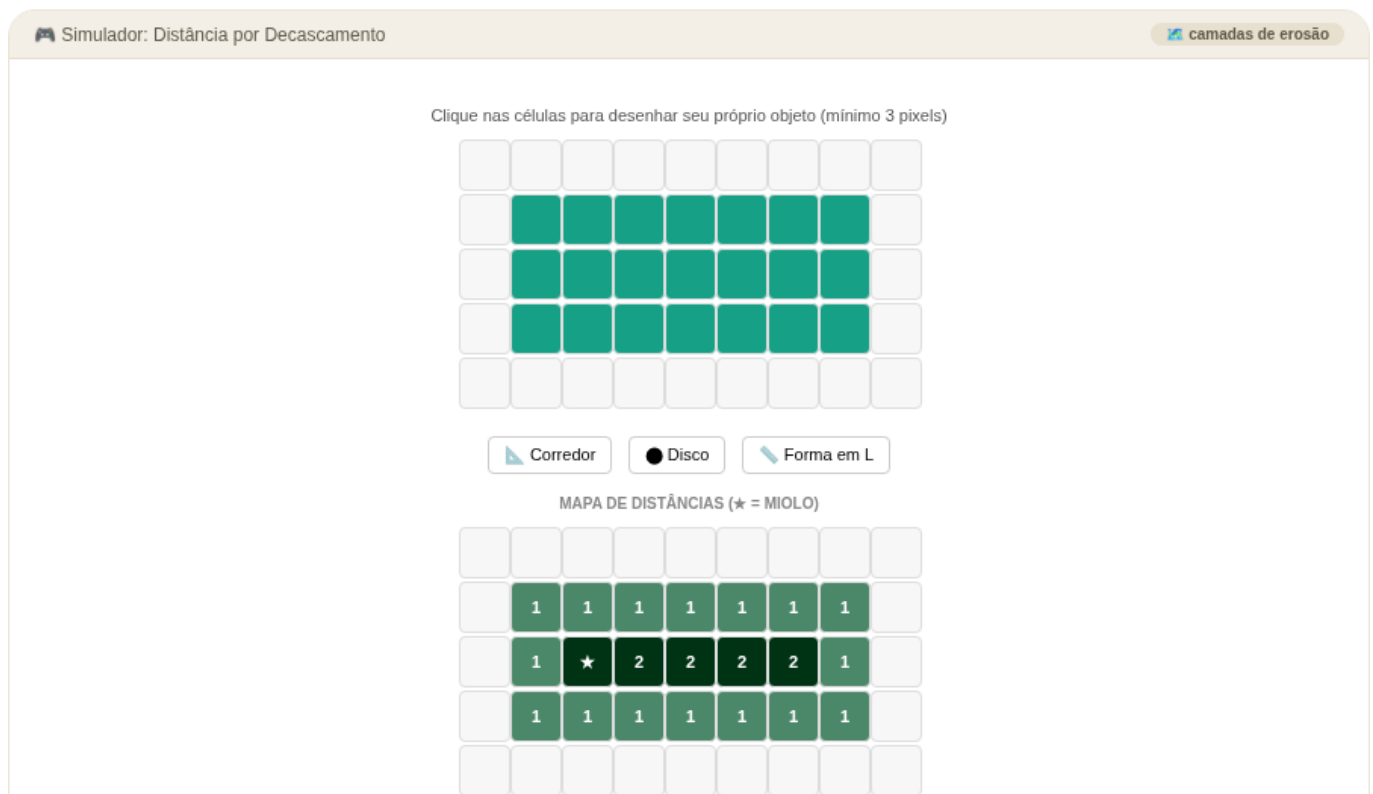



Figura 4.38: Simulador: Transformada de Distância

Writing EP04_09.py

```
TestSuite("EP04_09.py").run()
```

4.9.10 EP04_10 Separação de *Blobs*, Rotulação e Descritores

Em uma **linha de produção de moedas**, é comum que peças encostem umas nas outras na esteira, formando uma única mancha conectada na imagem — uma contagem ingênua erraria o total. A solução clássica combina operações morfológicas e análise de conectividade: primeiro uma **erosão** reduz ou rompe conexões frágeis entre objetos, e depois a **rotulação de componentes conectados** separa cada objeto em uma região distinta. Por fim, **descritores geométricos** (área e caixa delimitadora) resumem cada componente encontrado.

Ver na Figure 4.39 uma simulação deste EP.

4.9.10.1 Diretrizes de Implementação

1. **Dimensões da imagem:** ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** ler a matriz B , contendo valores 0 ou 1, linha a linha.
4. **Dados:** ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Separação:** calcular

$$f_{ero} = f \ominus B$$

usando erosão binária plana (como no EP04_04), eliminando conexões frágeis entre objetos.

6. **Rotulação:** sobre f_{ero} , identificar componentes conectados usando conectividade definida pela vizinhança B . A rotulação deve seguir varredura *raster*: ao encontrar um pixel 1 ainda não rotulado, atribuir um novo rótulo inteiro crescente a partir de 1 e propagar esse rótulo a toda a região conectada.

7. **Descritores:** para cada rótulo k , calcular:

- **Área:** número de pixels pertencentes ao rótulo;
- **Caixa delimitadora:**

$$(y_{min}, x_{min}, y_{max}, x_{max})$$

8. **Saída:** exibir o número total de rótulos e, em seguida, uma linha por rótulo no formato:

$$k, \text{ área}, y_{min}, x_{min}, y_{max}, x_{max}$$

4.9.10.2 Restrições Computacionais

- A erosão deve ser aplicada antes da rotulação.
- A conectividade é fixa e definida pela vizinhança acima.
- O elemento estruturante B não interfere na conectividade da rotulação.
- Sem padding em qualquer etapa.
- A ordem dos rótulos segue a primeira descoberta em varredura *raster*.

4.9.10.3 Fundamentação Teórica

Conceito	Significado	Impacto
Ponte fina	Conexão estreita entre objetos	Pode ser removida pela erosão morfológica
Conectividade	Definida pelo conjunto $\mathcal{N}(y, x)$	Determina quais pixels pertencem ao mesmo componente
Área	Número de pixels por componente	Estimativa direta do tamanho do objeto
Caixa delimitadora	Extensão espacial do rótulo	Resumo geométrico do componente

4.9.10.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro L
- Linha 2: inteiro C
- Linha 3: inteiro L_B
- Linha 4: inteiro C_B
- Próximas L_B linhas: matriz B
- Próximas L linhas: matriz f

Saída:

- Linha 1: número total de rótulos encontrados
- Linhas seguintes:

$$k, \text{ área}, y_{min}, x_{min}, y_{max}, x_{max}$$

```
%%writefile EP04_10.py
# Código Python
```



Figura 4.39: Simulador: Separação de Blobs, Rotulação e Descritores

```
Writing EP04_10.py
```

```
TestSuite("EP04_10.py").run()
```

Capítulo 5

Transformadas e Compressão

 Executar Colab  Abrir si-md2  GitHub

Nos capítulos anteriores, todas as operações foram realizadas **no domínio espacial**: filtros de convolução, morfologia e segmentação atuam diretamente sobre os valores de intensidade dos pixels. Este capítulo apresenta uma perspectiva complementar e poderosa — o **domínio da frequência** — que revela como a imagem é construída a partir de padrões oscilatórios de diferentes escalas.

A ideia central é simples: **qualquer imagem digital discreta pode ser representada como uma soma de senoides bidimensionais ortogonais**, cada uma com frequência, amplitude e fase próprias. Baixas frequências correspondem a variações suaves (fundo, iluminação global); altas frequências correspondem a bordas, texturas e ruído.

Esta decomposição abre três caminhos práticos:

1. **Filtragem espectral** — suprimir ou realçar seletivamente faixas de frequência;
2. **Análise multi-escala com *wavelets*** — capturar estruturas em diferentes resoluções e posições;
3. **Compressão** — eliminar coeficientes imperceptíveis ao olho humano, reduzindo o volume de dados.

5.1 Objetivos

Ao concluir este capítulo, você será capaz de:

- **Interpretar o espectro de Fourier** de uma imagem: distinguir magnitude de fase, localizar baixas e altas frequências, e ler o espectro de forma intuitiva;
- **Aplicar o Teorema da Convolução** para implementar filtragem eficiente via FFT, compreendendo o ganho computacional em relação à convolução direta;
- **Projetar e comparar filtros espectrais** — passa-baixa (Ideal, Gaussiano, Butterworth), passa-alta e notch — com foco nos seus efeitos visuais e nos artefatos que produzem;
- **Compreender wavelets e multirresolução**: interpretar as subbandas LL/LH/HL/HH e relacionar a análise *wavelet* às camadas de CNNs;
- **Entender o pipeline JPEG**: desde a DCT em blocos 8×8 até a quantização perceptual e os artefatos de compressão;
- **Escolher o formato de imagem** adequado (JPEG, PNG, WebP) com base no conteúdo e na aplicação.

5.2 Configuração do Ambiente

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt
import cv2
from scipy import ndimage

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
```

```
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

print(" Ambiente pronto")
```

Ambiente pronto

5.3 Transformada de Fourier Discreta 2D

A análise de Fourier fundamenta-se em um princípio poderoso: qualquer sinal periódico admite representação como soma de senoides com diferentes frequências, amplitudes e fases. Ver exemplo em uma dimensão na Figure 5.1.

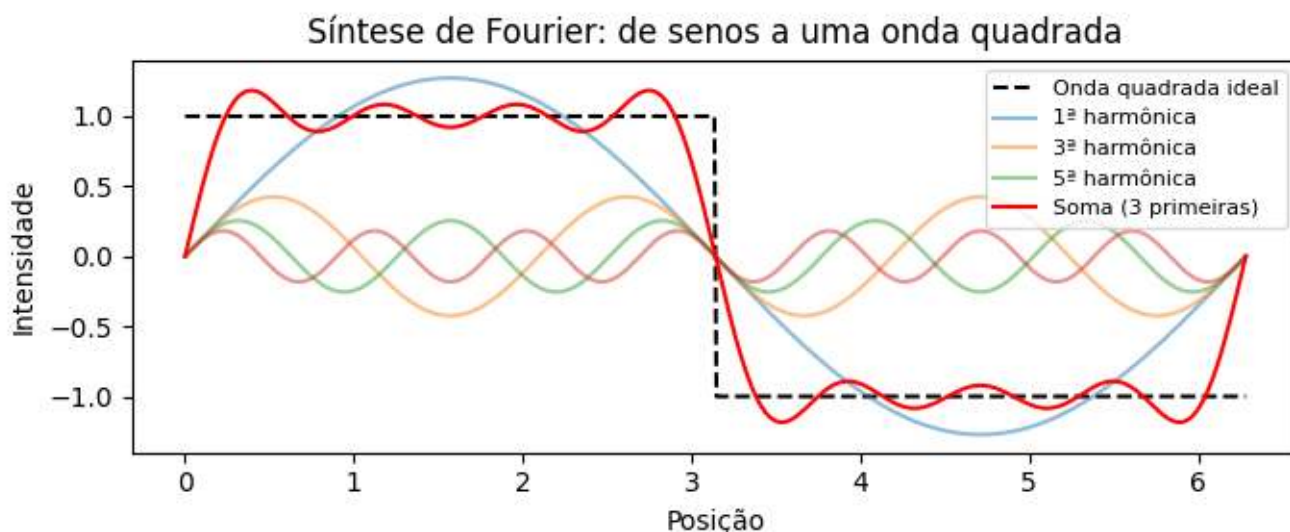


Figura 5.1: Decomposição de Fourier 1D: uma onda quadrada (linha tracejada) é aproximada pela soma das primeiras senoides (linhas coloridas). Quanto mais termos, melhor a aproximação.

Aplicada a imagens digitais, essa ideia transforma uma matriz de valores de intensidade $f(x, y)$ em uma representação no **domínio da frequência**, revelando quais padrões oscilatórios compõem a cena visual.

5.3.1 Fenômeno: o que uma imagem “parece” no domínio da frequência?

Observe o que acontece quando se transforma uma fotografia: ela se transforma em um padrão radial brilhante ao centro, com energia decaindo progressivamente em direção às bordas. Esse padrão é o **espectro de magnitude** da imagem — uma radiografia da distribuição de frequências.

Região do espectro	O que representa	Exemplos visuais
Centro (baixas freq.)	Variações lentas — iluminação, cor de fundo, forma global	Gradientes suaves, blocos uniformes
Meio (médias freq.)	Texturas, padrões repetitivos	Tecidos, tijolos, grama
Bordas (altas freq.)	Variações abruptas — contornos, ruído	Bordas de objetos, grão de imagem

Intuição-chave: suprimir o centro do espectro remove a estrutura global e enfatiza bordas;

suprimir as bordas suaviza a imagem e remove ruído. É precisamente isso que os filtros espectrais fazem — mas com precisão cirúrgica impossível no domínio espacial.

5.3.2 O Experimento da Grade: Construindo a Imagem do Zero

Antes de mergulhar nas fórmulas complexas, vamos fazer o caminho inverso (IDFT). O que acontece se tivermos um espectro completamente vazio (preto) e acendermos **apenas um único ponto** brilhante (um coeficiente de frequência)?

O resultado no espaço real é uma grade perfeitamente senoidal. A posição desse ponto no espectro define a **direção** e a **frequência** (espessura) dessa grade.

```
N_grid = 100
espectro_vazio = np.zeros((N_grid, N_grid), dtype=complex)

# Acendendo um único ponto (frequência) fora do centro
u0, v0 = 10, 5
espectro_vazio[N_grid//2 - v0, N_grid//2 - u0] = 1000

# Retornando para o domínio espacial (IDFT)
onda_2d = np.real(np.fft.ifft2(np.fft.ifftshift(espectro_vazio)))

onda_vis = cv2.normalize(onda_2d, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
espectro_vis = cv2.normalize(np.abs(espectro_vazio), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# Destaque visual do ponto
espectro_color = cv2.cvtColor(espectro_vis, cv2.COLOR_GRAY2BGR)
cv2.circle(espectro_color, (N_grid//2 - u0, N_grid//2 - v0), 2, (0, 0, 255), -1)

mm.show([espectro_color, onda_vis], titles=["Espectro (1 ponto ativo)", "Onda 2D Resultante (IDFT)"], co
```

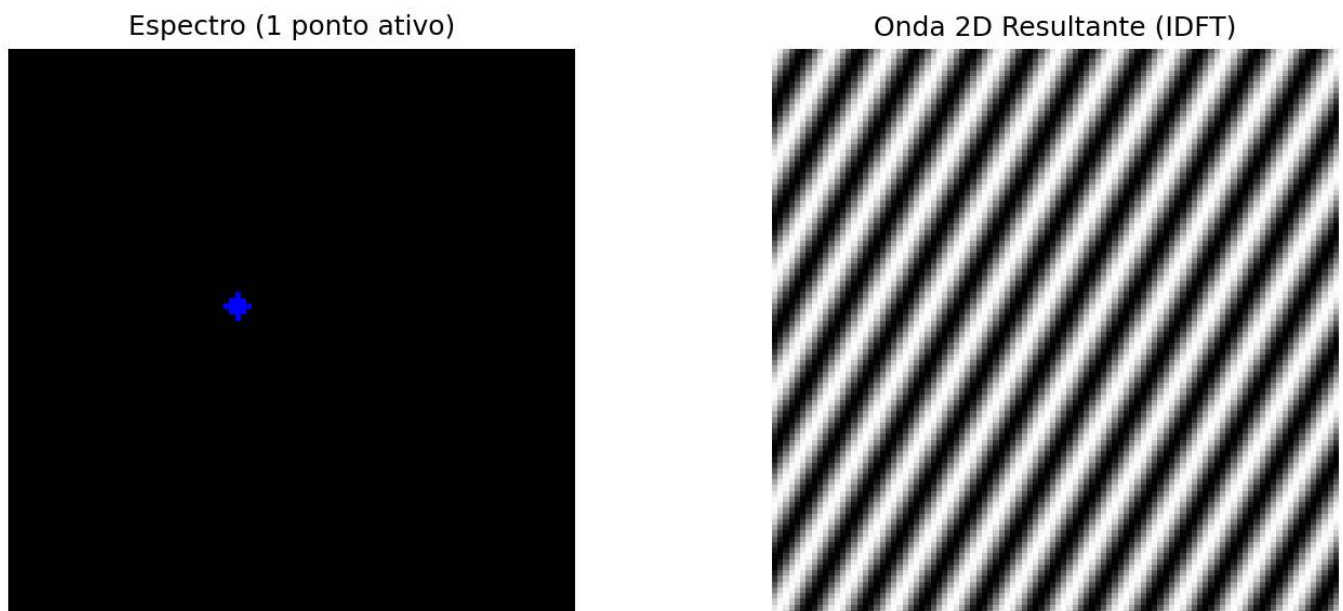


Figura 5.2: Toda frequência no espectro (ponto isolado) corresponde a uma onda senoidal 2D rotacionada no domínio espacial.

5.3.3 Definição Matemática

Dada uma imagem $f(x, y)$ de dimensões $M \times N$, sua **Transformada de Fourier Discreta 2D** (DFT) é definida como:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})} \quad (5.1)$$

onde $u = 0, 1, \dots, M-1$ e $v = 0, 1, \dots, N-1$ são as **frequências discretas** nas direções horizontal e vertical, respectivamente. O exponencial complexo $e^{-j2\pi(ux/M + vy/N)}$ representa uma oscilação bidimensional de frequência espacial normalizada ($u/M, v/N$) em ciclos por pixel.

A **Transformada Inversa de Fourier Discreta 2D** (IDFT) recupera a imagem original a partir de seu espectro:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})} \quad (5.2)$$

i O que é o componente DC?

O coeficiente $F(0, 0)$ — chamado **DC** (*Direct Current*) — corresponde à **soma de todos os pixels**, ou seja, $F(0, 0) = MN \bar{f}$, sendo $\bar{f}$ a média global da imagem. Na grande maioria das imagens naturais, esse coeficiente possui a maior magnitude no espectro — imagens com variações abruptas e periódicas podem concentrar energia em outras frequências. Os demais coeficientes ($u, v \neq 0, 0$) representam variações em torno da média.

Após **fftshift** — deslocamento circular que move o DC para o centro —, o espectro fica intuitivo: centro = baixas frequências, bordas = altas frequências.

5.3.4 Implementação e Visualização



Figura 5.3: Diagrama conceitual do espectro de Fourier 2D centrado.

5.3.5 O que a Magnitude e a Fase carregam?

Uma das demonstrações mais reveladoras em PDI é trocar os espectros de magnitude e fase entre duas imagens distintas e reconstruir cada uma. O resultado mostra que:

- **A fase carrega a estrutura semântica** — contornos, posição dos objetos, geometria da cena;
- **A magnitude carrega o conteúdo energético e textural** — contraste, iluminação geral.

Quando reconstruímos uma imagem com a **magnitude de A** mas a **fase de B**, o resultado se parece com B — mesmo usando a “energia” de A. A fase possui papel predominante na preservação da estrutura geométrica e organização espacial da imagem, enquanto a magnitude controla a distribuição energética, contraste e textura. Ver um exemplo na Figure 5.4.

Analogia auditiva: no áudio, a fase determina a percepção de posição (esquerda/direita). Na imagem, ela determina a posição e forma dos objetos. Inverter a fase de uma música torna-a irreconhecível mesmo que a magnitude (timbre) permaneça intacta.

```
# Experimento: A Importância da Fase
# Carregamento da imagem
url      = "https://upload.wikimedia.org/wikipedia/commons/2/25/GAZI.MD.AHAD_11.jpg"
caminho  = "imagens/coins.jpg"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_color = np.array(img_obj)
img_gray  = mm.gray(img_color)

img_a = cv2.resize(img_gray, (400, 400))

# Criar uma imagem B sintética (padrão geométrico)
img_b = np.zeros((400, 400), dtype=np.uint8)
cv2.rectangle(img_b, (100, 100), (300, 300), 255, -1)
cv2.circle(img_b, (200, 200), 150, 128, 10)

FA = np.fft.fft2(img_a)
FB = np.fft.fft2(img_b)

# Troca de Fase
rec_A_mag_B_fase = np.real(np.fft.ifft2(np.abs(FA) * np.exp(1j * np.angle(FB))))
rec_B_mag_A_fase = np.real(np.fft.ifft2(np.abs(FB) * np.exp(1j * np.angle(FA))))

mm.show(
    [img_a, img_b, rec_A_mag_B_fase, rec_B_mag_A_fase],
    titles=["Imagem A", "Imagem B", "Mag(A) + Fase(B)", "Mag(B) + Fase(A)"],
    cols=4, figsize=(16, 4)
)

print(" Interpretação: Observe que a estrutura (formas) segue a FASE, não a magnitude.")
```

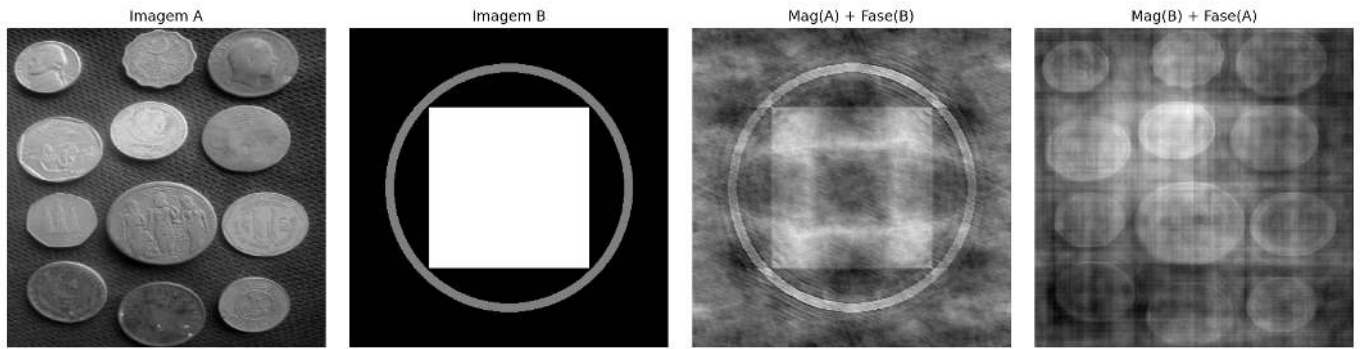


Figura 5.4: Experimento de troca de fase: Imagem A (moedas) e Imagem B (padrão geométrico) reconstruídas com magnitudes e fases trocadas. O resultado confirma que a estrutura visual segue a fase — a imagem reconstruída com a fase de B se parece com B, independentemente de qual magnitude foi usada.

Interpretação: Observe que a estrutura (formas) segue a FASE, não a magnitude.

5.3.6 Simulador: Reconstruindo Sinais com Senoides

Antes de prosseguir para imagens 2D, o simulador interativo da Figure 5.5 permite explorar a essência da análise de Fourier 1D: **qualquer forma de onda pode ser reconstruída somando-se senoides simples**.

Observe como, ao adicionar mais termos, a soma (curva preta) converge progressivamente para a forma alvo (tracejada). O espectro abaixo mostra as amplitudes de cada frequência — é a “receita” da forma de onda.

Explore ativamente: (i) Com quantos termos a onda quadrada fica visualmente aceitável? (ii) Qual das três formas converge mais rapidamente — por quê? (iii) O que acontece com o espectro ao trocar de onda quadrada para triangular?

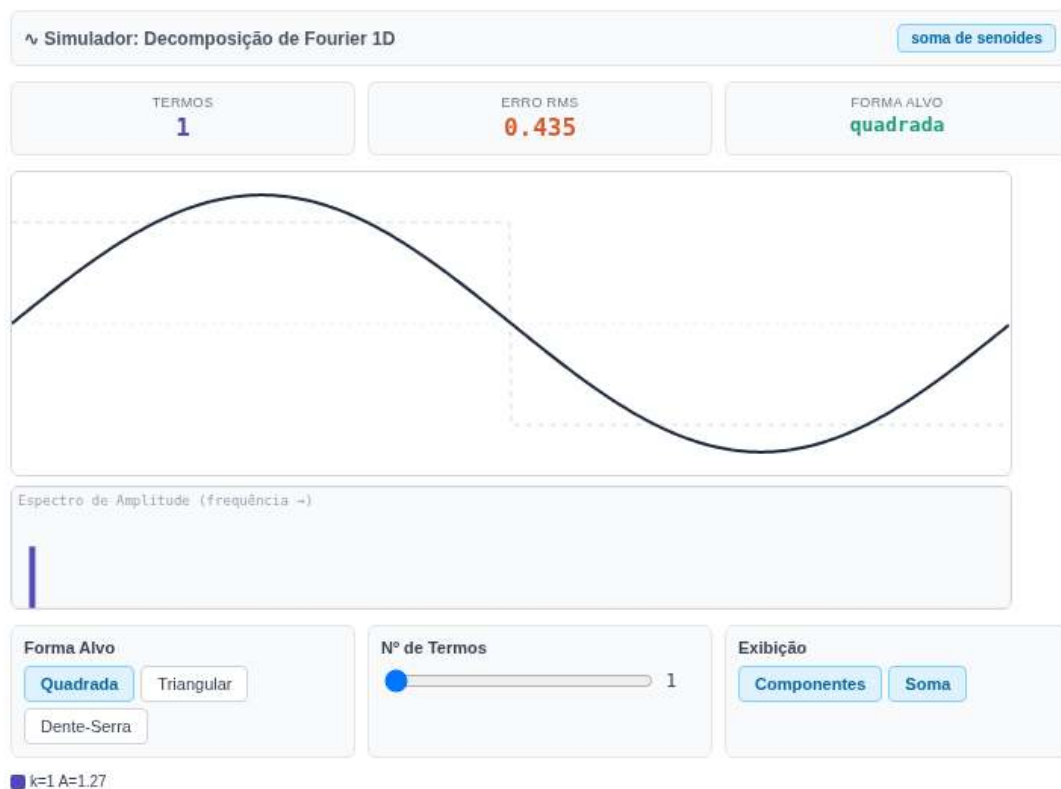


Figura 5.5: Simulador interativo da decomposição de Fourier 1D: visualização da soma de senoides com diferentes frequências, amplitudes e fases. Adicione termos e observe a convergência para formas de onda arbitrárias.

5.4 Teorema da Convolução e Estratégias de Filtragem

O **Teorema da Convolução** estabelece uma das relações mais importantes entre os domínios espacial e de frequência:

$$f(x, y) \otimes h(x, y) \xleftrightarrow{\mathcal{F}} F(u, v) H(u, v) \quad (5.3)$$

onde $\otimes$ representa a convolução circular discreta. Para reproduzir a convolução linear convencional sem artefatos de borda, utiliza-se *zero-padding* antes da FFT. Isso significa que, em vez de deslizar uma máscara espacialmente sobre cada pixel, podemos realizar uma multiplicação ponto a ponto no domínio da frequência.

Contudo, no domínio espacial, existe uma propriedade matemática crucial para filtros como o Gaussiano e o *Box Filter*: a **separabilidade**.

5.4.1 Kernel Separável vs. Não Separável

A diferença entre um *kernel* separável e um não separável reside na capacidade de decompor uma matriz bidimensional em duas operações unidimensionais independentes.

- **Kernel Não Separável:** É uma matriz de filtro $K \times K$ cujas linhas ou colunas são linearmente independentes. Para calcular o valor de um único pixel na imagem resultante, o algoritmo precisa realizar K^2 multiplicações e somas, varrendo toda a vizinhança bidimensional simultaneamente.
- **Kernel Separável:** Pode ser escrito exatamente como produto externo entre dois vetores 1D: um vetor coluna e um vetor linha ($H = v \cdot h^T$).

Na prática, isso transforma uma varredura bidimensional pesada em duas passagens unidimensionais consecutivas: primeiro, filtra-se cada linha da imagem com o vetor horizontal de tamanho $1 \times K$; em seguida, filtra-se o resultado verticalmente com o vetor de tamanho $K \times 1$.

5.4.2 Análise de Eficiência Computacional em Alta Resolução

Para uma imagem de dimensões $M \times N$ (como o caso de 2560×1920 para a imagem das moedas) e um filtro quadrado de tamanho $K \times K$, o impacto de cada abordagem na complexidade algorítmica e no tempo de execução é drástico, ver Table 5.1.

Tabela 5.1: Comparação de complexidade entre convolução direta, separável e via FFT.

Método de Filtragem	Complexidade Assintótica	Operações por Pixel	Comportamento com o crescimento de K
Espacial Não Separável	$\mathcal{O}(M \cdot N \cdot K^2)$	K^2	Quadrático: O tempo de processamento explode rapidamente.
Espacial Separável	$\mathcal{O}(M \cdot N \cdot 2K)$	$2K$	Linear: Altamente eficiente, mesmo para <i>kernels</i> moderados.
Filtragem via FFT	$\mathcal{O}(M \cdot N \log(M \cdot N))$	$\log(M \cdot N)$	Constante: O tamanho do <i>kernel</i> não altera o tempo de execução.

5.4.3 Discussão do Gráfico Experimental

O gráfico obtido no ensaio experimental utilizando a imagem das moedas (2560×1920), apresentado na Figure 5.6, traduz de forma consistente a teoria da complexidade computacional para cenários reais de processamento intensivo. Os tempos medidos (em milissegundos) variam conforme o hardware, a implementação da biblioteca e os mecanismos de paralelização disponíveis no computador utilizado.

1. **O Colapso da Convolução Não Separável ($\mathcal{O}(M \cdot N \cdot K^2)$):** Como previsto pela curva laranja, a abordagem direta bidimensional sofre uma penalidade severa. Para um *kernel* pequeno de 5×5 , o tempo é desprezível, mas ao atingir um *kernel* de 51×51 , o tempo de execução ultrapassa a marca dos **25.000 ms (25 segundos)** para processar apenas três iterações. Ela se torna proibitiva para *pipelines* de tempo real.
2. **A Consistência Absoluta da FFT ($\mathcal{O}(M \cdot N \log(M \cdot N))$):** A linha roxa demonstra a independência da FFT em relação ao tamanho do filtro. Uma vez mapeados a imagem e o *kernel* expandido para o domínio da frequência, a operação passa a ser uma multiplicação ponto a ponto de complexidade fixa. O tempo flutua estavelmente em torno de **1.500 ms**, independentemente de o *kernel* ser 5×5 ou 51×51 . A FFT cruza a curva não separável por volta de $K = 11$.
3. **A Supremacia Prática da Convolução Separável ($\mathcal{O}(M \cdot N \cdot 2K)$):** Representada pela linha verde rente ao eixo horizontal, a convolução separável aproveita a otimização de baixo nível (como vetorização SIMD do `cv2.sepFilter2D`). Como o custo cresce apenas de forma linear com K ($2K$ operações), ela consegue processar a imagem de 5 MP em **poucos milissegundos**, superando inclusive a FFT em toda a faixa testada devido à ausência do *overhead* de cálculo das transformadas direta e inversa.

O ganho da FFT torna-se evidente principalmente para *kernels* grandes não separáveis ou quando múltiplas convoluções são realizadas no domínio espectral. Para filtros gaussianos, a separabilidade reduz drasticamente o custo espacial.

$$g = \mathcal{F}^{-1}[\mathcal{F}(f) \cdot \mathcal{F}(h)] \quad \text{vs.} \quad g = (f \otimes K) \quad \text{vs.} \quad g = (f \otimes v) \otimes h^T \quad (5.4)$$

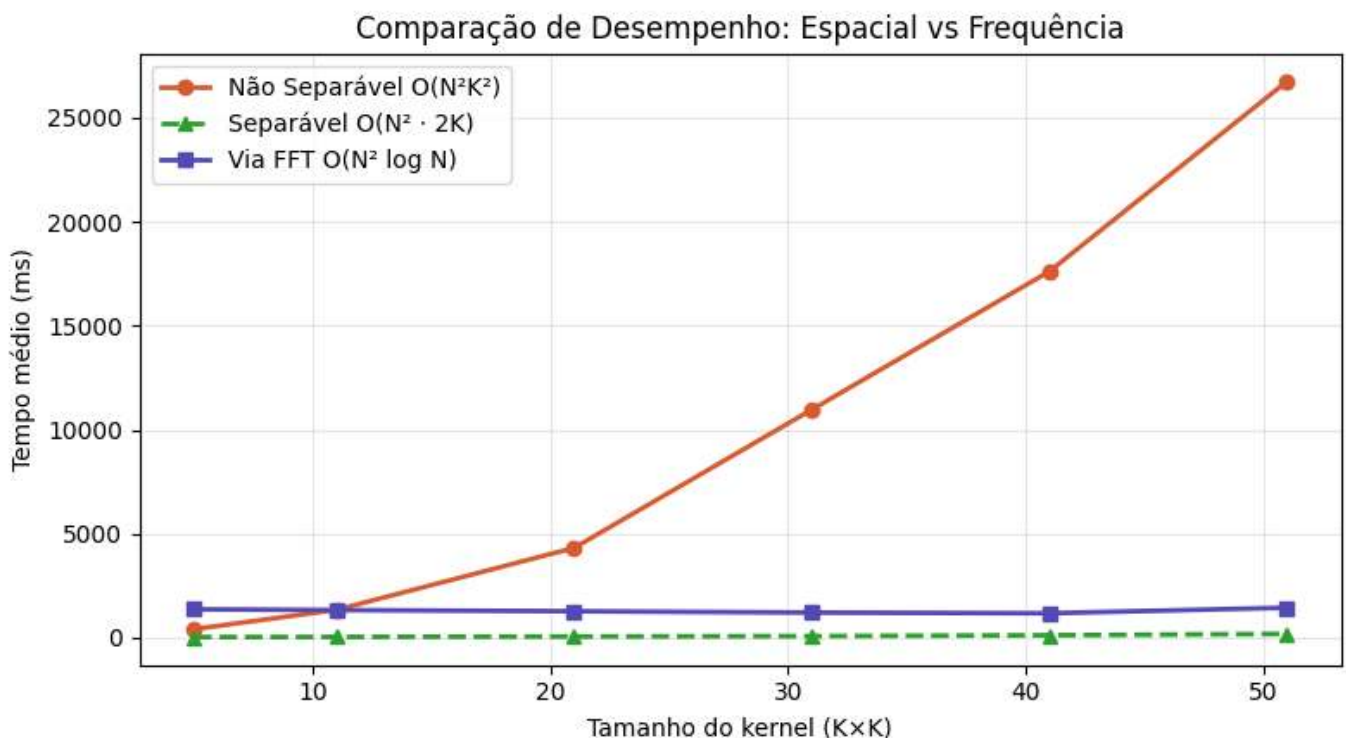


Figura 5.6: Comparação de eficiência: Convolução Não Separável (Espacial 2D), Separável (Espacial 1D) e via FFT.

! O Perigo da Convolução Circular (*Wrap-around Error*)

A DFT pressupõe que a imagem se repete infinitamente como um azulejo. Se aplicarmos um filtro de frequência sem esticar a imagem com bordas de zeros (*zero-padding*), o topo da imagem se mistura com a base, e a esquerda com a direita. O *padding* evita esse vazamento.

```
# Definir M e N
M, N = img_gray.shape # linha adicionada

# Simulação de um filtro de deslocamento brutal
H_shift = np.zeros_like(img_gray, dtype=complex)
for u in range(M):
    for v in range(N):
        H_shift[u, v] = np.exp(-1j * 2 * np.pi * (u*120/M + v*120/N))

# Filtragem SEM padding (causa o wrap-around)
F_img = np.fft.fft2(img_gray)
img_vazada = np.real(np.fft.ifft2(F_img * H_shift))

img_vazada_vis = cv2.normalize(img_vazada, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
mm.show([img_gray, img_vazada_vis], titles=["Original", "Filtragem s/ Padding (Vazamento)"], cols=2, fig
```



Figura 5.7: Sem padding, um deslocamento severo faz a imagem vazar para o lado oposto (convolução circular).

```
# Kernel Gaussiano 11x11
sigma = 3.0
K = 11
ks = np.arange(K) - K // 2
gauss1d = np.exp(-ks**2 / (2 * sigma**2))
gauss1d /= gauss1d.sum()
kernel = np.outer(gauss1d, gauss1d) # kernel 2D separável

# Método 1: Convolução espacial direta
f_float = img_gray.astype(np.float64)
convEsp = cv2.filter2D(f_float, -1, kernel, borderType=cv2.BORDER_CONSTANT)

# Método 2: Multiplicação em frequência (via FFT)
M, N = f_float.shape
# Padding para convolução linear (evita aliasing circular)
Mpad = 2 ** int(np.ceil(np.log2(M + K - 1)))
```



```

Npad      = 2 ** int(np.ceil(np.log2(N + K - 1)))

# Posiciona o kernel com a origem no (0,0) e padding com zeros
kernel_pad = np.zeros((Mpad, Npad))
kh, kw     = kernel.shape
kernel_pad[:kh, :kw] = kernel

F_img     = np.fft.fft2(f_float, (Mpad, Npad))
F_kern    = np.fft.fft2(kernel_pad)
conv_freq = np.real(np.fft.ifft2(F_img * F_kern))

# Recorte para compensar o deslocamento introduído pelo posicionamento do kernel
offset    = K // 2
conv_freq_crop = conv_freq[offset:offset+M, offset:offset+N]

# Verificação numérica
diff = np.abs(conv_esp - conv_freq_crop)
print(f"Diferença máxima (|conv_esp - conv_freq|): {diff.max():.2e}")
print(f"Diferença média (|conv_esp - conv_freq|): {diff.mean():.2e}")
print(f"→ Teorema da Convolução verificado numericamente.")

conv_esp_vis = cv2.normalize(conv_esp, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
conv_freq_vis = cv2.normalize(conv_freq_crop, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
diff_vis     = cv2.normalize(diff, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

mm.show(
    [img_gray, conv_esp_vis, conv_freq_vis, diff_vis],
    titles=[
        "Original",
        "Convolução espacial",
        "Multiplicação em frequência",
        f"Diferença (máx={diff.max():.1e})"
    ],
    cols=4, figsize=(16, 5)
)

```

```

Diferença máxima (|conv_esp - conv_freq|): 2.56e-13
Diferença média (|conv_esp - conv_freq|): 2.79e-14
→ Teorema da Convolução verificado numericamente.

```



Figura 5.8: Verificação do Teorema da Convolução: a diferença pixel a pixel entre a convolução espacial (`cv2.filter2D`) e a multiplicação em frequência (FFT) é numericamente nula — confirmando a equivalência teórica.

5.5 Filtros no Domínio da Frequência

Um filtro no domínio da frequência é, essencialmente, uma **máscara aplicada ao espectro**: valores próximos de 1 deixam a frequência passar; valores próximos de 0 a atenuam. A forma dessa máscara determina o comportamento visual do filtro.

O dilema do corte abrupto: um filtro ideal (corte perfeito em D_0) parece ótimo na teoria, mas causa o **fenômeno de Ringing** — oscilações próximas às bordas dos objetos. Isso ocorre porque um corte abrupto no espectro corresponde, no domínio espacial, a uma convolução com uma função *sinc* de suporte infinito — que produz os anéis visíveis, ver Figure 5.9.

A solução: filtros com transição suave — Gaussiano ou Butterworth — eliminam o *ringing* ao custo de uma fronteira de corte menos precisa.

```
# Simulando o Filtro Ideal na Frequência (Cilindro) e sua representação Espacial (Sinc)
N_grid = 128
u = np.arange(-N_grid//2, N_grid//2)
U, V = np.meshgrid(u, u)
D = np.sqrt(U**2 + V**2)

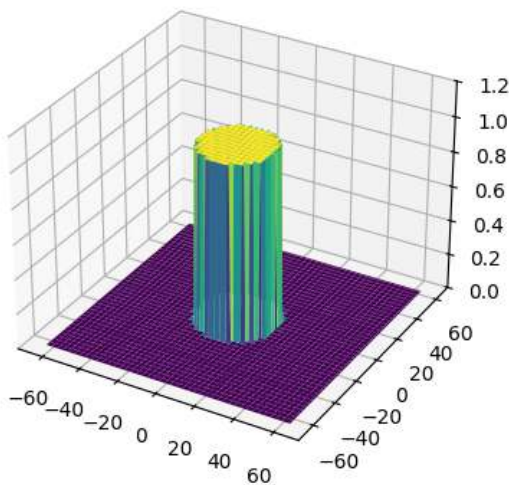
# Frequência: Cilindro Ideal (1 no centro, 0 fora do raio 20)
H_freq = np.zeros((N_grid, N_grid))
H_freq[D <= 20] = 1

# Espaço: A inversa resulta na famigerada Sinc 2D
h_space = np.fft.fftshift(np.real(np.fft.ifft2(np.fft.ifftshift(H_freq))))

fig, ax = plt.subplots(1, 2, subplot_kw={'projection': '3d'}, figsize=(12, 4))
ax[0].plot_surface(U, V, H_freq, cmap='viridis', edgecolor='none')
ax[0].set_title("Frequência: Filtro Ideal (Cilindro)")
ax[0].set_zlim(0, 1.2)

ax[1].plot_surface(U, V, h_space, cmap='plasma', edgecolor='none')
ax[1].set_title("Espaço Real: Ondulações da Sinc (Causa do Ringing)")
plt.tight_layout(); plt.show()
```

Frequência: Filtro Ideal (Cilindro)



Espaço Real: Ondulações da Sinc (Causa do Ringing)

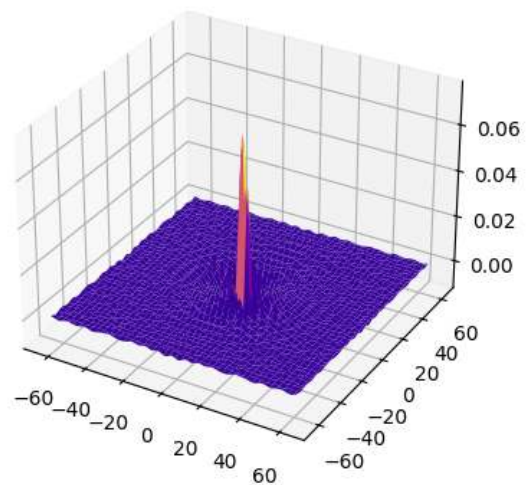


Figura 5.9: A Dualidade Perigosa: O corte abrupto na Frequência (Cilindro) transforma-se obrigatoriamente numa Sinc espacial. Suas ondulações causam o *ringing* fantasma nas bordas da imagem.

5.5.1 Filtros Passa-Baixa

Filtros passa-baixa atenuam altas frequências, suavizando a imagem e reduzindo ruído. Definem-se em termos da distância ao centro do espectro:

$$D(u, v) = \sqrt{\left(u - \frac{M}{2}\right)^2 + \left(v - \frac{N}{2}\right)^2} \quad (5.5)$$

Filtro Ideal (LPFI):

$$H_{\text{ideal}}(u, v) = \begin{cases} 1, & D(u, v) \leq D_0 \\ 0, & D(u, v) > D_0 \end{cases} \quad (5.6)$$

O corte abrupto em D_0 causa o **fenômeno de Ringing**: oscilações artificiais próximas às bordas dos objetos, análogas ao comportamento da série de Fourier truncada.

Filtro Gaussiano (LPFG):

$$H_{\text{gauss}}(u, v) = e^{-D^2(u, v)/(2\sigma^2)} \quad (5.7)$$

No domínio contínuo, a transformada de Fourier de uma Gaussiana é também Gaussiana — propriedade que garante transição suave sem *ringing*.

Filtro Butterworth (LPFB) de ordem n :

$$H_{\text{BW}}(u, v) = \frac{1}{1 + [D(u, v)/D_0]^{2n}} \quad (5.8)$$

O parâmetro n controla a inclinação da transição: valores baixos ($n = 1, 2$) produzem transições suaves (sem *ringing*); valores altos ($n \geq 5$) aproximam o comportamento do filtro ideal. Ver exemplos na Figure 5.10.



Figura 5.10: Filtros passa-baixa.

5.5.2 Filtros Passa-Alta e Passa-Banda

Filtros passa-alta são obtidos pelo complemento de qualquer filtro passa-baixa: $H_{\text{HP}} = 1 - H_{\text{LP}}$. O efeito visual é o inverso — preserva bordas e detalhes, atenua regiões uniformes.

Filtros passa-banda combinam um corte inferior D_L e um corte superior D_H , preservando apenas as frequências entre eles: $H_{\text{BP}} = H_{\text{HP}}^{(D_L)} \cdot H_{\text{LP}}^{(D_H)}$.

São especialmente úteis na **remoção de ruído periódico**: padrões regulares (interferência elétrica, grade de scanner) aparecem como picos no espectro de magnitude e podem ser suprimidos com um filtro rejeita-banda (*notch filter*) posicionado sobre esses picos. Ver exemplos de aplicações de filtros passa baixa na Figure 5.12 e passa alta na Figure 5.13.

Figura 5.11: Simulador interativo de filtros no domínio da frequência.

```

import io

def distancia_centro(M, N):
    """Matriz de distâncias ao centro do espectro."""
    u = np.arange(M) - M // 2
    v = np.arange(N) - N // 2
    V, U = np.meshgrid(v, u)
    return np.sqrt(U**2 + V**2)

def aplicar_filtro_freq(img, H):
    """Aplica filtro H (centrado) a imagem via FFT."""
    F = np.fft.fftshift(np.fft.fft2(img.astype(np.float64)))
    Fg = F * H
    g = np.real(np.fft.ifft2(np.fft.ifftshift(Fg)))
    return cv2.normalize(g, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

M, N = img_gray.shape
D = distancia_centro(M, N)
D0 = 30 # frequência de corte
n_bw = 2 # ordem do Butterworth

# Funções de transferência
H_ideal = (D <= D0).astype(np.float64)
H_gauss = np.exp(-D**2 / (2 * D0**2))
H_bw = 1.0 / (1.0 + (D / D0)**(2 * n_bw))

# Imagens filtradas
img_ideal = aplicar_filtro_freq(img_gray, H_ideal)
img_gauss = aplicar_filtro_freq(img_gray, H_gauss)
img_bw = aplicar_filtro_freq(img_gray, H_bw)

# Perfis de H(u,v)
def fig2img(fig):
    b = io.BytesIO(); fig.savefig(b, format='png', dpi=100); plt.close(fig); b.seek(0)
    return (plt.imread(b)[:,:,:3]*255).astype(np.uint8)

fig, ax = plt.subplots(figsize=(6, 3))
linha = M // 2
ax.plot(H_ideal[linha, :], label="Ideal", color="#D85A30", lw=1.5, ls="--")
ax.plot(H_gauss[linha, :], label="Gaussiano", color="#1D9E75", lw=1.5)
ax.plot(H_bw[linha, :], label="Butterworth", color="#534AB7", lw=1.5)
ax.axvline(N//2-D0, color="#aaa", lw=0.8, ls=":")
ax.axvline(N//2+D0, color="#aaa", lw=0.8, ls=":")
ax.set(title="Perfis H(u,v) - linha central", xlabel="v", ylabel="H(u,v)")
ax.legend(fontsize=8); plt.tight_layout()
perfil_img = fig2img(fig)

# Filtros H visualizados
def H_vis(H):
    return cv2.normalize((H*255).astype(np.uint8), None, 0, 255, cv2.NORM_MINMAX)

mm.show(
    [img_gray, img_ideal, img_gauss, img_bw,
     H_vis(H_ideal), H_vis(H_gauss), H_vis(H_bw), perfil_img],

```

```

titles=[
    "Original", "LPF Ideal", "LPF Gaussiano", "LPF Butterworth (n=2)",
    "H Ideal", "H Gaussiano", "H Butterworth", "Perfis H(u,v)"
],
cols=4, figsize=(16, 9)
)

```

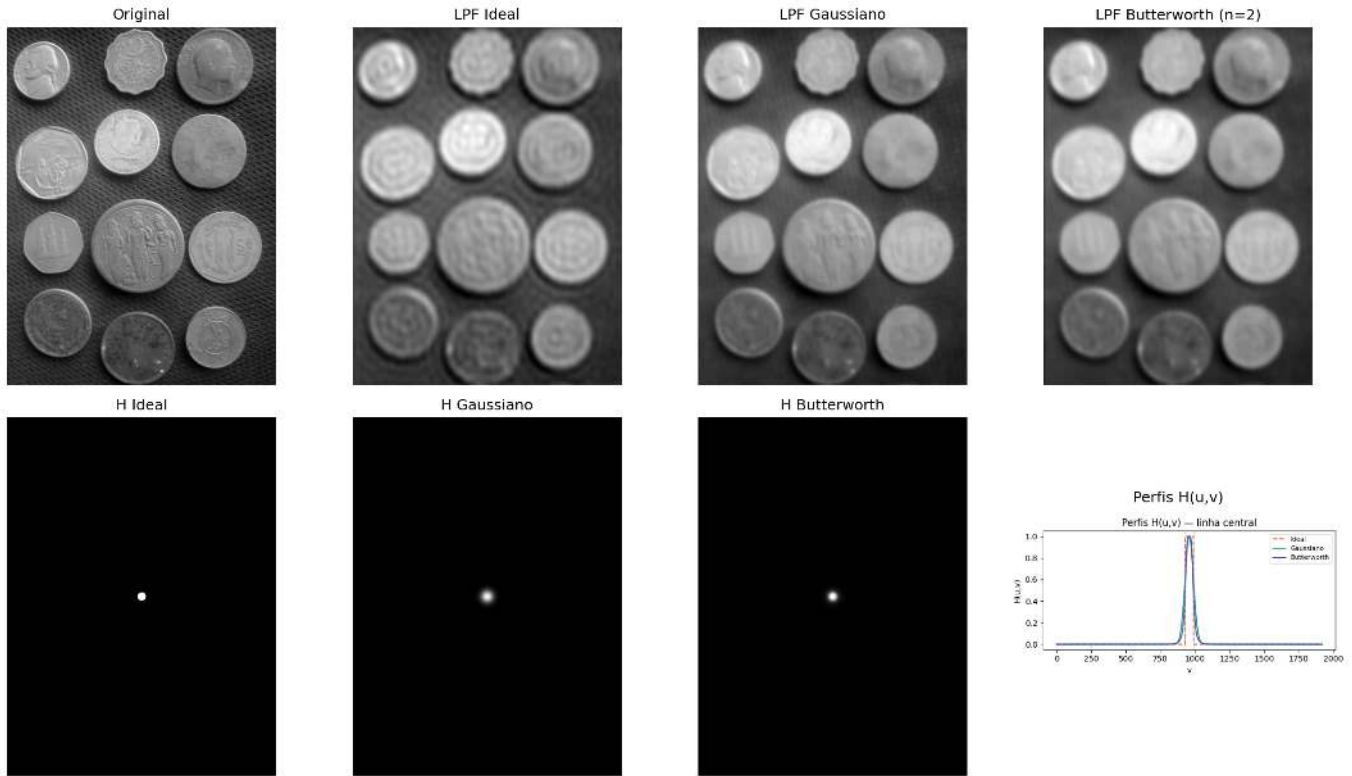


Figura 5.12: Comparação entre filtros passa-baixa: Ideal ($D = 30$), Gaussiano ($D = 30$) e Butterworth ($D = 30$, $n=2$). Perfis de $H(u,v)$ ao longo de uma linha central e imagens filtradas correspondentes.

```

# Filtro passa-alta: complemento do passa-baixa Gaussiano
# Reutiliza aplicar_filtro_freq() definida na célula anterior
H_alta = 1 - H_gauss
img_alta = aplicar_filtro_freq(img_gray, H_alta)

mm.show(
    [img_gray, img_alta],
    titles=["Original", "Passa-alta Gaussiano ($D_0=30$)"],
    cols=2
)

```

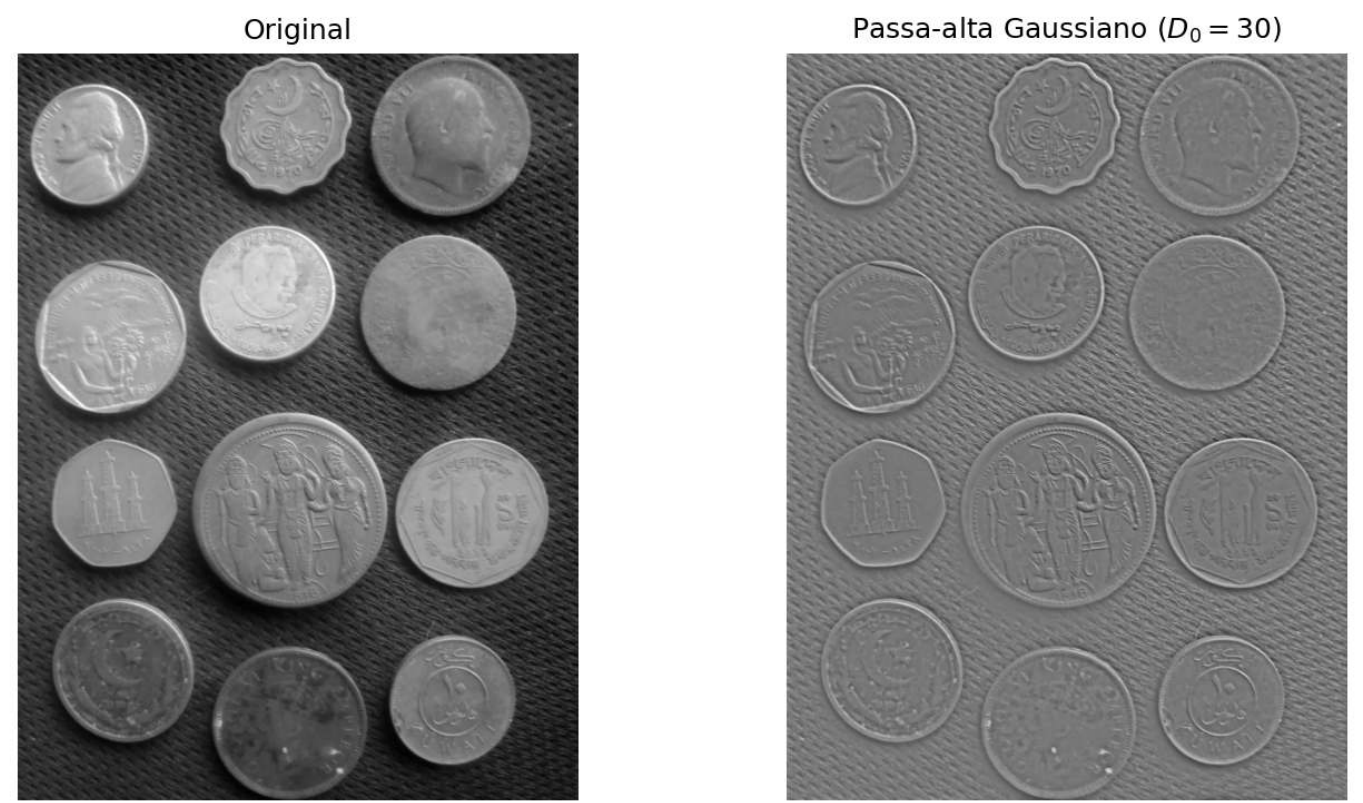


Figura 5.13: Filtro passa-alta Gaussiano. (a) Original; (b) Filtro passa-alta ($D = 30$) - as bordas das moedas e fundo texturizado são realçados.

5.5.3 Remoção de Ruído Periódico

Ruído periódico — proveniente de interferência elétrica, sensores com padrão regular ou artefatos de digitalização — manifesta-se no espectro de Fourier como **picos pontuais** simétricos em torno do centro. O filtro **rejeita-banda notch** suprime seletivamente essas frequências, preservando o restante da imagem.

📌 Síntese — Filtros Espectrais

Filtro	Efeito visual	Artefato	Uso
Passa-baixa Ideal	Suavização forte	Ringin (anéis)	Ilustrativo apenas
Passa-baixa Gaussiano	Suavização suave	Nenhum	Suavização geral
Passa-baixa Butterworth	Suavização controlada	Ringin leve (ordens altas)	Compromisso suavidade/precisão
Passa-alta	Realce de bordas	Pode amplificar ruído	Deteção de contornos
Notch	Remove frequências específicas	Pode criar artefatos locais	Remoção de ruído periódico

O design de filtros no domínio da frequência é direto e intuitivo — mas os artefatos espaciais (ringing, borramento) emergem de escolhas no espectro.

5.6 Wavelets e Multirresolução

A Transformada de Fourier decompõe o sinal em frequências **globais**: cada coeficiente $F(u,v)$ recebe contribuições de **toda** a imagem, sem informação sobre *onde* uma frequência ocorre. Uma borda localizada no canto da imagem se mistura ao espectro inteiro — o que limita sua capacidade de representar explicitamente onde determinadas frequências ocorrem espacialmente.

```

# Imagem com ruído periódico sintético
h_img, w_img = img_gray.shape
x = np.arange(w_img)
y = np.arange(h_img)
X, Y = np.meshgrid(x, y)

# Usar frequências inteiras e consistentes (importante para restauração perfeita)
u0, v0 = 20, 20 # frequências exatas do ruído

ruído = 40 * np.sin(2 * np.pi * (u0 * X / w_img + v0 * Y / h_img))
img_ruidosa = np.clip(img_gray.astype(np.float64) + ruído, 0, 255).astype(np.uint8)

# Espectro da imagem ruidosa
F_r = np.fft.fftshift(np.fft.fft2(img_ruidosa.astype(np.float64)))
mag_r = np.log1p(np.abs(F_r))
mag_vis = cv2.normalize(mag_r, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# Máscara notch
mascara = np.ones((h_img, w_img), dtype=np.float64)
r_notch = 8 # raio do notch (ajuste fino se necessário)

def suprimir_pico(mask, cy, cx, r):
    """Zera um disco de raio r centrado em (cy, cx)"""
    yy, xx = np.ogrid[:mask.shape[0], :mask.shape[1]]
    dist = np.sqrt((yy - cy)**2 + (xx - cx)**2)
    mask[dist <= r] = 0
    return mask

# Coordenadas centrais
cy, cx = h_img // 2, w_img // 2

# Suprimir os 4 picos simétricos (importante!)
for dy, dx in [(v0, u0), (-v0, -u0), (v0, -u0), (-v0, u0)]:
    mascara = suprimir_pico(mascara, cy + dy, cx + dx, r_notch)

mascara_vis = (mascara * 255).astype(np.uint8)

# Filtragem e reconstrução
F_filtrada = F_r * mascara
img_rest = np.real(np.fft.ifft2(np.fft.ifftshift(F_filtrada)))
img_rest_vis = cv2.normalize(img_rest, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# Avaliação
psnr = cv2.PSNR(img_gray, img_rest_vis)
#ssim = cv2.SSIM(img_gray, img_rest_vis) if hasattr(cv2, 'SSIM') else "N/A"
ssim = ssim_sk(img_gray, img_rest_vis, data_range=255)

print(f"PSNR (original vs restaurada): {psnr:.2f} dB")

# Visualização
mm.show(
    [img_ruidosa, mag_vis, mascara_vis, img_rest_vis],
    titles=[
        "Com ruído periódico",
        "Espectro (log)",
        "Máscara notch",
        f"Restaurada (PSNR={psnr:.1f} dB)"
    ],
    cols=4,
    figsize=(16, 4)
)

```

As **wavelets** (*ondaletas*) mitigam essa limitação com funções de base **compactas** — energia concentrada em uma região finita — que podem ser deslocadas e escaladas, oferecendo representação simultânea em **frequência e localização espacial**.

5.6.1 O Limite de Fourier: Onde ocorreu o evento?

Fourier nos diz perfeitamente *quais* frequências existem, mas perde totalmente a noção do *tempo/espço* de onde elas ocorreram. Veja o experimento abaixo: uma imagem com duas linhas nítidas, e outra com as mesmas linhas deslocadas. O espectro de magnitude é virtualmente idêntico.

```
img_sinal1 = np.zeros((128, 128)); img_sinal1[:, 20:25] = 1; img_sinal1[100:105, :] = 1
img_sinal2 = np.zeros((128, 128)); img_sinal2[:, 90:95] = 1; img_sinal2[30:35, :] = 1

mag1 = np.log1p(np.abs(np.fft.fftshift(np.fft.fft2(img_sinal1))))
mag2 = np.log1p(np.abs(np.fft.fftshift(np.fft.fft2(img_sinal2))))

mm.show([img_sinal1, mag1, img_sinal2, mag2],
        titles=["Sinal A", "Espectro A", "Sinal B (Deslocado)", "Espectro B"],
        cols=4, figsize=(14, 4))
```

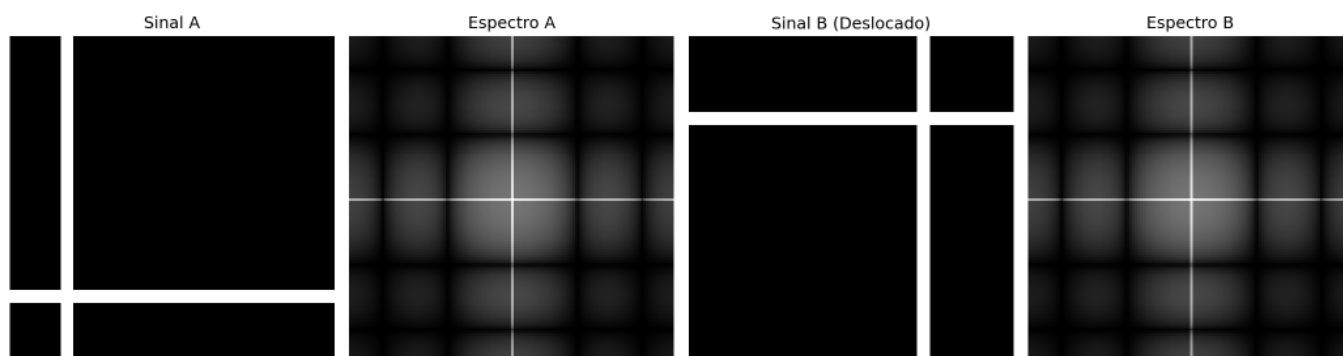


Figura 5.15: Fourier global é cego para a posição. Os espectros não dizem onde as bordas estão.

5.6.2 Transformada Wavelet Discreta 2D

A DWT aplica, separadamente em linhas e colunas, dois filtros complementares — **passa-baixa** h (aproximações) e **passa-alta** g (detalhes) — seguidos de subamostragem por 2, produzindo quatro subbandas:

$$\text{DWT}(f) = \left\{ \begin{array}{c} \underline{\underline{LL}} \\ \text{aprox.} \end{array}, \begin{array}{c} \underline{\underline{LH}} \\ \text{horiz.} \end{array}, \begin{array}{c} \underline{\underline{HL}} \\ \text{vert.} \end{array}, \begin{array}{c} \underline{\underline{HH}} \\ \text{diag.} \end{array} \right\}$$

Subbanda	Filtros aplicados	Conteúdo visual
LL	baixa × baixa	Aproximação suavizada — versão reduzida da imagem
LH	baixa × alta	Bordas horizontais e variações verticais
HL	alta × baixa	Bordas verticais e variações horizontais
HH	alta × alta	Detalhes diagonais, texturas em 45°, cantos

A decomposição é **recursiva**: aplicando a DWT novamente sobre LL obtém-se o próximo nível. Após J

níveis, a representação hierárquica contém $3J + 1$ subbandas — cada nível com metade da resolução do anterior.

i Conexão com CNNs

A análise multirresolução das *Wavelets* possui forte relação conceitual com representações hierárquicas utilizadas em métodos modernos de visão computacional, como as CNNs.

5.6.3 Famílias de Wavelets

Diferentes *wavelets* resultam em diferentes compromissos entre localização, suavidade e compactação:

Wavelet	Suporte	Momentos nulos	Simetria	Uso típico
Haar	2	1	Assimétrica	Análise básica, educação
Daubechies db4	8	4	Assimétrica	Compressão, análise geral
Symlet sym4	8	4	Quase simétrica	Reconstrução de sinais
Biortogonal 5/3	5/3	2/2	Simétrica	JPEG 2000 (sem perda)
Biortogonal 9/7	9/7	4/4	Simétrica	JPEG 2000 (com perda)

O número de **momentos nulos** controla a compactação: com p momentos nulos, a *wavelet* produz coeficientes exatamente nulos para polinômios de grau até $p - 1$ — quanto maior p , mais coeficientes próximos de zero em regiões suaves e maior a eficiência de compressão.

```
# Import pywt
try:
    import pywt
except ImportError:
    import subprocess
    subprocess.run(["pip", "install", "PyWavelets", "-q"])
    import pywt

wavelet_haar = pywt.Wavelet('haar')
wavelet_db4 = pywt.Wavelet('db4')

phi_h, psi_h, x_h = wavelet_haar.wavefun(level=4)
phi_d, psi_d, x_d = wavelet_db4.wavefun(level=4)

fig, ax = plt.subplots(1, 2, figsize=(10, 3))
ax[0].plot(x_h, psi_h, 'b', lw=2); ax[0].set_title("Ondaleta Haar ()")
ax[1].plot(x_d, psi_d, 'g', lw=2); ax[1].set_title("Ondaleta Daubechies 4 ()")
plt.tight_layout(); plt.show()
```

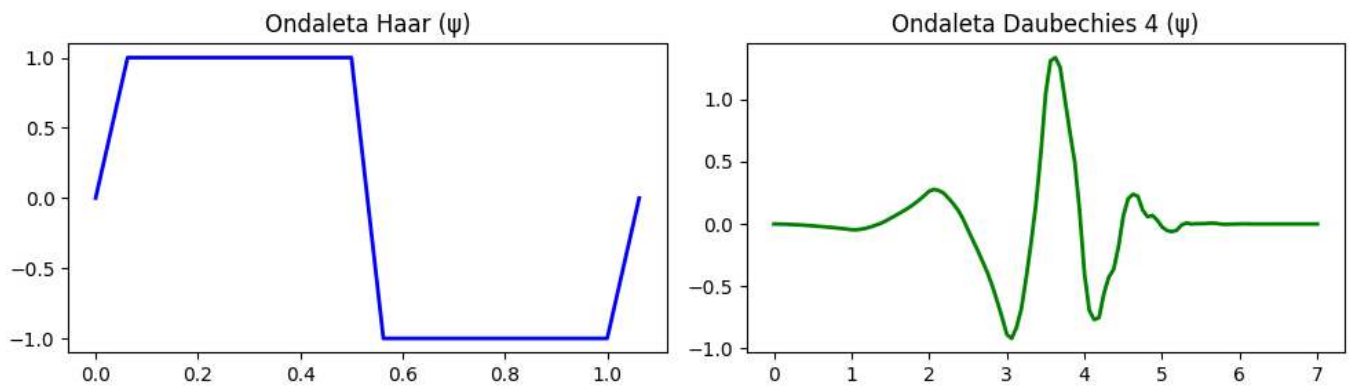



Figura 5.16: Funções da Wavelet (ψ). Note como elas rapidamente decaem para zero (suporte compacto), ao contrário das senoides infinitas de Fourier.

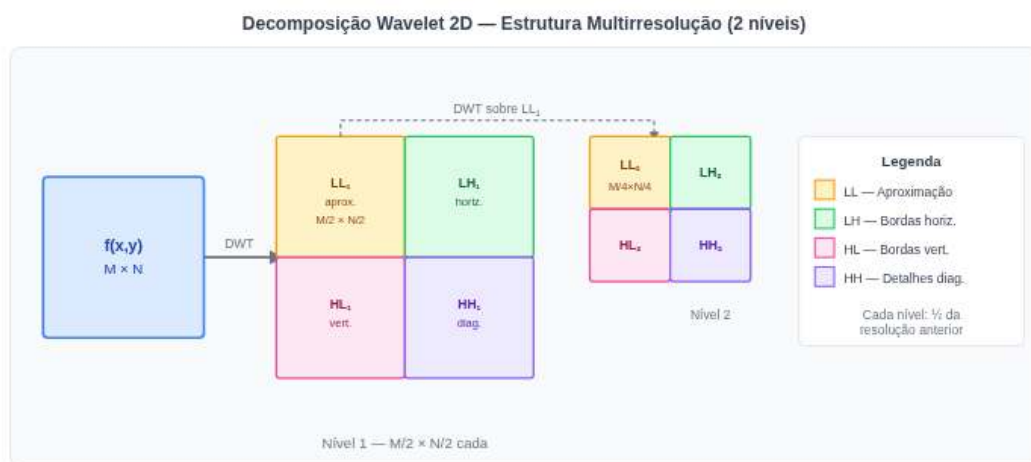


Figura 5.17: Diagrama da decomposição wavelet 2D em dois níveis.

```
try:
    import pywt
    HAS_PYWT = True
except ImportError:
    import subprocess
    subprocess.run(["pip", "install", "PyWavelets", "-q"])
    import pywt
    HAS_PYWT = True

# Decomposição wavelet 2 níveis
wavelet = "haar"
img_float = img_gray.astype(np.float64)

# Nível 1
coefs1 = pywt.dwt2(img_float, wavelet)
LL1, (LH1, HL1, HH1) = coefs1

# Nível 2 (aplicado sobre LL1)
coefs2 = pywt.dwt2(LL1, wavelet)
LL2, (LH2, HL2, HH2) = coefs2

def sb_vis(sb):
    """Normaliza subbanda para visualização [0,255]."""
    return cv2.normalize(np.abs(sb), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
```

```

print(f"Forma original      : {img_gray.shape}")
print(f"LL1 (nível 1)      : {LL1.shape} | LH1/HL1/HH1: {LH1.shape}")
print(f"LL2 (nível 2)      : {LL2.shape} | LH2/HL2/HH2: {LH2.shape}")

imgs_dwt = [img_gray, sb_vis(LL1), sb_vis(LH1), sb_vis(HL1), sb_vis(HH1),
                             sb_vis(LL2), sb_vis(LH2), sb_vis(HL2), sb_vis(HH2)]
titles_dwt = ["Original",
              "LL (aprox.)", "LH (horiz.)", "HL (vert.)", "HH (diag.)",
              "LL (aprox.)", "LH (horiz.)", "HL (vert.)", "HH (diag.)"]

mm.show(imgs_dwt, titles=titles_dwt, cols=5, figsize=(16, 7))

```

```

Forma original      : (2560, 1920)
LL1 (nível 1)      : (1280, 960) | LH1/HL1/HH1: (1280, 960)
LL2 (nível 2)      : (640, 480) | LH2/HL2/HH2: (640, 480)

```

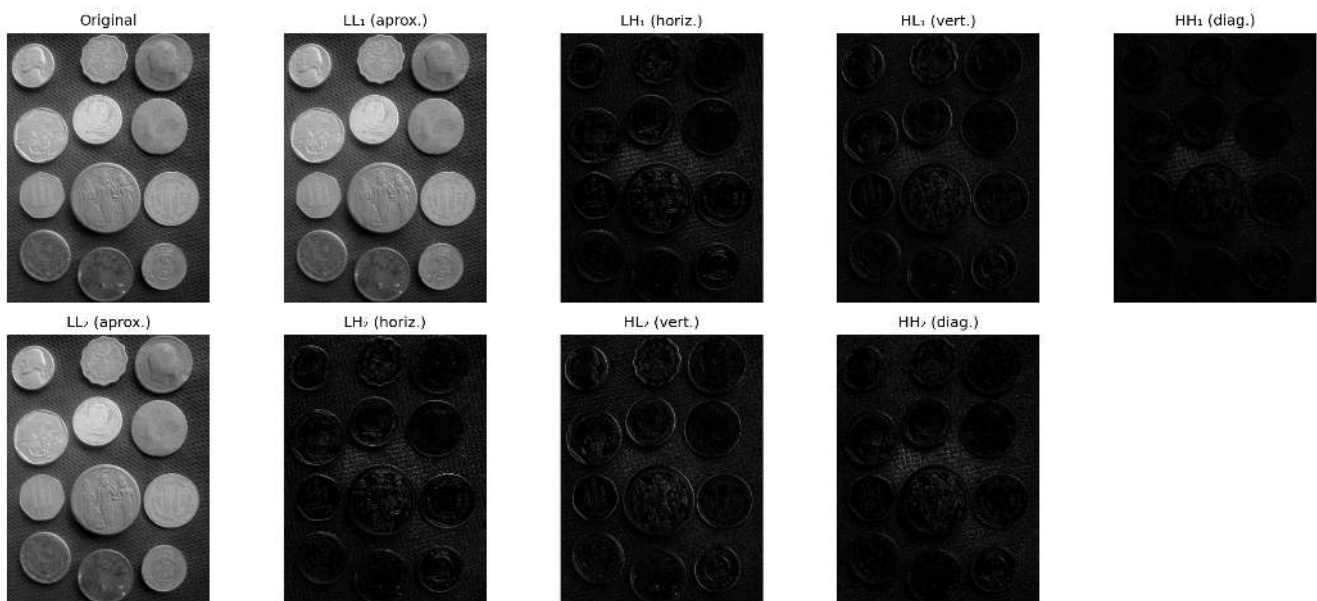


Figura 5.18: Decomposição wavelet 2D de 2 níveis com wavelet Haar: subbandas LL, LH, HL, HH em cada nível. As subbandas de detalhe revelam estruturas orientadas em diferentes escalas.

```

wavelets_comp = ["haar", "db4", "sym4", "bior2.2"]
imgs_comp, titles_comp = [], []

for wname in wavelets_comp:
    LL, (LH, HL, HH) = pywt.dwt2(img_float, wname)
    imgs_comp += [sb_vis(LL), sb_vis(HH)]
    titles_comp += [f"{wname} - LL ", f"{wname} - HH "]

mm.show(imgs_comp, titles=titles_comp, cols=4, figsize=(14, 8))

```

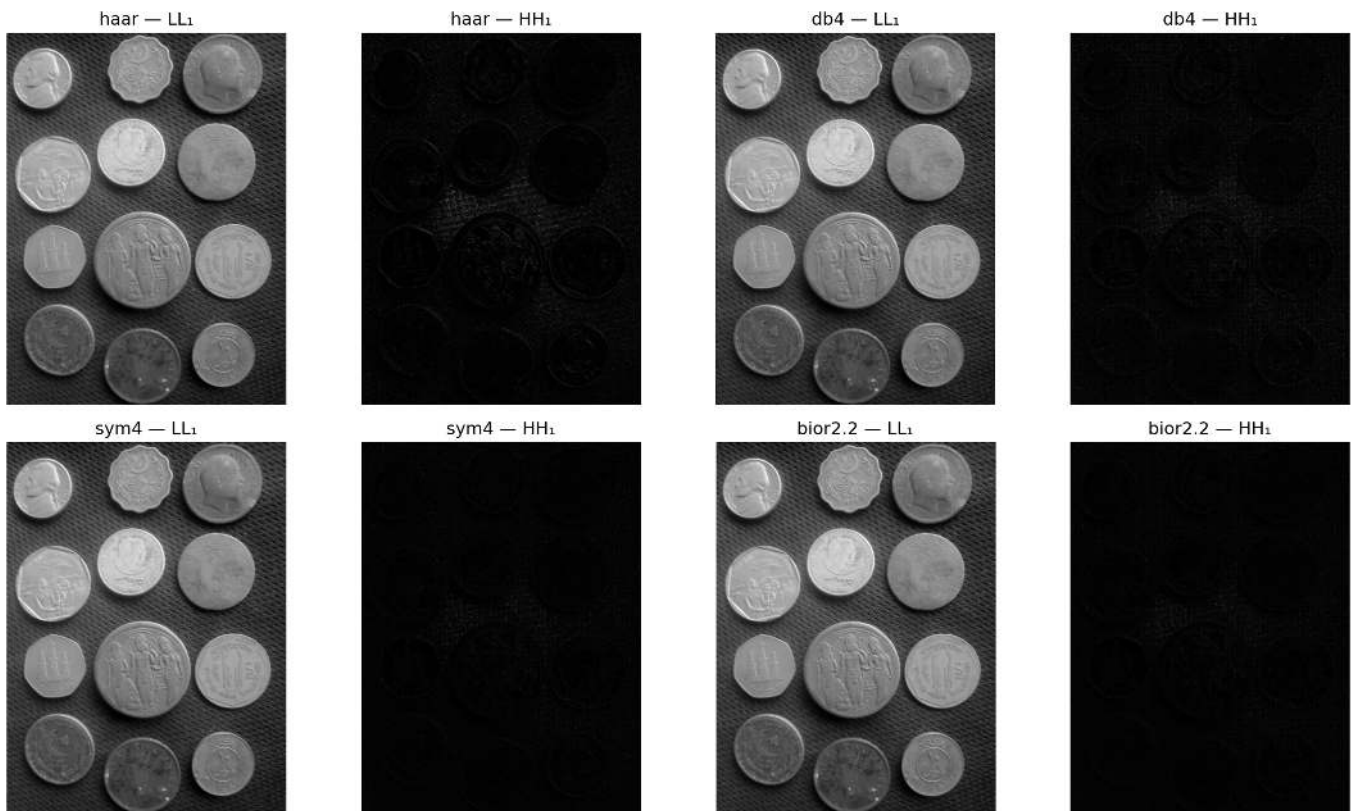


Figura 5.19: Comparação entre famílias de wavelets: Haar, db4, sym4 e bior2.2. Subbanda LL (aproximação) e HH (diagonal) para cada escolha, ilustrando o compromisso entre compactação e suavidade.

```
def dwt_threshold_reconstruct(img, wavelet='db4', nivel=2, threshold=0.0):
    """Decompõe, aplica limiar e reconstrói via IDWT."""
    coefs = pywt.wavedec2(img.astype(np.float64), wavelet, level=nivel)
    # Copia e aplica hard thresholding em todos os detalhe
    coefs_t = [coefs[0]] # LL final não é limiarizado
    for detalhe in coefs[1:]:
        coefs_t.append(tuple(pywt.threshold(sb, threshold, mode='hard') for sb in detalhe))
    rec = pywt.waverec2(coefs_t, wavelet)
    # Recorte para dimensão original
    rec = rec[:img.shape[0], :img.shape[1]]
    return np.clip(rec, 0, 255).astype(np.uint8)

thresholds = [0, 10, 30, 60, 100]
imgs_thr = [img_gray]
titles_thr = ["Original"]

for t in thresholds:
    rec = dwt_threshold_reconstruct(img_gray, threshold=t)
    psnr = cv2.PSNR(img_gray, rec)
    imgs_thr.append(rec)
    titles_thr.append(f"T={t} PSNR={psnr:.1f} dB")

mm.show(imgs_thr, titles=titles_thr, cols=3, figsize=(14, 10))
```

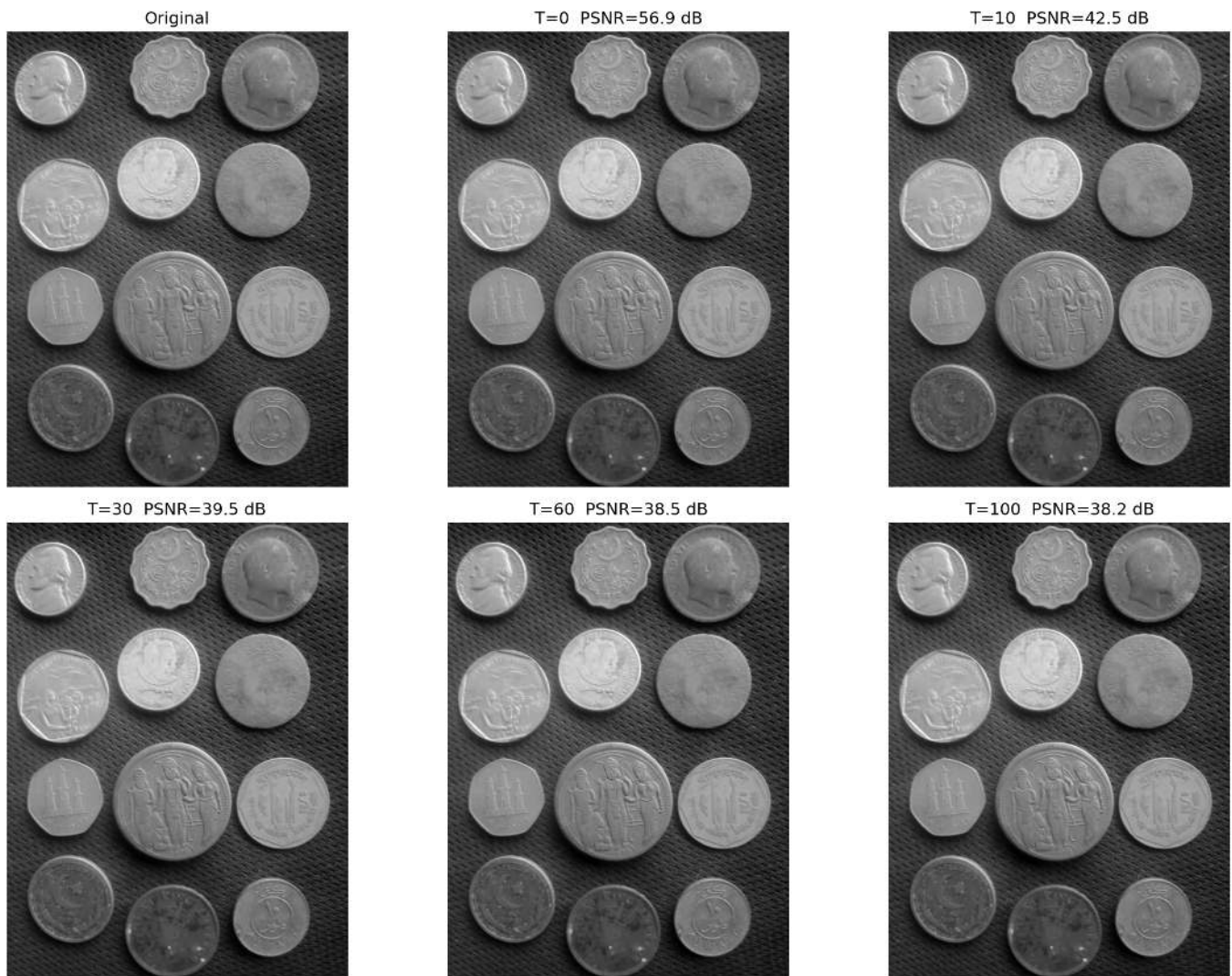


Figura 5.20: Reconstrução wavelet com limiamento de coeficientes (hard thresholding): à medida que o limiar aumenta, mais detalhes são zerificados, produzindo imagens progressivamente mais suaves. Métrica PSNR quantifica a perda de qualidade.

5.6.4 📌 Síntese - Fourier vs *Wavelets*: quando usar cada uma?

Critério	Fourier (DFT)	<i>Wavelet</i> (DWT)
Base	Senoides de suporte infinito	Funções compactas, localizadas
Localização espacial	— informação global	✓ — frequência e posição
Filtragem espectral	✓ — controle preciso de banda	Limitado
Compressão	DCT (JPEG) — excelente em blocos	DWT (JPEG 2000) — melhor para imagens inteiras
Análise multi-escala		✓ — hierarquia natural
Ruído periódico	✓ — localização exata no espectro	
Texturas não-estacionárias		✓

Regra prática: use Fourier para filtragem espectral e remoção de ruído periódico; use *wavelets* para compressão, análise multi-escala e processamento de sinais não-estacionários.

5.7 Compressão de Imagens

As wavelets estabelecem a fundação teórica do padrão JPEG 2000, mas o padrão JPEG original — dominante em fotografia digital — utiliza uma transformada mais simples e igualmente eficaz: a **Transformada de Cossenos Discreta (DCT)**. Ambas exploram o mesmo princípio: concentrar a energia da imagem em poucos coeficientes e descartar os demais com impacto visual mínimo.

A compressão visa reduzir a quantidade de dados necessária para armazenar ou transmitir uma imagem, explorando **redundâncias** presentes na representação original.

5.7.1 Taxonomia das Redundâncias

Três categorias principais de redundância são exploradas por algoritmos de compressão:

Tabela 5.2: Tipos de redundância em imagens e como são exploradas.

Tipo	Definição	Explorada por
Espacial (interpixel)	Pixels vizinhos são altamente correlacionados	DCT, codificação preditiva
Espectral (interchannel)	Canais de cor são correlacionados (R G B em imagens naturais)	Conversão YCbCr
Psicovisual	O sistema visual humano é insensível a certas variações de alta freq.	Quantização JPEG

A compressão é classificada em duas categorias fundamentais:

- **Sem perda (*lossless*):** reconstrução bit-a-bit idêntica ao original. Usada quando integridade é crítica (imagens médicas, documentos).
- **Com perda (*lossy*):** permite distorção controlada para ganhos maiores de compressão. Adequada para fotografia e vídeo, onde o SVH tolera imperfeições.

5.7.2 As Funções de Base da DCT

A **frequência espacial** ($u/M, v/N$) representa a taxa de variação espacial da intensidade ao longo da imagem.

Valores baixos correspondem a variações suaves e estruturas globais; valores altos correspondem a mudanças rápidas de intensidade, como bordas, texturas e detalhes finos.

Em termos discretos, os índices (u, v) indicam quantos ciclos da oscilação ocorrem ao longo da largura e da altura da imagem.

Cada bloco 8×8 da imagem é decomposto em uma combinação ponderada de **64 padrões de base** da DCT — funções cossenoidais de frequência crescente em x e em y . O coeficiente $C(u, v)$ representa o “peso” do padrão de frequência (u, v) naquele bloco específico.

A figura abaixo exhibe as 64 bases: o canto superior esquerdo corresponde ao componente DC (padrão uniforme); ao avançar para a direita e para baixo, a frequência espacial aumenta progressivamente.

5.7.3 Transformada de Cossenos Discreta (DCT)

A **Transformada de Cossenos Discreta (DCT)** é a operação central do padrão JPEG. Diferentemente da DFT (que usa base complexa), a DCT é composta por funções de base puramente reais, o que simplifica as implementações computacionais e favorece aplicações de compressão e compactação de energia em imagens naturais. Para um bloco $f(x, y)$ de dimensões $N \times N$, a **DCT-II 2D** é definida como:

$$C(u, v) = \alpha(u) \alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{\pi(2x+1)u}{2N} \right] \cos \left[\frac{\pi(2y+1)v}{2N} \right] \quad (5.9)$$

onde $\alpha(0) = \sqrt{1/N}$ e $\alpha(k) = \sqrt{2/N}$ para $k > 0$ (fator de normalização ortogonal). O coeficiente $C(0, 0)$ corresponde ao **componente DC**.

i DCT vs DFT: vantagem da compactação de energia

Tanto a DCT quanto a DFT transformam um bloco $N \times N$ em $N \times N$ coeficientes. A diferença crítica para compressão é a **compactação de energia**: para imagens naturais. Para imagens naturais, a DCT frequentemente apresenta melhor compactação de energia nos coeficientes de baixa frequência do que a DFT, favorecendo aplicações de compressão perceptual. Isso ocorre porque a DCT assume simetria par do sinal — equivalente a uma extensão periódica sem descontinuidade — reduzindo o *ringing* nas bordas do bloco. A consequência prática é que mais coeficientes DCT podem ser zerados sem degradação visível, resultando em maior compressão.

```
from scipy.fft import dct, idct # linha adicionada

fig, axes = plt.subplots(8, 8, figsize=(6, 6))
fig.subplots_adjust(hspace=0.05, wspace=0.05)
for i in range(8):
    for j in range(8):
        coef = np.zeros((8, 8)); coef[i, j] = 1
        b = idct(idct(coef.T, norm='ortho').T, norm='ortho')
        axes[i, j].imshow(b, cmap='gray')
        axes[i, j].axis('off')
plt.suptitle("As 64 Bases da DCT 8x8", y=0.92, fontsize=12, fontweight='bold')
plt.show()
```

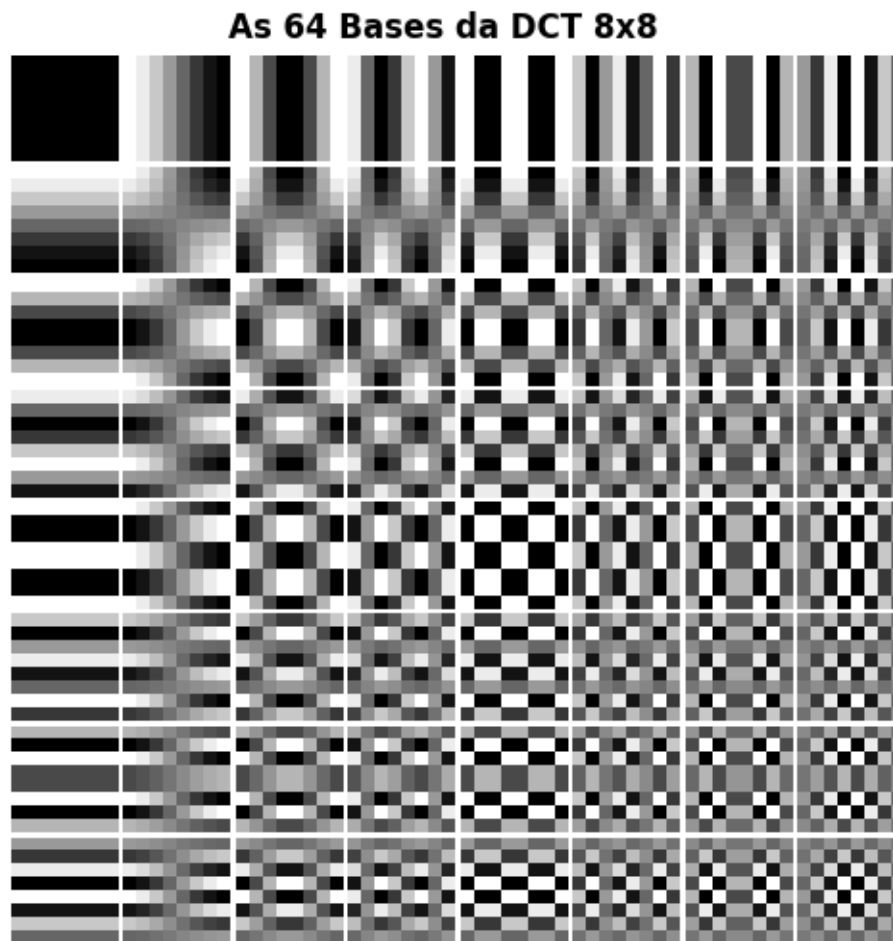


Figura 5.21: O Alfabeto Visual do JPEG: As 64 funções de base da DCT-II. O coeficiente DC fica no topo esquerdo (suave). Ao descer e avançar à direita, a oscilação espacial aumenta drasticamente.

5.7.4 Quantização: a principal fonte de compressão

Após a DCT, cada coeficiente $C(u, v)$ é dividido por um valor da **tabela de quantização** $Q(u, v)$ e arredondado para o inteiro mais próximo. Coeficientes de alta frequência — menos perceptíveis ao SVH — recebem valores altos em Q , forçando o resultado ao arredondamento para **zero**. Longas sequências de zeros são então comprimidas eficientemente pela codificação entrópica.

Quando o fator de qualidade é baixo, o descarte de altas frequências é agressivo: o bloco 8×8 é reconstruído com poucos coeficientes, o que produz os característicos **artefatos de bloco** visíveis nas fronteiras entre blocos adjacentes.

```
from scipy.fft import dct, idct

def dct2(bloco):
    """DCT-II 2D ortogonal (separável)."""
    return dct(dct(bloco.T, norm='ortho').T, norm='ortho')

def idct2(coefs):
    """IDCT-II 2D ortogonal."""
    return idct(idct(coefs.T, norm='ortho').T, norm='ortho')

# Bloco 8x8 centralizado da imagem
cy, cx = img_gray.shape[0]//2, img_gray.shape[1]//2
bloco = img_gray[cy:cy+8, cx:cx+8].astype(np.float64) - 128.0
```



```

C = dct2(bloco)

print("Coeficientes DCT do bloco 8x8:")
print(np.round(C).astype(int))
print(f"\nEnergia DC      : {C[0,0]**2:.1f}")
print(f"Energia total   : {(C**2).sum():.1f}")
print(f"Fração no DC    : {C[0,0]**2 / (C**2).sum():.1%} ← concentração de energia")

# Reconstrução progressiva
imgs_rec = [cv2.normalize((bloco+128).astype(np.uint8), None, 0, 255, cv2.NORM_MINMAX)]
titles_rec = ["Bloco original\n(8x8 pixels)"]

for keep in [1, 4, 10, 20, 40, 64]:
    C_trunc = np.zeros_like(C)
    indices = sorted([(u,v) for u in range(8) for v in range(8)], key=lambda p: p[0]+p[1])
    for u, v in indices[:keep]:
        C_trunc[u, v] = C[u, v]
    rec = np.clip(idct2(C_trunc) + 128, 0, 255).astype(np.uint8)
    imgs_rec.append(rec)
    titles_rec.append(f"{keep} coef.\n({keep/64:.0%} do total)")

mm.show(imgs_rec, titles=titles_rec, cols=4, figsize=(12, 7))

```

Coeficientes DCT do bloco 8x8:

```

[[192 -48  1  8  0 -1  0  0]
 [-96  54  7 -10  0  0  0  0]
 [ 14 -14  8  0  0  0  0  0]
 [  0  0 -11  1  0  0  0  0]
 [  9 -11  0  0  1  0  1  0]
 [  0  0  0  0  0  0  0  0]
 [  0  0  0  0 -1  0  0  0]
 [  0  0  0  0  0  0  0  0]]

```

```

Energia DC      : 36816.0
Energia total   : 52253.0
Fração no DC    : 70.5% ← concentração de energia

```

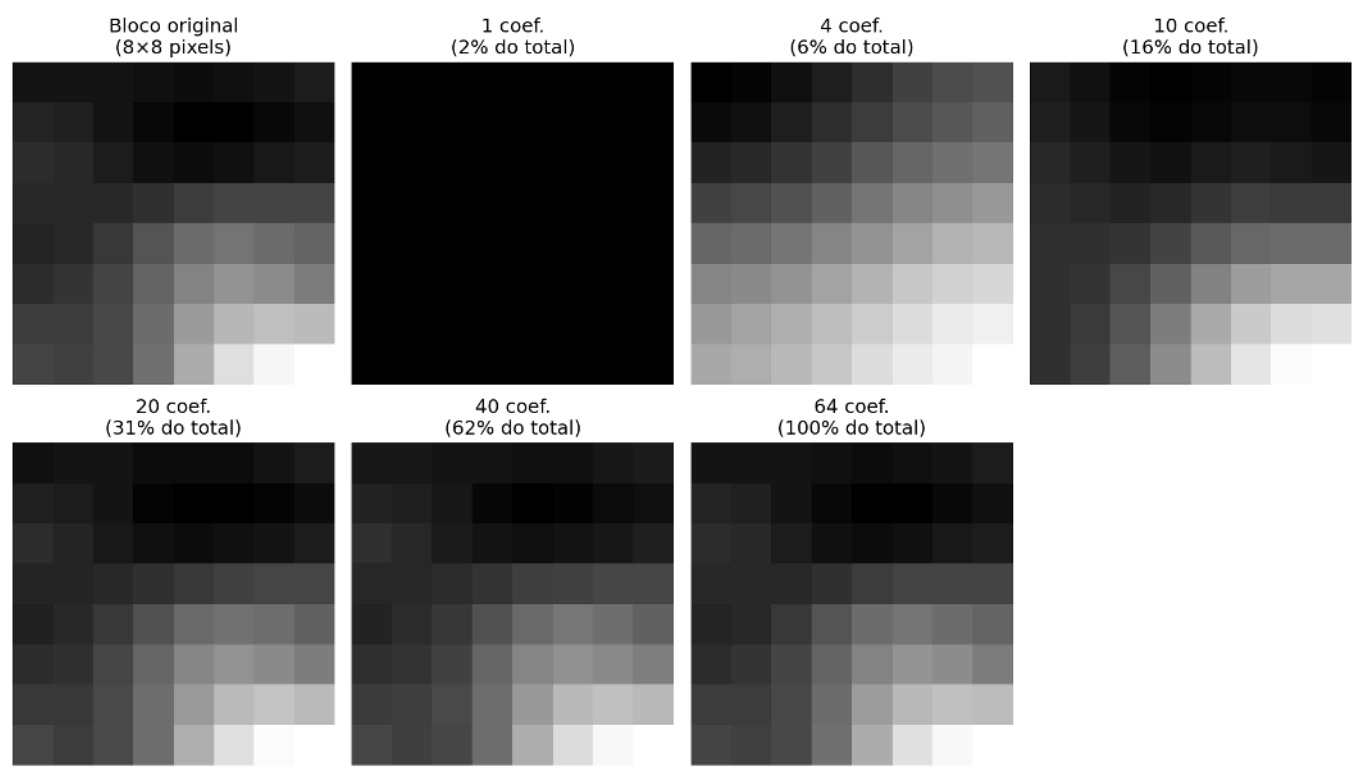
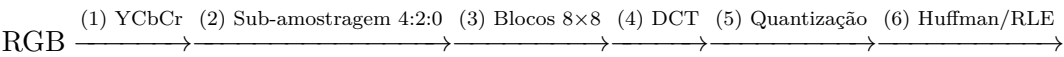


Figura 5.22: DCT 2D em bloco 8×8: coeficientes e reconstrução progressiva.

5.7.5 Pipeline JPEG

O padrão JPEG aplica a DCT em blocos disjuntos de 8 × 8 pixels. O pipeline completo envolve seis etapas principais:



Etapa		Operação	Fundamento perceptual
1		RGB → YCbCr: separa luminância (Y) de crominância (Cb, Cr)	O SVH é ~4× mais sensível a variações de luminância do que de cor
2		Sub-amostragem 4:2:0: Cb e Cr reduzidos à metade da resolução	Elimina ~50% dos dados de cor com impacto visual mínimo
3–4		Blocos 8×8 centrados em zero e transformados pela DCT	Concentra energia nos primeiros coeficientes
5		Divisão por $Q(u, v)$ e arredondamento: $\tilde{C}(u, v) = \text{round}[C(u, v)/Q(u, v)]$	Zeros em altas frequências → principal fonte de compressão
6		Varredura zigzag + RLE + Huffman	Comprime longas sequências de zeros resultantes da quantização

A **tabela de quantização** Q é o parâmetro central do compromisso qualidade-compressão: o fator de qualidade JPEG (1–100) escala Q globalmente — valores altos preservam mais coeficientes; valores baixos geram mais zeros e maiores artefatos.

Por que o Ziguezague? A DCT acumula a energia vital no canto superior esquerdo (baixas frequências) e empurra os zeros para a direita e para baixo. Ler a matriz na diagonal (ziguezague) cria uma “corrida de zeros” contínua no final do vetor, permitindo que o algoritmo simplesmente anote “e os próximos 45 elementos são zero”, comprimindo o dado violentamente (*Run-Length Encoding*).

```
# Tabela de quantização luminância (padrão JPEG)
Q_luma = np.array([
    [16,11,10,16,24,40,51,61],
    [12,12,14,19,26,58,60,55],
    [14,13,16,24,40,57,69,56],
    [14,17,22,29,51,87,80,62],
    [18,22,37,56,68,109,103,77],
    [24,35,55,64,81,104,113,92],
    [49,64,78,87,103,121,120,101],
    [72,92,95,98,112,100,103,99]
], dtype=np.float64)

def jpeg_compress_block(bloco, Q_table):
    """DCT → quantização → dequantização → IDCT em bloco 8×8."""
    C = dct2(bloco.astype(np.float64) - 128)
    Cq = np.round(C / Q_table) * Q_table # quantiza e dequantiza
    return np.clip(idct2(Cq) + 128, 0, 255)

def jpeg_quality_compress(img, qualidade=50):
    """JPEG simplificado: comprime imagem inteira por blocos 8×8."""
    # Fator de escala: qualidade 50 = sem escala; >50 = maior qualidade
    if qualidade < 50:
        escala = 5000 / qualidade
    else:
        escala = 200 - 2 * qualidade
    Q = np.clip(np.round(Q_luma * escala / 100), 1, 255)

    h, w = img.shape
    result = np.zeros_like(img, dtype=np.float64)
    for r in range(0, h-7, 8):
        for c in range(0, w-7, 8):
            result[r:r+8, c:c+8] = jpeg_compress_block(img[r:r+8, c:c+8], Q)
    return result.astype(np.uint8)

# Comparação de fatores de qualidade
qualidades = [10, 25, 50, 75, 90]
imgs_jpeg = [img_gray]
titles_jpeg = ["Original"]

for q in qualidades:
    rec = jpeg_quality_compress(img_gray, qualidade=q)
    psnr = cv2.PSNR(img_gray, rec)
    imgs_jpeg.append(rec)
    titles_jpeg.append(f"Q={q} PSNR={psnr:.1f}dB")

mm.show(imgs_jpeg, titles=titles_jpeg, cols=3, figsize=(14, 10))
```

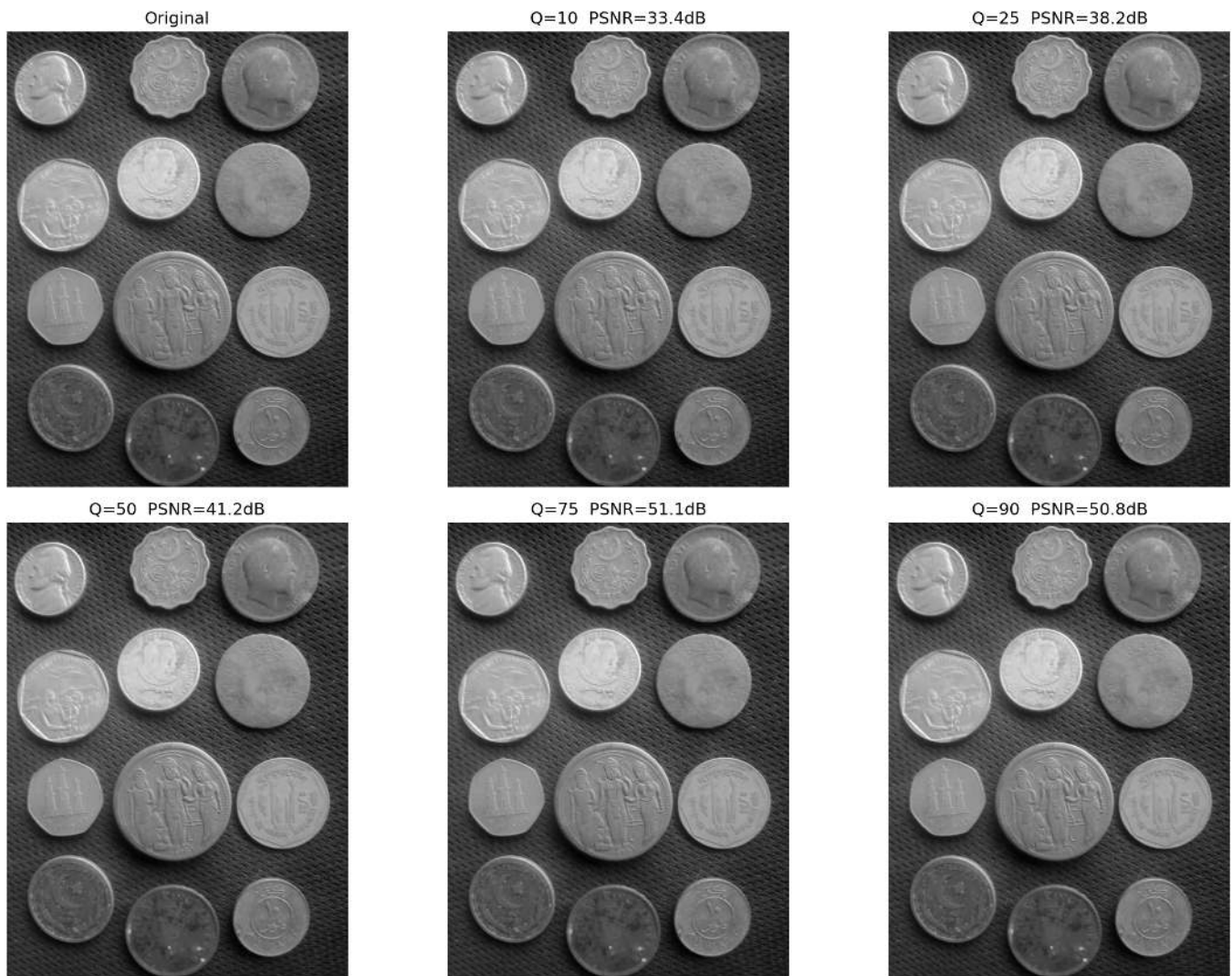


Figura 5.23: Pipeline JPEG simplificado: DCT em blocos 8×8 , quantização com diferentes fatores de qualidade e reconstrução via IDCT. O artefato de blocos torna-se visível para qualidades baixas ($Q=10-20$).

5.7.6 Simulador Interativo: Quantização DCT

O simulador abaixo permite explorar o efeito da quantização sobre um bloco 8×8 real, visualizando em tempo real:

- os **coeficientes DCT** (mapa de calor — quanto maior, mais quente);
- os **coeficientes quantizados** (observe os zeros aparecerem com qualidade baixa);
- o **bloco reconstruído** e o erro de quantização.

5.8 Comparação de Formatos de Imagem

A escolha do formato de arquivo impacta diretamente o compromisso entre qualidade, tamanho e velocidade de decodificação. Os três formatos mais relevantes para aplicações web e computação visual são JPEG, PNG e WebP.

5.8.1 Características dos Formatos



Figura 5.24: Simulador interativo de compressão DCT-JPEG: ajuste o fator de qualidade e visualize em tempo real os coeficientes zerados, o bloco reconstruído e o erro de quantização.

Tabela 5.3: Comparação entre os principais formatos de imagem rasterizados.

Característica	JPEG	PNG	WebP
Compressão	Com perda	Sem perda	Com e sem perda
Transparência (alfa)		✓	✓
Suporte a animação		Limitado	✓
Algoritmo base	DCT + Huffman	DEFLATE (LZ77 + Huffman)	VP8 / VP8L
Melhor para	Fotografia	Gráficos, texto, ícones	Web (substituto universal)
Pior para	Texto, bordas nítidas	Fotos de alta resolução	Compatibilidade legada

5.8.2 Métricas de Qualidade

Duas métricas objetivas são amplamente usadas para avaliar a qualidade de imagens comprimidas:

PSNR (Peak Signal-to-Noise Ratio):

$$\text{PSNR} = 10 \log_{10} \left(\frac{L^2}{\text{MSE}} \right) \quad [\text{dB}] \tag{5.10}$$

onde $L = 255$ para imagens de 8 bits e MSE é o erro quadrático médio. Valores típicos: PSNR > 40 dB indica qualidade excelente; 30–40 dB, qualidade boa; < 30 dB, degradação visível.

SSIM (Structural Similarity Index):

$$\text{SSIM}(f, g) = \frac{(2\mu_f\mu_g + c_1)(2\sigma_{fg} + c_2)}{(\mu_f^2 + \mu_g^2 + c_1)(\sigma_f^2 + \sigma_g^2 + c_2)} \tag{5.11}$$

O SSIM combina três fatores medidos em janelas locais: **luminância** (μ_f, μ_g), **contraste** (σ_f, σ_g) e **estrutura** (σ_{fg}), ponderados por constantes de estabilidade c_1, c_2 . O resultado varia em $[-1, 1]$, com 1 indicando identidade perfeita. Diferentemente do PSNR, o SSIM é sensível à organização espacial dos erros,

sendo mais alinhado com a percepção humana — consulte Gonzalez; Woods (2018) para a formulação completa.

i PSNR vs SSIM: qual usar?

O PSNR é simples e rápido, mas pode superestimar a qualidade em imagens com artefatos localizados (blocos JPEG) ou subestimá-la quando o ruído é perceptualmente irrelevante (diferença de brilho global). O SSIM captura melhor a percepção humana, mas é mais caro computacionalmente. Para avaliação rigorosa de algoritmos de compressão, recomenda-se reportar ambas as métricas, além de avaliação subjetiva em estudos perceptuais.

5.8.3 Inspeção Visual: Qualidade não é só PSNR

A filosofia de compressão dita o tipo de degradação. O JPEG corta a imagem em quadrados (DCT), gerando “Mosaicos”. Formatos baseados em wavelets ou filtros preditivos modernos (como WebP/JPEG2000) evitam blocos, mas sofrem de perda de textura e “derretimento” (borramento).

```
# Cria o diretório e garante a gravação dos arquivos comprimidos em Q=10 antes da leitura
os.makedirs("imagens/comp_test", exist_ok=True)
cv2.imwrite("imagens/comp_test/coins_q10.jpg", img_gray, [cv2.IMWRITE_JPEG_QUALITY, 10])
cv2.imwrite("imagens/comp_test/coins_q10.webp", img_gray, [cv2.IMWRITE_WEBP_QUALITY, 10])

# Recorte de área de alto contraste das imagens já comprimidas (Dando zoom de 4x)
zoom_original = cv2.resize(img_gray[120:200, 150:230], (320, 320),
                           interpolation=cv2.INTER_NEAREST)

rec_jpeg = cv2.imread("imagens/comp_test/coins_q10.jpg", 0)
zoom_jpeg = cv2.resize(rec_jpeg[120:200, 150:230], (320, 320),
                      interpolation=cv2.INTER_NEAREST)

rec_webp = cv2.imread("imagens/comp_test/coins_q10.webp", 0)
zoom_webp = cv2.resize(rec_webp[120:200, 150:230], (320, 320),
                       interpolation=cv2.INTER_NEAREST)

mm.show([zoom_original, zoom_jpeg, zoom_webp],
        titles=["Zoom Original", "JPEG Q=10 (DCT Blocos)", "WebP Q=10 (Suavização)"],
        cols=3, figsize=(14, 5))
```

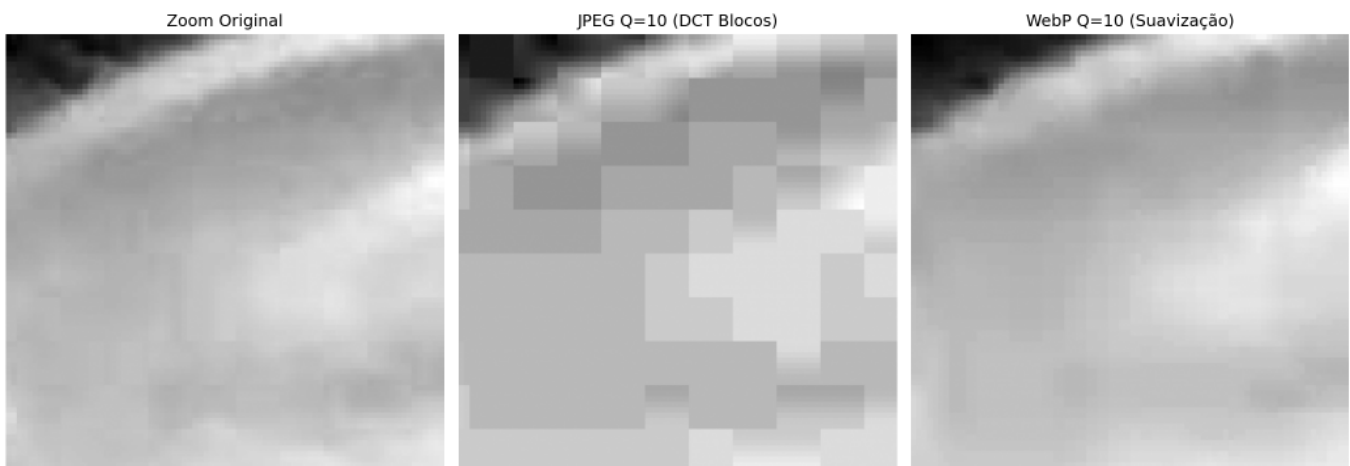


Figura 5.25: Forte compressão (Q=10). À esquerda o artefato de bloco clássico da DCT. À direita, a suavização de bordas característica do WebP.


```

os.makedirs("imagens/comp_test", exist_ok=True)
resultados = []

# JPEG
for q in [10, 20, 30, 40, 50, 60, 70, 80, 90, 95]:
    path = f"imagens/comp_test/coins_q{q}.jpg"
    cv2.imwrite(path, img_gray, [cv2.IMWRITE_JPEG_QUALITY, q])
    rec = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    resultados.append({"formato": "JPEG", "qualidade": q,
                      "PSNR": cv2.PSNR(img_gray, rec),
                      "KB": os.path.getsize(path)/1024})

# PNG
path_png = "imagens/comp_test/coins.png"
cv2.imwrite(path_png, img_gray, [cv2.IMWRITE_PNG_COMPRESSION, 9])
resultados.append({"formato": "PNG", "qualidade": "lossless",
                  "PSNR": float('inf'), "KB": os.path.getsize(path_png)/1024})

# WebP
for q in [50, 75, 90]:
    path_w = f"imagens/comp_test/coins_q{q}.webp"
    cv2.imwrite(path_w, img_gray, [cv2.IMWRITE_WEBP_QUALITY, q])
    rec_w = cv2.imread(path_w, cv2.IMREAD_GRAYSCALE)
    resultados.append({"formato": "WebP", "qualidade": q,
                      "PSNR": cv2.PSNR(img_gray, rec_w),
                      "KB": os.path.getsize(path_w)/1024})

# Curva taxa-distorção
jpeg_r = [r for r in resultados if r["formato"]=="JPEG"]
webp_r = [r for r in resultados if r["formato"]=="WebP"]
png_r = [r for r in resultados if r["formato"]=="PNG"]

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot([r["KB"] for r in jpeg_r], [r["PSNR"] for r in jpeg_r],
        "o-", label="JPEG", color="#D85A30", lw=2, ms=5)
ax.plot([r["KB"] for r in webp_r], [r["PSNR"] for r in webp_r],
        "s-", label="WebP", color="#534AB7", lw=2, ms=5)
ax.axhline(50, color="#1D9E75", lw=2, ls="--",
           label=f"PNG sem perda ({png_r[0]['KB']:.0f} KB)")
ax.axhspan(40, 60, alpha=0.05, color="#1D9E75", label="Qualidade excelente (PSNR>40)")
ax.set(xlabel="Tamanho do arquivo (KB)", ylabel="PSNR (dB)",
       title="Curva Taxa-Distorção: JPEG × WebP × PNG")
ax.legend(fontsize=9); ax.grid(True, alpha=0.3); plt.tight_layout()
plt.show()

print(f"\nTamanho bruto (sem compressão): {img_gray.nbytes/1024:.0f} KB")
print(f"\n{'Formato':>8} {'Qual.':>6} {'KB':>7} {'PSNR (dB)':>11}")
print("-"*38)
for r in resultados:
    psnr_s = f"{r['PSNR']:>11.2f}" if r['PSNR']!=float('inf') else f"∞ (lossless)':>11}"
    print(f"{r['formato']:>8} {str(r['qualidade']):>6} {r['KB']:>7.1f} {psnr_s}")

```

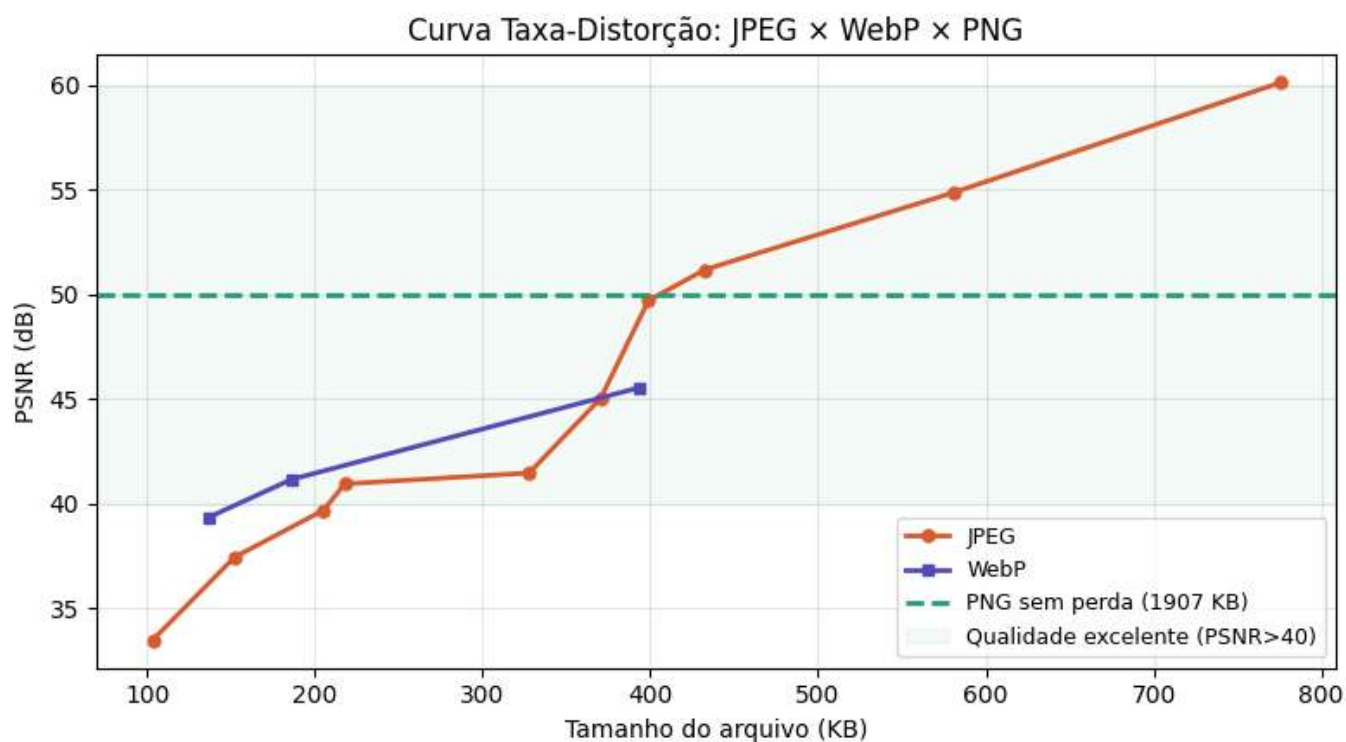



Figura 5.26: Curva taxa-distorção: PSNR vs tamanho de arquivo para JPEG, WebP e PNG.

Tamanho bruto (sem compressão): 4800 KB

Formato	Qual.	KB	PSNR (dB)
JPEG	10	103.3	33.50
JPEG	20	152.0	37.45
JPEG	30	205.1	39.70
JPEG	40	217.8	40.96
JPEG	50	327.7	41.48
JPEG	60	370.4	45.04
JPEG	70	398.8	49.71
JPEG	80	432.4	51.19
JPEG	90	581.0	54.88
JPEG	95	775.3	60.12
PNG lossless		1906.8	(lossless)
WebP	50	137.0	39.37
WebP	75	185.9	41.16
WebP	90	393.1	45.55

```
try:
    from skimage.metrics import structural_similarity as ssim
except ImportError:
    import subprocess; subprocess.run(["pip", "install", "scikit-image", "-q"])
    from skimage.metrics import structural_similarity as ssim

qualidades_ssim = [25, 50, 75, 95]
imgs_ssim = [img_gray]
titles_ssim = ["Original"]

for q in qualidades_ssim:
    path = f"imagens/comp_test/coins_q{q}.jpg"
    rec = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if rec is None: continue
```

```

if rec.shape != img_gray.shape:
    rec = cv2.resize(rec, (img_gray.shape[1], img_gray.shape[0]))

psnr_v = cv2.PSNR(img_gray, rec)
ssim_v, _ = ssim(img_gray, rec, full=True)
diff_vis = cv2.normalize(np.abs(img_gray.astype(float)-rec.astype(float)),
                        None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

imgs_ssim += [rec, diff_vis]
titles_ssim += [f"Q={q} PSNR={psnr_v:.1f}dB\nSSIM={ssim_v:.3f}",
               f"Mapa de erro (Q={q})\n(bordas = artefatos de bloco)"]

mm.show(imgs_ssim, titles=titles_ssim, cols=3, figsize=(14, 14))

```

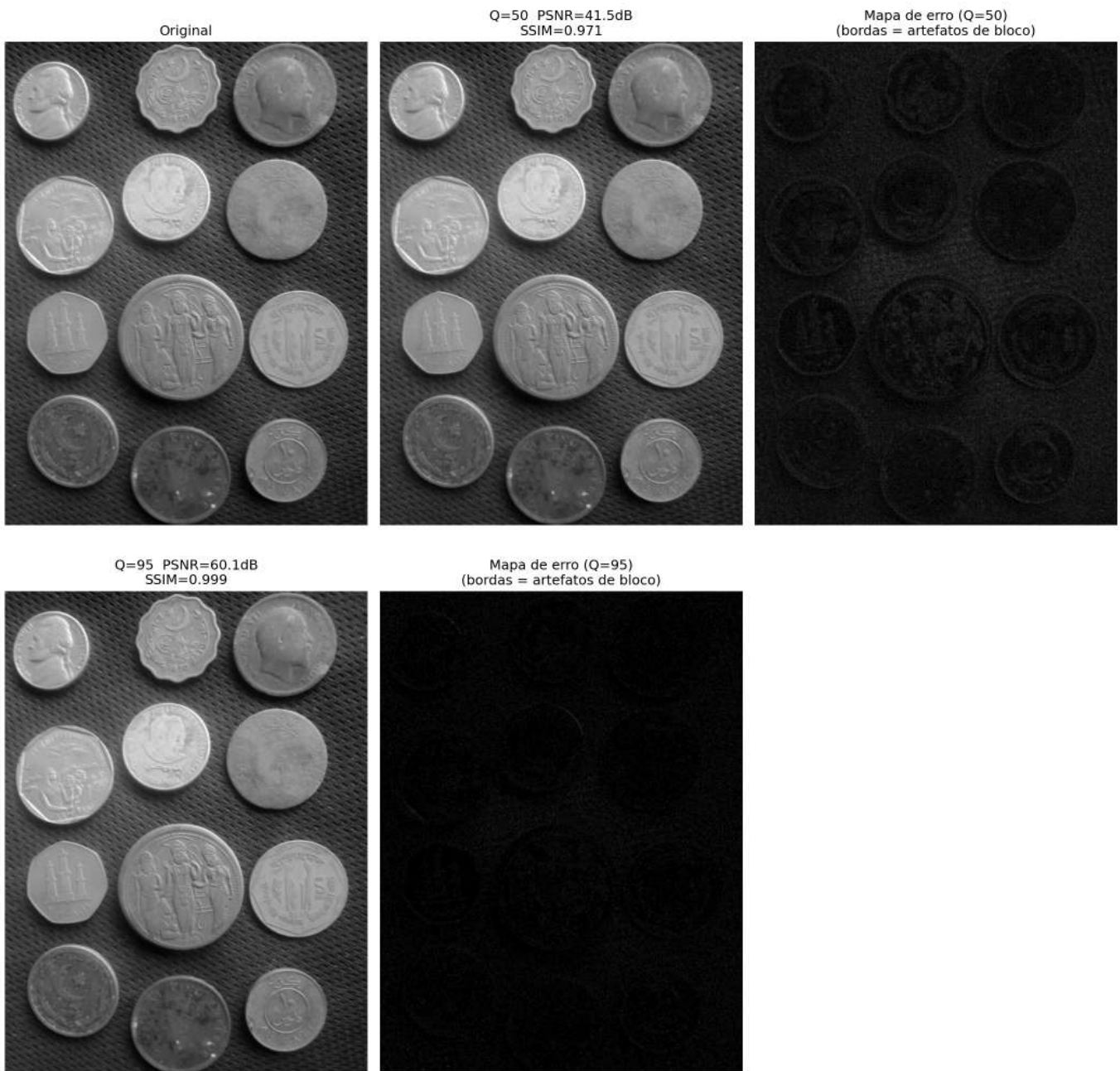


Figura 5.27: Mapas de erro JPEG em diferentes qualidades: artefatos de bloco nas bordas.

Síntese — Compressão JPEG

Etapa	O que faz	Ganho
YCbCr	Separa luminância de crominância	Modela percepção humana
Subamostragem 4:2:0	Reduz Cb/Cr à metade	~50% de dados a menos
DCT 8×8	Concentra energia nos primeiros coeficientes	Alta compactação
Quantização	Zera coeficientes imperceptíveis	Principal fonte de compressão
Huffman	Codifica estatisticamente os zeros	~30-50% adicional

Artefatos típicos de JPEG: - **Artefato de bloco** (qualidades baixas): fronteiras 8×8 visíveis; - **Ringings** (qualidades muito baixas): oscilações ao redor de bordas nítidas; - **Perda de textura** (qualidades baixas): regiões com textura fina ficam “plásticas”.

5.9 Aplicação Prática: Remoção de Ruído por Filtragem Espectral

Reunindo as técnicas do capítulo, apresenta-se um *pipeline* completo de **remoção de ruído** que combina análise no domínio da frequência com filtragem adaptativa. O objetivo é remover um ruído misto (Gaussiano + periódico) preservando ao máximo a estrutura original da imagem.

Ruidosa $\xrightarrow{\text{FFT}}$ $\xrightarrow{\text{Identif. picos}}$ $\xrightarrow{\text{Filtro Butterworth + Notch}}$ $\xrightarrow{\text{IFFT}}$ Restaurada

Avaliação: PSNR e SSIM

O par de métricas PSNR / SSIM fornece uma avaliação complementar: - **PSNR** penaliza uniformemente todos os erros pixel a pixel. - **SSIM** avalia a preservação de estruturas perceptualmente relevantes. Um bom filtro maximiza ambos, mas na prática existe um compromisso: filtros muito agressivos reduzem o ruído de alta frequência mas destroem bordas e texturas, o que pode melhorar o PSNR em relação à imagem ruidosa mas reduz o PSNR em relação à imagem original e degrada o SSIM.

```
try:
    from skimage.metrics import structural_similarity as ssim_sk
except ImportError:
    import subprocess
    subprocess.run(["pip", "install", "scikit-image", "-q"])
    from skimage.metrics import structural_similarity as ssim_sk

def suprimir_pico_gaussiano(mask, cy, cx, r, sigma=3.0):
    yy, xx = np.ogrid[:mask.shape[0], :mask.shape[1]]
    dist = np.sqrt((yy - cy)**2 + (xx - cx)**2)
    notch = np.exp(-dist**2 / (2 * sigma**2))
    mask *= (1 - notch)          # multiplicação correta
    return mask

# 1. Construção do ruído misto
np.random.seed(42)
h_img, w_img = img_gray.shape
X2, Y2 = np.meshgrid(np.arange(w_img), np.arange(h_img))

u0, v0      = 15, 10
ruído_gauss = np.random.normal(0, 15, img_gray.shape)
ruído_period = 30 * np.sin(2 * np.pi * (u0 * X2 / w_img + v0 * Y2 / h_img))
img_noisy   = np.clip(img_gray.astype(float) +
                      ruído_gauss + ruído_period, 0, 255).astype(np.uint8)
```

```

# 2. Espectro e identificação dos picos
F_n = np.fft.fftshift(np.fft.fft2(img_noisy.astype(np.float64)))
mag_n = cv2.normalize(np.log1p(np.abs(F_n)), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# 3. Notch gaussiano nos picos periódicos
cy0, cx0 = h_img // 2, w_img // 2
mascara_notch = np.ones((h_img, w_img), dtype=np.float64)
for dy, dx in [(+v0, +u0), (-v0, -u0), (+v0, -u0), (-v0, +u0)]:
    mascara_notch = suprimir_pico_gaussiano(mascara_notch, cy0 + dy, cx0 + dx, r=5)

img_notch = np.real(np.fft.ifft2(np.fft.ifftshift(F_n * mascara_notch)))
img_notch = np.clip(img_notch, 0, 255).astype(np.uint8)

# 4. Bilateral: remove gaussiano residual preservando bordas
img_den = cv2.bilateralFilter(img_notch, d=7, sigmaColor=25, sigmaSpace=7)

# Métricas nas três etapas
psnr_n, ssim_n = cv2.PSNR(img_gray, img_noisy), ssim_sk(img_gray, img_noisy)
psnr_no, ssim_no = cv2.PSNR(img_gray, img_notch), ssim_sk(img_gray, img_notch)
psnr_d, ssim_d = cv2.PSNR(img_gray, img_den), ssim_sk(img_gray, img_den)

print(f"{'Etapa':>20} | {'PSNR (dB)':>9} | {'SSIM':>6}")
print("-" * 42)
print(f"{'Ruidosa (gauss+per)':>20} | {psnr_n:>9.2f} | {ssim_n:>6.4f}")
print(f"{'Após notch':>20} | {psnr_no:>9.2f} | {ssim_no:>6.4f}")
print(f"{'Notch + bilateral':>20} | {psnr_d:>9.2f} | {ssim_d:>6.4f}")

mascara_vis = (mascara_notch * 255).astype(np.uint8)
mm.show(
    [img_gray, img_noisy, mag_n, mascara_vis, img_notch, img_den],
    titles=[
        "Original",
        f"Ruidosa\nPSNR={psnr_n:.1f} dB",
        "Espectro\n(picos visíveis)",
        "Máscara notch\n(gaussiana suave)",
        f"Após notch\nPSNR={psnr_no:.1f} dB",
        f"Notch + bilateral\nPSNR={psnr_d:.1f} dB SSIM={ssim_d:.3f}"
    ],
    cols=6, figsize=(20, 4)
)

```

Etapa	PSNR (dB)	SSIM
Ruidosa (gauss+per)	19.97	0.3478
Após notch	24.55	0.3777
Notch + bilateral	31.55	0.7892

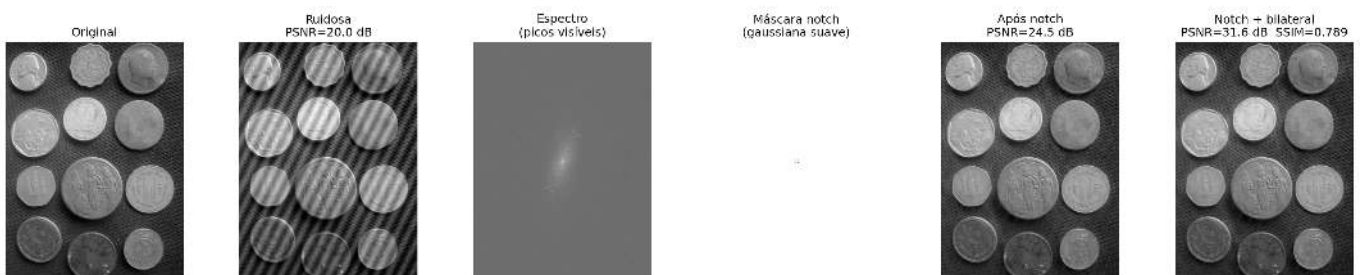


Figura 5.28: Pipeline completo de remoção de ruído misto: (1) ruído gaussiano + periódico adicionado; (2) picos periódicos identificados no espectro; (3) filtro notch de supressão suave (Gaussiana) aplicado; (4) filtro bilateral remove o gaussiano residual.

5.10 Resumo do Capítulo

A transição do **domínio espacial** para o **domínio da frequência** revela a distribuição de energia da imagem, estabelecendo a base matemática para filtragem avançada e compressão de dados. Ver um mapa conceitual do domínio da frequência na Figure 5.29.

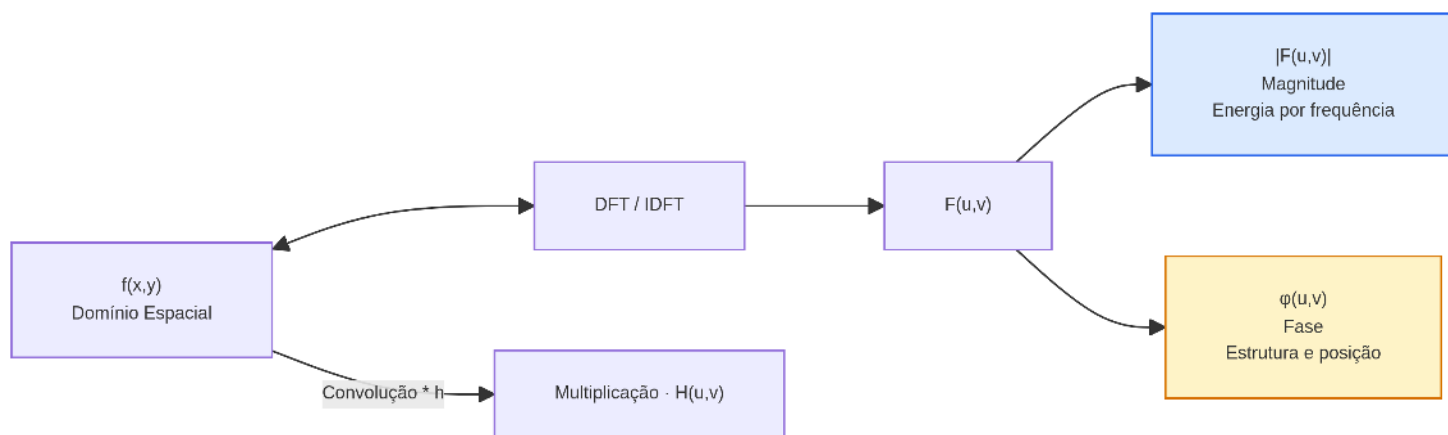


Figura 5.29: Mapa conceitual do domínio da frequência.

🔑 Fundamentos Essenciais

- **DFT e Percepção Visual:** O espectro decompõe a imagem em ondas. A **fase** carrega a inteligibilidade da forma e os contornos; a **magnitude** dita apenas a distribuição de contraste.
- **Eficiência Algorítmica:** O Teorema da Convolução permite que filtros de grande escala sejam aplicados via FFT, despendendo a complexidade de $O(N^2K^2)$ para $O(N^2 \log N)$.
- **O Artefato do Ringing:** Cortes abruptos de frequência (Filtro Ideal) geram ondulações indesejadas no espaço. Filtros **Gaussianos** e **Butterworth** garantem transições suaves.
- **A Era das Wavelets:** Superando a análise global de Fourier, as Wavelets capturam frequência e localização espacial simultaneamente (multirresolução), fundamentando o JPEG 2000 e influenciando técnicas modernas de análise multi-escala, incluindo representações hierárquicas usadas em CNNs.
- **Compressão DCT (JPEG):** O algoritmo explora a cegueira humana a altas frequências espaciais. A DCT compacta a energia visual em blocos de 8×8 , quantizando e descartando detalhes finos (o que gera artefatos de bloco em qualidades baixas).

Próximos Passos: O Capítulo 6 inicia a Parte II da obra, estendendo estas ferramentas aos **Espaços de Cor** e inaugurando os conceitos de **Visão Computacional**.

5.11 🤖 Uso do NotebookLM como Tutor Complementar

Nesta edição, incentiva-se o uso do **NotebookLM** como ferramenta complementar de aprendizagem. Baseado em inteligência artificial, o sistema utiliza exclusivamente os documentos fornecidos pelo autor como fonte de conhecimento, produzindo respostas alinhadas ao conteúdo e à abordagem adotada ao longo do livro.

🎓 Estude com o Tutor Inteligente

🚀 [ACESSAR NOTEBOOKLM: CAPÍTULO 05](#)

⚠️ **Aviso sobre Conteúdo Gerado por IA**

A IA é uma poderosa aliada nos estudos, mas o conteúdo gerado pode conter **erros ou imprecisões**. Consulte sempre **livros, artigos científicos e outras fontes acadêmicas confiáveis** para validar as informações. Sempre que possível, execute os exemplos práticos fornecidos neste capítulo para verificar

os resultados.

5.12 Lista de Exercícios

1. (10%) Implemente a DFT 2D manualmente (sem `np.fft.fft2`) usando a definição da Equation 5.1 para uma imagem 16×16 . Compare com `np.fft.fft2` e verifique que as diferenças absolutas são menores que 10^{-8} . Meça o tempo de execução de ambas as implementações e explique a diferença.
2. (15%) Adicione ruído senoidal com frequências $(u_0, v_0) \in \{(5, 10), (20, 5), (30, 30)\}$ à imagem de moedas. Para cada frequência, projete um filtro notch e remova o ruído. Avalie quantitativamente a restauração com PSNR e SSIM. Discuta o compromisso entre remoção de ruído e preservação de detalhes.
3. (15%) Compare os filtros passa-baixa Ideal, Gaussiano e Butterworth (ordens $n = 1, 2, 4$) com $D_0 = 20, 40, 60$ pixels. Para cada combinação, calcule o PSNR e o SSIM da imagem filtrada em relação à original. Apresente os resultados em uma tabela e plote as curvas de resposta em frequência $H(u, 0)$.
4. (15%) Implemente a decomposição wavelet 2D manualmente usando a wavelet Haar: calcule os filtros h (passa-baixa) e g (passa-alta) de Haar, aplique-os por separabilidade em linhas e colunas, e subamostrasse por 2. Compare o resultado com `pywt.dwt2(img, 'haar')` e verifique a equivalência numérica.
5. (15%) Aplique o limiamento de coeficientes wavelet (*wavelet thresholding*) com limiares $T \in \{5, 10, 20, 40, 80\}$ e wavelets Haar, db4 e sym4. Para cada combinação, calcule PSNR e SSIM após reconstrução com `pywt.waverec2`. Identifique a combinação wavelet $\times$ limiar que maximiza o SSIM.
6. (15%) Implemente o pipeline JPEG completo incluindo: conversão RGB $\rightarrow$ YCbCr, subamostragem 4:2:0 de Cb/Cr, DCT em blocos 8×8 , quantização com tabela padrão escalada por fator de qualidade, e reconstrução com IDCT. Compare os resultados com `cv2.imencode('.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, q])` para qualidades 20, 50 e 80.
7. (15%) Crie uma imagem sintética composta por três regiões: fotografia de textura natural (canto superior esquerdo), região de texto nítido (centro) e região de gradiente suave (canto inferior direito). Compare JPEG, PNG e WebP para essa imagem mista em termos de PSNR, SSIM e tamanho de arquivo. Justifique qual formato é mais adequado considerando o conteúdo heterogêneo.

Referências do Capítulo

A fundamentação teórica deste capítulo baseia-se nas seguintes obras:

- Gonzalez; Woods (2018) para os conceitos de DFT 2D, filtros no domínio da frequência, DCT e fundamentos de compressão de imagens.
- Oppenheim (1999) para a teoria de sinais e sistemas no domínio discreto, incluindo o Teorema da Convolução e propriedades da DFT.
- Mallat (1999) para a teoria de wavelets, análise multirresolução e bancos de filtros.
- Wallace (1991) para o padrão de compressão JPEG original e o fundamento da quantização DCT.
- Szeliski (2022) para métricas de qualidade (PSNR, SSIM) e comparação de formatos de imagem modernos.

Parte II

Parte II — Visão Computacional

Capítulo 6

Inspeção Industrial e Análise de Documentos

 Executar  Colab  Abrir si-md2  GitHub

Na **Parte I — Processamento Digital de Imagens (PDI)**, foram estudadas técnicas para transformação e aprimoramento de imagens, como operações morfológicas, filtragem espacial, convoluções, limiarização, segmentação e processamento no domínio da frequência.

A **Parte II — Visão Computacional (VC)** amplia esse escopo ao tratar da interpretação automática do conteúdo visual, envolvendo a extração de informações, o reconhecimento de padrões e a tomada de decisões a partir de imagens.

Este capítulo apresenta essa transição por meio de duas aplicações representativas:

1. **Inspeção Industrial Automatizada**, voltada ao controle de qualidade e à detecção de defeitos em linhas de produção;
2. **Análise Automatizada de Documentos**, aplicada ao processamento de formulários, avaliações e outros documentos estruturados por meio de sistemas de reconhecimento óptico de marcas (*Optical Mark Recognition* – OMR).

Essas aplicações integram técnicas de detecção de estruturas geométricas, extração de descritores invariantes, reconhecimento de padrões e classificação de objetos, constituindo a base de diversos sistemas modernos de inspeção visual e automação.

6.1 Objetivos do Capítulo

Ao final deste capítulo, o estudante deverá ser capaz de:

- Aplicar técnicas de **alinhamento e retificação geométrica de documentos** utilizando a Transformada de Hough e transformações projetivas;
- **Detectar e segmentar regiões de interesse** com base em operações morfológicas, contornos e propriedades geométricas;
- **Decodificar marcadores bidimensionais e códigos de barras**, integrando bibliotecas de Visão Computacional a *pipelines* de processamento documental;
- **Implementar sistemas de reconhecimento óptico de marcas (OMR)** para leitura automatizada de avaliações e formulários;
- **Extrair informações estruturadas** a partir da análise de documentos digitalizados;
- **Avaliar a robustez de algoritmos** frente a ruídos, variações de iluminação e distorções geométricas;
- **Desenvolver *pipelines* completos de Visão Computacional** para análise automatizada de documentos e inspeção visual.

Este capítulo marca a transição do **Processamento Digital de Imagens**, voltado à transformação de imagens, para a **Visão Computacional**, cujo objetivo é interpretar o conteúdo visual, extrair informações relevantes e utilizá-las na tomada de decisões automatizadas.

6.2 Configuração do Ambiente

Os exemplos deste capítulo utilizam bibliotecas amplamente empregadas em Processamento Digital de Imagens e Visão Computacional. O bloco abaixo instala os pacotes necessários; em ambientes que já os possuam, a execução pode ser ignorada.

```
# Instalar apenas as bibliotecas ausentes
import importlib
import subprocess
import sys

for mod, pkg in {
    "cv2": "opencv-python",
    "skimage": "scikit-image",
    "numpy": "numpy",
    "pdf2image": "pdf2image",
    "pandas": "pandas",
    "tabulate": "tabulate",
    "PyPDF2": "PyPDF2",
    "bcrypt": "bcrypt",
    "pyarrow": "pyarrow",
    "pyzbar": "pyzbar",
}.items():
    try:
        importlib.import_module(mod)
    except ImportError:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg], check=True)

# Imports
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import io, data, color
```

Além dessas bibliotecas, será utilizado o módulo didático `morph.py`, desenvolvido para simplificar operações de leitura, visualização e processamento de imagens ao longo deste livro. O código a seguir verifica sua disponibilidade, realiza o *download* quando necessário e confirma a versão carregada.

```
import os
import urllib.request

if not os.path.exists("morph.py"):
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph/morph.py",
        "morph.py",
    )

import morph
from morph import mm

print(f" Ambiente pronto. morph {getattr(morph, '__version__', 'local_file')}")
```

Ambiente pronto. morph 1.1.2

6.3 Bases de Imagens para Experimentação

Os exemplos desta parte do livro utilizam, sempre que possível, documentos digitalizados, folhas de respostas, códigos de barras, *QR Codes* e outras imagens provenientes de aplicações reais. Para facilitar a reprodução dos experimentos, também são empregadas imagens públicas amplamente utilizadas no ensino e na pesquisa

em Visão Computacional.

6.3.1 Imagens Públicas com `skimage.data`

A biblioteca `skimage.data` disponibiliza uma coleção de imagens de referência frequentemente utilizada em PDI-VC. O conjunto inclui fotografias, documentos digitalizados, padrões sintéticos e outros exemplos adequados para segmentação, extração de contornos, análise geométrica e reconhecimento de padrões.

A Tabela 6.1 apresenta algumas das principais imagens dessa coleção, enquanto a Figura 6.1 ilustra exemplos representativos. Neste capítulo, destacam-se as imagens `page()`, `text()`, `coffee()` e `brick()`, particularmente adequadas para experimentos de OCR (*Optical Character Recognition*), OMR (*Optical Mark Recognition*) e análise documental.

6.3.2 Imagens Reais com OpenCV

Embora as imagens do `skimage.data` sejam adequadas para demonstrações e validação de algoritmos, aplicações reais normalmente utilizam imagens obtidas por digitalização, câmeras ou outros dispositivos de aquisição.

Para esses casos, a biblioteca OpenCV (`cv2`) oferece funções para leitura de imagens, captura de vídeo e acesso a dispositivos de aquisição. Ao longo deste livro, essas funções são encapsuladas pelo módulo didático `morph.py`, por meio de rotinas como `mm.read`, proporcionando uma interface única para manipulação das imagens. Dessa forma, os algoritmos desenvolvidos podem ser aplicados tanto às imagens públicas quanto a documentos e cenas reais, preservando o mesmo fluxo de processamento.

Tabela 6.1: Principais imagens públicas disponíveis no módulo `skimage.data`.

Função	Descrição
<code>data.astronaut()</code>	Fotografia colorida de um astronauta.
<code>data.camera()</code>	Fotografia clássica em tons de cinza amplamente utilizada em PDI.
<code>data.cat()</code>	Fotografia colorida de um gato.
<code>data.chelsea()</code>	Retrato colorido da gata Chelsea.
<code>data.clock()</code>	Relógio analógico para detecção de formas e contornos.
<code>data.coins()</code>	Conjunto de moedas utilizado em segmentação e <i>watershed</i> .
<code>data.coffee()</code>	Fotografia colorida de uma xícara de café.
<code>data.horse()</code>	Silhueta binária de um cavalo.
<code>data.moon()</code>	Imagem da Lua em tons de cinza.
<code>data.page()</code>	Página digitalizada de documento.
<code>data.text()</code>	Imagem contendo texto impresso para experimentos de OCR.
<code>data.checkerboard()</code>	Padrão xadrez para calibração e transformações geométricas.
<code>data.binary_blobs()</code>	Blobs binários sintéticos para estudos de conectividade e morfologia.

```
import matplotlib.pyplot as plt
from skimage import data

imgs = {
    "camera": data.camera(),
    "coins": data.coins(),
    "text": data.text(),
    "page": data.page(),
    "moon": data.moon(),
    "cat": data.cat(),
    "horse": data.horse(),
    "binary_blobs": data.binary_blobs()
}
```

```
fig, ax = plt.subplots(2, 4, figsize=(10, 5))

for a, (nome, img) in zip(ax.ravel(), imgs.items()):
    a.imshow(img, cmap="gray")
    a.set_title(nome)
    a.axis("off")

plt.tight_layout()
```

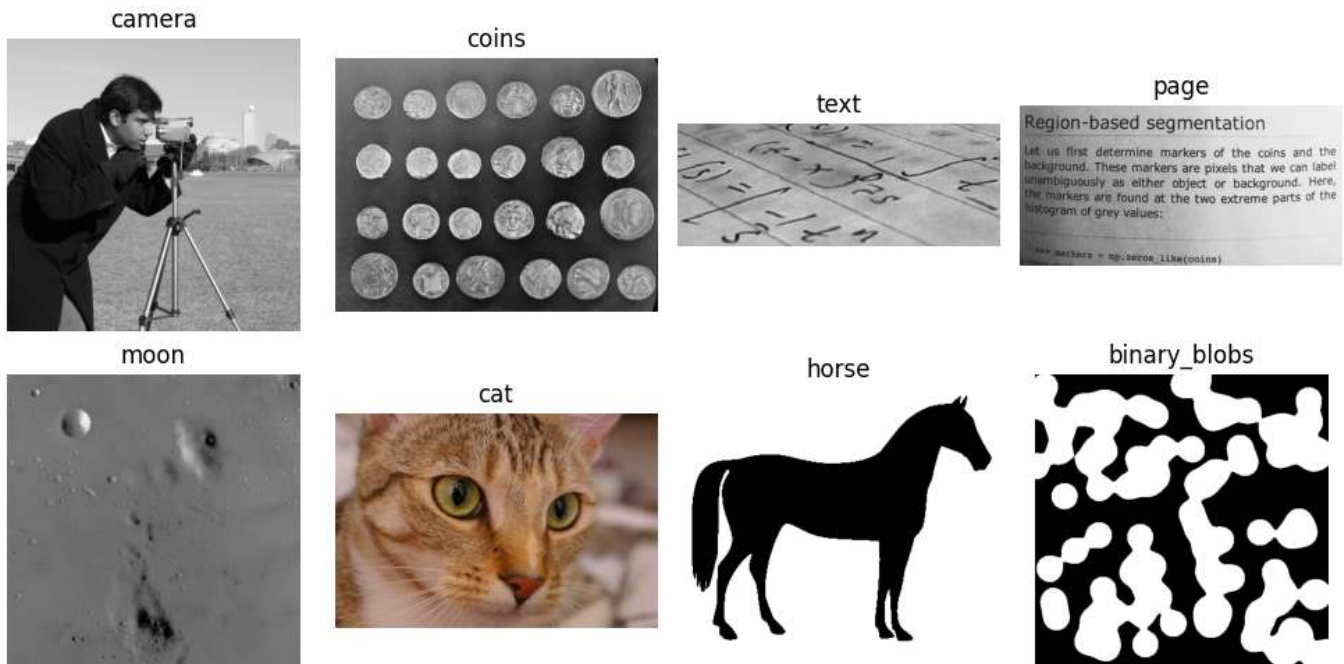


Figura 6.1: Algumas imagens públicas disponíveis em *skimage.data*.

6.4 Fundamentos de OMR e Inspeção Industrial

O **Reconhecimento Óptico de Marcas** (*Optical Mark Recognition* — OMR) é uma técnica de Visão Computacional destinada à identificação automática de marcações em posições previamente definidas de um formulário. Suas aplicações incluem folhas de respostas, questionários, formulários administrativos e outros documentos estruturados.

Diferentemente do OCR (*Optical Character Recognition*), que reconhece caracteres e palavras, o OMR determina a presença, a ausência ou a intensidade de marcas em regiões previamente conhecidas. Em vez de interpretar texto, explora propriedades geométricas e estatísticas associadas ao preenchimento dessas regiões.

Os sistemas modernos de OMR processam imagens obtidas por *scanners*, câmeras ou dispositivos móveis, automatizando tarefas que anteriormente dependiam de equipamentos especializados.

De forma geral, um sistema de OMR compreende as seguintes etapas:

1. **Aquisição:** conversão do documento físico em formato digital;
2. **Pré-processamento:** correção geométrica, redução de ruídos e binarização;
3. **Localização das regiões de interesse:** identificação das áreas destinadas às marcações;
4. **Análise das marcações:** avaliação do preenchimento das regiões candidatas;
5. **Interpretação:** conversão das marcações em respostas ou dados estruturados.

Esses princípios se estendem naturalmente à **Inspeção Industrial Automatizada**. Em linhas de produção, o mesmo encadeamento — aquisição, pré-processamento, segmentação, extração de características e decisão — é empregado para detectar defeitos superficiais, verificar a integridade de componentes e medir dimensões

com precisão subpixel. A diferença reside no domínio de aplicação: enquanto o OMR opera sobre documentos com estrutura predefinida, a inspeção industrial lida com objetos cujas variações geométricas e radiométricas devem ser modeladas de forma mais flexível.

Nas seções seguintes, ambas as aplicações são desenvolvidas por meio de projetos práticos que reproduzem etapas típicas de sistemas reais.

6.5 Projetos Práticos: Construção de um *Pipeline* de Análise Documental

Os conceitos deste capítulo serão desenvolvidos por meio de projetos que reproduzem etapas típicas de sistemas reais de análise documental, introduzindo técnicas reutilizáveis em aplicações de OCR, OMR, inspeção visual e processamento de formulários.

6.5.1 Alinhamento Automático de Documentos (*OCR/OMR Pre-processing*)

A correção de inclinação (*deskew*) é uma etapa fundamental no processamento de documentos. Rotações introduzidas durante a digitalização ou captura comprometem a localização de regiões de interesse e reduzem a precisão das etapas subsequentes.

Neste projeto será desenvolvido um sistema para estimar automaticamente a orientação predominante do documento e corrigir sua inclinação. Para isso, serão empregadas técnicas clássicas de detecção de bordas com o operador de Canny e detecção de retas pela Transformada de Hough. A partir das linhas identificadas, será estimado o ângulo de rotação e aplicada uma transformação afim para produzir uma versão alinhada do documento.

Como formulários e folhas de resposta são frequentemente distribuídos em formato PDF, o *pipeline* inicia-se com a rasterização de cada página, convertendo-a em uma imagem matricial. Neste capítulo, essa etapa será realizada com a biblioteca `pdf2image`, gerando imagens PNG com resolução de 300 DPI (*dots per inch*). A partir delas, poderão ser aplicadas as técnicas de detecção de bordas, Transformada de Hough, segmentação, extração de contornos e reconhecimento automático de padrões estudadas ao longo do capítulo.

```
from pdf2image import convert_from_path

# Diretório dos microdados e folhas de respostas do exame institucional
file_path = "dados/provas_qrcode_EP.pdf"
print(f"PDF de folhas de prova digitalizadas: {file_path}")

if os.path.exists(file_path):
    # Rasterização das páginas com resolução otimizada de 300 DPI
    pages = convert_from_path(file_path, dpi=300)
    for i, page in enumerate(pages):
        saida = f"test{i+1:02d}.png"
        page.save(saida)
        print(f"[INGESTÃO] Página PDF convertida com sucesso: {saida}")
else:
    print("[AVISO] Arquivo PDF não localizado no path. Ativando fallback via skimage.data.")
    # Injeta matriz de texto pública para garantir a execução contínua do pipeline
    img_fallback = data.text()
    cv2.imwrite("test02.png", img_fallback)
    print("[INGESTÃO] Imagem de fallback estruturada: test02.png")

# Carrega e exibe a imagem rasterizada inicial utilizando o ecossistema morph
img_original = mm.read('test02.png')
mm.show(img_original, figsize=(4, 3))
```



```
PDF de folhas de prova digitalizadas: dados/provas_qrcode_EP.pdf
[INGESTÃO] Página PDF convertida com sucesso: test01.png
[INGESTÃO] Página PDF convertida com sucesso: test02.png
[INGESTÃO] Página PDF convertida com sucesso: test03.png
```

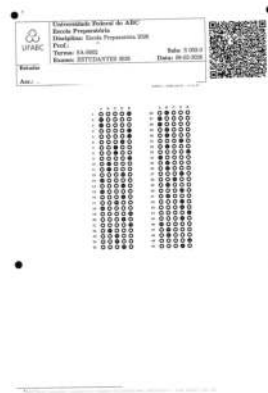


Figura 6.2: *Pipeline* de ingestão de documentos: rasterização adaptativa de páginas PDF para matrizes discretas em formato PNG, exibindo a página dois.

6.5.2 Algoritmo de Retificação de Inclinação (*Deskew*)

A etapa de *deskew* tem como objetivo estimar e corrigir a inclinação global de um documento digitalizado. A Figure 6.4 apresenta o fluxo de processamento, desde a imagem original até o resultado após a correção geométrica. Além disso, simulador Figure 6.3 ajuda a entender a Transformada de Hugh.

O procedimento é composto por três etapas principais:

1. detecção de bordas com o operador de Canny;
2. estimação da orientação predominante pela Transformada de Hough;
3. correção da inclinação por meio de uma transformação afim de rotação.

Após a retificação, o documento fica alinhado aos eixos da imagem, favorecendo as etapas subsequentes de segmentação, extração de componentes conexos e reconhecimento de marcas.

6.5.3 Modelagem Matemática

Esta seção apresenta os fundamentos matemáticos das três etapas que compõem o processo de retificação: detecção de bordas, estimação da orientação e correção geométrica.

6.5.3.1 Detecção de Bordas

Inicialmente, a imagem é suavizada por um filtro Gaussiano para reduzir ruídos. Os conceitos de filtragem espacial e convolução foram apresentados no **Capítulo 3**. Em seguida, o operador de Canny calcula a magnitude do gradiente,

$$|\nabla f| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}.$$

Após a supressão de não-máximos e a limiarização, obtém-se uma imagem binária contendo as principais bordas do documento.

6.5.3.2 Transformada de Hough

As bordas detectadas são analisadas pela Transformada de Hough Linear. Cada ponto (x, y) contribui para o conjunto de retas descrito por

$$\rho = x \cos \theta + y \sin \theta.$$

Os máximos da matriz acumuladora correspondem às estruturas lineares predominantes, como bordas da página, linhas de formulários e linhas de texto.

Para estimar a orientação global, são considerados apenas os ângulos pertencentes ao intervalo $[-45^\circ, 45^\circ]$. A mediana desses valores é utilizada como estimativa da inclinação, reduzindo a influência de detecções espúrias.

6.5.3.3 Rotação Afim

Conhecido o ângulo de inclinação, aplica-se uma transformação afim de rotação em torno do centro da imagem. As transformações afins foram estudadas no **Capítulo 2**, juntamente com as operações de translação, escala, cisalhamento e rotação. A função `mm.rotate` implementa essa transformação utilizando interpolação bicúbica para preservar a qualidade visual de contornos e caracteres.



Figura 6.3: Simulador interativo de correção de inclinação (*deskew*): mova o slider para inclinar o documento e observe as três etapas do *pipeline* — imagem inclinada, bordas Canny e resultado corrigido.

```
def retificar_inclinacao_documento(img):
    gray = mm.gray(img) if img.ndim == 3 else img
    edges = cv2.Canny(cv2.GaussianBlur(gray, (5,5), 0), 50, 150)
    lines = cv2.HoughLines(edges, 1, np.pi/180, 200)
```

```

if lines is None:
    return edges, img

angulos = []
for line in lines:
    angulo = np.rad2deg(line[0][1]) - 90
    if -45 < angulo < 45:
        angulos.append(angulo)

if not angulos:
    return edges, img

return edges, mm.rotate(img, np.median(angulos), interp="bicubic")

# Execução do pipeline de deskew
img_edges, img_final = retificar_inclinacao_documento(img_original)

# Exibição múltipla padronizada com o formato nativo do livro
mm.show(
    [img_original, img_edges, img_final],
    titles=["Imagem Original", "Bordas de Canny", "Documento Retificado"],
    cols=3,
    figsize=(12, 4)
)

```

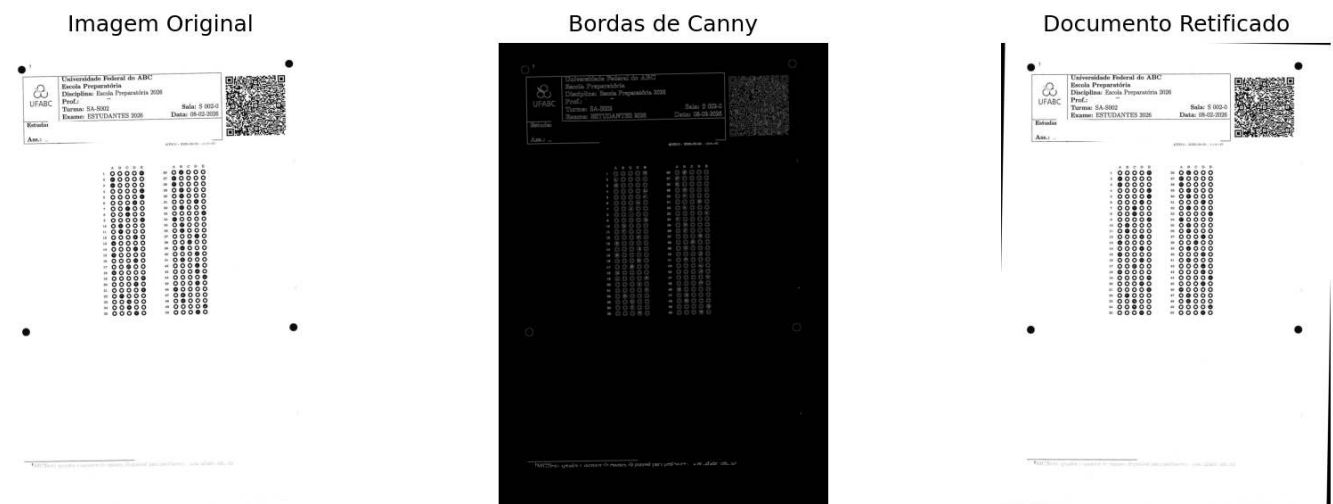


Figura 6.4: *Pipeline* de retificação axial: exibição comparativa entre a entrada rotacionada original, o mapa de gradientes estruturais de Canny e o resultado final alinhado com fundo normalizado em branco.

Por que funciona? — Transformada de Hough

Na Transformada de Hough, cada pixel de borda contribui com votos para todas as retas que podem passar por sua posição. Em vez de selecionar apenas a reta com maior número de votos, o algoritmo considera todas as retas cuja quantidade de votos excede um limiar mínimo e calcula seus respectivos ângulos. A inclinação global do documento é então estimada pela mediana desses ângulos, uma medida robusta a valores discrepantes. Assim, retas espúrias produzidas por sombras, ruídos ou outros elementos da imagem exercem pouca influência sobre a estimativa final, desde que a maioria das retas detectadas corresponda às bordas do documento.

6.5.4 Limitações Práticas

Embora apresente bom desempenho em condições usuais de digitalização, o método depende da existência de estruturas lineares suficientemente definidas para serem detectadas pela Transformada de Hough, como bordas da página, linhas de formulários ou linhas de texto. Sua precisão pode ser reduzida em imagens com baixa resolução, ruído excessivo, sombras intensas ou grandes inclinações. Em geral, documentos digitalizados com resolução próxima de 300 DPI e iluminação homogênea fornecem resultados adequados para aplicações de OCR e OMR.

A implementação apresentada neste capítulo possui finalidade didática, ilustrando os princípios da correção automática de inclinação por meio da detecção de bordas, da Transformada de Hough e da rotação afim. Por utilizar apenas a orientação das estruturas lineares predominantes, o método pode ser aplicado a diferentes tipos de documentos, sem depender de marcadores específicos.

Em sistemas reais de análise documental, entretanto, o alinhamento normalmente utiliza marcadores geométricos previamente conhecidos. No modelo de folha de respostas empregado pelo ecossistema MCTest, por exemplo, são utilizados quatro discos pretos de referência, além das regiões correspondentes ao cabeçalho, ao *QRCode* e aos quadros de respostas. A localização desses elementos permite estimar simultaneamente a rotação, a escala e a translação da folha, tornando o registro menos sensível à quantidade de texto, à ausência de linhas estruturais e às variações de impressão ou digitalização.

Por esse motivo, a abordagem baseada na Transformada de Hough é utilizada neste capítulo para introduzir os fundamentos do problema, enquanto as etapas posteriores adotam o alinhamento por marcadores geométricos, estratégia predominante em sistemas de OMR e análise documental.

6.6 Normalização de Fundo e Equalização Local de Contraste

A qualidade da segmentação depende diretamente do contraste da imagem de entrada. Em documentos digitalizados, variações de iluminação, sombras, regiões superexpostas e diferenças de tonalidade do papel dificultam a separação entre primeiro plano e fundo por meio de um limiar global.

Duas estratégias complementares são utilizadas para compensar essas variações:

- **Normalização de fundo:** divide a imagem por uma versão fortemente suavizada de si mesma (fundo estimado), cancelando gradientes de iluminação de baixa frequência — sombras, variação lateral de luz — antes da binarização.
- **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*), apresentada no **Capítulo 4**: divide a imagem em pequenas regiões (*tiles*) e equaliza o histograma de cada uma individualmente, limitando a amplificação do contraste para evitar o realce excessivo do ruído. É mais eficaz quando a iluminação é localmente heterogênea, mas o gradiente global já foi removido.

A Figure 6.5 compara as duas abordagens sobre a imagem `page()` da biblioteca `skimage.data`. Cinco versões são exibidas: a imagem original, a imagem com fundo normalizado, a binarização direta por Otsu (sem pré-processamento, como referência), o resultado após normalização de fundo seguida de Otsu, e o resultado após CLAHE seguido de Otsu. Essa comparação permite visualizar tanto o efeito intermediário da normalização quanto a qualidade final da segmentação em cada estratégia, sendo a normalização de fundo seguida de Otsu a que produz a binarização mais limpa para este tipo de documento.

No projeto de retificação apresentado na seção anterior, a folha de respostas foi digitalizada em condições controladas, tornando suficiente a aplicação direta da limiarização. Em aplicações reais, entretanto, documentos são frequentemente capturados por *scanners* ou câmeras de dispositivos móveis, sob iluminação não uniforme. Nessas situações, a combinação de normalização do fundo, CLAHE e limiarização produz segmentações mais robustas, beneficiando as etapas subsequentes do *pipeline* de Visão Computacional.

```
import cv2
from skimage import data
from morph import mm
```

```

img = data.page() # ou: img = mm.gray(img_final) - com imagem da folha de prova

# Método 1: Normalização de fundo + Otsu
# Estima o fundo com um filtro Gaussiano de sigma grande (variações lentas de luz)
# e divide pixel a pixel para cancelar o gradiente de iluminação
bg = cv2.GaussianBlur(img, (0, 0), sigmaX=25)
img_norm = cv2.divide(img, bg, scale=255)
img_norm_otsu = mm.threshold(img_norm)

# Método 2: CLAHE + Otsu
# tileGridSize define o tamanho de cada região local (tile)
# clipLimit controla o teto de amplificação - valores altos aumentam contraste
# mas também amplificam ruído
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
img_clahe = clahe.apply(img)
img_clahe_otsu = mm.threshold(img_clahe)

# Método 0: Otsu direto (sem pré-processamento) - referência
img_otsu = mm.threshold(img)

mm.show(
    [img, img_otsu, img_norm, img_clahe_otsu, img_norm_otsu],
    titles=["Original", "Otsu direto", "Fundo normalizado", "CLAHE + Otsu", \
           "Normaliz. fundo + Otsu"],
    cols=3,
    figsize=(14, 8)
)

```

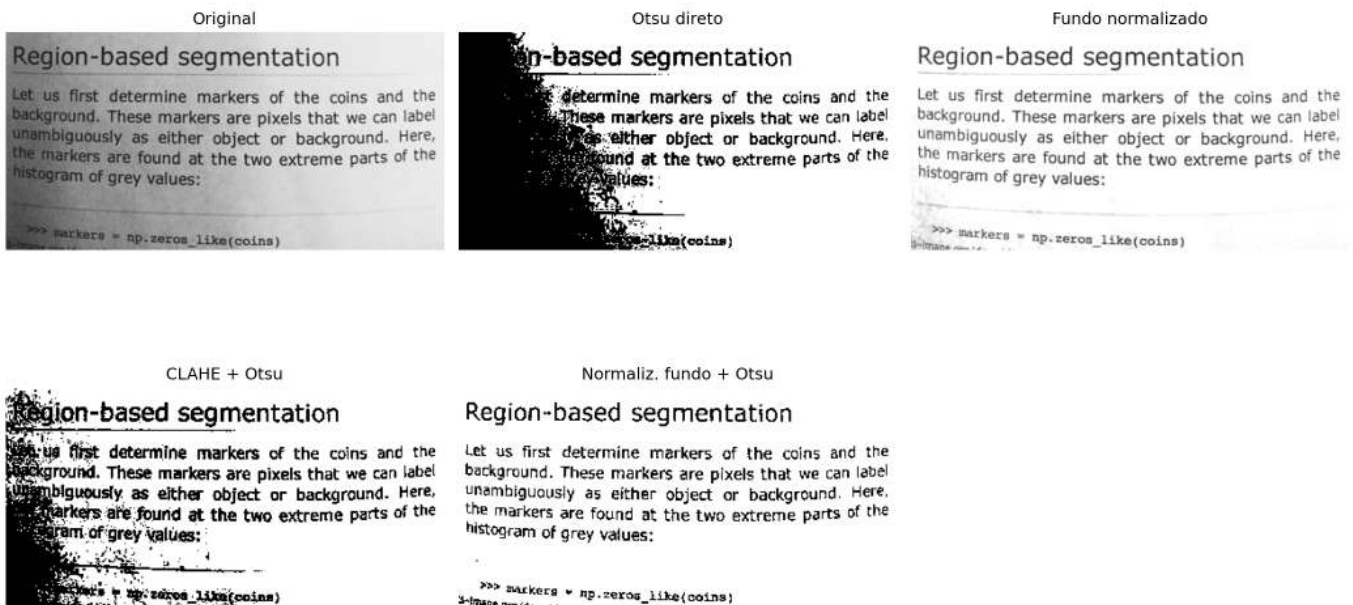


Figura 6.5: Efeito da normalização de fundo e do CLAHE na limiarização por Otsu: da esquerda para a direita, qualidade crescente de segmentação.

i Por que funciona? — Normalização de fundo vs. CLAHE

Normalização de fundo: ao dividir a imagem por uma versão fortemente suavizada de si mesma, eliminam-se as variações lentas de luminosidade (gradiente de luz, sombra de borda) sem afetar os

detalhes finos — texto, linhas, bolhas. O resultado é uma imagem com iluminação aproximadamente uniforme, onde o limiar global de Otsu passa a funcionar bem em toda a página.

CLAHE: um histograma global equalizado “estica” os tons de toda a imagem de uma vez — útil quando a iluminação é uniforme, mas problemático quando não é. O CLAHE divide a imagem em pequenos blocos (*tiles*) e equaliza cada um separadamente, com um limite máximo de amplificação (`clipLimit`) para não explodir o ruído. É especialmente eficaz para realçar regiões subexpostas localmente, mas não elimina gradientes globais — por isso, aplicá-lo após a normalização de fundo tende a produzir resultados mais consistentes.

6.6.1 Detecção de Bordas e Contornos

A localização precisa das regiões de interesse é uma etapa essencial em sistemas de OMR. No modelo de folha de respostas utilizado neste capítulo, o cabeçalho e o quadro de respostas estão contidos em um retângulo virtual delimitado por quatro discos pretos posicionados nos cantos. A identificação desses marcadores permite localizar a região de interesse e corrigir distorções geométricas introduzidas durante a aquisição da imagem.

O procedimento é composto por cinco etapas. Inicialmente, aplica-se um **fechamento morfológico** (dilatação seguida de erosão), operação estudada no **Capítulo 4**, utilizando um elemento estruturante em disco (`mm.sedisk(33)`). Essa operação reduz pequenas descontinuidades e preserva os discos de referência, tornando-os mais homogêneos. Em seguida, a imagem é invertida (`mm.neg`), de modo que os discos passem a constituir componentes claros sobre fundo escuro.

Na etapa seguinte, aplica-se a operação `mm.edgeoff` (**Capítulo 4**), que remove componentes conectados às bordas da imagem, eliminando artefatos como sombras de digitalização, marcas de corte e outros objetos espúrios nas margens. Os componentes remanescentes são então analisados a partir de seus contornos e filtrados por propriedades geométricas, como área e circularidade, para identificar os discos candidatos. A implementação do MCTest torna esse processo mais robusto ao selecionar, entre todos os candidatos, os quatro cujos centros formam um retângulo com largura compatível com a da imagem, reduzindo a ocorrência de falsos positivos.

Por fim, os centros dos quatro discos são ordenados espacialmente (superior esquerdo, superior direito, inferior esquerdo e inferior direito) e utilizados como pontos de controle em uma transformação de perspectiva (*perspective warp*). Essa transformação retifica a imagem, produzindo uma representação alinhada e com dimensões conhecidas, adequada às etapas subsequentes de segmentação e reconhecimento.

As principais etapas desse *pipeline*, desde o processamento morfológico até a imagem retificada, são ilustradas a seguir.

```
import cv2
import numpy as np
from morph import mm

# img: imagem em escala de cinza da folha de prova
img = mm.gray(img_final)

# 1. Fechamento morfológico: preserva os discos escuros, removendo tudo menor que o disco
img_close = mm.close(img, mm.sedisk(41))

# 2. Inversão: discos escuros tornam-se componentes claros sobre fundo escuro
img_neg = mm.neg(img_close)

# 3. Remove componentes conectados que tocam a borda da imagem
img_edgeoff = mm.edgeoff(img_neg)

# 4. Extração dos contornos externos
contornos, _ = cv2.findContours(img_edgeoff, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```



```

# 5. Filtragem por área e circularidade, mantendo apenas os 4 discos
centros = []
for c in contornos:
    area = cv2.contourArea(c)
    perimetro = cv2.arcLength(c, True)
    if area < 50 or perimetro == 0:
        continue
    circularidade = 4 * np.pi * area / (perimetro ** 2)
    if circularidade > 0.6:
        M = cv2.moments(c)
        cx, cy = M["m10"] / M["m00"], M["m01"] / M["m00"]
        centros.append((cx, cy))

# Verificação robusta: interrompe o pipeline com mensagem clara em vez de AssertionError
if len(centros) != 4:
    print(f"[AVISO] Esperado 4 discos marcadores, encontrado {len(centros)}.")
    print("  Verifique se a imagem é uma folha de respostas MCTest válida")
    print("  ou ajuste os parâmetros de circularidade e área mínima.")
    img_retificada = img # fallback: preserva a imagem sem retificação
else:
    # 6. Ordenação dos centros: superior-esquerdo, superior-direito,
    # inferior-esquerdo, inferior-direito
    pts = np.array(centros, dtype=np.float32)
    soma = pts.sum(axis=1)
    diff = pts[:, 0] - pts[:, 1]
    tl = pts[np.argmin(soma)]
    br = pts[np.argmax(soma)]
    tr = pts[np.argmax(diff)]
    bl = pts[np.argmin(diff)]
    pts_ordenados = np.array([tl, tr, bl, br], dtype=np.float32)

    # 7. Retificação por transformação de perspectiva (warp)
    largura, altura = 400, 400
    destino = np.array(
        [[0, 0], [largura, 0], [0, altura], [largura, altura]], dtype=np.float32
    )
    M_persp = cv2.getPerspectiveTransform(pts_ordenados, destino)
    img_retificada = cv2.warpPerspective(img, M_persp, (largura, altura))

    mm.show(
        [img_close, img_edgeoff, img_retificada],
        titles=["Fechamento (sedisk 41)", "edgeoff", "Retificada (warp)"],
        cols=3,
        figsize=(12, 4)
    )

mm.write(img_retificada, "img_beetween_disks.png")

```

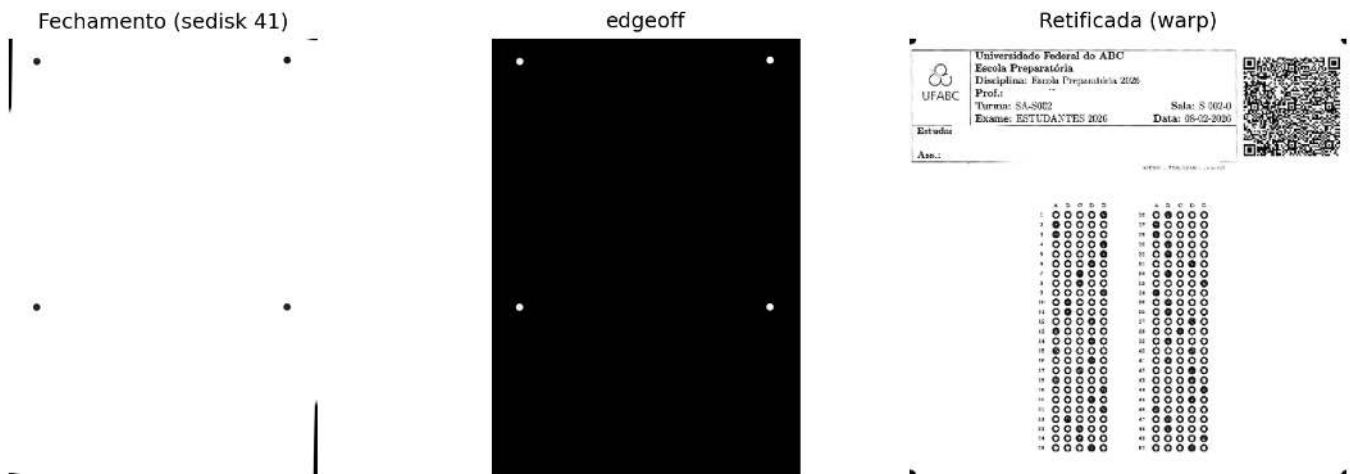


Figura 6.6: Detecção dos discos marcadores, extração de contornos e retificação por transformação de perspectiva.

i Por que funciona? — Do fechamento morfológico à retificação

Fechamento morfológico: a dilatação seguida de erosão preenche pequenas descontinuidades e suaviza os contornos dos objetos sem alterar significativamente sua forma global. Ao utilizar um elemento estruturante grande (`sedisk(41)`), detalhes finos, como textos, linhas do formulário e pequenos ruídos, tendem a ser incorporados ao fundo durante o processamento, enquanto objetos de maior escala, como os discos de referência, permanecem preservados e tornam-se mais homogêneos.

Circularidade: após o isolamento dos componentes candidatos, a métrica $C = \frac{4\pi A}{P^2}$ quantifica o quanto sua forma se aproxima de um círculo. Seu valor é igual a 1 para um círculo perfeito e diminui à medida que o contorno se torna mais irregular. Assim, um limiar como $C > 0,6$ permite descartar a maior parte dos falsos positivos sem recorrer a modelos de aprendizado. Um simulador dessa métrica é apresentado na Figure 6.7.

Transformação de perspectiva (*perspective warp*): uma vez identificados os quatro discos de referência, seus centros são utilizados como pontos de controle para estimar a transformação projetiva que mapeia a imagem capturada para o plano do documento. Essa transformação corrige as distorções introduzidas pela perspectiva durante a aquisição da imagem, produzindo uma representação frontal com dimensões conhecidas e adequada às etapas subsequentes de segmentação e reconhecimento.

6.6.2 Isolamento, Segmentação e Decodificação do *QRCode*

Após a retificação geométrica da folha de respostas, realiza-se a detecção e a decodificação do *QRCode* presente no formulário. Esse marcador armazena informações utilizadas pelo sistema de OMR (*Optical Mark Recognition*), como a identificação do aluno, o código da prova e sua variação, possibilitando a recuperação do gabarito correspondente no banco de dados. Por segurança, essas informações são criptografadas antes da geração do *QRCode*. Assim, a sequência decodificada corresponde a uma string hexadecimal, cuja interpretação é realizada exclusivamente pelo sistema MCTest. O procedimento é composto por três etapas: pré-processamento morfológico, isolamento da região do *QRCode* e decodificação de seu conteúdo.

Inicialmente, a imagem retificada em escala de cinza é binarizada por meio da operação `mm.threshold`. Em seguida, aplica-se uma **abertura morfológica** (erosão seguida de dilatação), estudada no **Capítulo 4**, utilizando um elemento estruturante quadrado (`mm.sebox(2)`). Essa operação remove pequenos ruídos e suaviza imperfeições sem comprometer a estrutura do marcador. Por fim, a imagem é invertida (`mm.neg`), de modo que o *QRCode* passe a constituir um componente claro sobre fundo escuro, facilitando a extração de seus contornos.

A localização do *QRCode* é realizada pela análise dos contornos externos da imagem binarizada. Entre os componentes detectados, seleciona-se aquele com maior área e geometria aproximadamente quadrada,

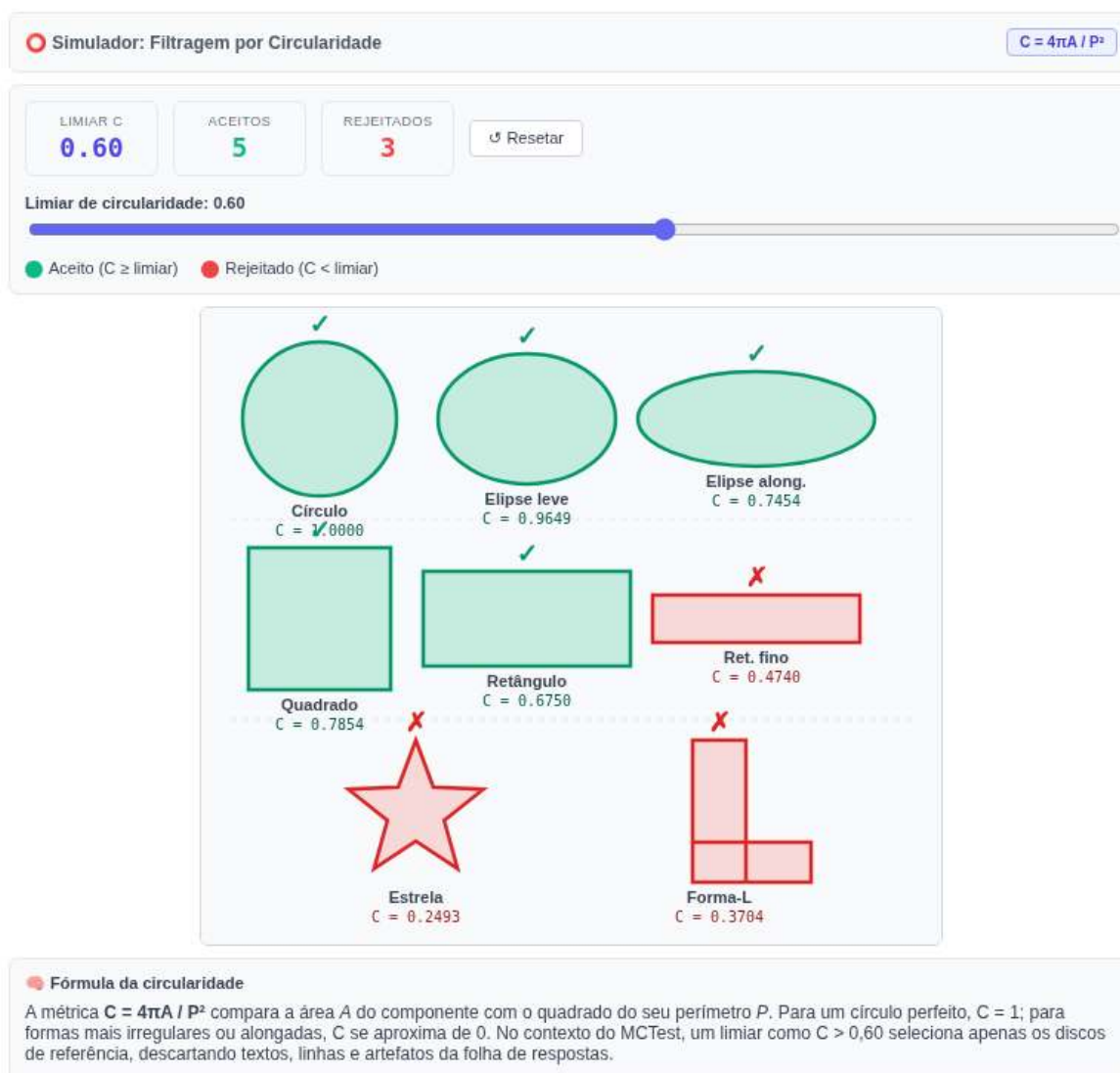


Figura 6.7: Simulador interativo de filtragem por circularidade: mova o slider para ajustar o limiar C e observe quais componentes são aceitos (verde) ou rejeitados (vermelho).

descartando os demais elementos impressos da folha. Em seguida, a região correspondente é expandida por uma pequena margem de segurança, garantindo a preservação integral do marcador.

O *QRCode* é então extraído diretamente da imagem retificada em escala de cinza, preservando sua qualidade radiométrica. Como essa região geralmente apresenta dimensões reduzidas, aplica-se um redimensionamento com interpolação cúbica, aumentando a resolução espacial e favorecendo a identificação de seus módulos. A leitura é realizada pelo detector de *QRCode* do OpenCV (`cv2.QRCodeDetector`), que recupera a sequência de caracteres originalmente codificada.

O fluxo completo desse processamento, desde o pré-processamento morfológico até a decodificação do *QRCode*, é ilustrado na Figure 6.8. O trecho exibido na saída corresponde apenas ao início da string hexadecimal criptografada; sua interpretação completa é realizada internamente pelo MCTest após a decodificação.

```
import cv2
import numpy as np
from morph import mm

# f: imagem retificada convertida para o tipo correto de 8 bits (0-255)
f = img_retificada.astype('uint8')

# 1. Limiarização: conversão da imagem em tons de cinza para binária
f_thresh = mm.threshold(f)

# 2. Abertura morfológica: elimina pequenos ruídos e suaviza o contorno dos blocos
f_open = mm.open(f_thresh, mm.sebox(2))

# 3. Inversão morfológica: módulos escuros tornam-se componentes claros sobre fundo escuro
f_inv = mm.neg(f_open)

# 4. Conversão segura para uint8 com escala 0-255
img_uint8 = (f_inv.astype(np.uint8) * 255) if f_inv.max() == 1 else f_inv.astype(np.uint8)

# 5. Detecção de contornos externos
contornos, _ = cv2.findContours(img_uint8, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

if not contornos:
    raise ValueError("Nenhum contorno encontrado. Verifique o limiar ou a imagem de entrada.")

# 6. Filtragem pelo maior contorno com proporção aproximadamente quadrada
# (aspect ratio entre 0.7 e 1.3 descarta retângulos alongados da folha)
def is_square_like(contorno, tol=0.3):
    x, y, w, h = cv2.boundingRect(contorno)
    ratio = w / h if h > 0 else 0
    return (1 - tol) <= ratio <= (1 + tol)

candidatos = [c for c in contornos if is_square_like(c)]

if not candidatos:
    raise ValueError(
        "Nenhum contorno quadrado encontrado. "
        "Verifique se o QRCode está presente na imagem ou ajuste a tolerância."
    )

# Seleciona o maior candidato quadrado por área de bounding box
maior_contorno = max(candidatos, key=lambda c: cv2.boundingRect(c)[2] * cv2.boundingRect(c)[3])
x, y, w, h = cv2.boundingRect(maior_contorno)

# 7. Expansão da bounding box com margem de segurança (evita truncamento do QR Code)
margem = 5
h_img, w_img = img_uint8.shape[:2]
```

```

x1 = max(x - margem, 0)
y1 = max(y - margem, 0)
x2 = min(x + w + margem, w_img)
y2 = min(y + h + margem, h_img)

# 8. Recorte da região de interesse a partir da imagem original (nítida, em cinza)
img_qrcode_final = img_retificada[y1:y2, x1:x2]

# 9. Ampliação para resolução mínima de decodificação (400 px no lado maior)
# cv2.QRCodeDetector requer módulos com ao menos 3-4 px de largura para decodificar
# com segurança; imagens menores que ~400 px tendem a falhar.
lado = max(img_qrcode_final.shape[:2])
escala = max(400 / lado, 1.0)
img_para_leitura = cv2.resize(
    img_qrcode_final, None,
    fx=escala, fy=escala,
    interpolation=cv2.INTER_CUBIC
)

# 10. Inicialização do detector nativo de QRCode do OpenCV
detector = cv2.QRCodeDetector()

# 11. Detecção geométrica e decodificação dos dados textuais
dados, pontos, qrcode_reto = detector.detectAndDecode(img_para_leitura)

# Visualização intermediária: progressão da binarização ao isolamento do QRCode
mm.show(
    [f_thresh, f_open, f_inv, img_qrcode_final],
    titles=["1. Limiarização", "2. Abertura morfológica", "3. Inversão", "Imagem final"],
    cols=3, figsize=(12, 4)
)

# Validação e saída dos metadados extraídos
if dados:
    print(f"QRCode decodificado com sucesso: \n{dados[:50]}...")
else:
    raise ValueError(
        "Falha na decodificação do QRCode. "
        "Verifique o limiar, as margens da região ou a qualidade da imagem."
    )

```

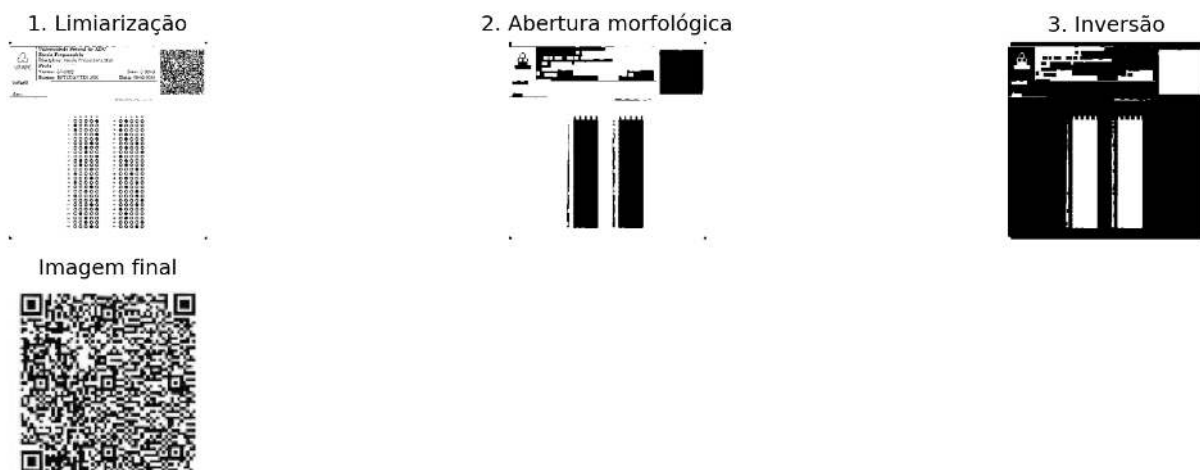


Figura 6.8: *Pipeline* de processamento do *QRCode*: limiarização, abertura morfológica, inversão e recorte final para decodificação.

QRCode decodificado com sucesso:

325a356b71367266556955646b7233454149624a694f417730...

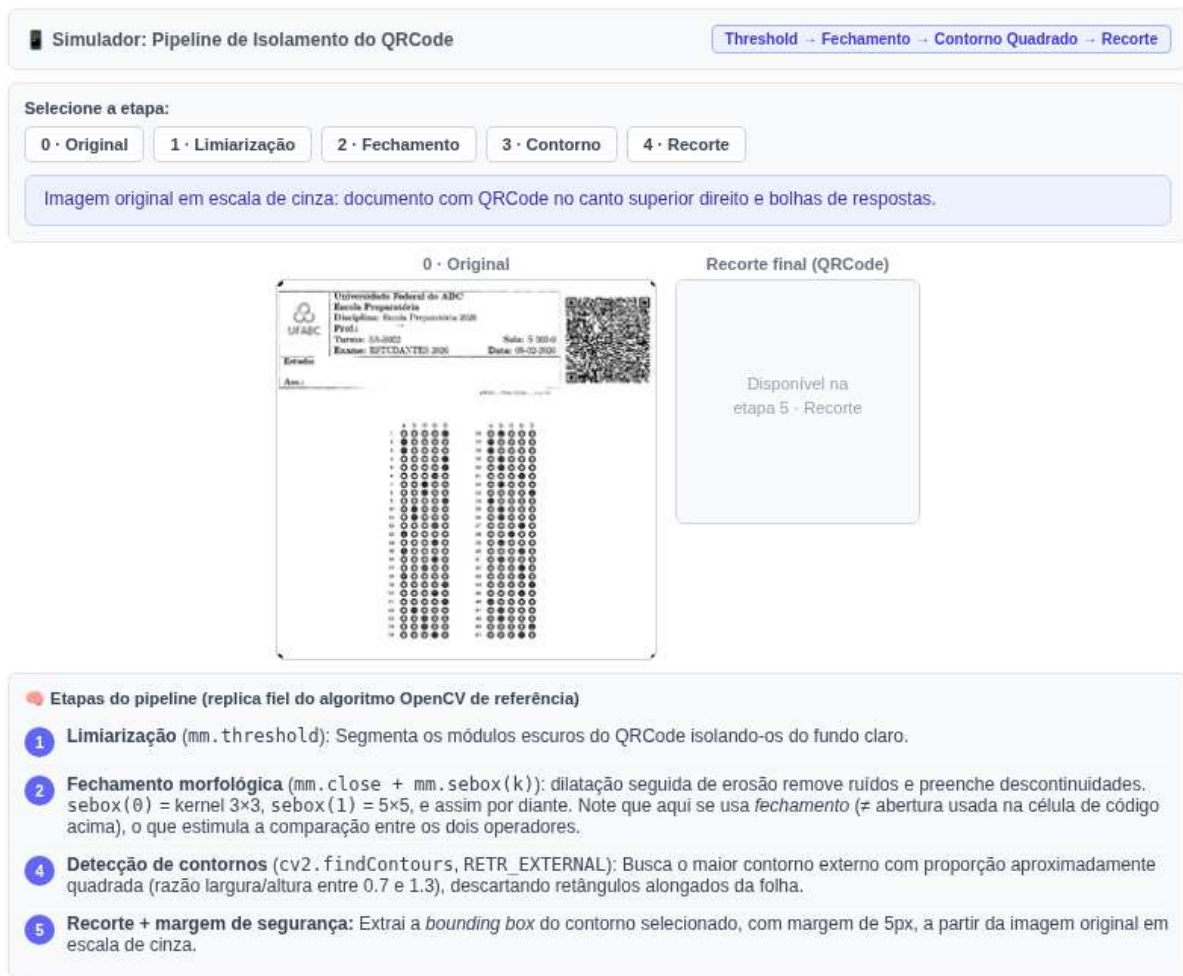


Figura 6.9: Simulador interativo do *pipeline* de isolamento do QRCode: navegue pelas etapas de filtragem, ajuste o elemento estruturante do fechamento morfológica e veja a detecção do contorno quadrado sobre a imagem original do gabarito.

i Por que funciona? — Isolamento e decodificação do QRCode

Abertura morfológica: diferentemente do fechamento, a abertura (erosão seguida de dilatação) remove pequenos ruídos e protuberâncias sem alterar significativamente a geometria dos objetos maiores. Assim, preserva a estrutura do QRCode enquanto elimina componentes espúrios que poderiam dificultar sua localização.

Seleção por geometria: o QRCode possui formato aproximadamente quadrado ($w/h \approx 1$). A combinação desse critério com a seleção do componente de maior área descarta linhas do formulário, textos e outros elementos impressos, permitindo isolar o marcador sem o uso de modelos de aprendizado.

Redimensionamento antes da decodificação: quando o QRCode ocupa poucos pixels na imagem, seus módulos tornam-se difíceis de distinguir. O redimensionamento com interpolação cúbica aumenta a resolução espacial da região de interesse, facilitando a identificação dos padrões do código pelo cv2.QRCodeDetector e tornando a decodificação mais robusta.

6.6.3 Decodificação de Código de Barras

Além de QRCodes, a biblioteca `pyzbar` permite decodificar diversas simbologias de códigos de barras lineares (1D), como EAN-13 e Code 128. O procedimento consiste em localizar o símbolo na imagem e interpretar a

sequência de barras e espaços, produzindo a cadeia de caracteres correspondente.

A Figure 6.10 apresenta um exemplo de código de barras e o resultado de sua decodificação.

```
import os
import numpy as np
from pyzbar.pyzbar import decode
from morph import mm

barcode_path = 'dados/barcode.png'

if os.path.exists(barcode_path):
    image = mm.read(barcode_path)
else:
    # Fallback sintético: gera um padrão de barras verticais que simula um Code-128
    print("[AVISO] Arquivo 'dados/barcode.png' não encontrado.")
    print("      Usando imagem sintética para demonstração do pipeline.")
    h, w = 100, 400
    img_synth = np.ones((h, w), dtype=np.uint8) * 255
    # Barras escuras em posições regulares (padrão simplificado)
    for x in range(20, w - 20, 8):
        if (x // 8) % 3 != 0:
            img_synth[:, x:x+4] = 0
    image = img_synth

mm.show(image)

barcodes = decode(image)

if barcodes:
    dados_bc = barcodes[0].data.decode("utf-8")
    tipo = barcodes[0].type
    print(f"Código de barras decodificado com sucesso [{tipo}]:\n{dados_bc}")
else:
    print("[INFO] Nenhum código de barras detectado na imagem.")
    print("Em imagem sintética isso é esperado - substitua pelo arquivo real para decodificar.")
```



Figura 6.10: Decodificação de código de barras linear com *pyzbar*: imagem de entrada e dados extraídos.

Código de barras decodificado com sucesso [EAN13]:
0000000000055

6.6.4 O MCTest como estudo de caso: do protótipo ao sistema em produção

Até este ponto, cada etapa do *pipeline* foi implementada e analisada isoladamente: correção da inclinação (*deskew*), detecção de marcadores, retificação por perspectiva e leitura do *QRCode*. O **MCTest** integra essas etapas em um sistema utilizado desde 2012 na UFABC para a correção automatizada de avaliações de centenas de estudantes por semestre, oferecendo suporte a diferentes modelos de folhas de resposta, gabaritos individualizados e geração automática de relatórios de desempenho (Zampirolli, 2023).

A partir deste ponto, o objetivo deixa de ser implementar novos algoritmos e passa a ser compreender como eles são integrados em uma aplicação real, atendendo a requisitos de robustez, manutenção e escalabilidade.

Desenvolvido na UFABC e disponibilizado como software de código aberto, o MCTest será utilizado como estudo de caso. Nas próximas seções, será analisado o módulo de Visão Computacional, implementado no

arquivo `CVMCTest.py`, destacando como os conceitos apresentados neste capítulo são organizados em um sistema completo.

6.6.4.1 Obtenção e Preparação do Módulo

O arquivo `CVMCTest.py` integra o sistema `MCTest` e depende de modelos e configurações do *framework* Django, indisponíveis fora do ambiente Web. Para utilizá-lo neste capítulo, o arquivo é obtido com `requests` e essas dependências são removidas com `sed`, tornando o módulo autocontido.

```
import requests
CVMCTest = requests.get(
    "https://raw.githubusercontent.com/fzampirolli/mctest/master/exam/CVMCTest.py"
)
with open('CVMCTest.py', 'w') as writefile:
    writefile.write(CVMCTest.text)
```

Cada comando `sed` remove importações específicas do ambiente Django, tornando o arquivo `CVMCTest.py` utilizável de forma independente neste capítulo.

```
# remove linhas with "from django.", ...
!sed --in-place '/from django./d' CVMCTest.py
!sed --in-place '/from exam./d' CVMCTest.py
!sed --in-place '/from mctest./d' CVMCTest.py
!sed --in-place '/from student./d' CVMCTest.py
!sed --in-place '/from topic./d' CVMCTest.py
!sed --in-place '/from .models import VariationExam/d' CVMCTest.py
```

Por que remover as dependências do Django?

O arquivo `CVMCTest.py` foi desenvolvido para integrar uma aplicação Django e, por isso, importa modelos de banco de dados e configurações do sistema que não estão disponíveis neste capítulo. Essas dependências impedem a importação do módulo, embora não sejam necessárias para executar as rotinas de Visão Computacional.

Os comandos `sed` removem apenas essas importações, preservando a lógica de processamento de imagens. Esse procedimento exemplifica uma prática comum de engenharia de software: desacoplar um módulo de sua infraestrutura para permitir sua reutilização em outros contextos.

6.6.4.2 Extração da Área de Respostas

A função `getAnswerArea` encapsula as etapas de localização dos discos de referência e retificação por perspectiva, retornando a região da folha que contém os quadros de marcação, já alinhada e com dimensões fixas. A Figure 6.11 apresenta a imagem original em escala de cinza, enquanto a Figure 6.12 mostra a área de respostas extraída.

```
import os
from pdf2image import convert_from_path
from skimage import data as skdata
import cv2
from morph import mm

file = "dados/provas_qrcode_EP.pdf"
MYFILES = 'extra02.qrcode'

if os.path.exists(file):
    pages = convert_from_path(file, 200) # dpi 100=min 500=max
    numPAGES = 0
    for page in pages:
        myfile0 = MYFILES + '_p' + str(numPAGES) + '.png'
```

```

page.save(myfile0)
numPAGES += 1
print(f"[INGESTÃO] Página convertida: {myfile0}")
pages.clear()
img_color = mm.read(myfile0)
img_inicial = mm.gray(img_color)
else:
    print("[AVISO] Arquivo 'dados/provas_qrcode.pdf' não encontrado.")
    print("        Usando imagem pública skimage.data.page() como substituto.")
    img_inicial = skdata.page()
mm.show(img_inicial)

```

```

[INGESTÃO] Página convertida: extra02.qrcode_p0.png
[INGESTÃO] Página convertida: extra02.qrcode_p1.png
[INGESTÃO] Página convertida: extra02.qrcode_p2.png

```

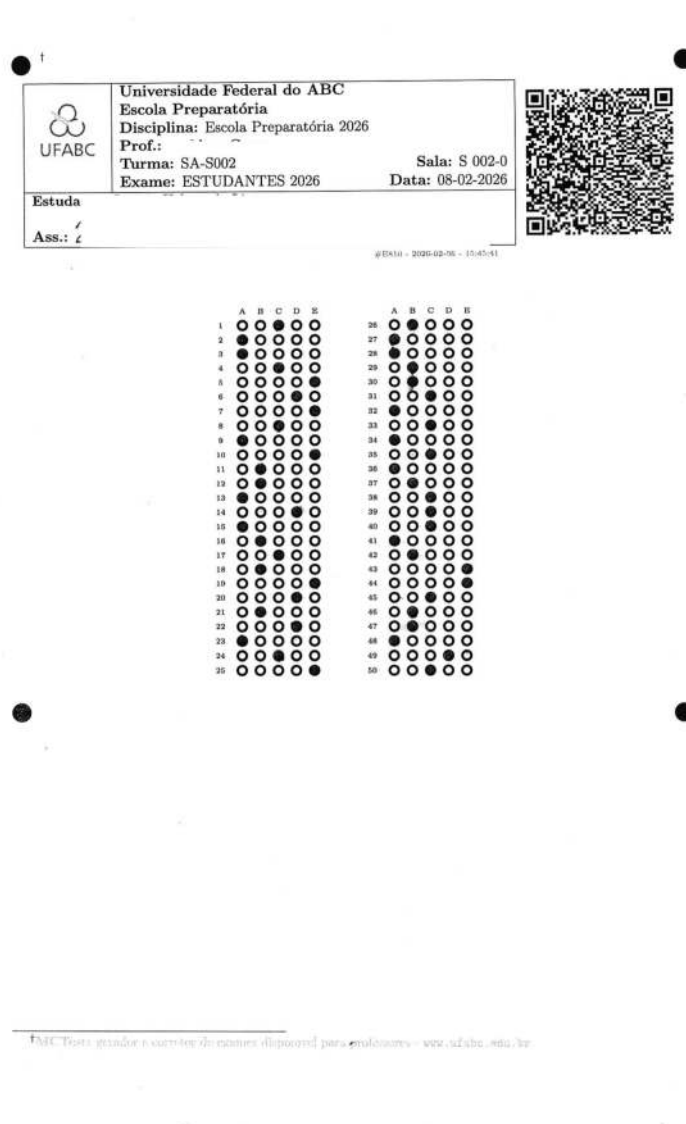


Figura 6.11: Imagem da folha de respostas em escala de cinza carregada a partir do PDF rasterizado.

```
import CVMCTest
countPage = 0
img_getAnswerArea = CVMCTest.cvMCTest.getAnswerArea(img_inicial, countPage)
mm.show(img_getAnswerArea)
```

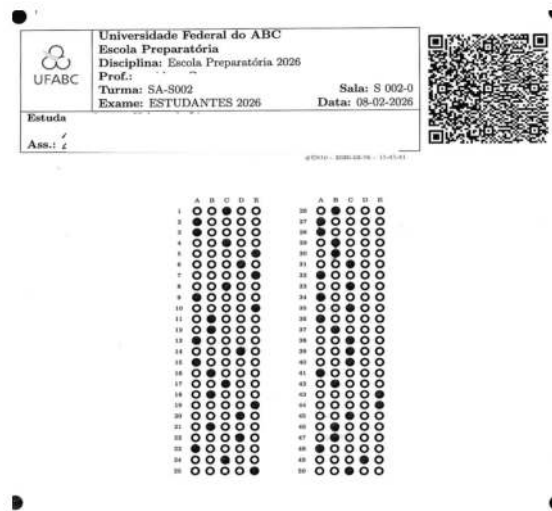


Figura 6.12: Área de respostas extraída por *getAnswerArea*: região retificada contendo os quadros de marcação.

Nota de compatibilidade: versões recentes do NumPy (2.0) removeram o alias `np.int0`. Caso `CVMCTest.py` utilize esse tipo, o comando abaixo aplica a correção diretamente no arquivo antes de recarregá-lo:

```
!sed -i 's/box = np.int0(cv2.boxPoints(rect))/box = cv2.boxPoints(rect).astype(np.intp)/' \
./CVMCTest.py
```

```
import importlib
import CVMCTest

importlib.reload(CVMCTest)
```

```
<module 'CVMCTest' from '/home/fz/fz/VSCoDe/pdi-vc/all/cap06/CVMCTest.py'>
```

6.6.4.3 Segmentação do *QRCode* e Localização dos Quadros

Após a extração da área de respostas, o `MCTest` executa duas etapas preparatórias para a leitura das bolhas: `segmentQRcode` e `findSquares`.

Segmentação do *QRCode*. A função `segmentQRcode` isola o *QRCode* para uma tentativa de decodificação. Em seguida, `getQRcode` retorna o indicador `myFlagArea`, que informa se a área de respostas foi localizada corretamente, e o dicionário `qr`, utilizado para armazenar os metadados da prova. Caso a decodificação não seja bem-sucedida, o processamento das respostas é mantido, mas a identificação do aluno, obtida a partir do *QRCode*, permanece em branco no arquivo CSV de saída. Dessa forma, todas as folhas são processadas e registradas, uma por linha, mesmo quando o *QRCode* não pode ser interpretado. A Figure 6.13 apresenta o *QRCode* segmentado.

```
import CVMCTest
imgQRcode = CVMCTest.cvMCTest.segmentQRcode(img_getAnswerArea, countPage)
mm.show(imgQRcode)
```



Figura 6.13: Região do *QRCode* isolada por *segmentQRcode* dentro da área de respostas retificada.

A função `CVMCTest.cvMCTest.decodeQRcode(imgQRcode)` descriptografa a *string* hexadecimal e retorna o dicionário `qr` com os metadados da prova, apresentado na próxima seção.

Localização dos Quadros de Respostas

A área de respostas retornada por `getAnswerArea` engloba o cabeçalho da folha, incluindo o *QRCode*, e os quadros de respostas posicionados logo abaixo. Como apenas essa segunda região é utilizada na leitura das bolhas, o recorte `img_getAnswerArea[300:, :]` isola a região de interesse processada pela função `findSquares` para localizar os quadros de respostas.

```
img_getAnswerArea_aux = img_getAnswerArea[300:, :]
mm.show(img_getAnswerArea_aux)
```

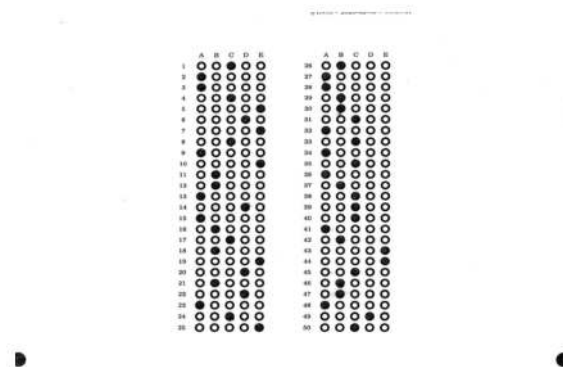


Figura 6.14: Recorte inferior da área de respostas, concentrando os quadros de bolhas a serem segmentados.

Os principais campos do dicionário `qr`, retornado pelo código a seguir, são:

- **date:** identificador temporal da prova, composto pela data de geração e um *timestamp* interno do MCTest.
- **idClassroom, idExam, idStudent:** identificadores da turma, da prova e do aluno, utilizados para localizar o gabarito e registrar os resultados.
- **term:** período letivo da prova.
- **stylesheet:** folha de estilo utilizada na geração do formulário, definindo seu *layout*.
- **var1 a var5:** número de questões dos níveis de dificuldade 1 a 5, conforme a configuração da prova.
- **text:** número de questões dissertativas.
- **answer:** número de alternativas por questão.
- **numquest:** número total de questões da prova.

- **correct, dbtext:** campos preenchidos após a correção automática, contendo o gabarito e informações adicionais das questões.
- **variations, variant:** indicam a existência de variações da prova e a versão atribuída ao estudante.

Esses metadados permitem ao MCTest identificar a prova e o estudante, recuperar o gabarito correspondente e configurar as etapas subsequentes de leitura e correção das respostas. Na prática, a função `CVMCTest.cvMCTest.decodeQRcode(imgQRcode)` é chamada internamente por `CVMCTest.cvMCTest.getQRCode(img, countPage)`, apresentada na próxima seção, que integra a segmentação e a decodificação do *QRCode* em uma única etapa do *pipeline*.

```
myFlagArea, qr = CVMCTest.cvMCTest.getQRCode(img_inicial, countPage)
myFlagArea, qr
```

```
(True,
 {'date': '260208-1770403541363',
  'idClassroom': '955',
  'idExam': '810',
  'idStudent': '448898',
  'term': '0',
  'stylesheet': '1',
  'var1': '50',
  'var2': '0',
  'var3': '0',
  'var4': '0',
  'var5': '0',
  'text': '0',
  'answer': '5',
  'numquest': 50,
  'correct': '',
  'dbtext': '',
  'variations': '0',
  'variant': '0'})
```

```
rectSquares = CVMCTest.cvMCTest.findSquares(qr, img_getAnswerArea, countPage)
rectSquares
```

```
[[[np.int64(395), np.int64(350)], [np.int64(950), np.int64(516)]],
 [[np.int64(394), np.int64(602)], [np.int64(951), np.int64(767)]]]
```

6.6.4.4 Leitura Automática das Respostas

As etapas desenvolvidas anteriormente são integradas no trecho de código a seguir para processar cada quadro de respostas e compor as marcações do estudante. Para cada quadro identificado em `rectSquares`, as funções `setColumns` e `setLines` estimam, respectivamente, o número de alternativas por questão e o número de questões, com base na distribuição espacial das bolhas. Em seguida, `segmentAnswers` avalia o grau de preenchimento de cada bolha e identifica a alternativa marcada. Por fim, `setAnswersOneLine` consolida as respostas de todos os quadros no campo `qr['answers']`, produzindo uma única sequência de respostas. Quando o MCTest é utilizado de forma independente, sem integração com seu banco de questões, essa sequência é comparada ao gabarito presente na primeira página do PDF, que corresponde ao modelo de prova sem enunciados adotado neste capítulo. A `?@fig-mctest-answers` apresenta o conteúdo final de `qr['answers']`, contendo as respostas lidas automaticamente.

```
testAnswers = []
if myFlagArea:

    imgQ_all = []

    for countSquare in range(len(rectSquares)):
```



```

p1, p2 = rectSquares[countSquare]

if True:
    imgQi = CVMCTest.cvMCTest.imgAnswers[p1[0]:p2[0], p1[1]:p2[1]]
    [NUM_COLUMNS, img] = CVMCTest.cvMCTest.setColumns(imgQi, countPage, countSquare)
    [NUM_LINES, img] = CVMCTest.cvMCTest.setLines(imgQi, countPage, countSquare)
    NUM_RESPOSTAS = NUM_COLUMNS
    NUM_QUESTOES = NUM_LINES

    imgQiNC = CVMCTest.cvMCTest.imgAnswers[p1[0]:p2[0], p1[1]:p2[1]]
    testAnswers.append(CVMCTest.cvMCTest.segmentAnswers(
        [imgQi, imgQiNC], countPage, countSquare, NUM_QUESTOES, qr
    ))

    imgQ_all.append(imgQiNC)

qr = CVMCTest.cvMCTest.setAnswersOneLine(testAnswers, qr)
# deixa as respostas de cada quadro em uma linha

mm.show(imgQ_all)
print(f"Respostas lidas das {len(qr['answers'].split(","))} questões: \
      \n{qr['answers'][:-19]}...")

```

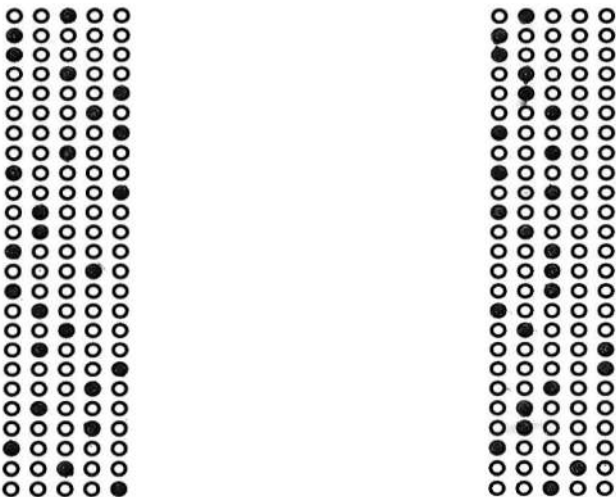


Figura 6.15: Respostas lidas automaticamente pelo MCTest após segmentação e classificação de todas as bolhas.

Respostas lidas das 50 questões:

C,A,A,C,E,D,E,C,A,E,B,B,A,D,A,B,C,B,E,D,B,D,A,C,E,B,A,A,B,B,C,A,C,A,C,A,B,C,C,C,...

💡 Conectando os pontos

O dicionário `qr['answers']` representa o resultado integrado das etapas desenvolvidas neste capítulo, desde a rasterização e a retificação geométrica até a decodificação do *QRCode* e a interpretação do preenchimento das bolhas.

A principal diferença entre o protótipo implementado neste capítulo e o MCTest está na engenharia de software: tratamento de erros, suporte a múltiplos formatos, integração com banco de dados e interface Web. Os algoritmos de Visão Computacional permanecem essencialmente os mesmos.

No modo de operação utilizado neste capítulo, a primeira página do PDF é assumida como gabarito, enquanto as páginas subsequentes correspondem às folhas de respostas dos estudantes. O MCTest

compara automaticamente cada folha com esse gabarito para calcular a pontuação.

6.7 Inspeção Industrial Automatizada

A inspeção visual automatizada constitui uma importante aplicação da Visão Computacional. Em sistemas de inspeção, câmeras capturam imagens de peças ou produtos, que são analisadas por algoritmos para verificar critérios de qualidade previamente definidos. Dependendo da aplicação, essa abordagem pode reduzir a variabilidade da inspeção humana e aumentar a produtividade do processo.

Neste capítulo são ilustradas duas estratégias clássicas para detecção de defeitos:

- **Subtração de imagens:** compara a imagem capturada com uma referência livre de defeitos. Regiões cuja diferença de intensidade excede um limiar são classificadas como anomalias. Essa abordagem requer iluminação e posicionamento consistentes entre as imagens.
- **Análise de textura:** avalia a uniformidade local da superfície sem depender de uma imagem de referência. Pode ser aplicada a materiais homogêneos, como tecidos, papéis e metais, nos quais irregularidades locais podem indicar defeitos.

As duas abordagens são ilustradas a seguir com imagens sintéticas baseadas em `skimage.data`, permitindo reproduzir os experimentos sem dependência de bases de dados externas.

6.7.1 Referências e *Datasets* Públicos

Em aplicações reais, algoritmos de detecção de defeitos são frequentemente avaliados em *datasets* públicos. Neste capítulo, utilizam-se imagens sintéticas para garantir a reprodutibilidade dos exemplos, ver Figure 6.16, mas os seguintes conjuntos de dados podem ser empregados para experimentação e comparação de algoritmos:

- **MVTec Anomaly Detection Dataset (MVTec AD):** reúne imagens de objetos e texturas com defeitos anotados em nível de pixel (Bergmann et al., 2019). Disponível em: <https://www.mvtec.com/company/research/datasets/mvtec-ad>
- **Kolektor Surface-Defect Dataset (KolektorSDD):** contém imagens de componentes industriais com defeitos superficiais anotados (Tabernik et al., 2020). Disponível em: <https://www.vicos.si/resources/kolektorsdd/>
- **NEU Surface Defect Database:** disponibiliza imagens de superfícies de aço laminado contendo seis categorias de defeitos (Song; Yan, 2013). Disponível em: http://faculty.neu.edu.cn/songkechen/zh_CN/zdylm/263270/list/index.htm

```
import numpy as np
from skimage import data, color
from morph import mm

# Imagem de referência (produto sem defeito)
product_color = data.coffee()
product_gray = color.rgb2gray(product_color)

# Inserção de defeito simulado: arranhão escuro de 10×100 px
defect_image = np.copy(product_gray)
defect_image[100:110, 200:300] = 0.1

# Detecção por subtração e limiarização
difference = np.abs(product_gray - defect_image)
defect_threshold = 0.15 # ajustável conforme a aplicação
detected_defect = (difference > defect_threshold).astype(np.uint8) * 255

mm.show()
```

```
[product_gray, defect_image, detected_defect],
titles=["Referência", "Com defeito", "Defeito detectado"],
cols=3,
figsize=(12, 4)
)

status = "Defeito detectado." if detected_defect.any() else "Produto conforme."
print(status)
```



Figura 6.16: Detecção de defeito por subtração de imagem: produto de referência, imagem com defeito simulado e máscara de anomalia detectada.

Defeito detectado.

i Subtração de Imagens e Registro Geométrico

A subtração de imagens pressupõe que a imagem de inspeção esteja geometricamente alinhada à imagem de referência. Quando há diferenças de posicionamento, rotação, escala ou perspectiva entre as imagens, a subtração pode evidenciar essas variações em vez dos defeitos, gerando falsos positivos.

Em muitas linhas de produção, entretanto, o processo de aquisição é rigorosamente controlado. Dispositivos de posicionamento, esteiras, gabaritos mecânicos e câmeras fixas garantem que todas as imagens sejam capturadas na mesma posição e orientação. Nesses casos, a subtração pode ser aplicada diretamente. Um exemplo é a inspeção de uma caixa de ferramentas, na qual a caixa é sempre posicionada no mesmo local para verificar automaticamente a ausência de alguma ferramenta.

Quando esse controle não é possível, emprega-se o **registro de imagens**, um processo que estima uma transformação geométrica para alinhar a imagem de inspeção à imagem de referência, compensando diferenças de posicionamento, rotação, escala e, quando necessário, perspectiva. Após esse alinhamento, a subtração tende a destacar apenas as diferenças causadas por defeitos, e não aquelas decorrentes da aquisição das imagens. A Figure 6.17 apresenta um simulador que ilustra o efeito do desalinhamento e a importância do registro antes da subtração.

6.7.2 Detecção de Defeitos por Análise de Textura

Na ausência de uma imagem de referência, a inspeção pode explorar a **homogeneidade local da textura**. Superfícies uniformes, como metais polidos, tecidos e papéis, apresentam baixa variância local. Defeitos como riscos, bolhas ou manchas aumentam essa variância em sua vizinhança, permitindo sua detecção.

A Figure 6.18 ilustra essa abordagem com a imagem `skimage.data.brick()`. Inicialmente, calcula-se a variância local em uma janela deslizante; em seguida, o mapa resultante é limiarizado para destacar as regiões heterogêneas.

```
import numpy as np
import cv2
from skimage import data as skdata
from morph import mm
```



Figura 6.17: Simulador interativo de inspeção industrial por subtração de imagens: controle as distorções geométricas de desalinhamento (registro) e o limiar de detecção para observar o impacto nos falsos positivos.

```

# Imagem de textura uniforme (tijolo)
texture = skdata.brick().astype(np.float32) / 255.0

# Inserção de defeito sintético: mancha clara 20×80 px
texture_defect = np.copy(texture)
texture_defect[60:80, 80:160] = 0.95

# Mapa de variância local (janela 15×15)
def variancia_local(img, ksize=15):
    img_f = img.astype(np.float32)
    mean = cv2.blur(img_f, (ksize, ksize))
    mean2 = cv2.blur(img_f ** 2, (ksize, ksize))
    return np.clip(mean2 - mean ** 2, 0, None)

var_ref = variancia_local(texture)
var_defect = variancia_local(texture_defect)
diff_var = np.abs(var_defect - var_ref)

# Normaliza e limiariza
diff_norm = (diff_var / diff_var.max() * 255).astype(np.uint8)
_, mask = cv2.threshold(diff_norm, 30, 255, cv2.THRESH_BINARY)

mm.show(
    [texture, texture_defect, diff_norm, mask],
    titles=["Textura original", "Com defeito", "Δ variância local", "Anomalia detectada"],
    cols=4,
    figsize=(16, 4)
)

status = "Defeito de textura detectado." if mask.any() else "Superfície conforme."
print(status)

```

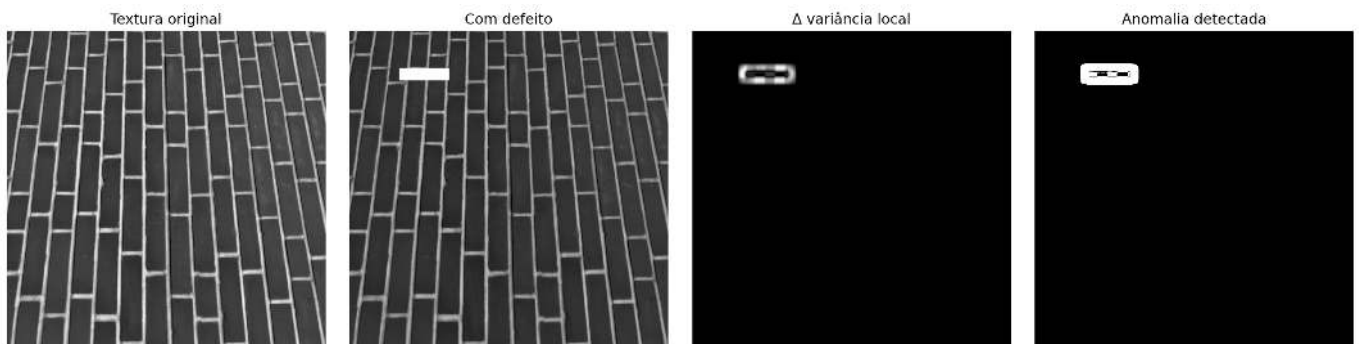


Figura 6.18: Detecção de heterogeneidade de textura: mapa de variância local e máscara de anomalia.

Defeito de textura detectado.

i 🧠 Por que funciona? — Detecção de defeitos por subtração

A subtração pixel a pixel entre a imagem de referência e a imagem capturada produz valores próximos de zero em regiões idênticas e valores elevados onde há diferenças. A operação `np.abs` torna o método insensível ao sinal da diferença, permitindo detectar tanto defeitos mais claros quanto mais escuros que a referência. Em seguida, a limiarização converte o mapa de diferenças em uma decisão binária: região conforme ou defeituosa.

O principal parâmetro do método é o limiar (`defect_threshold`). Valores muito baixos tornam o algoritmo sensível ao ruído e aumentam a ocorrência de falsos positivos, enquanto valores muito altos

podem impedir a detecção de defeitos sutis.

6.8 Resumo

Neste capítulo foram apresentadas técnicas de Visão Computacional aplicadas à inspeção industrial e à análise automatizada de documentos.

- **Alinhamento de documentos (*deskew*):** a combinação entre detecção de bordas com Canny, estimação da orientação predominante pela Transformada de Hough e rotação inversa corrige automaticamente a inclinação de documentos digitalizados.
- **Normalização de fundo e CLAHE:** a divisão da imagem por uma versão suavizada cancela gradientes globais de iluminação; o CLAHE complementa esse processo equalizando o contraste localmente, tornando a limiarização por Otsu mais robusta em documentos com iluminação não uniforme.
- **Detecção de marcadores e retificação de perspectiva:** o fechamento morfológico com elemento estruturante grande isola os discos de referência; a filtragem por circularidade seleciona os quatro marcadores; a transformação de perspectiva retifica a folha para dimensões conhecidas.
- **Decodificação de *QRCode* e código de barras:** a abertura morfológica remove ruídos, a seleção pelo maior contorno quadrado isola o marcador e o redimensionamento por interpolação cúbica viabiliza a decodificação com `cv2.QRCodeDetector` e `pyzbar`.
- **Sistema MCTest:** integração das etapas anteriores em um *pipeline* completo de correção automatizada de avaliações, ilustrando a transição entre protótipo didático e sistema em produção.
- **Inspeção industrial:** detecção de defeitos por subtração de imagem de referência e por análise de variância local de textura, com exemplos sintéticos reproduzíveis e referência ao *dataset* MVTec AD.

O próximo capítulo aprofundará técnicas de **extração de características e reconhecimento de padrões**, ampliando o escopo da Visão Computacional para tarefas de classificação e reconhecimento de objetos.

6.9 Uso do NotebookLM como Tutor Complementar

Nesta edição, o uso do **NotebookLM** é incentivado como ferramenta complementar de aprendizagem. Baseado em inteligência artificial, o sistema utiliza exclusivamente os documentos fornecidos pelo autor como fonte de conhecimento, produzindo respostas alinhadas ao conteúdo e à abordagem adotada ao longo deste capítulo.

 Estude com o Tutor Inteligente

 [ACESSAR NOTEBOOKLM: CAPÍTULO 06](#)

 **Aviso sobre Conteúdo Gerado por IA**

Embora seja uma ferramenta valiosa de apoio aos estudos, o NotebookLM pode eventualmente produzir respostas incompletas, imprecisas ou incorretas. Recomenda-se validar as informações consultando o material do capítulo, livros, artigos científicos e outras fontes acadêmicas confiáveis. Sempre que possível, execute e experimente os exemplos práticos apresentados ao longo do texto para consolidar a compreensão dos conceitos.

6.10 Exercícios Práticos

Os exercícios a seguir consolidam os conceitos apresentados neste capítulo, propondo extensões do *pipeline* de processamento documental desenvolvido ao longo do texto.

6.10.1 Exercício 1: Adaptação do *Pipeline* para Código de Barras

Contexto: O *pipeline* desenvolvido neste capítulo converte um PDF em imagens, corrige a inclinação (*deskew*) e decodifica um *QRCode*. Códigos de barras lineares (1D) também são amplamente empregados em documentos institucionais, notas fiscais e etiquetas logísticas, seguindo um fluxo de processamento bastante semelhante.

Desafio: Adapte esse *pipeline* para processar o arquivo `dados/provas_barcode.pdf`, cujas páginas contêm códigos de barras lineares no lugar de *QRCodes*. Para isso:

1. Rasterize o PDF com `pdf2image` utilizando resolução de 300 DPI;
2. Aplique a etapa de correção de inclinação (*deskew*) desenvolvida na seção anterior;
3. Utilize a biblioteca `pyzbar` (apresentada na próxima seção) para decodificar o código de barras;
4. Exiba a imagem retificada e a sequência de caracteres decodificada.

Saída esperada: A imagem retificada, seguida da string decodificada do código de barras.

Dica: Ao contrário do *QRCode*, que é bidimensional, códigos de barras lineares normalmente podem ser decodificados diretamente da imagem rasterizada em 300 DPI, sem a necessidade de redimensionamento da região de interesse.

6.10.2 Exercício 2: Validação Robusta da Leitura

Contexto: A detecção das bolhas utiliza critérios geométricos e de preenchimento. Em imagens de baixa qualidade, ruídos e manchas podem gerar falsos positivos.

Desafio:

1. Investigue critérios adicionais para validar as bolhas detectadas;
2. Avalie diferentes limiares de preenchimento;
3. Compare os resultados em imagens com diferentes níveis de ruído;
4. Discuta o impacto das escolhas na taxa de acertos e de falsos positivos.

Saída esperada: Uma tabela ou gráfico comparando o desempenho para diferentes limiares e exemplos de detecções corretas e incorretas.

6.10.3 Exercício 3: Detecção de Marcação Inválida

Contexto: Em avaliações reais, uma questão pode apresentar dupla marcação, rasuras ou ausência de resposta.

Desafio:

1. Detecte questões com duas ou mais alternativas preenchidas;
2. Identifique respostas em branco;
3. Defina um critério para sinalizar preenchimentos ambíguos;
4. Gere um relatório indicando o estado de cada questão.

Saída esperada: Um relatório em JSON ou `pandas.DataFrame` contendo, para cada questão, a alternativa lida e seu estado (OK, BRANCO, DUPLA MARCAÇÃO ou SUSPEITA).

6.10.4 Exercício 4: *Pipeline* Completo

Contexto: As etapas desenvolvidas neste capítulo podem ser integradas em uma única aplicação.

Desafio:

1. Implemente uma função `processar_prova(caminho_pdf)` que:

- converta o PDF em imagem;
 - retifique a folha;
 - decodifique o *QRCode*;
 - leia as respostas;
 - retorne os metadados e as respostas do estudante.
2. Avalie o *pipeline* em diferentes condições de aquisição (rotação, resolução e qualidade de digitalização).
 3. Documente as principais limitações observadas e proponha possíveis melhorias.

Saída esperada: Um programa reutilizável acompanhado de exemplos de execução em diferentes folhas de resposta.

6.11 Próximos Passos

Neste capítulo foi desenvolvido um *pipeline* completo para análise de documentos, desde a conversão de PDFs em imagens até a leitura automática de folhas de resposta por OMR. As técnicas estudadas — operações morfológicas, transformações geométricas, análise de contornos e decodificação de marcadores — constituem a base de diversos sistemas de Visão Computacional. Os próximos capítulos ampliam esse escopo para problemas mais gerais de reconhecimento visual.

Referências

- BARRERA, Junior *et al.* MMach: a mathematical morphology toolbox for the KHOROS system. **Journal of Electronic Imaging**, v. 7, n. 1, p. 174–210, 1998.
- BERGMANN, Paul *et al.* [MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection](#). *In*: 2019.
- BRADSKI, Gary; KAEHLER, Adrian. **Learning OpenCV: Computer vision with the OpenCV library**. [S.l.]: " O'Reilly Media, Inc.", 2008.
- DOUGHERTY, Edward R.; LOTUFO, Roberto A. **Hands-on morphological image processing**. [S.l.]: SPIE press, 2003. v. 59
- GONZALEZ, R. C.; WOODS, R. E. **Digital Image Processing**. 4th. ed. New York: Pearson, 2018.
- ITU-R. **Recommendation BT.601: Studio encoding parameters of digital television for standard 4:3 and wide screen 16:9 aspect ratios**. <https://www.itu.int/rec/R-REC-BT.601/>, 2011.
- KNUTH, Donald E. [Literate Programming](#). **The Computer Journal**, v. 27, n. 2, p. 97–111, 1984.
- LEWIS, Patrick *et al.* Retrieval-augmented generation for knowledge-intensive nlp tasks. **Advances in neural information processing systems**, v. 33, p. 9459–9474, 2020.
- LOTUFO, R. A.; ZAMPIROLI, F. A. [Fast multidimensional parallel Euclidean distance transform based on mathematical morphology](#). *In*: 2001.
- LOTUFO, Roberto A. *et al.* MMachLib functions and MMach operators. *In*: 1997.
- MALLAT, Stéphane. **A wavelet tour of signal processing**. [S.l.]: Elsevier, 1999.
- MATHERON, Georges. **Random Sets and Integral Geometry**. New York: John Wiley & Sons, 1975.
- OPPENHEIM, Alan V. **Discrete-time signal processing**. [S.l.]: Pearson Education India, 1999.
- OTSU, Nobuyuki. [A Threshold Selection Method from Gray-Level Histograms](#). **IEEE Transactions on Systems, Man, and Cybernetics**, v. 9, n. 1, p. 62–66, 1979.
- REDMON, Joseph *et al.* [You Only Look Once: Unified, Real-Time Object Detection](#). *In*: 2016.
- SERRA, Jean. **Image Analysis and Mathematical Morphology**. London: Academic Press, 1982. v. 1
- SINGH, Himanshu. **Practical machine learning and image processing**. [S.l.]: Springer, 2019.
- SINGH, Shobhna; LLOYD, Jerome; FLICKER, Felix. [Hamiltonian Cycles on Ammann-Beenker Tilings](#). **Physical Review X**, v. 14, n. 3, p. 031005, 10 jul. 2024.
- SONG, Kechen; YAN, Yunhui. [A Noise Robust Method Based on Completed Local Binary Patterns for](#)

- [Hot-Rolled Steel Strip Surface Defects](#). **Applied Surface Science**, v. 285, p. 858–864, 2013.
- SZELISKI, Richard. **Computer Vision: Algorithms and Applications**. 2. ed. *[S.l.]*: Springer, 2022.
- TABERNIK, Domen *et al.* [Segmentation-Based Deep-Learning Approach for Surface-Defect Detection](#). **Journal of Intelligent Manufacturing**, v. 31, n. 3, p. 759–776, 2020.
- WALLACE, Gregory K. [The JPEG Still Picture Compression Standard](#). **Communications of the ACM**, v. 34, n. 4, p. 30–44, 1991.
- ZAMPIROLI, Francisco A. **MCTest: Como Criar e Corrigir Exames Parametrizados**. *[S.l.]*: Independente, 2023.
- ZAMPIROLI, Francisco de Assis *et al.* A Practical Digital Image Processing Course with morph.py. *In*: Porto Alegre, RS, Brasil: Sociedade Brasileira de Computação (SBC), 2024. Disponível em: <<https://sol.sbc.org.br/index.php/educomp/article/view/28199>>
- ZAMPIROLI, Francisco de Assis *et al.* [Teaching Hands-On Digital Image Processing with morph.py: Methods and Comprehensive Results](#). **Revista Brasileira de Informática na Educação**, v. 33, p. 990–1026, 2025.